

Theory of Automata, Languages and Computation

About the Author

Rajendra Kumar is Assistant Professor and Head of the Department of Computer Science and Engineering at Vidya College of Engineering, Meerut. He has been a member of the Board of Studies of Uttar Pradesh Technical University, Lucknow, and is a member of ISTE Delhi and Amnesty International, UK.

Prof. Kumar has over twelve years of teaching experience. He has taught at the Meerut Institute of Engineering and Technology, Meerut, for eight years, and has also been a guest faculty at the Bundelkhand Institute of Engineering and Technology, Jhansi.

Prof. Kumar has authored three textbooks, namely, *Human Computer Interaction, Information and Communication Technologies*, and *Modeling and Simulation Concept*. Besides these, he has written distance learning books on *Computer Graphics* for MGU Kerala and MDU Rohtak; *Modeling and Simulation*, *System Simulation*, and *ICT in Public Life* for CDLU Sirsa; and *IT Enabled Services* for IASE University, Rajasthan. He has also published and presented several papers in international/national journals/conferences.

A popular academician, Prof. Kumar has guided two MTech dissertations and dozens of BTech projects. He has been appointed Head Examiner three times by UPTU Lucknow.

His current research area is Instruction Level Parallelism. His other areas of interest include Biometric Systems, Compiler Design, Multimedia Systems, and Software Engineering.



Theory of Automata, Languages and Computation

Rajendra Kumar

*Assistant Professor and Head of Department
Computer Science and Engineering
Vidya College of Engineering
Meerut*



Tata McGraw Hill Education Private Limited

NEW DELHI

McGraw-Hill Offices

New Delhi New York St Louis San Francisco Auckland Bogotá Caracas
Kuala Lumpur Lisbon London Madrid Mexico City Milan Montreal
San Juan Santiago Singapore Sydney Tokyo Toronto



Tata McGraw-Hill

Published by Tata McGraw Hill Education Private Limited,
7 West Patel Nagar, New Delhi 110 008

Theory of Automata, Languages and Computation

Copyright © 2010, by Tata McGraw Hill Education Private Limited

No part of this publication may be reproduced or distributed in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise or stored in a database or retrieval system without the prior written permission of the publishers. The program listings (if any) may be entered, stored and executed in a computer system, but they may not be reproduced for publication.

This edition can be exported from India only by the publishers,
Tata McGraw Hill Education Private Limited.

ISBN (13 digits): 978-0-07-070204-2

ISBN (10 digits): 0-07-070204-7

Managing Director: *Ajay Shukla*

Head—Higher Education Publishing and Marketing: *Vibha Mahajan*

Manager: Sponsoring—SEM & Tech Ed: *Shalini Jha*

Asst Sponsoring Editor: *Surabhi Shukla*

Development Editor: *Surbhi Suman*

Executive—Editorial Services: *Sohini Mukherjee*

Sr Production Executive: *Suneeta Bohra*

Dy Marketing Manager—SEM & Tech Ed.: *Biju Ganesan*

General Manager—Production: *Rajender P Ghansela*

Asst General Manager—Production: *B L Dogra*

Information contained in this work has been obtained by Tata McGraw-Hill, from sources believed to be reliable. However, neither Tata McGraw-Hill nor its authors guarantee the accuracy or completeness of any information published herein, and neither Tata McGraw-Hill nor its authors shall be responsible for any errors, omissions, or damages arising out of use of this information. This work is published with the understanding that Tata McGraw-Hill and its authors are supplying information but are not attempting to render engineering or other professional services. If such services are required, the assistance of an appropriate professional should be sought.

Typeset at Text-o-Graphics, B1/56 Arawali Apartment, Sector 34, Noida 201301 and
printed at Sheel Print-N-Pack, D-132, Hoisery Complex, Phase-II, Noida.

Cover Printer: Sheel Print-N-Pack

RALBCRCZRBBYZ

Dedicated to

Late Shri R P Singh (Palwal)

*“We will remember
Because you made us smile
We are just happy that you
Lived a life so worthwhile”*

Preface

Overview of the Subject

Automata theory is the foundation of computer science. Its applications have spread to almost all areas of computer science and many other disciplines. We see how the concepts of automata theory are applied in the design of compilers, text editors, etc. In addition, there is a growing number of software systems designed to manipulate automata, regular expressions, grammars, and related structures.

Formal languages provide the theoretical underpinning for the study of programming languages as well as lay the foundations for compiler design. They are important in such areas as the study of biological systems, data transmission and compression, computer networks, and the like.

This subject introduces the theoretical basis of computation, in the sense, we see the origin of the computer as we know it today. In this subject, we see mathematical models for computation and how they can be formally described and reasoned about. It covers the basic models of state machines to Turing machines, the difference in computational power and how the concept of computing relates to formal languages. It also discusses deterministic and nondeterministic models of machines and compares them with each other.

The automata models are a valuable part of the repertoire for any computer scientist or engineer. This course introduces progressively more powerful models of computation, starting with formal languages and finite automata before moving on to Turing machines. It also presents regular, context free, recursive and recursively enumerable languages, and shows how they correspond to the various models of computation and generate mechanisms such as regular expressions and grammar. The emphasis is on understanding the properties of these models, the relationship among them, and how modifications such as non-determinism and resource bounds affect them. The course includes applications of these concepts to problems arising in other parts of computer science.

The course on Automata Theory is mainly concerned with the concepts related to validation of statements in a high-level language program. Thus, the main focus of this course is on strings, finite automata, regular expression, grammars, pushdown automata, Turing machines, etc.

By studying this subject, the readers will be able to recognize and use terminology and formalisms related to grammars for programming languages which are a prerequisite for compiler construction. As we know, the purpose of a compiler is to translate a program in some language (the *source program*) into a low-level language (the *machine language*). The compiler itself is written in a language called the *implementation language*. To construct a compiler, software engineers have to think about and understand the source language and its validation in order to convert it into machine language.

The compiler uses a class graph to construct a finite intermediate automaton which is used in conjunction with an adaptive program to generate an object-oriented program in a target language. The intermediate automaton enables general-case compilation of most combinations of adaptive programs and class graphs. The automaton also enables the use of standard minimization techniques which reduce the size of the generated object-oriented code.

Motivation behind this Book

During my several years of teaching I realised that amongst the plethora of standard books available in the market, some overburden the students with the reigor of content and difficult language, while others oversimplify the concepts and fail to cover important topics like designing Mealy and Moore machines, GTG, MyHill–Nerode theorem, two-way FA, unambiguous grammar, auxiliary and two-stack PDA, Gödel numbering and the Markov algorithm. Besides these, most of the local books available on this subject contain numerous mistakes and provide poor presentation of concepts. Hence, there was a great demand for a book that could cover the flaws of the existing books and also attract students of different skill levels. Thus, this inspired me to start the work of this manuscript.

Scope/Target Audience

A book is considered good if it is able to deliver the required knowledge to its readers. These days, a majority of students feel comfortable if the whole syllabus is covered in a single book and the concepts are explained in simple language. This book also consists of simple and real-life explanations of complicated ideas with a wide variety of examples. Hence, it can be adopted as a textbook in an undergraduate course and as a reference book in postgraduate courses like MTech. This book would also be useful for BCA/MCA students of regular and distance education programmes of universities like IGNOU. In addition to this, this book will be helpful for competitive examinations like GATE, etc. Thus, I hope this book will be the first choice for a majority of students.

Roadmap for Various Target Courses

This book has 11 chapters in all which fulfill the syllabi requirements of the major Indian universities offering courses on Automata Theory. Hence, I recommend the usage of all chapters of the book in the current sequence. However, after reading **Chapter 1**, readers can go for **Appendix A** (Propositions and Predicate Calculus) on the OLC (Online Learning Center). The readers looking for Applications of Automata in Parsing can go for **Appendix B** (*LL* and *LR* Grammar) on the OLC after reading chapters 6, 7, and 8. Students having a strong background in discrete mathematics can skip Chapter 1 of the textbook and Appendix A (Propositions and Predicate Calculus) of the OLC.

About the Book

This book is specially designed to suit the requisites of all levels of students studying this subject. It endeavours to provide an excellent and user-friendly presentation of the concepts, essential for an introductory course on automata and computation. The text includes the straightforward explanation of complicated ideas like transition functions, two-way FA, equivalence of two automata, equivalence of two regular expressions, pumping lemma, auxiliary and two-stack PDA, equivalence of two-stack PDAs and Turing machines, PDA for regular and context free languages, and Turing machines for regular and non-regular languages.

This book explains how a reader can define the transition function if (s)he knows the functioning of an automaton and vice-versa in a very easy way. Every concept is followed by examples. The text is illustrated with diagrams. Most of the exercise questions are accompanied with hints, making it easy for the student to solve the problems.

In several sections of this book, there are *Did You Know* and *Good to Know* boxes that contain interesting information about the major contributors in this field. This is an attractive feature which adds flavour to the text and acts as mental relaxation while reading serious stuff. It keeps the reader's knowledge updated with some remarkable facts that are hard to find in any textbook on this subject as such.

Salient Features

- Detailed coverage of important topics such as Finite Automata, Pushdown Automata, Context-Free and Regular Languages.
- Exhaustive yet simple and lucid explanation of concepts
- Good balance between theoretical and mathematical rigor
- Hints and solutions to review exercises, graded problems and multiple-choice questions
- 'Good to Know' and 'Did You Know' features provide additional information and facts about the subject and its history
- Excellent pedagogy includes:
 - ✓ Over 140 Solved Examples
 - ✓ Over 175 Review Questions
 - ✓ Over 100 Graded Questions
 - ✓ Over 280 Multiple Choice Questions

Organization of the Book

The book comprises 11 chapters. **Chapter 1** on Mathematical Preliminaries introduces the concept of set theory, strings, languages, relations, functions, graphs and trees. **Chapter 2** deals extensively with Finite Automata and related topics. Formal Languages are taken up in **Chapter 3**. Arithmetic Expressions and Chomsky Hierarchy are also explained in this chapter. Regular Language and Regular Grammar are discussed in **Chapter 4**. The Myhill–Nerode theorem is taken up at this stage. **Chapter 5** details the Properties of Regular Languages.

Context Free Grammar and Context Free Language are described in **Chapter 6**. The Chomsky and Greibach normal forms and the CYK algorithm are discussed here. **Chapter 7** on Push Down Automata explains the topic in detail. Properties of Regular and Context Free Languages are described in **Chapter 8**. The very important topic of Turing Machines is thereafter discussed in **Chapter 9**. **Chapter 10** is on Undecidability and Computability. Finally, NP-Completeness is dealt with in **Chapter 11**.

The excellent pedagogical features include a bulleted **Summary** for every chapter, along with a large number of **Review Questions**, **Graded Exercises** and **Objective Questions**. Interested readers may also go through the sections on **References for Extra Reading** and **Online Sources**.

Web Supplements

The accompanying website <http://www.mhhe.com/kumar/talc> is designed as Online Learning Center (OLC) for both students and instructors. The solutions to the difficult problems are available on the OLC. This page also contains PowerPoint Slides of the book along with my notes on Automata. Appendices on ‘**Propositions and Predicates**’ and ‘**LR[k] and LL[k] Grammars**’ are present as extra reading material. Apart from this, there are several test papers (based on university examination pattern) with solutions. My notes and sample test papers can also be found on my web site www.rkronline.in.

Acknowledgements

I would first like to acknowledge my students because of whom I have been learning and improving my logical skills for more than a decade.

I am indebted to Dr P K Singh of MMM Engineering College, Gorakhpur; Dr V K Sharma of Vidya Knowledge Park, Meerut; Dr Phalguni Gupta of IIT Kanpur; Dr Vinay Pathak, Vice Chancellor Uttranchal Open University; Dr D S Yadav of IET, Lucknow; Dr Vikas Saxena of IIIT, Noida; Dr A K Solanki of BIET, Jhansi; Dr Ravindra Kumar of DIT, Greater Noida; Dr A H Siddiqui, Dr R C Singh and Dr Bhaskar Bhattacharya of Sharda University, Greater Noida.

I am thankful to Ms Vibha Mahajan, Ms Shalini Jha, Ms Surabhi Shukla, Ms Surbhi Suman, Ms Sohini Mukherjee, Ms Suneeta Bohra and the entire team of Tata McGraw Hill Education for their efforts to make this book possible.

My special thanks are due to Dr B P Sharma, Advocate B S Khokhar, Advocate Y S Dhaka, and R K Pathak for their guidance. I am grateful to Ajay Pratap Singh, Rohit Khokhar, Saurabh Sharma, Priya Sharma for their kind support from time to time.

And of course, I would like to express my sincere gratitude towards my parents; brother Jitendra; wife, Richa; and daughters Devanshi and Shreyanshi, for their love, cooperation and patience with my endless late night keyboard tapping.

Thanks are also due to the following reviewers for their constructive suggestions which helped in giving a final shape to this project. Their names are given below:

Dinesh K Sharma

University of Maryland, Eastern Shore, Princess Anne, MD, USA

Abhilash Sharma

Meerut Institute of Engineering and Technology, Meerut, Uttar Pradesh

Jay Prakash

Madan Mohan Malviya Engineering College, Gorakhpur, Uttar Pradesh

Naveen Chauhan

National Institute of Technology, Hamirpur, Himachal Pradesh

Manjari Gupta

Banaras Hindu University, Varanasi, Uttar Pradesh

Parmanand Astya

Galgotia College of Engineering, Ghaziabad, Uttar Pradesh

Raghuraj Singh

Harcourt Butler Technological Institute (HBTI), Kanpur, Uttar Pradesh

Krishna K Mishra

Motilal Nehru National Institute of Technology, Allahabad, Uttar Pradesh

Satinder Bal Gupta

Vaish College of Engineering, Rohtak, Haryana

Saiful Islam

Zakir Hussain College of Engineering and Technology, Aligarh Muslim University, Aligarh, Uttar Pradesh

Suneeta Agarwal

Motilal Nehru National Institute of Technology, Allahabad, Uttar Pradesh

C S Yadav

Noida Institute of Engineering and Technology, Greater Noida, Uttar Pradesh

Nagresh Kumar

Meerut Institute of Engineering and Technology, Meerut, Uttar Pradesh

Subir Halder

Dr Bidhan Chandra Roy Engineering College, Durgapur, West Bengal

Jaydeep Nath

Future Institute of Technology, Kolkata, West Bengal

D Sarkar

Indian Institute of Technology Kharagpur, Kharagpur, West Bengal

K V Santhilata

Birla Institute of Technology, Goa

M Janaki Meena

PSG College of Technology, Coimbatore, Tamil Nadu

Latha R Nair

School of Engineering, Cochin University of Science and Technology (CUSAT), Cochin, Kerala

Kamala Krithivasan

Indian Institute of Technology (IIT) Madras, Chennai, Tamil Nadu

N Guruprasad

People's Education Society (PES) Institute of Technology, Bangalore, Karnataka

Tilottama Goswami

Swami Vivekananda Institute of Technology (SVIT), Secunderabad, Andhra Pradesh

K Raja Sekhar

Koneru Lakshmaiah University, Vijaywada, Andhra Pradesh

RAJENDRA KUMAR

email: kumar_rajen@yahoo.com,

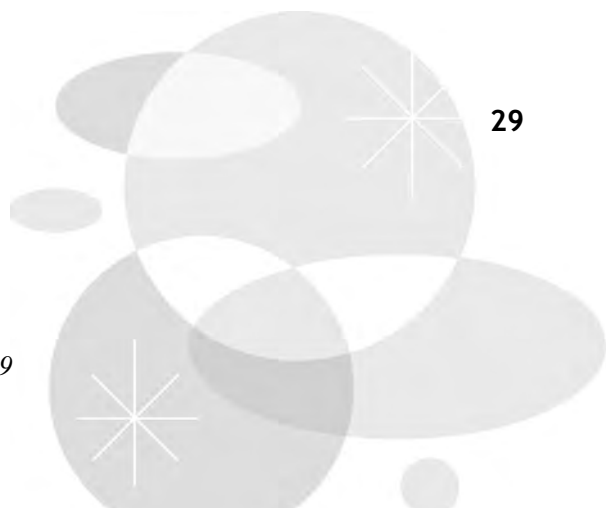
website: www.rkronline.in

Feedback

We welcome suggestions for improvement of the book. Please drop in an email with your views/feedback/suggestions at tmh.csefeedback@gmail.com (kindly mention the title and author name in the subject line). We look forward to receiving information on any piracy spotted by you.

Contents

<i>Preface</i>	vii
<i>List of Important Symbols and Notations</i>	xix
1. Mathematical Preliminaries	1
<i>Chapter Objective</i>	1
<i>Introduction</i>	1
1.1 SET Theory	2
1.2 Alphabets	7
1.3 Strings and Languages	7
1.4 Relations	9
1.5 Functions	13
1.6 Graphs and Trees	16
1.7 Proof Techniques	22
<i>Summary</i>	24
<i>Review Questions</i>	25
<i>Graded Exercises</i>	26
<i>Objective Questions</i>	27
<i>Reference for Extra Reading</i>	27
<i>Online Sources</i>	28
2. Finite Automata	29
<i>Chapter Objective</i>	29
<i>Introduction</i>	29
2.1 Finite State Machines and its Model	30
2.2 Deterministic Finite Automata	31
2.3 Simplified Notation	33
2.4 FA with and without Epsilon Transitions	39
2.5 Language of Deterministic Finite Automata	39
2.6 Acceptability of a String by a DFA	40
2.7 Processing of Strings by DFA	40



2.8	Nondeterministic Finite Automata	41
2.9	Language of NFA	43
2.10	Equivalence between DFA and NFA	44
2.11	NFA With and Without Epsilon Transitions	47
2.12	Two Way Finite Automata	52
2.13	FA with Output: Moore and Mealy Machines	55
2.14	From Finite Automata to Moore Machine	63
2.15	Interconversion between the Machines	64
2.16	Equivalence between Moore and Mealy Machines	68
2.17	Minimisation of FA	69
2.18	Properties of Transition Function (d)	76
2.19	Extending Transition Function to Strings	77
2.20	Applications of Finite Automata	79
2.21	Limitations of Finite State Machines	83
	<i>Summary</i>	83
	<i>Review Questions</i>	84
	<i>Graded Exercises</i>	89
	<i>Objective Questions</i>	91
	<i>Reference for Extra Reading</i>	91
	<i>Online Sources</i>	91

3. Formal Languages

97

	<i>Chapter Objective</i>	97
	<i>Introduction</i>	97
3.1	Theory of Formal Languages	98
3.2	Kleene and Positive Closure	99
3.3	Defining Languages	102
3.4	Recursive Definition of Languages	102
3.5	Arithmetic Expressions	104
3.6	Grammars	105
3.7	Classification of Grammars and Languages	119
3.8	Languages and their Relations	120
3.9	Operations on Languages	122
3.10	Chomsky Hierarchy	124
	<i>Summary</i>	127
	<i>Review Questions</i>	127
	<i>Graded Exercises</i>	130
	<i>Objective Questions</i>	132
	<i>Reference for Extra Reading</i>	137
	<i>Online Sources</i>	137

4. Regular Language and Regular Grammar

138

	<i>Chapter Objective</i>	138
	<i>Introduction</i>	138

4.1	Regular Language	139
4.2	Regular Expressions	139
4.3	Operators of Regular Expressions	140
4.4	Identity Rules	141
4.5	Algebraic Laws for RE	142
4.6	Finite Automata and Regular Expressions	144
4.7	Equivalence of Two Regular Expressions	152
4.8	Myhill–Nerode Theorem	153
4.9	Regular Sets	155
4.10	Closure Properties of Regular Sets	156
4.11	Regular Grammar and FA	159
4.12	Regular Expressions and Regular Grammar	163
4.13	Left Linear and Right Linear Regular Grammar	164
4.14	Applications of Regular Expressions	165
4.15	Non-Regular Languages	167
	<i>Summary</i>	167
	<i>Review Questions</i>	168
	<i>Graded Exercises</i>	173
	<i>Objective Questions</i>	174
	<i>Reference for Extra Reading</i>	177
	<i>Online Sources</i>	177

5. Properties of Regular Languages

178

	<i>Chapter Objective</i>	178
	<i>Introduction</i>	178
5.1	Closure Properties of Regular Languages	178
5.2	Decision Properties of Regular Languages	182
5.3	Pumping Lemma for Regular Languages	182
5.4	Proving Languages not to be Regular Languages	184
5.5	Regular Language and Right Linear Grammar	189
	<i>Summary</i>	189
	<i>Review Questions</i>	190
	<i>Graded Exercises</i>	190
	<i>Objective Questions</i>	191
	<i>Reference for Extra Reading</i>	191
	<i>Online Sources</i>	191

6. Context Free Grammar and Context Free Language

192

	<i>Chapter Objective</i>	192
	<i>Introduction</i>	192
6.1	Definition of Context Free Grammar	193
6.2	Context Free Language	194
6.3	Deterministic Context Free Language (DCFL)	194
6.4	Derivations	196

6.5	Parse Trees	196
6.6	From Inference to Tree	199
6.7	Derivation Tree and New Notation of Arithmetic Expressions	201
6.8	Sentential Forms	202
6.9	Rightmost and Leftmost Derivation of Strings	203
6.10	Ambiguity in Grammar and Language	210
6.11	Removal of Ambiguity	214
6.12	Ambiguous to Unambiguous Context-Free Grammar	214
6.13	Useless Symbols in CFG	219
6.14	Elimination of Null and Unit Productions	224
6.15	Chomsky and Greibach Normal Form	228
6.16	CYK Algorithm	235
6.17	Applications of CFG	237
	<i>Summary</i>	290
	<i>Review Questions</i>	241
	<i>Graded Exercises</i>	243
	<i>Objective Questions</i>	244
	<i>Reference for Extra Reading</i>	248
	<i>Online Sources</i>	248

7. Push Down Automata

249

	<i>Chapter Objective</i>	249
	<i>Introduction</i>	249
7.1	Description and Definition	250
7.2	Definition and Model of PDA	250
7.3	Language of PDA	251
7.4	Graphical Notations for PDA	251
7.5	Acceptance by Final State and Empty Stack	255
7.6	From Empty Stack to Final State and Vice Versa	255
7.7	Deterministic Push Down Automata	258
7.8	Nondeterministic Push Down Automata	259
7.9	Equivalence of PDA and Context Free Language	261
7.10	PDA and Regular Languages	274
7.11	Equivalence of PDA and Context Free Grammar	274
7.12	Two Stack PDA	279
7.13	Auxiliary Push Down Automata	282
7.14	Parsing and PDA (Top Down and Bottom Up)	283
7.14	Deterministic PDA and Deterministic CFL	286
	<i>Summary</i>	286
	<i>Review Questions</i>	287
	<i>Graded Exercises</i>	288
	<i>Objective Questions</i>	290
	<i>Reference for Extra Reading</i>	293
	<i>Online Sources</i>	293

8. Properties of Regular and Context Free Languages 294

Chapter Objective 294

Introduction 294

8.1 Pumping Lemma for Context Free Languages 294

8.2 Decision Properties and Algorithm 301

8.3 Closure Properties of CFLs 305

8.4 Mixing of CFLs and RLs 309

Summary 313

Review Questions 313

Graded Exercises 314

Objective Questions 314

Reference for Extra Reading 316

Online Sources 316

9. Turing Machines 317

Chapter Objective 317

Introduction 317

9.1 Model of Turing Machines 318

9.2 Definition of Turing Machine 319

9.3 Halt and Crash Conditions 321

9.4 Equivalence of Two Turing Machines 321

9.5 Representation of Turing Machines 321

9.6 Designs for Turing Machines 325

9.7 Programming Techniques 333

9.8 Turing Machine and Computation 334

9.9 Types of Turing Machines 336

9.10 Universal Turing Machine 341

9.11 Church-Turing Hypothesis 342

9.12 Language Accepted by Turing Machine 345

9.13 Recursive and Recursively Enumerable Language 345

9.14 Turing Machine and Type-0 Grammar 349

9.15 Undecidable Problems about Turing Machines 351

9.16 Turing Machine as Language Acceptor and Generator 353

9.17 Turing Transducer 354

Summary 356

Review Questions 356

Graded Exercises 359

Objective Questions 361

Reference for Extra Reading 363

Online Sources 364

10. Undecidability and Computability 365

Chapter Objective 365

Introduction 365

10.1	Unsolvable Problems Involving CFLs	366
10.2	Undecidable Problems that are Recursively Enumerable	367
10.3	Post Correspondence Problem	367
10.4	Modified Post Correspondence Problem	370
10.5	Languages that are not Recursively Enumerable	370
10.6	Context Sensitive Languages	370
10.7	Computability	371
10.8	Recursive Function Theory	371
10.9	Ackermann's Function	376
10.10	Reducing One Undecidable Problem to Another	371
10.11	Rice's Theorem	378
10.12	Computational Complexity	379
10.13	Rewriting Systems	380
10.14	Matrix Grammar	380
10.15	Markov Algorithm	381
	<i>Summary</i>	383
	<i>Review Questions</i>	384
	<i>Graded Exercises</i>	385
	<i>Objective Questions</i>	385
	<i>Reference for Extra Reading</i>	388
	<i>Online Sources</i>	389

11. NP-Completeness

390

	<i>Chapter Objective</i>	390
	<i>Introduction</i>	390
11.1	Time Complexity	391
11.2	Growth Rate of Functions	392
11.3	Polynomial Time	397
11.4	Polynomial Time Reduction	399
11.5	<i>P</i> and <i>NP</i> Classes	399
11.6	<i>NP</i> -Completeness	400
11.7	<i>NP</i> -Hard	400
11.8	Cook's Theorem	402
11.9	Some <i>NP</i> -Complete Problems	404
	<i>Summary</i>	405
	<i>Review Questions</i>	406
	<i>Graded Exercises</i>	406
	<i>Objective Questions</i>	407
	<i>Reference for Extra Reading</i>	409
	<i>Online Sources</i>	409

Appendix: Answers to Objective Questions

411

Index

415

List of Important Symbols and Notations

<i>Notation</i>	<i>Meaning</i>		
$\Rightarrow$ <i>LMD</i>	leftmost derivation	ϕ	the null set
$\Rightarrow$ <i>RMD</i>	rightmost derivation	π	the partition corresponding to equivalence states
$\vdash$	move relation	δ	the transition function
$\vdash_M^*$	move relation in machine M	Σ^*	the set of strings including ε or made from elements of Σ
U_i^n	projection function	$\varepsilon, \wedge$	the null string or string of length zero
$\$$	the right end marker of a tape	$\omega, w, \text{ or } s$	a string
$\alpha \Rightarrow_G \beta$	α drives β directly from grammar G	Σ^+	the set of strings excluding ε or made from elements of Σ
$\alpha \Rightarrow^* \beta$	α drives β in more than one steps	π_k	the partition corresponding to k -equivalence states
$\alpha \Rightarrow \beta$	α directly drives β	ω^R	the reverse of string ω
$\wedge$	logical connective <i>AND</i>	ω^T	the transpose of string ω
$\Rightarrow$	logical connective <i>IF...THEN...</i>	(V_N, Σ, P, S)	the grammar
$\vee$	logical connective <i>OR</i>	$\lfloor x \rfloor$	the largest integer $\leq x$
$\neg$	negation	$\lceil x \rceil$	the largest integer $\geq x$
Σ	set of input symbol on input tape	$ V_N $	the number of nonterminals in V_N
Γ	set of symbols on tape, set of push down symbol	$ x $	the length of string x
ψ	the left end marker of a tape	2^Q	the power set of Q

xx □ List of Important Symbols and Notations

$A - B$	the subtraction of set B from A	$Nil(x)$	image of x under nil function
$A \cap B$	the intersection of set A and B	P	set of productions
$A \subseteq B$	the set A is subset of B	$perd(x)$	image of x under predecessor function
$A \cup B$	the union of set A and B	Q	finite nonempty set of states
a	regular expression corresponding to $\{a\}$	q_0	the initial state
C_a	a nonterminal deriving $C_a \rightarrow a$	r	regular expression r
$cons(a, x)$	concatenation of a and x	r^*	the closure of regular expression r
F	the set of final states, logical false	$r_1 + r_2$	union of regular expressions r_1 and r_2
$f(x)$	image of x under f	$r_1 r_2$	concatenation of regular expressions r_1 and r_2
$f: x \rightarrow y$	function f mapping from x to y	S	the start symbol
G	the grammar	$S(x)$	image of x under successor function
ID	instantaneous description	$sbtr(x, y)$	the subtractor function
L'	the complement of L	T	logical true
$L(G)$	language generated by grammar G	$T(M)$	the set accepted by final state in PDA M
L_0	the class of type-0 languages	V_N	set of nonterminals or variables
L_{CFL}	the class of context free languages	$x \in L$	x belongs to L
L_{CSL}	the class of context sensitive languages	$x \notin L$	x does not belong to L
L_{RL}	the class of regular languages	$Z(x)$	image of x under zero function
$N(M)$	the set accepted by null or empty store in PDA M	Z_0	push down symbol

Visual Walkthrough

Chapter Objective

Each chapter contains a chapter objective which outlines the key concepts of the chapter.

Benefit In a short paragraph, the Chapter Objective makes the reader aware of the concepts to be discussed in the chapter.



Mathematical Preliminaries

❖ Chapter Objective ❖

In this chapter we will discuss some elementary topics of discrete mathematics which are a prerequisite for automata theory. We start our discussion from set theory by covering basic notations of sets, then types of sets and various operations on set. We will discuss what alphabets are and how strings are generated from sets. Then we will show how strings form a language. In this sequence, we will describe functions, relations and graphs. Finally we will discuss proof techniques. Our emphasis will be on mathematical induction as it will be applied in so many later topics.

□ Introduction □

Discrete mathematics is the study of mathematical structures that are fundamentally discrete rather than continuous. Discrete mathematics has also been characterised as the branch of mathematics dealing with sets, functions, relation, graphs, trees, and proof techniques.

Discrete mathematics has become popular in recent years because of its applications to theoretical computer science. Concepts and notations from discrete mathematics have been found useful in studying and describing objects. The problems in computer algorithms and programming languages have been solved by discrete mathematics. At an advanced level discrete mathematics has applications in cryptography, automated theorem proving, and software development.

The history of discrete mathematics has involved a number of challenging problems which have focused attention within areas of the field. In graph theory, much research was motivated by attempts to prove the four color theorem, first stated in 1852, but not proved till 1976 (by Kenneth Appel and Wolfgang Haken, using substantial computer assistance).

In logic, the second problem on David Hilbert's, is a typical problem among the list of open problems presented in 1900. It was to prove that the axioms of arithmetic are consistent. Kurt Gödel's second incompleteness theorem was proved in 1931. Hilbert's tenth problem determined whether a given polynomial Diophantine equation with integer coefficients has an integer solution. By the year 1970, Yuri Matiyasevich proved that this could not be done.

Brief Introduction

A brief introduction mentioning the history of the topics to be discussed is given following the chapter objective.

Benefit This helps in providing to a brief background of the chapter to the readers.

Diagrams

Over 200 diagrams are given in the text.

Benefit Diagrams present the text material in a clear and lucid manner. These enable the reader to effectively understand the various automata concepts.

ser checks input stream for syntactic correctness and prepares a framework for subse processing. A parser is implemented as a push down automaton (PDA). YACC (yet an compiler) is a tool that takes context free grammar and generates a parser for that lang C program).

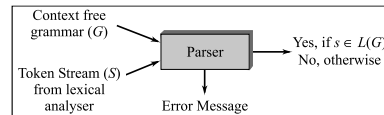


Fig. 7.22 Function of a parser

ser works on a stream of tokens as shown in the figure below. The smallest item is a token

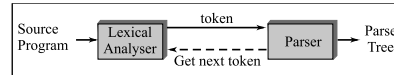


Fig. 7.23 Working of a parser

❖ Did you Know ?

In 1959, Michael Rabin (with Dana Scott) introduced the notion of nondeterminism. This initiated an idea by which the idea that a computer program can make guesses and the concept of parallelism. Rabin defined the inherent computational difficulty of a problem by inspiring from a puzzle posed by John McCarthy.

In 1976, Michael Rabin and Dana Scott shared the Turing award for their joint paper "Finite Automata and Their Decision Problem". This paper introduced the idea of nondeterministic machines. This concept proved to be an extremely valuable theory in mathematical models of automatic machines. This classic paper has been a continuous source of inspiration for further work in this field.

Description of NDFA

A nondeterministic finite automaton is defined as 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

$Q \rightarrow$ finite non-empty set of states

$\Sigma \rightarrow$ set of inputs called alphabet

$\delta \rightarrow$ transition function which maps from $Q \times \Sigma \rightarrow 2^Q$

$q_0 \rightarrow$ initial state (i.e., there is only initial state), and $q_0 \in Q$

$F \rightarrow$ set of final states, and $F \subseteq Q$.

❖ Good to Know!

2^Q is the number of elements in power set of Q , the set of all subset of Q .

For the formal description of transition δ of NDFA, let us consider following example:

Good to Know/ Did You Know Boxes

Useful information related to the topic are given in highlighted boxes throughout the text.

Benefit These will help students gain additional information and facts about the topic and its history.

Definitions

Key definitions related to the topic are given in highlighted boxes throughout the text.

Benefit These will help the students to quickly locate and revise the important definitions before an exam.

construct a derivation tree. Therefore, a derivation 's' may be obtained either by leftmost derivation or rightmost derivation.

❖ Definitions

❖ **A-tree** A subtree looks like a derivation tree except that the label of the root node may not be S (the start symbol) if and only if S does not appear in the right side of any production. It is called an A-tree if the label of its root is A . Similarly if there is another subtree whose label of root node is B then it is called a B-tree.

❖ **Working string** The arrow symbol ' $\Rightarrow$ ' is employed between the unfinished stages of the generation of a string or a derivation. It means that there exists a possible substitution in $XY \Rightarrow abY$ if there exists a production $X \rightarrow ab$. These unfinished stages are strings of terminals and nonterminals that are generally called working strings.

❖ **Internal nodes** A node in a derivation tree whose label is a variable (or say nonterminal) is called a leaf node or an external node. For example, if the label of a node is $A \in V_n$, then A is an internal node.

❖ **Leaf node** These are also called external nodes. A node in a derivation tree, whose label is a terminal $a \in \Sigma$ or $\wedge$ is called a leaf node or external node or simply a leaf.

❖ **Sentential Form** A string of terminals and variables (say α) is called a sentential form if $S \Rightarrow \alpha$, where $S \in V_n$ and $\alpha \in (V_n \cup \Sigma)^*$.

❖ **Nullable variable** A variable E in CFG G is said to be nullable if it derives (or say generates) $\wedge$ i.e., $E \Rightarrow \wedge$.

Solved Examples

Close to 150 Solved Examples are present in the text.

Benefit Stepwise approach used for solving examples make it easier for the reader to understand the problem and henceforth, apply the learnt concepts.

Example 5.3

Show that the language $L = \{0^n \mid p \text{ is a prime number}\}$ is not regular.

Solution

Step 1 Let us suppose L is regular language and if get a contradiction then L is not regular. Let n be number of states in the finite automaton accepting language L .

Step 2 Let p be the prime number greater than n . By pumping lemma, we have $z = 0^p = uvw$ and $|z| = |uvw| = |0^p| = p$.

By using pumping lemma, we can write $z = uvw$, with $|uv| \leq n$ and $|v| \geq 1$. u, v, w are the strings of 0's. Therefore, $v = 0^m$ for some $n \geq m \geq 1$. So $|v| = m$.

Step 3 Let $i = p + 1$. Then,

$$\begin{aligned} |uv^i w| &= |uvw| + |v^{i-1}| \\ &= p + (i-1)m \\ &= p + (i+1-1)m \quad \text{by } i = p+1 \\ &= p + pm \\ &= p(1+m). \end{aligned}$$

By pumping lemma $uv^i w \notin L$ with $i = p+1$, because $|uv^i w| = p(1+m)$, and $p(1+m)$ is not a prime number, since it is divisible by p and $(1+m)$ where $(1+m) \geq 2$. Here we get a contradiction to say that L is not regular.

Example 5.4

By using pumping lemma show that the language $L = \{a^{n^2} \mid n > 1\}$ is not regular.

Solution

Step 1 Let us assume that $L = \{a^{n^2} \mid n > 1\}$ is regular. If the language $L = \{a^{n^2} \mid n > 1\}$ is regular then by mathematical induction $L = \{a^{(m+1)^2} \mid m > 0\}$ is also regular. Let us assume $n = m + 1$.

Step 2 Let $z = a^{(m+1)^2}$ then $|z| = (m+1)^2$.

By pumping lemma we write

$$z = uvw \text{ such that } |uv| \leq (m+1)^2 \text{ and } |v| \geq 1.$$

Step 3 As the string $a^{(m+1)^2}$ contains occurrences of a 's we have

$$z = a^{(m+1)^2} = a^{m^2+2m+1} = a^{m^2} a^{2m} a$$

Let $u = a^{m^2}$, $v = a^{2m}$, and $w = a$, then by pumping lemma, we have

Theorem 5.5 The set of strings accepted by a finite automaton M with n states is:

- (i) nonempty if and only if the finite automaton accepts a sentence of length less than n .
- (ii) infinite if and only if the automaton accepts some strings of length where $n \leq l < 2n$.

This shows that there is an algorithm to determine whether a finite automaton accepts zero, a finite number or an infinite number of strings.

Proof (i) Suppose finite automaton M accepts a nonempty set. Let ' s ' be a string as short as any other strings accepted. By using pumping lemma we can write $|s| < n$ if the string s is shortest and $|s| \leq n$ otherwise. Then we can write $s = uvw$, and uv is a shorter string in the language. This shows whether $L(M)$ (language accepted by finite automaton M) is empty or not. Hence there is such a procedure guaranteed to halt.

(ii) If the string $s \in L(M)$ and $n \leq |s| < 2n$, then by using pumping lemma, we can say that $L(M)$ is infinite. Let us suppose $s = s_1 s_2 s_3$, and for all i , $s_i s_2 s_3 \in L(M)$. Conversely if $L(M)$, the language accepted by finite automaton M is infinite, then there exists $s \in L(M)$, where $|s| \geq n$.

If there is no word whose length lies between n and $2n-1$, suppose the length of string s is at least $2n$, but as short as any string is $L(M)$ whose length is greater than or equal to $2n$. Again by using pumping lemma we can write $s = s_1 s_2 s_3$ with $n \leq |s_2| < 2n$ and $s_1 s_2 \in L(M)$. Then either s was not a shortest string of length $2n$ or more, or $n \leq |s_2| < 2n-1$, a contradiction in either case.

This decides whether $L(M)$ is infinite. For example, if any string of length between n and $2n-1$ is in $L(M)$. Hence there is such a procedure that is guaranteed to halt.

Lemma 5.1 (Pumping Lemma) If $M = (Q, \Sigma, \delta, q_0, F)$ is a finite automaton with n states. Suppose L is a regular set accepted by finite automaton M . Let $z \in L$ and $|z| = m$. If $m \geq n$, then there exists u, v, w such that $z = uvw$ with $|v| \neq 0$ and $uv^i w \in L$ for each positive integer values of i (i.e., $i \geq 0$).

Proof The pumping lemma considers that a string $z \in L$ has length greater equal to m and the number of states in FA representing z are n such that $m \geq n$. It is possible only when there exists some indirect or direct loop (say closure, i.e., present state and next states are same on input $a \in \Sigma$) in at least one state of FA. In practice we see that if there is no closure on any state of FA, the length of string is one less than total number of states.

Let the string z be composed as

$$z = a_1 a_2 a_3 \dots a_m \text{ with } m \geq n$$

$$\delta(q_0, a_i a_2 a_3 \dots a_m) = q_i \text{ for } i = 1, 2, 3, \dots, m$$

and

$$Q = \{q_0, q_1, q_2, \dots, q_{m-1}\}$$

Theorems, Lemmas and Proofs

Over 70 theorems are provided in the text.

Benefit Theorems and proofs make the reader realize how mathematics works, and how one can get from one step to the next.

Summary

A concise Summary is provided at the end of each chapter in a bulleted format.

Benefit It underlines the important concepts and aids in reviewing the ideas covered in the chapter at a faster pace.

Summary

- **Context-Free Grammar** A context-free grammar (CFG) is defined as 4-tuples (V_N, Σ, P, S) , where P is a set of production rules of the form
 one nonterminal $\rightarrow$ finite string of terminals and/or nonterminals
- **Context Free Languages** The languages generated by the context-free grammars are called context-free languages.
- **Deterministic Context-Free Language** A language L is a deterministic context-free language (abbreviated as DCFL) if there is a deterministic push down automaton (abbreviated as DPDA) accepting language L .
- **Deterministic Context-Free Grammar** A context-free grammar is deterministic if and only if its rules allow a (left-to-right, bottom-up) parser to proceed without having to guess the correct continuation at any point.
- **Derivations** A derivation can be defined as the process to generate a string of terminals by replacing the nonterminals in the right hand side of a production.
- **Parse Trees** The strings generated by a context-free grammar $G = (V_N, \Sigma, P, S)$ can be represented by a hierarchical structure called tree.
- **Yield of a Derivation Tree** The yield of a derivation tree is the concatenation of the labels of leaf nodes in left-to-right ordering without repetition.

Objective Questions

1. A problem is *NP*-complete if
 - (a) solution can be verified quickly
 - (b) a quick algorithm to solve this problem can be used to solve all other *NP* problems quickly
 - (c) it can be solved in polynomial time
 - (d) all of the above
2. If a problem A is polynomial time reducible to B (i.e., $A \leq_p B$) and $B \in P$, then
 - (a) problem A is also in P
 - (b) problem A can not be in P
 - (c) problem A is not harder than B
 - (d) both (a) and (b)
3. If $A \leq_p B$ and B is in *NP*, then
 - (a) if B is recursive, A is also recursive
 - (b) A is in *NP*
 - (c) both (a) and (b)
 - (d) none of these.
4. Which of the following problems is not *NP*-complete
 - (a) satisfiability problem
 - (b) vertex cover problem
 - (c) integer linear programming problem
 - (d) none of these

Objective Questions

Over 280 Objective-Type Questions with Answers are present in the text.

Benefit These help readers have a quick revision of the concepts discussed in the chapter and prepare for competitive examinations.

Review Questions with Hints

Over 175 Review Questions are given in the text.

Benefit These are set to test the theoretical grasp of the principles discussed and check the understanding of the student. These also serve the purpose of preparing assignment/test exercise questions. The hints given gently guide the student in arriving at the answer.

Review Questions

- 10.1 Prove that if L_1 and L_2 are languages over alphabets Σ_1 and Σ_2 , respectively, and $L_1 \leq L_2$, then if L_2 is recursive, L_1 is also recursive.
- 10.2 Show that, for arbitrary context-free grammar G_1 and G_2 , the problem " $L(G_1) \cap L(G_2)$ is context-free" is undecidable.
- 10.3 Prove that for $|\Sigma| = 1$, that the post correspondence problem is decidable.
- 10.4 Give the solution of giving post correspondence problems:
 - (i) $A = \{aab, aabb, bb, bab\}$ and $B = \{ab, bbb, bbb, aba\}$
 - (ii) $A = \{ab, a, b\}$ and $B = \{abb, ba, bb\}$
 - (iii) $A = \{bb, baa, bbb\}$ and $B = \{bbb, aab, bb\}$
 - (iv) $A = \{a, bba, aab\}$ and $B = \{ba, aaa, ba\}$
- 10.5 Show that concatenation function (denoted by *concat*) is a primitive recursive function.
- 10.6 Define a function for subtraction.
- 10.7 Evaluate Ackermann's function for $A(2, 3)$ and $A(3, 2)$.
- 10.8 Determine Ackermann's function for $A(10, 8)$

Hints

10.3 For a single-letter alphabet, there is a solution of *PCP* if and only if there is some subset I of $\{1, 2, 3, \dots, n\}$ such that

$$\sum_{i \in I} |w_i| = \sum_{i \in I} |v_i|$$

Since there exists only a finite number of subsets, they can all be checked and therefore we can say that the problem is decidable.

- 10.4 (i) The problem has a solution for the sequence given by $x_3 x_1 x_1 = y_3 y_1 y_1$.
 (ii) The problem has no solution because for each substring $x_i \in A$ and $y_j \in B$, we have

(iv) The problem has no solution because no pair has a common non-empty initial substring.

10.5 The concatenation function can be defined as $concat(x, y) = xy$. It can be represented by composition and recursion to the initial functions.

10.6 First we define a predecessor function as:

$$\begin{aligned} pred(0) &= 0, \\ pred(y + 1) &= y. \end{aligned}$$

by using this definition we define the subtraction function as:

$$\begin{aligned} subtr(x, 0) &= x, \\ subtr(x, y + 1) &= pred(subtr(x, y)). \end{aligned}$$

$$(i) 1 + 2 + 3 + \dots + n = \frac{n \times (n+1)}{2}, (ii) 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n \times (n+1)(2n+1)}{6}$$

1.20 Give the difference between proof by contradiction and proof by contra positive.

Graded Exercises

- * 1.1 Prove by contradiction that the product of odd integers is not even.
- * 1.2 Prove that for any positive integer n , $2^n > n$.
- ** 1.3 What is wrong with the following proof by mathematical induction?
We will prove that for any positive integer n , n is equal to 1 more than n . Assume that $P(k)$ is true.
 $k = k + 1$
Adding 1 to both side of this equation we get
 $k + 1 = k + 2$
thus, $P(k + 1)$ is true.
- ** 1.4 Let f be a function, $f: P \rightarrow Q$, show that for all subsets A and B of P , $f(A \cap B) \subseteq f(A) \cap f(B)$.
- * 1.5 Determine whether the following function is onto or it is one-to-one:
 $f: R \rightarrow R, f(x) = 2x + 1$
- *** 1.6 Prove that the number of leaf nodes in a binary tree graph is $\frac{n+1}{2}$, where n is the number of vertices.
- * 1.7 Show that the function $f: P \rightarrow P$, give by $f(x) = 2x$ in one-to-one but not an onto.
- * 1.8 Show that the function $f: P \rightarrow P - \{1\}$, give by $f(x) = \frac{x+1}{x-1}$ in an onto function.
- * 1.9 Prove that if x is an odd integer then x^2 is also odd.
- * 1.10 Prove that the sum $(P + Q)$ of any two integers P and Q is odd if and only if exactly one of the integers P or Q is odd.

* Difficulty level 1

** Difficulty level 2

*** Difficulty level 3

Hints

1.9 Let $x = 2n + 1$, where $n \in N$

The square of both sides gives

$$\begin{aligned} x^2 &= (2n + 1)^2 \\ &= 4n^2 + 4n + 1 = 4(n^2 + n) + 1 \end{aligned}$$

The value $4(n^2 + n) + 1$ is an odd integer.

1.10 Assume P is odd and Q is even, then

$$\begin{aligned} P + Q &= 2n + 1 + 2m \quad \text{for } P = 2n + 1, \text{ and } Q = 2m \\ &= 2(m + n) + 1 \\ &= \text{odd} \end{aligned}$$

Graded Exercises with Hints

Over 100 Graded Exercises with hints are given.

Benefit These help to check the students' analytical grasp of the concepts. The gradation of the questions into easy (*), difficult (**) and very difficult (***) help the student gain confidence in his/her ability of solving various range of simple and complex problems.

References for Extra Reading/Online Sources

At the end of each chapter, a comprehensive list of useful references and online sources are listed.

Benefit These offer helpful guidance in finding material for further studies on the topics covered in the chapter.

248 □ Theory of Automata, Languages and Computation

References for Extra Reading

1. John C Martin, *Introduction to Languages and the Theory of Computation*. McGraw Hill, 2003.
2. Łukasiewicz, Jan, *Elements of Mathematical Logic*. Translated from Polish by Olgierd Wojtasiewicz, New York, Macmillan, 1964.
3. L. Bala Sundara Raman, Ishwar, S, Sanjeeth Kumar Ravindranath, *Context-free Grammar for Natural Language Constructs*. Proceedings of Tamil Internet, Chennai, 2003.
4. Michael Sipser, *Introduction to the Theory of Computation*, PWS Publishing, 1997.
5. Nederhof, M. J., *Regular approximations of CFLs: A grammatical view*, In International Workshop on Parsing Technologies, Massachusetts Institute of Technology, 1997.
6. S. Ginsburg and E. H. Spanier, *Quotients of context-free languages*, Journal of the Association for Computing Machinery 10:4, 487–492, 1963.
7. Sheila A. Greibach, *A New Normal-Form Theorem for Context-Free Phrase Structure Grammars*, Journal of the ACM, 1965.
8. Sheila A. Greibach, *An Infinite Hierarchy of Context-Free Languages*, Journal of the ACM, 1969.
9. Shieber, Stuart, *Evidence against the context-freeness of natural language*. Linguistics and Philosophy, 1985.

Online Sources

1. <http://www.cs.sun.ac.za/~rw324/documents/rw324w5.pdf>
2. <http://www.kornai.com/MatLing/cflfinal.pdf>
3. www.brics.dk/~amoeller/papers/ambiguity/journal.pdf
4. <http://www.eecs.harvard.edu/~shieber/Biblio/Papers/shieber85.pdf>
5. <http://www.cs.uiowa.edu/~hzhang/c135/contextfree-4p.pdf>



Mathematical Preliminaries

❖ Chapter Objective ❖

In this chapter we will discuss some elementary topics of discrete mathematics which are prerequisite for automata theory. We start our discussion from set theory by covering basic notations of sets, then types of sets and various operations on set. We will discuss what alphabets are and how strings are generated from sets. Then we will show how strings form a language. In this sequence, we will describe functions, relations and graphs. Finally we will discuss proof techniques. Our emphasis will be on mathematical induction as it will be applied in so many later topics.

□ Introduction □

Discrete mathematics is the study of mathematical structures that are fundamentally discrete rather than continuous. Discrete mathematics has also been characterised as the branch of mathematics dealing with sets, functions, relation, graphs, trees, and proof techniques.

Discrete mathematics has become popular in recent years because of its applications to theoretical computer science. Concepts and notations from discrete mathematics have been found useful in studying and describing objects. The problems in computer algorithms and programming languages have been solved by discrete mathematics. At an advanced level discrete mathematics has applications in cryptography, automated theorem proving, and software development.

The history of discrete mathematics has involved a number of challenging problems which have focused attention within areas of the field. In graph theory, much research was motivated by attempts to prove the four color theorem, first stated in 1852, but not proved till 1976 (by Kenneth Appel and Wolfgang Haken, using substantial computer assistance).

In logic, the second problem on David Hilbert's, is a typical problem among the list of open problems presented in 1900. It was to prove that the axioms of arithmetic are consistent. Kurt Gödel's second incompleteness theorem was proved in 1931. Hilbert's tenth problem determined whether a given polynomial Diophantine equation with integer coefficients has an integer solution. By the year 1970, Yuri Matiyasevich proved that this could not be done.

1.1

SET THEORY

1.1.1 Set Notation and Operations

A set is a collection of elements without any structure or order. In other words a set may be defined as collection of objects (i.e., the elements of members of set). The statement “0 is an element of Σ ”, or equivalently, “0 belongs to Σ ” is written as $0 \in \Sigma$.

The statement that “0 is not element of Σ ” that is the negation of $0 \in \Sigma$, is written as $0 \notin \Sigma$. Some more examples of sets are as follows:

- (i) $A_1 = \{1, 3, 5, 7, 9, \dots\}$, a set of all odd numbers.
- (ii) $A_2 = \{a, e, i, o, u\}$, a set of all vowels in English alphabets.
- (iii) $A_3 = \{1, 2, 3, 5, 7, 11, 13, \dots\}$, a set of all prime numbers.
- (iv) $A_4 = \{aa, aaaa, aaaaaa, \dots\}$, set of all strings of a 's with even occurrences.

The method by which the above sets are represented is called **Roster** or **tabulation** method. In this method the elements of the set are listed within brackets (i.e., within $\{\}$) separated by commas.

Another method to represent a set is called the set-builder method. In this method the elements of the set are represented by concluded properties (characteristics). For example,

$$A = \{w \mid \text{properties of } w\}$$

Suppose we wish to represent above sets A_1 , A_2 , A_3 , and A_4 by set builder method, we have the following equivalent representations :

$$A_1 = \{x \in N \mid x \% 2 = 1 \text{ and } N \text{ is a natural number}\}$$

$$A_2 = \{x \in E \mid x \text{ is a vowel from English alphabet } E\}$$

$$A_3 = \{x \in P \mid P \text{ is a prime number}\}$$

$$A_4 = \{x \in a^n \mid n \geq 2 \text{ and } n \% 2 = 0\}$$

Cardinality The number of distinct elements in a set is called the cardinality of the set. For example, there is a set A , having elements

$$A = \{0, 1, 2\}.$$

Number of distinct element in $A = 3$, therefore the cardinality of set A is 3.

1.1.2 Types of Sets

Equivalent Sets Two finite sets A and B will be called equivalent sets if and only if they have the same cardinality. For example, sets A and B are equivalent sets represented as

$$A = \{0, 1, 2\}, B = \{a, b, c\}.$$

Equal Sets Two sets A and B will be said to be equal set if and only if they have the same elements. For example, the set A and B represented as

$$A = \{a, e, i, o\}, B = \{a, e, i, o\}$$

are equal sets.

Empty Sets An empty set is defined as a set with no elements. It is also called a null set. It is denoted by ϕ . An empty set is a sub set of all sets. A set consisting of at least one element is called non-empty or non-void set.

Singleton Sets A set containing only one (single) element is called a singleton set. For example, the following sets are singleton sets:

$$A_1 = \{0\}, A_2 = \{1\}, A_3 = \{x\}.$$

Sub-Sets If every element in a set A is also an element of set B , then A is called a subset of B . It is written as

$$A \subseteq B \text{ or } B \supseteq A.$$

The empty set is a subset of every set.

Super Set When A is a subset of B (i.e., $A \subseteq B$) then B is a super set of A which is written as $B \supset A$. Let us consider sets A and B defined in the following ways :

1. $A = \{1, 2, 3\}, B = \{1, 2, 3, 4, 5\}$ then $A \subseteq B$.
2. $A = \{x \mid x \text{ is a vowel in English alphabets}\},$
 $B = \{x \mid x \text{ is an English alphabet}\},$
then $A \subseteq B$.
3. $A = \{\alpha, \beta, \gamma\}, B = \{\gamma, \beta, \alpha\},$ then $A \subseteq B$ and $B \subseteq A$.

Proper Subsets If $A \subseteq B$ then it is still possible that $A = B$. When $A \subseteq B$ but $A \neq B$, we say A is a proper subset of B . We write it $A \subset B$ when A is a proper subset of B . For example, $A = \{1, 2\}, B = \{1, 2, 3\}, C = \{1, 3, 2\}$, then both A and B are subsets of C , but A is a proper subset of C , whereas C is not a proper subset of B since $C = B$.

Finite Sets A set is said to be finite if it contains exactly m distinct elements where m denotes some non-negative integer, otherwise, the set is said to be infinite. For example, the empty set ϕ , and the set of digits in the decimal number system are finite sets, whereas the set of all odd positive integers, $\{1, 3, 5, 7, \dots\}$, is infinite.

Power Set The power set of a set S is denoted by 2^S . If we have a set Q defined as $Q = \{q_1, q_2\}$, then the power set of Q has elements $(\phi, \{q_1\}, \{q_2\}, \{q_1, q_2\})$. It is a set of all subsets of a given set.

Universal Set A set U is called universal set if U is the superset of all the sets which are under our consideration. For example, if three sets A, B and U are defined as:

$$\begin{aligned} A &= \{1, 3, 5, 7, 9\} \\ B &= \{2, 4, 6, 8, 10\} \\ U &= \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\} \end{aligned}$$

As we see that $A \subseteq U$ and $B \subseteq U$ therefore U is the universal set and U is the superset of A and B .

Disjoint Sets Two sets A and B are said to be disjoint set if

$$A \cap B = \{\phi\}$$

For example, if set A and B are defined as

$$A = \{a, b, c, d\}, B = \{A, B, C, D\}$$

then $A \cap B = \{\phi\}$.

Therefore, A and B are disjoint sets.

1.1.3 Operations on Sets

Union and Intersection The union of two sets A and B , is denoted by $A \cup B$. It is the set of elements which belong to A or B . For example,

$$A \cup B = \{x \mid x \in A \text{ or } x \in B\}$$

The intersection of two sets A and B is denoted by $A \cap B$. It is the set of elements which belong to both A and B . For example,

$$A \cap B = \{x \mid x \in A \text{ and } x \in B\}$$

Let us consider two sets A and B defined as

$$(i) \quad A = \{1, 2, 3, 4, 5, 6\}, B = \{2, 4, 6, 8, 10, 12\},$$

$$\text{then } A \cup B = \{1, 2, 3, 4, 5, 6, 8, 10, 12\} \text{ and } A \cap B = \{2, 4, 6\}$$

$$(ii) \quad A = \{x \mid x \text{ is an English alphabet}\}, B = \{x \mid x \text{ is vowel in English alphabet}\}$$

$$\text{then } A \cup B = \{x \mid x \text{ is an English alphabet}\}$$

$$= \{a, b, c, d, \dots, z\}$$

$$\text{and } A \cap B = \{x \mid x \text{ is vowel in English alphabet}\}$$

$$= \{a, e, i, o, u\}$$

$$(iii) \quad A = \{a, b, c\}, B = \{0, 1, 2, 3\}, C = \{A, B\}$$

then

$$A \cup B \cup C = \{a, b, c, 0, 1, 2, 3, A, B\}$$

and

$$A \cap B \cap C = \{\emptyset\}, \text{ i.e., an empty or null set.}$$

Complement The absolute complement or simply complement of a set A , denoted by A' or $\overline{A}$, is the set of elements which belong to U but which do not belong to A , i.e.,

$$A' = \{x \mid x \in U, x \notin A\}$$

For example, if sets A and U are defined as

$$A = \{2, 4, 6, 8, 10\},$$

$$U = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\},$$

$$\text{then } A' = \{1, 3, 5, 7, 9\}.$$

Set Difference The set difference of a set A with respect to set B (denoted by $A - B$) is the set of all elements that are in A but not in B . The set difference of a set A with respect to set B is defined as

$$A - B = \{x \mid x \in A \text{ and } x \notin B\}$$

For example, if two sets A and B are given as

$$A = \{a, c, d, g, h, i\}$$

$$B = \{a, b, c, e, f, g, i\}$$

then

$$A - B = \{d, h\}$$

and

$$B - A = \{b, e, f\}$$

Symmetric Difference The symmetric difference of two sets A and B (denoted by $A \Delta B$) is defined as

$$A \Delta B = (A - B) \cup (B - A)$$

For example, if two sets A and B are given as

$$A = \{1, 3, 5, 7, 10\}$$

$$B = \{1, 2, 4, 5, 6, 8, 9, 10\}$$

then

$$A - B = \{3, 7\}$$

and

$$B - A = \{2, 4, 6, 8, 9\}$$

Therefore, $A \Delta B = \{3, 7\} \cup \{2, 4, 6, 8\}$

$$= \{2, 3, 4, 6, 7, 8\}$$

Representation of Set Operations By using a Venn diagrams set operations can be represented very easily. Venn diagrams are nothing but diagrammatic representation using rectangles and circles. In a Venn diagram a universal set is represented by a rectangle and the remaining sets are represented by circles or ellipses. Let us look on the following Venn diagrams:

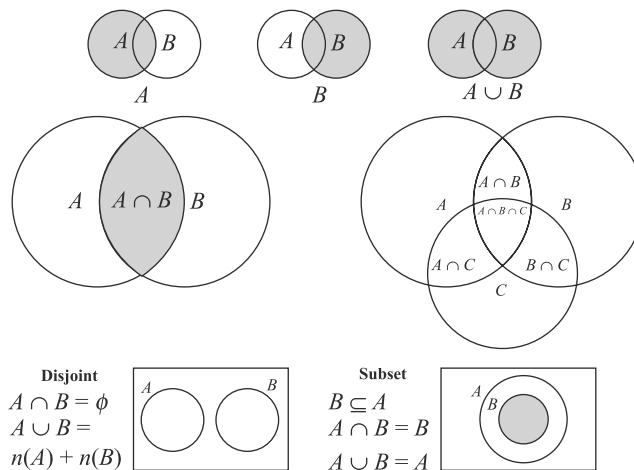


Fig. 1.1 Representation of set operations by Venn diagrams

The shaded (or say filled by a pattern) area in the above Venn diagram shows the result of a set operation.

1.1.4 Set Identities

Following table shows some important identities used in the simplification of set operations:

Table 1.1 Set Identities

Identity	Law
$A \cup \phi = A, A \cap U = A, U$ is universal set	Identity Laws
$A \cup U = U, A \cap \phi = \phi$	Domination Laws
$A \cup A = A, A \cap A = A$	Idempotent Laws
$(A')' = A$	Complementation Laws
$A \cup B = B \cup A, A \cap B = B \cap A$	Commutative Laws
$A \cap (B \cup C) = (A \cap B) \cup (A \cap C), A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$	Distributive Laws
$(A \cup B)' = A \cap B', (A \cap B)' = A' \cup B'$	DeMorgan's Laws
$(A \cup B) \cap A = A, (A \cap B) \cup A = A$	Absorption Laws
$A - B = A \cap B', B - A = B \cap A'$	Difference Laws
$A \Delta B = (A - B) \cup (B - A), B \Delta A = (B - A) \cup (A - B)$	Symmetric Difference Laws

1.1.5 Representation of Sets in a Computer

Suppose we wish to represent a set A in a computer and if the set A is of finite order then it has finite number of distinct elements. For representation, an arbitrary order is specified in the set A . For example, if set A has elements

$$A = \{1, 2, 3, 4, 5, \dots, n\}$$

Then computer representation of set A will be 1111.....1 (i.e., n occurrences of 1's).

The above is the bit string of length n and all bits are 1. If some element between 1 to n is missing then the corresponding bit is assigned as 0. For example if set A has elements

$$A = \{1, 4, 5, 7, 8, 9, 10\}$$

Then A will be represented by bit string 1001101111, where missing elements in the range of set are represented by 0's. Therefore it is a must to decide the range of set. Let a universal set be $U = \{1, 2, 3, \dots, 12\}$

then

$$\{3, 4, 5\} = 001110000000$$

and

$$\{2, 3, 4, 5, 6\} = 011111000000.$$

1.1.6 Relations between Sets

Partitions of a Set Let A be a nonempty set. The family of set $\{A_1, A_2, A_3, \dots, A_n\}$ is a partition of the set A if

$$(i) \quad A = \bigcup_{i=1}^m A_i,$$

$$(ii) \quad A_i \cap A_j = \phi, \text{ if } i \neq j.$$

For example, if $A = \{a, b, c, d, e, f, g, h\}$, and $A_1 = \{a, b, c, f\}$, $A_2 = \{d, e, g, h\}$, then $\{A_1, A_2\}$ is a partition of A .

Dual (Duality Principle) Let A be any identity involving set, and set operations are $\cup$ and $\cap$, if B is a set obtained by replacing $\cup$ by $\cap$, and replacing $\cap$ by $\cup$, ϕ by U , and U by ϕ , then B is also true and is called dual of A . for example, the dual of

$$X \cup \{Y \cap X\} = X$$

is

$$X \cap \{Y \cup X\} = X.$$

Cartesian Product The set of all ordered pairs of elements (x, y) if $x \in A$ and $y \in B$, is called the Cartesian product of sets A and B . The Cartesian product of sets A and B is denoted by $A \times B$. the Cartesian product $A \times B$ is defined as

$$A \times B = \{(x, y) \mid x \in A, y \in B\}$$

For example, if sets A and B are given as

$$A = \{1, 2\}, \text{ and } B = \{3, 4, 5\}$$

then

$$A \times B = \{(1, 3), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5)\}$$

and

$$B \times A = \{(3, 1), (3, 2), (4, 1), (4, 2), (5, 1), (5, 2)\}$$

1.2

ALPHABETS

An alphabet can be defined as a finite set of symbols. These are the input symbols from which strings are constructed by applying certain operations. In this book we denote alphabet by the set Σ . The common alphabets in practice are:

1. $\Sigma = \{0, 1\}$, for binary sequences,
2. $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$, for decimal numbers
3. $\Sigma = \{a, b, c, \dots, z\}$, for character string in lowercase.
4. $\Sigma = \{A, B, C, \dots, Z\}$, for character string in uppercase.

1.3

STRINGS AND LANGUAGES

While we combine some symbols (or say alphabets) with each other by applying some rule we get a new string. The string can also be obtained by applying an operation on a particular symbol itself. If the operation (for example some arithmetic operation) is applied on numbers rather than the character symbols, we obtain a numeric value.

The alphabets A is a set of strings from a language while using some criteria. For example, let us consider the set of all natural numbers N . If we allow only even numbers among from N , then the collection or set of all even integers will be called language of all even integers, and it can be represented as

$$L = \{0, 2, 4, 6, \dots, 100, 102, \dots, 22562426, 22562428, \dots\}$$

$$= \{n \in N \mid N \bmod 2 = 0\}$$

The major operations that can be applied on alphabets are union, intersection, concatenation, etc. If we wish to obtain a string without accessing any alphabet we get a string of length zero and such string is usually denoted by ε (epsilon), π (pie), or λ (lambda).

❖ Good to Know!

In this book an empty string is denoted by either ε (epsilon) or $\wedge$ (pie). Some text book authors have used λ (lambda). Another symbol like epsilon (ε), is $\in$ (belongs to). Readers are requested to read accordingly.

Length of String The length of string is the number of letters in the string. Suppose we define the function 'length' to compute the length of a string. For example, if $a = xxxx$ is the string in the language L , then

$$\text{length}(a) = 4$$

if $m = 329cwy$ is in language L , then

$$\text{length}(m) = 6$$

or we could write also the way in L

$$\text{length}(gb) = 2.$$

In any language which includes the empty string $\wedge$, we have

$$\text{length}(\wedge) = 0.$$

Reverse of String Now we introduce a new function 'reverse'. If ω is a word in language L , then $\text{reverse}(\omega)$ is the same string of letters traversed backward, called the reverse of ω , even if this backward string is not a word in language L . Consider following examples :

$$\text{reverse}(xyz) = zyx$$

$$\text{reverse}(2361) = 1632.$$

Concatenation Basically concatenation is the operation applicable to strings to combine them. A particular string can be concatenated with any string and itself also. For example, a string $r_1 r_2 r_3$ can be treated as:

1. concatenation of r_1 and $r_2 r_3$
2. concatenation of $r_1 r_2$ and r_3
3. concatenation of $\wedge$ and $r_1 r_2 r_3$
4. concatenation of $r_1 r_2 r_3$ and $\wedge$

The concatenation operation is also applicable to sets. For example,

$$\{a, b\} \{a, b\} = \{aa, ab, ba, bb\}.$$

Palindrome Let us define special strings language called PAL over the alphabet $\Sigma = \{a, b\}$

$$\text{PAL} = \{ \wedge, \text{all string 's' such that } \text{reverse}(s) = s \}.$$

If we start the listing of elements in PAL, we have

$$\text{PAL} = \{ \wedge, a, b, aa, bb, aba, bab, aaa, bbb, \dots \}.$$

Sometimes, when we concatenate two elements in PAL, we obtain another palindrome such as when a is concatenated with 'aaa'. Here PAL contains all palindromes (even length as well as odd length) over $\{a, b\}$. If we wish to represent all palindromes over $\{a, b\}$ of even length, we can write

$$\text{PAL-E} = \{\text{set of all strings 's' made from } a\text{'s and } b\text{'s such that } s = \text{reverse}(s) \text{ and } |s| \% 2 = 0\}$$

Similarly, all odd length palindromes over $\{a, b\}$ can be represented as

$$\text{PAL-O} = \{\text{set of all strings 's' made from } a\text{'s and } b\text{'s such that } s = \text{reverse}(s) \text{ and } |s| \% 2 = 1\}$$

Prefix and Suffix A prefix of a string is a substring of leading symbols of that string. 'w' is a prefix of 'X' if there exists y in Σ^* such that $X = wy$. Then we write $w \leq X$. For example, the string '123' has 4 prefixes $\wedge, 1, 12, 123$.

A suffix of a string is a substring of trailing symbols of that string. 'w' is a suffix of 'X' if there exists $y \in \Sigma^*$ such that $X = yw$. For example, '123' has four suffixes $\wedge, 3, 23, 123$.

1.4

RELATIONS

1.4.1 Definition of a Relation

A relation from A to B is the subset of $A \times B$. Suppose R is the relation from A to B , then R is a set of ordered pair where each first element comes from A and each second element comes from B . That is, for each pair $a \in A$ and $b \in B$, exactly one of the following is true:

- (i) $(a, b) \in R$ (i.e., a is R -related to b , written as aRb)
- (ii) $(a, b) \notin R$ (i.e., a is not R -related to b)

The domain of a relation R is the set of all first elements of the ordered pair which belong to R , and the range of R is the set of second elements. Let us consider two sets N_1 and N_2 of natural numbers. If we define R as relation between them such that N_1 is cube of N_2 . then $1R1, 2R8, 3R27, 4R64, \dots$

In terms of ordered pair, these relations can also be written as

$$\begin{aligned} R &= \{(1, 1), (2, 8), (3, 27), (4, 64), \dots\} \\ &= \{(a, b) \mid a, b \in N_1 \text{ or } N_2, \text{ and } b = a^3\} \end{aligned}$$

Thus, we see that if R is the relation from A to B , then $R \subseteq A \times B$. In particular, if any subset $A \times A$ defining a relation in A is called a **binary relation**. Below are some more examples of relation

- If relation R is from set A to set B defined as $A - B = 2$, if set $A = \{1, 2, 3\}$, and $B = \{0, 1, 2\}$, then

$$R = \{(2, 0), (3, 1)\}.$$

Here domain of R is $\{2, 3\}$ and range is $\{0, 1\}$.

- If $A = \{1, 2, 3, 4, 5\}$, and $B = \{6, 7, 8, 9, 10\}$, and suppose $R = \{(1, 7), (2, 8), (5, 9), (5, 10)\}$ is a relation from A to B , then domain of $R = \{1, 2, 5\}$ and range of $R = \{7, 8, 9, 10\}$
- If $A = \{1, 2, 3\}$, and $B = \{a, b, c\}$ and $R = \{(1, b), (2, c), (1, a), (3, a)\}$ is subset of $A \times B$, then R is a relation from A to B . Therefore,

$$\{(1, b), (2, c), (1, a), (3, a)\} \in R.$$

can also be written as $1Rb, 2Rc, 1Ra, 3Ra$. But $(2, b) \notin R$, therefore 2 is not related to b .

❖ Good to Know!

If A and B are two nonempty finite sets consisting of m and n elements respectively, then $A \times B$ consists of total $m.n$ ordered pairs. Therefore total number of subsets in $A \times B$ are $2^{m.n}$

1.4.2 Inverse of a Relation

If R is a relation from a set A to set B , then the inverse of R , denoted by R^{-1} , is the relation from B to A which consists of those ordered pairs which, when reversed belong to R . for example

$$R^{-1} = \{(b, a) \mid (a, b) \in R\}.$$

For example, let $A = \{a, b, c, d\}$, and $B = \{1, 2, 3, 4\}$, let $R = \{(a, 1), (a, 3), (b, 3), (c, 4), (d, 1)\}$ be a relation from A to B then

$$R^{-1} = \{(1, a), (3, a), (3, b), (4, c), (1, d)\}$$

1.4.3 Types of Relations

Reflexive Relation A relation R on a set A is reflexive if aRa for every $a \in A$, i.e., if $(a, a) \in R$ for every $a \in A$. Therefore, R is not reflexive if there exists $a \in A$ such that $(a, a) \notin R$. For example, aRa for all $a \in A$, is a reflexive relation. As another example, the relation " $\leq$ " (i.e., less than or equal to) in the set of natural number, is a reflexive relation.

Symmetric and Anti-symmetric Relations A relation R on a set A is said to be symmetric if whenever aRb then bRa , i.e., if whenever $(a, b) \in R$, then $(b, a) \in R$. R is not said to be symmetric if there exists $(a, b) \in A$ such that $(a, b) \in R$ but $(b, a) \notin R$. Following are some examples of symmetric relations:

- (i) Let l_1 and l_2 be two lines parallel to each other. It means whenever l_1 is parallel to l_2 then l_2 is parallel to l_1 . Therefore the relation "parallel" is said to be symmetric.
- (ii) Let $A = \{1, 2, 3, 4\}$, and relations R_1 and R_2 are defined as

$$R_1 = \{(1, 3), (3, 1), (2, 4), (4, 2)\}$$

and

$$R_2 = \{(2, 3), (3, 2), (1, 4), (4, 1)\}$$

Then relations R_1 and R_2 are symmetric relations.

Transitive Relation A relation R on a set A is said to be transitive if whenever aRb and bRc , then aRc , it means, if whenever

$$(a, b), (b, c) \in R$$

then

$$(a, c) \in R$$

R will not be said transitive if there exists $a, b, c \in A$ such that $(a, b), (b, c) \in R$ but $(a, c) \notin R$. As an example, if l_1, l_2 , and l_3 are three lines such that l_1 is parallel to l_2 and l_2 is parallel to l_3 , then l_1 is parallel to l_3 . Therefore the relation “is parallel to” is a transitive relation.

Equivalence Relation A binary relation R on a set A will be called an equivalence relation if and only if

- (1) R is reflexive relation
- (2) R is symmetric relation, and
- (3) R is transitive relation.

The following are some examples of equivalence relations:

- The equality relation ($=$) on a set of numbers such as $\{1, 2, 3\}$ is an equivalence relation.
- The congruent modulo m relation on the set of integers i.e., $\{(a, b) | a \equiv b \pmod{m}\}$, where m is a positive integer greater than 1, is an equivalence relation.

Equivalence Class For an equivalence relation R on a set A , the set of the elements of A that are related to an element, say a , of A is called the equivalence class of element a and it is denoted by $[a]$.

The set of equivalence classes of an equivalence relation on a set A is a partition of A . Conversely, a partition of a set A determines an equivalence relation on A . Let $\{A_1, \dots, A_n\}$ be a partition of a set A . We can define a binary relation R on A as follows: $(a, b) \in R$ if and only if $a \in A_i$ and $b \in A_i$ for some i , $1 \leq i \leq n$. Then R is an equivalence relation. Let R_1 and R_2 be equivalence relations, Then $R_1 \cap R_2$ is an equivalence relation, but $R_1 \cup R_2$ is not necessarily an equivalence relation.

1.4.4 Closure of a Relation

Reflexive and Symmetric Closure If R is a relation on a set A then

- (i) $R \cup \Delta_A$ is reflexive closure of R , where $\Delta_A = \{(a, a) | a \in A\}$.
- (ii) $R \cup R^{-1}$ is the symmetric closure of R where R^{-1} is the inverse of R . For example, if R is a relation defined as

$$R = \{(a, a), (a, b), (b, a), (c, b)\}$$

on set $A = \{a, b, c\}$, then we can find reflexive closure of as:

By inspection we see that

$$(b, b), (c, c) \notin R$$

Therefore, $S = R \cup \Delta_A$ is a reflexive closure of R , where $\Delta_A = \{(b, b), (c, c)\}$.

To obtain the reflexive closure of a relation R , we just add the diagonal relation elements to R (i.e., $\{(a, a) | a \in A\}$). To obtain the symmetric closure of a relation R , we just add its inverse to it. For example, let a relation R on set A be given as

$$R = \{(a, a), (a, b), (b, b), (c, b), (c, c)\}$$

and

$$A = \{a, b, c\}$$

by inspection we see that

$$(b, a), (b, c) \notin R$$

Therefore, $S = R \cup R^{-1}$ is a symmetric closure of R , where R^{-1} is inverse of R . Hence $S = \{(a, a), (a, b), (b, a), (b, b), (c, b), (b, c), (c, c)\}$.

Transitive Closure If R is a relation on set A , then the connectivity relation R^* consists of the pairs (a, b) such that there is a path between a and b in relation R , or we can say that

$$R^* = \bigcup_{n=1}^{\infty} R^n.$$

The transitive closure of a relation R is equal to the connectivity relation R^* . We are going to prove two things:

(i) R^* is transitive relation and

(ii) S is a transitive relation on set A , with $R \subseteq S$, then $R^* \subseteq S$.

First we show that R^* is transitive. If $(a, b) \in R^*$ and $(b, c) \in R^*$, then there are paths from a to b and b to c . Therefore, there will be a path from a to c by starting with a to b and following it with path b to c . Hence, $(a, c) \in R^*$.

Therefore, R^* is a transitive relation. Now we show that S is a transitive relation on set A with $A \subseteq S$. As we have $R \subseteq S$, then $R^* \subseteq S$ by following definition of R^*

$$R^* = \bigcup_{i=1}^{\infty} R^i.$$

Therefore R^* contains R . Since S is transitive, then S^n is also transitive, also $S^n \subseteq S$, since $S^* = \bigcup_{i=1}^{\infty} S^i$ and $S^i \subseteq S$. Also, any path in relation R is also a path in S , because $R^* \subseteq S^*$ if $R \subseteq S$. Now we have $R^* \subseteq S^*$, and $S^* \subseteq S$. Hence, any transitive relation that contains R must also contain R^* . Therefore R^* is transitive closure of R .

Partial Ordering A relation R on a set A is said to be in partial order relation if R is

- (i) a reflexive relation
- (ii) an anti-symmetric relation, and
- (iii) a transitive relation.

Lexicographic Order The Lexicographic Ordering of S on $A_1 \times A_2$ is defined by the dictionary ordering of pairs i.e., which pair should be placed first, which pair at second place and so on. For example, if there is a relation R on set A defined as

$$R = \{(a, a), (a, b), (b, a), (b, b)\}.$$

The relation R on set $A = \{a, b\}$ is in *Lexicographic Ordering*, since the listing of pairs is alphabetical. If we wish to represent the set of all strings made from any possible combinations from set (a, b) , we generally use to write strings as

$$\{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots, bbb, aaaa, \dots, bbbb, \dots\}$$

In the above set, the strings are written in ascending order of length alphabetically. Always we try to write all strings up to specific length to reflect the nature of strings. In contrast to this, if we write the strings in a set as $\{a, abba, bba, abbbba, \dots\}$, we will not be able to draw any conclusion about what kind of strings this set has.

1.5

FUNCTIONS

A function, denoted by f , from a set A to a set B is a relation from A to B that satisfies :

- (i) for each element a in A , there is an element b in B such that (a, b) is in the relation, and
- (ii) if (a, b) and (a, c) are in the relation, then $b = c$.

The set A in the above definition is called the domain of the function and B its co-domain. This way, f is a function if it covers the domain and it is single valued. The relation given by f between a and b represented by the ordered pair (a, b) is denoted as $f(a) = b$, and b is called the image of a under f . The set of such images of the elements of a set S under a function f is known as the image of the set S under f , and is represented by $f(S)$, that is, $f(S) = \{f(a) \mid a \in S\}$, where S is a subset of the domain A of f .

The image of the domain under f is called the range of f . As an example, let f be the function from the set of natural numbers N to N that maps each natural number x to x^2 . Then the domain and co-domain of function f are N . For example, the image of 4 under this function is 16, and its range is the set of squares, i.e., $\{0, 1, 4, 9, 16, 25, \dots\}$.

1.5.1 Types of Functions

Onto Function A function f from a set A to a set B is said to be onto (surjective), if and only if for every element y of B , there is an element x in A such that $f(x) = y$, that is, f is onto if and only if $f(A) = B$. In other words, a function $f: A \rightarrow B$ is said to be an onto function if every element of B is mapped by at least one element of A .

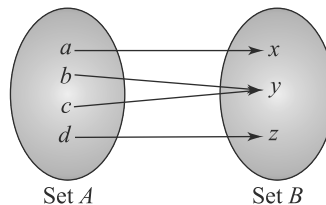


Fig. 1.2 Onto mapping

One-to-One Function A function $f: A \rightarrow B$ is said to be one-to-one (injective). In these types of functions all distinct elements of set A map to all distinct elements of set B , and no element of set B is without mapping.

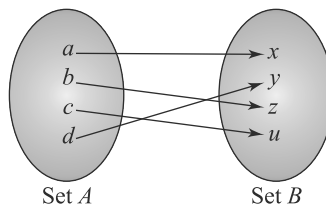


Fig. 1.3 One-to-One mapping

Into Function A function $f: A \rightarrow B$ is said to be into function if there is at least one element of set B which is not mapped by any element of set A . The following mapping shows into function.

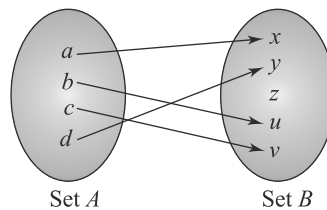


Fig. 1.4 Into mapping

One-to-One into Function A function $f: A \rightarrow B$ is said to be one-to-one into function if it is both one-to-one and into function. The following mapping shows one-to-one into function.

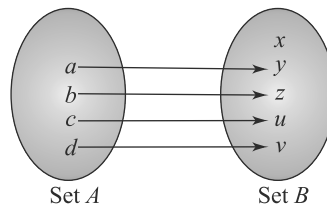


Fig. 1.5 One-to-one into mapping

Many-to-One Function A function $f: A \rightarrow B$ is said to be many-to-one function at least one element of co-domain B is mapped by two or more elements of domain A . The following mapping shows one-to-one into function.

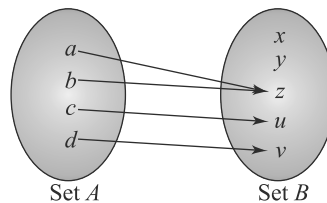


Fig. 1.6 Many-to-one mapping

Many-to-One into Function A function $f: A \rightarrow B$ is said to be many-to-one into function if it is both many-to-one and into function. In this type of mapping two or more elements of set A map to some elements of set B and some element of set B are not mapped by any element of set A . Following mapping shows many-to-one into function.

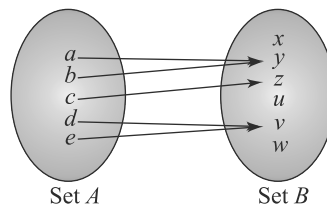


Fig. 1.7 Many-to-one into mapping

Many-to-One onto Function A function $f: A \rightarrow B$ is said to be many-to-one onto function if it is both many-to-one and onto function. In this type of function one element of set B is mapped by at least one element of set A , and two or more elements of set A map to some elements of set B . Following mapping shows many-to-one onto function.

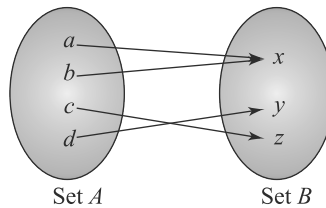


Fig. 1.8 Many-to-one onto mapping

1.5.2 Some Other Functions

Even Function A function $f: A \rightarrow B$ is said to be an even function if and only if

$$f(-x) = f(x) \quad \forall x \in A$$

for example,

$$\text{if } f(x) = x^2 + 1$$

$$\begin{aligned} f(-x) &= (-x)^2 + 1 \\ &= x^2 + 1 \\ &= f(x) \end{aligned}$$

Odd Function A function $f: A \rightarrow B$ is said to be an odd function if and only if

$$f(-x) = -f(x) \quad \forall x \in A$$

for example,

$$\text{if } f(x) = -x$$

$$\begin{aligned} f(-x) &= -(-x) \\ &= -(f(x)) \\ &= -f(x) \end{aligned}$$

Constant Function A function $f: A \rightarrow B$ is said to be a constant function if every element of set A maps to a single element of set B . A constant function is always a straight line parallel to any of the axis. The following figure shows mapping of such function.

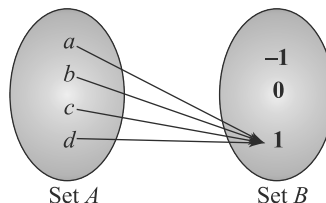


Fig. 1.9 Mapping of constant function $f(x) = 1$

Inverse Function A function $f: A \rightarrow B$ with another function $g: B \rightarrow A$ is said to be inverse function if the function $g: B \rightarrow A$ associates each element $b \in B$ to a unique element $a \in A$ such that $f(a) = b$ is called the inverse of $f: A \rightarrow B$.

Modulus Function A function $f: A \rightarrow B$ is said to be an modulus function if

$$f(x) = |x| \text{ for all } x \in A$$

i.e., the function $f(x)$ can also be defined as

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ x & \text{if } x < 0 \\ 0 & \text{if } x = 0 \end{cases}$$

Thus the modulus function can also be viewed as $f(x) = f(-x)$, therefore a modulus function is always an even function.

1.6

GRAPHS AND TREES

Graph History A graph is a very simple structure consisting of set of vertices and a family of lines, called edges (undirected) or arcs (directed), each of them linking some pair of vertices (or say nodes).

The famous problem “bridges of konigsberg” was solved by Euler. This problem is considered as the first formal result in graph theory. This theory was developed by Hamilton alongwith Heawood, Kempe, Kirchhoff, Petersen, and Tait during the second half of nineteenth century. Graph theory has boomed since the 1930s with efforts of Hall, Kuratowski, Konig, Erdos, Seymour and many other people. Graph theory is clearly related with combinatorics, algebra, topology etc.

The main application of the graph theory in computer science includes operations research decision theory, game theory and others.

Graphs A graph is a finite set of nodes (or say vertices) connected by links called edges. Symbolically, a graph G is defined as triple $G = (V, E, \theta_G)$

Where

V = nonempty set of vertices

E = set of edges

θ_G = a function that assigns to each edge, a subset (u, v) of V , u and v need not to be distinct. If e is an edge and u, v are vertices such that

$$\theta_G(e) = uv$$

Then e is an edge between u and v , and the vertices u and v are the endpoints of the edge e .

For example, if $G = (V, E, \theta_G)$ is graph where, V, E, θ_G are defined as

$$V = \{v_1, v_2, v_3, v_4\}$$

$$E = \{e_1, e_2, e_3, e_4, e_5\}$$

$$\theta_G(e_1) = \{v_1, v_2\}$$

$$\theta_G(e_2) = \{v_2, v_2\}$$

$$\theta_G(e_3) = \{v_2, v_3\}$$

$$\theta_G(e_4) = \{v_1, v_3\}$$

$$\theta_G(e_5) = \{v_3, v_4\}$$

The pictorial representation of graph G is given in Fig. 1.10.

In above diagram the edge e_2 joins the vertex v_2 to itself; such an edge is known as self loop or simply a loop.

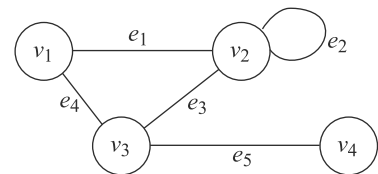


Fig. 1.10 Pictorial representation of graph

1.6.1 Types of Graphs

Connected Graph A graph G is said to be a connected graph if there exists a path connecting every pair of vertices. That means any vertex of the graph is reachable from any other vertex. A non-connected graph can be divided into connected components called a disjoint connected sub-graph. For example, the following graph made of three connected components is not a connected graph

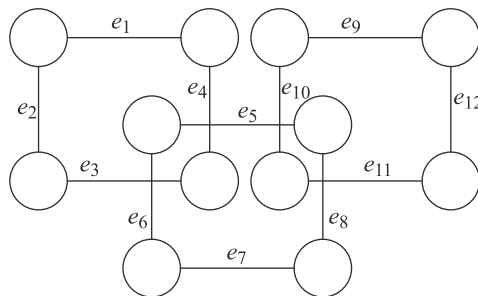


Fig. 1.11 Three connected components

With the help of at least two edges and three connected components the above figure can constitute a connected graph. One of the possible connected graphs is given below.

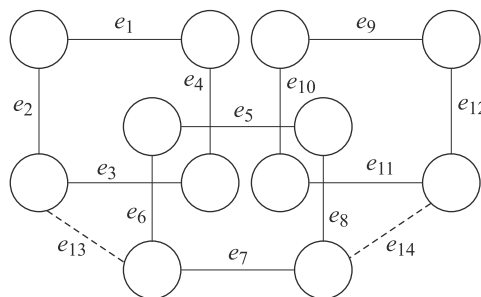


Fig. 1.12 A connected graph

The dotted lines in the above figure show the connectivity of the connected components to constitute a connected graph.

A vertex is said to be an isolated vertex if its degree is zero. The degree (also called valence) of a vertex is the number of edge ends at the vertex. For example, in the graph given below each vertex has degree 3. A vertex with degree 1 is called a pendant vertex.

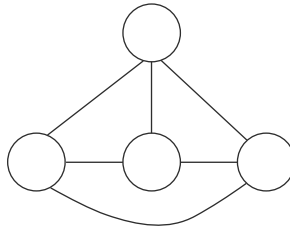


Fig. 1.13 A graph of degree 3

The in-degree of a vertex v is the number of edges from which the terminal vertices are connected if and only if the vertex v is the initial vertex. The out-degree of a vertex v is the number of edges with v as their initial vertex. The self loop at a vertex contributes both in-degree and out-degree as one of its vertex.

Directed Graph A graph G is said to be a directed graph (also called digraph) if the edges are directed. The edges in the directed graph are represented by ordered pairs like (v_1, v_2) to indicate that the edge is directed from v_1 to v_2 . The direction is shown by an arrow. For example, the following is the directed graph

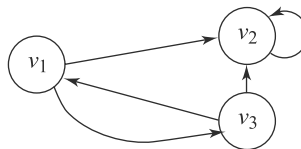


Fig. 1.14 A directed graph

The directed edges of above graph are (v_1, v_2) , (v_2, v_1) , (v_1, v_3) , (v_3, v_2) . The vertex u in an ordered pair (u, v) is called the initial vertex and v is called the terminal vertex.

Transition diagram (or say transition graph) are directed graphs with three different types of vertices i.e., initial state, final state and intermediates. The out degree of a vertex v is the number of edges with v as their initial vertex. Let us consider following directed graph:

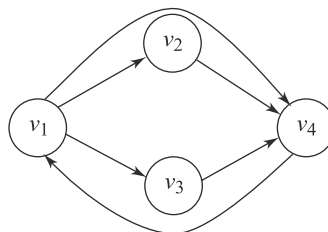


Fig. 1.15 Another directed graph

The in-degree of vertices v_1, v_2, v_3, v_4 are 1, 1, 2, 3 respectively and the out degree of vertices v_1, v_2, v_3, v_4 are 3, 2, 1, 1 respectively.

Multi-Graph A graph is said to be multi-graph if there are multiple edges between the same vertices. The multiple edges are also called parallel edges. A path in a graph is a sequence of consecutive edges and the length of the path is the number of edges traversed. By default it is assumed that each edge between two vertices has length 1. Consider the following graph

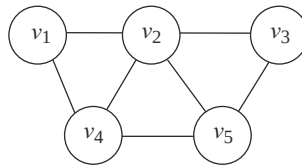


Fig. 1.16 Multi-graph

If we traverse vertex v_1 then goto v_4 , then v_2 then v_3 , the length of path is 3.

Pseudo Graph A graph G is said to be a pseudo graph if it has self loops and is parallel edged. Therefore, every simple graph and every multi graph is a pseudo graph, but the converse is not. For example, the following graph G is a pseudo graph, but neither a simple graph nor a multi graph.

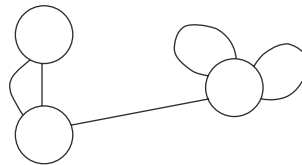


Fig. 1.17 Pseudo graph

Complete Graph A simple graph is said to be a complete graph if its each pair of distinct vertices are joined by an edge. A complete graph with n vertices is denoted by K_n . The following are some examples of complete graphs:

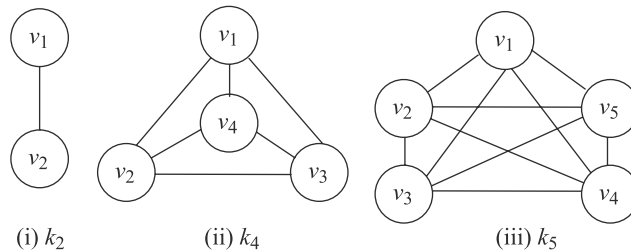


Fig. 1.18 Complete graphs

K-Regular Graph A graph G is said to be k -regular (or regular of degree k) if every vertex of G has degree k . For example, following graph is a 3-regular graph.

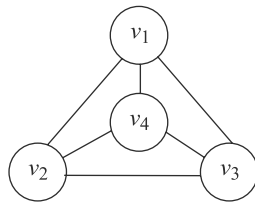


Fig. 1.19 3-regular graph

Bipartite Graph A simple graph G is said to be a bipartite graph if its vertices set V is partitioned into two disjoint nonempty sets V_1 and V_2 such that every edge in the graph connects a vertex in V_1 and a vertex in V_2 . Also, there is no edge that connects either two vertices in V_1 or two vertices in V_2 . For example, the following graph G is a bipartite graph:

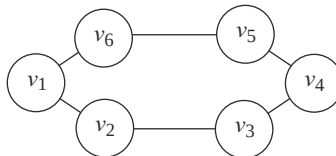


Fig. 1.20 Bipartite graph

The vertices of G are $V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$. The set V set can be partitioned into V_1 and V_2 as $V_1 = \{v_1, v_3, v_5\}$, $V_2 = \{v_2, v_4, v_6\}$. Also, every edge in graph G connects a vertex in V_1 and a vertex in V_2 .

Complete Bipartite Graph A graph G is said to be a complete bipartite graph $K_{m,n}$ if its vertices are partitioned into two nonempty subsets of m and n vertices, respectively. If there is an edge between two vertices in a complete bipartite graph $K_{m,n}$ then its one vertex is in the first subset and the other vertex is in the second subset. The following is a complete bipartite graph $K_{2,3}$

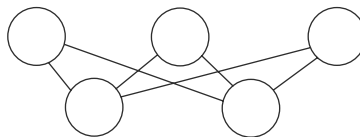


Fig. 1.21 A complete Bipartite graph

Tree, or Tree-Graph A graph (directed or undirected) is said to be a tree or tree-graph if it is a connected graph without circuits (or say without loops). In a tree-graph, there is one and only one path between every pair of vertices. Also, a tree graph with n vertices has exactly $n-1$ edges. Following are some examples of tree graphs:

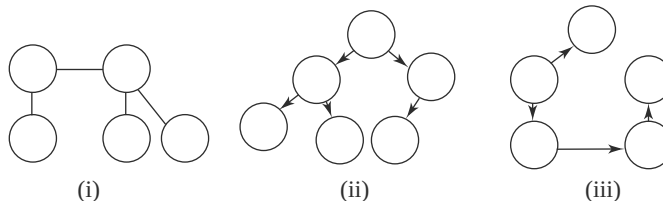


Fig. 1.22 Tree or tree-graph

Planar Graph A graph G is said to be planar graph if it can be drawn on a plane so that the edges intersect only at vertices. For example, following are some planar graphs:

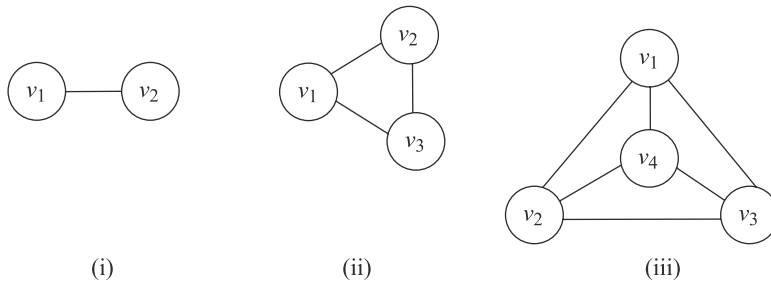


Fig. 1.23 Planar graphs

1.6.2 Operations on Graphs

Complement of a Graph The Complement of a Graph (denoted by G') is the graph with only those edges which are not present in graph G . Therefore, uv is an edge of graph G' if and only if uv is not an edge of graph G . For example, below is a graph and its complemented graph:

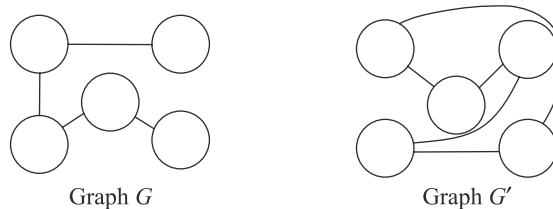


Fig. 1.24 A graph with its complement

Union and Intersection of Two Simple Graphs

If $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are two simple graphs, then a graph G_u obtained by union of G_1 and G_2 is defined as

$$G_u = G_1 \cup G_2 = \{(V_1 \cup V_2), (E_1 \cup E_2)\}.$$

The intersection of graphs G_1 and G_2 is defined as

$$G_i = G_1 \cap G_2 = \{(V_1 \cap V_2), (E_1 \cap E_2)\}.$$

The following figures show two simple graphs G_1 and G_2 , and their union and intersection:

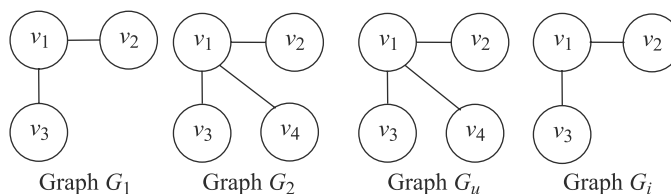


Fig. 1.25 Union and Intersection of two simple graphs

Weighted Graph A graph G is said to be a weighted graph if every edge of graph G is assigned with a real number. For example, the following graph is a weighted graph:

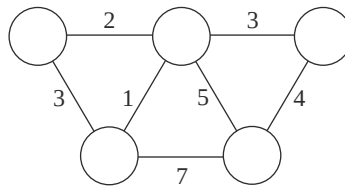


Fig. 1.26 A weighted graph

Acyclic Graph A directed graph G which has no cycle is called an acyclic graph.

Cyclic Graph A directed graph G which has one or more cycles is called a cyclic graph. In other words, if there is at least one vertex in graph G such that there is a path from this vertex to itself. For example, the following are acyclic and cyclic graphs:

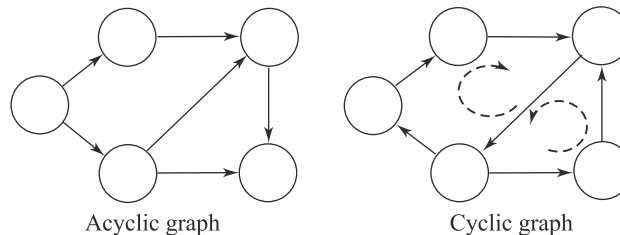


Fig. 1.27 Acyclic and cyclic graphs

1.7

PROOF TECHNIQUES

In mathematics, the proofs employ logic but they usually include some amount of natural language which indeed admits some ambiguity. In the context of proof theory the formal proofs are considered entirely formal demonstrations called social proofs. This characteristic of proofs has led to much examination of modern and historical mathematical practice. The viewpoint of mathematics is directly concerned with the role of formal language and logic in proofs, and mathematics as language as well.

Apart from one's attitude to formalism, the result that is proven to be true is a theorem completely in a formal proof. The complete proof represents that how it follows from the axiom alone. Once a theorem is proved it can be applied as basis step to prove further statements. There is a special class of statements that don't need to prove they are foundations of mathematics. Such statements are supposed to be true by default. These are true since the primary study of philosophy of mathematics. Today focus is more than this. The modern approach is based on practical, that is, acceptable technique. Some common proof techniques are divided into two parts: formal proofs and inductive proofs.

1.7.1 Formal Proofs

Direct Proof Where the conclusion is established by logically combining the axioms, definitions and earlier theorems. In this proof technique, we prove implication statements that contain two parts: An "if-part" which is called the Premise and another part called "then-part" known as conclusion.

Proof by Contradiction This kind of proof can be applied to all types of statements. Where it is shown that if some property were true, a logical contradiction occurs, hence the property must be false.

Proof by Contra Positive Contra positive proof technique contains contra positive of the statements, which is of the form “if A then B ” concludes “if not A then not B ”.

Proof by Exhaustion Where the conclusion is established by dividing it into a finite number of cases and proving each one separately.

Proof by Deduction A deductive proof consists of a sequence of statements whose truth leads some initial statements, called hypothesis or the given statements, to a concluding statement.

1.7.2 Inductive Proof

Proof by Mathematical Induction Where a base case is proved, and an induction rule is used to prove a (often in finite) series of other cases.

A probabilistic proof should mean a proof in which an example is shown to exist by methods of probability theory—not an argument that a theorem is “probably true”. The latter type of reasoning can be called a “possibility argument”. In the case of collatz conjecture it is clear how far that is from a genuine proof. Probabilistic proof is one of many ways to show existence theorem, rather than proof by construction.

If we are trying to prove, for example, “some X satisfies $f(X)$ ”, an existence or non-constructive proof will prove that there is a X that satisfies $f(X)$, but does not tell how such X will be obtained. A constructive proof conversely will do so.

A **conjecture** may be defined as a statement which is considered to be true but has not been proven yet. Sometimes it is possible and desirable to prove that a certain statement cannot possibly be proved from a given set of axioms. In most axiom systems, there are statements which neither is proven nor disproved.

Mathematical Induction Mathematical induction is a technique to assertion like

- (i) $n < 2^n$ for all positive integers n
- (ii) the sum of the first n odd integers is n^2

$$(iii) \sum_{m=0}^n 2^m = 2^{n+1} - 1.$$

There are several other assertions like the above that can be proved very easily by using techniques called Mathematical induction. This technique is extensively used to prove results of discrete objects.

A proof by using Mathematical induction that $P(n)$ is true for each positive integer n having the two following steps:

(i) **Basis step** The statement $P(n)$ is shown to be true for $n = 1, 2$ by putting $n = 1, n = 2$.

(ii) **Induction step** The statement $P(n)$ is assumed to be true for $n = m$. Now the statement $P(m+1)$ is shown to be true for every positive integer m . Here the statement $P(m)$ for a fixed positive integer m is called induction hypothesis. Now we prove the following assertions by Mathematical induction steps:

$$(i) \quad 1 + 2 + 3 + \dots + n = \frac{1}{2}n(n+1)$$

(ii) A finite set A with n elements, then power set of A has 2^n elements.

Proof (i) Suppose $P(n)$ is the statement that the sum of first n natural numbers is $\frac{1}{2}n(n+1)$.

Basis step $P(1)$ is true, since the first number is 1 and $\frac{1}{2}n(n+1)$ is 1 for $n = 1$.

Induction step Suppose $P(m)$ is true for $n = m$. Now we have

$$P(m): 1 + 2 + 3 + \dots + m = \frac{1}{2}m(m+1)$$

To show that $P(m+1)$ is also true, we add $(m+1)$ to both side of equation given by $P(m)$ to get

$$\begin{aligned} P(m): 1 + 2 + 3 + \dots + m + (m+1) &= \frac{1}{2}m(m+1) + (m+1) \\ &= \frac{1}{2}(m+1)(m+2) \end{aligned}$$

Hence, $P(n)$ is true for $n = m+1$, therefore $P(n)$ is true for all values of n .

Proof (ii) Let $P(n)$ be the statement that if a set A has n elements, then its power set has 2^n elements.

Basis step $P(0)$ is true, since power-set of a set with zero elements has only one element, (i.e., 2^0).

Induction step Suppose $P(m)$ is true for $n = m$. Now we have to show that

$P(m+1)$ is also true, i.e., $P(n)$ for $n = m+1$.

Any set with $(m+1)$ elements should have 2^{m+1} subsets. Let B be a set with $m+1$ elements. Now we can write

$$B = A \cup (X)$$

where X is one of the elements of set B , and $A = B - (X)$.

The subsets of set B can be obtained as :

For every subset A_1 of set A , there are exactly two subsets of B :

- (i) A_1
- (ii) $A_1 \cup (X)$

These subsets constitute all the subsets of set B . since there are 2^m subsets of A_1 , therefore there are $2 \cdot 2^m$ (i.e., 2^{m+1}) subsets of B . Hence $P(n)$ is true for $n = m+1$.

Summary

- **Set** A set is a collection of element without any structure or order.
- **Cardinality** The number of distinct elements in a set is called the cardinality of the set.
- **Empty Sets** An empty set is defined as a set with no elements.

- **Operations on Sets** Union, intersection, difference, complement.
- **Alphabets** An alphabet can be defined as a finite set of symbols.
- **Relation** A relation from A to B is the subset of $A \times B$. Suppose R is the relation from A to B , then R is a set of ordered pair where each first element comes from A and each second element comes from B .
- **Inverse of a Relation** If R is a relation from a set A to set B , then the inverse of R , denoted by R^{-1} , is the relation from B to A which consists of those ordered pairs which, when reversed belong to R .
- **Equivalence Relation** A binary relation R on a set A is an equivalence relation if and only if R is a reflexive, symmetric, and transitive relation.
- **Equivalence Class** For an equivalence relation R on a set A , the set of the elements of A that are related to an element, say a , of A is called the equivalence class of element a .
- **Lexicographic Order** The Lexicographic Ordering of S on $A_1 \times A_2$ is defined by the dictionary ordering of pairs i.e., which pair should be placed first, which pair at second place and so on.
- **Graph** A graph is a finite set of nodes (or say vertices) connected by links called edges.
- **Connected Graph** A graph G is said to be connected if there is a path connecting every pair of vertices.
- **Multi-graph** A graph is said to be multi graph if there are multiple edges between the same vertices.
- **Pseudo Graph** A graph G is said to be a pseudo graph if it has self loops and parallel edged.
- **Complete Graph** A simple graph is said to be a complete graph if its each pair of distinct vertices are joined by an edge.
- **Tree or Tree-Graph** A graph (directed or undirected) is said to be a tree or tree-graph if it is a connected graph without circuits (or say without loops).
- **Planar Graph** A graph G is said to be planar graph if it can be drawn on a plane so that the edges intersect only at vertices.
- **Inductive Proof** where a base case is proved, and an induction rule is used to prove a (often in finite) series of other cases.

Review Questions

- 1.1 What is a set? Explain how a set can be partitioned?
- 1.2 Prove that a set is subset of itself.
- 1.3 What are DeMorgan's laws? Illustrate with the help of Venn diagrams.
- 1.4 Illustrate DeMorgan's laws without using Venn diagram.
- 1.5 What are various operations on strings?
- 1.6 Give the language of all palindromes over (a, b) starting and ending with b .
- 1.7 Draw the graphical representation of the relation "les than" on $\{1, 2, 3, 4\}$.
- 1.8 Find the reflexive, symmetric, and transitive closure of the relation $\{(a, a), (b, b), (c, c), (a, c), (a, d), (b, d), (c, a), (d, a)\}$ on the set $S = \{a, b, c, d\}$
- 1.9 Let $A = \{1, 2, 3, 5, 10\}$, $B = \{2, 4, 7, 8, 9\}$, $C = \{5, 8, 10\}$ be subsets of $S = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$, find (i) $A \cup B$, (ii) $A - C$, (iii) $B' \cap (A \cup C)$
- 1.10 Let $S = \{2, 4, 6, 8\}$ and $R = \{k, l, m, n\}$, find the number of functions from S to R .
- 1.11 Find a recursive and non-recursive definition for the following sequence 2, 5, 8, 11, 14, 17,
- 1.12 What is a relation? Give an example of an equivalence relation.

- 1.13 Give the language of all strings over $(0, 1)$ up to length 4 in lexicographic order.
- 1.14 Give the applications of inverse and modulus functions.
- 1.15 Give the difference between one-to-one and many-to-one functions.
- 1.16 Give the applications of graphs.
- 1.17 Explain the condition when a graph is also a tree.
- 1.18 Define, (i) Weighted graph (ii) Cyclic graph (iii) Complete graph.
- 1.19 With the help of mathematical induction prove that:

$$(i) 1 + 2 + 3 + \dots + n = \frac{n \times (n+1)}{2}, (ii) 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n \times (n+1)(2n+1)}{6}$$

- 1.20 Give the difference between proof by contradiction and proof by contra positive.

Graded Exercises

- * 1.1 Prove by contradiction that the product of odd integers is not even.
- * 1.2 Prove that for any positive integer n , $2^n > n$.
- ** 1.3 What is wrong with the following proof by mathematical induction?
We will prove that for any positive integer n , n is equal to 1 more than n . Assume that $P(k)$ is true.
 $k = k + 1$
Adding 1 to both side of this equation we get
 $k + 1 = k + 2$
thus, $P(k+1)$ is true.
- ** 1.4 Let f be a function, $f: P \rightarrow Q$, show that for all subsets A and B of P , $f(A \cap B) \subseteq f(A) \cap f(B)$.
- * 1.5 Determine whether the following function is onto or it is one-to-one:
 $f: R \rightarrow R, f(x) = 2x + 1$
- *** 1.6 Prove that the number of leaf nodes in a binary tree graph is $\frac{n+1}{2}$, where n is the number of vertices.
- * 1.7 Show that the function $f: P \rightarrow P$, give by $f(x) = 2x$ in one-to-one but not an onto.
- * 1.8 Show that the function $f: P \rightarrow P - \{1\}$, give by $f(x) = \frac{x+1}{x-1}$ in an onto function.
- * 1.9 Prove that if x is an odd integer then x^2 is also odd.
- * 1.10 Prove that the sum $(P + Q)$ of any two integers P and Q is odd if and only if exactly one of the integers P or Q is odd.

* Difficulty level 1

** Difficulty level 2

*** Difficulty level 3

Hints

- 1.9 Let $x = 2n + 1$, where $n \in N$

The square of both sides gives

$$\begin{aligned} x^2 &= (2n + 1)^2 \\ &= 4n^2 + 4n + 1 = 4(n^2 + n) + 1 \end{aligned}$$

The value $4(n^2 + n) + 1$ is an odd integer.

- 1.10 Assume P is odd and Q is even, then

$$\begin{aligned} P + Q &= 2n + 1 + 2m \quad \text{for } P = 2n + 1, \text{ and } Q = 2m \\ &= 2(m + n) + 1 \\ &= \text{odd} \end{aligned}$$

Objective Questions

- Which of the following statements is false?
 - The sets $A = \{\{a, b\}, \{aa, bb\}\}$ and $B = \{a, b, aa, bb\}$ are same.
 - $\{\{a, b\} \{aa, bb\}\} = \{a, b, aa, bb\}$
 - $\{\{a, b\} \{aa, bb\}\} = \{aaa, abb, baa, bbb\}$
 - (a) and (b)
- If U is the universal set then which of the following is false?
 - $U \cup \Phi = U$
 - $U \cap \Phi = \Phi$
 - $U - \Phi = U$
 - none of these
- If $A = \{1, 2, 3, 4\}$, and $B = \{a, b\}$, then
 - $A \cup B = \{a, b, 1, 2, 3, 4\}$
 - $B \cup A = \{1, 2, 3, 4, a, b\}$
 - $A, B \subseteq A \cup B$
 - all of these
- $A \cap (B \cup A)'$ is equal to
 - A'
 - B'
 - A
 - none of these
- If $A = \{a, b\}^*$, and $B = b^*$, then $A - B$ is
 - $(a, ab, ba)^*$
 - b^*
 - a^*
 - $(a, ab, ba) (a, ab, ba)^*$
- If $A = \{a, b\}^*$, and $B = (b, c)^*$, then $A \cap B$ is
 - a^*
 - b^*
 - c^*
 - none of these
- If N is the set of natural numbers then which of the following relation is one-to-many?
 - $\{(1, 2), (2, 3), (3, 4)\}$
 - $\{(2, 7), (8, 4), (2, 5), (7, 4)\}$
 - $\{(1, 3), (2, 3), (4, 5)\}$
 - none of these
- The function $f: R^+ \rightarrow R^+$ defined over $f(n) = n + x$, is
 - onto
 - bijective
 - one-to-one
 - none of these
- When the proof of the a statement is initiated by an assumption that the statement is false, the proof technique is called
 - contra positive
 - contradiction
 - direct proof
 - none of these
- Which of the following statements is false?
 - There exist no positive integers a and b such that $a^2 - b^2 = 10$.
 - If P divides Q and R as well separately then P divides $Q + R$.
 - The contra positive of "If A then not B " is "If not A then not B ".
 - None of these

References for Extra Reading

- Whitehead, A.N. and B. Russell, *Principia Mathematica I-III*
- Nagle, H. Troy, *An Introduction To Computer Logic*, PHI, 2000.

28 □ Theory of Automata, Languages and Computation

3. Judith L. Gersting, *Mathematical Structures for Computer Science: A Modern Treatment of Discrete Mathematics*, W. H. Freeman and Company, New York, 5th Edition, 2003.
4. Kneet H. Rosen, *Discrete Mathematics and Applications*, TMH, 1999.
5. Mandelson E., *Introduction to Mathematical Logic*, Chapman and Hall, 1997.
6. McDermott, D. and J. Doyle (1980), *Nonmonotonic logic Artificial Intelligence*, ACM 13(1980), 41-72.
7. Nilsson, N., *Logic and Artificial Intelligence*, Journal of Artificial Intelligence: 31-55, 1991.
8. Tremblay, Manohar, *Discrete Mathematical Structures with Application to computer Science*, McGraw Hill 1995.
9. McGee, V. *Tarski's Staggering Existential Assumptions*, Syntheses 142: 371–387, 2004. Kluwer Academic Publishers.

Online Sources

1. <http://csc.colstate.edu/bosworth/GraphTheory/ResearchNotes.htm>
2. <http://www.liafa.jussieu.fr/~jep/PDF/MPRI/MPRI.pdf>
3. www.wi-consortium.org/pdf/McCarthy-humanlevelAI.ps

2

Finite Automata

❖ Chapter Objective ❖

In this chapter we start our discussion with first automaton (finite automaton deterministic version), it is also called deterministic finite state machine (DFSM or DFA). In automata theory, we have four kinds of automata (machine model) namely Finite Automata, Push Down Automata (PDA), Linear Bounded Automata (LBA), and Turing Machine (TM). The PDA, LBA and Turing Machines are discussed in later chapters.

The DFA is defined by its transition function and then its working is explained by presenting its model. Its representation is given with the help of transition equation, transition graph (also called transition diagram or transition system), and state table (called transition table). On the basis of inputs, finite automata with and without epsilon (represented by ϵ or $\wedge$) transitions/moves are discussed. The elimination of epsilon transitions is discussed supported by equivalence of finite automata with and without epsilon transitions. The language of DFA is described by illustrating acceptability of strings by DFAs. We will construct optimised DFA by showing minimisation of DFA. Then we will discuss the nondeterministic version of finite automata (abbreviated as NDFA or simply NFA) by describing its transition function followed by converting an NFA into equivalent DFA.

We will see the finite automata with output in the form of Mealy and Moore machines. In this sequence we show the equivalence and differences between Mealy and Moore machines. Finally we discuss extended transition function for strings. We will see how it works with NFAs

□ Introduction □

A finite state automaton is used to recognise a formal language called regular language. A formal language can be defined as a set of strings accepted by a machine. A finite automaton consists of a finite number of states, therefore it is also called a finite state machine. Two major traditional applications in computer science of Finite automata or finite state automata have been found as: Modeling of finite state system and description of regular sets of finite words. Several new applications of finite state automata have emerged, for example optimisation of logic programs, and specification and verification of protocols in the last few years. These applications use finite state automata to describe regular sets of infinite words and trees.

When we do programming using some high level language, the validity of a statement is checked in two parts:

- (i) All individual strings should be according to the symbol table of compiler.
- (ii) The order or sequence of strings should be according to the syntax of that language.

The point (i) refers to finite automata and point (ii) is related to push down automata. The statements written in high level language are broken into primitive parts called tokens, and these tokens are checked by lexical analyser (the practical implementation of finite automata) whether they are correct or not. If there is any spelling mistake, the syntax error is generated by the debugger. In this chapter, we will see how finite automata help to check the validity of a string. For the validity of a statement, readers are requested to refer the CFG and PDA.

2.1

FINITE STATE MACHINES AND ITS MODEL

Basically, finite state machines are studied in automata theory, which is a subfield of theoretical computer science. A finite state machine (FSM) or finite state automata (automaton is singular of automata) is an abstract machine used in the study of computation and languages that has only a finite, constant amount of memory (states). It can be conceptualised as a directed graph. There are finite numbers of states, and each state has transition into next state. There is an input string that determines which transition is to be followed.

A finite automaton may operate on the language of finite words (the standard case), infinite words, or various types of trees, to name the most important cases.

An automaton is defined as a system where information and materials are transformed, transmitted and utilised for performing some processes without direct involvement of a human. Some popular examples are automatic printing machines and automatic machines for production of various products. The model of finite automaton (discrete) is shown in Fig. 2.1 below:

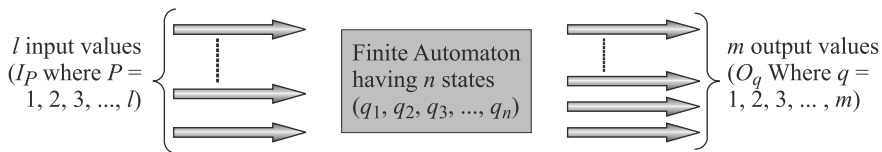


Fig. 2.1 Model of finite automaton

The characteristics of model of FA in Fig. 2.1 are:

- (i) **Input** $I_1, I_2, \dots, I_l$ are input values from the input alphabet at discrete instance of time.
- (ii) **Output** $O_1, O_2, \dots, O_m$ are different outputs of this model. Each output can take finite numbers of fixed values from the set of outputs O .
- (iii) **States** At any instance of time the finite automaton will be in any one of the states from $\{q_1, q_2, q_3, \dots, q_{n-1}, q_n\}$.
- (iv) **State relation** At any instance of time the next state of automaton is determined by the present input and present state.

(v) **Output relation** As output either only state, or both the output value and state are obtained. When an automaton reads an input symbol, it moves to the next state which is given by the state relation.

Different kinds of automata may be developed by applying some restrictions. Some of them are:

Automaton Without Memory An automaton in which only a state is obtained as output on scanning the input string.

Automaton with a Finite Memory An automaton in which the output is also obtained on input, for example, Mealy and Moore machines.

- ❖ **Moore machine** The automaton in which the output depends only on the present states of the machine.
- ❖ **Mealy machine** The automaton in which the output depends on the present state and the input at any instance of time.

2.2

DETERMINISTIC FINITE AUTOMATA

If not stated, a finite automaton is deterministic by default. In a deterministic finite automaton there is only one next state $q_0 \in Q$ on an input $a \in \Sigma$. The definition below is of deterministic finite automata. The maximum numbers of outgoing transitions from a particular state are number of different symbols from Σ .

❖ Definition

A finite automaton (deterministic) is defined as 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where,

$Q \rightarrow$ finite non-empty set of states i.e., there is at least one state

$\Sigma \rightarrow$ finite non-empty set of inputs called alphabet

$\delta \rightarrow$ transition function which maps $Q \times \Sigma$ into Q .

$q_0 \rightarrow$ initial state (i.e., there is only one initial state and $q_0 \in Q$)

$F \rightarrow$ Set of final states, and $F \subseteq Q$

❖ Good to Know!

1. The transition function describes the change of state during the transitions. The transition function $Q \times \Sigma$ into Q can also be written as $Q \times \Sigma \rightarrow Q$. The transition function $Q \times \Sigma \rightarrow Q$ describes that at a time a machine is in some state (element of Q), it will take some input (element of Σ), as a result it goes to some next state (element of Q).
2. The minimum number of final states in a finite automaton are zero; and as maximum, all states of finite automata may be final states.
3. In a transition function, Σ represents input symbol and Σ^* represents input string.

The mapping given by transition function is usually represented by a transition table or a transition diagram. The transition function which maps $Q \times \Sigma \rightarrow Q$ is called transition function working on a single symbol at a time. The mapping $Q \times \Sigma^* \rightarrow Q$ indicates that the finite automaton is in some state q ($q \in Q$) and by taking a string as input it goes to next state q' ($q' \in Q$).

The model of finite automaton can be represented graphically by Fig. 2.2.

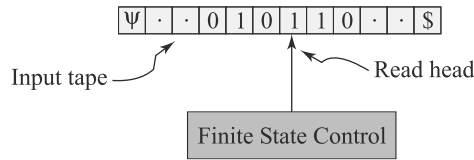


Fig. 2.2 Components of finite automaton

The main components of finite automata are explained as follows:

Input Tape The input tape is divided into blocks (or squares), each block contains a single symbol from the input alphabet $\Sigma = \{0, 1\}$. The end blocks of the input tape contain end-markers ψ at the left-end, and $\$$ at the right-end. The symbol ψ indicates the starting of input tape and the symbol $\$$ indicates the end of tape. The absence of these symbols indicates that the tape is of infinite length. The input string to be processed lies between these end marker symbols from left-to-right.

Reading Head The read scans only one block at a time and moves one block either left or the right. In case of general finite automata, we restrict the movement of read head only from left to the right side.

Finite Control The symbol under the read head is the input to the automaton. Suppose the scanned symbol is '0' and the present state of the machine is q ($q \in Q$), it performs two operations:

- (i) The read head moves to next block of input tape (the read head may remain at the same block when there is a null move).
- (ii) The machine goes to next state q' , where $q' \in Q$
The transition function δ also indicates this output.

$$\begin{array}{ccccc} Q & \times & \Sigma & \rightarrow & Q \\ \uparrow & & \uparrow & & \uparrow \\ q & & 0 & & q' \end{array} \quad \begin{array}{l} q = \text{present state} \\ 0 = \text{present input symbol} \\ q' = \text{next state} \end{array}$$

This operation can also be represented by

$$\delta(q, 0110) = \delta(q', 110)$$

The above transition shows that q is present state, the current input symbol is 0 (the leftmost symbol of string 0110). When read head scans symbol 0 then machine goes to $q' \in Q$ (the next state), and the remaining string is 110.

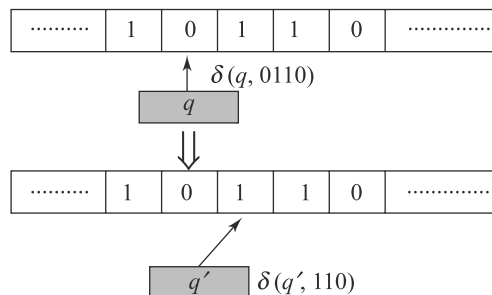


Fig. 2.3 Transition in finite automata

The machine starts in the start state (initial state) and reads in a string of symbols from its alphabet. It uses the transition function δ to determine the next state using the current state and the symbol just read. If, when it has finished reading, it is in some accepting state, it is said acceptance of the string, otherwise it is said rejection of the string. The set of strings it accepts to form a language is the language the FA recognises.

2.3

SIMPLIFIED NOTATIONS

The presentations by which a finite automaton can be represented are:

- Transition equations
- State Transition graph
- Transition table

2.3.1 Transition Equations

A finite automaton can be represented with the help of transition equations. For each transition there exists a transition equation. For example, consider a finite automaton M defined as

$M = (\{q_1, q_2, q_3\}, \{0, 1\}, \delta, q_1, \{q_3\})$ is a DFA, where δ is given by

$$\begin{aligned} \delta(q_1, 0) &= q_2, & \delta(q_1, 1) &= q_1, \\ \delta(q_2, 0) &= q_3, & \delta(q_2, 1) &= \phi, \\ \delta(q_3, 0) &= q_2, & \delta(q_3, 1) &= q_3, \end{aligned}$$

This finite automaton is DFA as it follows the transition function $Q \times \Sigma \rightarrow Q$. This represents that there are total three states in this DFA $\{q_1, q_2, q_3\}$ with q_1 as initial or start state and q_3 as final state or accepting state, the option of input are $\{0, 1\}$.

The transition equation $\delta(q_1, 0) = q_2$, shows that there is a transition from state q_1 to q_2 on input 0.

The transition equation $\delta(q_1, 1) = q_1$, shows that there is a transition from state q_1 to q_1 on input 1.

The transition equation $\delta(q_2, 0) = q_3$, shows that there is a transition from state q_2 to q_3 on input 0.

The transition equation $\delta(q_2, 1) = \phi$, shows that there exists no transition from state q_2 on input 1.

The transition equation $\delta(q_3, 0) = q_2$, shows that there is a transition from state q_3 to q_2 on input 0.

The transition equation $\delta(q_3, 1) = q_3$, shows that there is a transition from state q_3 to q_3 on input 1.

2.3.2 State Transition Graph

A transition diagram (also called transition graph or transition system) is a finite directed labelled graph in which each vertex represents a state and directed edges indicate the transition from one state to another. Edges are labelled with input or input/output.

In this representation the initial state is represented by a circle with an arrow towards it, the final state is represented by two concentric circles, and intermediate states (neither initial nor final) are represented by just a circle. Let us consider the following transition diagram (Fig. 2.4):

q_0 = initial state

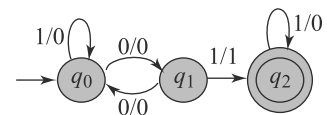


Fig. 2.4 A transition diagram

q_2 = final state

q_1 = intermediate state.

If the system is in state q_0 and the input 1 is applied, the system remains in state q_0 and the output is 0. If the system is in state q_1 and the input 0 is applied, the system goes to state q_0 and output is 0.

❖ Good to Know!

- (i) In some text books the initial state and final states are denoted by (–) and (+) respectively.
- (ii) In all transition diagrams an arrow without a label should be treated as an arrow with label $\wedge$ or ε , as you will see for initial state in all transition diagrams.

❖ Definition

A transition system is defined as 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where,

$Q \rightarrow$ finite non-empty set of states i.e., there is at least one state

$\Sigma \rightarrow$ set of input symbols called alphabet

$q_0 \rightarrow$ initial state (i.e., there is only one initial state and $q_0 \in Q$)

$F \rightarrow$ Set of final states, and $F \subseteq Q$

$\delta \rightarrow$ transition function, it is finite subset of $Q \times \Sigma^*$ into Q .

If (q, w, q') is in δ , it means the transition starts at vertex q , goes along a set of edges, and reaches the vertex q' . w is the string obtained by concatenation of the label of all the edges.

Example 2.1

Consider the following transition system:

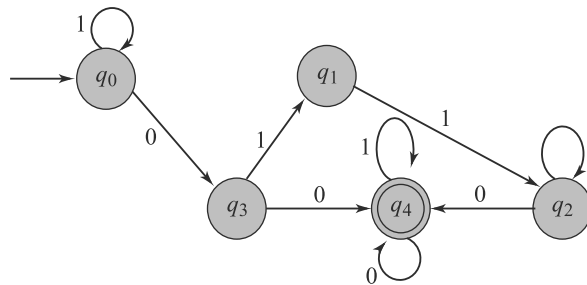


Fig. 2.5 The transition diagram of Example 2.1

Determine initial state, final states, and acceptability of '000101'.

Solution The initial state is q_0 . There is only one final state q_4 . The path value of state sequence $q_0 q_3 q_4 q_4 q_4 q_4 q_4$ is '000101' and q_4 is final state. So '000101' is accepted by the transition system (Fig. 2.5). the transitions may be represented as:

0 0 0 1 0 1
 $q_0 \rightarrow q_3 \rightarrow q_4 \rightarrow q_4 \rightarrow q_4 \rightarrow q_4 \rightarrow q_4$

Generalised Transition Graph

Generalised transition graphs are transition graphs where the labels of edges are regular expressions (the strings obtained by closure, concatenation, and union operation on input symbols). Thus an NDFA can be considered to be a generalised transition graph, which is just like an NDFA except that the edges may be labelled with arbitrary regular expressions. Since the labels on the edges of NDFA may be either $\wedge$ or elements of Σ , each of these can be considered to be a regular expression.

To find a generalised transition graph from a given transition graph, we adopt following steps:

Step 1 If the NDFA has more than one final states, convert NDFA to have only one final state.

Step 2 Remove states one by one from the NDFA, re-labelling edges as we go, until only the initial and final state remain.

Step 3 Read the final regular expression from the two-state (*i.e.* initial and final) automaton that results generalised transition graph.

The regular expression derived this way accepts the same language as the original NDFA. Since we can convert an NDFA into a regular expression, and a regular expression into an NDFA, therefore the both describe the same class of languages *i.e.*, the regular languages. Every transition graph is obviously a generalised transition graph, since a generalised transition graph is a generalisation of a transition graph.

We show the construction of generalised transition graph from an NDFA by applying above method (steps 1 to 3). This method can be applied to any NDFA. It contains following steps:

- (i) If necessary, get a generalised transition graph to have exactly one initial state having no incoming transitions:

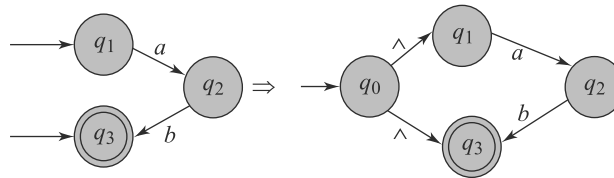


Fig. 2.6 Transition graph and equivalent generalised transition graph

- (ii) If necessary, get a generalised transition graph to have exactly one final state having no outgoing transitions:

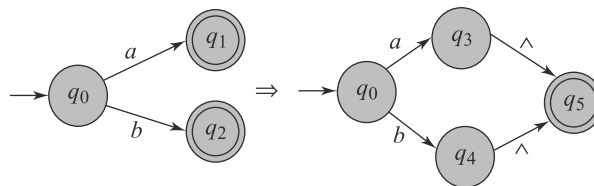


Fig. 2.7 Transition graph and generalised transition graph

- (iii) Consolidate multiple transition in the same direction between states into one:

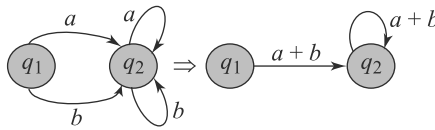


Fig. 2.8 Transition graph and generalised transition graph

(iv) Eliminate intermediate states one by one:

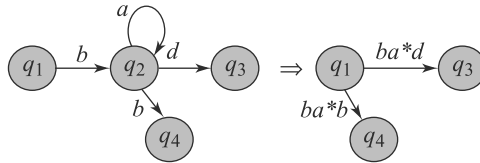


Fig. 2.9 Transition graph and generalised transition graph

2.3.3 Transition Table

A transition table is the tabular representation of the transition system of an automaton. A transition table is also known as transition function table or state table. In this representation the initial (start) state is represented by an arrow towards it, and a final state is represented by a circle. Let us consider the following transition table (Table 2.1) and its equivalent transition diagram (Fig. 2.10):

Table 2.1 A transition table of an FA (deterministic)

PRESENT STATE	INPUTS	
	<i>a</i>	<i>b</i>
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_0
(q_2)	q_2	q_0

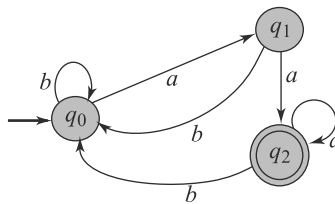


Fig. 2.10 Transition diagram of finite automata given in Table 2.1

In the transition table (Table 2.1), q_0 is initial state and q_2 is final state. The C++ program given below simulates this finite automaton.

```
/*Program to implement following transition table:
```

```

-----
|      | a | b |
| -----
|  q0  | q1 | q0 | q0 is initial state
|  q1  | q2 | q0 | q2 is final state
|  q2  | q2 | q0 |
| -----

```

```
*/
```

```

#include<iostream.h>
#include<stdio.h>
#include<conio.h>
int s = 0, i, n;
char a[5];
void state0();
void state1();
void state2();
void main(){
    clrscr();
    cout<<"enter the characters in a string:";
    cin>>n;
    for(int j = 0; j < n; j++)
        for(int i = 0; i < n; i++)
        {
            cin>>a[i];
            if(s == 0)
            {
                state0();
                break;
            }
            else
            {
                if(s == 1)
                {state1();
                break;
                }
                else
                {
                    if(s == 2)
                    {
                        state2();
                        break;
                    }
                }
            }
            {
                if(s == 2)
                {cout<<"\nString accepted !!";
                }
                else
                {cout<<"\nString not accepted !!";
                }
            }
        }
    getch();
}

```

```
void state0()
{
    if(a[i]=='a')
    {
        s = 1;
    }
    else
    if(a[i] == 'b')
    {
        s = 0;
    }
    else
    {
        cout<<"invalid character";
    }
}
void state1()
{
    if(a[i]=='a')
    {
        s = 2;
    }
    else
    if(a[i]=='b')
    {
        s = 0;
    }
    else
    {
        cout<<"invalid character";
    }
}
void state2()
{
    if(a[i]=='a')
    {
        s = 2;
    }
    else
    if(a[i]=='b')
    {
        s = 0;
    }
    else
    {
        cout<<"invalid character";
    }
}
```

2.4

FA WITH AND WITHOUT EPSILON TRANSITIONS

The FA that takes input from $\Sigma \cup \{\epsilon\}$ is called a FA with epsilon transitions. The transition function of such FA can be given as $Q \times \Sigma \cup \{\epsilon\} \rightarrow Q$. For example, consider a finite automaton M defined as

$M = (\{q_1, q_2, q_3\}, \{0, 1\} \cup \{\epsilon\}, \delta, q_1, \{q_3\})$, where δ is given by

$$\delta(q_1, \epsilon) = q_2,$$

$$\delta(q_1, 1) = q_1,$$

$$\delta(q_2, \epsilon) = q_3,$$

$$\delta(q_3, 0) = q_2,$$

This represents that there are total three states in this FA $\{q_1, q_2, q_3\}$ with q_1 as initial or start state and q_3 as final state or accepting state, the option of input are $\{\epsilon, 0, 1\}$.

The transition equation $\delta(q_1, \epsilon) = q_2$, shows that there is a transition from state q_1 to q_2 on input ϵ , or say without input.

The transition equation $\delta(q_1, 1) = q_1$, shows that there is a transition from state q_1 to q_1 on input 1.

The transition equation $\delta(q_2, \epsilon) = q_3$, shows that there is a transition from state q_2 to q_3 on input ϵ , or say without input.

The transition equation $\delta(q_3, 0) = q_2$, shows that there is a transition from state q_3 to q_2 on input 0.

❖ Good to Know!

1. If a finite automaton takes an epsilon move then the present state and the next state can be considered to be same.
2. The transition $\delta(q_i, \epsilon) = q_i$, is true for all the states in Q . As it has no significance therefore it is not mentioned in set of transitions.

2.5

LANGUAGE OF DETERMINISTIC FINITE AUTOMATA

The set of all strings accepted by a DFA is called the language of DFA. The number of strings accepted by a DFA depends upon the number of unique paths from initial state to final state while processing strings with a DFA. On this basis a deterministic finite automaton can accept a finite or infinite number of states. If there are more than one final states in a deterministic finite automaton then the language accepted by that automaton is defined as:

$$L = \{\text{set of all strings consumed from initial state to all final states}\}$$

For example, the DFA below accepts infinite number of strings because there are infinite numbers of paths from initial state to final state.

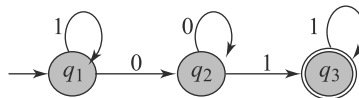


Fig. 2.11 A DFA

The language accepted by this DFA is $L = \{1^n 0^{m+1} 1^{l+1} \mid n, m, l \geq 0\}$. This language consists of strings starting with zero or more occurrences of 1's followed by at least one occurrence 0 ended by at least one occurrence 1. In a DFA, a particular string is accepted by one and only one unique path.

2.6

ACCEPTABILITY OF A STRING BY A DFA

A string w is accepted by a finite automaton $M = (Q, \Sigma, \delta, q_0, F)$ if

$$\delta(q_i, w) = q_j \in F$$

where q_0 is initial (start) state and F is set of final (accepting) states.

This is basically the acceptability of a string by the final state. Needless to say that if there is no final state in a particular finite automaton then the language accepted by that finite automaton will be ϕ .

Let us consider following finite state machine (Fig. 2.12) with q_0 as initial state. If we wish to determine its language if:

- (i) only q_0 is the final state.
- (ii) only q_1 is the final state.
- (iii) only q_2 is the final state.
- (iv) only q_3 is the final state.
- (v) only q_0 and q_2 are the final states.
- (vi) only q_1 and q_3 are the final states.
- (vii) all are final states.

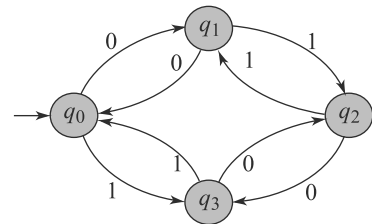


Fig. 2.12 FA with four states

Following are the languages of above finite automaton as per different final states:

- (i) It accepts all strings over $\{0, 1\}$ having even number of 0's and even number of 1's.
- (ii) It accepts all strings over $\{0, 1\}$ having odd number of 0's and even number of 1's. The overall length of input string will be odd.
- (iii) It accepts all strings over $\{0, 1\}$ having odd number of 0's and odd number of 1's. The overall length of input string will be even.
- (iv) It accepts all strings over $\{0, 1\}$ having even number of 0's and odd number of 1's. The overall length of input string will be odd.
- (v) It accepts all strings over $\{0, 1\}$ of even length.
- (vi) It accepts all strings over $\{0, 1\}$ of odd length.
- (vii) It accepts all strings over $\{0, 1\}$, i.e., $\{0, 1\}^*$

2.7

PROCESSING OF STRINGS BY DFA

The processing of a string by a DFA is started from initial state and terminated at any one state of the DFA. If the string's processing is terminated at some nonfinal state then the string is not supposed to be part of language. A string w_1 is accepted by a finite automaton $M = (Q, \Sigma, \delta, q_0, F)$ if

$$\delta(q_i, w_1) = q_j \in F$$

Similarly, A string w_2 is not accepted by a finite automaton $M = (Q, \Sigma, \delta, q_0, F)$ if

$$\delta(q_i, w_2) = q_k \notin F$$

where q_0 is initial (start) state and F is set of final (accepting) states.

Let us consider a DFA given below.

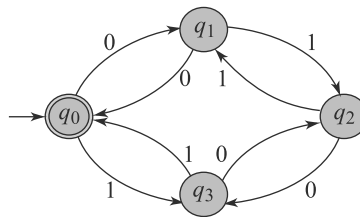


Fig. 2.13 DFA with four states

The figure below, processes the strings 1, 00, 01, 010, 011, 1010 and 0100. The processing of two strings 00 and 0110 is terminated to q_0 , which is the final state, and rest strings are terminated to a nonfinal state. Therefore, the strings 00 and 0101 are accepted and rest are not accepted.

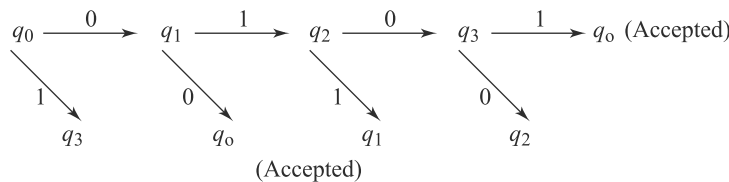


Fig. 2.14 Recognition of strings 00 and 0101

2.8

NONDETERMINISTIC FINITE AUTOMATA

A mechanism equivalent of DFA is non-deterministic FA. The deterministic finite automaton is a useful concept in proving theorems. Also the concept of nondeterminism plays an important role in both theory of computation and theory of languages.

Let us consider a modification in finite automata model to allow zero, one, or more transitions from some state on same input. This modified model of finite automaton is called nondeterministic finite automata (NFA or NFA). Let us consider the transition diagram given by Fig. 2.15.

In transition system (Fig. 2.15) there are two outgoing edges labelled 0 from state q_0 , one returning back to q_0 and one going state q_3 . We cannot decide whether the next state is q_0 or q_3 . So this finite automaton is nondeterministic. The same condition occurs when input 1 is applied when the automaton is in state q_0 .

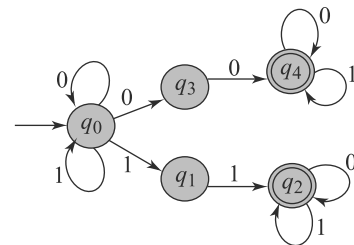


Fig. 2.15 A nondeterministic finite automaton

❖ Did you Know ?

In 1959, Michael Rabin (with Dana Scott) introduced the notion of nondeterminism. This initiated an idea by which the idea that a computer program can make guesses and the concept of parallelism. Rabin defined the inherent computational difficulty of a problem by inspiring from a puzzle posed by John McCarthy.

In 1976, Michael Rabin and Dana Scott shared the Turing award for their joint paper “Finite Automata and Their Decision Problem”. This paper introduced the idea of nondeterministic machines. This concept proved to be an extremely valuable theory in mathematical models of automatic machines. This classic paper has been a continuous source of inspiration for further work in this field.

Description of NDFA

A nondeterministic finite automaton is defined as 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

$Q \rightarrow$ finite non-empty set of states

$\Sigma \rightarrow$ set of inputs called alphabet

$\delta \rightarrow$ transition function which maps from $Q \times \Sigma \rightarrow 2^Q$

$q_0 \rightarrow$ initial state (i.e., there is only initial state), and $q_0 \in Q$

$F \rightarrow$ set of final states, and $F \subseteq Q$.

❖ Good to Know!

2^Q is the number of elements in power set of Q , the set of all subset of Q .

For the formal description of transition δ of NDFA, let us consider following example:

Suppose an NDFA is in state q_0 and there are total two states in the NDFA (e.g., q_0 and q_1), then there are four possible transitions (2^Q here $Q = 2$) from state q_0 , which are:

- (i) No transition,
- (ii) q_0 to q_0 ,
- (iii) q_0 to q_1
- (iv) q_0 to q_0 and q_1 .

Hence, if there are total Q number of states in an NDFA, then there are at most 2^Q transitions from any state. In case of three states (q_0, q_1, q_2) there will be at most eight possible transitions. It can be described as all combinations of 0's with three bits (e.g., 000, 001, 010, 011, 100, 101, 110 and 111). Suppose the first bit is assigned for q_0 , second bit for q_1 and third bit for q_2 . Then, bits sequence '011' represents transition from q_0 to q_1 and q_2 . Similarly '110' represents the transition from q_0 to q_0 and q_1 , if q_0 is the present state. Also the bits sequence '000' represents no transition at all.

Let us consider, if there is a transition from state q_1 to q_2 and q_3 on input b , then according to transition function

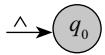
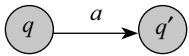
$$\begin{array}{ccc}
 \begin{array}{c} Q \\ \uparrow \\ \text{Present} \\ \text{state } (q_1) \end{array} & \times & \begin{array}{c} \Sigma \\ \uparrow \\ \text{Present} \\ \text{input } (b) \end{array} & \rightarrow & \begin{array}{c} 2^Q \\ \uparrow \\ \text{a set of next states} \\ (q_2 \text{ and } q_3) \end{array}
 \end{array}$$

NFA consists of

- A variable that can only take on a finite number of different states
- A read head by which the input tape will be read from left to right
- A transition relation δ that forms the control of the automaton
- an initial state and
- zero one or several final states

Let us now look more carefully at the correspondence between NFA and transition diagram (Table 2.2)

Table 2.2 States of NFA and their transitions

STATES/TRANSITIONS OF NFA	TRANSITION DIAGRAM	EXPLANATION
For any state $q \in Q$		An intermediate state
An initial state q_0		state with an incoming arrow on no input (or on input Λ)
A final state $q_f \in Q$		A state with two concentric circles.
$(q, a, q') \in \delta$		Transition from state q to q' on input a .

An important relation of finite automata and regular expression is given in following theorem:

Theorem 2.1 For every regular expression r , there is a nondeterministic finite automaton that accepts the regular set described by r .

The proof of this theorem can be given by the procedure of converting from regular expression to NFA. We will discuss this conversion in Chapter 4.

2.9

LANGUAGE OF NFA

The language of an NFA is set of all strings accepted by an NFA. The number of strings accepted by a NFA depends upon the number of unique paths from initial state to final state while processing strings with a NFA. On this basis a nondeterministic finite automaton can accept finite or infinite number of states. If there are more than one final states in a nondeterministic finite automaton then the language accepted by that NFA is defined as:

$$L = \{\text{set of all strings consumed from initial state to all final states}\}$$

For example, the NFA below accepts infinite number of strings because there are infinite numbers of paths from initial state to final state.

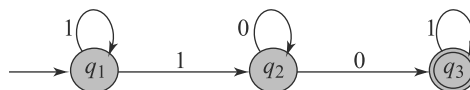


Fig. 2.16 AN NFA

The language accepted by this DFA is $L = \{1^n 0^m 1^l \mid n, m \geq 1, l \geq 0\}$. In an NFA, a particular string may be accepted by one or more paths. Therefore the NFA uses the concept of parallelism.

2.10

EQUIVALENCE BETWEEN DFA AND NFA

In contrast to the NFA (NDFA), the deterministic finite automaton (DFA) has:

- no ϵ -transitions or say $\wedge$ -moves
- for every $\delta(q, a)$ with $q \in Q$ and $a \in \Sigma$ there is at most one successor state.

Therefore, for every word $w \in \Sigma^*$, there is at most one w path in the transition diagram for DFA M . If w is in the language of M , then this path leads from the initial state to a final state without M having to employ the ‘guesswork’ of an NFA. So, DFA are preferred for practical use.

A deterministic finite automaton can simulate the behavior of NFA by increasing the number of states. A DFA $(Q, \Sigma, \delta, q_0, F)$ can be viewed as an NFA $(Q, \Sigma, \delta', q_0, F)$ by defining

$$\delta'(q, a) = \{\delta(q, a)\}$$

2.10.1 Conversion of NFA into DFA

Theorem 2.2 If a language L is accepted by an NFA then there is a DFA that accepts L .

Proof 1 The proof of the above theorem is constructive and uses the subset construction. If there are two w paths, the corresponding states from a common state in the DFA will be:

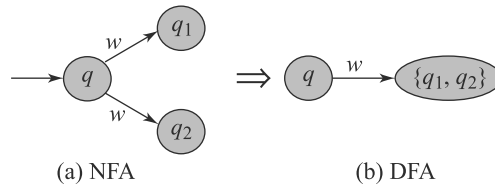


Fig. 2.17 An NFA and its equivalent DFA

Proof 2 Let $M = (Q, \Sigma, \delta, q_0, F)$ be an NFA accepting the language L . We construct DFA M' as follows:

$$M' = (Q', \Sigma, \delta', q_0', F')$$

where $Q' = 2^Q$ (any state in Q' is represented by $\{q_1, q_2, q_3, \dots, q_i\}$ where $q_1, q_2, q_3, \dots, q_i \in Q$)

$$q_0' = q_0,$$

$F' =$ set of all subsets of Q containing an element of F .

Let us consider the construction of Q' , q_0' and F' . NFA M is initially at q_0 , the initial state. By applying an input symbol ‘ a ’, M can reach any of the states in $\delta(q_0, a)$. To describe M , we require all the possible states that M can reach after the application of a . So, M' has to remember all these possible states at any instance of time. Hence the states of M' may be defined as subset of Q . As q_0 is initial state so q_0' is defined as $\{q_0\}$. If the string w is accepted by M , then M reaches a final state on processing w . Therefore a final state in DFA M' is any subset of Q having some final state of NFA M .

Now we can define transition function δ' of DFA M' :

$$\delta'([q_1, q_2, q_3, \dots, q_k], a) = \delta(q_1, a) \cup \delta(q_2, a) \cup \delta(q_3, a) \dots \cup \delta(q_k, a)$$

or

$$\delta'([q_1, q_2, q_3, \dots, q_k], a) = [P_1, P_2, P_3, \dots, P_k] \text{ if and only if}$$

$$\delta(\{q_1, q_2, q_3, \dots, q_k\}, a) = [P_1, P_2, P_3, \dots, P_k]$$

Now we prove:

$$\delta'(q_0', x) = [q_1, q_2, q_3, \dots, q_k] \text{ if and only if}$$

$$\delta(q_0, s) = \{q_1, q_2, q_3, \dots, q_k\} \text{ for all } s \text{ in } \Sigma^*$$

when $|s| = 0$, $\delta(q_0, \wedge) = q_0$ and by definition of δ' we have

$$\delta'(q_0', \wedge) = q_0' = [q_0']$$

Assume there is another string t with $t \leq m$. Let s be a string of length $m + 1$. So we can write s as ta , where

$$|t| = m \text{ and } a \in \Sigma. \text{ Let } \delta(q_0, t) = [P_1, P_2, P_3, \dots, P_k] \text{ and}$$

$$\delta(q_0, ta) = \{r_1, r_2, r_3, \dots, r_k\} \text{ as } t \leq m, \text{ by induction hypothesis we can say}$$

$$\delta'(q_0', t) = [P_1, P_2, P_3, \dots, P_k]$$

and

$$\{r_1, r_2, r_3, \dots, r_k\} = \delta(q_0, ta) = \delta(\delta(q_0, t), a) = \delta(\{P_1, P_2, P_3, \dots, P_k\}, a)$$

Now by definition of δ'

$$\delta'([P_1, P_2, P_3, \dots, P_k], a) = \{r_1, r_2, r_3, \dots, r_k\}$$

Hence

$$\begin{aligned} \delta'(q_0', ta) &= \delta'(\delta'(q_0', t), a) = \delta'([P_1, P_2, P_3, \dots, P_k], a) \\ &= [r_1, r_2, r_3, \dots, r_k] \end{aligned}$$

Thus we can say that $s = ta$. Now, $s \in T(M)$ if and only if $\delta(q_0, s)$ contains a state of F . $\delta(q_0, s)$ contains a state of F iff $\delta'(q_0', s)$ is in F' . Hence $s \in T(M)$ if and only if $s \in T(M')$. This proves that DFA M' accepts the language L . ($s \in T(M)$ represents the acceptability of string s by M).

When transition for NFA M is given in form of transition table, the construction of DFA M' equivalent to NFA M is done by following method:

$\delta(q_i, a)$ is given by the row corresponding to q_i and column corresponding to input symbol say ' a '. To construct $\delta'([q_1, q_2, q_3, \dots, q_k], a)$, consider the states present in the rows corresponding to $q_1, q_2, \dots, q_k$ and the column corresponding to ' a '. These states will construct $\delta'([q_1, q_2, q_3, \dots, q_k], a)$. The Table 2.4 shows the DFA equivalent to NFA given by Table 2.3.

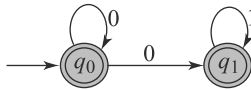
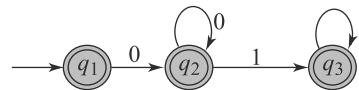
Example 2.2

(i) Construct a DFA equivalent to the NFA given by transition Table 2.3.

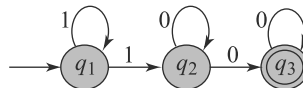
Table 2.3 Transition table of Example 2.2

PRESENT STATE	NEXT STATE	
	<i>a</i>	<i>b</i>
$\rightarrow q_1$	q_1	q_2
q_2	q_2	q_1, q_2

(ii) Show that the following NFA and DFA are equivalent:

**Fig. 2.18** $FA M_1$ (NFA)**Fig. 2.19** $FA M_2$ (DFA)

(iii) Convert the following NFA into equivalent DFA:

**Fig. 2.20** An NFA**Solution** (i) Let M' be equivalent DFA of M . For M'

- states are $\phi, [q_1], [q_2], [q_1, q_2]$
- $[q_1]$ is initial state
- $[q_1]$ and $[q_1, q_2]$ are the final states as these are the only states containing q_1 , and
- δ is defined by the transition table given below:

Table 2.4 DFA equivalent to NFA given by Table 2.3

PRESENT STATE	NEXT STATE	
	<i>a</i>	<i>b</i>
$\rightarrow [q_1]$	$[q_1]$	$[q_2]$
$[q_2]$	$[q_2]$	$[q_1, q_2]$
$\rightarrow [q_1, q_2]$	$[q_1, q_2]$	$[q_1, q_2]$

The states q_1 and q_2 appear in the rows corresponding to q_1 and q_2 and the column corresponding to a . Therefore, $\delta([q_1, q_2], a) = [q_1, q_2]$

(ii) Both the finite automata accept the strings represented by $0^* + 0^*1^*$, therefore both the finite automata are equivalent.

(iii) Same as part (i).

2.10.2 Difference between DFA and NDFA

In deterministic finite automaton, for each present state there is at most one transition for each possible input. In contrast to deterministic version, the nondeterministic finite automaton, there can be more than one transition from a state for a given possible input.

Non-deterministic finite automata are usually converted into deterministic finite automata before their use. In most of the cases, the generated deterministic finite automaton is exponentially bigger (by having few more states) than the non-deterministic automaton. It is always desirable to substantially optimise the converted DFA. The standard acceptance condition for non-deterministic finite automata requires that some computation accepts the input. It also provides a dual notion, where all non-deterministic computations must be accepted, that means a string may be accepted by following two or more different paths.

Beyond the theory and software implementation, finite automata occur also in hardware circuits, where the input, the state and the output are bit vectors of fixed size. The Mealy and Moore machines are such examples.

2.11

NFA WITH AND WITHOUT EPSILON TRANSITIONS

Now we will extend our existing model of NFA to include transitions without giving any input (By providing empty input ϵ or $\wedge$). Let us consider the transition diagram (Fig. 2.21).

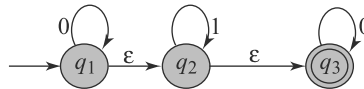


Fig. 2.21 A finite automaton with ϵ -moves

The non-deterministic finite automaton (Fig. 2.21) accepts the language consisting any occurrence of 0's followed by any occurrence of 1's followed by any occurrence of 0's. This NDFA accepts a string 's' if there is some path 'w' from initial state q_1 to final state q_3 in which edges labelled may be included. For example, the string '00110' is accepted by the NDFA by following path:

$$\begin{array}{ccccccccc} 0 & 0 & \epsilon & 1 & 1 & \epsilon & 0 \\ q_1 \rightarrow q_1 & \rightarrow q_1 & \rightarrow q_2 & \rightarrow q_2 & \rightarrow q_2 & \rightarrow q_3 & \rightarrow q_3 \end{array}$$

here the input string is $00\epsilon 11\epsilon 0$ or $00\wedge 11\wedge 0$. Needless to say that

$$00\epsilon 11\epsilon 0 = 00\wedge 11\wedge 0 = 00110$$

A nondeterministic finite automaton with $\wedge$ -moves is defined as 5-tuples $(Q, \Sigma, \delta, q_0, F)$ with all elements stated as before, but the transition function δ maps from $Q \times (\Sigma \cup \{\wedge\}) \rightarrow 2^Q$. $(\Sigma \cup \{\wedge\})$ represents set of input symbols. For example in transition $\delta(q, x) = q'$, x is a symbol from $\Sigma \cup \{\wedge\}$.

NFA Without $\wedge$ -Moves The transition diagram which are associated with label $\wedge$ (or ϵ) are called transition diagram with $\wedge$ -moves. These transitions take place when no input (or say input $\wedge$) is applied at present state. These transitions can be eliminated by determining $\wedge$ -closure of all states of NDFA. After elimination of $\wedge$ -moves the resultant NDFA can have either less or same number of states as in NDFA with $\wedge$ -moves. The initial state of NDFA without $\wedge$ -moves is the closure of initial state of NDFA

with $\wedge$ -moves. For example, if q_0 is the initial state of NDFA with moves then $\wedge$ -closure(q_0) will be the initial state of NDFA without $\wedge$ -moves. $\wedge$ -closure(q_0) is the set of all states reachable from q_0 on input $\wedge$. The final state of NDFA without $\wedge$ -moves will be all those states that contain final state of NDFA with $\wedge$ -moves. Thus the transitions of NDFA without $\wedge$ -moves will be different from NDFA with $\wedge$ -moves.

Let the NDFA with $\wedge$ -moves be

$$M = (Q, \Sigma, \delta, q_0, F),$$

and let the constructed NDFA without $\wedge$ -moves be

$$M' = (Q', \Sigma, \delta', q_0', F'),$$

Where δ' is defined in terms of δ as

$$\delta'(q, x) = \wedge\text{-closure}((q, x)), q \in Q, \text{ and } x \in \Sigma.$$

and Q' is the set $\wedge$ -closure of all states of NDFA with $\wedge$ -moves

Equivalence of NDFA with $\wedge$ -moves and FA without $\wedge$ -moves

If L is accepted by an NDFA with $\wedge$ -transitions then L is accepted by an NDFA without $\wedge$ -transitions.

Let $M = (Q, \Sigma, \delta, q_0, F)$, be an NDFA with $\wedge$ -moves. Let us construct NDFA M' without $\wedge$ -moves, such that

$$M' = (Q, \Sigma, \delta', q_0, F')$$

F' (the set of final states) of M' is defined as

$$F' = \begin{cases} F \cup \{q\} & \text{if } \wedge\text{-closure}(q) \text{ contains a state of } F \\ F & \text{otherwise} \end{cases}$$

and δ' is defined as

$$\delta'(q, a) = \hat{\delta}(q, a) = \wedge\text{-closure}(\delta(q, a)) \text{ for all } a \in \Sigma$$

Note Here $\hat{\delta}(q, a)$ is the state of NDFA after reading the symbol a when NDFA was in state q . $\wedge$ -closure(q_0) is the set of all states reachable from q_0 on input $\wedge$.

We can show this by using induction on $|x|$ such that $\delta'(q_0, a) = \wedge\text{-closure}(\delta(q_0, a))$. However, this is not true for $|x| = 0$. We know that $\hat{\delta}(q_0, a) = \{q_0\}$ and $\hat{\delta}(q, \wedge) = \wedge\text{-closure}(q_0)$. Our basis of proof is $|x| = 1$; by definition of δ' we have

$$\delta'(q, a) = \hat{\delta}(q, a) = \wedge\text{-closure}(\delta(q_0, a))$$

Now we look $|x| > 1$. Let $x = sa$ for $a \in \Sigma$, Then

$$\delta'(q, sa) = \delta'(\delta'(q, s), a)$$

$$\delta'(q_0, s) = P \text{ by induction hypothesis.}$$

and therefore, $\delta'(P, a) = \text{because } P = \hat{\delta}(q_0, a)$

So, $\delta'(q, sa) = \hat{\delta}(q_0, sa)$

Now we must show that $\delta'(q_0, x)$ contains a state in F' if and only if $\hat{\delta}(q_0, x)$ contains a state in F . If $x = \wedge$, then $\delta'(q_0, \wedge) = \{q_0\}$ and q_0 is placed in F' whenever $\hat{\delta}(q, \wedge) = \wedge\text{-closure}(q_0)$ contains a state in F .

If $x \neq \wedge$ (i.e., $x = sa$), then if $\hat{\delta}(q_0, x)$ contains a state of F then it is sure that $\delta'(q_0, x)$ contains the same state in F' .

Example 2.3

Consider the following NDFA with $\wedge$ -moves. Construct an equivalent NDFA without $\wedge$ -moves.

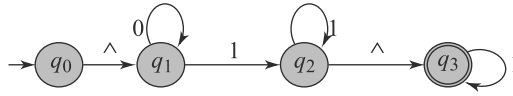


Fig. 2.22 Finite automaton with $\wedge$ -moves of Example 2.3

Solution The given NDFA with $\wedge$ -moves is

$$M = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, \{q_3\})$$

where δ is defined as

$$\delta(q_0, \wedge) = q_1, \quad \delta(q_1, 0) = q_1, \quad \delta(q_1, 1) = q_2,$$

$$\delta(q_2, \wedge) = q_3, \quad \delta(q_2, 1) = q_2, \quad \delta(q_3, 1) = q_3,$$

Let the NDFA without $\wedge$ -moves be

$$M' = (Q', \{0, 1\}, \delta', q_0', F')$$

Where

$$Q' = \{\text{set of } \wedge\text{-closure of all states of } M\}$$

$$= \{\wedge\text{-closure}(q_0), \wedge\text{-closure}(q_1), \wedge\text{-closure}(q_2), \wedge\text{-closure}(q_3)\}$$

Let us determine $\wedge$ -closures of q_0, q_1, q_2, q_3

$$\begin{aligned} \wedge\text{-closure}(q_0) &= \{\text{set of all states reachable from } q_0 \text{ on input } \wedge\} \\ &= \{q_0, q_1\} \end{aligned}$$

$$\begin{aligned} \wedge\text{-closure}(q_1) &= \{\text{set of all states reachable from } q_1 \text{ on input } \wedge\} \\ &= \{q_1\} \end{aligned}$$

$$\begin{aligned} \wedge\text{-closure}(q_2) &= \{\text{set of all states reachable from } q_2 \text{ on input } \wedge\} \\ &= \{q_2, q_3\} \end{aligned}$$

$$\begin{aligned} \wedge\text{-closure}(q_3) &= \{\text{set of all states reachable from } q_3 \text{ on input } \wedge\} \\ &= \{q_3\} \end{aligned}$$

Now we can define NDFA M' as

$$M' = (\{\{q_0, q_1\}, \{q_1\}, \{q_2, q_3\}, \{q_3\}\}, \{0, 1\}, \delta', \{q_0, q_1\}, \{\{q_3\}, \{q_2, q_3\}\})$$

where δ' is computed as

$$\begin{aligned} \delta'(\{q_0, q_1\}, 0) &= \wedge\text{-closure}(\delta(\{q_0, q_1\}, 0)) \\ &= \wedge\text{-closure}(\delta(\{q_0\}, 0) \cup \delta(\{q_1\}, 0)) \\ &= \wedge\text{-closure}(\emptyset \cup q_1) \\ &= \wedge\text{-closure}(q_1) \\ &= \{q_1\} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_0, q_1\}, 1) &= \wedge\text{-closure}(\delta(\{q_0, q_1\}, 1)) \\ &= \wedge\text{-closure}(\delta(\{q_0\}, 1) \cup \delta(\{q_1\}, 1)) \\ &= \wedge\text{-closure}(\emptyset \cup q_2) \\ &= \{q_2, q_3\} \end{aligned}$$

$$\begin{aligned}
 \delta'(\{q_1\}, 0) &= \wedge\text{-closure}(\delta(\{q_1\}, 0)) \\
 &= \wedge\text{-closure}(q_1) \\
 &= \{q_1\} \\
 \delta'(\{q_1\}, 1) &= \wedge\text{-closure}(\delta(\{q_1\}, 1)) \\
 &= \wedge\text{-closure}(q_2) \\
 &= \{q_2, q_3\} \\
 \delta'(\{q_2, q_3\}, 0) &= \wedge\text{-closure}(\delta(\{q_2, q_3\}, 0)) \\
 &= \wedge\text{-closure}(\delta(\{q_2\}, 0) \cup \delta(\{q_3\}, 0)) \\
 &= \wedge\text{-closure}(\phi \cup \phi) \\
 &= \{\phi\} \\
 \delta'(\{q_2, q_3\}, 1) &= \wedge\text{-closure}(\delta(\{q_2, q_3\}, 1)) \\
 &= \wedge\text{-closure}(\delta(\{q_2\}, 1) \cup \delta(\{q_3\}, 1)) \\
 &= \wedge\text{-closure}(q_2 \cup q_3) \\
 &= \wedge\text{-closure}(q_2) \cup \wedge\text{-closure}(q_3) \\
 &= \{q_2, q_3\} \cup \{q_3\} \\
 &= \{q_2, q_3\} \\
 \delta'(\{q_3\}, 0) &= \wedge\text{-closure}(\delta(\{q_3\}, 0)) \\
 &= \wedge\text{-closure}(\phi) \\
 &= \{\phi\} \\
 \delta'(\{q_3\}, 1) &= \wedge\text{-closure}(\delta(\{q_3\}, 1)) \\
 &= \wedge\text{-closure}(q_3) \\
 &= \{q_3\}
 \end{aligned}$$

Now the transition table for NFA without $\wedge$ -moves can be drawn with the help of δ' as

Table 2.5 Transition table of Example 2.3

PRESENT STATE	INPUTS	
	<i>a</i>	<i>b</i>
$\rightarrow\{q_0, q_1\}$	$\{q_1\}$	$\{q_2, q_3\}$
$\{q_1\}$	$\{q_1\}$	$\{q_2, q_3\}$
$\{q_2, q_3\}$		$\{q_2, q_3\}$
$\{q_3\}$		$\{q_0\}$

For simplicity, let us assume that

$$\{q_0, q_1\} = q_A, \{q_1\} = q_B, \{q_2, q_3\} = q_C, \text{ and } \{q_3\} = q_D$$

The transition diagram of resultant NFA without $\wedge$ -moves is given by Fig. 2.23.

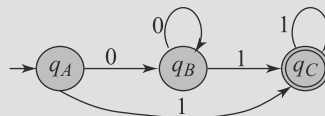
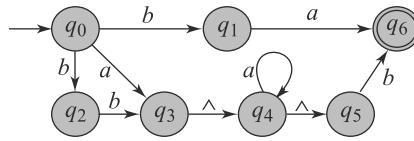


Fig. 2.23 Transition diagram of NFA without $\wedge$ -moves of Example 2.3

Example 2.4Construct an NFA without $\wedge$ -moves corresponding to the following NFA.**Fig. 2.24** FA with $\wedge$ -moves of Example 2.4**Solution** Constructed NDFA with $\wedge$ -moves is

$$M = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}, \{a, b\}, \delta, q_0, \{q_6\})$$

Let us determine $\wedge$ -closure of all the states of M , i.e., $q_0, q_1, q_2, q_3, q_4, q_5$, and q_6 as we have determined in previous example.

$$\wedge\text{-closure}(q_0) = \{q_0\}$$

$$\wedge\text{-closure}(q_1) = \{q_1\}$$

$$\wedge\text{-closure}(q_2) = \{q_2\}$$

$$\wedge\text{-closure}(q_3) = \{q_3, q_4, q_5\}$$

$$\wedge\text{-closure}(q_4) = \{q_4, q_5\}$$

$$\wedge\text{-closure}(q_5) = \{q_5\}$$

$$\wedge\text{-closure}(q_6) = \{q_6\}$$

Let the NDFA without $\wedge$ -moves equivalent to NDFA having $\wedge$ -moves be defined as

$$M' = (\{\{q_0\}, \{q_1\}, \{q_2\}, q_3, q_4, q_5\}, \{q_4, q_5\}, \{q_5\}, \{q_6\}\}, \{a, b\}, \delta', q_0', F')$$

where, δ' is defined as

$$\delta'(\{q_0\}, a) = \{q_3, q_4, q_5\},$$

$$\delta'(\{q_0\}, b) = \{q_1\} \cup \{q_2\} = \{q_1\}, \{q_2\}$$

$$\delta'(\{q_1\}, a) = \{q_6\},$$

$$\delta'(\{q_1\}, b) = \{\emptyset\}$$

$$\delta'(\{q_2\}, a) = \{\emptyset\},$$

$$\delta'(\{q_2\}, b) = \{q_3, q_4, q_5\}$$

$$\delta'(\{q_3, q_4, q_5\}, a) = \{q_4, q_5\},$$

$$\delta'(\{q_3, q_4, q_5\}, b) = \{q_6\}$$

$$\delta'(\{q_4, q_5\}, a) = \{q_4, q_5\},$$

$$\delta'(\{q_4, q_5\}, b) = \{q_6\}$$

$$\delta'(\{q_5\}, a) = \{\emptyset\},$$

$$\delta'(\{q_5\}, b) = \{q_6\}$$

$$\delta'(\{q_6\}, a) = \{\emptyset\},$$

$$\delta'(\{q_6\}, b) = \{\emptyset\}$$

Let us assume, $\{q_0\} = q_A, \{q_1\} = q_B, \{q_2\} = q_C, \{q_3, q_4, q_5\} = q_D, \{q_4, q_5\} = q_E, \{q_5\} = q_F, \{q_6\} = q_G$. By using above transitions δ' , following finite automaton (NDFA without $\wedge$ -moves) can be drawn

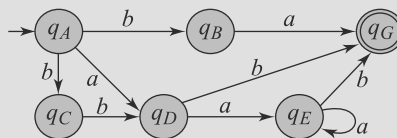
**Fig. 2.25** Finite automaton without $\wedge$ -moves

Table 2.6 Transition table of NDFA without $\wedge$ -moves

PRESENT STATE	INPUTS	
	<i>a</i>	<i>b</i>
$\rightarrow q_A$	q_D	q_B, q_C
q_B	q_F	--
q_C	--	q_D
q_D	q_E	q_G
q_E	q_E	q_G
q_G	--	--

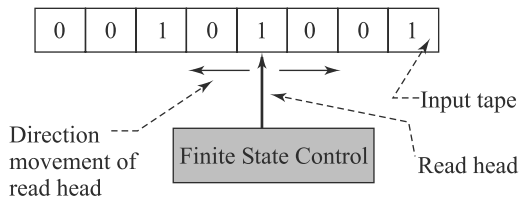
❖ Did you Know ?

Rabin got a book by S. C. Kleene called *Mata-mathematics* during his study. It had a chapter on Alan Turing's work, whose focus was mainly the notion of computability and the Turing machine. This book really appealed to Rabin. Since Rabin was very eager to study science, he broke the promise to his father. He was driven away from the sciences by his charm the exact sciences, but that was only the cause that made Rabin successful in the other areas of his life.

2.12

TWO-WAY FINITE AUTOMATA

We have studied finite automata as a control unit that reads a magnetic tape, moving one block right side


Fig. 2.26 A two way finite automaton

at each move. We made some change by adding non-determinism to this model which allowed many copies of the unit to read the magnetic tape simultaneously. Then we added $\wedge$ -transitions, which allowed change of state without reading any input from the tape. When read head is allowed to move in either direction (left or right), the modified model is called two-way finite automaton given in Fig. 2.26.

❖ Definition

A two way finite automaton M is defined as 5-tuples $(Q, \Sigma, \delta, q_0, F)$, where

$Q \rightarrow$ finite non-empty set of states

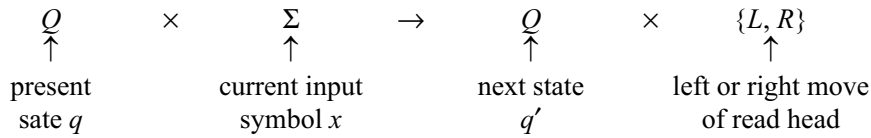
$\Sigma \rightarrow$ finite non-empty set of inputs called alphabet

$q_0 \rightarrow$ initial state, and $q_0 \in Q$

$F \rightarrow$ set of final states, and $F \subseteq Q$

$\delta \rightarrow$ transition function which maps from $Q \times \Sigma \rightarrow Q \times \{L, R\}$

The transition function in above definition indicates if two-way finite automaton is in state q , scanning input symbol x , ($x \in \Sigma$) it goes to next state q' and moves its read head one block left or right. For example:



The transition $\delta(q_i, a) = \{q_j, L\}$ indicates that there is transition from q_i state to q_j state on input a and machines takes a left move. For example, if the input string is $babb$, the first symbol b is already scanned and the current input is a the machine is in state q_i then transition can clearly be explained as following:

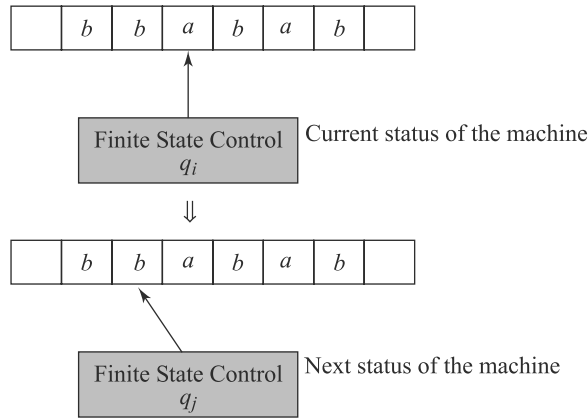


Fig. 2.27 Left move taken by a two way finite automata

If this machine never takes a left move then it shows behavior of a normal finite automaton.

Example 2.5

Design a 2-way finite automata to accept language $L = \{a^n b^m \mid m, n \geq 0\}$.

Solution The following is the required 2-way F.A.

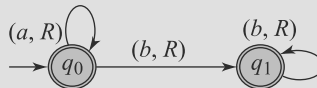


Fig. 2.28 Two way finite automata of Example 2.5

The above 2-way FA can also be represented in the form of transitions as following:

$M = (\{q_0, q_1\}, \{a, b\}, \delta, q_0, \{q_0, q_1\})$, where δ is defined as:

$\delta(q_0, a) = \{q_0, R\}$, $\delta(q_0, b) = \{q_1, R\}$, $\delta(q_1, b) = \{q_1, R\}$,

Example 2.6

Design a 2-way finite automata to accept language $L = \{a^{3n+2}b \mid n \geq 0\}$.

Solution In the previous example, we have not allowed 2-way FA to take any left move. In this example, we will see how the length of input tape in 2-way FA is restricted by taking left moves.

The basic approach to design 2-way FA for language $L = \{a^{3n+2}b \mid n \geq 0\}$ is that initially FA is in start state q_0 , it reads input symbol a to reach in state q_1 by taking a right move, it again reads input a to reach in state q_2 by taking a right move, now on input a it returns back to state q_0 by taking a left move otherwise on input b the FA reaches to final state q_3 . The following (Fig. 2.29) is the transition diagram of the required 2-way FA:

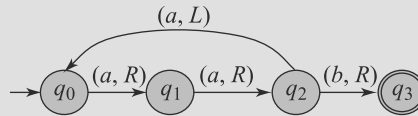


Fig. 2.29 Two way finite automata of Example 2.6

The above 2-way FA can also be represented in the form of transitions as following:

$M = (\{q_0, q_1, q_2, q_3\}, \{a, b\}, \delta, q_0, \{q_3\})$, where δ is defined as:

$$\begin{aligned} \delta(q_0, a) &= \{q_1, R\}, & \delta(q_1, a) &= \{q_2, R\}, \\ \delta(q_2, a) &= \{q_0, L\}, & \delta(q_2, b) &= \{q_3, R\} \end{aligned}$$

If you know the functioning of an automaton then you can write the transition function δ of that automaton (or say machine) vary easily, and if you know the transition function of some automaton then you can understand the functioning of that automaton very easily. For example if an automaton is in some state $q \in Q$ takes some input $a \in \Sigma$ from input tape, as a result automaton goes to some next state $q' \in Q$, and produces some output $o \in \Delta$, then the transition function of the automaton is

$$\delta: Q \times \Sigma \rightarrow Q \times \Delta$$

Conversely, the transition function $\delta: Q \times \Sigma \rightarrow Q \times \Gamma^* \times \{L, R\}$, (where Q , Σ , and $\{L, R\}$ have their usual meanings and Γ is set of writable tape symbol), says that an automaton is in some present state $q \in Q$ takes some input $a \in \Sigma$ from input tape, as a result automaton goes to some next state $q' \in Q$, the read/write head writes a string on the tape and read/write head takes a left or right move.

Theorem 2.3 If L is accepted by a 2-way deterministic finite automaton (2DFA), then L is regular set.

Proof Suppose $M = (Q, \Sigma, \delta, q_0, F)$, is a two-way deterministic finite automaton. Let us construct an NFA M' which accepts language $L(M)$. M' is defined as:

$$M' = (Q', \Sigma, \delta', q'_0, F')$$

where Q' = set of states that consists of all valid crossing sequences for M

q'_0 = crossing sequence consisting of q_0 alone

F' = set of all crossing sequence of length one consisting of a state in F .

$\delta'(c, a) = \{b \mid b \text{ is a valid crossing sequence that is right-matched by } c \text{ on input } a\}$

The intuitive idea is that M' puts together pieces of the computation of M as it reads the input string. This process is done by guessing successive crossing sequences. If M' has guessed that c is the crossing sequence at a boundary, and a is the next input symbol, then M' can guess any valid crossing sequence that c right-matches on input a . If the guessed computation results in M moving off the right end of the input in an accepting state, then M' accepts.

Now we can show that $L(M) = L(M')$. Let $s \in L(M)$. Now each crossing sequence right-machine the one at the next boundary, therefore M' can guess the proper crossing sequence and accept.

In other words, if $s \in L(M')$, consider the crossing sequences $C_0, C_1, \dots, C_n$ of M corresponding to the states of M' , as M' reads $s = a_1 a_2 a_3 \dots a_n$. For each i , $0 \leq i < n$, c_i right-matches c_{i+1} on a_i . Now we can construct an accepting computation of M on input s by determining when the head reverses direction. In particular, we prove that induction on i that M' on reading $a_1 a_2 a_3 \dots a_i$ can enter state $c_i = [q_1, q_2, q_3, \dots, q_k]$ only if:

1. M started in state q_0 (the initial state) on $a_1 a_2 a_3 \dots a_i$ will first move right from position i in state q_1 , and
2. for $j = 2, 4, 6, \dots$, if M is started at position i in state q_j , M will eventually move right from position i in the state q_{j+1} (in the case k will be odd).

Initially, we take $i = 0$. As $c_0 = [q_0]$, then point 1 is satisfied because M begins its computation by 'moving right' from position 0 in state q_0 .

By induction, let us assume that hypothesis is true for $i - 1$. Suppose that M' on scanning $a_1 a_2 a_3 \dots a_i$ can enter in state $c_i = [P_1, P_2, \dots, P_l]$ from state $c_{i-1} = [q_1, q_2, q_3, \dots, q_k]$. Since k and l are odd, and c_{i-1} right-matched c_i on a_i , there should exist an odd j such that, in state q_j on input a_i , M moves right. Suppose j_1 is smallest such j . By definition of 'right-matches' it follows that $\delta(q_{j_1}, a_i) = (P_1, R)$. This proves the point 1. Now if $\delta(q_j, a_i) = (P_{j+1}, R)$ for all even j , then point 2 is followed immediately.

2.13

FA WITH OUTPUT: MOORE AND MEALY MACHINES

Mealy and Moore machines are finite automata with output. Some of the readers may already have met these machines in courses on computer logic or architecture. These machines are commonly used to describe the action of sequential circuits that involve flip-flops and the feedback electronic devices for which the circuit is not only a function of the specific instantaneous inputs, but also a function of the previous state of the system. The history of the input string that can be 'remembered' in a finite automaton is bounded by a function of the number of state in the finite automaton. Sometimes the term transducer is used to call an automaton with input/output because of electronics related characteristics.

Both kinds of machines consist of flip-flop whose inputs are logical combinations of the input signals and the values of flip-flop Q and Q' . Q' is the complement of Q . These machines differ in how the outputs of the machines are determined. For a Moore machine the outputs depend only on the Q and Q' flip-flop outputs. For a Mealy machine, the outputs may depend on both the Q and Q' flip-flop outputs and the current inputs.

Thus the outputs of a Mealy machine can be changed whenever the input changes, even if this occurs between clock pulses. In Fig. 2.30 we see that the output of a Moore machine is associated with the state itself, whereas with a

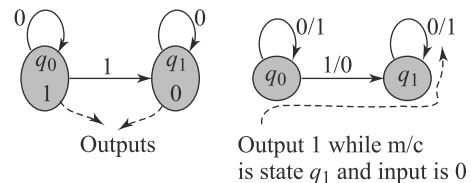


Fig. 2.30 A Moore and a Mealy machine

Mealy machine the output is associated with both the state of machine and the current input value. The architecture of Moore and Mealy machines is shown in block diagrams below (Figs 2.31 and 2.32).

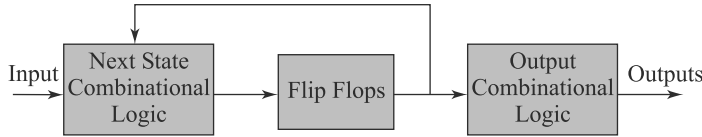


Fig. 2.31 A Moore machine

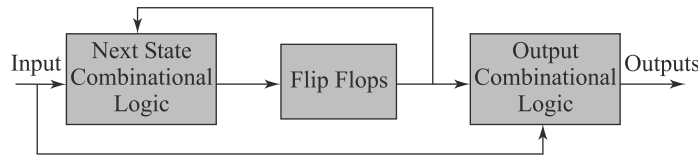


Fig. 2.32 A Mealy machine

In general, if a machine has M states, it will require M flip-flops, where N is the smallest number such that M is less than or equal to 2^N . If the system has D inputs, there are 2^D paths flowing out of each state. If there are k outputs for a Moore machine each state is labelled with k output values, and for a Mealy machine each path is labelled with k outputs.

Mealy and Moore machine models are those models where the outputs can be chosen from some other alphabet. When the value of the output function $O(t)$ is a function of the present state $q(t)$ and the present input $i(t)$, the generalised model is called Mealy machine. The output function may be written as

$$O(t) = \lambda(q(t), i(t))$$

where λ is called output function.

If the output function $O(t)$ depends only on the present state, the restricted model is called Moore machine and the output function may be written as:

$$O(t) = \lambda(q(t))$$

Between these two models, the Moore machine model is more convenient than Mealy machine. A Moore machine with its state table, is shown in Fig. 2.33.

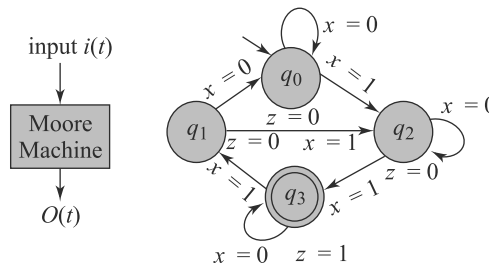
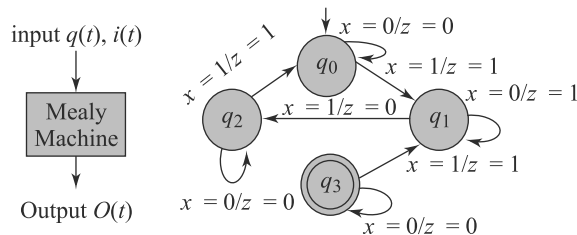


Fig. 2.33 State transition diagram or transition diagram of a Moore machine

Table 2.7 State or transition table of a Moore machine given by Fig. 2.33

PRESENT STATE	NEXT STATE		PRESENT OUTPUT
	$x = 0$	$x = 1$	
$\rightarrow q_0$	q_0	q_1	0
q_1	q_0	q_2	0
q_2	q_2	q_3	0
q_3	q_3	q_1	1

Let us consider a Mealy machine. A Mealy machine with its state table (Table 2.7) is shown in Fig. 2.34:

**Fig. 2.34** Transition diagram of a Mealy machine**Table 2.8** A Mealy machine of Fig. 2.34

PRESENT STATE	NEXT STATE			
	$x = 0$	OUTPUT	$x = 1$	OUTPUT
$\rightarrow q_0$	q_0	0	q_1	1
q_1	q_1	1	q_2	0
q_2	q_2	0	q_0	1
q_3	q_3	0	q_1	1

A state diagram is a convenient method to organise a solution to a problem in preparation for construction of a state machine. The following steps are used to create a state diagram from a problem statement:

- Determine the number of states needed from the problem description.
- Draw a circle representing each state.
- For each state, draw an arrow for each transition-causing event from the present state to next state. For m inputs, there will be 2^m transitions from each state. An input combination that causes the machine to stay in the same state is still a transition and must be included in the diagram.
- If the machine is a Moore machine, write the value of the output in each state.
- If the machine is a Mealy machine, write the output value along with the values of the inputs for each state.

❖ Definitions

Moore Machine The Moore machine is defined as 6-tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where

$Q \rightarrow$ finite non-empty set of states

$\Sigma \rightarrow$ input alphabet

$\Delta \rightarrow$ output alphabet

$\delta, \rightarrow$ transition function which maps $Q \times \Sigma \rightarrow Q$

$\lambda \rightarrow$ output function which maps $Q \rightarrow \Delta$, and

$q_0 \rightarrow$ initial state.

Mealy Machine A Mealy machine is also a 6-tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where all the symbols except λ have the same meaning as in Moore machine. λ is the output function mapping

$Q \times \Sigma \rightarrow \Delta$.

A finite automaton can be converted into a Moore machine by introducing $\Delta = \{0,1\}$ and defining $\lambda(q) = 1$ if $q \in F$ and $\lambda(q) = 0$ if $q \notin F$.

Let us consider the transition table of a Mealy machine (Table 2.9):

Table 2.9 Transition table of Mealy machine

PRESENT STATE	NEXT STATE			
	$x = 0$	OUTPUT	$x = 1$	OUTPUT
$\rightarrow q_1$	q_3	0	q_2	0
q_2	q_1	1	q_4	0
$\odot q_3$	q_2	1	q_1	1
q_4	q_4	0	q_3	0

State $q_1 \rightarrow q_3 \rightarrow q_2 \rightarrow q_4 \rightarrow q_3$

Input 0 0 1 1

Output 0 1 0 0

length of input string = 4

length of output string = 4

For the input string '0011', the transition of states is given by $q_1 \rightarrow q_3 \rightarrow q_2 \rightarrow q_4 \rightarrow q_3$ and the output string is '0100'. Thus, we get an output only on the application of an input symbol. So for the input string $\wedge$, the output is only $\wedge$. It may be observed that in case of Moore machine, we get $\lambda(q_0)$ for the input string $\wedge$. For this now we consider a Moore machine (Table 2.10):

Table 2.10 Transition table of a Moore machine

PRESENT STATE	NEXT STATE		PRESENT OUTPUT
	$x = 0$	$x = 1$	
$\rightarrow q_0$	q_3	q_1	0
q_1	q_1	q_2	1
q_2	q_2	q_3	0
q_3	q_3	q_0	0

For the input string '0111', the transition of states is given by $q_0 \rightarrow q_3 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$. As an output the string '00010' is obtained, and for null string $\wedge$, the output is 0.

Input 0 1 1 1 length of input string = 4)
 States $q_0 \rightarrow q_3 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$
 Output 0 0 0 1 0 (length of output string = 5)

Thus we see that for a Moore machine if the input string is of length n , the output string is of length $n + 1$. But in case of Mealy machine if the length of input string is n , the length of output string is also n .

Example 2.7

Construct the transition and output table for Moore machine given in Fig. 2.35.

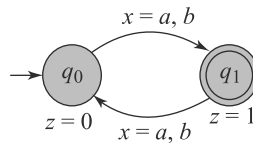


Fig. 2.35 Moore machine of Example 2.7

Solution The above Moore machine can be represented by the following state table (Table 2.11):

Table 2.11 State table of Moore machine given in Fig. 2.33

PRESENT STATE	NEXT STATE		PRESENT OUTPUT
	$x = 0$	$x = 1$	
$\rightarrow q_0$	q_1	q_1	0
q_1	q_0	q_0	1

Example 2.8

Design a Mealy machine that prints 1's complement of an input bits string.

Solution In 1's complement if the input is 0, the output must be 1 and vice-versa. Let us construct a state table (Table 2.12) which could solve our purpose.

Table 2.12 Mealy machine of Example 2.8

PRESENT STATE	NEXT STATE			
	$x = 0$	OUTPUT	$x = 1$	OUTPUT
$\rightarrow q_0$	q_0	1	q_0	0

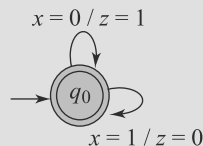


Fig. 2.36 Transition diagram of Table 2.12

Now we can see for input string '0100' we have '1011' which is 1's complement of '0100'.

Example 2.9

Design a Mealy machine that accepts all strings over $\{0,1\}$ not ending with 11 and producing incremented output.

Solution The first step to solve this problem is that we have to design a finite automaton accepting all strings over $\{0,1\}$ not ending with 11. As a second step we have to produce output option that is incremented bit pattern of input. The finite automaton for accepting all strings over $\{0,1\}$ not ending with 11 is given below:

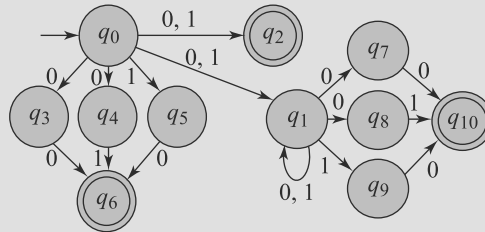


Fig. 2.37 Finite automaton of Example 2.9

As second step, we have to produce the required output in the following manner:

last digits in input :	0	1	00	01	10
output :	1	0	01	10	11

This way the required Mealy machine is:

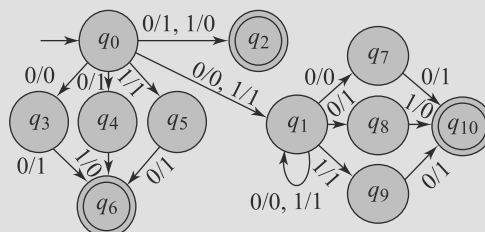


Fig. 2.38 Mealy machine of Example 2.9

Example 2.10

Design a Mealy machine that recognises the double occurrence of symbol 'a' in input string $s \in \Sigma^*$ where $\Sigma = \{a,b\}$.

Solution The required Mealy machine is drawn as shown in Fig. 2.39.

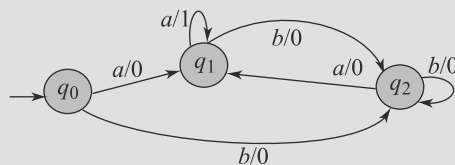


Fig. 2.39 Mealy machine to recognise double occurrences of a's

For some input sequences, the related output sequences are given in Table 2.13:

Table 2.13 Output sequences for some input sequences accepted by Mealy machine of Example 2.10

INPUT SEQUENCE	OUTPUT SEQUENCE
aaa	011
bab	000
abaabb	000100
aababba	0100000

From the Table 2.13, we observe that when we scan the input sequence left to right, we have output bit '1' for 'aa' otherwise '0'.

Example 2.11

Design a Moore machine to accept language $L = \{a^n b^m \mid m, n \geq 1\}$ and produce output in a manner such that first a produces output 0 followed by 1's for additional a 's followed by output 1 for first b followed by 0's for additional b 's, i.e.

input = a a a a a b b b b b
output = 0 1 1 1 1 1 0 0 0 0

Solution The solution of this problem contains two steps. In the first step we construct Mealy machine for this purpose and in the second step we convert this Mealy machine into an equivalent Moore machine.

The conversion from a Mealy machine into Moore machine is described in Section 2.17. The Mealy machine for this purpose is



Fig. 2.40 Mealy machine of Example 2.11

When this Mealy machine is converted into equivalent Moore machine we get

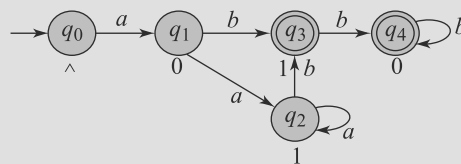


Fig. 2.41 Moore machine of Example 2.11

When we apply input *aaaaabbbb* to the above Moore machine we get output 0111110000.

Example 2.12

Design a Mealy machine that accepts a binary number and produces 2's complement of input bits pattern.

Solution The following Mealy machine accepts all binary numbers and as output the 2's complement value is obtained.

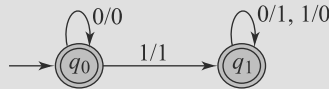


Fig. 2.42 Mealy machine of Example 2.12

The 2's complement of 1101 is 0011, which can also be produced by this Mealy machine.

Input	1	0	1	1
Transitions	$q_0 \rightarrow q_1$	$\rightarrow q_1$	$\rightarrow q_1$	$\rightarrow q_1$
Output	0	0	1	1

This machine reads the input and output from right to left (LSB i.e., least significant bit). Therefore, we treat this as

input = 1101
output = 0011

Example 2.13

Design a Moore machine accepting all strings over {0, 1} producing the counting of substring '001'.

Solution We design the required Moore machine in the manner such that when an occurrence of substring '001' is found the machine gives output XXY to show the single occurrence of substring '001'. Similarly for input 100011001 the machine produces output XXXXYXXX to indicate that the substring '001' occurs two times in the input string. Number of Y's in output shows this. The required Moore machine is given by following diagram:

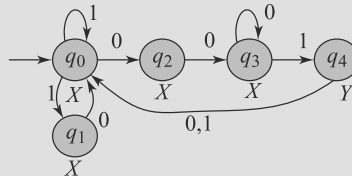


Fig. 2.43 Moore machine of Example 2.13

For input string 100011 001 following transitions take place.

Input	1	0	0	0	1	1	0	0	1
Transitions	$q_0 \rightarrow q_1$	$\rightarrow q_0$	$\rightarrow q_2$	$\rightarrow q_3$	$\rightarrow q_4$	$\rightarrow q_0$	$\rightarrow q_2$	$\rightarrow q_3$	$\rightarrow q_4$
Output	X	X	X	X	X	Y	X	X	Y

and also

Input	1	0	0	0	1	1	0	0	1
Transitions	$q_0 \rightarrow q_0$	$\rightarrow q_2$	$\rightarrow q_3$	$\rightarrow q_3$	$\rightarrow q_4$	$\rightarrow q_0$	$\rightarrow q_2$	$\rightarrow q_3$	$\rightarrow q_4$
Output	X	X	X	X	X	Y	X	X	Y

This means the string 100011001 is accepted by two different paths and output in both cases is XXXXYXXX to show that the substring 001 in input is occurred two times.

Example 2.14

Design a Mealy machine that scans binary strings over $\{0,1\}$, and produces output X for each occurrence of substring 101, produces output Y for each occurrence of substring 110, otherwise produces output Z .

Solution The first we design Moore machine for this purpose and then we will get equivalent Mealy machine. For designing the Moore machine required, we identify the presence of substrings 101 and 110. When substring 101 is identified then following way to decide the output can be adopted:

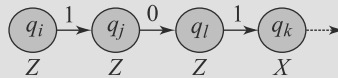


Fig. 2.44 A fragment of a Moore machine

If the substring 110 is identified then following way to decide the output can be adopted:

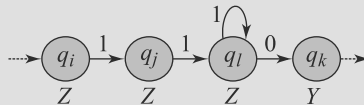


Fig. 2.45 A fragment of a Moore machine

By applying these approaches, the designed Moore machine and its equivalent Mealy machine is given by the following diagrams:

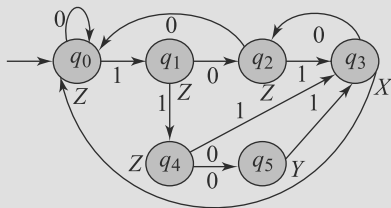


Fig. 2.46 Moore machine of Example 2.14

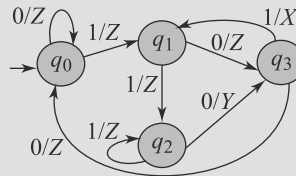


Fig. 2.47 Mealy machine of Example 2.14

❖ Did you Know ?

Edward F. Moore, the creator of Moore machine, was the first person to use the type of finite state machine that is most commonly used today. He, with Claude Shannon, did seminal work on computability theory and designed reliable circuits using less reliable relays.

2.14

FROM FINITE AUTOMATA TO MOORE MACHINES

A finite automaton can be converted into a Moore machine by introducing output alphabet $\Delta = \{0,1\}$ and defining the output function $\lambda(q)$, such that

$$\lambda(q) = \begin{cases} 1 & \text{if } q \in F \\ 0 & \text{if } q \notin F \end{cases}$$

Example 2.15

Consider the finite automaton given by Table 2.14. Convert this finite automaton into a Moore machine.

Table 2.14 Finite automata of Example 2.15

PRESENT STATE	INPUTS	
	$a = 0$	$x = 1$
$\rightarrow q_0$	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

Solution Let $\Delta = \{0,1\}$ be the introduced alphabet. The value of output is defined by output function $\lambda(q)$ as defined in this section.

We have,

$$\lambda(q_0) = 1, \lambda(q_1) = 0$$

$$\lambda(q_2) = 0, \lambda(q_3) = 0$$

Let us construct a Moore machine, with the help of the FA given by Table 2.14, by writing output values calculated corresponding to states of FA. The constructed Moore machine is given by Table 2.15.

Table 2.15 Moore machine constructed from an FA

PRESENT STATE	NEXT STATE		OUTPUT
	$a = 0$	$x = 1$	
$\rightarrow q_0$	q_2	q_1	1
q_1	q_3	q_0	0
q_2	q_0	q_3	0
q_3	q_1	q_2	0

2.15

INTERCONVERSION BETWEEN THE MACHINES

It is sometimes desirable to convert a Mealy machine into a Moore machine and vice versa. In this section we will see the conversion of a Moore machine into a Mealy machine and then from a Mealy machine into a Moore machine.

2.15.1 From Moore Machines to Mealy Machines

The acceptability of input string by a Moore machine is modified by neglecting the response of Moore machine on input Λ , the null string. This way, we define that Mealy machine Me and Moore machine Mo , are equivalent if, for all input strings s , of length n ,

$$PZ_{Me(s)} = Z_{Mo(s)}$$

where

P = the output of Moore machine for its initial state

$Z_{Me(s)}$ = the output string given by Mealy machine on input string s of length n

$Z_{Mo(s)}$ = the output string given by Moore machine on input string s of length $n + 1$.

Suppose $Mo = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$ is a Moore machine. Then the following procedure may be adopted to construct an equivalent Mealy machine Me .

Procedure

Step 1 The output function λ' for Mealy machine is defined as a function of present state and present input by:

$$\lambda'(q, a) = \lambda(\delta(q, a)) \text{ for } q \in Q, a \in \Sigma$$

Step 2 The transition function δ is same as that of the Moore machine.

Example 2.16

Construct a Mealy machine equivalent to the Moore machine given by Table 2.15.

Table 2.16 Moore machine of Example 2.16

PRESENT STATE	NEXT STATE		OUTPUT
	$a = 0$	$a = 1$	
$\rightarrow q_1$	q_4	q_2	0
q_2	q_2	q_3	1
q_3	q_3	q_4	0
q_4	q_4	q_1	0

Solution To obtain an equivalent Mealy machine corresponding to the Moore machine given by Table 2.17, we introduce two output columns corresponding to inputs $a = 0$ and $a = 1$. The value in output columns depends upon the value of output in the Moore machine corresponding to present state. So, what we do is, we assign the output value in a Mealy machine corresponding to present state and present input. We construct a transition table as follows (Table 2.17):

Table 2.17 Mealy machine without assigning output

PRESENT STATE	NEXT STATE	
	$a = 0$	$a = 1$
$\rightarrow q_1$	q_4	q_4
q_2	q_2	q_3
q_3	q_3	q_4
q_4	q_4	q_1

Table 2.16, is the Mealy machine without assigning the output values. We eliminate the ‘output column’ of the Moore machine and insert two output columns in next state column corresponding to $a = 0$ and $a = 1$.

In the table of the Moore machine (Table 2.16) the state $q_1 - q_4$ has the following outputs:

$$\begin{aligned} q_1 &\rightarrow 0 \\ q_2 &\rightarrow 1 \\ q_3 &\rightarrow 0 \\ q_4 &\rightarrow 0. \end{aligned}$$

These output values at state, $q_1 - q_4$ will be written in Table 2.17, corresponding to current states in next state column (the output column). This way we write 1 in front of all occurrences of q_2 , we write 0 in front of all occurrences of state q_1, q_3 and q_4 because output for these states is 0. Thus, the constructed Mealy machine is given by Table 2.18.

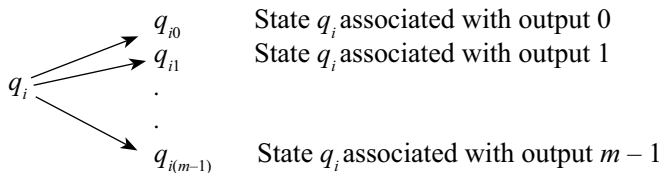
Table 2.18 Mealy machine of Example 2.16

PRESENT STATE	NEXT STATE			
	$a = 0$	OUTPUT	$a = 1$	OUTPUT
$\rightarrow q_1$	q_4	0	q_2	1
q_2	q_2	1	q_3	0
q_3	q_3	0	q_4	0
q_4	q_4	0	q_1	0

2.15.2 From Mealy Machines to Moore Machines

In this section we will transform a Mealy machine into a Moore machine so that for a given input string the output strings are the same (except for the first symbol) in both the machines.

In a Mealy machine, the output depends both upon the present input and present state, while in a Moore machine the output depends upon only the present state. At the first stage we develop a procedure so that both the machines accept the same set of input sequences. We look into the next state column for any state q_i , and determine the number of different output values associated with q_i in that column. We split q_i into m parts if q_i has m outputs, in the following way:



This way, all states are associated with single output and all these states are considered as unique states. This process is repeated for all states having more than one output.

❖ Good to Know!

While constructing a Moore machine equivalent to a Mealy machine, the states of Mealy machine associated with single output (in 'next state' column of Mealy machine) are not split. If a state of Mealy machine q_i is associated with m outputs, then for a split state q_{ik} ($0 \leq k \leq m - 1$), the transitions of present state q_i are followed.

Example 2.17

Consider the mealy machine given by following transition Table 2.19. Construct a Moore machine equivalent to this given Mealy machine.

Table 2.19 Mealy machine of Example 2.17

PRESENT STATE	NEXT STATE			
	$a = 0$	OUTPUT	$a = 1$	OUTPUT
$\rightarrow q_1$	q_3	1	q_2	0
q_2	q_1	1	q_4	1
q_3	q_2	0	q_1	1
q_4	q_4	1	q_3	0

Solution Let us look into the state columns for these states which are associated with more than one output. There is only one state q_3 , which is associated with two outputs 0 and 1. Hence, state q_3 is split into q_{30} and q_{31} associated with output 0 and 1, respectively. Now we will construct a transition table (Table 2.20) by introducing new states q_{30} and q_{31} in place of q_3 , see Table 2.20 :

Table 2.20 Transition table of Mealy machine with newly introduced states

PRESENT STATE	NEXT STATE			
	$a = 0$	OUTPUT	$a = 1$	OUTPUT
$\rightarrow q_1$	q_{31}	1	q_2	0
q_2	q_1	1	q_4	1
q_{31}	q_2	0	q_1	1
q_{30}	q_2	0	q_1	1
q_4	q_4	1	q_{30}	0

Note that, each state q_i of Mealy machine having m output has the property that state q_{ik} in next state column is associated with output k (shown in Table 2.20). The Mealy machine (Table 2.19) has five states q_1, q_2, q_{31}, q_{30} and q_4 . All these states are now associated with a single output only.

Now we transform this table (Table 2.20) into a Moore machine by eliminating outputs within 'next' state column. Here, we introduce a new 'output' column adjacent to Next state column. The values in this 'output' column are corresponding to present state column. For example, for state q_1, q_{31} , and q_4 we write 1 and for states q_2 and q_{30} we write 0 (see Table 2.21):

Table 2.21 Moore machine of Example 2.17

PRESENT STATE	NEXT STATE		OUTPUT
	$a = 0$	$a = 1$	
$\rightarrow q_1$	q_{31}	q_2	1
q_2	q_1	q_4	0
q_{31}	q_2	q_1	1
q_{30}	q_2	q_1	0
q_4	q_4	q_3	1

The transition Table 2.21 shows the constructed Moore machine equivalent to the given Mealy machine. Here we see that the initial state q_1 is associated with output 1. This shows that when we apply input we get an output 1, if the Moore machines starts at state q_1 . Thus, this machine accepts an empty string (zero-length sequence) which is not accepted by the Mealy machine. To overcome this situation, either we must add a new state starting at q_0 or we must neglect the response of a Moore machine to input. If we add a new initial state q_0 then its state transitions will be similar to q_1 but the output will be 0.

Therefore the modified transition table of the constructed Moore machine is given in Table 2.22:

Table 2.22 Moore machine of Example 2.17

PRESENT STATE	NEXT STATE		OUTPUT
	$a = 0$	$a = 1$	
$\rightarrow q_0$	q_{31}	q_2	0
q_1	q_{31}	q_2	1
q_2	q_1	q_4	0
q_{31}	q_2	q_1	1
q_{30}	q_2	q_1	0
q_4	q_4	q_3	1

2.16

EQUIVALENCE BETWEEN MOORE AND MEALY MACHINES

A Moore machine can never be correctly equivalent to a Mealy machine because the length of output string from a Moore machine is one larger than that from a Mealy machine by giving the same input. In other words, we can say that for a Moore machine if the input string is of length n , the output string is of length $n + 1$. The first output is $\lambda(q_0)$ for all output strings, as the machine is in initial state q_0 . But in case of Mealy machine if the input string is length n , the output string is also of the same length n .

A Mealy machine is equivalent to a Moore machine in the sense both can accept the same language or same set of strings. They are not equivalent in the sense the outputs are different for same input, as we know the length of outputs are different.

2.17

MINIMISATION OF FA

It is always desirable to use the least amount of computer memory whether it is length of code or programming data (stored in variables). If we wish to simulate a particular DFA by using some programming language, we always try to deal with the DFA with minimum number of states.

Two finite automata M and M' are said to be equivalent if and only if both accept the same language or the same set of strings. If automaton M contains n_1 states and there exists another automaton M' equivalent to M containing n_2 states with $n_2 \leq n_1$ then it is desirable to use M' .

In this section we will construct a DFA M' with minimum number of states equivalent to given DFA M . Minimisation (or say optimisation) of DFA refers to detecting those states of given DFA whose presence or absence does not affect the language accepted by the automaton. Hence, these states can be eliminated from the automaton to save the memory and to ensure faster execution. The states that can be eliminated are:

Unreachable States These states are those states of a DFA, which are not reachable from the initial state of DFA on any possible input. The unreachable or non-reachable states are useless states in an automaton and these should be eliminated immediately.

Dead States These states are those states of a DFA whose transitions on every input symbol terminate on itself.

The dead states in a finite automaton are useless states and these should be eliminated immediately. For example q_i is a dead state if $\delta(q_i, a) = q_i$ for $q_i \in Q$ and $a \in \Sigma$.

Non-distinguishable States These states are those states of a DFA for which there exist no distinguishing string. Hence, these states cannot be distinguished from one another.

2.17.1 Method I (Equivalence Class): Construction of Minimum State DFA

Input: Given FA $M = (Q, \Sigma, \delta, q_0, F)$

Output: Minimise FA $M' = (Q', \Sigma, \delta', q'_0, F')$

In this section we will construct a finite automaton M' with minimum number of state equivalent to given finite automaton M . Two states q and q' are equivalent if both are final states or both are non-final states for all input strings $w \in \Sigma^*$. Now we will determine k -equivalent of two states.

Two states q and q' are k -equivalent ($k \geq 0$) if both $\delta(q, x)$ and $\delta(q', x)$ are final state or both are non-final states for all string x having length k or less ($|x| \leq k$). By default, any two final states are 0-equivalent and also any two non-final states are 0-equivalent.

Now we define some properties for these relations:

Property 1 The defined relations are equivalence relations. They are reflexive symmetric and transitive.

Property 2 Any equivalence relation R on a set S performs partitions S into disjoint equivalence classes. This induces the partition of Q . Partitions are denoted by π and π_k . All elements of π_k are k -equivalence classes.

Property 3 If any two states q_1 and q_2 are k -equivalent for all $k \geq 0$, then they are equivalent.

Property 4 If q_1 and q_2 are $(k + 1)$ -equivalent, then are also k -equivalent.

Property 5 $\pi_n = \pi_{n+1}$ for some n .

Thus, two states q_1 and q_2 are $(k + 1)$ -equivalent if:

- (i) they are k -equivalent
- (ii) $\delta(q_1, x)$ and $\delta(q_2, x)$ are also k -equivalent for every $x \in \Sigma$.

Construction of Minimum State Finite Automaton

The following are the four steps:

(i) **Construction of π_0** By using the definition of 0-equivalence $= (Q_1^0, Q_2^0)$, where

Q_1^0 = set of all final states, and

Q_2^0 = set of all non-final states

π_0 = {set of all states}

$Q = Q_1^0 \cup Q_2^0$

So we have

$Q_2^0 = Q - Q_1^0$

By the help of π_0 we will construct π_1 , and by the help of π_1 we will construct π_2 , and so on.

(ii) **Construction of π_{k+1} from π_k** Suppose Q_j^k is any subset in π_k . If q_1 and q_2 are in Q_j^k , they are $(k + 1)$ -equivalent because $\delta(q_1, x)$ and $\delta(q_2, x)$ are k -equivalent. Determine whether $\delta(q_1, x)$ and $\delta(q_2, x)$ are in same equivalence class of π_k for every $x \in \Sigma$. Then q_1 and q_2 will be $(k + 1)$ -equivalent classes. Repeat this process for every Q_j^k to find all elements of π_{k+1} .

(iii) Now, we construct π_n for $n = 1, 2, 3, 4, \dots$ until π_n and π_{n+1} have same elements.

(iv) **Construction of minimum state automaton** The states obtained in step (iii) are equivalence classes. Now we obtain the transition table by replacing a state q by the corresponding equivalence classes, i.e., $[q]$.

Example 2.18

Construct the minimum state automata equivalent to the state given in the following Table 2.23 by Method I:

Table 2.23 A finite automaton of Example 2.18

PRESENT STATE	NEXT STATE	
	a	b
$\rightarrow q_1$	q_2	q_1
q_2	q_1	q_3
q_3	q_4	q_2
q_4	q_4	q_1
q_5	q_4	q_6
q_6	q_7	q_5
q_7	q_6	q_7
q_8	q_7	q_4

Solution Since there is only one final state q_4 ,

$Q_1^0 = q_4$ and $Q_2^0 = Q - Q_1^0$. Therefore,

$\pi_0 = [\{q_4\}, \{q_1, q_2, q_3, q_5, q_6, q_7, q_8\}]$. As q_4 is single element so it cannot be partitioned further,

$Q_1^1 = \{q_4\}$

q_1 is 1-equivalent to $\{q_1, q_2, q_6, q_7\}$, but not to $\{q_3, q_5, q_8\}$

So $Q_1^2 = \{q_1, q_2, q_6, q_7\}$

q_3 is 1-equivalent to q_5 , therefore, $Q_1^3 = \{q_3, q_5\}$

The only element left in is q_8

So $Q_1^4 = q_8$, thus

$\pi_1 = [\{q_4\}, \{q_1, q_2, q_6, q_7\}, \{q_3, q_5\}, \{q_8\}]$

q_1 is 2-equivalent to q_7 but not to q_2 or q_6 .

we have

$Q_2^2 = \{q_1, q_7\}$

As q_2 is 2-equivalent to q_6

$Q_3^2 = \{q_2, q_6\}$

As q_3 is 2-equivalent to q_5 , so

$Q_4^2 = \{q_3, q_5\}$, $Q_5^2 = \{q_8\}$

Therefore

$\pi_2 = [\{q_4\}, \{q_1, q_7\}, \{q_2, q_6\}, \{q_3, q_5\}, \{q_8\}]$

$Q_4^3 = \{q_4\}$

As q_1 is 3-equivalent to q_7 , so $Q_2^3 = \{q_1, q_7\}$

As q_2 is 3-equivalent to q_6 , so $Q_3^3 = \{q_2, q_6\}$

As q_3 is 3-equivalent to q_5 , so $Q_4^3 = \{q_3, q_5\}$

and $Q_5^3 = \{q_8\}$

Therefore

$\pi_3 = [\{q_4\}, \{q_1, q_7\}, \{q_2, q_6\}, \{q_3, q_5\}, \{q_8\}]$

Now we see that $\pi_3 = \pi_2$. π gives the equivalence classes. So the minimum state automaton M' is given is

$M' = (Q', \{a, b\}, \delta', q'_1, F')$

$Q' = [\{q_4\}, \{q_1, q_7\}, \{q_2, q_6\}, \{q_3, q_5\}, \{q_8\}]$

$q'_1 = [q_1, q_7]$ (initial state)

$F' = [q_4]$ (final state)

δ' is given by following transition table:

Table 2.24 Minimum state finite automaton of Example 2.18

PRESENT STATE	NEXT STATE	
	<i>a</i>	<i>b</i>
$\rightarrow[q_1, q_7]$	$[q_2, q_6]$	$[q_1, q_7]$
$[q_2, q_6]$	$[q_1, q_7]$	$[q_3, q_5]$
$[q_3, q_5]$	$[q_4]$	$[q_2, q_6]$
$\odot[q_4]$	$[q_4]$	$[q_1, q_7]$
$[q_8]$	$[q_1, q_7]$	$[q_4]$

As the state $[q_8]$ is an unreachable state from the initial state, therefore this state can be eliminated from Q' . Thus, the optimised finite automaton is given by Table 2.24.

Table 2.25 Minimum state finite automaton of Example 2.18

PRESENT STATE	NEXT STATE	
	<i>a</i>	<i>b</i>
$\rightarrow[q_1, q_7]$	$[q_2, q_6]$	$[q_1, q_7]$
$[q_2, q_6]$	$[q_1, q_7]$	$[q_3, q_5]$
$[q_3, q_5]$	$[q_4]$	$[q_2, q_6]$
$\odot[q_4]$	$[q_4]$	$[q_1, q_7]$

2.17.2 Method II (Subset Construction): Construction of Minimum State DFA

Input: Given FA $M = (Q, \Sigma, \delta, q_0, F)$

Output: Minimise FA $M' = (Q', \Sigma, \delta', q'_0, F')$

The states of given DFA can be minimised by detecting unreachable, dead, and non-distinguishable states. Initially, we have two classes of states from Q , a class of final states say class 1, and a class of non-final states say class 2.

We make partitions as subclasses class 1 and class 2 corresponding to distinguishable states on input $a \in \Sigma$. For example,

$$\delta(q_i, a) = q_j \text{ for some } i, j = 1, 2, 3, \dots$$

If next states q_j for some j 's belong to different classes then states are distinguished and can be partitioned further. This process of partitioning based on distinguishing the states is repeated till the possibility of partitioning is possible. Total numbers of classes of distinguished states are considered as number of states in minimised or optimised DFA. For example, if the given DFA is

$$M = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}, \Sigma, \delta, q_0, \{q_6\})$$

and after the partitioning the classes of distinguished states are $\{q_0, q_2\}$, $\{q_1, q_3, q_4\}$, $\{q_5\}$, $\{q_6\}$ then the minimised DFA M' will have only four states including $\{q_0, q_2\}$ as initial state and $\{q_6\}$ as final state. The transition, from state $\{q_1, q_3, q_4\}$ on input a is defined as

$$\delta(\{q_1, q_3, q_4\}, a) = \{ \delta(q_1, a) \cup \delta(q_3, a) \cup \delta(q_4, a) \}$$

Example 2.19

Construct the minimum state DFA equivalent to the DFA given by Table 2.22 by method II.

Solution Since there is only one final state q_4 , therefore initially two classes of final and non-final states are

$$\underbrace{\{q_4\}}_{\text{class 1}} \quad \underbrace{\{q_1, q_2, q_3, q_5, q_6, q_7, q_8\}}_{\text{class 2}}$$

As the class 1 has only one element (i.e., q_4) the transition from q_4 on input ' a ' is $\delta(q_4, a) = q_4$ i.e., the next states (here only q_4) belong to only class 1 so it can not be partitioned further. This is also supported by following facts:

The transition of q_4 on input b is $\delta(q_4, b) = q_1$ i.e., the next states (here only q_1) belong to only class 2, so it cannot be partitioned further. Also q_4 is a single state in class 1, therefore, a single state (here q_4) cannot be partitioned further on any basis.

Let us find distinguishable classes from class 2 for every input $x \in \Sigma$. We have following transitions on input a when each state from class 2 is a present state:

$$\begin{aligned} \delta(q_1, a) &= q_2 \\ \delta(q_2, a) &= q_1 \\ \delta(q_3, a) &= q_4 \\ \delta(q_5, a) &= q_4 \\ \delta(q_6, a) &= q_7 \\ \delta(q_7, a) &= q_6 \\ \delta(q_8, a) &= q_7 \end{aligned}$$

As the transitions $\delta(q_3, a) = q_4$ and $\delta(q_5, a) = q_4$ belong to class 1 and remaining belong to class 2, therefore the elements of class 2 can be partitioned further into two subclasses say class 3 and class 4. Now we have following of states

$$\underbrace{\{q_4\}}_{\text{class 1}} \quad \underbrace{\{q_3, q_5\}}_{\text{class 3}} \quad \underbrace{\{q_1, q_2, q_6, q_7, q_8\}}_{\text{class 4}}$$

Now we will find distinguishable classes of states again from class 2 on input b . We have following transitions:

$$\begin{aligned} \delta(q_1, b) &= q_1 \\ \delta(q_2, b) &= q_3 \\ \delta(q_3, b) &= q_2 \end{aligned}$$

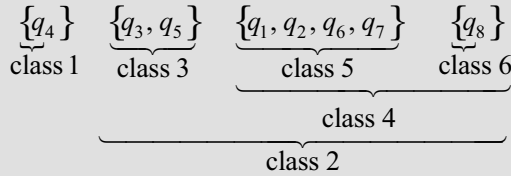
$$\delta(q_5, b) = q_6$$

$$\delta(q_6, b) = q_5$$

$$\delta(q_7, b) = q_7$$

$$\delta(q_8, b) = q_4$$

As the transitions $\delta(q_1, b) = q_1$, $\delta(q_3, b) = q_2$, $\delta(q_5, b) = q_6$ and $\delta(q_7, b) = q_7$ belong to class 4, the transitions $\delta(q_2, b) = q_3$, $\delta(q_6, b) = q_5$ belong to class 3, and the transition $\delta(q_8, b) = q_4$ belongs to class 1, therefore $\{q_3, q_5\}$, $\{q_1, q_2, q_6, q_7\}$ and $\{q_8\}$ are distinguishable in class 2 on input b , as $\{q_3, q_5\}$ is already in different class (i.e., class 3), therefore class 4 can be partitioned further into $\{q_1, q_2, q_6, q_7\}$ and $\{q_8\}$ denoted by class 5 and class 6 respectively. Now we have following classes of states:



Now we try to make partitions further in class 3 by finding distinguishable classes of states. We have flowing transition on input a :

$$\delta(q_3, a) = q_4$$

$$\delta(q_5, a) = q_4$$

As these transition belong to only a single class (i.e. class 1), therefore class 3 cannot be partitioned further. On input b the possible transitions from class 3 are:

$$\delta(q_3, b) = q_2$$

$$\delta(q_5, b) = q_6$$

As these transitions belong to only a single class (i.e., class 5), therefore class 3 cannot be partitioned further. Now we try to make partitions further in class 5 by finding distinguishable classes of states. We have the following transitions on input a :

$$\delta(q_1, a) = q_2$$

$$\delta(q_2, a) = q_1$$

$$\delta(q_6, a) = q_7$$

$$\delta(q_7, a) = q_6$$

As these transitions belong to only a single class (i.e. class 5), therefore class 5 cannot be partitioned further on the basis of input a . For input b , the states of class 5 have following transitions:

$$\delta(q_1, b) = q_1$$

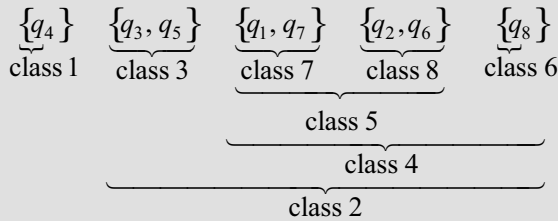
$$\delta(q_2, b) = q_3$$

$$\delta(q_6, b) = q_5$$

$$\delta(q_7, b) = q_7$$

The states q_1 and q_7 distinguishable from states q_2 and q_6 because the transitions $\delta(q_1, b) = q_1$ and $\delta(q_7, b) = q_7$ belong to class 5, while the transitions $\delta(q_2, b) = q_3$ and $\delta(q_6, b) = q_5$ belong to class 3.

Therefore, states of classes 5 can be partitioned further into $\{q_1, q_7\}$ and $\{q_2, q_6\}$ denoted by class 7 and class 8 respectively. Now, we have the following classes of states:



Now we can see that next states of class 7, class 8 and class 6 belong to only a single class on input a and b . Therefore $\{q_1, q_7\}$, $\{q_2, q_6\}$ and $\{q_8\}$ cannot be partitioned further.

Thus, the minimum state DFA M' has only five states with $\{q_1, q_7\}$ as initial state and $\{q_4\}$ as final state. The minimum DFA M' can be defined as

$$M' = (\{q_1, q_7\}, \{q_2, q_6\}, \{q_3, q_5\}, \{q_4\}, \{q_8\}, \{a, b\}, \delta', \{q_1, q_7\}, \{q_4\})$$

where δ' is defined as

$$\delta'(\{q_1, q_7\}, a) = \delta(q_1, a) \cup \delta(q_7, a) = \{q_2 \cup q_6\} = \{q_2, q_6\}$$

$$\delta'(\{q_1, q_7\}, b) = \delta(q_1, b) \cup \delta(q_7, b) = \{q_1 \cup q_7\} = \{q_1, q_7\}$$

$$\delta'(\{q_2, q_6\}, a) = \delta(q_2, a) \cup \delta(q_6, a) = \{q_1 \cup q_7\} = \{q_1, q_7\}$$

$$\delta'(\{q_2, q_6\}, b) = \delta(q_2, b) \cup \delta(q_6, b) = \{q_3 \cup q_5\} = \{q_3, q_5\}$$

$$\delta'(\{q_3, q_5\}, a) = \delta(q_3, a) \cup \delta(q_5, a) = \{q_4\}$$

$$\delta'(\{q_3, q_5\}, b) = \delta(q_3, b) \cup \delta(q_5, b) = \{q_2 \cup q_6\} = \{q_2, q_6\}$$

$$\delta'(\{q_4\}, a) = \delta(q_4, a) = \{q_4\}$$

$$\delta'(\{q_4\}, b) = \delta(q_4, b) = \{q_1, q_7\} \text{ (As there is only state } \{q_1, q_7\} \text{ having } q_1 \text{ in } M')$$

$$\delta'(\{q_8\}, a) = \delta(q_8, a) = \{q_1, q_7\} \text{ (As there is only state } \{q_1, q_7\} \text{ having } q_7 \text{ in } M')$$

$$\delta'(\{q_8\}, b) = \delta(q_8, b) = \{q_4\}$$

For simplicity let us assume that

$$q_A = \{q_1, q_7\}, \quad q_B = \{q_2, q_6\}, \quad q_C = \{q_3, q_5\}, \quad q_D = \{q_4\}, \text{ and } q_E = \{q_8\}$$

The transition table of minimum state DFA is given below:

Table 2.26 Transition table of minimised DFA M' of Example 2.19

PRESENT STATE	NEXT STATE	
	a	b
$\rightarrow q_A$	q_B	q_A
q_B	q_A	q_C
q_C	q_D	q_B
$\odot q_D$	q_D	q_A
q_E	q_A	q_D

In Table 2.25 state q_E is unreachable state therefore the state q_E can be eliminated from Q' and its transitions can be eliminated from δ' , thus the minimum state DFA M' is defined as:

$$M' = (\{q_A, q_B, q_C, q_D, q_E\}, \{a, b\}, \delta', q_A, \{q_D\})$$

where δ' is defined by the following transition table:

Table 2.27 Final transition table of minimised DFA M' of Example 2.19

PRESENT STATE	NEXT STATE	
	a	b
$\rightarrow q_A$	q_B	q_A
q_B	q_A	q_C
q_C	q_D	q_B
q_D	q_D	q_A

We can see that the minimum state DFA M' given by Tables 2.25 and 2.27 are equivalent.

Example 2.20

Minimise the finite automaton (with all states as final states) given by following transitions diagram:

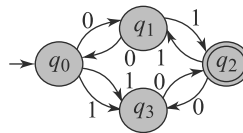


Fig. 2.48 DFA with four states

Solution The finite automaton in this example accepts all strings over $(0, 1)$. The required minimised finite automaton is



Fig. 2.49 Minimised DFA with one state

On application of minimisation procedure q is obtained as $[q_0, q_1, q_2, q_3]$ single combined state.

2.18

PROPERTIES OF TRANSITION FUNCTIONS (δ)

Property 1 $\delta(q_i, \wedge) = q_i$ exists in a finite automaton. It means that the state of the transition system can be changed only by giving an input. Without input (or say input $\wedge$), the system remains in the same state. In other words, $\wedge$ is considered as zero length string.

Property 2 For all strings s and input symbol a ,

$$\delta(q_i, as) = \delta(\delta(q_i, a), s)$$

$$\delta(q_i, sa) = \delta(\delta(q_i, s), a)$$

Second property gives the state after the automaton reads the first symbol of string 'as' and the state after the automaton reads a prefix 's' of the string 'sa'.

Theorem 2.4 If $M = (Q, \Sigma, \delta, q_0, F)$, then for any transition function δ , there exists

$$\delta(q_i, s_1 s_2) = \delta(\delta(q_i, s_1), s_2)$$

for any two strings $s_1, s_2 \in \Sigma^*$, and $q_i \in Q$.

Proof We apply mathematical induction on length of string s_2 i.e., $|s_2|$. As basis of induction, we have $|s_2| = 1$ for $s_2 = x \in \Sigma$.

Let us take L.H.S. of $\delta(q_i, s_1 s_2) = \delta(\delta(q_i, s_1), s_2)$ with $s_2 = x$. We have,

$$\begin{aligned} \text{L.H.S.} &= \delta(q_i, s_1 tx) \\ &= \delta(\delta(q_i, s_1), x) \quad [\text{by using Property 2}] \\ &= \text{R.H.S.} \end{aligned}$$

Let the result of $\delta(q_i, s_1 s_2) = \delta(\delta(q_i, s_1), s_2)$ for all strings s_1 and s_2 with $|s_2| = n$. Assume s_2 is a string of length $n+1$. Now we can write $s_2 = tx$, where $|t| = n$.

Let us take L.H.S. of $\delta(q_i, s_1 s_2) = \delta(\delta(q_i, s_1), s_2)$ with $s_2 = tx$. We have,

$$\begin{aligned} \text{L.H.S.} &= \delta(q_i, s_1 tx) \\ &= \delta(\delta(q_i, s_1 t), x) \quad [\text{by using Property 2}] \\ &= \delta(\delta(\delta(q_i, s_1), t), x) \quad [\text{by induction hypothesis}] \\ &= \text{R.H.S.} \end{aligned}$$

Now we take R.H.S. of $\delta(q_i, s_1 s_2) = \delta(\delta(q_i, s_1), s_2)$ with $s_2 = tx$, where $|t| = n$.

$$\begin{aligned} \text{R.H.S.} &= \delta(\delta(q_i, s_1), tx) \\ &= \delta(\delta(\delta(q_i, s_1), t), x) \quad [\text{by using Property 2}] \end{aligned}$$

Thus, we have L.H.S. = R.H.S. This proves $\delta(q_i, s_1 s_2) = \delta(\delta(q_i, s_1), s_2)$ for any string s_2 of length $n+1$. We have $\delta(q_i, s_1 s_2) = \delta(\delta(q_i, s_1), s_2)$ true for all strings by the principle of mathematical induction.

2.19

EXTENDED TRANSITION FUNCTIONS TO STRINGS

When transition function of finite automaton is defined as

$$Q \times \Sigma^* \text{ into } Q$$

then Σ^* is string rather than a single symbol. In this case transition function is denoted by δ^* called extended transition function.

For an NFA M , $\delta^*(q, x)$ is to be set of states in which NFA M can be, after scanning input string x in state q .

Note that δ^* is read as delta star. Let us consider an example, if an NFA M is in state q_1 it scans input a and goes to next state q_2 then

$$\delta^*(q_1, abb) = \delta^*(q_2, bb)$$

$$\delta^*(q_1, abb) = \delta^*(\delta(q_1, a), bb)$$

where abb is any string in Σ^* .

Here,

$$\delta(q_1, a) = q_2$$

Nonrecursive Definition of δ^* for an NDFA

For NDFA $M = (Q, \Sigma, \delta, q_0, F)$ and for any $P \in Q$, $\delta^*(P, \wedge) = P$. For any $P \in Q$ and for any $x = a_1 a_2 a_3 \dots a_n \in \Sigma^*$, $\delta^*(P, x)$ is the set of all states q for which there is a sequence of states $P = P_0, P_1, P_2, \dots, P_n = q$ satisfying.

$$P_i \in \delta(P_{i-1}, a_i) \text{ for each } i, 1 \leq i \leq n$$

Recursive Definition of δ^* for an NDFA

Suppose $M = (Q, \Sigma, \delta, q_0, F)$ is the NDFA. The function $\delta: Q \times \Sigma \rightarrow 2^Q$ is defined as:

- (i) for any $q \in Q$, $\delta^*(q, \wedge) = q$
- (ii) for any $y \in \Sigma^*$, $a \in \Sigma$ and $q \in Q$

$$\delta^*(q, ya) = \bigcup_{P \in \delta^*(q, y)} \delta(P, a)$$

Example 2.21

Consider the following finite automaton. Determine the state described by:

- (i) $\delta^*(q_0, abc)$
- (ii) $\delta^*(q_1, bcabb)$

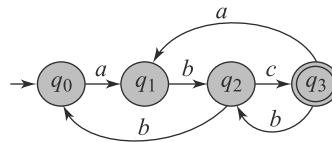


Fig. 2.50 Finite automaton of Example 2.21

Solution We know that $\delta^*(q, \wedge) = q$. By definition (recursive)

$$\begin{aligned} \delta^*(q, xyz) &= \delta(\delta^*(q, x), yz) \\ &= \delta(\delta(\delta^*(q, x), y), z) \end{aligned}$$

- (i) $\delta^*(q_0, abc) = \delta(\delta^*(q_0, a), bc)$
 $= \delta(\delta(\delta^*(q_0, a), b), c)$
 $= \delta(\delta(\delta^*(q_0, \wedge), a), b), c)$
 $= \delta(\delta(\delta(q_0, \wedge), a), b), c)$
 $= \delta(\delta(q_1, b), c)$

$$= \delta(q_2, c)$$

$$= q_3$$

Hence, $\delta^*(q_0, abc) = q_3$. Note that the underlined q_0 is equal to $\delta^*(q_0, \wedge)$.

$$\begin{aligned} \text{(ii) } \delta^*(q_1, bcabb) &= \delta(\delta^*(q_1, b), cabb) \\ &= \delta(\delta(\delta^*(q_1, b), c), abb) \\ &= \delta(\delta(\delta(\delta^*(q_1, b), c), a), bb) \\ &= \delta(\delta(\delta(\delta(\delta^*(q_1, b), c), a), b), b) \\ &= \delta(\delta(\delta(\delta(\delta^*(q_1, \wedge), b), c), a), b), b) \\ &= \delta(\delta(\delta(\delta(q_1, b), c), a), b), b) \\ &= \delta(\delta(\delta(q_2, c), a), b), b) \\ &= \delta(\delta(q_3, a), b), b) \\ &= \delta(q_1, b), b) \\ &= \delta(q_2, b) \\ &= q_0 \end{aligned}$$

Thus, $\delta^*(q_1, bcabb) = q_0$

❖ Good to Know!

In NDFA, we may have more than one next states on one input. Let us consider part (ii) of Example 2.2, when current state is q_0 by taking input 1 the NDFA has q_1 and q_2 as next states, so we have written this transition as $\delta(q_0, 1) = \{q_1, q_2\}$. Similarly, NDFA goes to state $\{q_0, q_1\}$ from $\{q_1, q_2\}$ by taking input 1. In this case we have used transition from state q_1 and q_2 for input 1, and the next states q_0 and q_1 are combined (or say union) as $\{q_0, q_1\}$.

Extended Transition Function of NFA

The extended transition function of NFA is defined as

$$Q \times \Sigma^* \rightarrow 2^Q$$

After the processing of string from Σ^* , there may be zero, one or more next states.

Example 2.22

Consider the following transition table of an NDFA

Table 2.28 Transition table of Example 2.22

PRESENT STATE	NEXT STATE	
	0	1
$\rightarrow q_0$	$\{q_1\}$	$\{q_1, q_2\}$
q_1	$\{q_2\}$	$\{q_0\}$
q_2	$\{q_2\}$	$\{q_1\}$

Determine the state described by

$$\text{(i) } \delta^*(q_1, 010), \quad \text{(ii) } \delta^*(q_0, 111)$$

Solution

$$\begin{aligned}
\text{(i)} \quad \delta^*(q_1, 010) &= \delta(\delta^*(q_1, 0), 10) \\
&= \delta(\delta(\delta^*(q_1, 0), 1), 0) \\
&= \delta(\delta(\delta(\delta^*(q_1, \wedge), 0), 1), 0) \\
&= \delta(\delta(\delta(q_1, 0), 1), 0) \\
&= \delta(\delta(\{q_2\}, 1), 0) \\
&= \delta(\{q_1\}, 0) \\
&= \{q_2\} \\
\text{(ii)} \quad \delta^*(q_0, 111) &= \delta(\delta^*(q_0, 1), 11) \\
&= \delta(\delta(\delta^*(q_0, 1), 1), 1) \\
&= \delta(\delta(\delta(\delta^*(q_0, \wedge), 1), 1), 1) \\
&= \delta(\delta(\delta(q_0, 1), 1), 1) \\
&= \delta(\delta(\{q_1, q_2\}, 1), 1) \\
&= \delta(\delta(\{q_0, q_2\}, 1) \\
&= \{q_0, q_1, q_2\} \\
\text{Thus, } \delta^*(q_0, 111) &= \{q_0, q_1, q_2\}
\end{aligned}$$

2.20**APPLICATIONS OF FINITE AUTOMATA**

There are several software design problems that are simplified by converting into regular expression representation (see chapter 4) to corresponding finite automata. Some of these are:

- Lexical Analyser
- Switching Circuit Design
- Word Processor Programmes
- Text Editors, *etc.*

Lexical analyser is a phase in compiler design; its function is called lexical analysis. The lexical analyser scans the source program and breaks it into primitive parts called tokens. These tokens are transformed to syntax analyser (the next phase of compiler).

Problems involving searches on the web and other on-line textual information base, require finding all the documents containing some or all of a given set of words. We call such words in a given set as keywords. The approach used for searching documents for keyword mainly use either automata or inverted indexes for the purpose. The search engines generally use inverted indexes. However, the search based on automata is more suitable than inverted indexes in the type of information base in which -

- most of the information contents change frequently as happen in the case of daily on line news items, and
- the document being searched are not, and even cannot be indexed which makes searches based on inverted indexes difficult.

In these situations, the task is performed using FA, through the following steps:

- (i) First of all an NFA is designed that accepts and signals the acceptance of the keywords.
- (ii) As the NFA is not a form of programme, it is implemented by the following approaches:

- (a) A DFA equivalent to NFA is constructed.
- (b) A programming implementation is done that simulates the NFA.
- (iii) Writing a program by simulating a DFA is obtained in (ii) step. A combination of two approaches is actually used in the text processing programs like 'egrep' and 'fgrep' which are advanced forms of Unix grep program.

Example 2.23

Design an FA to recognise the string 'cat' and 'rat'.

Solution The following diagrams represent on FA that recognises the strings 'cat' and 'rat'

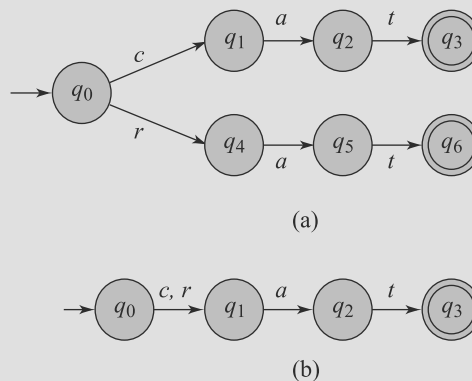


Fig. 2.51 The DFA to accept strings 'cat' and 'rat'

In programming implementation we have used the second diagram having four states. The C++ implementation of this FA is given below:

```

/*Programme to implement FSM that accepts string rat or cat */
#include<iostream.h>
#include<stdio.h>
#include<conio.h>
int s = 0;
int i, n;
char a[5];
void state0();
void state1();
void state2();
void state3();
void main (){
    clrscr ();
    count <<"enter the characters in a string:";
    cin >>n ;
    for (int j = 0 ; j<n ; j++)
        for (int i = 0 ; i<n ; i++)
        {
            cin>>a [i] ;

```

```

if (s == 0){
    state0( ) ;
    break ;
}
else if (s == 2) {
    state2( ) ;
    break ;
}
else if (s == 2) {
    state2( ) ;
    break ;
}
else if (s == 3) {
    state3( ) ;
    break ;
}
}
{
if (s == 3){
    cout <<"\nstring accepted !!";
}
else {
    cout <<"\nstring not accepted !!";
}
}
getch ( ) ;
}
void state0() {
    if ((a [i] == 'r') (a [i] == 'c' )){
        s = 1;
    }
    else if ((a [i] != 'r' (a [i] != 'c'))
    {
        s = 0;
    }
}
void state1() {
    if (a [i] == 'a' ) {
        s = 2;
    }
    else if ((a [i] == 'r' (a [i]== 'c')) {
        s = 1;
    }
}
void state2() {
    if (a [i] == 't' ) {
        s = 3;
    }
    else if (a [i] != 't' ) {
        s = 0;
    }
}

```

```

}
void state3() {
if (a [i] == ' ')
s = 3;
}
}
// output 1
//enter the characters in a string: 3
//rat
// string accepted !!
//output2
//enter the characters in a string: 3
//mat
//string not accepted !!

```

2.21

LIMITATIONS OF FINITE STATE MACHINES

In this chapter we have seen that finite state machines are very simple. As a consequence, there are limitations to what they can do. For example, it is not possible to design a finite state machine that accepts the language $a^n b^n$, i.e. the set of all strings which consist of a (possibly empty) block of a 's followed by a (possibly empty) block of b 's of exactly the same length.

Basically the language of the form $a^n b^n$ or palindromes is a context free language, which cannot be recognised by a simple computation model like FA. For such types of languages we need a machine more powerful than FA like pushdown automata. FA have certain expressive weaknesses. This also limits their expressive adequacy from the point of view of linguistics, because several linguistic phenomena can only be described by language which cannot be generated by FA. In fact, even the language $a^n b^n$ is linguistically relevant, since several linguistic constructions require 'balanced' structure. Additionally, a finite state machine cannot remember arbitrarily a large amount of information.

Furthermore, Hidden Markov models, which are very common for speech recognition and part of speech tagging, can be seen as a variant of finite state machines, which assign probabilities to their transitions. Thus, finite state machines have their limitations for NLP (Natural Language Processing) purposes.

Summary

- **Automaton** An automaton is defined as a system where information and materials are transformed, transmitted and utilised for performing some processes without direct involvement of human beings.
- **DFA** A finite automaton is DFA if it follows $Q \times \Sigma$ into Q .
- **Representation of DFA** The presentations by which a finite automaton can be represented are: Transition equations, State Transition graph, and Transition table.
- **Transition Graph** A transition diagram (also called transition graph or transition system) is a finite directed labelled graph in which each vertex represents a state and directed edges indicate the transition from one state to another. Edges are labelled with input or input/output.
- **Generalised Transition Graph** Generalised transition graphs are transition graphs where the labels of edges are regular expressions

- **Transition Table** A transition table is the tabular representation of the transition system of an automaton.
- **FA with Epsilon Transitions** The FA that takes input from $\Sigma \cup \{\epsilon\}$ is called a FA with epsilon transitions.
- **Language of DFA** The set of all strings accepted by a DFA is called the language of DFA. The number of strings accepted by a DFA depends upon the number of unique paths from initial state to final state while processing strings with a DFA.
- **Acceptability of a String by a DFA** A string w is said to be accepted by a finite automaton $M = (Q, \Sigma, \delta, q_0, F)$ if $\delta(q_i, w) = q_j \in F$, where q_0 is initial (start) state and F is set of final (accepting) states.
- **NFA** A finite automaton is NFA if it follows $Q \times \Sigma \rightarrow 2^Q$.
- **Two way FA** When read head is allowed to move in either direction (left or right), the modified model is called two-way finite automaton.
- **FA with Output** Mealy and Moore machines are finite automata with output.
- **Equivalence between Moore and Mealy Machines** A Moore machine can never be correctly equivalent to a Mealy machine because the length of output string from a Moore machine is one larger than that from a Mealy machine by giving the same input. In other words, we can say that for a Moore machine if the input string is of length n , the output string is of length $n + 1$. A Mealy machine is equivalent to a Moore machine in the sense both can accept the same language or same set of strings. They are not equivalent in the sense the outputs are different for same input, as we know the length of outputs are different.
- **Equivalence of Two FA** Two finite automata M and M' are said to be equivalent if and only if both accept the same language or the same set of strings. If automaton M contains n_1 states and there exists another automaton M' equivalent to M containing n_2 states with $n_2 \leq n_1$, then it is desirable to use M' .
- **Unreachable States** These states are those states of a DFA, which are not reachable from the initial state of DFA on any possible input. The unreachable or non-reachable states are useless states in an automaton and these should be eliminated immediately.
- **Dead States** These states are those states of a DFA whose transitions on every input symbol terminate on itself.
- **Non-distinguishable States** These states are those states of a DFA for which there exist no distinguishing string. Hence, these states cannot be distinguished from one another.
- **Extending Transition Function** When transition function of finite automaton is defined as $Q \times \Sigma^*$ into Q , then Σ^* is string rather than a single symbol.
- **Applications of FA** There are several software design problems that are simplified by converting into regular expression representation to corresponding finite automata. Some of these are: Lexical Analyser, Switching Circuit Design, Word Processor Programmes, Text Editors, etc.

Review Questions

2.1 A transition function table for finite state machine M is given:

Table 2.29 Transition table of Exercise 2.1

PRESENT STATE	NEXT STATE	
	a	b
$\rightarrow q_0$	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

For above find out the strings which are accepted by M : (a) 101101 (b) 111111 (c) 000000.

2.2 FA is denoted by $M = (Q, \Sigma, \delta, q_0, F)$, where

$$Q = \{q_0, q_1, q_2, q_3\}, \Sigma = \{0, 1\}$$

$$F = \{q_0\} \text{ and } \delta \text{ is given by Table 2.27}$$

Construct transition diagram.

2.3 Convert following NDFAs into DFAs

(a)

Table 2.30 Transition table of Exercise 2.3 (a)

PRESENT STATE	NEXT STATE	
	a	b
$\rightarrow q_0$	q_0	q_1
(q_1)	q_1	q_0, q_1

(b)

Table 2.31 Transition table of Exercise 2.3 (b)

PRESENT STATE	NEXT STATE	
	a	b
$\rightarrow q_0$	q_0, q_1	q_2
q_1	q_0	q_1
(q_2)	q_1	q_0, q_1

2.4 $M = (\{q_1, q_2, q_3\}, \{0, 1\}, \delta, q_1, \{q_3\})$ is an NDFA, where δ is given by

$$\delta(q_1, 0) = \{q_2, q_3\}, \quad \delta(q_1, 1) = \{q_1\}$$

$$\delta(q_2, 0) = \{q_1, q_2\}, \quad \delta(q_2, 1) = \{\emptyset\}$$

$$\delta(q_3, 0) = \{q_2\}, \quad \delta(q_3, 1) = \{q_1, q_2\}$$

Construct an equivalent DFA

2.5 Prove that NDFA = DFA.

2.6 Convert the following Mealy machine into an equivalent Moore machine.

Table 2.32 Transition table of Exercise 2.8

PRESENT STATE	NEXT STATE			
	$x = 0$	OUTPUT	$x = 1$	OUTPUT
$\rightarrow q_1$	q_3	0	q_2	0
q_2	q_1	1	q_4	0
q_3	q_2	1	q_1	1
(q_4)	q_4	1	q_3	0

2.7 Convert the following Moore machine into an equivalent Mealy machine.

Table 2.33 Transition table of Exercise 2.9

PRESENT STATE	NEXT STATE		OUTPUT
	$a = 0$	$a = 1$	
$\rightarrow q_1$	q_1	q_2	0
q_2	q_1	q_3	0
(q_3)	q_1	q_3	1

2.8 Construct a minimum state automaton equivalent to the finite automaton given below:

Table 2.34 A finite automaton of Exercise 2.10

PRESENT STATE	NEXT STATE	
	0	1
$\rightarrow q_0$	q_1	q_5
q_1	q_6	q_2
(q_2)	q_0	q_2
q_3	q_2	q_6
q_4	q_1	q_5
q_5	q_2	q_6
q_6	q_6	q_4
q_7	q_6	q_2

2.9 For each of the following Moore machines, construct the transition and output tables:

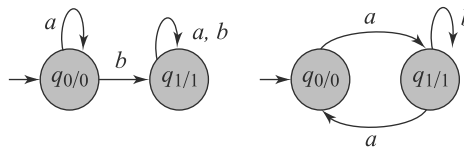


Fig. 2.52 Mealy and Moore machines of Exercise 2.9

2.10 An NFA with states $q_1 - q_5$ and input alphabet $\{a, b\}$ is described by the transition Table 2.35 given below:

Table 2.35 Transition table of NFA of Exercise 2.10

PRESENT STATE	NEXT STATE	
	a	b
$\rightarrow q_1$	$\{q_1, q_2\}$	$\{q_1\}$
q_2	$\{q_3\}$	$\{q_3\}$
q_3	$\{q_4\}$	$\{q_4\}$
q_4	$\{q_5\}$	—
(q_5)	—	$\{q_5\}$

(a) Calculate $\delta^*(q_1, ab)$, (b) Calculate $\delta^*(q_1, abaab)$, (c) Calculate $\delta^*(q_2, abb)$

2.11 Construct a Moore machine equivalent to a Mealy machine given by Table 2.36:

Table 2.36 Mealy machine of Exercise 2.11

PRESENT STATE	NEXT STATE			
	$a = 0$	OUTPUT	$a = 1$	OUTPUT
$\rightarrow q_1$	q_3	0	q_2	0
q_2	q_1	1	q_4	0
q_3	q_2	1	q_1	1
q_4	q_4	1	q_3	0

2.12 Construct a Mealy machine equivalent to a Moore machine given by Table 2.37:

Table 2.37 Moore machine of Exercise 2.12

PRESENT STATE	NEXT STATE		OUTPUT
	$a = 0$	$a = 1$	
$\rightarrow q_0$	q_3	q_1	0
q_1	q_1	q_2	1
q_2	q_2	q_3	0
q_3	q_3	q_0	0

2.13 Design two way finite automaton to accept the language $L = \{a^{2n+1} \mid n \geq 0\}$

2.14 Give the advantage of two way finite automaton over one way (left to right) FA.

2.15 Design Mealy machine for accepting all decimal numbers divisible by 3.

2.16 Design Moore machine for recognising all even integers between 100 and 1000.

2.17 Determine $\delta^*(q_1, 010)$ and $\delta^*(q_4, 1001)$

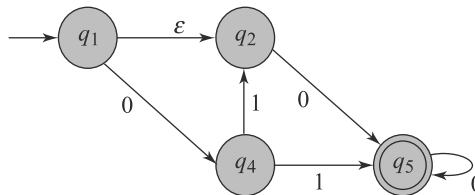


Fig. 2.53 An NFA

Hints

- 2.1 (a) Accepted: $q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_3 \xrightarrow{1} q_2 \xrightarrow{1} q_3 \xrightarrow{0} q_1 \xrightarrow{1} q_0$
- (b) Accepted: $q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_0$
- (c) Accepted: $q_0 \xrightarrow{0} q_2 \xrightarrow{0} q_0 \xrightarrow{0} q_2 \xrightarrow{0} q_0 \xrightarrow{0} q_2 \xrightarrow{0} q_0$

- 2.10 (a) $\delta^*(q_1, ab) = \delta(\delta^*(q_1, a), b)$
 $= \delta(\delta(\delta^*(q_1, \wedge), a), b)$
 $= \delta(\delta(q_1, a), b)$
 $= \delta(\{q_1, q_2\}, b)$
 $= \{q_1, q_3\}$
- (b) $\delta^*(q_1, abaab) = \delta(\delta^*(q_1, a), baab)$
 $= \delta(\delta(\delta^*(q_1, a), b), aab)$
 $= \delta(\delta(\delta(\delta^*(q_1, a), b), a), ab)$
 $= \delta(\delta(\delta(\delta(\delta^*(q_1, a), b), a), a), a), b)$
 $= \delta(\delta(\delta(\delta(\delta(\delta^*(q_1, \wedge), a), b), a), a), a), b)$
 $= \delta(\delta(\delta(\delta(q_1, a), b), a), a), b)$
 $= \delta(\delta(\delta(\delta(\{q_1, q_2\}, b), a), a), b)$
 $= \delta(\delta(\delta(\delta(\{q_1, q_2\}, b), a), a), b)$
 $\cdot$
 $\cdot$
 $= \{q_1, q_3, q_4, q_5\}$
- (c) Try yourself like (a) and (b).

- 2.11 The constructed Moore machine is given by the following transition Table 2.38:

Table 2.38 Moore machine of Exercise 2.11

PRESENT STATE	NEXT STATE		OUTPUT
	$a = 0$	$a = 1$	
$\rightarrow q_0$	q_3	q_{20}	0
q_1	q_3	q_{20}	1
q_{20}	q_1	q_{40}	0
q_{21}	q_1	q_{40}	1
q_3	q_{21}	q_1	0
q_{40}	q_{41}	q_3	0
q_{41}	q_{41}	q_3	1

- 2.12 The constructed Mealy machine is given by the following transition table:

Table 2.39 Mealy machine of Exercise 2.12

PRESENT STATE	NEXT STATE			
	$a = 0$	OUTPUT	$a = 1$	OUTPUT
$\rightarrow q_0$	q_3	0	q_1	1
q_1	q_1	1	q_2	0
q_2	q_2	0	q_3	0
q_3	q_3	0	q_0	0

Graded Exercises

- *2.1** Draw NFA with specified number of states to accept the following languages:
- (i) All strings that contain the substring 0101 (using 5 states)
 - (ii) All strings such that the third symbol from the right end is a '0' (4 states).
- *2.2** Consider the FA below and construct the, smallest DFA which accepts the same language.

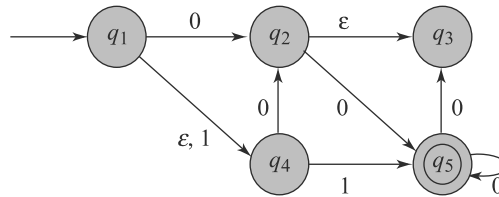


Fig. 2.54 FA of Exercise 2.2

- **2.3** Give Mealy and Moore machines for the following:
- (a) For input from $(0 + 1)^*$, if the input ends in '000', output A; if the input ends in '111', output B; otherwise output C.
 - (b) For input from $(0 + 1 + 2)^*$ print the residue modulo 5 of the input treated as a ternary (base 3, with digits 0, 1, 2,) number.
- *2.4** Construct an FA accepting all strings over $\{0, 1\}$ ending in 010 or 0010.
- **2.5** Show that there exists no FA accepting all palindromes over $\{a, b\}$.
- **2.6** Draw DFA for the following language over $\{0, 1\}$:
- (i) All strings with even number of 0's and even number of 1's.
 - (ii) All strings of length at most five.
- ***2.7** Design a Mealy machine that accepts binary string divisible by 3.
- **2.8** Design a finite automaton that reads strings of 0's and 1's and recognises modulo 4 as based on the number of 1's in the string.
- **2.9** Design a finite automaton that reads strings of 0's and 1's and recognises $n \bmod 4 = 3$. That means the strings of 0's and 1's with number of 1's as 3, 7, 11, ...
- **2.10** Design a finite automaton that accepts all decimal integers divisible by 3.

* Difficulty level 1

** Difficulty level 2

*** Difficulty level 3

Hints

- 2.2** In this FA, q_3 is the dead state. After elimination of q_3 we get the following FA:

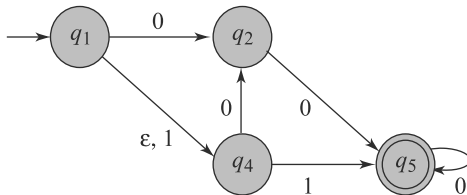


Fig. 2.55 FA of Exercise 2.2

- 2.3** (b) Required Mealy and Moore machines are given by Fig. 2.21 (iii) and (iv) respectively.

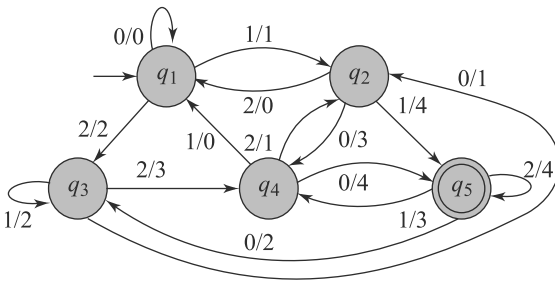


Fig. 2.56 Mealy machines of Exercise 2.3(b)

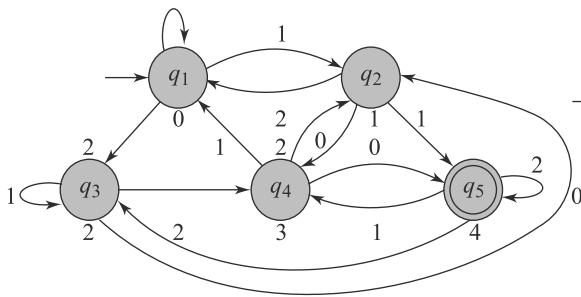


Fig. 2.57 Moore machines of Exercise 2.3(b)

- 2.4** The following is the FA accepting all strings over $\{0, 1\}$ ending in 010 or 0010.

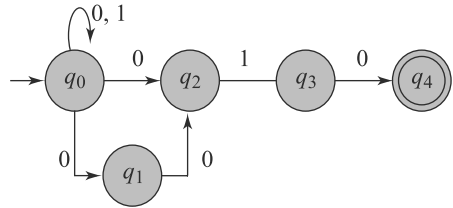


Fig. 2.58 FA of Exercise 2.4

- 2.5** The language of all palindromes is context free language, and to recognise a context free language we require PDA or higher machine. Therefore, no FA exists to accept palindromes.

- 2.6** (i) The required DFA is given below:

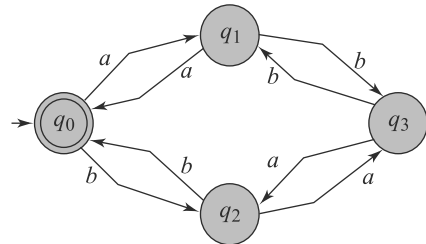


Fig. 2.59 FA of Exercise 2.6 (i)

- (ii) The required DFA is given below:

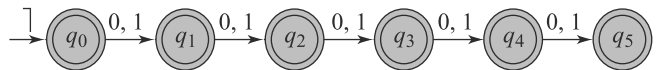


Fig. 2.60 FA of Exercise 2.6 (ii)

- 2.7** The following is the required Mealy machine that accepts binary strings $(0^*, 0^*11, 0^*110^*, 1001, 10010^*, 10101, \dots)$ whose decimal equivalents are 0, 3, 6, 9, 12, 15, 18, $\dots$

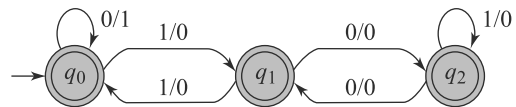


Fig. 2.61 FA of Exercise 2.7

Objective Questions

- Which one of the following is false for FA $M = (\{q_0, q_1\}, \{a, b\}, \delta, q_0, \{q_1\})$?
 (a) $q_0 \in Q$ (b) $q_0 \in F$ (c) $abb \in \Sigma^*$ (d) none of these
- All notations in Mealy are equal to a Moore machine with only the difference of:
 (a) output function (b) initial state (c) output alphabet (d) input alphabet
- A final state as well as initial in a transition diagram is denoted by
 (a)  (b)  (c)  (d) none of these
- A transition system accepts a string $w \in \Sigma^*$ if:
 (a) there exists a path which originates from some initial state
 (b) there exists a path which terminates at some final state
 (c) there exists a path which originates from some initial state, goes along the arrows, and terminates at some final state
 (d) none of these
- The relation between Σ^* and Σ^+ is:
 (a) $\Sigma^* \subseteq \Sigma^+$ (b) $\Sigma^* = \Sigma^+ \cup \{\Lambda\}$ (c) $\Sigma^+ = \Sigma^* \cup \{\Lambda\}$ (d) none of these
- $\delta(q, xy)$ is similar to
 (a) $\delta(q, x), y$ (b) $\delta(\delta(q, x), y)$ (c) $\delta(q, y), x$ (d) none of these
- In an NDFA $M = (Q, \Sigma, \delta, q_0, F)$ the out-come is
 (a) a subset of Q (b) a state $q \in Q$ (c) neither (a) nor (b) (d) none of these
- A string $s \in \Sigma^*$ is accepted by an NDFA if
 (a) $\delta(q_0, s)$ contains a next state (b) $\delta(q_0, s)$ contains some final state
 (c) $\delta(q_0, s)$ contains some initial state (d) none of these
- A transition table is also known as
 (a) state table (b) transition function table
 (c) transition system table (d) all of these
- A Moore machine can never be correctly equivalent to a Mealy machine because
 (a) the length of output string in a Moore machine is larger than one from a Mealy machine
 (b) the length of output string in a Mealy machine is one larger than length of output string in a Moore machine
 (c) in Mealy machine the output depends upon present state and present input
 (d) none of the above
- Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA and $M' = (Q', \Sigma, \delta', q'_0, F')$ be a minimum state FA, then
 (a) $Q' \subseteq Q$ (b) $F' \subseteq F$ (c) both (a) and (b) (d) none of these

12. Consider the following transition system

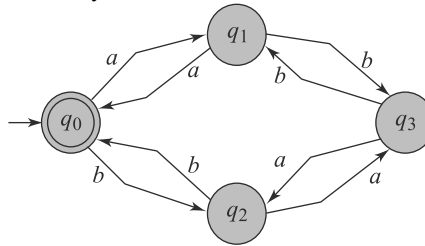


Fig. 2.62 FA of Question (12)

- The string accepted by the transition system is
- (a) *bbbbba* (b) *aaaa* (c) *bbaaa* (d) *aaba*
13. A transition diagram is also known as
- (a) transition system (b) transition graph (c) both (a) and (b) (d) none of these
14. For a given DFA $M = (\{q_1, q_2, q_3\}, \{a, b\}, \delta, q_1, \{q_2\})$, then $\delta(q_2, \wedge)$ is
- (a) q_1 (b) q_2 (c) q_3 (d) both (a) and (b)
15. The difference between the DFA and NFA is
- (a) only in Σ (b) only in initial state
(c) only in final state (d) only in δ (the transition function)
16. If $\Sigma = \{1, 2, 3\}$, then
- (a) $222 \in \Sigma^*$ (b) $321 \in \Sigma^*$ (c) $112233 \in \Sigma^*$ (d) All of these
17. The transition $\delta(q, a) = (q', L)$ in two-way finite automaton describes
- (a) on scanning symbol 'a', when in state q , its read head moves one block left
(b) on scanning symbol 'a', when in state q , it goes to next state q'
(c) both (a) and (b)
(d) none of the above
18. Which of the following statements is false?
- (a) the finite automaton in which the output is associated with state only is called a Moore machine.
(b) the finite automaton in which the output is associated with state and input is called a Mealy machine.
(c) the finite automaton in which the movement of read head is in either direction (left or right) is called a two-way finite automaton
(d) none of the above
19. Which of the following statement is true?
- (a) if M_1 is a Moore machine, then there is a Mealy machine M_2 equivalent to M_1 .
(b) if M_1 is a Mealy machine, then there is a Moore machine M_2 equivalent to M_1 .
(c) both (a) and (b) are true
(d) none of these
20. The applications of finite automata include:
- (a) lexical analyzers (b) text editors (c) operating system (d) only (a) and (b)

21. The string accepted by following finite automaton are:

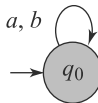


Fig. 2.63 Finite automaton of Question (21)

- (a) $\{a, aa, aaa, \dots\}$ (b) $\{b, bb, bbb, \dots\}$
 (c) $\{a, b, ab, ba, aa, bb, \dots\}$ (d) $\{\emptyset\}$
22. Which of the following statements is wrong ?
- (a) A finite state machine can be considered to be a Turing machine of finite tape length rewinding capability, and unidirectional tape movement.
 (b) Palindromes cannot be recognised by any finite state machine because a finite state machine cannot deterministically fix the mid-point.
 (c) Palindromes cannot be recognised by any finite state machine because even if the mid-point is known, a finite state machine cannot find whether the second half of the string matches the first half
 (d) none of these.
23. Consider the transition diagram of a finite automaton given below:

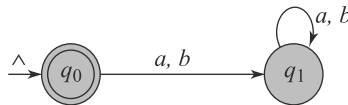


Fig. 2.64 Finite automaton of Question (23)

This automaton accepts:

- (a) all strings over $\{a, b\}$
 (b) all strings over $\{a, b\}$ including $\wedge$
 (c) only $\wedge$, that means this automaton accepts no string of length greater than zero.
 (d) all string over $\{a, b\}$ excluding $\wedge$
24. If the language L is recognised by an FA, then the language $\text{FRONT}(L) = \{s \mid s \text{ is the prefix of } t, \text{ here } t \in L\}$ is a
- (a) regular language (b) context-free language (c) context-sensitive language (d) all of these
25. Which of the following statement is true ?
- (a) A Mealy machine generates no language as such.
 (b) A Moore machine generates no language as such.
 (c) A Mealy machine has no terminating state.
 (d) all of the above
26. A finite state machine can recognise:
- (a) only regular grammar (b) only context-free grammar
 (c) only unambiguous grammar (d) any grammar

27. Which of the following are not regular ?
- string of 0's whose length is a perfect square
 - string of 0's whose length is a prime number
 - string of palindromes over $\{0, 1\}$
 - all of the above
28. The main limitation of finite state machine is that
- it cannot remember arbitrary large amount of information
 - it sometimes recognises grammars that are not regular
 - it sometimes fail to recognise grammar that are regular
 - both (a) and (c)
29. If a language L is recognised by a finite automaton, then the language $\text{REVERSE}(L) = \{s \mid s \text{ is the reverse of } t \text{ where } t \in L\}$ is a
- recursively enumerable language
 - context-sensitive language
 - context-free language
 - regular language
30. Any transition graph has an equivalent
- regular expression
 - deterministic finite state machine
 - nondeterministic finite state machine
 - all of the above
31. The FA given below accepts:

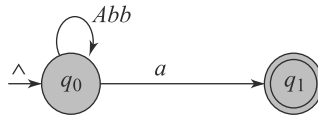


Fig. 2.65 Finite automaton of Question (31)

- all strings that contain the sub string 'abb' and end with 'a'
 - all strings that contain the sub string 'abba' and end with 'a'
 - all strings over $\{a, b\}$
 - both (b) and (c)
32. The language accepted by following FA is

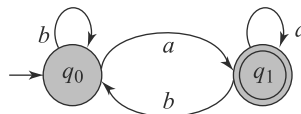


Fig. 2.66 Finite automaton of Question (32)

- $(a + b)^* b$
 - $(a + b)^* a$
 - $a^* b$
 - $a^* b^*$
33. A palindrome cannot be recognised by any finite state machine because
- a finite state machine cannot remember arbitrary large amount of information
 - finite state machine cannot deterministically fix the mid-point
 - even if the mid-point is known, a finite state machine cannot find whether the second half of the string matches the first half.
 - all of the above

34. The FA given below accepts

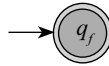


Fig. 2.67 Finite automaton of Question (34)

(a) all strings

(b) only $\wedge$

(c) no string

(d) none of these

References for Extra Reading

1. Aho, A. V., R. Sethi, and J. D. Ullman *Compilers. Principles, Techniques and Tools*. Addison Wesley, 1986.
2. Booth, Taylor L., *Sequential Machines and Automata Theory*, New York: John Wiley and Sons, 1967.
3. Claude Shannon and John McCarthy, *Representation of Events in Nerve Nets and Finite Automata*, in Automata Studies, 1956.
4. Crochemore, M. & D. Perrin, *Twoway string matching*, Journal of the ACM, 1991.
5. Huffman, D. A. *The synthesis of sequential switching circuits*, Journal of the Franklin Institute (1954).
6. Moore, E. F. *Gedanken experiments on sequential machines*, in Automata Studies, 129–153, Princeton University Press, Princeton, New Jersey, 1956.
7. Mealy, G. H. *A method for synthesising sequential circuits*, Bell System Technical Journal (1955).
8. Gill, A., *Introduction to the Theory of Finite-state Machines*, McGraw-Hill, 1962.
9. Goldstine, J., C.M.R. Kintala and D. Wotschke, *On measuring nondeterminism in regular languages*, Information and Computation 1990.
10. Goldstine, J., H. Leung & D. Wotschke, *On the relation between ambiguity and non-determinism in finite automata*, Information and Computation, 1992.
11. Kintala, C.M.R. & D. Wotschke, *Amounts of nondeterminism in finite automata*, Acta Informatica, (1980).
12. Kohavi, Z., *Switching and Finite Automata Theory*, McGraw-Hill, 1978.
13. M. Daley, M. G. Eramian, and S. Yu (eds.), *Conference on Implementation and Application of Automata CIAA'2000*. London, Ontario, Canada, University of Western Ontario.
14. Rabin, M. O. and D. Scott, *Finite automata and their decision problems*, IBM Journal of Research and Development 3, 114–125, 1959.
15. Mateescu, A., A. Salomaa, K. Salomaa & S. Yu, *Lexical analysis with a simple finite-fuzzy-automaton model*, Journal of Universal Computer Science, 1995.
16. Mealy, G. H., *A method for Synthesizing Sequential Circuits*, Bell system Technical Journal, 1955
17. Minsky, Marvin, *Computation: Finite and Infinite Machines*, New Jersey: Prentice-Hall, 1967.
18. N. Immerman, *Nondeterministic space is closed under complementation*, SIAM Journal of Computing 1988.
19. Rabin, M.O., Scott, D., *Finite Automata and Their Decision Problems*, IBM Journal of Research and Development, 1959.
20. Eilenberg, S. *Automata, languages and machines*, Acad. Press 1974.
21. Tiziano Villa, *Synthesis of Finite State Machines: Logic Optimisation*, Kluwer Academic Publishers, Boston 1997.

Online Sources

1. http://public.beuth-hochschule.de/~merceron/pub/DP-DFA_eng.pdf
2. http://www.cis.uoguelph.ca/~sawada/3620/notes/fsa_proofs-1x2.pdf
3. <http://www.cs.uky.edu/~lewis/texts/theory/automata/fin-auto.pdf>
4. <http://www.let.rug.nl/~vannoord/papers/preds.pdf>

3

Formal Languages

❖ Chapter Objective ❖

In this chapter we proceed towards a detailed discussion on formal languages. We will discuss how to define and present languages obtained by various operations on formal languages. We will discuss the rules by which a language is generated. These rules are called grammars. We will see how to construct rules by which language can be generated. Then we proceed towards the classification of grammars based on types of production rules. This classification is called the Chomsky Hierarchy. In this hierarchy, we will see four types of machines (automata) namely finite automata corresponding to regular languages (already seen in chapter 2), Push Down Automata corresponding to context free languages, Linear Bounded Automata corresponding to context sensitive languages, and Turing Machine corresponding to recursively enumerable languages. A detailed discussion on Push Down Automata, Linear Bounded Automata and Turing Machines is given in later chapters. Finally, we discuss the relations among all classes of languages.

□ Introduction □

In the formal approach to languages, words are a collection of symbols which fit together in regular ways. The possibilities of combination are their syntax, which is often expressed as a set of grammatical rules. The study of grammar is of course very ancient, but it is only in recent times that grammars have become formal. The modern approach is credited to Noam Chomsky, and it coincides with the advent of computers. When we talk about Chomsky classification of formal languages, we see a definite progression. All regular languages are context-free languages, and we shall see that all context-free languages are Turing machine languages. Historically, the order of these ideas is as follows:

- ◆ Regular languages and finite automata were developed by Kleene, Mealy, Moore, Rabin, and Scott in the 1950s. Regular language is the simplest form of formal languages.
- ◆ Context-free grammars and pushdown automata were developed later, by Chomsky, Oettinger, Schutzenberger, and Evey in the 1960s.
- ◆ Turing machines and their theory were developed by Alan Mathison Turing and Emil Post in the 1930s and 1940s.

Linguists were trying in the decade of 1950 to define the precise description of valid sentences and structural descriptions of sentences. They wanted to describe a grammar (i.e., formal grammar) to describe English.

In 1956, Noam Chomsky gave a mathematical model of grammar. Although it was not useful for describing natural languages, but it was useful for computer languages. ALGOL is precisely described by Backus-Naur form.

With the concept of natural language grammar, the syntax rules of formal language are developed. This generation of language is initiated with the concept of English and other languages. The word “formal” refers to the fact that all the rules for the languages are explicitly stated in terms of what strings of symbols can occur.

A “symbol” is an abstract entity. Letters and digits are examples of frequently used symbols. A string (or word) is a finite sequence of symbols. For example, a , b , and c are symbols and ‘ $abcd$ ’ is a string. The length of a string ω is denoted by $|\omega|$, which is the number of symbols composing the string. For example, ‘ $abcd$ ’ has length 4. The empty string or string of length zero denoted by ε or $\wedge$, is the string consisting of zero symbols. Therefore, $|\varepsilon| = 0$. Set of strings for a language while following certain syntax rules.

3.1

THEORY OF FORMAL LANGUAGES

A formal language is a set of strings of symbols from some alphabet. The empty set ϕ , and the set consisting of the empty string (ε) are languages. The set of palindromes (strings that read the same forward and backward) over the alphabet $\{0, 1\}$ is an infinite language. Some members of this language are ε , 0, 00, 11, 101 and 1101011 etc. So a certain specified set of strings of characters from the alphabets will be called the language. Those strings that are permissible in the language we call words. A string to have no letters we call empty string or null string, and we denote it by symbol ε or $\wedge$. No matter what “alphabet” we are considering, the null is always $\wedge$ and for all languages the null word, if it is a word in the language, is also $\wedge$.

Two words are considered to be same if all their letters are same and in the same order, so there is only one possible word of no letters. For clarity, we usually do not allow the symbol $\wedge$ to be part of the alphabet for any language.

We denote the language that has no words by the standard symbol for the null set, ϕ . It is not true that $\wedge$ is a word in the language ϕ since this language has no words at all. If a certain language L does not contain the word $\wedge$ and we wish to add it to L , we use the “union of sets” operation denoted by “+”, “ $\cup$ ”, or “,” to form $L + \{\wedge\}$, $L \cup \{\wedge\}$, or $(L, \{\wedge\})$ respectively. All have same meanings. This language is not the same as L . On the other hand, $L + \phi$ is the same as L since no new word has been added.

Now we consider the most familiar example of language, English. The alphabet is usually the combination of letters with apostrophe and ‘hyphen’. Let us denote the whole alphabet by Σ :

$$\Sigma = \{a, b, c, d, e, \dots, z, ', _ \}.$$

This set can be used to denote a collection of letters to form an alphabet for the words in the language L .

We can now state which strings of these are valid words in our language by listing them all, as is done in the English dictionary. It is a long, but finite list, and it makes a perfect definition of the language. If we call this language E-WORDS, then

$E\text{-WORDS} = \{\text{collection of all words in a Standard English dictionary}\}.$

Backtracking to the language of $E\text{-WORDS}$, we can say that this is not what we usually mean by “English”. To know all the words in a finite language like English does not imply the ability to create a viable sentence.

If we wish to make a formal definition of the language of the sentences in English, we must start by saying that this time our basic alphabet is the entries in the dictionary. Let us call this alphabet τ :

$\tau = \{\text{the entries in a standard dictionary, with blank space and usual punctuation marks}\}.$

In order to specify which strings of elements from τ produce valid words in the language $E\text{-SENT}$ (We are using $E\text{-SENT}$ for English sentences), we must consult the grammatical rules of English. This is because we could never produce a complete list of all possible words in this language; that would have to be a list of all possible words in this language; that would have to be a list of all valid English sentences. So there are different infinite words in the language $E\text{-SENT}$. For example,

I take one cup of tea.

I take two cups of tea.

I take three cups of tea.

... ..

The method of defining the language $E\text{-SENT}$ by listing all rules of English grammar allows us to give a finite description of an infinite language.

If we follow the rules of grammar only, several strings of alphabet letters seem to be valid words, for example,

“I take four Sundays daily”.

In a formal language we must allow this string. It is grammatically correct; only its meaning indicates that it is ridiculous.

3.2

KLEENE AND POSITIVE CLOSURE

If Σ is the set of alphabets, we wish to define a language in which any string of letters from Σ is a word, even the null string $\wedge$, we call this language ‘closure’ of the alphabet. It is denoted by the symbol $*$ (an asterisk) after the name of the alphabet e.g., Σ^* . This notation is also known as the ‘Kleene star’ after the logician Stephen Kleene.

If $\Sigma = \{a\}$, then

$\Sigma^* = \{\wedge, a, aa, aaa, \dots\}.$

= {The set of all strings having 0 to ∞ occurrences of a ’s}

If $\Sigma = \{a, b\}$ then

$\Sigma^* = \{\wedge, a, b, aa, ab, ba, bb, \dots\}$

= {The set of all strings made from either a ’s or b ’s or combination of both having 0 to ∞ occurrences}.

Similarly, if $\Sigma = \{a, b, c\}$, then

$\Sigma^* = \{\wedge, a, b, c, ab, ac, ba, bc, ca, cb, \dots\}$

We can say that by using Kleene star (or say closure) operation we can make an infinite language of strings of letters out of an alphabet. So an ‘infinite language’ refers to infinitely many words, each of finite length.

❖ Good to Know!

We write the words in increasing order of length called “lexicographic order”. First of all we list Λ , then all the words of the one or more length alphabetically. We usually follow this method of sequencing in all languages.

If S is a set of words, then S^* is the set of all strings formed by concatenating words from S , where any word may be used any number of times (by including the null string also), and in any order. S^* is called transitive closure of S .

Consider the following example:

Let $S = \{a, ab\}$, the

$$\begin{aligned} S^* &= \{ \Lambda, \text{any word obtained by composition of } a \text{ and } ab \text{ as often we like} \}. \\ &= \{ \Lambda, \text{all strings of } a\text{'s and } b\text{'s except those that start with } b \text{ and those contain} \\ &\quad \text{two or more consecutive } b\text{'s} \} \\ &= \{ \Lambda, a, aa, ab, aaa, aab, aba, aaaa, aaab, \dots \} \end{aligned}$$

If we wish to modify the concept of closure to refer to only the concatenation of some strings (not zero) from a set denoted by S , we use the notation ‘+’ instead of ‘*’. For example,

If $\Sigma = \{a\}$, then

$$\Sigma^+ = \{a, aa, aaa, \dots\}.$$

If S is a set of strings not including Λ , then S^+ is the language S^* without the word Λ . In other words, if S is a language that does contain Λ , then $S^+ = S^*$.

This ‘+’ (plus operation) is sometimes called positive closure. If $S = \{aa, aaa\}$, then S^+ is the same as S^* except for the word Λ which is not in S^+ .

Relation between Kleene and Positive Closures

Basically the main difference in Kleene and positive closure is of Λ . When we determine the Kleene closure of a symbol, string or set, the first element is always Λ , i.e. the zero occurrences of the symbol or string of what we are determining the Kleene closure. The Kleene closure of A is defined as:

$$\begin{aligned} A^* &= \bigcup_{i=1}^{i=\infty} A^i \\ &= (A^0 \cup A^1 \cup A^2 \cup A^3 \cup A^4 \cup A^5 \cup \dots \cup A^\infty) \\ &= (A^0, \cup A^1, \cup A^2, \cup A^3, \cup A^4, \cup A^5, \dots, A^\infty) \\ &= (A^0 + A^1 + A^2 + A^3 + A^4 + A^5 + \dots + A^\infty) \\ &= (\Lambda, A^1, A^2, A^3, A^4, A^5, \dots, A^\infty) \end{aligned}$$

But in case of positive closure definition is same as above with only the difference of initial value of i .

$$\begin{aligned} A^+ &= \bigcup_{i=1}^{i=\infty} A^i \\ &= (A^1 \cup A^2 \cup A^3 \cup A^4 \cup A^5 \cup \dots \cup A^\infty) \end{aligned}$$

$$\begin{aligned}
 &= (A^1, A^2, A^3, A^4, A^5, \dots, A^\infty) \\
 &= (A^1 + A^2 + A^3 + A^4 + A^5 + \dots + A^\infty) \\
 &= (A^1, A^2, A^3, A^4, A^5, \dots, A^\infty)
 \end{aligned}$$

This way, we can establish following relationship between positive and Kleene closure:

$$A^* = A^+ \cup \{\Lambda\}$$

If Σ is an alphabet, then a string over Σ is some element of Σ^* . If $\Sigma = \{0,1\}$, some of the strings over Σ are 0, 100, 010, 00110 and 11011. The null string is denoted by the symbol Λ .

The set of all strings over Σ is denoted by Σ^* , if Σ is any alphabet, and a language over Σ is a subset of Σ^* . Let $\Sigma = \{a, b\}$, then

$$\{a, b\}^* = \{\Lambda, a, b, aa, ab, ba, bb, \dots\}.$$

A few examples of language generated over Σ are

- (i) $\{\Lambda, a, aa, aab, abb\} \in \{a, b\}^*$
- (ii) $\{x \in \{a, b\}^* \mid |x| \leq 5\} \in \{a, b\}^*$
- (iii) $\{x \in \{a, b\}^* \mid |x| = 1, 3, 5, 7, \dots\} \in \{a, b\}^*$
- (iv) $\{s \in \{a, b\}^* \mid \text{number of } a\text{'s in } s = \text{number of } b\text{'s in } s\} \in \{a, b\}^*$

Languages are sets of strings, and new languages can be constructed using set operations. Here we are considering union, intersection and difference operations on sets. Any two languages can be considered to be languages over a common alphabet. If $L_1 \subseteq \Sigma_1$ and $L_2 \subseteq \Sigma_2$ then L_1 and L_2 are both subsets of $(\Sigma_1 \cup \Sigma_2)$.

If $L_1, L_2 \subseteq \Sigma^*$, then

$$L_1 L_2 = \{xy \mid x \in L_1, y \in L_2\}.$$

For example, if $L_1 = \{\text{help, hope}\}$ and $L_2 = \{\text{fully, less}\}$, then

$$\begin{aligned}
 L_1 L_2 &= \{\text{help, hope}\} \{\text{fully, less}\} \\
 &= \{\text{helpfully, helpless, hopefully, hopeless}\}.
 \end{aligned}$$

If $\Sigma = \{a\}$, $w \in \Sigma$ and $L \subseteq \Sigma^*$ then

$$\begin{aligned}
 a^i &= aaa \dots a \quad (i \text{ times } a) \\
 w^i &= www \dots w \quad (i \text{ times } w) \\
 L^i &= LLL \dots L \quad (i \text{ times } L) \\
 \Sigma^i &= \Sigma \Sigma \Sigma \dots \Sigma \quad (\{w \in \Sigma^* \mid |w| = i\})
 \end{aligned}$$

Consider $i = 0$

$$a^0 = \Lambda, w^0 = \Lambda, L^0 = \Lambda, \text{ and } \Sigma^0 = \Lambda.$$

L^i is the set of all strings that may be obtained by concatenating i number of elements of L .

❖ Did you Know ?

Stephen Cole Kleene is the person behind Kleene closure. The name pronounced by Kleene was a little bit different as pronounced in different languages by persons in his circle. Ken Kleene, the son of S C Kleene, wrote the following about mispronunciations:

“As far as I am aware the pronunciations made are incorrect. I believe this pronunciation was invented by my father.”

3.3

DEFINING LANGUAGES

Let us consider some examples of languages. Suppose we have an alphabet Σ having only one letter x .

$$\Sigma = \{x\}.$$

We can define a language by a non-empty string of characters called word:

$$L_1 = \{x, xx, xxx, \dots\}.$$

This could also be written as:

$$L_1 = \{x^n \mid n = 1, 2, 3, \dots\}.$$

This language does not include the null string. We could have defined it so as to include Λ . In this language we can define the operation of concatenation, in which two strings are written down, in combination to form a longer string. In other words, concatenation of strings s_1 and s_2 is the string s_1 followed s_2 . Thus, when concatenate the string xxx with the string xx , we obtain ' $xxxxx$ '. The operation of concatenation is like:

x^i concatenated with x^j is the string x^{i+j} . For example, we can say that if the string ' xxx ' is a and the string xx is b , then to denote the string formed by concatenating a and b , we write:

$$ab = xxxxx.$$

But it does not happen always that when two strings are concatenated they produce another string or word in the language. For example, if the language is

$$\begin{aligned} L_2 &= \{x, xxx, xxxxx, xxxxxxx, \dots\} \\ &= \{\text{occurrence of odd powers of } x\} \\ &= \{x^{2i+1} \mid i = 0, 1, 2, 3, \dots\} \end{aligned}$$

then let $a = xxx$ and $b = xxxxx$ are both strings in L_2 , but their concatenation ' ab ' (e.g., ' $xxxxxxx$ ') is not in L_2 (say $xxxxxxx \notin L_2$).

Now consider another language generated by alphabets given in Σ

$$\Sigma = \{0, 1, 2, 3, \dots, 9\}$$

and we define the set of words as

$$L = \{\text{any finite string of alphabets that does not start with the letter zero}\}.$$

then we have

$$L = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, \dots\}$$

3.4

RECURSIVE DEFINITION OF LANGUAGES

A new method of defining sets is called recursive definition. A recursive definition is a three-step process. First, specify some basic objects in the set. Second, define rules for constructing more objects in the set from the ones already known. Third, declared that no objects except those constructed in the way are allowed in the list.

Let us consider the example of define the set of positive even integers. One standard way of defining this is:

EVEN = the set of all positive integers divisible by 2.

Another way is:

EVEN = the set of all $2n$ where $n = 1, 2, 3, \dots$

Third method by using recursive definition:

The set EVEN can be defined by these three steps:

- (i) 2 is in EVEN
- (ii) If x is in EVEN, then $x + 2$ is also EVEN.
- (iii) The only elements in EVEN are those which can be produced by using rule (i) and (ii).

The last rule i.e., (iii) is not to be included necessarily. The reason is that the third rule is less popular than the others. It is very difficult to implement in most practical applications. Now suppose we want to prove that 16 is in the set EVEN. By using rule one, we can say that 16 is EVEN, because 16 is divisible by 2. Now we will prove by recursive definition that 16 is in EVEN:

By step (i) we know that 2 is in EVEN

By step (ii) we know that $2 + 2 = 4$ is also in EVEN

Again by step (ii), we know that $4 + 2 = 6$ is also in EVEN

Similarly we can say that $14 + 2 = 16$ is also in EVEN.

Example 3.1

Give the recursive definition for positive integers.

Solution The following is the recursive definition of the positive integers:

- (i) 1 is in INT
- (ii) If x is in INT, then $x + 1$ is also in INT.

If we want the set INT to be defined to include both positive and negative integers. We use following recursive definition:

- (i) 1 is in INT
- (ii) If x and y both are in INT, then $x - y$ and $x + y$ are also in INT.

Since $1 - 1 = 0$, $2 - 1 = 1$, $1 + 1 = 2$. Thus we can say that for all positive x we also have $0 - x = -x$ (for all positive and negative integers).

Example 3.2

Give the recursive definition for finding factorial of a positive integer.

Solution

- (i) $0! = 1$
- (ii) $n! = n(n - 1)!$

Thus we can define $(n + 1)!$ in terms of $n!$ by allowing a procedure to call itself, we refer it as recursive definition.

3.5

ARITHMETIC EXPRESSIONS

In this section we discuss the validation of an arithmetic expression (AE) that can be scanned by computers. If Σ is set of alphabets for this language then:

$$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, *, /, (,)\}.$$

We know that following are not valid arithmetic expressions:

$$(4 + 3) - 2), 5 (*7 + 2), 3) - (9,$$

The first expression contains unbalanced parenthesis, the second expression contains forbidden substring (*, and the third has a close parenthesis before the corresponding open parentheses. The other sequences like //, */ , + -, - +,), may also be in other invalid arithmetic expression.

A recursive definition may be defined for validity of an arithmetic expression. The definition is:

- (1) Any number (positive, negative, or zero) is in *AE*.
- (2) If '*a*' is in *AE*, then
 - (i) (*a*) is in *AE*
 - (ii) -*a*, is also in *AE*.
- (3) If '*a*' and '*b*' are in *AE*, then following are also in *AE*:
 - (i) *a* + *b* (when the first symbol in '*b*' is not + or -)
 - (ii) *a* - *b* (when the first symbol in '*b*' is not + or -)
 - (iii) *a* * *b*
 - (iv) *a* / *b*
 - (v) *a* ** *b* (exponentiation).

Example 3.3

Give the recursive definition of all odd length palindromes O-PAL over {*a*, *b*}.

Solution

1. *a* and *b* are in O-PAL as they are length one strings.
2. If *x* is in O-PAL, then *axa* and *bxb* are also in O-PAL.
3. Strings obtained by using rules 1 and 2 are also in O-PAL.

Example 3.4

Check whether the following arithmetic expression is correct?

$$(x + y) * (z * (w - v) / y / y * (x + u) - s$$

Solution

- (*x* + *y*) is correct 3(i), and 2(ii)
 (*w* - *v*) is correct 3(ii), and 2(ii)
z * (*w* - *v*) is correct by 3(iii),

$z^* (w - v) / y$ is correct by 3(iv),
 $z^* (w - v) / y / y$ is correct by 3(iv),
 $(z^* (w - v) / y / y)$ is correct by 2(i),
 $(x + y) * (z^* (w - v) / y / y)$ is correct by 3 (iii)
 $(x + y) * (z^* (w - v) / y / y) * (x + u)$ is correct by 3 (iii)
 $(x + y) * (z^* (w - v) / y / y) * (x + u) - s$ is correct by 3 (ii).

Thus the above *AE* is valid.

An important use of recursive definitions is to determine what expressions are valid in symbolic logic. The valid expressions in sentential calculus or propositional calculus are traditionally called well-formed formulae (wff).

3.6

GRAMMARS

We have already studied what the formal languages are? How and why they are used in computation? In this section we introduce the concept of formal grammars and we will discuss Chomsky classification of languages.

Before giving the definition of grammar, we will study the analogy between computer languages and English language. Some of the rules of English grammar are as follows:

- (i) sentence $\rightarrow$ subject followed by a predicate
- (ii) subject $\rightarrow$ noun phrase
- (iii) noun-phrase $\rightarrow$ adjective followed by a noun-phrase
- (iv) noun-phrase $\rightarrow$ article followed by a noun-phrase
- (v) noun-phrase $\rightarrow$ noun
- (vi) predicate $\rightarrow$ verb followed by a noun-phrase
- (vii) noun $\rightarrow$ *Akash, Aman, Ajay, Paridhi*
- (viii) verb $\rightarrow$ *speaks, eats, drinks, gets, meets, beats*
- (ix) adjective $\rightarrow$ *naughty, shy, innocent, vibrant*
- (x) article $\rightarrow$ *a, an, the*

By using this small model of English language, we can construct hundreds of sentences like.

“naughty Aman beats innocent Akash”

Now we look on the method by which the above sentences is derived:

sentence $\Rightarrow$ subject predicate.	[by rule (i)]
$\Rightarrow$ noun-phrase predicate.	[by rule (ii)]
$\Rightarrow$ adjective noun-phrase predicate	[by rule (iii)]
$\Rightarrow$ adjective noun predicate	[by rule (v)]
$\Rightarrow$ adjective noun verb noun-phrase	[by rule (vi)]
$\Rightarrow$ adjective noun verb adjective noun-phrase	[by rule (iii)]
$\Rightarrow$ adjective noun verb adjective noun	[by rule (v)]
$\Rightarrow$ <i>naughty</i> noun verb adjective noun	[by rule (ix)]

$\Rightarrow$ <i>naughty Aman</i> verb adjective noun	[by rule (vii)]
$\Rightarrow$ <i>naughty Aman beats</i> adjective noun	[by rule (viii)]
$\Rightarrow$ <i>naughty Aman beats innocent</i> noun	[by rule (ix)]
$\Rightarrow$ <i>naughty Aman beats innocent Akash</i>	[by rule (vii)]

Above are the derivation steps to derive the sentence “*naughty Aman beats innocent Akash*.” The arrow $\Rightarrow$ indicates that a substitution was made according to the preceding rules of the grammar. In most cases, we have some options in selecting which rule we want to apply. There is a qualitative difference between the word ‘noun’ and the word ‘*Akash*’. To represent this, the words are written in bold (thick) format to be replaced by other possible words. In these grammar rules, the left hand side can be replaced by the right hand side but not the vice-versa. For example noun can be replaced by adjective followed by noun-phrase or, by article followed by noun-phrase or simply by noun. These replacements are called substitutions. The words that cannot be replaced by anything are called terminals (shown in italic) and, the words that must be replaced by other words are called non-terminals or variables in our terminology of grammar.

The grammar rules of the form

$$A \rightarrow aB$$

are called production rules or simply productions. This production says that word ‘*aB*’ can be substituted on place of ‘*A*’ but not the vice-versa.

A production $A \rightarrow aB$ is read as *A* to *aB*. In all productions all capital letters are nonterminals (also called variables) and, all small letters and operators (like +, *, etc.) are called terminals.

Just a single terminal or string of terminals (without a non-terminal) in the left side of the production is an invalid production. For example, $a \rightarrow Ba$ is not a valid production. Thus, a collection of finite productions constitutes a grammar.

❖ Definition

Grammar A grammar (also called phrase-structure grammar) can be defined as 4-tuples (V_N, Σ, P, S) , where

- (i) V_N = Finite non-empty set of variables or non-terminals.
- (ii) Σ = Finite non-empty set of input alphabets or terminals.
- (iii) P = Finite set of productions or productions rules or rewriting rules
- (iv) S = the start symbol
- (v) $V_N \cap \Sigma = \phi$ (i.e., the set V_N and Σ have no common elements)

The language generated by a grammar G is the set of all strings of terminals that can be produced from the start symbol S using the productions as substitutions.

Let G be some grammar and $L(G)$ be the language generated by grammar G . If $G = (V_N, \Sigma, P, S)$, then we say,

$$\begin{aligned} L(G) &= \{\text{Language generated by grammar } G\} \\ &= \{\text{Set of all terminal strings represented by start symbol 'S'}\} \end{aligned}$$

❖ Good to Know!

If there is absence of start symbol S in a given set of productions representing a grammar G , then the variable in the left hand side of the first productions is considered as start symbol. For example in the grammar:

$$\begin{aligned} E &\rightarrow E + T \mid T, \\ T &\rightarrow T * F \mid F, \\ F &\rightarrow id. \end{aligned}$$

E will be considered as the start symbol.

The general form of a valid production is

String of terminals and nonterminals $\rightarrow$ a finite string of terminals and/or non-terminals

where the strings in left hand side can consist of only non-terminals, or any combination of terminals and non-terminals. At least one nonterminal is mandatory in LHS of any valid production. While, the right hand side of a production consist of only terminal, or non-terminals, or any combination of terminals and non-terminals or even the empty string denoted by ϵ or $\wedge$. To represent a grammar there will be at least one production that has non-terminal S in its left side. The following are some examples of valid productions:

$$\begin{aligned} aS &\rightarrow ASb \\ Ab &\rightarrow aSb \\ B &\rightarrow Aba \\ AB &\rightarrow aba \end{aligned}$$

❖ Good to Know!

A production $A \rightarrow a$, is read as A to a . And the production $B \rightarrow bA \mid c$, is read as B to bA and B to c . It can also be assumed to be union of B to bA and B to c .

If $G = (V_N, \Sigma, P, S)$ is a grammar, where $V_N = \{S\}$, $\Sigma = \{a, b\}$, and $P = \{S \rightarrow aSb, S \rightarrow ab\}$ then $V_N \cup \Sigma = \{S, a, b\}$. If α and β are strings on $V_N \cup \Sigma$, then we say α derives β . It is denoted by

$$\alpha \xRightarrow[G]{*} \beta$$

The symbol $*$ indicates that more than one substitutions are applied to derive string β . The symbol G indicates that string β is derived by using production of grammar G . The combination of $*$, $\Rightarrow$ and G (written as $\xRightarrow[G]{*}$) denotes the reflexive-transitive closure of relation in $(V_N \cup \Sigma)^*$. The $(V_N \cup \Sigma)^*$ denotes any string obtained by any combination of elements of V_N and Σ in any order and any number of times.

Language Generated by Grammar The language generated by a grammar G (denoted by $L(G)$) is defined as $L(G) = \{s \in \Sigma^* \mid S \xRightarrow[G]{*} s\}$ where s is a string formed by elements of only Σ , i.e., terminals and,

S is the start symbol. The elements of $L(G)$ are called sentences. In other words $L(G)$ is the set of all terminals strings derived from the start symbol S .

If $\{S \xRightarrow{*}_G s\}$ then s is called a sentential form. The elements of $L(G)$ are sentential forms but the reverse is not true.

The language generated by a context free grammar is sometimes called the language defined by the context free grammar, the language produced by the context free grammar, or the language derived from the context free grammar.

Example 3.5

Derive the language from the following productions representing the grammar

$$G = (\{S\}, \{a, b\}, P, S),$$

where P consists of

$$S \rightarrow aSb, S \rightarrow \wedge.$$

Solution From the production $S \rightarrow \wedge$, we can say that the smallest string generated by the grammar G is empty string that is $\wedge$. The next string generated by the grammar G is 'ab'. It is generated by substituting ' $\wedge$ ' in the right hand side of first productions on place of non-terminal S .

If we substitute aSb in the right hand side of first productions n times on place of S , we will have

$$S \Rightarrow aa...aSb.....bbb \text{ (or } a^n Sb^n \text{)}$$

in which total number of a 's and b 's are same. Let us substitute $S \rightarrow \wedge$, we have

$$S \Rightarrow a^n b^n$$

The smallest value of n must be 0 (zero), in that case there will be empty string, which is the smallest string represented by the grammar G . Therefore the language $L(G)$ derived by the grammar G is, $L(G) = \{a^n b^n | n \geq 0\}$.

Example 3.6

Let $G = (\{S\}, \{a, b\}, P, S)$, where P is defined as

$$\begin{aligned} S &\rightarrow a, \quad S \rightarrow b \\ S &\rightarrow aS, \quad S \rightarrow bS. \end{aligned}$$

Find the language $L(G)$ derived by the grammar G .

Solution The smallest string generated by the grammar G is either a or b . In the right hand side of productions $S \rightarrow aS$ and $S \rightarrow bS$, S can be replaced by a , b , aS or bS in any order. Hence all strings having any combination of a 's and b 's can be derived except the empty string. Therefore,

$$\begin{aligned} L(G) &= (a + b)^* - \wedge \\ &= (a + b)^+. \end{aligned}$$

To show $(a + b)^+ \subseteq L(G)$, let us consider any string $a_1 a_2 a_3 a_4 \dots a_n$, where each a_k is either a or b . The first production in the derivation of $a_1 a_2 a_3 a_4 \dots a_n$, is $S \rightarrow aS$ or $S \rightarrow bS$ according to the value of $a_1 = a$ or $a_1 = b$.

Other productions can be obtained in the same way. The last productions is $S \rightarrow a$ or $S \rightarrow b$ according to the value of $a_n = a$ or $a_n = b$.

Example 3.7

Let G be the grammar having productions

$$S \rightarrow aS \mid aA \mid a, A \rightarrow aAb \mid ab$$

Give the concise description of $L(G)$.

Solution We observe that the symbol A generates $a^n b^n$ for all $n \geq 1$. The rules for S using aA and a simply add another 'a' in front of $a^n b^n$, so that it gives the strings of the form $aa^n b^n$ for all $n \geq 0$. The rule aS for S allows an arbitrary number of a 's to be added at beginning.

Therefore,

$$L(G) = \{a^m b^n \mid m > n \geq 0\}$$

Here, minimum value of m is 1, so the smallest string in the language is a , i.e. $a \in L(G)$. This 'a' can be generated by $S \Rightarrow a$ directly.

Example 3.8

Following are the productions of a given grammar G :

$$S \rightarrow aABa, A \rightarrow baABb$$

$$B \rightarrow Aab, aA \rightarrow baa, bBb \rightarrow abab.$$

Test whether the string $s = ba^2b^2aba^3b^2aba$ is in language $L(G)$

Solution We will start with the production having start symbol S in its left hand side. Such production is $S \rightarrow aABa$. At every stage we will substitute a suitable production in the right hand side of the production $S \rightarrow aABa$ to derive the string s

$$S \Rightarrow aABa$$

In the string 's' we see that first symbol is b so we will have to eliminate first symbol 'a' from the right hand side of the production $S \rightarrow aABa$. For this we will substitute $aA \rightarrow baa$ because first symbol in the right hand side of this productions is b ,

$$\begin{aligned} S &\Rightarrow baaBa && \text{(After replacing } aA \text{ by } baa) \\ &\Rightarrow ba^2Ba \end{aligned}$$

first three symbols of the derivation are matched with the test string. Now we will substitute B such that the first symbol of replace string is the fourth symbol of test string or a nonterminal

$$S \Rightarrow ba^2Aaba \quad \text{(After replacing } B \text{ by } Aab)$$

Now

$$\begin{aligned} S &\Rightarrow ba^2baABbaba && \text{(After replacing } A \text{ by } baABb) \\ &\Rightarrow ba^2bbaaBbaba && \text{(After replacing } aA \text{ by } baa) \\ &\Rightarrow ba^2b^2aaAabbaba && \text{(After replacing } B \text{ by } Aab) \\ &\Rightarrow ba^2b^2abaaabbba && \text{(After replacing } aA \text{ by } baa) \\ &\Rightarrow ba^2b^2aba^3b^2aba \end{aligned}$$

Therefore

$$s \in L(G)$$

Example 3.9

If G is a grammar given by the following productions

$$S \rightarrow AB$$

$$A \rightarrow BB$$

$$B \rightarrow AA$$

Find the language generated by grammar G .

Solution Since $\Sigma = \{\phi\}$, because all three productions have no terminals on the right-hand side. Hence $L(G) = \{\phi\}$.

Example 3.10

Let $G = (\{S, A\}, \{a, b, c\}, P, S)$, where P contains $S \rightarrow aSAc$, $S \rightarrow abc$, $cA \rightarrow Ac$, $bA \rightarrow bb$. Find the language generated by grammar G .

Solution As the grammar has the productions $S \rightarrow abc$, we have smallest string abc generated by the grammar G , i.e., $abc \in L(G)$.

If we apply $S \rightarrow aSAc$, $n - 1$ times in the right hand side of this production, we will have,

$$S \Rightarrow^* a^{n-1} A (Ac)^{n-1}$$

If we apply $S \rightarrow abc$, we will have,

$$S \Rightarrow a^n bcA (Ac)^{n-1}$$

If we apply $cA \rightarrow Ac$ several times, we will have

$$S \Rightarrow^* a^n bA^{n+1} c^n$$

Now by applying $bA \rightarrow bb$, $n - 1$ times, we will have,

$$S \Rightarrow^* a^n b^n c^n$$

Therefore the language generated by the grammar G is:

$$L(G) = \{a^n b^n c^n | n \geq 1\}$$

Example 3.11

Determine the language generated by grammar having following productions:

$$S \rightarrow aS \mid A$$

$$A \rightarrow bB \mid b$$

$$B \rightarrow cC \mid \wedge$$

$$C \rightarrow \wedge$$

Solution The production $S \rightarrow aS$ can generate

$$S \Rightarrow^{n-1} a^n S$$

when $S \rightarrow aS$ is applied $n - 1$ times.

To eliminate S from left side of the derivation we substitute $S \rightarrow A$ to get

$$S \Rightarrow a^n A.$$

Now there are two options to eliminate A , first is $A \rightarrow bB$ and second $A \rightarrow b$, this way we have

$$S \Rightarrow a^n S$$

$$\Rightarrow a^n A \begin{cases} \nearrow a^n b \text{ (here } n \geq 0, \text{ because } S \Rightarrow A \Rightarrow b) \\ \searrow a^n bB \end{cases}$$

B can be replaced by either cC or $\wedge$ to get

$$S \Rightarrow a^n bB \begin{cases} \nearrow a^n bcC \\ \searrow a^n b \end{cases}$$

we have only one option to eliminate C , and the option is $\wedge$, to generate

$$S \Rightarrow a^n bc \wedge$$

$$\Rightarrow a^n bc$$

Therefore,

$$L = \{a^n b \mid n \geq 0\} \cup \{a^n bc \mid n \geq 0\}$$

Example 3.12

Determine the language generated by grammar having productions

$$S \rightarrow aAb \mid ab, A \rightarrow bAa \mid \wedge.$$

Solution The production $S \rightarrow ab$, can generate

$$S \Rightarrow ab$$

which is the smallest string in the language. The production $A \rightarrow bAa \mid \wedge$ can generate

$$S \Rightarrow b^n A n^n$$

when $A \rightarrow bAa$ is applied $n - 1$ times to itself. Finally, by substituting $A \rightarrow \wedge$, A is capable to generate

$$A \Rightarrow b^n a^n$$

By substituting $A \Rightarrow b^n a^n$ in production $S \rightarrow aAb$ we get.

$$S \Rightarrow ab^n a^n b$$

As we have seen that the smallest string represented by S is 'ab', therefore the value of n in $ab^n a^n b$ is minimum zero and maximum infinite.

Therefore,

$$L = \{ab^n a^n b \mid n \geq 0\}$$

Example 3.13

Determine the language generated by a grammar having productions

$$S \rightarrow A \mid B \mid C \mid D$$

$$A \rightarrow aA \mid aa$$

$$B \rightarrow bB \mid bb$$

$$C \rightarrow cC \mid cc$$

$$D \rightarrow dD \mid dd.$$

Solution The language of this grammar is the union of all strings represented by A, B, C and D . The productions for A, B, C and D are identical in nature. Here,

A generates $\{a^n \mid n \geq 2\}$

B generates $\{b^m \mid m \geq 2\}$

C generates $\{c^l \mid l \geq 2\}$

D generates $\{d^p \mid p \geq 2\}$

This way, the language generated by given grammar is:

$$L = \{a^n \mid n \geq 2\} \cup \{b^m \mid m \geq 2\} \cup \{c^l \mid l \geq 2\} \cup \{d^p \mid p \geq 2\}$$

or

$$L = \{a^n \cup b^m \cup c^l \cup d^p \mid n, m, l, p \geq 2\}$$

Example 3.14

Determine the language generated by a grammar having productions
 $S \rightarrow aS \mid bS \mid aa$.

Solution As we look on productions $S \rightarrow aS$ and $S \rightarrow bS$ we have options a and b as first symbol in any string. Similarly, for second, third and onward places again we have option of symbol a and b . Therefore $S \rightarrow aS$ and $S \rightarrow bS$ can generate any string that is composed of a 's and b 's, any number of times in any order followed by S . This way,

$$S \Rightarrow (a, b)^* S$$

finally, S can be eliminated by aa . So we get

$$S \Rightarrow (a, b)^* aa$$

Therefore,

$$L = \{\text{set of all strings over } (a, b) \text{ ending with } aa\}$$

Example 3.15

Determine the language generated by grammar $G = (\{S, A\}, \{a, b\}, P, S)$, where P is defined as

$$A \rightarrow aA \mid bA \mid \wedge, S \rightarrow aAb.$$

Solution The A -productions are capable of generating all strings over $\{a, b\}$, i.e. $(a, b)^*$. If the strings represented by A are substituted in production $S \rightarrow aAb$, we get all strings over $\{a, b\}$ starting with a and ending with b .

Therefore,

$$\begin{aligned} L &= \{\text{set of all strings over } \{a, b\} \text{ not starting with } b \text{ and not ending with } a\} \\ &= \{s \in (a, b)^* \mid s \text{ is starting with } a \text{ and ending with } b\} \\ &= \{a(a, b)^*b\}. \end{aligned}$$

Example 3.16

Determine the language generated by a grammar having productions.

$$S \rightarrow 0S11 \mid \wedge.$$

Solution When we substitute $S \rightarrow 0S11$, again and again in right side of the production $S \rightarrow 0S11$, then after $n - 1$ substitutions we get

$$S \Rightarrow^{n-1} 0^n S(11)^n$$

Finally, we have to substitute $S \rightarrow \wedge$ to eliminate S from right side of $S \Rightarrow^{n-1} 0^n S(11)^n$. We get

$$\begin{aligned} S &\Rightarrow 0^n \wedge (11)^n \\ &\Rightarrow 0^n (11)^n \\ &\Rightarrow 0^n (1)^{2n} \end{aligned}$$

As we have $S \rightarrow \wedge$ production, therefore the smallest string of the language is $\wedge$. This way the minimum value of n must be zero.

Therefore,

$$L = \{0^n 1^{2n} \mid n \geq 0\}$$

Example 3.17

Determine the language generated by a grammar having productions
 $S \rightarrow aSbb \mid aabbb$.

Solution When we substitute $S \rightarrow aSbb$ again and again to itself, we get

$$S \Rightarrow a^{n-1} S (bb)^n \text{ (which is equivalent to } a^n S b^{2n} \text{)}$$

To eliminate S from right side of $S \Rightarrow a^{n-1} S (bb)^n$, we will have to substitute $S \rightarrow aabbb$. This way we get

$$\begin{aligned} S &\Rightarrow a^n S (b)^{2n} \\ &\Rightarrow a^n aabbb (b)^{2n} \\ &\Rightarrow a^{n+2} b^{2n+3} \end{aligned}$$

As the production $S \rightarrow aabbb$, says that smallest string in the language is 'aabbb', therefore the minimum value of n is zero.

$$L = \{a^{n+2} b^{2n+3} \mid n \geq 0\}$$

Example 3.18

Find the grammar generating the language $L(G) = \{a^n b a^n \mid n \geq 1\}$.

Solution For this language $L(G)$, there must be a production of the form

$$\text{start symbol} \rightarrow (\text{terminal } a) \text{ start symbol } (\text{terminal } a)$$

We take production

$$S \rightarrow aSa.$$

If we substitute aSa on the place of S in the right hand side of the production $n - 1$ times, we have

$$S \Rightarrow a^{n-1} S a^{n-1}$$

now substitute $S \rightarrow aba$, we have

$$S \Rightarrow a^n b a^n$$

aba is the smallest string given by $L(G)$ for $n = 1$. In this case we can not substitute $S \rightarrow aSa$ in the right hand side of this production exactly ' n ' times, because we will have

$$S \Rightarrow a^n S a^n$$

If we substitute $S \rightarrow b$, then the smallest string in the language is ' b ' which is true when $n \geq 0$, but in the given language n is greater than or equal to 1.

Therefore, the grammar generating the language is defined as (V_N, Σ, P, S) , where

$$V_N = \{S\},$$

$$\Sigma = \{a, b\}$$

$$P = \{S \rightarrow aSa, S \rightarrow aba\}$$

$$S = \text{start symbol.}$$

Example 3.19

Determine the grammar G generating the language defined as:
 $L = \{\text{set of all strings over } \{0, 1\} \text{ starting with } 00 \text{ and ending with } 11\}$.

Solution The regular expression corresponding to the language L is $00(0, 1)^*11$. We can define the grammar for this expression by dividing it into three parts (say A, B, C) as $A = 00, B = (0, 1)^*$ and $C = 11$.

Therefore,

$$S \rightarrow ABC$$

$$A \rightarrow 00$$

$$B \rightarrow 0B \mid 1B \mid \wedge$$

$$C \rightarrow 11$$

is the required grammar.

The other three solutions to this problem are:

$$(i) \quad S \rightarrow 00A11, A \rightarrow 0A \mid 1A \mid \wedge$$

$$(ii) \quad S \rightarrow 00A, A \rightarrow 0A \mid 1A \mid 11$$

$$(iii) \quad S \rightarrow 00A, A \rightarrow B11, B \rightarrow 0B \mid 1B \mid \wedge$$

Example 3.20

Determine the grammar that does not generate all strings over $\{0,1\}$ ending with substring 11.

Solution The Set of all strings over $\{0,1\}$ not ending with 11 can be represented as $(0,1)^*(00, 01, 10)$.

In other words, this is the language of all strings over $\{0,1\}$ ending with either 00, or 01, or 10. The required grammar could have the productions as following:

$$S \rightarrow 0S \mid 1S$$

to generate all strings over $\{0,1\}$ ending with S . If we substitute 00, 01, or 10 on place of S then our purpose is solved. Thus the required grammar is:

$$S \rightarrow 0S \mid 1S \mid 00 \mid 01 \mid 10$$

But this grammar does not generate the strings $\wedge, 0$, and 1 , which are also the part of language. So the above grammar can be modified as

$$S' \rightarrow S \mid 0 \mid 1 \mid \wedge, S \rightarrow 0S \mid 1S \mid 00 \mid 01 \mid 10$$

to be the resultant grammar with S' as start symbol.

Therefore, the required grammar is

$$G = (\{S, S'\}, \{0, 1\}, \{S' \rightarrow S \mid 0 \mid 1 \mid \wedge, S \rightarrow 0S \mid 1S \mid 00 \mid 01 \mid 10\}, S')$$

Example 3.21

If $L_1 = \{a^n b c^n \mid n \geq 0\}$ and $L_2 = \{a^m b^{2m} \mid m \geq 1\}$ find grammar G such that (i) $L(G) = L_2 L_1$, (ii) $L(G) = (L_2 + L_1)^*$.

Solution The grammar G_1 corresponds to the language L_1 is:

$$G_1 = (V'_N, \Sigma', P', S')$$

where,

$$V'_N = \{S'\}$$

$$\Sigma' = \{a, b, c\}$$

$$P' = \{S' \rightarrow aS'c \mid b\}$$

The grammar G_2 corresponds to the language L_2 is:

$$G_2 = (V'_N, \Sigma'', P'', S'')$$

where,

$$V'_N = \{S''\}$$

$$\Sigma'' = \{a, b, c\}$$

$$P'' = \{S'' \rightarrow aS'bb \mid abb\}$$

- (i) The language defined as $L(G) = L_2L_1$, is the concatenation of languages L_2 and L_1 . So the language $L(G)$ is the language of strings of L_2 followed by strings of L_1 . Therefore, the required grammar is

$$G = (V_N, \Sigma, P, S)$$

where,

$$V_N = \{S, S', S''\}$$

$$\Sigma = \{a, b, c\}$$

$$P = \{S \rightarrow S'S', S' \rightarrow aS'bb \mid abb, S' \rightarrow aS'c \mid b\}$$

- (ii) The language defined as $L(G) = (L_2 + L_1)^*$ is the closure of union of languages L_1 and L_2 . First we determine the productions that generate $L_1 + L_2$, then we determine the productions to find its closure.

The grammar, $G = (V_N, \Sigma, P, S)$, with productions $S_1 \rightarrow S' \mid S''$ can generate $L_1 + L_2$, and the productions $S \rightarrow S S_1 \mid \wedge$, can produce the closure of strings represented by nonterminal S_1 i.e., closure of $L_1 + L_2$. Therefore, the required grammar is

$$G = (V_N, \Sigma, P, S)$$

where,

$$V_N = \{S, S_1, S', S''\}$$

$$\Sigma = \{a, b, c\}$$

$$P = \{S \rightarrow S S_1 \mid \wedge, S_1 \rightarrow S' \mid S'', S' \rightarrow aS'c \mid b, S'' \rightarrow aS'bb \mid abb\}$$

Example 3.22

Determine the grammar G for the language $L = \{a^n b^m c^n \mid m \geq 2, n \geq 1\}$.

Solution First we define the productions that can generate the strings of the form $a^n X c^n$, then we will substitute X for b^m . This way the required productions are

$$S \rightarrow aSc \mid aAc, A \rightarrow bA \mid bb$$

The production $A \rightarrow bb$ ensures the starting value of m as 2.

Example 3.23

Determine the grammar G for the language $L = \{a^n b^n c^n \mid n \geq 1\} \cup \{a^n (bc)^n \mid n \geq 1\}$.

Solution In case of infinite language there exists direct or indirect infinite loop in the productions. The production $S \rightarrow aS$, contains direct loop and, the productions set $S \rightarrow aA$, $A \rightarrow bB$, $B \rightarrow aS$ contains indirect loop. It means both the production sets generate infinite numbers of strings.

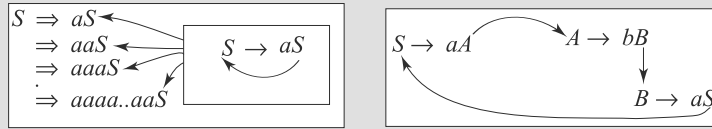


Fig. 3.1 (a) Direct loop by single production (b) Indirect loop by three productions

As we know the given language has three different symbols with minimum zero and maximum infinite occurrences. Here we define the production system that contains a direct loop and all three symbols a , b , and c appeared in the right hand side of the production with the nonterminal appearing in left side of that production.

The production

$$S \rightarrow aSBC$$

Will determine equal occurrences of a 's, b 's and c 's, when replacing B by b , C by c . Here, we will use production of the form $CB \rightarrow BC$ to maintain the correct order a 's, followed by b 's followed by c 's. If the production $CB \rightarrow BC$ is not applied then remaining productions will generate language $L = \{a^n(bc)^n | n \geq 1\}$. The following set of productions generates G :

$$S \rightarrow aSBC, CB \rightarrow BC, B \rightarrow b, C \rightarrow c$$

Example 3.24

Define grammar G for generating all positive integers and floating point values.

Solution The productions

$$A \rightarrow 0A \mid 1A \mid 2A \mid 3A \mid 4A \mid 5A \mid 6A \mid 7A \mid 8A \mid 9A$$

$$A \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

can generate any integer value from 0 to any order. If we add productions

$$S_1 \rightarrow A.B$$

$$B \rightarrow 0B \mid 1B \mid 2B \mid 3B \mid 4B \mid 5B \mid 6B \mid 7B \mid 8B \mid 9B,$$

$$B \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

Then S_1 with the help of A -productions and B -productions can produce any floating point value. The required grammar is:

$$G = (\{S, S_1, S_2, A, B\}, \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}, P, S)$$

where, P is defined as

$$P = \{S \rightarrow S_1 \mid S_2, S_2 \rightarrow B, S_1 \rightarrow A.B,$$

$$A \rightarrow 0A \mid 1A \mid 2A \mid 3A \mid 4A \mid 5A \mid 6A \mid 7A \mid 8A \mid 9A,$$

$$A \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9,$$

$$B \rightarrow 0B \mid 1B \mid 2B \mid 3B \mid 4B \mid 5B \mid 6B \mid 7B \mid 8B \mid 9B,$$

$$B \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9\}$$

Example 3.25

Determine the grammar for the language $L = \{a^n b^m \mid m + n \text{ is even}\}$.

Solution We define the grammar having production $S \rightarrow AB$. The variable A will represent a 's and B will represent b 's. The value of $m + n$ is even if and only if

- (i) m and n both are even,
- (ii) m and n both are odd.

Therefore we can use productions $S \rightarrow AB, S \rightarrow CD$. The production $S \rightarrow AB$, for case (i) and $S \rightarrow CD$, for case (ii). The way we define A and B is

$$A \rightarrow aaA \mid \wedge, B \rightarrow bbB \mid \wedge \text{ (} A \text{ generates } a^{2n} \text{ and } B \text{ generates } b^{2n} \text{)}$$

and C, D are defined as

$$C \rightarrow aaC \mid a, D \rightarrow bbD \mid b \text{ (} C \text{ generates } a^{2n+1} \text{ and } D \text{ generates } b^{2n+1} \text{)}$$

Thus, the required grammar is

$$G = (\{S, A, B, C, D\}, \{a, b\}, P, S)$$

where, P is defined as

$$P = \{S \rightarrow AB, S \rightarrow CD, A \rightarrow aaA \mid \wedge, B \rightarrow bbB \mid \wedge, C \rightarrow aaC \mid a, D \rightarrow bbD \mid b\}.$$

Example 3.26

Construct the grammar G for set L of all palindromes over $\{0, 1\}$.

Solution A set of all palindromes contains even as well as odd length strings. For the construction of grammar G to generate the set of all palindromes, we can use recursive definitions in following cases:

- (i) Any string of zero length ($\wedge$) is a palindrome
- (ii) Any single symbol is palindrome i.e., 0, 1.
- (iii) If p is a palindrome then $0p0$ and $1p1$ will also be palindromes.

Therefore we can define productions as:

- (i) $S \rightarrow \wedge$, which will give palindrome of zero length.
- (ii) $S \rightarrow 0 \mid 1$, which will give palindromes of length one.
- (iii) $S \rightarrow 0S0 \mid 1S1$, which will give palindromes of length two or more with productions $S \rightarrow \wedge$, and $S \rightarrow 0 \mid 1$

For all even length palindrome, productions $S \rightarrow 0S0 \mid 1S1$ will be substituted several times on place of S in right hand side of these productions and $S \rightarrow \wedge$ once. This will give strings in the form of palindromes (i.e., $\wedge, 00, 11, 0000, 0110, 1001, 1111, \dots$)

For all odd length palindromes, productions $S \rightarrow 0S0 \mid 1S1$ will be substituted several times on place of S in the right hand side of these productions and $S \rightarrow 0 \mid 1$ once. This will give odd length palindromes (i.e., 0, 1, 000, 010, 101, 111, ...).

Therefore the constructed grammar is:

$$G = (\{S\}, \{0, 1\}, P, S)$$

where, P is the set of following productions

$$P = \{S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \wedge\}$$

Example 3.27

Determine grammar G to generate all palindromes over $\{a, b\}$ starting and ending with a .

Solution The productions

$$A \rightarrow aAa \mid bAb \mid a \mid b \mid \wedge$$

generates all palindromes over $\{a, b\}$.

If we substitute variable ' A ' in the production

$$S \rightarrow aAa$$

with S as start symbol, then this set of production will generate all palindromes starting and ending with a .

Therefore, the required grammar is,

$$G = (\{S, A\}, \{a, b\}, \{S \rightarrow aAa \mid a, A \rightarrow aAa \mid bAb \mid a \mid b \mid \wedge\}, S)$$

Example 3.28

Construct grammar G generating the language

$$L(G) = \{x \in \{0, 1\}^* \mid n_0(x) \neq n_1(x)\}$$

where,

$n_0(x)$ = occurrences of 0's in string x ,

$n_1(x)$ = occurrences of 1's in string x .

Solution Let L_0 and L_1 be the languages of strings with more 0's than 1's and, more 1's than 0's respectively.

The language $L(G)$ is equal to $L_0 \cup L_1$, we concentrate first on finding a grammar G_0 generating the language L_0 .

Clearly, $0 \in L_0$ and for any $x \in L_0$, both $x0$ and $0x$ are in L_0 . This suggests the production:

$$S \rightarrow 0 \mid S0 \mid 0S$$

Now we also need to add 1's to our strings. We cannot expect that the addition of one 1 to a string of L_0 will always produce a string of L_0 . However, if we have two strings in L_0 , then the concatenation of those strings produces a string with at least two more 0's than 1's, and therefore adding a single 1 will still yield a string of L_0 . We can add it at the left, at the right, or in between of two. The corresponding productions will be:

$$S \rightarrow 1SS \mid SS1 \mid S1S$$

It is not so hard to derive any string of L_0 by using the productions:

$$S \rightarrow 1SS \mid SS1 \mid S1S \mid S0 \mid 0S \mid 0$$

In the converse direction, we can do something better; If G_0 is the grammar with following productions:

$$S \rightarrow 1SS \mid SS1 \mid S1S \mid 0S \mid 0$$

By this set of productions every string in L_0 can be derived in grammar G_0 . A similar set of productions can be designed for language L_1 . Thus the productions for grammar G_1 include:

$$S \rightarrow 0SS \mid SS0 \mid SOS \mid 1S \mid 1$$

This can also be done by just replacing 0 by 1 and 1 by 0 in set of productions designed for G_0 .

Now we have to combine all these designed productions. Let 'S' be the start symbol of combined grammar G and A be the start symbol of grammar G_0 generating the language L_0 ; and variable B be the start of grammar G_1 generating the language L_1 . Thus, the resulting grammar G has the productions:

$$S \rightarrow A \mid B$$

$$A \rightarrow 1AA \mid AA1 \mid A1A \mid 0A \mid 0$$

$$B \rightarrow 0BB \mid BB0 \mid B0B \mid 1B \mid 1$$

3.7

CLASSIFICATION OF GRAMMARS AND LANGUAGES

In practice, formal grammars are supposed to consist of a finite set of terminal symbols (the letters of the words in the formal languages), a finite set of non-terminal (also called variables) symbols, a set of production rules with a left and a right-hand side consisting of a word of these symbols, and a start symbol. A rule may be applied to a word by replacing the left-hand side by the right-hand side. A derivation is a sequence of rule applications. Such a grammar defines the formal languages of all words consisting exclusively of terminal symbols that can be reached by a derivation from the start symbol.

As we have discussed earlier non-terminals are usually represented by upper case (capital) letters, terminals by lower case (small) letters, and another symbol called start symbol is represented by symbol S (upper case) which is a nonterminal (i.e., $S \in V_N$). The start symbol S defines language of all words in any form, for example $a^n b^m$ is some language from $\Sigma = \{a, b\}$.

3.7.1 Types of Grammars

The Chomsky hierarchy consists of the following four levels:

Type-0 grammars These are unrestricted grammars which include all formal grammars. Any kind of productions can be included in type-0 grammar. They generate exactly all languages (Type-0 language) that can be recognised by a Turing machine. The language that is recognised by a Turing machine is defined as all the strings on which it halts. These languages are also called the recursively enumerable languages. Note that this is different from the recursive languages which can be decided by an always halting Turing machine.

Type-1 grammars These are also called context-sensitive grammars (CSGs), and used to derive the context-sensitive languages (CSLs). These grammars have productions rules of the form

$$\alpha A \beta \rightarrow \gamma$$

where α is the left context and β is the right context with $A \in V_N$ and, $\beta \in (V_N \cup \Sigma)^*$. The string α and β may be empty but γ must be non empty $\gamma \in (V_N \cup \Sigma)^+$. It also includes the production $S \rightarrow \wedge$ but if it is, then it must not have an S on the right hand side of any production. These languages are exactly all languages that can be recognised by a nondeterministic Turing machine, or linear bounded automaton (LBA). All natural languages are context-sensitive in nature.

A type-1 production $\alpha A \beta \rightarrow \gamma$, does not increase the length of the working string. In other words, $|\alpha A \beta| \leq |\gamma|$, $\alpha \neq \Lambda$. But if $\alpha \rightarrow \beta$ is a production such that $|\alpha| \leq |\beta|$, then it is not necessary that it is a type-1 production. The grammar having productions $S \rightarrow 0SBA \mid 01A$, $AB \rightarrow BA$, $1B \rightarrow 11$, $1A \rightarrow 10$, $0A \rightarrow 00$ is a context sensitive grammar generating context sensitive language $L = \{0^n 1^n 0^n \mid n \geq 1\}$. It is necessary that at least one production in context sensitive grammar should have left, right or both the contexts.

The production $AB \rightarrow BA$, can be treated in following two ways:

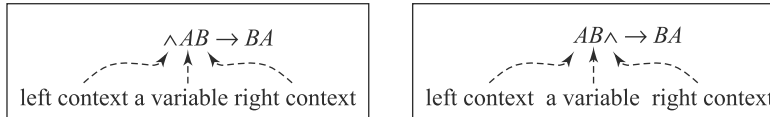


Fig. 3.2 Illustration of left and right context in a context sensitive grammar

In the first diagram left context is Λ , and right context is B ; in the second diagram left context is A , and right context is Λ .

Type-2 Grammars These are also called context-free grammars (CFGs), therefore these grammars generate context-free languages (CFLs), which are also known as type-2 languages. Context free languages are the languages which can be accepted or recognised by a push down automaton or some higher machines like Linear Bounded Automaton and Turing machine. Type-2 grammars are defined by the productions of the form

$$A \rightarrow \alpha$$

with $A \in V_N$ and $\alpha \in (V_N \cup \Sigma)^*$ i.e., A is nonterminal or variable, and α is any string of terminals and nonterminals. There is no left or right context in the left hand side of the productions. It indicates that there is no restriction in right hand side of the productions. The context-free languages are the theoretical basis for the syntax of most programming languages.

The grammar represented by productions $S \rightarrow aSb \mid \Lambda$, is context free grammar because the left hand side of the production does not contain any context, i.e., there is neither left nor right context.

Type-3 Grammars These grammars are also called regular grammars (RGs). The languages generated by regular grammars are called regular languages (RLs) or type-3 languages. Type-3 grammars restrict their productions to a single non-terminal on the left-hand side and the right side consisting of a single terminal, possibly followed by a single nonterminal. For example,

$$A \rightarrow a, A \rightarrow aB$$

are type-3 productions, where $A \in V_N$ and $a \in \Sigma$. The production $S \rightarrow \Lambda$ is also allowed in this grammar if S does not appear on the right side of any production. Regular languages (Type-3 language) are exactly all languages that can be accepted or recognised by a finite state automaton.

3.8

LANGUAGES AND THEIR RELATIONS

In this section of this chapter, we discuss the relationship among the classes of languages classified by Noam Chomsky.

Let L_0 , L_{CSL} , L_{CFL} , and L_{RL} denote the family of type-0 (or unrestricted languages), context sensitive languages, context free languages and regular languages, respectively. To show the relation among the classes of languages we discuss following properties of languages:

Property 1 According to the definition we know that the regular language is subset of context free language (i.e., $L_{RL} \subseteq L_{CFL}$), the context free language is subset of context sensitive language (i.e., $L_{CFL} \subseteq L_{CSL}$), the context sensitive language is subset of type-0 language (i.e., $L_{CSL} \subseteq L_0$).

Property 2 The regular language is subset of type-0 language (i.e., $L_{RL} \subseteq L_0$), the context free language is subset of type-0 language (i.e., $L_{CFL} \subseteq L_0$). The inclusion relation is not immediate as we follow $A \rightarrow \wedge$ in context free grammar when $A \neq S$, but not in context sensitive grammars.

A context free grammar G with productions of the form $A \rightarrow \wedge$ is equivalent to a context free grammar G_1 which has no null production (see in chapter context free language)

A production of the form $A \rightarrow \wedge$ is called a null production. Also, when grammar G_1 has the production $S \rightarrow \wedge$ and S does not appear in the right hand side of any production of G_1 , then G_1 is context-sensitive grammar. This proves that

$$L_{CFL} \subseteq L_{CSL}$$

Property 3 On the basis of property 1 and property 2 we have inclusion relation among L_0 , L_{CSL} , L_{CFL} , and L_{RL} as

$$L_{RL} \subseteq L_{CFL} \subseteq L_{CSL} \subseteq L_0$$

Property 4 $L_{RL} \subset L_{CFL} \subset L_{CSL} \subset L_0$ where $\subset$ is proper set, which defines that all contents of A are present in B but not the vice-versa if there is a relation $A \subset B$.

3.8.1 Recursively Enumerable Languages

A recursively enumerable set is a set S for which there is a procedure to determine whether or not a given element belongs to set S . According to the definition a set S is recursive if there is an algorithm to determine whether or not a given element belongs to set S .

Theorem 3.1 Every context-sensitive language is recursive.

Proof Let $G = (V_N, \Sigma, P, S)$ be a context-sensitive grammar and a word ' s ' in Σ^* of length n . We can test whether ' s ' is in $L(G)$ or not as follows:

We construct a transition graph whose vertices are the strings in $(V_N \cup \Sigma)^*$ of length n or less. We put an edge from α to β if $\alpha \Rightarrow \beta$. The paths in the transition graph correspond to derivations in grammars G , and string ' s ' $\in L(G)$ if and only if there is a path from the vertex for S to the vertex for ' s '. We can use any number of path-finding algorithms to decide whether such a path exists or not.

Theorem 3.2 There is a recursive language that is not Context Sensitive Language.

Proof For the proof of the theorem the study of halting of Turing machines for enumeration of some set is a must. So we will discuss this in Turing machines.

3.9

OPERATIONS ON LANGUAGES

Theorem 3.3 Each of the classes L_0 , L_{CSL} , L_{CFL} , and L_{RL} is closed under union.

Proof Suppose L_1 and L_2 are languages of the same type i . Let $G_1 = (V'_N, \Sigma', P', S')$ be the grammar from which the language L_1 is generated and $G_2 = (V''_N, \Sigma'', P'', S'')$ be the grammar from which the language L_2 is generated. The grammars G_1 and G_2 both are of type i . So any production in grammar G_1 or G_2 will be either $\alpha \rightarrow \beta$ or $A \rightarrow a$, where, $\alpha, \beta, A \in (V'_N \cup V''_N)$ and $a \in \Sigma$.

Let us assume that $V'_N \cap V''_N = \phi$ (this can be achieved by renaming the nonterminals of V''_N if they occur in V'_N).

Let us define a new grammar G_n such that:

$$G_n = (\{V'_N \cup V''_N \cup \{S\}\}, \Sigma' \cup \Sigma'', P_n, S)$$

where S is a new start symbol, such that $S \notin V'_N \cup V''_N$.

The set of productions P_n in G_n is defined as

$$P_n = \{S \rightarrow S_1, S \rightarrow S_2\} \cup \{P' \cup P''\}$$

Now we have to prove that the language generated by G_n i.e., $L(G_n)$ is equivalent to the union of languages L_1 and L_2 that is:

$$L(G_n) = L_1 \cup L_2$$

If s is a string that belongs to language $L_1 \cup L_2$ then the string ' s ' can be derived by grammar G_1 , such that or, by grammar G_2 such that .

Therefore,

$$S \xRightarrow{G_n} S_1 \text{ and } S_1 \xRightarrow{G_1} s \text{ or } S \xRightarrow{G_n} S_2 \text{ and } S_2 \xRightarrow{G_2} s$$

It means, $s \in L(G_n)$.

Hence,

$$L_1 \cup L_2 = L(G_n).$$

For the proof of $L(G_n) = L_1 \cup L_2$ let us consider a derivation of string ' s '. The first step is to derive $S \Rightarrow S_1$ or $S \Rightarrow S_2$. $S \Rightarrow S_1$ is derived first then S_1 is changed in subsequent steps. As we have assumed that $V'_N \cap V''_N = \phi$, these steps should involve only variables of V'_N and the productions we apply are in P' . Therefore, s is derived by G_n ,

$$S \xRightarrow{*}_{G_n} s$$

Similarly, if the first step is to derive $S \Rightarrow S_2$, then $S \xRightarrow{*}_{G_n} S_2 \xRightarrow{*}_{G_n} s$ Hence,

$$L(G_n) = L_1 \cup L_2$$

Also $L(G_n)$ is of type-0 or type-2 according as L_1 and L_2 are of type-0 or type-2. If $\wedge \notin L_1 \cup L_2$, then $L(G_n)$ is of type-3 or type-1 according as L_1 and L_2 are of type-1 or type-3.

Let us suppose $\wedge \in L_1$. In this case the grammar G_n is defined as:

$$G_n = (\{V'_N \cup V''_N \cup \{S, S''\}\}, \Sigma' \cup \Sigma'', P_n, S')$$

where

- (i) S' = a new start symbol, i.e., $S' \notin V'_N \cup V''_N \cup \{S\}$,
- (ii) $P_n = P' \cup P'' \cup \{S' \rightarrow S, S \rightarrow S_1, S \rightarrow S_2\}$

Therefore, $L(G_n)$ is of type-1 or type-3 according as L_1 and L_2 are of type-1 or type-3.

Let us suppose now $\wedge \in L_2$. The proof is similar as in case of $\wedge \in L_1$.

Theorem 3.4 Each of the classes L_0 , L_{CSL} , L_{CFL} , and L_{RL} is closed under concatenation.

Proof Suppose L_1 and L_2 are two languages of same type say i . Let the grammar $G_1 = (V''_N, \Sigma', P', S')$ generates the language L_1 and the grammar $G_2 = (V''_N, \Sigma'', P'', S'')$ generates the language L_2 . Therefore the grammars G_1 and G_2 will also be of same type i.e., type- i . The concatenation of languages L_1 and L_2 is L_1L_2 . Now our goal is to prove that the type of language L_1L_2 is type- i .

Let us construct a new grammar G_c that will generate the language L_1L_2 :

$$G_c = (\{V'_N \cup V''_N \cup \{S\}\}, \Sigma' \cup \Sigma'', P_c, S)$$

where, S is the start symbol such that $S \notin V'_N \cup V''_N$ and $P_c = P' \cup P'' \cup \{S \rightarrow S'S''\}$.

If the language generated by grammar G_c is $L(G_c)$ then we have to prove that $L(G_c) = L_1L_2$. If $w_1 \in L_1$, $w_2 \in L_2$ and $w_c \in L_1L_2$, then $w_1w_2 \in L_1L_2$.

The string w_1 is derived by grammar G_1 such that, $S_1 \xRightarrow{*}_{G_1} w_1$, and the string w_2 is derived by the grammar G_2 such that $S_2 \xRightarrow{*}_{G_2} w_2$.

Now, according to grammar G_c ,

$$S \xRightarrow{G_c} S_1S_2 \quad \text{and} \quad S_1S_2 \xRightarrow{G_c} w_1w_2$$

Hence,

$$L_1L_2 \subseteq L(G_c)$$

If $w_c \in L(G_c)$, then to generate the string w_c the first step is $S \Rightarrow S_1S_2$. We assume that $V'_N \cap V''_N = \emptyset$ and the productions in grammar G_1 or G_2 involve only nonterminals, $w_c = w_1w_2$. Therefore, $L_1L_2 = L(G_c)$. Also grammar (obtained by concatenation of G_1 and G_2) G_c is of type-0 or type-2 according as G_1 and G_2 are of type-2. The above construction is sufficient when G_1 and G_2 are also of type-3 or type-1 with $\wedge \notin L_1 \cup L_2$.

Let G_1 and G_2 be the type-1 or type-3 and $\wedge \in L_1$, $\wedge \in L_2$. Let us define languages L_A and L_B such that

$$L_A = L_1 - \{\wedge\}, \text{ and } L_B = L_2 - \{\wedge\}$$

Then L_1L_2 is defined as:

$$L_1L_2 = \begin{cases} L_AL_B \cup L_B & \text{if } \wedge \in L_1, \text{ but } \wedge \notin L_2 \\ L_AL_B \cup L_A & \text{if } \wedge \in L_2, \text{ but } \wedge \notin L_1 \\ L_AL_B \cup L_A \cup L_B \cup \{\wedge\} & \text{if } \wedge \in L_1, L_2 \end{cases}$$

As, we have shown in the previous theorem that L_{CSL} and L_{RL} are closed under union, so concatenation of L_1 and L_2 is of type-1 or type-3 according as L_1 and L_2 are of type-1 or type-3.

3.10

CHOMSKY HIERARCHY

The Chomsky hierarchy is a containment hierarchy of classes of formal grammars that generate formal languages. This hierarchy was described by Noam Chomsky in 1956. Chomsky and Miller proved the equivalence of regular grammars and regular sets. The four classes of languages are often called the Chomsky hierarchy, after Noam Chomsky. These classes of languages are defined as potential models of natural languages.

A regular grammar is a context free grammar by default, as we say the left hand side of all the productions in regular grammar is free from any context. Additionally, it is not necessary that a grammar which is context free will also be regular. This family of formal languages can be obtained by regular expressions. Regular languages are commonly used to define search patterns and the lexical structure of the programming languages.

A grammar $G = (V_N, \Sigma, P, S)$ is monotonic grammar (or length-increasing grammar), if every production in set P is of form (i) $\alpha \rightarrow \beta$ with $|\alpha| \leq |\beta|$ or (ii) $S \rightarrow \wedge$.

In the case (ii), S must not appear in the right-hand side of any production in productions set P .

Figure 3.3 describes the types of languages and their corresponding automata. This figure also describes the relation among four types of languages; and relation among automata. This says that the function performed (i.e., the language recognised) by finite automaton will also be performed by pushdown automaton; the function performed by pushdown automaton will also be performed by linear bounded automaton (LBA); and the function performed by linear bounded automaton will also be performed by a Turing machine.

The similar concept is among these four types of languages. The highest type number of grammar for a given set of productions (suppose production set consists of two or more grammars for a given set of productions) is the type described by the outer circle.

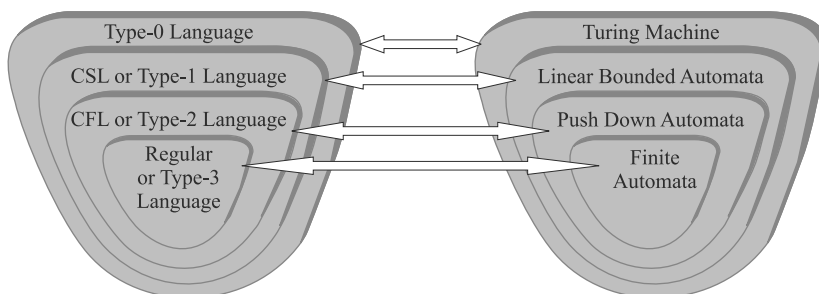


Fig. 3.3 Languages and their corresponding machines (The Chomsky Hierarchy)

For example if a set of productions consists of type-3 and type-2 production then highest type number or grammar will be type-2, because a type-3 production is a type-2 production by default. Therefore the following statements are true:

- (i) Every regular language is context-free.
- (ii) Every context-free language is context-sensitive, and
- (iii) Every context-sensitive language is recursively enumerable.

When we relate these languages with corresponding grammars then the following statements are true:

- (i) Every regular grammar is context-free. For example the production $A \rightarrow aB$ is context free also, as there is neither a left nor right context in the left hand side of the production.
- (ii) Every context-free grammar is context-sensitive. For example, a production $A \rightarrow ab$ in context free grammar, can be treated as $\wedge A \wedge \rightarrow ab$ with $\wedge$ as left as well as right context, and
- (iii) Every context-sensitive grammar is type-0 or unrestricted grammar.

These are all proper inclusions, meaning that there exist recursively enumerable languages which are not context-sensitive, context-sensitive languages which are not context free and context free languages which are not regular. The following table (Table 4.1) summarises each of Chomsky four types of grammar and their corresponding languages with machines that accept them. The class of languages the grammar generates is also represented by the Chomsky hierarchy.

Table 4.1 Chomsky classification of grammars and languages

GRAMMAR	PRODUCTION	LANGUAGE	AUTOMATON	PRACTICAL EXAMPLE
Type-0 Grammar or Unrestricted Grammar	No restriction	Type-0 Language or Recursively enumerable Language	Turing machine	Languages like to find GCD of two numbers
Type-1 Grammar or Context-sensitive Grammar	$\alpha A \beta \rightarrow \gamma, A \in V_N,$ $\gamma \in (V_N \cup \Sigma)^+$ and $\alpha, \beta \in (V_N \cup \Sigma)^*$	Type-1 Language or Context-sensitive Language	Linear Bounded Automata or Non-deterministic Turing machine	$L = \{a^n b^n c^n \mid n \geq 0\}$
Type-2 Grammar or Context-free Grammar	$A \rightarrow \alpha, A \in V_N,$ $\alpha \in (V_N \cup \Sigma)^*$	Type-2 Language or Context-free Language	Pushdown Automata	$L = \{a^n b^n \mid n \geq 0\}$
Type-3 Grammar or Regular Grammar	$A \rightarrow a, A \rightarrow a\alpha,$ $A \in V_N, a \in \Sigma,$ and $\alpha \in (V_N \cup \Sigma)^*$	Type-3 Language or Regular Language	Finite state automata	$L = \{a^n \mid n \geq 0\}$

Example 3.29

Find the highest type number which can be applied to the grammar represented by the following productions:

- (i) $A \rightarrow Ba, B \rightarrow c \mid Ca, C \rightarrow abc,$
- (ii) $B \rightarrow aB, A \rightarrow BAC \mid dab,$
- (iii) $A \rightarrow aAb, Ab \rightarrow ab.$

Solution

- (i) The productions $A \rightarrow Ba, B \rightarrow Ca,$ and $C \rightarrow abc,$ are type-2 productions, and the production $B \rightarrow c,$ is type-3 production, therefore the highest type number is type-2.
- (ii) The productions $A \rightarrow BAC \mid dab,$ are type-2 productions, and the production $B \rightarrow aB,$ is type-3 production, therefore the highest type number is type-2.
- (iii) The production $A \rightarrow aAb,$ is type-2 production, and the production $Ab \rightarrow ab$ is type-1 production, therefore the highest type number is type-1.

Theorem 3.5 If G is a type-0 grammar then an equivalent grammar G_1 can be found in which each production is either of the form $\alpha \rightarrow \beta$, where $\alpha, \beta \in V_N$, or $A \rightarrow a$, where, $A \in V_N$ and $a \in \Sigma$. Then, G_1 is of type-1, type-2, or type-3 according as the grammar G is of type-1, type-2, or type-3.

Proof The grammar G_1 is constructed as follows: For the construction of grammar G_1 , let us consider the production $\alpha \rightarrow \beta$, is in grammar G for α and β have some terminals. If we replace all terminals in α and β by a new variable C_x and get α' and β' . Thus, corresponding to every $\alpha \rightarrow \beta$, we construct $\alpha_1 \rightarrow \beta_1$, and productions of the form $C_x \rightarrow x$, for every $x \in \Sigma$ appearing in α or β . This type of construction is performed for all such production of the form $\alpha \rightarrow \beta$. Now the productions of G_1 are the updated productions obtained by the above construction. For grammar G_1 , the set of nonterminals is the set of nonterminals in grammar G with newly introduced variables of the form C_x . The set of variables and the start symbol S in grammar G_1 is same as grammar G . Thus the grammar G_1 satisfies the required conditions, therefore, both the languages generated by grammars G and G_1 are equivalent. Hence,

$$L(G_1) = L(G).$$

Example 3.30

Construct (i) a context sensitive but not context free grammar, (ii) context free but not regular grammar, (iii) a regular grammar to derive the language

$$L = \{a^n \mid n \geq 1\}.$$

Solution

- (i) A context sensitive grammar consists of productions which contain left or right or both the contexts in the left hand side of the productions. For the language $L = \{a^n \mid n \geq 1\}$, the strings will be $\{a, aa, aaa, \dots, a^n\}$. So we have to design the productions such that total number of a 's are increased by one, when the same production is applied repeatedly in the right hand side of the production. The required productions are

$$\begin{aligned} A &\rightarrow aS \mid A, \\ aS &\rightarrow aa, \\ A &\rightarrow a, \end{aligned}$$

In the above set of productions the production $aS \rightarrow aa$, shows context sensitivity.

- (ii) A context free but not regular grammar for language $L = \{a^n \mid n \geq 1\}$ is,

$$\begin{aligned} S &\rightarrow AS \mid a, \\ A &\rightarrow a, \end{aligned}$$

- (iii) A regular grammar to derive the language $L = \{a^n \mid n \geq 1\}$ is,

$$S \rightarrow aS, S \rightarrow a.$$

❖ Did you Know ?

Noam Chomsky has made a great contribution to formal languages especially in classification known as Chomsky classification of languages and grammars. Chomsky's important research appeared in *Logical Structures of Linguistic Theory* in 1955.

Chomsky received honorary degrees from reputed universities worldwide specially Cambridge University, Columbia University, Harvard University, Loyola University of Chicago, McGill University, University of Chicago, University of Connecticut, University of London, University of Massachusetts, University of Toronto, and University of Western Ontario. At present Chomsky is a Fellow of the American Academy of Arts and Sciences and the National Academy of Science.

Summary

- **Formal Languages** A formal language is a set of strings of symbols from some alphabet.
- **Recursive Definition of Languages** A new method of defining sets is called recursive definition. A recursive definition is a three-step process. First, specify some basic objects in the set. Second, define rules for constructing more objects in the set from the ones already known. Third, declaring that no objects except those constructed in the way are allowed in the list.
- **Grammar** A grammar (also called phrase-structure grammar) is defined as syntax rule to generate a language.
- **Language Generated by Grammar** The language generated by a grammar G (denoted by $L(G)$) is defined as $L(G) = \{s \in \Sigma^* \mid S \xRightarrow[G_c]{*} s\}$ where s is a string formed by elements of only Σ , i.e., terminals and, S is the start symbol.
- **Types of Grammars** According to Chomsky hierarchy, there are four types of grammar - unrestricted, context sensitive, context free, and regular grammars.
- **Type-0 Grammars** These are unrestricted grammars which include all formal grammars. Any kind of productions can be included in type-0 grammar.
- **Type-1 Grammars** These are also called context-sensitive grammars (CSGs), and used to derive the context-sensitive languages (CSLs). These grammars have productions rules of the form $\alpha A \beta \rightarrow \gamma$.
- **Type-2 Grammars** Type-2 grammars are defined by the productions of the form $A \rightarrow \alpha$, with $A \in V_N$ and $\alpha \in (V_N \cup \Sigma)^*$.
- **Type-3 Grammars** Type-3 grammars restrict their productions to a single non-terminal on the left-hand side and the right side consisting of a single terminal, possibly followed by a single nonterminal. For example, $A \rightarrow a$, $A \rightarrow aB$, are type-3 productions, where $A \in V_N$ and $a \in \Sigma$.
- **Recursively Enumerable Languages** A recursively enumerable set is a set S for which there is a procedure to determine whether or not a given element belongs to set S .
- **Operations on Languages** Each of the classes L_0 , L_{CSL} , L_{CFL} , and L_{RL} is closed under union, concatenation, and, closure.
- **Chomsky Hierarchy** Every regular language is context-free. Every context-free language is context-sensitive, and every context-sensitive language is recursively enumerable.

Review Questions

- 3.1 If $S = \{aa, b\}$, find S^*
- 3.2 Consider the language S^* , where $S = \{a, b\}$. How many words this language has of:
 - (i) length 2
 - (ii) length 3
 - (iii) length n .
- 3.3 Consider the language denoted by $S^* = (ab, ba)^*$. Write out all the words in S^* that have seven or fewer letters. Can any word in this language contain the substrings aaa or bbb . What is the smallest word that is not in this language?

- 3.4** Prove or disprove that for all sets S ,
- (i) $(S^+)^* = (S^*)^*$
 - (ii) $(S^+)^+ = S^+$
 - (iii) $(S^*)^+ = (S^+)^*$
 - (iv) $(S^*)^* = S^*$.
- 3.5** (i) Consider the language S^* , where $S = (aa, ab, ba, bb)$. Given another description of this language.
(ii) Give an example of set S such that S^* only contains all possible strings at a 's and b 's that have length divisible by 3.
- 3.6** If $S = \{xx, xxx\}$ then explain the different meanings of S^* .
- 3.7** For Exercise 3.6 explain the occurrence of $xxxxxxx$.
- 3.8** Consider the language S^* , where $S = (aa, aba, baa)$. Show that the words $aabaa$, $baaaba$, and $baaaaababaa$ are all in the language.
- 3.9** Consider the language S^* , where $S = (xx, xxx)$. In this many ways can x^{19} be written as the products of words in S ? This means: How many different factorisations are there of x^{19} into xx and xxx .
- 3.10** (i) Let $S = (a, b)$ and let $T = (a, b, aa, bb, bbb)$. Show that $S^* = T^*$.
(ii) Let $S = (ab, bb)$ and let $T = (aa, bb, bbb)$. Show that $S^* \neq T^*$, but that $S^* \subset T^*$.
- 3.11** If $S = \{a, b\}$ and $T^* = S^*$, prove that T must contain S .
- 3.12** Give the recursive definition for all negative integers.
- 3.13** Give the recursive definition to compute n^{th} Fibonacci term.
- 3.14** Give the recursive definition for valid AE , and check whether the following AE is valid or not:
 $- (- (- (- (a))))$.
- 3.15** How the validity of a Boolean expression can be checked?
- 3.16** Show that a language is a regular language that is defined by regular expression.
- 3.17** Explain the condition in which two languages are the same.
- 3.18** Write the grammar to represent
- (i) all even numbers between 1 to 100
 - (ii) all odd numbers between 101 to 200
- 3.19** Show that $L = \{a^n b^n \mid n \geq 0\}$ is not regular language.
- 3.20** Prove that $L = \{x \in \{0, 1\}^* \mid x \text{ contains equal number of 0's and 1's}\}$ is not regular.
- 3.21** Determine the grammar generating the language $L = \{\omega\omega \mid \omega \in \{0, 1\}^*\}$.
- 3.22** Construct regular grammar to generate the following:
- (a) $L = \{a^{2n} \mid n \geq 1\}$
 - (b) The set of all strings over $\{a, b\}$ ending in aa .
 - (c) The set of all strings over $\{a, b\}$ beginning with bb .
 - (d) $L = \{(ab)^n \mid n \geq 1\}$
- 3.23** If each production in a grammar G has some variable on its right-hand side, what can you say about $L(G)$?
- 3.24** Show that $\{abc, bca, cab\}$ can be generated by a regular grammar whose $\Sigma = \{a, b, c\}$.
- 3.25** Generate a grammar G such that $L(G) = \{\emptyset\}$.
- 3.26** Consider the grammar G given by $S \rightarrow 0SA2$, $S \rightarrow 012$, $2A \rightarrow A2$, $1A \rightarrow 11$. Test whether (a) $00112 \in L(G)$ and (b) $001122 \in L(G)$.

- 3.27** If grammar G is $S \rightarrow aS \mid bS \mid a \mid b$, find $L(G)$.
- 3.28** If $G = (\{S, C\}, \{a, b\}, P, S)$ where P consists of $S \rightarrow aCa, C \rightarrow aCa \mid b$, find $L(G)$.
- 3.29** If $G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S1 \mid 0\}, S)$ find $L(G)$.
- 3.30** Let L be the set of all palindromes over $\{a, b\}$. Construct G generating L .
- 3.31** Find the highest type number which can be applied to the following grammars:
- $S \rightarrow Aa, A \rightarrow c \mid Ba, B \rightarrow abc$
 - $S \rightarrow ASB \mid d, A \rightarrow aA$
 - $S \rightarrow aS \mid b$
- 3.32** If the grammar G is given by the productions $S \rightarrow aSa \mid bSb \mid aa \mid bb \mid \wedge$, then show that
- $L(G)$ has no string of odd length.
 - Any string in $L(G)$ is of length $2n, n \geq 0$
 - The number of strings of length $2n$ are 2^n .
- 3.33** Define the grammar to generate language $L = \{s \in (a, b)^* \mid n_a(s) = n_b(s) + 2\}$
- Is there any possibility of left and right contexts in a context free grammar?
 - find the regular grammar equivalent to CFG having productions $S \rightarrow Sa \mid Sb \mid \wedge$.
- 3.34** Show that a context free grammar is also a context sensitive grammar.
- 3.35** How can you convert a regular grammar into context free grammar?
- 3.36** Show that each of the class L_0, L_{CSL}, L_{CFL} , and L_{RL} is closed under transpose operation.
- 3.37** Show that each of the families of L_0, L_{CSL}, L_{CFL} , and L_{RL} is closed under intersection.

Hints

- 3.1** $S^* = \{\wedge, \text{any word composed of factors of } aa \text{ and } b\}$
 $= \{\wedge, \text{all strings of } a\text{'s and } b\text{'s in which the } a\text{'s occur in even clumps}\}$
 $= \{\wedge, b, aa, bb, aab, baa, bbb, aaaa, aabb, baab, bbaa, bbbb, aaaab, aabaa, \dots\}$
 The string $aabaaab$ is not in S^* since it has a clump of a 's of length 3.
- 3.2** (i) 4, (ii) 8, (iii) 2^n
- 3.3** Strings of seven or fewer letters = $\{\wedge, ab, ba, abab, abba, baab, baba, ababab, \dots, bababa\}$
- 3.4** $(S^*)^* = (S, SS, SSS, \dots)^*$
 $= \{\wedge, S, SS, SSS, \dots\}$
 $(S^*)^* = (\wedge, S, SS, SSS, \dots)^*$
 $= \{\wedge, S, SS, SSS, \dots\}$
- 3.5** (i) The set of all even length strings over (a, b)
 (ii) Strings are from $(aaa, aab, aba, abb, baa, bab, bba, bbb)^*$.
- 3.6** $S^* = \{\wedge, \text{all strings having more than one occurrences } x\}$
 $= \{x^n \text{ for } n = 0, 2, 3, 4, 5, \dots\}$
 $= \{\wedge, xx, xxx, xxxx, xxxxx, \dots\}$.
- 3.7** $xxxxxx$ is x^7 , now x^7 is either $x^2x^2x^3$ or $x^2x^3x^2$ or $x^3x^2x^2$.
 Where $x^2x^3x^2$ is the concatenation of x^2, x^3 and x^2 .
- 3.18** (i) $S \rightarrow ABC \mid 100, A \rightarrow 0, B \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 8 \mid 9, C \rightarrow 0 \mid 2 \mid 4 \mid 6 \mid 8$.
- 3.19** To show this, pumping lemma (see chapter 4) can be applied. And also, we have $L = \{\wedge, ab, aabb, aaabbb, \dots\}$. Let us note, that it is a subset of many regular languages, such as a^*b^* , which however, also includes strings aab and bb that $a^n b^n$ does not. So $a^n b^n$ cannot be represented in terms of regular expression. Therefore the language L is not a regular language.

- 3.20** Pumping lemma (Chapter 5) can be applied, and also no regular grammar exists for this language.
- 3.21** It is language of all even-length strings of 0's and 1's whose first and second halves are identical. The required grammar is
 $S \rightarrow AA, A \rightarrow 0A \mid 1A \mid \wedge$.
- 3.22** (a) $S \rightarrow aA, A \rightarrow aS, S \rightarrow aB, B \rightarrow a$
 (b) $S \rightarrow aS \mid bS \mid aa$
 (c) $S \rightarrow bB, B \rightarrow bA, A \rightarrow aA, A \rightarrow bA, A \rightarrow \wedge$
 (d) $S \rightarrow aA \mid aB, A \rightarrow bS, B \rightarrow b$
- 3.23** $L(G) = \{\phi\}$
- 3.24** Find Σ^* , and show that $abc, bca, cab \in \Sigma^*$.
- 3.25** Design production in such a way that the start symbol is not able to represent a terminal string.
- 3.26** (a) No.
 (b) Yes, $S \Rightarrow 0SA2 \Rightarrow 0012A2 \Rightarrow 001A22 \Rightarrow 001122$
- 3.27** $L(G) = (a, b)^+$, i.e., set of all strings over $\{a, b\}$ with at least one length.
- 3.28** The minimum length string generated by G is 'aba'. The productions $C \rightarrow aCa \mid b$, will give $C \Rightarrow a^{n-1}ba^{n-1}$. Hence the language generated will be $L(G) = \{a^nba^n \mid n \geq 1\}$.
- 3.29** The minimum length string is 0. The occurrence of 0's is one more than occurrences of 1's. Hence the language will be $L(G) = \{0^{n+1}1^n \mid n \geq 0\}$.
- 3.30** $S \rightarrow aSa \mid bSb \mid a \mid b \mid \wedge$.
- 3.31** (a) type-2, (b) type-2, (c) type-3.

Graded Exercises

- *3.1** Give a recursive definition for the language E-PAL (all palindromes of even length).
- *3.2** Let $S = (a, b, ab)$ find S^* and S^+ .
- 3.3** Construct the grammar, accepting of each of the following sets:
 *(a) $L = \{0^n1^{2n} \mid n \geq 1\}$
 **(b) $L = \{0^n1^n \mid n \geq 1\} \cup \{1^m0^m \mid m \geq 1\}$
 **(c) $L = \{s \in (a, b)^* \mid n_a(s) = n_b(s)\}$
 **(d) $L = \{a^n b^m c^{n+m} \mid n, m \geq 0\}$
- *3.4** Construct a grammar generating $L = \{\omega c \omega^T \mid \omega \in \{0, 1\}^*\}$.
- 3.5** Find the language generated by following grammars:
 **(a) $S \rightarrow 0S1 \mid 0A1, A \rightarrow 1A \mid 1$
 **(b) $S \rightarrow 0S1 \mid A \mid B, A \rightarrow 0A \mid 0, B \rightarrow 1B \mid 1$
 ***(c) $S \rightarrow 0SBA \mid 01A, AB \rightarrow BA, 1B \rightarrow 11, 1A \rightarrow 10, 0A \rightarrow 00$.
- *3.6** State whether the following statements are true or false, justify your answer with a proof or example:
 (a) If G_1 and G_2 are equivalent if they are of the same type.
 (b) If L is a finite subset of Σ^* , then L is CFL
 (c) If L is a finite subset of Σ^* , then L is a regular language.
- *3.7** Construct a grammar which generates all
 (i) even integers 0 to 98, (ii) odd integers 1 to 99.
- **3.8** What is the difference and similarity between grammars G_1 and G_2 defined as
 $G_1 = \{A \rightarrow aA \mid \wedge\}$, and $G_2 = \{B \rightarrow Ba \mid \wedge\}$.
- *3.9** Show that L_{CFL} is not closed under intersection. But the intersection of CFL and RL is context free.

****3.10** Construct grammar for language $L = \{a^n b^n c^n d^n \mid n \geq 0\}$, and then determine the type of this language.

*****3.11** Show that no regular or context-free grammar exists for $L = \{a^n b^n c^n \mid n \geq 0\}$.

****3.12** Determine what kind of strings are generated by the grammar having productions

$$S \rightarrow aA \mid bS$$

$$A \rightarrow bA \mid a$$

****3.13** Give the difference and similarity between the languages generated by following grammars

$$G_1: S \rightarrow aSa \mid aSb \mid bSa \mid bSb \mid a \mid b \mid \wedge$$

$$G_2: S \rightarrow aS \mid bS \mid \wedge$$

****3.14** Determine the property of strings are generated by the grammar having productions

$$S \rightarrow aSa \mid bSb \mid X$$

$$X \rightarrow aXb \mid bXa \mid aYb \mid bYa \mid S$$

$$Y \rightarrow aY \mid bY \mid \wedge$$

* Difficulty level 1

** Difficulty level 2

*** Difficulty level 3

Hints

3.1 Step 1. $\wedge$, aa and bb are in E-PAL as they are strings of length zero and two.

Step 2. If x is in E-PAL, then axa and $bx b$ are also in E-PAL.

Step 3. Strings obtained by using steps 1 and 2 repeatedly are also in E-PAL.

3.2 $S^* = \{ \wedge, \text{ and all strings of one or more than one symbols containing } a\text{'s and } b\text{'s} \}$

$$= \{ \wedge, a, b, aa, ab, ba, bb, \dots \}$$

$$S^+ = S^* - \wedge$$

$$= \{ a, b, aa, ab, ba, bb, \dots \}$$

3.3 (a) $S \rightarrow 0S11 \mid 011$

(b) $S \rightarrow A \mid B, A \rightarrow 0A1 \mid 01, B \rightarrow 1B0 \mid 10$

(c) $S \rightarrow SS \mid aSb \mid bSa \mid \wedge$

(d) $S \rightarrow aSc \mid A \mid \wedge, A \rightarrow bAc \mid \wedge$

3.4 $S \rightarrow 0S0 \mid 1S1 \mid c$.

3.5 (a) $\{0^n 1^m \mid m > n \geq 1\}$

(b) $\{0^n 1^m \mid m \neq n\}$

(c) $\{0^n 1^n 0^n \mid n \geq 1\}$

3.6 (a) False, G_1 and G_2 are equivalent if and only if $L(G_1) = L(G_2)$.

3.7 (a) $S \rightarrow 0 \mid 2 \mid 4 \mid 6 \mid 8 \mid 10 \mid 1A \mid 2A \mid 3A \mid 4A \mid 5A \mid 6A \mid 7A \mid 8A \mid 9A, A \rightarrow 0 \mid 2 \mid 4 \mid 6 \mid 8$.

3.8 The difference between grammars G_1 and G_2 is that the grammar G_1 is regular while the grammar G_2 is context free. The similarity in G_1 and G_2 is that both the grammars generate same set of strings i.e., same language, $L(G_1) = L(G_2)$, as we have,

$$L(G_1) = \{ \wedge, a, aa, aaa, \dots \} = \{ a^n \mid n \geq 0 \}$$

and

$$L(G_2) = \{ \wedge, a, aa, aaa, \dots \} = \{ a^m \mid m \geq 0 \}$$

Therefore, $L(G_1) \subseteq L(G_2)$, and $L(G_2) \subseteq L(G_1)$

3.13 The strings in the languages generated by both grammars are the same and the types of languages are different.

3.14 This Grammar generates all non palindromes over $\{a, b\}$.

Objective Questions

- If Σ is alphabet, then Σ^* is
 - closure of alphabet Σ
 - Kleene closure notation Σ
 - a language in which any string of letters from Σ
 - all of these
- Which of the following has maximum number of strings?
 - $(a)^*$
 - $(a, b)^*$
 - $(a, b, c)^*$
 - all have infinite strings
- If $\Sigma = \phi$ (the empty set), the Σ^* has
 - number of infinite strings
 - only $(\wedge)$
 - infinite string excluding $(\wedge)$
 - none of these
- Consider two language $A = (a, b, ab)$ and $B = (b, a, ab)$, then
 - $A^* \subseteq B^*$
 - $A^* = B^*$
 - both (a) and (b)
 - none of these
- If $\Sigma = (a)$, then $\Sigma^* - \Sigma^+$ is
 - $\{a\}$
 - $\{aa\}$
 - $\{a, aa, aaa, \dots\}$
 - $\{\wedge\}$
- Which one of the following is false?
 - $(A^*)^* = A^{**}$
 - $A^* = A^+ + (\wedge)$
 - $A^* = A^{**}$
 - none of these
- If $A = \{ab, ba\}$, then
 - $bb \in A^*$
 - $aa \in A^*$
 - $abba \in A^*$
 - all of these
- Which one of the following is false?
 - $(A^+)^* = (A^*)^*$
 - $(A^+)^+ = A^+$
 - $(A^*)^* \subset A^*$
 - none of these
- If $A = (ab, bb)$ and $B = (ab, bb, bbbb)$, then
 - $A^* = B^*$
 - $A^* \neq B^*$
 - $A^* = B^+$
 - none of these
- Which one of the following can be defined by recursive definition?
 - all positive real numbers
 - all even numbers
 - the set of polynomials
 - all of these
- If $\Sigma = \{xx, xxx\}$, then Σ^* contains
 - all $x^n \mid n \text{ even and odd}$
 - all $x^n \mid n \text{ is even}$
 - all $x^n \mid n \text{ is odd}$
 - all $x^n \mid n \neq 1$
- If $S = \{aa, aaa\}$, then for S^* , which one of the following is true?
 - $(a^n \mid n = 0, 2, 3, 4, 5, 6, \dots)$
 - $\{\wedge \text{ and all strings of more than one } a\}$
 - both (a) and (b) are false
 - both (a) and (b) are true
- If $A = \{a, bb\}$, then $(A^*)^*$ contains
 - all strings composed of factors of a and bb
 - all strings composed of factors of a and bb with length at least three
 - all strings composed of factors a and bb with length at least one
 - $\wedge$ and all strings composed of factors a and bb .

14. The concatenation of L_1 and L_2 is
 (a) $\{xy \mid x \in L_1 \text{ and } y \in L_2\}$ (b) $\{xy \mid x \in L_2 \text{ and } y \in L_1\}$
 (c) $\{xx \mid x \in L_1\}$ (d) $\{yy \mid y \in L_2\}$
15. If $A = \{\wedge, a, b, aa, bb, aaa, bbb, aba, bab, \dots\}$ then A is defined as
 (a) all strings over $\{a, b\}$
 (b) the set in which concatenation of any two strings belongs to the set A
 (c) the set of palindromes over $\{a, b\}$ including zero length palindrome
 (d) all of the above
16. The strings that are permissible in the language are called
 (a) alphabets (b) words (c) both (a) and (b) (d) neither (a) nor (b)
17. Certain specified set of strings of characters from the alphabet is called
 (a) words (b) alphabets (c) both (a) and (b) (d) language
18. If $A = \{\wedge, a, b\}$ then which statement is true?
 (a) $A^* = \{\wedge, a, b, aa, ab, ba, bb, \dots\}$ (b) $A^+ = A^*$
 (c) $A^+ = \{a, b, aa, ab, ba, bb, \dots\}$ (d) both (a) and (b)
19. Consider the language $A = \{ab, ba\}$, then which of the following statements is true?
 (a) there are so many words in A^* , which contain the substring aaa
 (b) there are so many words in A^* , which contain the substring bbb
 (c) both (a) and (b) are true
 (d) both (a) and (b) are false
20. If L is a language, then
 (a) $L^+ = L^* L = LL^*$ (b) $L^+ = L^* - \wedge$ (c) $L^+ = \bigcup_{i=1}^{i=\infty} L^i$ (d) all of these
21. If A and B are two languages over Σ , then
 (a) $A \subseteq B$ (b) $A^* \subseteq B^*$ (c) both (a) and (b) (d) neither (a) nor (b)
22. If L is any language over Σ , then
 (a) $L^* = (L^*)^*$ (b) $(L^+)^* = (L^*)^+$ (c) $L^+ \subseteq L^*$ (d) all of these
23. Consider the language $L \subseteq (a, b)^*$, then which of the following statement is true?
 (a) $a, b \in L$
 (b) for any $x \in L$, $ax \in L$
 (c) for any x and y in L , all the strings bxy , xyb and xyb are in L
 (d) all of the above
24. If $A^* = \{\wedge, a, b, aa, ab, ba, bb, aaa, aba, baa, \dots\}$, then
 (a) $\{x \in A^* \mid |x| \text{ is odd}\}$ (b) for all $x \in A^* \mid n_a(x) > n_b(x)$
 (c) for all $x \in A^* \mid n_a(x) < n_b(x)$ (d) none of the above.
25. If $L_1 \subseteq A^*$ and $L_2 \subseteq B^*$, then
 (a) L_1 in subset of $(A \cup B)^*$ (b) L_2 is subset of $(A \cup B)^*$
 (c) both (a) and (b) (d) neither (a) nor (b)
26. If $A = \{a, b\}$ and $B = \{c, d\}$, then AB is
 (a) $\{ac, ad, bc, bd\}$ (b) $\{ca, cb, da, db\}$ (c) $\{a, b, c, d\}$ (d) all these

27. The word 'formal' in formal languages means
 (a) the symbols used have well-defined meaning
 (b) only the form of the string of symbols is significant
 (c) they are unnecessary, in reality
 (d) all of the above
28. If the number of possible strings of length ' n ' is
 (a) n^n (b) 2^n (c) $n!$ (d) none of these
29. Which of the following string can be obtained by the language $L = \{a^n b^{2n} \mid n \geq 1\}$?
 (a) $a^3 b^6$ (b) $a^2 b^3$ (c) $ab^2 ab^3 a$ (d) $a^4 b^3 ab^2$
30. If $\Sigma^* = \{\wedge, a, b, aa, ab, ba, bb, \dots\}$, then
 (a) (a, aa, aab) is a language on Σ (b) (b, bb, bba) is a language on Σ
 (c) $L = \{a^n b^n \mid n \geq 0\}$ is a language on Σ (d) all of the above
31. If $L = \{a^n b^n \mid n \geq 0\}$ is a language, then
 (a) $L^2 = \{a^n b^n a^m b^m \mid n \geq 0, m \geq 0\}$ (b) $L^R = \{b^n a^n \mid n \geq 0\}$, where L^R is reverse of L
 (c) both (a) and (b) (d) none of the above
32. A grammar G is defined as $G = (V_N, \Sigma, P, S)$, then which one is true
 (a) $V_N \cap \Sigma = \phi$ (b) $V_N \subseteq P$ (c) $V_N \cup \Sigma$ is a string (d) none of these.
33. In a grammar $G = (V_N, \Sigma, P, S)$, P is
 (a) set of productions (b) set of production rules
 (c) set of rewriting rules (d) all of these
34. The string generated by the grammar $G = (\{S\}, \{a, b\}, \{S \rightarrow aSb \mid ab\}, S)$ are
 (a) $ab, aabb$ (b) a, b, ab (c) $a^3 b^3, a^3 b^6$ (d) all of these
35. For the following set of productions, which one is true
 $S \rightarrow aSB \mid \wedge, B \rightarrow aB \mid b$
 (a) $S, A, B \in V_N$ (b) $a, b, \wedge \in \Sigma$ (c) $a, b, S, B \in \Sigma$ (d) both (a) and (b)
36. The language generated by grammar $G = (\{S\}, \{a, b\}, \{S \rightarrow SS\}, S)$, is
 (a) $\{a^n b^n \mid n \geq 1\}$ (b) $\{a^n b^m \mid m, n \geq 1\}$ (c) ϕ (d) both (a) and (b)
37. The language generated by the grammar $G = (\{S\}, \{a, b\}, \{S \rightarrow aSb \mid \wedge\}, S)$, is
 (a) $\{a^n b^n \mid n \geq 1\}$ (b) $\{a^n b^n \mid n \geq 0\}$ (c) $\{b^n a^n \mid n \geq 1\}$ (d) none of these
38. If S_1 and S_2 are the string on $(V_N \cup \Sigma)^*$ then S_1 derives S_2 if
 (a) $S_1 \xRightarrow{*} S_2$ (b) $S_2 \xRightarrow{*} S_1$ (c) both (a) and (b) (d) neither (a) nor (b)
39. For a grammar G , if $S \xRightarrow{G} \alpha$ then
 (a) α is a sentential form (b) $\alpha \in L(G)$ (c) $L(G) \subseteq \alpha$ (d) both (a) and (b)
40. Which one of the following is not a valid production for $V_N = \{S, A, B\}, \Sigma = \{0, 1\}$?
 (a) $S \rightarrow A \mid AB$ (b) $SA \rightarrow B \mid 01$ (c) $0A1 \rightarrow 10$ (d) $0 \rightarrow 1$
41. The language generated by grammar $G = (\{S\}, \{a, b\}, \{S \rightarrow aS \mid bS \mid a \mid b \mid \wedge\}, S)$, is
 (a) $\{a, b\}^*$ (b) $\{a, b\}$ (c) $\{a, b\}^+$ (d) $\{a\}^*, \{b\}^*$
42. State which of the following statements is true?
 (a) if $L(G_1)$ and $L(G_2)$ are equivalent, even then it is not necessary that they are of same type.
 (b) a context free language is also a regular language.

- (c) a context sensitive language is also a context free language.
 (d) none of the above.
43. A set of production is given by $P = \{S \rightarrow AB, A \rightarrow aA \mid \wedge, B \rightarrow bB \mid \wedge\}$ for a grammar G . What is $L(G)$?
 (a) ϕ (b) $\{a^m b^n \mid m, n \geq 0\}$ (c) $\{a^m b^n \mid m, n \geq 1\}$ (d) none of these
44. For a grammar $G = (\{S, A, B\}, \{a, b\}, P, S)$ the valid production set is
 (a) $A \rightarrow SB, B \rightarrow AS$ (b) $A \rightarrow aA \mid \wedge, B \rightarrow bB \mid \wedge$
 (c) $S \rightarrow a \mid b, A \rightarrow B, B \rightarrow A$ (d) all of these
45. If $L(G) = \{a^n b^n \mid n \geq 0\}$, then
 (a) $L(G) \subseteq \{a^n b^n \mid n \geq 0\}$ (b) $ab \in L(G)$ (c) both (a) and (b) (d) none of these
46. If $L(G)$ has all strings of even length over $\{0, 1\}$. The set of productions in grammars G generating $L(G)$ is
 (a) $S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1$ (b) $S \rightarrow 0S0 \mid 1S1 \mid 00 \mid 11 \mid \wedge$
 (c) $S \rightarrow 00S \mid 01S \mid 10S \mid 11S$ (d) $S \rightarrow SS \mid 0 \mid 1 \mid \wedge$
47. Which of the following statements is wrong?
 (a) every recursive language is recursively enumerable.
 (b) a language is accepted by a finite automaton if and only if it is context free.
 (c) recursive languages are closed under intersection.
 (d) a language is accepted by a finite automaton if and only if it is right-linear
48. Which of the following statements is true?
 (a) all language can be generated by context-free grammar.
 (b) the number of symbols necessary to stimulate a Turing machine with m input symbols and n states is mn .
 (c) any regular language has an equivalent context-free grammar.
 (d) the class of context-free grammar is not closed under union.
49. Which sentence can be generated by the grammar $G = \{S \rightarrow aS \mid bA, A \rightarrow d \mid cA\}$?
 (a) $aabccd$ (b) $ababcca$ (c) $abcca$ (d) $abababd$
50. Consider the productions $S \rightarrow AA, A \rightarrow aa \mid bb$, in grammar G . The language specified by the production grammar is:
 (a) $L = \{aaaa, aabb, bbaa, bbbb\}$ (b) $L = \{abab, abaa, aaab, baaa\}$
 (c) $L = \{aaab, baba, bbaa, bbbb\}$ (d) $L = \{aaaa, abab, bbaa, aaab\}$
51. Give a production grammar that specifies language $L = \{a^i b^{2i} \mid i \geq 1\}$,
 (a) $S \rightarrow aSbb \mid abb$ (b) $S \rightarrow aSb \mid b$
 (c) $S \rightarrow aS, S \rightarrow b, A \rightarrow b$ (d) none of these
52. Which of the following string can be obtained by the language $L = \{a^i b^{2i} \mid i \geq 1\}$,
 (a) $aaabbbbb$ (b) $aabbb$ (c) $abbabba$ (d) $aaaabbbabb$
53. The production grammar for the language L , if the number of a 's in x is a multiple of 3 is
 (a) $S \rightarrow bS \mid b \mid aA \mid bA, A \rightarrow aB, B \rightarrow bB \mid aS \mid a$ (b) $S \rightarrow aS \mid bA, A \rightarrow bB, B \rightarrow bBa \mid bB$
 (c) $S \rightarrow aaS \mid bba, A \rightarrow bB, B \rightarrow ba$ (d) none of the above
54. The production grammar for the language $L = \{a^i b^j \mid i, j \geq 1, i \neq j\}$, is
 (a) $S \rightarrow aS \mid aB, B \rightarrow ab \mid b, A \rightarrow aaB$
 (b) $S \rightarrow A \mid C, A \rightarrow aA \mid bB, B \rightarrow aBb \mid ab, C \rightarrow Cb \mid Bb$

- (c) $S \rightarrow A, A \rightarrow aA \mid aB, B \rightarrow ab$
 (d) none of the above
55. Consider two language $L_1 = \{a^i b^j c^i \mid i, j \geq 1\}$, and $L_2 = \{a^i b^j c^i \mid i, j \geq 1\}$. The intersection of L_1 and L_2 is given by
 (a) $\{a^i b^j c^i \mid j \geq 1\}$ (b) $\{a^i b^j c^i \mid i, j \geq 1, i \neq j\}$ (c) $\{a^i b^j c^i \mid i \geq 1\}$ (d) none of these
56. The production grammar in $\{S \rightarrow aSbb, S \rightarrow abb\}$ is:
 (a) type-3 grammar (b) type-2 grammar (c) type-1 grammar (d) type-0 grammar
57. Which of the following statements is wrong?
 (a) a Turing machine cannot solve halting problem.
 (b) set of recursively enumerable languages is closed under union.
 (c) a finite state machine with 3 stacks is more powerful than a finite state machine without any stack.
 (d) context-sensitive grammar can be recognised by a linear bounded memory machine.
58. Which of the following statements is wrong?
 (a) recursive languages are closed under union.
 (b) recursive language are closed under complementation.
 (c) if a language and its complement are both regular, then the language must be recursive.
 (d) a language is accepted by a finite automaton if and only if it is recursive.
59. Recursively enumerable languages are not closed under:
 (a) complementation (b) union (c) intersection (d) none of these
60. Which of the following statements is false?
 (a) regular language is subset of context free language.
 (b) context sensitive language is subset of context free language.
 (c) context free language is subset of context sensitive language.
 (d) context sensitive language is subset of type-0 language.
61. Which of the following statements is false?
 (a) pushdown automaton also perform the operation of finite automaton
 (b) turning machine can also perform the operation of finite automaton.
 (c) linear bounded automaton can also perform the operation of finite automaton.
 (d) none of the above
62. The grammar $G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S1 \mid 0S \mid S1 \mid 0\}, S)$ generates a
 (a) context-free language (b) context-sensitive language
 (c) regular language (d) none of these
63. A context-sensitive grammar can be recognised by a
 (a) determine pushdown automaton (b) non-deterministic pushdown automaton
 (c) finite state machine (d) linear bounded automaton
64. The language $L = \{0^n 1^n 2^k 3^k \mid n, k \geq 1\}$, is a
 (a) context-sensitive language (b) context-free language
 (c) regular language (d) none of the these
65. Which of the following sentence is generated by the grammar $S \rightarrow aSbA, A \rightarrow dccA$?
 (a) $abbbd$ (b) $abbcd$ (c) $aaaad$ (d) none of these

66. The language generated by the productions given below is

$$S \rightarrow AB \mid BA, A \rightarrow AC \mid A, A \rightarrow B, B \rightarrow C$$
 (a) Type-0 (b) Type-1 (c) Type-2 (d) Type-3
67. The grammar having productions $S \rightarrow AS \mid \wedge, A \rightarrow 0A1 \mid 1A0 \mid \wedge$, will generate
 (a) $L = \{(0^n 1^n \mid n \geq 0) \cup (1^m 0^m \mid m \geq 0)\}^*$ (b) all palindromes over $(0, 1)$
 (c) $(0, 1)^*$ (d) none of these
68. The language constructs, which are most useful in the description of nested structures like balancing of parentheses, balancing of braces, etc., are
 (a) regular expression (b) context-free grammar (c) both (a) and (b) (d) none of these

References for Extra Reading

1. Cohen, D. I. A. *Introduction to Computer Theory*, John Willy and Sons (asia) Pvt. Ltd. Singapore, 1991.
2. Salomaa, A. *Formal languages*, Acad. Press.
3. Michael, A. Harrison. *Introduction to Formal Language Theory*. Addison-Wesley.
4. Choffrut, C., J. Karhumäki, *Combinatorics of words*, in G. Rozenberg and A. Salomaa, eds., *Handbook of Formal Languages*, Springer, Berlin 1997.
5. Lothaire, M., *Combinatorics on Words*, Addison-Wesley, Reading, MA 1983.
6. Lothaire, M., *Algebraic Combinatorics of Words*, Cambridge University Press, Cambridge 2002.
7. Rozenberg, G. and A. Salomaa, *Handbook of Formal Languages Vol.2 Linear Modeling: Background and Application*, Springer.
8. Carroll, J., Long, D., *Theory of Finite Automata with an Introduction to Formal Languages*, Prentice Hall, Englewood Cliffs, 1989.
1. Chomsky, Noam, *Three models for the description of language*, Information Theory, IEEE Transactions, 1956.
2. Chomsky, Noam *Formal properties of grammars*, In R. Duncan Luce, Robert R. Bush, and Eugene Galanter, editors, *Handbook of Mathematical Psychology*; Volume II, John Wiley, 1963.
3. Chomsky, Noam *Three models for the description of language*, IRE Trans. Information Theory, IT-2, 1956.
4. Chomsky, Noam *Syntactic structures*, Mouton, 1957.
5. Landweber, P. S. *Decision problems of phrase structure grammars*, IEEE Transactions on Electronic Computers 1964.
6. Bar-Hillel, Y. M. Perles, and E. Shamir, *On formal properties of simple phrase structure grammars*, Zeitschrift für Phonetik, Sprachwissenschaft, and Kommunikationsforschung, 1961.

Online Sources

1. <http://www.igm.univ-mlv.fr/~perrin/Recherche/Enciclopedia/article.pdf>
2. <http://www.cs.duke.edu/cs/rodger/papers/iticse07.pdf>
3. www.home.comcast.net/~jhorning4/algol-60-report.pdf
4. <http://www.cs.grinnell.edu/~stone/courses/languages/Backus-Naur-form.pdf>

4

Regular Language and Regular Grammar

❖ Chapter Objective ❖

This chapter will start with an introduction to regular languages and regular expressions. We will describe the identities and algebraic laws to optimise a regular expression. Then we will proceed to correlate regular expressions with finite automata. We will apply three approaches to show the equivalence of two regular expressions. The My-Hill Nerode theorem will be discussed with the help of constructive proof

We will see how a set (here regular set) can represent the strings belonging to a regular language or regular expression. In this sequence we will discuss closure properties of regular sets. Then we will proceed to correlate regular grammar with finite automata. We will see how a regular grammar is constructed from a FA and vice versa. Then the concept of left linear and right linear grammar will be discussed followed by their interconversions. Finally, we will discuss applications of regular expressions and languages that are not regular.

□ Introduction □

Regular languages are those languages which are described by regular expressions. Thus, these form the basis of all computer languages for specifying the lexical analysis. (Lexical analysis is one of the phases of a compiler. You can study lexical analysis in a compiler design course in detail). Regular languages are recognised by finite state machines or finite automata (both DFA and NFA). Languages other than regular cannot be recognised by finite state machines. Hence finite state machines are less computationally powerful than Turing machines because Turing machines solve the decidable problems that are not computable using a FSM (finite state machine).

The origin of regular languages and regular expressions have been seen in theory of automata and formal language theory (both parts of theoretical computer science). These are seen as the study models of computation and ways to describe and classify formal languages.

In the decade of 1940, Warren McCulloch and Walter Pitts described the nervous system by modelling neurons (Neurons are defined as simple processing elements which operate in parallel, whose function is determined by network structure) as small simple automata. The mathematician Stephen Kleene, later described these models using his mathematical notations called regular sets. Ken Thompson built

this notation into the editor 'qed', then into Unix editor 'ed' and eventually into 'grep'. Ever since that time, regular expressions have been widely used in Unix-like utilities such as 'expr', 'awk' and 'perl' most of them using the regex library built by Henry Spencer.

Regular expressions may be used to describe a regular language recognised by finite automata. Regular expressions correspond to the type-3 grammars of the Chomsky hierarchy (see chapter 2). This means that they can be used to describe morphology (the study of the form of words) of a language.

4.1

REGULAR LANGUAGE

A regular language is a formal language that satisfies the following properties:

- (i) It can be accepted by a deterministic finite automaton.
- (ii) It can be accepted by a nondeterministic finite automaton.
- (iii) It can be described by a regular expression.
- (iv) It can be generated by a regular grammar.
- (v) It can be accepted by a read-only Turing machine.

4.2

REGULAR EXPRESSION

❖ Definition

Regular Expression is defined as a set of pattern matching rules which are encoded in a string according to certain syntax rules.

A regular expression is abbreviated as regex or regexp. A regular expression is an expression that describes a whole set of strings by following certain syntax rules of the programming language. Regular expressions are used by many text editors and utilities (like in UNIX operation system) to search a block of text for certain patterns, for example to replace the found strings with some other string.

Regular expressions consist of constants and operators that denote a set of strings and operations over these sets, respectively. For finite alphabet, the following constants are defined:

1. **Empty set**—denoting the set $\emptyset$
2. **Empty string**—denoting the set $\{\wedge\}$ or $\{\epsilon\}$
3. **Literal character**—'a' in denoting the set $\{a\}$

and the following operations:

1. Concatenation Let P and Q be two different regular expressions, PQ denoting the set $\{\alpha\beta \mid \alpha \in P, \beta \in Q\}$. For example,

$$\{a, bc\} \{de, f\} = \{ade, af, bcde, bcf\}.$$

2. Union $P \cup Q$, denoting the union of P and Q .

The symbols ‘+’ and ‘,’ are also used for union in regular expressions. This way, $P + Q$, $P \cup Q$ and (P, Q) are equivalent.

3. Kleene star P^* denoting the smallest superset of P that contains $\wedge$ and is closed under string concatenation. This is the set of all strings that can be obtained by concatenating zero or more strings in P . For example, $\{a, bc\}^* = \{\wedge, a, bc, aa, abc, \dots\}$

Let us consider the following:

- (i) $a \cup b^*$ denotes $\{a, \wedge, b, bb, bbb, \dots\}$ such that $a \cup b^* = \{a\} \cup \{\wedge, b, bb, bbb, \dots\}$
- (ii) $(a \cup b)^*$ denotes the set of all strings consisting of any occurrence of a ’s and b ’s including empty string .
- (iii) $ab^*(c \cup \wedge)$ denotes the set of strings starting with a , followed by any occurrence of b ’s and finally one optional c .
- (iv) $(ab^*)^*$ denotes the same set as described in (ii) above. As $(ab^*)^*$ can be written as

$$\begin{aligned} ((a)(b^*))^* &= (\{a\} \{ \wedge, b, bb, bbb, \dots \})^* \\ &= (a \cup b)^* \end{aligned}$$

The above are the strings derived by corresponding regular expressions.

❖ Did you Know ?

Kleene’s research was on the closure of regular sets, set theory, theory of algorithms and theory of recursive functions. He developed the field of recursion theory along with Alonzo Church, Kurt Gödel, Alan M. Turing and others. Kleene contributed to mathematical intuitionism which had been discovered by Brouwer.

Kleene’s work on recursive function theory helped to provide the foundations in theoretical computer science. Kleene’s work led to the study of which functions are computable, by providing methods of determining what problems are decidable. Kleene’s formulation of computable function via six schemata is one of the most concise and useful contributions to recursive function theory. Kleene’s work on lambda functions played a major role in supporting “Church Thesis” that these classes coincide with the intuitively computable functions. Kleene developed a diverse array of topics in computability that includes: Degrees of computability, the arithmetical hierarchy, computable ordinals and hyper arithmetic theory, Finite automata and regular sets with massive consequences for computer science.

4.3

OPERATORS OF REGULAR EXPRESSIONS

Three operations are performed on alphabets to obtain a regular expression. These operations are: Union, closure, and concatenation.

- (i) A terminal symbol (small letter $x \in \Sigma$) is a regular expression.
- (ii) The union of two different regular expressions P and Q is also a regular expression and, is written as $P + Q$. For example, if $P = \{ab\}^*$, $Q = (a + b)^*$, then $R = P + Q = (ab)^* + (a + b)^*$.

- (iii) The concatenation of two different regular expressions P and Q is also a regular expression and, is written as PQ . For example, $S = PQ = (ab)^* (a + b)^*$, where S is representing the concatenation of P and Q .
- (iv) The closure of a regular expression P is also regular expression and, is written as P^* .
- (v) The regular expression obtained by applying the above four operations any number of times in any order, is also a regular expression. For example, a and b are two different regular expressions by (i) as $a, b \in \Sigma$, $(a + b)$ is regular expression by (ii). $(a + b)^*$ is regular expression by (iv). The expression $(a + b)^* b$ is regular expression by (iii) and so on.

4.3.1 Precedence of Operators

To avoid brackets in regular expression it is assumed that the Kleene star has the highest priority, then concatenation and finally union. If there is no ambiguity then there is no need to include brackets. For example, $(ab)(c(d^*))$ can be written as $abcd^*$.

As we know there are three operations to be performed on alphabets to obtain a regular expression, namely: union, closure, and concatenation. Any operation can be performed first depending upon the use of regular expression. When we construct the finite automaton equivalent to a regular expression the precedence of operator is a big issue. The operation performed first while the regular expression was constructed, is eliminated in last, and the operation performed last while the regular expression was constructed, is eliminated in first. Therefore the operators are eliminated in the reverse order as opposite the order in which they were applied. For example, to construct a regular expression $(a + b)^*a$, the order of operations is:

1. union of a and b to form $(a + b)$
2. closure of $(a + b)$ to form $((a + b)^*)$
3. concatenation of $(a + b)^*$ and a to form $(a + b)^*a$

While we wish to construct the finite automaton equivalent to $(a + b)^*a$, we will have to eliminate concatenation first, followed by elimination of closure operation and finally union operation.

4.4

IDENTITY RULES

An identity for an operator can be defined as the value that when applied to some value results in some other value. The identities are:

1. $\phi + P = P$

Proof We have, L.H.S. = $\phi + P$
 $= \{\} + P$
 $= P + \{\}$
 $= \text{Elements of } P \cup \{\}$
 $= P$
 $= \text{R.H.S.}$

2. $\phi P = P\phi = \phi$

Proof We have, L.H.S. = ϕP
 $= \{\}P$

= Elements of $\{\}$ concatenated with elements of P
 = ϕ , as there is no element in $\{\}$
 = R.H.S.

$$3. \quad \wedge P = P \wedge = P$$

Proof We have, L.H.S. = $\wedge P$

= $\wedge$ followed by P
 = P , the concatenation of $\wedge$ with any symbol/string is the same symbol/string
 = R.H.S.

In the same way, $P \wedge = P$.

4.5

ALGEBRAIC LAWS FOR RE

To simplify regular expressions, certain algebraic laws are used. Let us assume P , Q , and R be different regular expressions.

$$1. \quad \wedge^* = \wedge \text{ and } \phi^* = \wedge$$

Proof We have, L.H.S. = $\wedge^*$
 = $\{\wedge, \wedge\wedge, \wedge\wedge\wedge, \wedge\wedge\wedge\wedge, \dots\}$
 = $\{\wedge, \wedge, \wedge, \wedge, \dots\}$
 = $\wedge$
 = R.H.S.

Now, we have, L.H.S. = ϕ^*
 = $\{\wedge, \phi, \phi\phi, \phi\phi\phi, \phi\phi\phi\phi, \dots\}$
 = $\{\wedge, \phi, \phi, \phi, \phi, \dots\}$ since $\phi\phi = \phi$
 = $\{\wedge, \phi\}$
 = $\wedge$
 = R.H.S.

$$2. \quad P + P = P$$

Proof We have, L.H.S. = $P + P$
 = $P(\wedge + \wedge)$
 = $P(\wedge)$ since $\wedge + \wedge = \wedge$
 = P
 = R.H.S.

$$3. \quad (P^*)^* = P^*$$

Proof We have, L.H.S. = $(P^*)^*$
 = $(\wedge, P, PP, PPP, \dots)^*$
 = $\{\wedge, P, PP, PPP, \dots\}$
 = P^*
 = R.H.S.

4. $P^*P^* = P^*$

Proof We have, L.H.S. = P^*P^*
 $= \{\wedge, P, PP, PPP, \dots\} \{\wedge, P, PP, PPP, \dots\}$
 $= \{\wedge\wedge, \wedge P, \wedge PP, \wedge PPP, \dots, P\wedge, PP, PPP, PPPP, \dots\}$
 $= \{\wedge, P, PP, PPP, \dots\}$
 $= P^*$
 $= \text{R.H.S.}$

5. $PP^* = P^*P$

Proof We have, L.H.S. = PP^*
 $= P(\wedge, P, PP, PPP, \dots)$
 $= \{P, PP, PPP, PPPP, \dots\}$
 $= P^+$
 $= \{P, PP, PPP, PPPP, \dots\}$
 $= (\wedge, P, PP, PPP, PPPP, \dots)P$
 $= P^*P$

6. $(PQ)^*P = P(QP)^*$

Proof We have, L.H.S. = $(\wedge, PQ, PQPQ, PQPQPQ, \dots)P$
 $= \{P, PQP, PQPQP, PQPQPQP, \dots\}$
 $= P(\wedge, QP, QPQP, QPQPQP, \dots)$
 $= P(QP)^*$
 $= \text{R.H.S.}$

7. $(P + Q)^* = (P^* + Q^*)^* = (P^*Q^*)^*$

Proof We have,

$$(P + Q)^* = (\wedge, P, Q, PP, PQ, QP, QQ, \dots)$$

= All possible string over $\{P, Q\}$

Now $(P^* + Q^*)^* = (\wedge, P, PP, PPP, \dots, \wedge, Q, QQ, QQQ, \dots)^*$
 = All possible string over $\{P, Q\}$

Now $(P^*Q^*)^* = ((\wedge, P, PP, PPP, \dots)(\wedge, Q, QQ, QQQ, \dots))^*$
 = (All possible string over $\{P, Q\})^*$
 = All possible string over $\{P, Q\}$

Therefore, $(P + Q)^* = (P^* + Q^*)^* = (P^*Q^*)^*$

8. $(P + Q)R = PR + QR,$

Proof We have,

$$(P + Q)R = P \text{ or } Q \text{ followed by } R$$

$$= PR \text{ or } QR$$

$$= PR + QR$$

$$= \text{R.H.S.}$$

Example 4.1Prove that $a^*(ab)^*(a^*(ab)^*)^* = (a + ab)^*$ **Solution**

$$\text{L.H.S.} = a^*(ab)^*(a^*(ab)^*)^*.$$

Let $a^*(ab)^*$ be P , then we have

$$\text{L.H.S.} = PP^*$$

$$= P^*$$

$$= (a^*(ab)^*)^*$$

$$= (a + ab)^* \quad [\text{By law 7}]$$

$$= \text{R.H.S.}$$

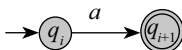
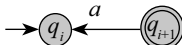
4.6**FINITE AUTOMATA AND REGULAR EXPRESSIONS**

In this section we shall study how a regular expression can be represented by a transition diagram (finite automata) and vice-versa.

Every regular expression P can be recognised by a transition diagram for every string s in P , and there exists a path from starting state to final state (accepting state) which follows path value s .

Now we look how a regular expression can be represented by a transition diagram. Let R be a regular expression consisting of one character. Then $R = a$, $R = \wedge$, or $R = \phi$. Consider Table 4.1 for their representation by a transition diagram:

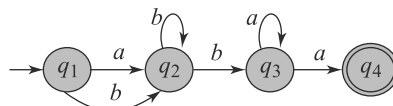
Table 4.1 Regular expressions and their corresponding transition diagrams

REGULAR EXPRESSION	TRANSITION DIAGRAM	REMARK
$R = a$		There exists a path from initial state q_i to final state q_{i+1} on input 'a'.
$R = \wedge$		This is the case when initial state and final states are same, language accepted is $\{\wedge\}$.
$R = \phi$		This is the case when final state is not reachable from initial state, therefore no language is accepted.

You can see above that final state represented by two concentric circles  and the initial state is represented by a single circle with an incoming arrow i.e.,  one another state, intermediate state is represented by a single circle only i.e., 

Example 4.2

Find initial, final and intermediate states by following transition diagram.

**Fig. 4.1** FA of Example 4.2

Solution The initial state is q_1 since it has an incoming arrow without label (or say label $\wedge$). Final state is only q_4 and remaining states i.e., q_2 and q_3 are intermediate states (neither initial nor final).

4.6.1 Construction: Machine From Regular Expression

If a regular expression is given then there is an associated regular language $L(r)$. Since, there is a finite state automaton for every language, so there is machine M for every regular expression such that, $L(M) = L(r)$ (i.e., the language accepted by machine M is same as the language represented by the regular expression r).

First of all the machine (transition system) is constructed with $\wedge$ -moves. Then we eliminate the $\wedge$ -moves present in the FA. Any regular expression consists of $(,), +, *$ and alphabets from Σ . To construct a transition diagram (FA) of given regular expression, we have to eliminate closure, union and concatenation in the reverse order in which they were applied. For example, in $(aa + b)^*$ we will eliminate closure first of all, then union and finally concatenation, because to construct $(aa + b)^*$, first of all concatenation of a with a is performed then union of aa and b is performed, and finally closure of $(aa + b)$ is performed. To construct a transition system from a given regular expression, we consider the following four steps:

- (1) Take two states on initial state and other one the final state and write the regular expression as path value from state to final state. (see Fig. 4.2)
- (2) Now divide regular expression in two or more sub-expressions (if possible).
- (3) Introduce new intermediate states between initial state and final state. If there are n sub-regular expressions, i.e., n different sub expressions are concatenated then new $n - 1$ intermediate states are introduced to eliminate the concatenation operation.
- (4) Now consider following table (Table 4.2) for construction of transition diagram for each sub-regular expression.

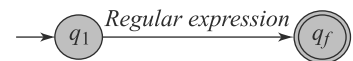


Fig. 4.2 Transition diagram accepting a regular expression

Table 4.2 Transition sub-diagrams corresponding to a portion of regular expression

RULE NO.	REGULAR SUB-EXPRESSION	TRANSITION SUB-SYSTEM (NFA WITH ϵ TRANSITIONS)
1	$R = \wedge$	$\xrightarrow{\wedge} (q_f)$ or simply $\rightarrow (q_f)$
2	$R = \phi$	$\rightarrow (q_i) (q_f)$ or $\rightarrow (q_i) \xrightarrow{a} (q_{i+1})$
3	$R = a$	$\cdots \rightarrow (q_i) \xrightarrow{a} (q_{i+1}) \cdots$
4	$R = a^*$	$\cdots \xrightarrow{\wedge} (q_i) \xrightarrow{a} (q_i) \cdots$
5	$R = P + Q$	$\cdots \rightarrow (q_i) \xrightarrow{P} (q_{i+1}) \xrightarrow{Q} \cdots$
6	$R = PQ$	$\cdots \rightarrow (q_i) \xrightarrow{P} (q_{i+1}) \xrightarrow{Q} (q_{i+2}) \cdots$

The six constructions given in step (4) (given by Table 4.2) correspond to the cases:

- (i) The entire regular expression is the null string, i.e., $L = \{\wedge\}$ or $L = \{\epsilon\}$, where $R = \epsilon$.
- (ii) The entire regular expression is empty, i.e., $L = \phi$
- (iii) An element of the input alphabet Σ , is in the regular expression $R = a$ where, $a \in \Sigma$.
- (iv) A regular expression has the Kleene closure (*) applied to it, i.e., $R = a^*$
- (v) Two regular expressions are joined by the union operator, +, i.e., $R = P + Q$
- (vi) Two regular expressions are joined by concatenation, i.e., $R = PQ$

4.6.2 Kleene's Theorem

Theorem 4.1 If r is a regular expression then there exists a nondeterministic finite automaton M with ϵ (or say $\wedge$) transitions that accepts or recognises the language $L(r)$.

The construct of transition subsystems (parts of NFA with ϵ transitions) is illustrated in Table 4.2.

Example 4.3

Construct transition diagram for regular expression, $R = (ab^*)(a+b)^*(b+ab)$.

Solution Step (i) Let q_i is initial and q_f final state. Then

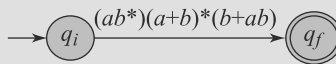


Fig. 4.3 The generalised transition graph of $(ab^*)(a+b)^*(b+ab)$ with two states

Step (ii) Divide this regular expression in three parts i.e., (ab^*) , $(a+b)^*$ and $(b+ab)$

Step (iii) Introduce two new states q_1 and q_2

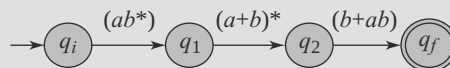


Fig. 4.4 The generalised transition graph with four states

Step (iv) Again insert a new state to divide ab^* in a and b^* . We have

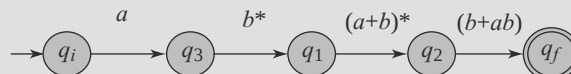


Fig. 4.5 The generalised transition graph with five states

Step (v) Now, modify this transition diagram according to Table 4.2 in rule (4). We have

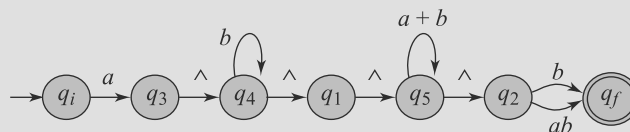


Fig. 4.6 Finite automaton with $\wedge$ -moves

Then introduce two new states to separate ab and $(a + b)$. We have

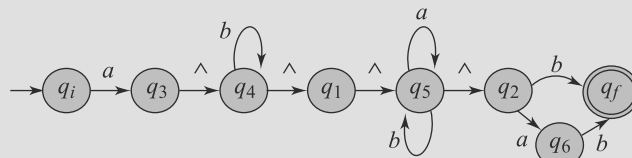


Fig. 4.7 Finite automaton of Example 4.3 with $\wedge$ -moves

Example 4.4

Design finite automata not to accept all strings over $\{0, 1\}$ starting with 000.

Solution The regular expression to represent all strings over $\{0, 1\}$ not starting with 000 is

$$R = \wedge + 0 + 1 + 00 + 01 + 10 + 11 + (001 + 010 + 011 + 100 + 101 + 110 + 111)(0+1)^*$$

The required FA is constructed below

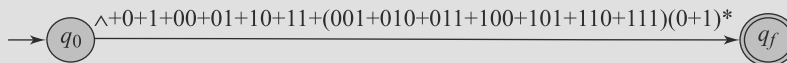


Fig. 4.8 The generalised transition graph of Example 4.4

After elimination of union concatenation and closure, we get FA given by Fig. 4.9.

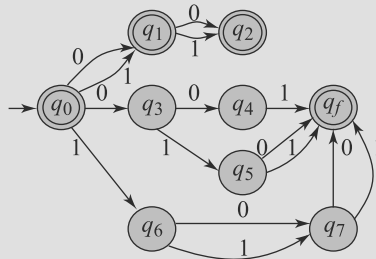


Fig. 4.9 The Finite automaton of Example 4.4

Example 4.5

Design FA to accept all strings over $\{0, 1\}$ containing 010 and 101 as substrings.

Solution The regular expression corresponding to all strings over $\{0, 1\}$ containing 010 and 101 as substrings is

$$(0, 1)^*(010(0, 1)^*101 + 101(0, 1)^*010) (0, 1)^*$$

The finite automata corresponding to this regular expression is

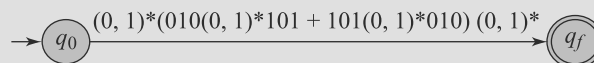


Fig. 4.10 The generalised transition graph of Example 4.5

After elimination of concatenation, union and closure, we get the following finite automaton:

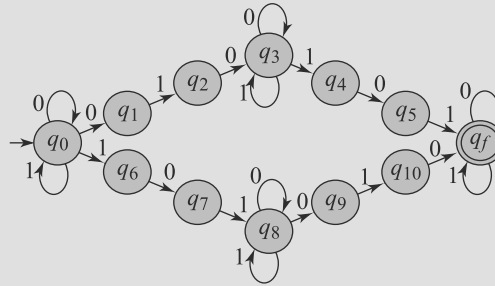


Fig. 4.11 The finite automaton of Example 4.5

Example 4.6

Design finite automata to accept all strings over $\{0, 1\}$ containing 010 or 101 as substring.

Solution The regular expression corresponding to all strings over $\{0, 1\}$ containing 010 and 101 as substrings is

$$(0+1)^*(010+101)(0+1)^*$$

The finite automata corresponding to this regular expression is

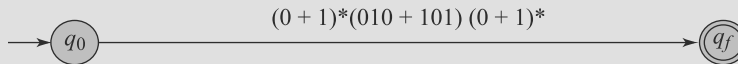


Fig. 4.12 The generalised transition graph of Example 4.6

After elimination of concatenation, union and closure, we get the following finite automaton:

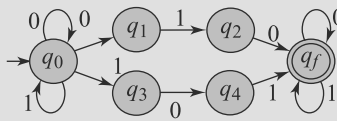


Fig. 4.13 Finite automaton of Example 4.6

4.6.3 ARDEN'S THEOREM

Theorem 4.2 (Arden's Theorem) If P and Q are two regular expressions over Σ and $\wedge \in P$ is not correct, then the equation $R = Q + RP$ in R has a unique solution given by $R = QP^*$.

Proof Let us consider the R.H.S. of equation $R = Q + RP$

$$\begin{aligned}
 Q + RP &= Q + (QP^*)P \\
 &= Q + QP^*P \\
 &= Q + QP^*
 \end{aligned}$$

$$\begin{aligned}
 &= Q(\wedge + P^*) \\
 &= QP^*.
 \end{aligned}$$

Therefore the equation $R = Q + RP$ is satisfied when $R = QP^*$. This shows that the equation $R = QP^*$ is a solution of $R = Q + RP$.

Let us prove the uniqueness of the solution. Consider the equation $R = Q + RP$. Now replace R by $Q + RP$. We have

$$\begin{aligned}
 R &= Q + RP = Q + (Q + RP)P \\
 &= Q + QP + RPP \\
 &= Q + QP + (Q + RP)PP \quad \text{put again } R = Q + RP \\
 &= Q + QP + QPP + RPPP \\
 &= Q + QP + QP^2 + RP^3 \\
 &\vdots \\
 &= Q + QP + QP^2 + QP^3 + QP^4 + \dots + QP^i + RP^{i+1} \\
 R &= Q(\wedge + P + P^2 + P^3 + \dots + P^i) + RP^{i+1} \text{ for } i \geq 0.
 \end{aligned}$$

Now we show that any solution of $R = Q + RP$ is equivalent to QP^* . Let R satisfies $R = Q + RP$ then it satisfies $R = Q(\wedge + P + P^2 + P^3 + \dots + P^i) + RP^{i+1}$.

Let s be a string of length i in the set R , then s belongs to the set $Q(\wedge + P + P^2 + P^3 + \dots + P^i) + RP^{i+1}$. As we know that P does not contain $\wedge$, so RP^{i+1} has no string of length less than $i+1$ and therefore string s is not in the RP^{i+1} . So s belongs to the set $Q(\wedge + P + P^2 + P^3 + \dots + P^i)$ i.e., QP^* .

Now consider a string s in QP^* . Then s is in the set QP^k for some $k \geq 0$. Thus, s is in R.H.S. of equation

$$R = Q(\wedge + P + P^2 + P^3 + \dots + P^i) + RP^{i+1}.$$

Therefore s is in L.H.S. of equation $R = Q(\wedge + P + P^2 + P^3 + \dots + P^i) + RP^{i+1}$. Now we can say that R and QP^* represent the same set of strings.

4.6.4 Application of Arden's Theorem

Arden's theorem is used to determine the regular expression represented by a transition diagram. The following points are assumed regarding transition diagrams:

- (i) The transition system does not have any $\wedge$ -move
- (ii) It has only one initial (starting) state, say q_1 .
- (iii) Its vertices are $q_1, q_2, q_3, \dots, q_n$.
- (iv) R_i is the regular expression representing the set of strings accepted by the transition system if q_i is the final state.
- (v) P_{ij} represents the regular expression representing the set of labels of edges from q_i to q_j (concatenation of labels from state q_i to state q_j). If there is no path from state q_i to q_j then $P_{ij} = \phi$.

We can get the following set of equations in terms of $q_1, q_2, q_3, \dots, q_n$.

$$\begin{aligned}
 q_1 &= q_1 P_{11} + q_2 P_{21} + \dots + q_n P_{n1} + \wedge \\
 q_2 &= q_1 P_{12} + q_2 P_{22} + \dots + q_n P_{n2} \\
 &\vdots \\
 q_n &= q_1 P_{1n} + q_2 P_{2n} + \dots + q_n P_{nn}.
 \end{aligned}$$

By repeatedly applying substitutions and Arden's theorem we can express final state q_i in terms of symbols in Σ , i.e., P_{ij} .

To obtain the resultant set of strings recognised by the transition system, we take the union of all q_i 's (final states) if there are more than one final states in the transition diagram. This also supports the fact that if there is no final state (i.e., there is no path from initial state to final state) then the language accepted by that FA is $L = \{\emptyset\}$. Therefore, if an FA has no final state, it will not recognise any input.

Example 4.7

Consider the transition diagram given in Fig. 4.14. Construct the regular expression corresponding to this transition diagram.

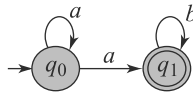


Fig. 4.14 Finite automaton of Example 4.7

Solution There will be two equations, one for state q_0 and one for q_1 . Equations will be written as we have explained set of equations in Section 4.8.4,

$$q_0 = q_0 a + \wedge + q_0 a$$

and

$$q_1 = q_0 a + q_1 b.$$

By applying Arden's theorem on equation $q_0 = \wedge + q_0 a$, we get

$$q_0 = \wedge a^* = a^*$$

Put $q_0 = a^*$ in equation $q_1 = q_0 a + q_1 b$, we get

$$q_1 = a^* a + q_1 b$$

By applying Arden's theorem on equation $q_1 = a^* a + q_1 b$, we get

$$q_1 = a^* a b^* = a^+ b^*$$

As there is only one final state in transition diagram of this example, so the resultant regular expression is $a^+ b^*$ or $a^* a b^*$.

Example 4.8

Consider the transition system of a finite automaton given in Fig. 4.15. Construct a regular expression corresponding to this transition system.

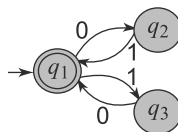


Fig. 4.15 Finite automaton of Example 4.8

Solution According to the transition system given by Fig. 4.15. We have three equations for q_1 , q_2 and q_3

$$q_1 = q_2 1 + q_3 0 + \wedge$$

$$q_2 = q_1 0$$

$$q_3 = q_1 1$$

Put $q_2 = q_1 0$ and $q_3 = q_1 1$ in equation $q_1 = q_2 1 + q_3 0 + \wedge$, we get

$$q_1 = q_1 01 + q_1 10 + \wedge$$

$$q_1 = q_1 (01 + 10) + \wedge = \wedge + q_1 (01 + 10)$$

By applying Arden's theorem on equation $q_1 = \wedge + q_1 (01 + 10)$, we get

$$q_1 = \wedge (01 + 10)^*$$

$$= (10 + 10)^*$$

Since, there is only one final state, so the regular expression corresponding to the given transition system will be expressed by q_1 (the final state). Hence, the resultant regular expression is

$$(01 + 10)^*.$$

Example 4.9

Determine the regular expression corresponding to the transition system given by Fig. 4.16.

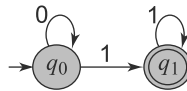


Fig. 4.16 Finite automaton of Example 4.9

Solution We have the following equations for q_0 and q_1

$$q_0 = q_0 0 + \wedge,$$

and

$$q_1 = q_0 1 + q_1 1$$

By applying Arden's theorem on equation $q_0 = \wedge + q_0 0$, (equivalent to $q_0 = q_0 0 + \wedge$), we have

$$q_0 = \wedge 0^* = 0^*$$

Put $q_0 = 0^*$ in equation $q_1 = q_0 1 + q_1 1$. We get

$$q_1 = 0^* 1 + q_1 1$$

Now apply Arden's theorem on equation $q_1 = 0^* 1 + q_1 1$, we get

$$q_1 = (0^* 1) 1^*$$

As the transition system in Fig. 4.16, shows that both q_0 and q_1 are final states. Therefore.

$$\begin{aligned} q_0 + q_1 &= 0^* + (0^* 1) 1^* \\ &= 0^* + 0^* 1^* \\ &= 0^* (\wedge + 1^*) \\ &= 0^* 1^* \end{aligned}$$

Hence, the resultant regular expression is $0^* 1^*$.

4.7

EQUIVALENCE OF TWO REGULAR EXPRESSIONS

Two regular expressions are equivalent if:

- They represent same sets (or strings).
- Their corresponding FA are same.
- One regular expression is proved equivalent to another by using identities.

To find the equivalence of two regular expressions by (i), we find regular sets of both regular expressions. If both sets have same strings then both the regular expressions are equivalent. Let us consider two regular expressions $P = (a^* + b)^*$ and $Q = (a + b^*)^*$. Suppose P_s and Q_s are the sets of strings corresponding to P and Q respectively. The sets of strings P_s and Q_s are determined as following:

$$\begin{aligned} P_s &= (a^* + b)^* \\ &= (a^* \cup b)^* \\ &= \{\wedge, a, aa, aaa, aaaa, \dots, b\}^* \\ &= \{\wedge, a, b, aa, ab, ba, bb, aaa, aab, \dots\} \end{aligned}$$

and

$$\begin{aligned} Q_s &= (a + b^*)^* \\ &= (a \cup b^*)^* \\ &= \{a, \wedge, b, bb, bbb, \dots\}^* \\ &= \{\wedge, a, b, aa, ab, ba, bb, aaa, aab, \dots\} \end{aligned}$$

We can see that the set of strings represented by P_s and Q_s are same. Therefore, both the regular expressions are equivalent.

To find the equivalence of two regular expressions by (ii), we construct finite automata for both the regular expressions. Two finite automata M_1 and M_2 are said to be equivalent if they accept the same set of strings over Σ . Two finite automata M_1 and M_2 are said not be equivalent if one FA reaches into final state on application of $s \in \Sigma^*$, whereas the other FA reaches into a nonfinal state.

Let us construct FA M_1 and M_2 corresponding to regular expressions P and Q .

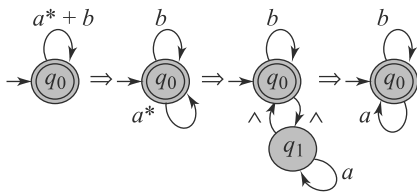


Fig. 4.17 Finite automata M_1 corresponding to P

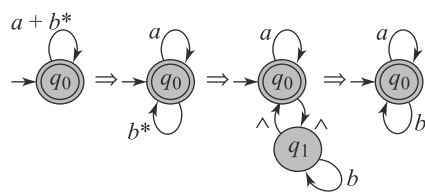


Fig. 4.18 Finite automata M_2 corresponding to Q

To find the equivalence of two regular expressions by (iii), we prove one regular expression equivalent to the other by using algebraic laws and identities. Let us take $P = (a^* + b)^*$ first

$$\begin{aligned} P &= (a^* + b)^* \\ &= ((a^*)^* b^*)^* && \text{by } (P + Q)^* = (P^* Q^*)^* \\ &= (a^* b^*)^* && \text{by } (P^*)^* = P^* \\ &= (a^* (b^*)^*)^* && \text{by } P^* = (P^*)^* \\ &= (a^* + (b^*)^*)^* && \text{by } (P^* Q^*)^* = (P + Q)^* \\ &= (a + b^*)^* && \text{by } (P^* + Q^*)^* = (P + Q)^* \\ &= Q \end{aligned}$$

Thus, regular expression P and Q are equivalent.

4.8

MYHILL-NERODE THEOREM

The Myhill–Nerode theorem is used for minimisation of DFAs. It is one of the decision properties of regular languages.

A binary relation R on a set X is an equivalence relation if only if

1. R is reflexive, that is xRx for all $x \in X$
2. R is symmetric, that $xRy \Rightarrow yRx$ is for all $x, y \in X$
3. R is transitive, that xRy and $yRz \Rightarrow xRz$ for all $x, y, z \in X$.

An equivalence relation R on Σ^* partitions R into subsets X_i (where $i \in I$) called the equivalence classes of R , defined by:

1. For all $i \in I$ and $x, y \in X_i$ it holds that xRy .
2. For all $i, j \in I$ and $i \neq j$ for all $x \in X_i$ and $y \in X_j$ it holds that not xRy .

The number $|I|$ of equivalence classes is the *index* of R . If $C \subseteq X$ is an equivalence class containing x , we call x a *representative* of C and write $C = [X]_R$.

❖ Definitions

Right invariant relation: An equivalence relation R on Σ^* is right invariant if for all $u, v, w \in \Sigma^*$, it holds that $uRv \Rightarrow uRvw$.

Binary relation: Let A be a DFA with starting state q_0 . Define a binary relation R_A on Σ^* by $uR_A v$ if and only if $\delta(q_0, u) = \delta(q_0, v)$. This is an equivalence relation because “=” is one and R_A is defined via “=” of states

Obviously R_A is right-invariant, has finite index, and $L(A)$ is the union of some of the equivalence classes of R_A (namely those for which $\delta(q_0, v)$ leads into an accepting state). This insight is considerably refined in the following theorem.

Theorem 4.3 (The Myhill–Nerode theorem; the algebraic characterisation of regular language). Let $L \subseteq \Sigma^*$ be a language. The following three statements are equivalent:

1. $L \subseteq \Sigma^*$ is regular, that is, $L = L(A)$ for some DFA A .
2. L is the union of some of the equivalence classes of a right-invariant equivalence relation on all strings taken from Σ^* of finite index.
3. Let an equivalence relation R_L on Σ^* be defined by:

$$uR_L v \text{ if and only if for all } w \in \Sigma^*: uw \in L \Leftrightarrow uv \in L$$

Then R_L is of finite index.

The Myhill–Nerode theorem gives yet another characterisation of the regular languages. Its proof involves the construction of minimal automata, which is an important technique in its own right.

Proof

1. We have seen it already.
2. Any equivalence relation R satisfying 2 could be shown is a refinement of R_L , that means, every equivalence class of R is entirely contained in some equivalence class of R_L . This way, the index

of R_L cannot be greater than the index of R and therefore it is finite. Let us assume that uRv . Since R is right invariant, then for every $w \in \Sigma$, we have $uwRvw$, and therefore, uR_Lv . Thus, R is a refinement of uR_Lv .

3. We first prove that R_L is right invariant. Let us assume that uR_Lv . For any $y \in \Sigma^*$ we must show that uyR_Lvy , that means, for any $z \in \Sigma^*$, $uyz \in L \Leftrightarrow vyz \in L$. But this follows from the definition of R_L in 3 if we put $w = yz$.

We now construct a DFA A_{MN} accepting L . The basic technique is to use states as the equivalence classes of R_L (of which there is only a finite number). So,

$$A_{MN} = (Q, \Sigma, \delta, q_0, F) = (\{[w]_{RL} \mid w \in \Sigma^*\}, \Sigma, q_0, \delta_{MN}, F_{MN}) \text{ We put}$$

- (i) $q_0 = [\epsilon]_{RL}$
- (ii) for all $w \in \Sigma^*$, $a \in \Sigma$ and $\delta_{MN}([w]_{RL}, a) = [wa]_{RL}$
- (iii) $F_{MN} = \{[w]_{RL} \mid w \in L\}$

We first have to show that this is well-defined, especially that (ii) makes sense, that is, the definition of δ_{MN} is independent of the choice of representative w of the equivalence class $[w]_{RL}$. So we have to show that if wR_Lv , then $[w]_{RL} = \delta_{MN}([w]_{RL}, a) = \delta_{MN}([v]_{RL}, a) = [va]_{RL}$. But if wR_Lv , then waR_Lva because R_L is right-invariant, therefore $[wa]_{RL} = [va]_{RL}$. So δ_{MN} is well-defined.

Finally we have to show that $L = L(A_{MN})$. This follows from $\hat{E}_{MN}([\epsilon]_{RL}, w) = [w]_{RL}$, where $\hat{E}(q, x) = \wedge\text{-closure}(\delta(q, x))$ and $\wedge\text{-closure}(q)$ is set of all states reachable from q on input $\wedge$.

The following is an example that illustrates the Myhill–Nerode theorem. Let L be the language $0^*10^*1(0+1)^*$, that is, all words over $\{0, 1\}$ that contain at least two 1's. L is accepted, for instance, by the DFA A shown in Fig. 4.19.

DFA A yields a right-invariant equivalence relation R_A as follows:

R_A has six equivalence classes corresponding to the states of A . These equivalence classes each contain the words that from the starting state into that class:

$$\begin{array}{ll} C_a = (00)^* & C_b = (00)^*0 \\ C_c = (00)^*1 & C_d = (00)^*01 \\ C_e = (00)^*100^* + (00)^*0100^* & C_f = 0^*10^*1(0+1)^* \end{array}$$

L is the union of C_c , C_d and C_e , because these are final states in given DFA A . The relation R_L for L has wR_Lv if and only if either

- (a) w and v have no 1's [= equivalence class D_1]
- (b) w and v each have one [= equivalence class D_2]
- (c) w and v each have more than one 1 [= equivalence class D_3].

The classes C_i define the classes D_j by $D_1 = C_a \cup C_b$, $D_2 = C_c \cup C_d \cup C_e$, and $D_3 = C_f$

$$\begin{aligned} D_1 &= C_a \cup C_b = (00)^* \cup (00)^*0 \\ &= (00)^*(\wedge+0) \\ &= 0^* \end{aligned}$$

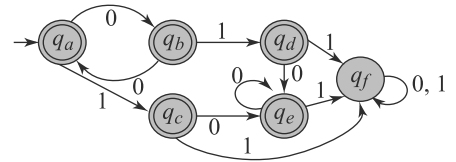


Fig. 4.19 DFAA accepting $L = 0^*10^*1(0+1)^*$

$$\begin{aligned}
 D_2 &= C_c \cup C_d \cup C_e = (00)^*1 \cup (00)^*01 \cup (00)^*100^* + (00)^*0100^* \\
 &= (00)^*(1+01) + (00)^*(1+01)00^* \\
 &= 00^*(1+01)(\wedge+00^*) \\
 &= 00^*(1+01)0^* \\
 &= 0^*10^* \\
 D_3 &= C_f = 0^*10^*1(0+1)^*
 \end{aligned}$$

We may denote the three equivalence classes of R_L by $D_1 = [\wedge]_{RL} = 0^*$, $D_2 = [1]_{RL} = 0^*10^*$, $D_3 = [11]_{RL} = 0^*10^*1(0+1)^*$. The DFA A_{MN} we obtained according to the construction in the proof of the Myhill-Nerode theorem is the following:

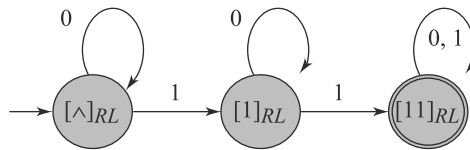


Fig. 4.20 The DFA A_{MN}

As another extension, the Myhill-Nerode theorem provides a necessary and sufficient condition for a language to be a regular language. This prepares a framework that can be used to prove that a given language is not regular.

For a given a language L , we define a relation R_L on strings by the rule $xR_L y$ if there is no unique extension z having the property of exactly one of the strings xz and yz is in L .

Now we can show that R_L is an equivalence relation on strings, this way it divides the set of all finite strings into one or more equivalence classes.

The Myhill-Nerode Theorem states that the number of states in the optimised (or say minimised) finite automaton accepting the language L is equal to the number of equivalence classes in R_L . The perception is that if we start with such an optimised automaton, then any strings 'x' and 'y' that drive it to the same state will be in the same equivalence class. Also, if we start with a partition into equivalence classes, we can easily construct a finite automaton that uses its state to keep track of the equivalence class containing the part of the string scanned so far.

4.9

REGULAR SETS

Any set represented by a regular expression is known as regular set. Let us consider the Table 4.3 (below):

Table 4.3 Some regular sets corresponding to given regular expressions

REGULAR EXPRESSION	REGULAR SET
a	$\{a\}$
abb	$\{abb\}$
a^*	$\{\wedge, a, aa, aaa, aaaa, \dots\}$
$(a+b)^*$ or $(a b)^*$	$\{\wedge, a, b, aa, ab, ba, bb, aaa, \dots\}$
$(a+b)^*bb$	$\{bb, abb, bbb, aabb, abbb, babbb, bbbbb, \dots\}$

4.10

CLOSURE PROPERTIES OF REGULAR SETS

In earlier section on regular expressions in this chapter, we have seen that the class of regular sets is closed under union, concatenation and closure. Union, concatenation, closure (iteration) *etc.* are such operations to regular sets, which result in regular sets. For example the union of two regular sets is a regular set, since if r_1 and r_2 are regular expressions denoting regular sets L_1 and L_2 , respectively, then $r_1 + r_2$ denotes $L_1 \cup L_2$. Hence, $L_1 \cup L_2$ is also regular. Similarly the concatenation of regular sets *e.g.*, $L_1 L_2$ is also a regular set, and the Kleene closure of a regular set is also regular.

When a class of language is closed under a particular operation, we call it a closure property of the class of languages.

In this section we will discuss closure properties of regular sets under (i) union, (ii) concatenation, (iii) closure (iteration), (iv) transpose, (v) intersection, and (vi) complement.

Theorem 4.4 The regular sets are closed under union, concatenation and Kleene closure, i.e., the union, concatenation and closure of regular sets is also a regular set.

Proof Let Σ be a finite set of symbol, and let L, L_1 and L_2 be sets of strings from Σ^* . The concatenation of L_1 and L_2 denoted by $L_1 L_2$, is the set $\{xy \mid x \text{ is in } L_1 \text{ and } y \text{ is in } L_2\}$. That is, the strings in $L_1 L_2$ are formed by choosing a string L_1 and following it by a string in L_2 , in all possible combinations.

Theorem 4.5 If L is regular then its transpose (denoted by L^T) is also regular.

Proof Let us assume that L is regular set. Then $L = T(M)$ for some deterministic finite automaton,

$$M = (Q, \Sigma, \delta, q_0, F),$$

where $T(M)$ is the set accepted by DFA M . We can construct a DFM M such that $T(M) = L$.

We construct a transition diagram M' by starting with the transition diagram of DFA M , and reversing the direction of the directed edges. The set of initial states of M' is defined as set of final states F ; q_0 (the initial state) is defined as the final state of M' , and the set of final states F is defined as $\{q_0\}$ the initial states of M . This way the DFA M' is defined as $M' = (Q, \Sigma, \delta, F, \{q_0\})$.

The string ' s ' the path value from state q_0 to some final state in F , if $s \in T(M)$. By reversing the direction of edges, we get a pair in M' from final state in F to the initial state q_0 . Its paths value is s^T (i.e., the reverse of string s). So $s^T \in T(M)$. In the same way, we can see this for some other strings also like if $s_1 \in T(M')$, then $s_1^T \in T(M)$. Thus from the transition diagrams given by Fig. 4.21 (a) and (b), it is easy to see that $T(M) = T(M)^T$.



Fig. 4.21 (a), (b) DFA M and its transpose

Thus, we can prove for any string s , that $s \in T(M)$ if and only if $s^T \in T(M)$ by induction on s hence, $T(M)^T = T(M)$ this proves the theorem.

Example 4.10

Consider the automaton M given by Fig. 4.22. What is $T(M)$? Show that $T(M)^T$ is regular.

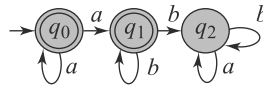


Fig. 4.22 FA of Example 4.10

Solution $T(M)$ is set of all path values from initial state (i.e., q_0) to final states. In this example final states are q_0 and q_1 . Therefore $T(M)$ consists of path values from state q_0 to itself and from state q_0 to q_1 .

From state q_0 to q_0 the corresponding path value is a^i , $i \geq 1$ because from q_0 to itself there is a loop repeated any number of times.

As q_1 is also final state, the paths from q_0 to q_1 are of the form

$$q_0 \xrightarrow{a} q_0 \xrightarrow{a} \cdots \xrightarrow{a} q_0 \xrightarrow{b} q_1 \xrightarrow{b} q_1 \cdots \xrightarrow{b} q_1$$

The corresponding path values are $a^i b^j$, where $i \geq 0, j \geq 0$. As q_0 is also final state therefore $\wedge$ must be in $T(M)$. Thus,

$$T(M) = \{ a^i b^j \mid i, j \geq 0 \}$$

Let us look on the transpose of $T(M)$. The transpose of $T(M)$ is

$$T(M)^T = \{ b^j a^i \mid i, j \geq 0 \}$$

The construction of finite automaton of $T(M)^T$, say M' is given by Fig. 4.23. The transition diagram for the transpose of M (denoted by M') is constructed by following steps:

- (i) The initial state of M' are q_0 and q_1 , because q_0 and q_1 are the final states of M .
- (ii) The only final state M' is q_0 because q_0 is the only initial state of M .
- (iii) The direction of the directed edges is reversed.

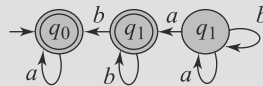


Fig. 4.23 FA M' for transpose of $T(M)$

By the above construction through (i)-(iii), we can say, that

$$T(M') = T(M')^T$$

Therefore, $T(M)^T$ is regular

Reverse of an FA The language that the reverse FA should accept uses the operator reverse or its specification contains words like *from right to left*. For example, if $10011 \in L$, then $11001 \in L'$.

To obtain the finite automaton M' having $\wedge$ -moves accepting L' (the reverse language of L), we modify the given finite automaton M as the following:

1. Reverse all arrows of the transition diagram M ,
2. Make the initial state of M the one and only accepting state of the NFA, and
3. Add a new initial state q' and $\wedge$ -transitions from q' to all accepting states M .

Let us consider the language,

$$L = \{\omega \mid \omega \% 4 = 0 \text{ when interpreted as a binary integer and is read from right to left}\}.$$

The finite automaton for language L is following:

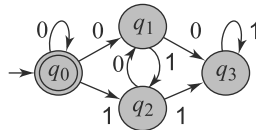


Fig. 4.24 FA accepting L

The reverse finite automaton is given by the following figure:

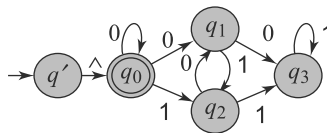


Fig. 4.25 FA accepting L^T

Theorem 4.6 The class of regular sets is closed under complementation, i.e., the complement of regular set is also a regular set.

Proof Let us assume L is regular. We can construct a DFA $M = (Q, \Sigma, \delta, q_0, F)$, accepting L , or in other words $L = T(M)$.

Now we construct another DFA $M' = (Q, \Sigma, \delta, q_0, F')$, by defining $F' = Q - F$, such that a final state of M is a nonfinal state of M' and vice-versa. Thus, the transition diagram of DFA M and M' are the same with only the difference of final states.

$s \in T(M')$ if and only if $\delta(q_0, s) \in F' = Q - F$, if only if $s \in L$. This proves

$$T(M') = \Sigma^* - L.$$

Theorem 4.7 The regular sets are closed under intersection, i.e., the intersection of two regular sets is also regular.

Proof Let L_1 and L_2 be two regular sets. The intersection of L_1 and L_2 is defined as:

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$

where, the over bar denotes complementation (e.g., $\overline{L_1}$ is the complement of L_1) with respect to an alphabet including the alphabets of L_1 and L_2 . Closure under intersection, is followed by closure under union and complementation.

Theorem 4.8 The class of regular sets is closed under substitution.

Here we are not giving the proof of this theorem. A simple induction on the number of operators in the regular expression completes this PROOF

Theorem 4.9 The class of regular sets is closed under homeomorphisms.

Closure under homeomorphisms follows immediately from closure under substitution, since every homeomorphism is a substitution.

❖ Definitions

Substitution Let Σ be an alphabet with $a \in \Sigma$, and we choose a language L_a that depends upon a function called substitution (say *sub*) on Σ , then we can refer L_a as *sub*(a) for each $a \in \Sigma$.

Homeomorphism A string homeomorphism is defined as a function on set of strings that substitute a particular string for each symbol. For example if a homeomorphism function H is defined as $H(0) = abb$ and $H(1) = ba$, then after homeomorphism of string 1011 gives *baabbbaba*

4.11

REGULAR GRAMMAR AND FA

Up to this point we have seen that, the sets accepted by deterministic finite automata are regular sets. In this section we see that the class of regular sets over Σ is precisely the regular language over the terminal set.

4.11.1 Construction of Regular Grammar Generating $T(M)$

Let M be a given DFA. The elements of $T(M)$ are defined as the path values concatenated, the path from initial state to the final state.

Suppose, 's' is obtained by concatenating the labels corresponding to several transitions from the initial state to some final state. So the construction of regular grammar, generating $T(M)$ for a given DFA, must be corresponding to the transitions in that DFA. Another important thing is that there should be a provision for terminating the derivation. Once a transition is terminated some final state is encountered. By keeping these things in mind we construct regular grammar G as:

$$G = (V_N, \Sigma, P, A_0)$$

where

A_0 is the start symbol and P , the set of productions is defined by the following rules:

- (i) $\{A_i \rightarrow aA_j\} \in P$ if $\delta(q_i, a) = q_j$ where q_j is any nonfinal state i.e., $q_j \notin F$.
- (ii) $\{A_i \rightarrow aA_j, A_i \rightarrow a\} \in P$ if where q_j is a final state (i.e., $q_j \in F$)

If there is an FA accepting only $\wedge$, is given below.

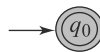


Fig. 4.26 FA accepting only $\wedge$

Then the corresponding grammar is $\{A_0 \rightarrow \wedge A_0 \mid \wedge\}$ or simply $A_0 \rightarrow \wedge$ because $A_0 \rightarrow A_0$ is useless.

Theorem 4.10 Prove that $L(G) = T(M)$, where G is the regular grammar and $T(M)$ is the set of strings accepted by DFA M .

Proof By using the construction of production set P in this section, we have production like

$$A_i \rightarrow aA_j \text{ if there exists a transition } \delta(q_i, a) = q_j$$

$$A_i \rightarrow a \text{ if and only if } \delta(q_i, a) \text{ is in } F.$$

Therefore

$$\begin{aligned} A_0 &\Rightarrow a_1 A_1 \\ &\Rightarrow a_1 a_2 A_2 \\ &\Rightarrow a_1 a_2 a_3 A_3 \\ &\Rightarrow a_1 a_2 a_3 \dots a_{k-2} a_{k-1} A_{k-1} \\ &\Rightarrow a_1 a_2 a_3 \dots a_k \end{aligned}$$

if and only if

$$\delta(q_0, a_1) = q_1$$

$$\delta(q_1, a_2) = q_2$$

$$\delta(q_{k-1}, a_k) = q_k \in F$$

this proves that

$$s = a_1 a_2 a_3 \dots a_k \text{ with } s \in L(G)$$

Hence, $s \in T(M)$.

Example 4.11

Construct a regular grammar G generating the regular expression $0^*(0 + 1)^*$.

Solution First of all we construct FA corresponding to regular expression $0^*(0 + 1)^*$. First we construct FA with $\wedge$ -transitions as given by Fig. 4.27.

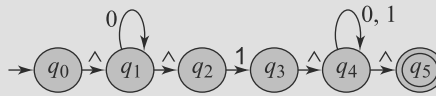


Fig. 4.27 Finite automaton with $\wedge$ -moves, of Example 4.11

After elimination of $\wedge$ -transitions, we have

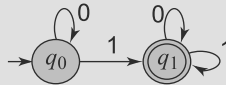


Fig. 4.28 Finite automaton without $\wedge$ -moves, of Example 4.11

The regular grammar G corresponding to this DFA is given by the following productions (as explained in Section 4.13.1):

$$(a) \quad A_0 \rightarrow 0A_0 \quad [\text{by rule (i)}]$$

$$(b) \quad A_0 \rightarrow 1A_1 \text{ and } A_0 \rightarrow 1 \quad [\text{by rule (ii)}]$$

$$(c) \quad A_1 \rightarrow 0A_1, A_1 \rightarrow 0 \text{ and } A_1 \rightarrow 1A_1, A_1 \rightarrow 1 \quad [\text{by rule (ii)}]$$

By the above productions we can easily decide that

$$V_N = \{A_0, A_1\}, \quad \Sigma = \{0, 1\}, \quad S = A_0.$$

Hence, $G = (\{A_0, A_1\}, \{0, 1\}, P, A_0)$, where P is given by (a), (b), and (c) (See above)

Example 4.12 Construct a regular grammar G generating the regular set represented by $\{a, aa, aaa, aaaa, \dots\}$.

Solution The regular expression corresponding to the regular set $\{a, aa, aaa, aaaa, \dots\}$ is $a(a)^*$ or we can say a^+ .

Let us construct DFA corresponding to regular expression $a(a)^*$ first. The constructed DFA is given by Fig. 4.29 after elimination of $\wedge$ -moves.

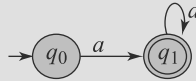


Fig. 4.29 DFA of Example 4.12

Let us construct set of production rules corresponding to this DFA (Fig. 4.29) by using rules (i) and (ii) given in Section 4.13.1.

$$(a) A_0 \rightarrow aA_1, A_0 \rightarrow a \quad [\text{by rule (ii)}]$$

$$(b) A_1 \rightarrow aA_1, A_1 \rightarrow a \quad [\text{by rule (ii)}]$$

Hence, $G = (\{A_0, A_1\}, \{a, b\}, P, A_0)$, where P is given by (a), and (b) (See above)

4.11.2 From Regular Grammar to FA

For given regular grammar $G = (V_N, \Sigma, P, A_0)$, where $V_N = \{A_0, A_1, A_2, A_3, \dots, A_n\}$ we construct a transition system in which:

1. all state of the transition system belong to V_N
2. initial state corresponding to variable A_0 , which is assumed as the start symbol
3. the transitions in the transition system correspond to the set of productions P

If the last productions applied in any derivation is of the form $A_i \rightarrow a$, the corresponding transition terminates at some next state (new state), and this is unique final state.

We can define the transition system M as

$$M = (\{q_0, q_1, q_2, q_3, q_4, \dots, q_n, q_f\}, \Sigma, \delta, q_0, \{q_f\})$$

where δ is defined as follows:

- (i) For each production of the form $A_i \rightarrow aA_j$, there is a transition from state q_i to q_j on input a .
- (ii) For each production of the form $A_m \rightarrow b$, there is a transition from state q_m to q_f on input b , where q_f is the final state.

The total number of states in M is one greater than total number of variables in G . From the above construction we can say that

$$\begin{aligned}
 A_0 &\Rightarrow a_1 A_1 \\
 &\Rightarrow a_1 a_2 A_2 \\
 &\Rightarrow a_1 a_2 a_3 A_3 \\
 &\vdots \\
 &\vdots \\
 &\Rightarrow a_1 a_2 a_3 \dots a_{n-2} a_{n-1} A_n \\
 &\Rightarrow a_1 a_2 a_3 \dots a_n
 \end{aligned}$$

$A_0 \Rightarrow a_1 a_2 a_3 \dots a_n$ is a derivation of $a_1 a_2 a_3 \dots a_n$ if and only if there is a path in the transition system M starting from initial state q_0 to final state q_f . Hence, the language $L(G)$ generated by regular grammar G , is equivalent to the set of string $T(M)$ accepted by the transition system. Therefore,

$$L(G) = T(M)$$

Example 4.13

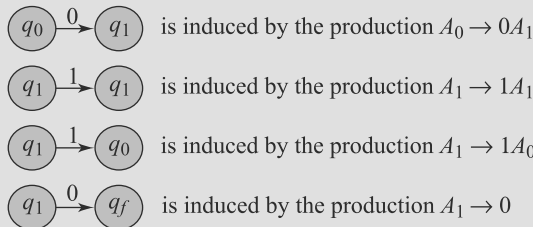
Consider the regular grammar $G = (\{A_0, A_1\}, \{0, 1\}, \{A_0 \rightarrow 0A_1, A_1 \rightarrow 1A_1 \mid 0 \mid 1A_0\}, A_0)$. Construct the transition system M , which accepts the language generated by grammar G .

Solution Suppose the transition system to be constructed is defined as

$$M = (\{q_0, q_1, q_f\}, \{0, 1\}, \delta, q_0, \{q_f\})$$

where q_0 and q_1 correspond to A_0 and A_1 respectively. q_f is the newly introduced state which is the final state, used for the productions of the form $A_m \rightarrow b$.

By applying the rules (i) and (ii) given in this section, we have the following transitions:



Let us combine these transition subsystems into a single transition system by denoting q_0 as initial state and q_f final state.

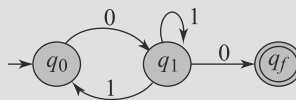


Fig. 4.30 Finite automaton of Example 4.13

Example 4.14

Construct transition system M accepting the regular language $L(G)$, for regular grammar given by productions $S \rightarrow aS \mid a \mid b$.

Solution As there is only one variable S in the grammar G , so there will be two states in M . One states is initial state q_0 which corresponds to S (the start symbol), and the another state is q_f (newly introduced state). Hence the transition system M is defined as

$$M = (\{q_0, q_f\}, \{a, b\}, \delta, q_0, \{q_f\})$$

Let us construct the sub-parts of the transition system M , as follows:

The transition sub-diagram

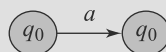


Fig. 4.31 A part of a transition diagram

is induced by the production $S \rightarrow aS$ by following rule (i) as given in this section.

The transition sub-diagram

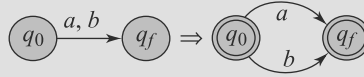


Fig. 4.32 Two equivalent transition sub-diagrams

is induced by the productions $S \rightarrow a$ and $S \rightarrow b$ by following rule (ii)

Let us combine these sub-parts by denoting q_0 as initial state and q_f as final state, we have

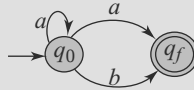


Fig. 4.33 Transition system of Example 4.14

4.12

REGULAR EXPRESSIONS AND REGULAR GRAMMAR

If there is a regular expression r then there exists a regular grammar G that generates all the strings represented by that regular expression. Also, if there is a regular grammar generating language $L(G)$ then there exists a regular expression the represents all the strings in $L(G)$.

From Regular Grammar to Regular Expression

Method I This is a two step approach:

1. Construct finite automaton M that accepts the language generated by regular grammar G .
2. Find regular expression r equivalent to finite automaton M using Arden's theorem.

Method II In this approach we transform the given production of regular grammar into regular expression directly one by one using the following rules:

Let $G = (V_N, \Sigma, P, S)$ be the regular grammar. A regular grammar consists of the productions of the form $S \rightarrow aS$, $S \rightarrow aB$, $B \rightarrow bC$, $C \rightarrow aC \mid b$, etc. Regular expression from the regular grammar can be converted by adopting the following rules.

- (i) If there is a production of the form $C \rightarrow aC \mid b$ then equivalent regular expression is

$$C \equiv (a^*b), \quad \text{where } a, b \in \Sigma$$

- (ii) If there is a production of the form $B \rightarrow bC$ then equivalent regular expression (with the help of rule (i) is

$$B \equiv b(ba^*b), \quad \text{where } a, b \in \Sigma$$

- (iii) If there is production of the form $S \rightarrow aS \mid aB$ then the productions are treated separately, and the equivalent regular expression will be

$$S \equiv a^*aB \quad \text{where } a \in \Sigma$$

or equivalently

$$S \equiv a^*aaba^*b = a^+ba^*b$$

Thus, the regular expression described by the start symbol is the resultant regular expression.

Example 4.15

Consider the following productions of the regular grammar G :

$$S \rightarrow aA \mid a, A \rightarrow aA \mid aB \mid a, B \rightarrow bB \mid c$$

Find the regular expression corresponding to regular grammar G .

Solution The regular expression corresponding production $B \rightarrow bB \mid c$ is:

$$B \equiv (b)^*c \quad [\text{by rule (i)}]$$

For the regular expression corresponding to production $A \rightarrow aA \mid aB \mid a$, substitute $(b)^*c$ on place of B , we have

$$A \rightarrow aA \mid a(b)^*c \mid a$$

This production is of the form $A \rightarrow \alpha A \mid \beta$, where $\alpha = a$ and $\beta = a(b)^*c \mid a$, therefore regular expression corresponding to production $A \rightarrow aA \mid aB \mid a$, is

$$A \equiv (a)^*(a(b)^*c \mid a).$$

For the resultant regular expression of the grammar we now substitute $(a)^*(a(b)^*c \mid a)$ on place of A in the production $S \rightarrow aA \mid a$, we have

$$\begin{aligned} S &\equiv a(a)^*(a(b)^*c \mid a) \mid a \\ &= a(a)^*(a(b)^*c + a) + a \\ &= a^+(ab^*c + a) + a \quad [\text{as } aa^* = a^+] \end{aligned}$$

Hence the regular expression corresponding to regular grammar G is

$$a^+(ab^*c + a) + a$$

From Regular Expression to Regular Grammar

This is a three step approach:

1. Construct finite automaton M with ε -transitions that accepts all the strings represented by regular expression.
2. Find finite automaton M' without ε -transitions.
3. Determine the regular grammar that generates the language accepted by M' .

4.13

LEFT LINEAR AND RIGHT LINEAR REGULAR GRAMMARS

A grammar is linear if and only if each production of a grammar has at most one nonterminal (or say variable) in the right hand side. A regular grammar is any left-linear or a right-linear grammar.

A grammar with productions $A \rightarrow Ba \mid a$, is a left-linear grammar, where $A, B \in V_N$, and $a \in \Sigma$.

A grammar is said to be a right-linear grammar if there is at most one nonterminal as the rightmost symbol in the right hand side of each production of the grammar. A grammar with productions $A \rightarrow aB \mid a$, is a right-linear grammar, where $A, B \in V_N$, and $a \in \Sigma$. Every right-linear grammar generates a regular language. To show this we construct NFA having epsilon transitions that can simulate left-most derivations by using its state to represent a single nonterminal in the current left sentential form.

Example 4.16

Construct the left linear and right linear grammars for the language representing strings by $1(01)^*$.

Solution The language $1(01)^*$ is generated by following right-linear grammar

$$A \rightarrow 1A, A \rightarrow 01A \mid \wedge$$

as well as the following left-linear grammar

$$S \rightarrow S01, S \rightarrow 1$$

4.13.1 Interconversion between the Two

A grammar G_1 with productions $S \rightarrow Sa \mid a$, generates language $L = \{a^n \mid n \geq 1\}$. This grammar is left-linear grammar. If the production $S \rightarrow Sa$ is replaced with $S \rightarrow aS$, then a new grammar G_2 is obtained such that

$$L(G_1) = L(G_2)$$

the grammar G_2 is right-linear grammar. This way, by shuffling the position of the nonterminal in the right hand side of a left-linear grammar, we can obtain the right-linear grammar.

4.14**APPLICATIONS OF REGULAR EXPRESSIONS**

Regular expressions are found useful in the three types of applications:

- (i) validation of input,
- (ii) in searching and selecting parts of a given text on the basis of a given pattern,
- (iii) the lexical analysis.

One of the common problem with the end user (of the computer) is incorrect type of inputs, for example, an identifier which is required to have its character as a symbol followed by a finite number of symbols or letters may be incorrectly given by starting with a dot.

To reduce (or eliminate) this problem, programmers use adhoc methods which are directly programmed into the input routine. A better technique is to use routines such as regex that takes the input string and a regular expression, and then returns true if the input string is matched to the regular expression specifying the input type, for example, the identifiers may be specified by the regular expression as

$$identifier \rightarrow [A-Z a-z][A-Z a-z 0-9]^*$$

and the set of positive integers may be specified by

$$P \rightarrow ([0-9]^+)$$

Apart from the validation of inputs, regular expressions are used in textual search and selection. One of the use of the 'grep' command is for searching for patterns specified by regular expressions. The general form of the 'grep' command is as follows:

$$grep < \text{regular expression in Unix notation} > < \text{file name} >$$

The use of regular expressions in searches can be seen from the way the use of regular expressions simplifies the specification of the problem involved in the complex searches and then solves the problem to find all words in the dictionary word list file, say

/user/dictionary/words

In which all the five vowels (i.e., a, e, i, o, u) occur at least once and one of these occurrences of vowels has the five vowels in the lexicographic order. It can easily be seen that without the use of regular expressions, the task of specification of the following command

```
grep ‘.*a.*e.*i.*o.*u’
```

can specify the problem very easily. This ‘grep’ command here prints words like

```
facetious
sacrilegious
```

Now, we look at application of regular expressions in lexical analysis. Lexical analysis is the task performed by the lexical analyser which is the first phase of the compilation process. The other phases are syntax analysis, intermediate code generation, code optimisation, code generation, table management, and also error handling.

High level language programmes are input to the lexical analyser which treats the programmes as a sequence of characters. These sequences of characters do not carry any meaning, but the lexical analyser converts a sequence of characters of a correctly written programme into a sequence of tokens (the primitive components in a source code). If the incorrect syntax is used to write the code, the tokens are not generated, instead error messages are displayed on the display unit. Let us consider a C/C++ programme fragment:

```
void main() {           /* line 1 */
    int n, i, j;         /* line 2 */
    n = 10;              /* line 3 */
```

In this fragment, the token `void` and `int` are keywords, the token `n`, `i` and `j` are identifiers, the token `=` is an operator, the token `10` is a literal and each of the tokens `,`, `{` and `;` is a separator.

Designing a lexical analyser module, is a complex task but a less suggested tool called LEX generates a lexical analyser automatically by using the great role of regular expressions. The output of LEX is lexical analyser. LEX uses regular expressions to simplify the tedious tasks of developing a lexical analyser for a given pair of high level language and a low level language. The following is a list of some classes and regular expressions associated with them:

Table 4.4 Regular expression based definitions of various classes

S. No.	CLASS	REGULAR EXPRESSION
1	alphabet	$[a-zA-Z]$
2	digit	$[0-9]$
3	Integer	$[0-9]^*[0-9]$ or $[0-9]^+$
4	Identifier	$[a-zA-Z][a-zA-Z0-9]^*$
5	Keyword	<code>int</code> <code>float</code> <code>if</code> <code>else</code> <code>while</code> <code>boolean</code> <code>main</code> <code>void</code>
6	Literal	$[0-9]^+$ <code>True</code> <code>False</code>
7	Boolean	<code>True</code> <code>False</code>
8	White space	<code>\n</code> <code>\t</code> <code>\r</code> (invisible space)
9	Comment	$//[a-zA-Z]^*$ $/*[a-zA-Z]^* */$
10	Operator	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>&&</code> <code> </code> <code>!</code> <code><</code> <code>></code> <code><=</code> <code>>=</code> <code>=</code>
11	Separator	<code>(</code> <code>)</code> <code>{</code> <code>}</code> <code>,</code> <code>;</code>

These regular expression based definitions along with the code for tool is fed into the LEX to generate a lexical analyser. Then any sequence of symbols and characters which consists of a high level language programme is fed to the lexical analyser generated by LEX. If the high level language programme is written as per the syntax of that language, the appropriate action is taken by the lexical analyser according to the classification of each segment of the input sequence.

4.15

NON-REGULAR LANGUAGES

The languages that cannot be accepted or recognised by finite automata are called non regular languages. A non regular language may be a context free language, a context sensitive language, or a recursively enumerable language. Examples of some non-regular languages are:

$$L = \{a^n b^n \mid n \geq 0\}$$

$$L = \{a^{n^2} \mid n \geq 0\}$$

$$L = \{a^n b^n \mid n \geq 0\}$$

$$L = \{a^n \mid n \text{ is a prime number}\}$$

We have already seen in chapter 2 that context free languages are accepted by PDA, LBA and Turing machine as well and context sensitive languages are accepted by LBA and Turing machines.

Summary

- **Regular Language** A regular language is a formal language that can be accepted by a deterministic finite automaton.
- **Regular Expression** Regular Expression is defined as a set of pattern matching rules which are encoded in a string according to certain syntax rules.
- **Operators of Regular Expressions** Three basic operations are performed on alphabets to obtain a regular expression. These operations are: Union, closure, and concatenation.
- **Precedence of Operators** To avoid brackets in regular expression it is assumed that the Kleene star has the highest priority, then concatenation and finally union. If there is no ambiguity then there is no need to include brackets. For example, $(ab)(c(d^*))$ can be written as $abcd^*$.
- **Identity Rules** An identity for an operator can be defined as the value that when applied to some value results in some other value.
- **Kleene's Theorem** If r is a regular expression then there exists a nondeterministic finite automaton M with ε (or say $\wedge$) transitions that accepts or recognises the language $L(r)$.
- **Arden's Theorem** If P and Q are two regular expressions over Σ and $P \in \wedge$ is not correct, then the equation $R = Q + RP$ in R has a unique solution given by $R = QP^*$. Arden's theorem is used to determine the regular expression represented by a transition diagram.
- **Equivalence of two Regular Expressions** Two regular expressions are equivalent if: (i) They represent the same sets (or strings), or (ii) their corresponding FA are same, or (iii) one regular expression is proved equivalent to another by using identities.

- **Myhill- Nerode Theorem** The Myhill-Nerode theorem is algebraic characterisation of a regular language. The Myhill-Nerode theorem is used for minimisation of DFAs. It is one of the decision properties of regular languages.
- **Equivalence Relation** An equivalence relation R on Σ^* is right invariant if for all $u, v, w \in \Sigma^*$, it holds that $uRv \Rightarrow uwRvw$.
- **Binary Relation** Let A be a DFA with starting state q_0 . Define a binary relation R_A on Σ^* by $uR_A v$ if and only if $\delta(q_0, u) = \delta(q_0, v)$. This is an equivalence relation because “=” is one and R_A is defined via “=” of states.
- **Regular Sets** Any set represented by a regular expression is known as a regular set.
- **Closure Properties of Regular Sets** Regular sets are closed under (i) union, (ii) concatenation, (iii) closure (iteration), (iv) transpose, (v) intersection, and (vi) complement.
- **Substitution** Let Σ be an alphabet with $a \in \Sigma$, and we choose a language L_a that depends upon a function called substitution (say *sub*) on Σ , then we can refer L_a as $sub(a)$ for each $a \in \Sigma$.
- **Homeomorphism** A string homeomorphism is defined as a function on set of strings that substitute a particular string for each symbol. For example if a homeomorphism function H is defined as $H(0) = abb$ and $H(1) = ba$, then after homeomorphism of string 1011 gives *baabbbaba*.
- **Equivalence of Regular Expressions and Regular Grammar** If there is a regular expression r then there exists a regular grammar G that generates all the strings represented by that regular expression. Also, if there is a regular grammar generating language $L(G)$ then there exists a regular expression that represents all the strings in $L(G)$.
- **Linear Grammar** A grammar is linear if and only if each production of a grammar has at most one nonterminal (or say variable) in the right hand side. A regular grammar is any left-linear or a right-linear grammar.
- **Left-linear Grammar** A grammar with productions $A \rightarrow Ba \mid a$, is a left-linear grammar, where $A, B \in V_N$, and $a \in \Sigma$.
- **Right-linear Grammar** A grammar with productions $A \rightarrow aB \mid a$, is a right-linear grammar, where $A, B \in V_N$, and $a \in \Sigma$. Every right-linear grammar generates a regular language.
- **Applications of Regular Expressions** Regular expressions are found useful in three types of applications: Validation of input, in searching and selecting parts of a given text on the basis of a given pattern, and lexical analysis.
- **Non-Regular Languages** The languages that can not be accepted or recognised by finite automata are called non-regular languages. A non-regular language may be a context free language, a context sensitive language, or a recursively enumerable language.

Review Questions

4.1 Consider the two regular expressions

$$r_1 = 0^* + 1^*, r_2 = 01^* + 10^* + 1^*0(0^*1^*)^*$$

- Find a string corresponding to r_1 but not to r_2 .
- Find a string corresponding to r_2 but not to r_1 .
- Find a string corresponding to both r_1 and r_2 .
- Find a string in $\{0, 1\}^*$ corresponding to neither r_1 and r_2 .

4.2 Find a regular expression corresponding to each of the following subset of $(0, 1)^*$

- (a) The language of all strings containing exactly two 0's.
- (b) The language of all strings containing at least two 0's.
- (c) The language of all strings that does not end with 01.
- (d) The language of all strings that begins and ends with either 00 or 11.
- (e) The language of all strings beginning with 1 and not having two consecutive 0's.
- (f) The language of all strings in which the number of 0's are even.
- (g) The language of all strings in which both the numbers of 0's and the number of 1's are even.
- (h) The language of all strings in which both the number of 0's and the number of 1's are odd.
- (i) The language of all strings in which every 0 is followed immediately by 11.
- (j) The language of all strings that contains 00 as substring.
- (k) The language of all strings containing both 11 and 010 as sub strings
- (l) The language of all strings containing 101 or 010 as sub string.
- (m) The language of all strings having at most one pair of 0's or at most one pair of 1's.
- (n) The language of all strings in which the number of occurrences of 0's divisible by 3.
- (o) The language of all strings in which there are at least two occurrences of 1's between any two occurrences of 0's.
- (p) The language of all strings with three consecutive 1's
- (q) The language of all strings beginning with 00.
- (r) The language of all strings ending with 00 and beginning with 1.
- (s) The language of all strings in which every pair of adjacent 0's appears before any pair of adjacent 1's.
- (t) The language of all strings not containing 10 as sub string.
- (u) The language of all strings having at least two 0's.
- (v) The language of all strings with exactly two 0's and two 1's.
- (w) The language of all strings having at least one 0 and one 1.
- (x) The language of all strings when 0 comes before 1 or 1 comes before 0.
- (y) The language of all strings in which an even number of 0's are followed by an odd number of 1's.
- (z) The language of all strings of even length strings.

4.3 Construct FA for the regular expressions:

- | | |
|---------------------------|-----------------------------------|
| (i) $(a + b)^*abb$ | (ii) $((a^* + b)^* + b^*)^*$ |
| (iii) $((a + b)^* + b)^*$ | (iv) $(a + (a + a(a + b)^*)^*)^*$ |

4.4 Construct a regular grammar which can generate the set of all strings starting with a letter (A to Z) followed by a string of digits (0 to 9)

4.5 Represent the following sets by regular expressions:

- (a) $\{0, 1, 2\}$
- (b) $\{1^{2n+1} \mid n > 0\}$
- (c) $\{w \in (a, b)^* \mid w \text{ has only one } a\}$
- (d) The set of all strings over $\{0, 1\}$ which has at the most two 0's.
- (e) $\{a^2, a^5, a^8, \dots\}$

- (f) $\{a^n \mid n \text{ is divisible by 2 or 3 or 5}\}$
 (g) $\{a, aa, aaa, aaaa, \dots\}$
 (h) $\{\wedge, a, b, ab, aa, bb, aaa, bbb, \dots\}$
 (i) $\{\wedge, ab, ba, abba, baab, abab, baba, \dots\}$

4.6 Find the set of all strings over $\{a, b\}$ recognised by the transition systems shown below:

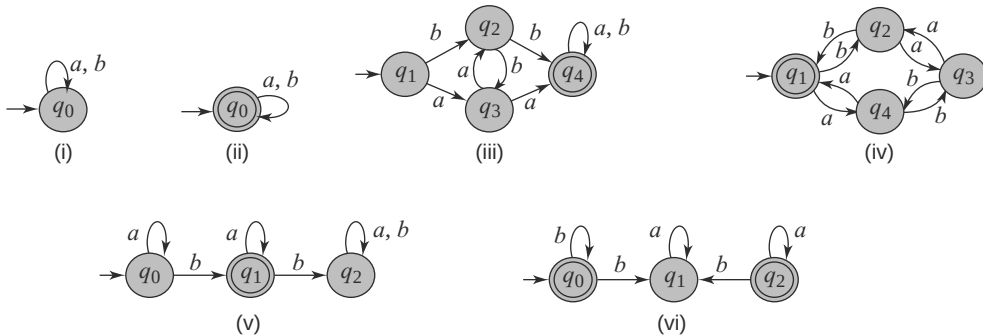


Fig 4.34 (i) – (vii) FA of Exercise 4.6

4.7 Construct a transition system/diagram corresponding to the following regular expressions:

- (a) $(ab + c^*)^*b$ (b) $a + bb + bab^*a$
 (c) $(0 + 1)^*(00 + 11)(0 + 1)^*$ (d) $0^*(1^*)$
 (e) $(ab + ba)^*$ (f) $0^*1(0 + 10^*1)^*$

4.8 Find regular expressions represented by the following transition systems:

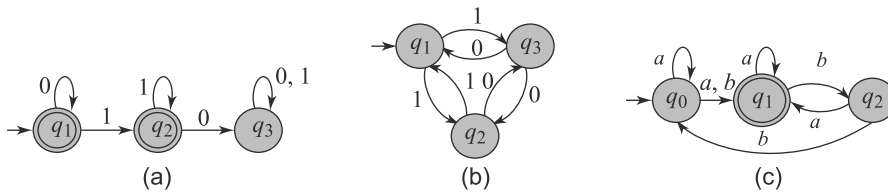


Fig. 4.35 (a)-(c) Transition systems of Exercise 4.8

4.9 The following figure is a transition diagram of an NFA- $\wedge$. For each string below, say whether the NFA- $\wedge$ accepts it:

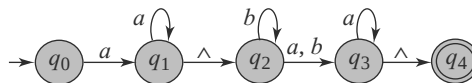


Fig. 4.36 NFA- $\wedge$ of Exercise 4.9

- (a) aba (b) $abab$ (c) $aaabbb$

4.10 Two NFAs are shown in each part of figures below. Decide whether the two accept the same language. Give reasons for your answer.

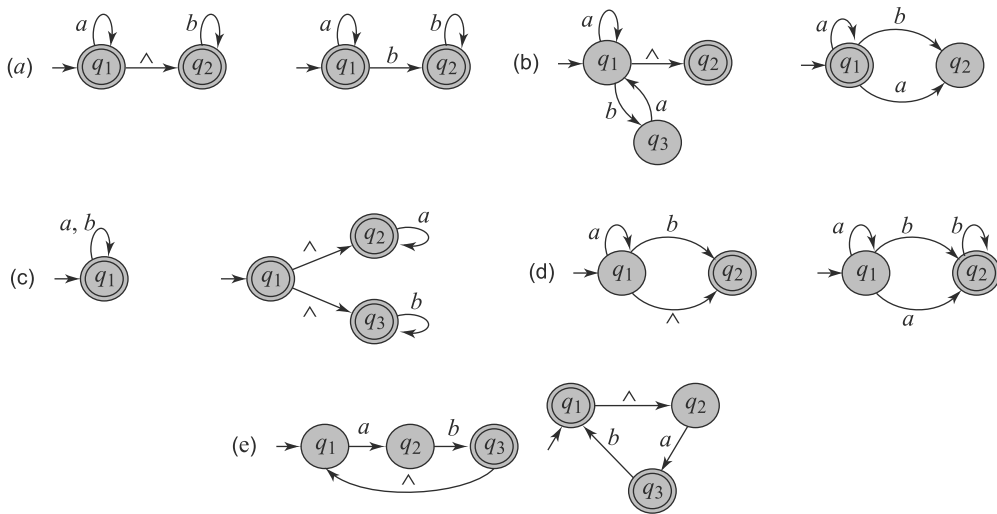


Fig. 4.37 (a)-(e) NFAs of Exercise 4.10

- 4.11 Consider a finite automaton with $\wedge$ -moves given in the figure below. Obtain an equivalent automaton without $\wedge$ -moves.

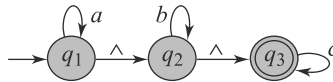


Fig. 4.38 FA with $\wedge$ -moves of Exercise 4.11

- 4.12 The FAs M_1 and M_2 are shown in figures below, recognising languages L_1 and L_2 respectively. Draw NFAs- $\wedge$ recognising each of the following languages:

- (a) $L_1 L_2$ (b) $L_1 L_1 L_2$ (c) $L_1 \cup L_2$ (d) L_1^*
 (e) $L_2^* \cup L_1$ (f) $L_2 L_1^*$ (g) $L_1 L_2 \cup (L_2 L_1)^*$

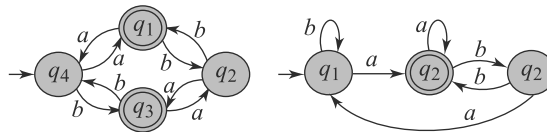


Fig. 4.39 FAs of Exercise 4.12

- 4.13 Prove the following identities for regular expressions r, s and t . Here, $r = s$ mean $L(r) = L(s)$:

- (a) $r + s = s + r$ (b) $(r + s) + t = r + (s + t)$
 (c) $(rs) t = r (st)$ (d) $r (s + t) = rs + rt$
 (e) $(r + s) t = rt + st$ (f) $\phi^* = \wedge$
 (g) $(r^*)^* = r^*$ (h) $(\wedge + r)^* = r^*$
 (i) $(r^* s^*)^* = (r + s)^*$

- 4.14** Construct a regular grammar G generating the set represented by:
- $(a + b)^*$
 - $a^*(ba^*)^*$
 - $ba + (a + bb)a^*b$
 - set of all strings of a 's and b 's ending in bab .
 - $\{\wedge, 11, 1111, 111111, \dots\}$.
- 4.15** Construct regular expression corresponding to the following productions representing regular grammar G .
- $S \rightarrow aS \mid B, B \rightarrow bB \mid a$
 - $S \rightarrow aS \mid bS \mid a \mid b$
 - $S \rightarrow aS \mid aA, A \rightarrow aA \mid a \mid b$
- 4.16** If ab^*aa and $ba(a + b)^*$ are two regular expressions then show that
- $ab^*aa + ba(a + b)^*$ is also a regular expression.
 - $ab^*aaba(a + b)^*$ is also a regular expression.
 - $(ab^*aa + ba(a + b)^*)^*$ is also a regular expression.
 - $(ab^*aa)^*$ and $(ba(a + b)^*)^*$ are also regular expressions.
- 4.17** Construct FA for regular expressions:
- ab^*aa and
 - $ba(a + b)^*$
- 4.18** With the help of FA of previous answer, construct FA for regular expressions:
- $ab^*aa + ba(a + b)^*$
 - $ab^*aaba(a + b)^*$
 - $(ab^*aa + ba(a + b)^*)^*$
 - $(ab^*aa)^*$ and $(ba(a + b)^*)^*$
- 4.19** Show that:
- $(a + b)^* = ((a + b)^*)^*$
 - $(a + b)^* = (a + b + ab + ba)^*$

Hints

4.1 (a) 11 (b) 011 (c) 0 (d) 010

4.2 Here we are giving all required sets (a-z)

- $1^*01^*01^*$
- $(0+1)^*0(0+1)^*0(0+1)^*$
- $(0+1)^*(00+01+10+11)$
- $00(0+1)^*00 + 11(0+1)^*11$
- $(1+10)^*$
- $(1^*01^*01^*)^*$
- $(00+11 + 0101 + 0110 + 1001 + 1010)^*$
- $(0+1)(00+11 + 0101 + 0110 + 1001 + 1010)^*$
- $(1+011)^*$

- $(0+1)^*00(0+1)^*$
- $(0+1)^*((11)(0+1)^*(010) + (010)(0+1)^*(11))(0+1)^*$
- $(0+1)^*(101 + 010)(0+1)^*$
- $1^*001^* + 0^*110^*$
- $(1^*01^*01^*01^*)^*$
- $(1 + 011((1)^*0))^*$
- 0^*1110^*
- $00(0+1)^*$
- $1(0+1)^*00$
- $(1+0011)^*$
- 0^*1^*

- (u) $(0+1)^*0(0+1)^*0(0+1)^*$
 (v) $(0011 + 0101 + 0110 + 1001 + 1010 + 1100)$
 (w) $(0+1)^*(01 + 10)(0+1)^*$
 (x) $(01)^* + (10)^*$
 (y) $(00)^*(11)^*1$ or $(00)^*1(11)^*$
 (z) $(00 + 01 + 10 + 11)^*$

4.4 The required grammar contains the following productions

4.5 (a) $(0 + 1 + 2)$ (b) $1(11)^*$

- (c) b^*ab^* (d) $1^*01^*01^* + 1^*01^* + 1^*$
 (e) $aa(aaa)^*$ (f) $(aa)^* + (aaa)^* + (aaaaa)^*$
 (g) aa^* (h) a^*b^*
 (i) $(ab + ba)^*$

- 4.6** (i) $\emptyset$, (as there is no final state)
 (ii) all strings over $\{a, b\}$
 (iii) the set of all strings over $\{a, b\}$ containing two successive a 's or two successive b 's
 (iv) set of all strings which have an even number of a 's and an even number of b 's
 (v) all strings over $\{a, b\}$ having exactly one b . The state q_2 has no importance in the finite automaton because q_2 is dead state
 (vi) all strings with any number of b 's. The states q_1 and q_2 are useless because q_1 is dead state and q_2 is non reachable state.

4.8 (a) 0^*1^* (b) $\emptyset$ (c) Try yourself

4.9 (a) yes (b) no (c) yes

4.10 (a) yes (b) no (c) no (d) no (e) no

4.12 (a)

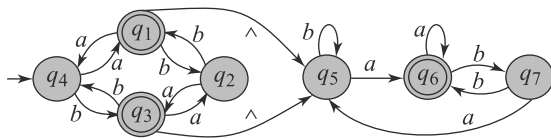


Fig. 4.40 FA of Hint 4.12(a)

- (b) try yourself, (similar approach as in part (a)).
 (c)

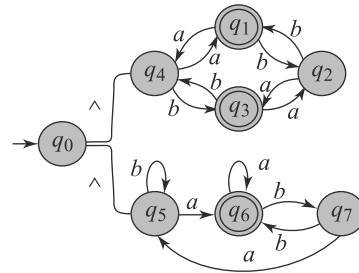


Fig. 4.41 FA of Hint 4.12(c)

(d)

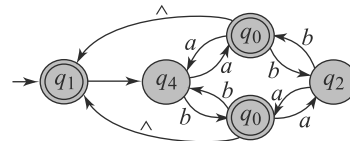


Fig. 4.42 FA of Hint 4.12(d)

4.14 (a) $S \rightarrow aS \mid bS \mid \wedge$

(b) $S \rightarrow aS \mid A \mid \wedge, A \rightarrow BA \mid \wedge, B \rightarrow bC, C \rightarrow aC \mid \wedge$

(c) $S \rightarrow S_1 \mid S_2, S_1 \rightarrow bA, A \rightarrow a, S_2 \rightarrow aB \mid bc, B \rightarrow aB \mid b, C \rightarrow bD, D \rightarrow aD \mid b$

(d) The corresponding regular expression is $(a, b)^*bab$. The regular grammar corresponding to the regular expression is

$S \rightarrow aS \mid bS \mid bA, A \rightarrow aB, B \rightarrow b$

(e) The corresponding regular expression is $(11)^*$. The regular grammar corresponding to the regular expression is

$S \rightarrow \wedge \mid 1A, A \rightarrow 1S$

4.15 (i) From the production $B \rightarrow bB \mid a$, we have $B \equiv b^*a$. if we put b^*a on place of B in $S \rightarrow aS \mid B$, we get $S \rightarrow aS \mid b^*a$. The production $S \rightarrow aS \mid b^*a$ will generate a^*b^*a .

(ii) $(a + b)^+$, (iii) $a^*aa^*(a + b)$

Graded Exercises

****4.1** Prove or disprove that

(ii) $(00^*1)^*1 = 1 + 0(0+10)^*11$

(iii) $(111^*)^* = (11 + 111)^*$

(iv) $(a^*ab + ba)^*a^* = (a + ab + ba)^*$

(v) $(a + b^*)^* = (a + b)^*$

***4.2** Prove or disprove whether following languages are regular or not.

- (i) The language L over the alphabet $\{0, 1\}$ of the form 0^n where n is not prime.
(ii) The set of all binary strings that read backward and the same as forward.
- *4.3** Find all strings of length 5 or less in the regular sets represented by the following:
(i) $(ab + a)^*(aa + b)$ (ii) $(a^*b + b^*a)^*a$ (iii) $a^* + (ab + a)^*$
(iv) $(ab + ba)^*$ (v) $(aa)^* + b(bb)^*$
- *4.4** Let $M = (Q, \Sigma, \delta, q_0, F)$, be an NFA. Show that for any $q \in Q$ and any $a \in \Sigma$
 $\delta^*(q, a) = \delta(q, a)$
- **4.5** Show that the transition system M with $\wedge$ -moves is equivalent to the transition system M' without $\wedge$ -moves.
- ***4.6** Give an example of a regular language L containing $\wedge$ that cannot be accepted by any nondeterministic finite automaton having only one final state.
- **4.7** Prove or disprove the following for regular expression r and s :
(a) $(rs + r)^*r = r(sr + r)^*$ (b) $s(rs + s)^*r = rr^*$ is $(rr^*s)^*$ (c) $(r + s)^* = r^* + s^*$
- *4.8** Are the following true or false? Support your answer by giving proofs.
(a) If $L_1 \cup L_2$ is regular L_1 is regular, that L_2 is also regular
(b) If L_1L_2 is regular and L_1 is regular then L_2 is also regular
(c) If L^* is regular then L is regular.
- **4.9** Show that the following set are not regular without using pumping lemma:
(a) $L = \{a^n b^n \mid n > 0\}$ (b) $L = \{a^n b^{2n} \mid n > 0\}$ (c) $L = \{a^n b^m \mid m > n > 0\}$
(d) $L = \{0^m 1^n \mid g.c.d. (m, n) = 1\}$ (e) $L = \{a^n b^m b^m c^n \mid m, n \geq 0\}$ (f) $L = \{a^n b^n c^n \mid n \geq 0\}$
- ***4.10** Determine regular and context free grammar corresponding to regular expression:
 $(0 + 11^*) + (01 + 101)^*$

Objective Questions

- Which of the following identity for regular expression is not true if P and Q are two regular expressions?
(a) $\wedge^* = \phi^*$ (b) $(P + Q)^* = (P^* + Q^*)^*$
(c) $P^*Q^* = (P^* + Q^*)^*$ (d) $(P + Q)^* = (P^*Q^*)^*$
- The set of all strings over $\{0, 1\}$ starting with 00 and ending in 11 is
(a) 0011 (b) 00(0+1)11 (c) (00)*(11)* (d) 0*1*
- The set of all string over $\{0, 1\}$ in which occurrence of b 's is followed by a or occurrence of a 's is followed by b is
(a) a^*b^* (b) $a^*b + b^*a$ (c) a^*b (d) b^*a
- If $r_1 = (a + b)^*$ and $r_2 = a^*(ba^*)^*$, then following statement is false?
(a) r_2 is the set of all strings over $\{a, b\}$ (b) the set r_1 is equivalent to r_2
(c) both (a) and (b) (d) none of these
- The set of all strings over $\{a, b\}$ which has at most three a 's then
(a) no such set is possible (b) infinite number of such strings are possible
(c) $b^* + b^*ab^* + b^*ab^*ab^* + b^*ab^*ab^*ab^*$ (d) both (b) and (c) are true.

6. The set of all strings over $\{0, 1\}$ which has exactly two 1's is
 (a) $0^*10^*10^*$ (b) 110^* (c) 10^*1 (d) all of these
7. The set of all strings over $\{0, 1\}$ having all strings of even length is
 (a) $(00 + 11)^*$ (b) $(00 + 01 + 10 + 11)^*$ (c) $(01 + 10)^*$ (d) both (a) and (c)
8. The set of all strings $\{0, 1\}$ in which strings end with 1 and do not contain the substring 00, is
 (a) $(1 + 01)^*1$ (b) $(1 + 01)^*$ (c) $(1 + 01)^*(1 + 01)$ (d) all of these
9. The set of all strings over $\{0, 1\}$ in which every 0 is followed immediately by 11 is
 (a) $(011)^*$ (b) $(011 + 1)^*$ (c) $011(011)^*$ (d) all of these
10. The set of all strings over $\{0, 1\}$ of odd lengths is
 (a) $0(00)^* + 1(11)^*$ (b) $(0 + 1)(00 + 01 + 10 + 11)^*$
 (c) $0(00 + 01 + 01 + 10 + 11)^*1$ (d) all of these
11. Regular expression $(x|y)(x|y)$ denotes the set
 (a) $\{xy, xy\}$ (b) $\{xx, xy, xy, yy\}$ (c) $\{x, y\}$ (d) $\{x, y, xy\}$
12. Regular expression $(x|y)$ denotes the set
 (a) $\{x, y\}$ (b) $\{xy\}$ (c) $\{x\}$ (d) $\{y\}$
13. Which of the following regular expressions denotes a language comprising all possible strings of even length over the alphabet $\{0, 1\}$?
 (a) $1 | 0(1 | 0)^*$ (b) $(0 | 1)(1 | 0)^*$ (c) $(1 | 0)$ (d) $(00 | 01 | 11 | 10)^*$
14. The regular expression which denotes zero or more instances of an x or y , is
 (a) $(x|y)$ (b) $(x|y)^*$ (c) $(x^*|y)$ (d) $(xy)^*$
15. $(ab)^*a$ is a regular expression defining a language over $\{a, b\}$. Another regular expression that defines the same language over $\{a, b\}$ is
 (a) a^*ba (b) $a^*(ba)^*$ (c) $a(ba)^*$ (d) none of these
16. The regular expression which has all strings over $\{0, 1, 2\}$ in which any number of 0's is followed by any number of 1's followed by any number of 2's is
 (a) $(0 | 1 | 2)^*$ (b) $0^*1^*2^*$ (c) $0^* + 1 + 2$ (d) $(0 + 1)^*2^*$
17. The regular expression which has all strings of 0's and 1's with no two consecutive 0's is
 (a) $(0 | 1)$ (b) $(0 | 1)^*$ (c) $(0 | \wedge)(1 + 10)^*$ (d) $(0 + 1)^*011$
18. The regular expression with all strings 0's and 1's with at least two consecutive 0's is
 (a) $(1 + 10)^*$ (b) $(0 + 1)^*00(0 + 1)^*$ (c) $(0 + 1)^*011$ (d) $0^*1^*2^*$
19. Which of the following is not the set of regular expression $R = (ab + abb)^*bbab$
 (a) $ababbbab$ (b) $abbbb$ (c) $ababbabbbab$ (d) $abababab$
20. Which of the following statements is wrong ?
 (a) the regular sets are closed intersection.
 (b) the class of regular sets is closed under substitution.
 (c) the class of regular sets is closed under homeomorphism.
 (d) context sensitive grammar (CSG) can be recognised by finite state machine (FSM).
21. Let $r = 1(1 + 0)^*$, $s = 11^*0$ and $t = 1^*0$ be three regular expressions. Which of the following is true?
 (a) $L(s) \leq L(r)$ and $L(s) \leq L(t)$ (b) $L(r) \leq L(s)$ and $L(s) \leq L(t)$
 (c) $L(s) \leq L(t)$ and $L(s) \leq L(r)$ (d) $L(t) \leq L(s)$ and $L(s) \leq L(r)$

22. Which of the following regular expression identities are true?
 (a) $(r^*)^* = r^*$ (b) $(r^* s^*)^* = r^* s^*$ (c) $(r + s)^* = r^* + s^*$ (d) both (a) and (b)
23. Let $\Sigma = \{a, b, c, d, e, f\}$. The number of strings in Σ^* of length 4 such that no symbol is used more than once in a string is
 (a) 28 (b) 35 (c) 49 (d) 360
24. Consider the regular expression $R = (ab|abb)^*bbab$, then which of the following is not in set R ?
 (a) $ababab$ (b) $abbbab$ (c) $abbabbbab$ (d) $ababbaabbab$
25. In the Fig. 4.43 the deterministic finite automaton M has final state q_D . Which of the following regular expressions denote the set of all words accepted by M ?

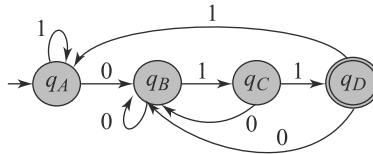


Fig. 4.43 DFA of Question 25

- (a) 001 (b) 10^*1^*0 (c) 1^*0^*011 (d) none of these.
26. The language recognised by the following FA is

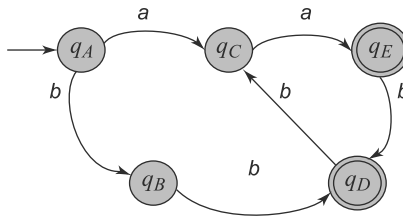


Fig. 4.44 FA of Question 26

- (a) $aabb^* + bab^*$ (b) $aab(\wedge + (bab)^*)^*$
 (c) $a(abb)^*(a + ab) + bb(bab)^* + bbba$ (d) none of these.
27. The language accepted by following FA is

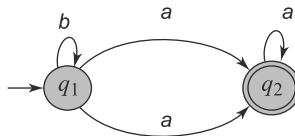


Fig. 4.45 FA of Question 27

- (a) $(a + b)^*(a + b)$ (b) b^*aa^* (c) $(a + b)^*b$ (d) a^*b .
28. Consider the following regular expressions
 $r_1 = (a + b)^*a$, $r_2 = (a + b)^*ab(a + b)^*$, $r_3 = (a + b)^*(aa + bb)(a + b)^*$
 According to above regular expressions which of the following statements is true ?
 (a) The machine that accepts r_1r_2 is the same as the machine which accepts r_2r_1 .
 (b) The machine which accepts r_2r_3 is the same as the machine which accepts r_3r_2 .
 (c) The machine that accepts r_2 is the same as the machine that accepts $b^*aa^*b(a + b)^*$.
 (d) The machine which accepts r_1r_1 is the same as the machine which accepts r_1^* .

References for Extra Reading

1. Brzozowski, J. A., *A survey of regular expressions and their applications*, IEEE Transactions on Electronic Computers.
2. John E. Hopcroft and Jeffrey D. Ullman, *Introduction to Automata Theory, Languages and Computation*, Addison-Wesley Publishing, 1997.
3. McNaughton, R. and H. Yamada, *Regular expressions and state graphs for automata*. IEEE Transactions on Electronic Computers 9:1 (1960), 39-47.
4. Van Noord, Gertjan., *The treatment of epsilon moves in subset construction*, In Finite-state Methods in Natural Language Processing, Ankara, 1998.

Online Sources

- 1 www.cs.uky.edu/~lewis/texts/theory/automata/dec-prob.pdf
- 2 www.let.rug.nl/~van Noord/papers/cl00/fsmnlp.pdf
- 3 www.peterasaro.org/writing/McCulloch.html
- 4 http://www.en.wikipedia.org/wiki/Automata_theory
- 5 <http://www.cs.uky.edu/~lewis/texts/theory/automata/autointr.pdf>
- 6 <http://www.abo.fi/~jboling/cdes/automata.pdf>
- 7 <http://www.cs.princeton.edu/courses/archive/spr01/cs126/lectures/T2-4up.pdf>

5

Properties of Regular Languages

❖ Chapter Objective ❖

This chapter highlights the closure and decision properties of regular languages. As closure properties, we will discuss union, concatenation, Kleene closure and positive closure, complement, etc operations on regular languages. We will illustrate how deterministic finite automata are used to find the complement of regular languages. We will describe pumping lemma and we will see how it can be applied to non-regular languages to prove that these languages are not regular.

□ Introduction □

The closure properties of regular languages allow us to design recognisers for languages which are obtained from the regular languages as the result of other regular languages. If we can design finite automaton for any regular language L then we can also design finite automata for LL , $L+L$, L^* , and L' . In the same way if we can design finite automata for any two regular languages L_1 and L_2 then we can also design finite automata for L_1L_2 , L_2L_1 , $L_1 + L_2$, $(L_1 + L_2)^*$, $(L_1L_2)^*$, $(L_2L_1)^*$, etc. This way the closure properties of regular languages help us to design complex finite automata.

5.1

CLOSURE PROPERTIES OF REGULAR LANGUAGES

Regular languages are closed under union, concatenation, closure, complementation, intersection and difference.

Union The union of two regular languages is also regular.

Concatenation The concatenation of two regular languages is also regular.

Closure The closure (both Kleene and Positive) of a regular language is also regular. The closure operation is also applicable to a set, *i.e.* the union of two or more regular languages.

Complementation The complement of a regular language is also regular.

Intersection The intersection of two regular languages is also regular.

Difference The difference of two regular languages is also regular.

Theorem 5.1 If L_1 and L_2 are two regular languages, $L_1 + L_2$, $L_1 L_2$, L_1^* and L_2^* are also regular languages.

Proof Let r_1 and r_2 be two regular expressions defining the regular languages L_1 and L_2 , respectively. The regular expression $(r_1 + r_2)$ defines the languages $L_1 + L_2$. The language $L_1 L_2$ can be defined by concatenation of r_1 and r_2 as resultant regular expression $r_1 r_2$. The language L_1^* and L_2^* can be defined by $(r_1)^*$ and $(r_2)^*$ respectively, that are formed by applying Kleene closure operations on regular expressions r_1 and r_2 .

For the above, the proof uses the fact that languages L_1 and L_2 must be defined by regular expressions if they are regular languages. Regular languages can also be defined in terms of machines (FA) to prove this theorem.

Complement of Regular Languages

Let L be a language over the alphabet Σ , then we can define L' as complement of L , which is the language of all strings of letters from Σ that are words not in language L . For example of L is the language over $\Sigma = \{0, 1\}$ of all words that have a triple 1 in them, then L' is the language of all words that do not have a triple 1.

To determine the complement of a regular language L over some alphabet Σ , we need to

1. Find the regular expression for regular language L .
2. Design NFA having $\wedge$ -moves from the regular expression.
3. Convert NFA obtained from step 2, into equivalent DFA M .
4. Find complement of DFA M by converting all final states in to non-final, and non-final states into final states to get the complemented DFA M' .
5. Find the language L' (It will be complement of L) of constructed DFA M' .

The important point regarding the construction of the DFA M is that it must show all transitions for every input symbol from Σ . If there are some missing transitions then there will be certain strings that will lead neither to an accepting nor a non-accepting configuration of DFA M .

Theorem 5.2 The complement of a regular language is also a regular language.

Proof Let L be regular language. A regular language L can be accepted by finite automaton. Some of the states of this finite automaton are final states and remaining are initial and intermediate (neither initial nor final) states. An initial state may also be a final state as well. Let us reverse the status of each state, i.e., if it was a final state make it a non-final state; and if it was a non-final state, make it a final state. Now if an input string is accepted by a non-final state, it now accepts in a final state. Now, if an input string is accepted by a non-final state, it now accepts in a final state and vice versa. So this machine accepts all input strings that were not accepted by the original finite automaton. Therefore this converted machine accepts exactly the language L' .

❖ Good to Know!

L' is complement of L only if we specify so, otherwise several times we have used notations L and L' to denote two different languages. Similarly G' is not complement of G until we specify so.

Complement of a Finite Automaton

To determine the complement of a deterministic finite automaton, we replace all final states with non-final states and vice versa. In this way, the resultant automaton accepts exactly the set of strings (*i.e.*, language) that the original automaton does not. For example, if we have to given a regular language

$$L = \{\omega \mid \omega \text{ is a binary integer and } \omega \% 3 \neq 0\},$$

i.e., bit sequences (strings of 0's and 1's) are not a multiple of 3 when interpreted as a binary integer. If we wish to design a complemented deterministic finite automaton for the language L , the complemented deterministic finite automaton will accept the language

$$L' = \{\omega \mid \omega \text{ is a binary integer and } \omega \% 3 = 0\},$$

i.e., bit sequences are a multiple of 3 when interpreted as a binary integer. For language L , the deterministic finite automaton is given by Fig. 5.1.

The deterministic finite automaton for language is $L' = \{\omega \mid \omega \text{ is a binary integer and } \omega \% 3 = 0\}$. The DFA for L' is given by Fig. 5.2.

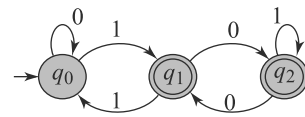


Fig. 5.1 A finite automaton accepting language L

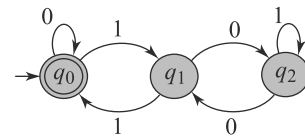


Fig. 5.2 A finite automaton accepting language L'

Intersection and Other Closures

A class of languages is simply a set of languages having some properties that determine whether a language is in that class or not. The class of languages are called regular languages if the set of all languages are regular.

Any language is a set of strings over a finite set of alphabets and the number of strings is countable. Also, a regular language may have an infinite number of strings. The strings in a regular language may be of any length, *i.e.*, of length 0, length 1, length 2, and so on. But a class of regular languages is considered to be the size of a power set of any given regular language. This is true if regular languages are closed under union that is true we know.

Intersection is constructed using De-Morgan's theorem and the difference is constructed as

$$L - M = L \text{ intersect complement } M.$$

The reverse is constructed from deterministic finite automata using final states as starting states and the starting states as the final states, reversing the direction of all transition arrows.

The construction of a DFA, M_3 , such that $L(M_3) = L(M_1) \cap L(M_2)$ is given by

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$

Let $S_1 \times S_2$ be the cross product of sets S_1 and S_2 , all ordered pairs, then

$$M_3 = (Q_1 \times Q_2, \Sigma, \delta_3, \{q_1, q_2\}, F_1 \times F_2)$$

where $\{q_1, q_2\}$ is an ordered pair from $Q_1 \times Q_2$.

δ_3 is constructed from

$$\delta_3(\{q, q'\}, a) = \{\delta_1(q, a) \cup \delta_2(q', a)\}$$

For all a in δ , and all $(\{q, q'\}$ in $Q_1 \times Q_1$.

We choose to say that the same alphabet is used in both machines, but this work is done by picking the alphabet of M_3 to be the intersection of the alphabets of M_1 and M_2 and using all a 's from this set.

Theorem 5.3 If L_A and L_B are two regular languages, then $L_A \cap L_B$ is also a regular language.

Proof According to De Morgan's law for any type of sets, we know,

$$L_A \cap L_B = (L'_A \cup L'_B)' = (L'_A + L'_B)'$$

where, L'_A and L'_B are complements of L_A and L_B respectively.

It states that the language $L_A \cap L_B$ contains all words that are neither in L'_A nor L'_B . As we know L_A and L_B are regular languages, so L'_A and L'_B are also regular languages. Therefore, $L'_A + L'_B$ will also be regular language, as we know that the union of two regular languages is also regular. And when $L'_A + L'_B$ is regular language, then $(L'_A + L'_B)'$ will also be regular language, which is equivalent to $L_A \cap L_B$.

❖ Definition

Shuffle: The shuffle operation on any two strings s_1 and s_2 is defined as the set of all strings that can be obtained by interleaving the positions of strings s_1 and s_2 in any way.

Union intersection and set-difference operations can be applied to regular language. The result of union, intersection and set-difference operations when applied to regular languages is itself a regular language. The complement of every regular language is a regular language as well. When every string in a regular language is reversed, another regular language is obtained. The result of concatenation of two regular languages also yields a regular language. When the shuffle operation is applied to two different regular languages, another regular language is formed. The right quotient and the left quotient of a regular language by an arbitrary language is also regular language. Also if L_1 and L_2 are two regular languages, the $L_1 \cap L_2$ is a regular language.

According to the Chomsky hierarchy (discussed in Chapter 3), every regular language is context free. But every context free language is not a regular language. For example, the language consisting of all strings which has the equal number of 0's and 1's is context free but not regular. To prove whether a language is not regular, pumping lemma can be applied, discussed later in this chapter.

Theorem 5.4 A language L over alphabet Σ is regular if and only if there is a finite automaton M that accepts language L .

This theorem says, that if M is any finite automaton, then there is a regular expression corresponding to the language $L(M)$ (the language accepted by finite automaton M); and, there is a regular expression r , for which there is a finite automaton that accepts the corresponding language.

5.2

DECISION PROPERTIES OF REGULAR LANGUAGES

The decision algorithm for regular sets requires the following points to be remembered:

- (a) An algorithm must always terminate to be called an algorithm. Basically, an algorithm needs to have the following four characteristics :
 - (i) An algorithm must be written using a finite number of unambiguous (one and only one meaning) steps.
 - (ii) For every possible input, only a finite number of steps are to be performed, and the algorithm is supposed to produce a result.
 - (iii) Every time, the same and correct result is to be produced for the same input.
 - (iv) Each step of the algorithm must have the properties as explained in (i), (ii) and (iii).
- (b) A regular language is just a set of strings over a finite alphabet. Every regular set can be represented by a regular expression and accepted by a minimum state DFA.

We choose DFAs represented by usual notations (as we have seen in Chapter 2) so that we can analyse every DFA and even simulate them.

The type of questions we are concerned in this section are such as :

- (i) Is the given language empty, finite or infinite ?
- (ii) Is one regular set equivalent to another ?

and so on. Now we have to establish an algorithm to answer such questions. For our purpose we use that regular sets are represented by finite automata.

At this point, we are able to determine whether a regular set is empty. Algorithms to determine whether a regular set is empty, finite or infinite may be based on the following theorem:

Theorem 5.5 The set of strings accepted by a finite automaton M with n states is:

- (i) nonempty if and only if the finite automaton accepts a sentence of length less than n .
- (ii) infinite if and only if the automaton accepts some strings of length where $n \leq l < 2n$.

This shows that there is an algorithm to determine whether a finite automaton accepts zero, a finite number or an infinite number of strings.

Proof (i) Suppose finite automaton M accepts a nonempty set. Let 's' be a string as short as any other strings accepted. By using pumping lemma we can write $|s| < n$ if the string s is shortest and $|s| \leq n$ otherwise. Then we can write $s = uvw$, and uw is a shorter string in the language. This shows whether $L(M)$ (language accepted by finite automaton M) is empty or not. Hence there is such a procedure guaranteed to halt.

(ii) If the string $s \in L(M)$ and $n \leq |s| < 2n$, then by using pumping lemma, we can say that $L(M)$ is infinite. Let us suppose $s = s_1s_2s_3$, and for all i , $s_1s_2^is_3 \in L(M)$. Conversely if $L(M)$, the language accepted by finite automaton M is infinite, then there exists $s \in L(M)$, where $|s| \geq n$.

If there is no word whose length lies between n and $2n - 1$, suppose the length of string s is at least $2n$, but as short as any string is $L(M)$ whose length is greater than or equal to $2n$. Again by using pumping lemma we can write $s = s_1s_2s_3$ with $n \leq |s_2| < 2n$ and $s_1s_3 \in L(M)$. Then either s was not a shortest string of length $2n$ or more, or $n \leq |s_1s_3| < 2n - 1$, a contradiction in either case.

This decides whether $L(M)$ is infinite. For example, if any string of length between n and $2n - 1$ is in $L(M)$. Hence there is such a procedure that is guaranteed to halt.

5.3

PUMPING LEMMA FOR REGULAR LANGUAGES

The term pumping lemma is made up of two words pumping and lemma. The word pumping refers to generate many input strings by pushing symbols in a string again and again. The word lemma refers to a subsidiary or intermediate theorem in a proof.

First of all we will see what is pumping lemma. We will give a proof of this theorem, then we will study pumping lemma for regular languages. It will prove that certain languages are not regular.

Lemma 5.1 (Pumping Lemma) If $M = (Q, \Sigma, \delta, q_0, F)$, is a finite automaton with n states. Suppose L is a regular set accepted by finite automaton M . Let $z \in L$ and $|z| = m$. If $m \geq n$, then there exists u, v, w such that $z = uvw$ with $|v| \neq 0$ and $uv^i w \in L$ for each positive integer values of i (i.e., $i \geq 0$).

Proof The pumping lemma considers that a string $z \in L$ has length greater equal to m and the number of states in FA representing z are n such that $m \geq n$. It is possible only when there exists some indirect or direct loop (say closure, i.e., present state and next states are same on input $a \in \Sigma$) in at least one state of FA. In practice we see that if there is no closure on any state of FA, the length of string is one less than total number of states.

Let the string z be composed as

$$z = a_1 a_2 a_3 \dots a_m, \text{ with } m \geq n$$

$$\delta(q_0, a_1 a_2 a_3 \dots a_m) = q_i \text{ for } i = 1, 2, 3, \dots, m$$

and

$$Q = \{q_0, q_1, q_2, \dots, q_{n-1}\}$$

where Q_1 is the sequence of states in the path with path value $z = a_1 a_2 a_3 \dots a_m$. As there are only n distinct states in FA, therefore at least two states in Q_1 must be considered. Let us take the first pair of states q_j and q_k among the various pairs of repeated states.

Assume $q_j = q_k$, then j and k satisfy the condition $0 \leq j < k \leq n$

The string z can be decomposed into three substrings u, v and w where

$$u = a_1 a_2 a_3 \dots a_j$$

$$v = a_{j+1} a_{j+2} a_{j+3} \dots a_k$$

$$w = a_{k+1} a_{k+2} a_{k+3} \dots a_m$$

As we know $k \leq n$, $|uv| \leq n$ and $z = uvw$. The path with path value z is by the transition diagram given by Fig. 5.3.

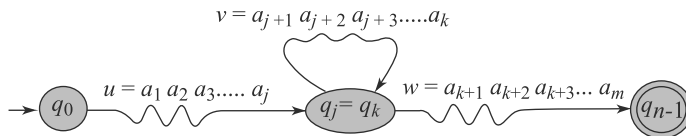


Fig. 5.3 FA recognising the string $a_1 a_2 a_3 \dots a_j a_{j+1} a_{j+2} a_{j+3} \dots a_k a_{k+1} a_{k+2} a_{k+3} \dots a_m$

The finite automaton M in Fig. 5.3 is initially in state q_0 (initial or start state). When the string u is applied it reaches to state q_j (or say q_k). On applying the string v it remains in state q_j (because $q_j = q_k$). Therefore after application of input v in state q_j again and again (in this case input v is applied i times

where $i \geq 0$) the automaton remains in state q_j . When we apply input string w on state q_j the automaton reaches to state q_{n-1} which is final state. Hence, $uv^i w \in L$. As every state in Q_1 is obtained by applying an input symbol, $v \neq \Lambda$.

5.4 PROVING LANGUAGES NOT TO BE REGULAR LANGUAGES

Pumping Lemma is generally used to prove that certain languages are not regular languages. If a DFA, NDFA or NDFA- Λ (NDFA having no Λ -moves) can be constructed to exactly accept a language, then the language is a regular language.

In another way, if a regular expression can be constructed to exactly generate the strings in a language, then the language is a regular language.

If a regular grammar can be constructed to exactly generate the same strings in a language, then the language is a regular language. To prove that a language is not regular requires a specific definition of the language i.e., the use of pumping lemma for regular language.

A Note about Using Pumping Lemma Suppose two formal statements A and B are given such that A implies B .

If we can prove that ' B ' is false, then we could have proved ' A ' is false. For the pumping lemma, the statement ' A ' is " L is regular language", and the statement ' B ' is a statement of pumping lemma.

Formal Statement of Pumping Lemma L is a regular language implies

(there exists n) (for all z) {
 $z \in L$ and $|z| \geq n$ implies {
 (there exists u, v, w) ($z = uvw$, $|uv| \leq n$ and $|v| \geq 1$ and (for all $i \geq 0$) ($uv^i w \in L$)
 }
 }

The two common ways to use the pumping lemma to prove that a language is NOT regular are :

- (i) Show that there is no possible ' n ' for the
 (there exists n)

this is usually accomplished by showing a contradiction such as

$$(n+1)(n+1) < n^2 + n.$$

- (ii) Show that there is no way to partition z into u, v and w such that $uv^i w$, typically for the value
 $i = 0$ or $i = 2$.

Pumping Lemma Steps

The following are the steps of pumping lemma required for proving that a given set is not regular.

Step 1 Let us assume that the given language L is regular. Let n be the number of states in the corresponding finite automaton FA.

Step 2 Choose a string z such that $|z| \geq n$. Let us use pumping Lemma to write $z = uvw$, with condition $|uv| \leq n$ and $|v| \geq 1$ or $|v| \neq 0$.

Step 3 Find a suitable integer i such that $uv^i w \notin L$. This contradicts our assumption. Therefore the language L is not regular. (The most important part of the procedure is to find the value of i such that $uv^i w \notin L$).

Example 5.1

Show that the language $L = \{0^n 1^n \mid n \geq 1\}$ is not regular.

Solution

Step 1 Let us suppose L is regular language and if we get a contradiction then L is not regular. Suppose finite automaton has n states that accepts language L

Step 2 Let $z = 0^n 1^n$, then $|z| = 2n > n$. By pumping lemma we write $z = uvw$ with $|uv| \leq n$ and $|v| \geq 1$

Step 3 Now we have to find i so that $uv^i w \notin L$ to get a contradiction. The string v can be any one of the three possible forms:

- (i) the string v is constructed by using only 0's, it means $v = 0^k$ for some $k \geq 1$
- (ii) the string v is constructed by using only 1's it means $v = 1^l$ for some $l \geq 1$
- (iii) the string v is constructed by using both symbols 0's and 1's. Then the string v will be of the form $v = 0^m 1^p$ for some $m \geq 1$ and $p \geq 1$

Case (i) By pumping lemma we write

$$z = 0^n 1^n = \underbrace{000000 \dots 0000}_{0^{n-k}} \underbrace{00000}_{0^k} \underbrace{1111111 \dots 11111}_{1^n} = 0^{n-k} 0^k 1^n$$

we now consider $u = 0^{n-k}$, $v = 0^k$ and $w = 1^n$

By pumping lemma we write

$$z = 0^{n-k} (0^k)^i 1^n$$

for $i = 0$ we get

$$z = 0^{n-k} 1^n$$

which is a contradiction because $0^{n-k} 1^n \notin L$. In $0^{n-k} 1^n$, the number of 0's are less than number of 1's as $k \geq 1$.

Case (ii) By pumping lemma we write

$$z = 0^n 1^n = \underbrace{000000 \dots 0000}_{0^n} \underbrace{111111}_{1^l} \underbrace{111111 \dots 11111}_{1^{n-l}} = 0^n 0^l 1^{n-1}$$

Here we considered $u = 0^n$, $v = 1^l$ and $w = 1^{n-l}$

By pumping lemma we write

$$z = 0^n (1^l)^i 1^{n-1}$$

for $i = 0$ we get

$$z = 0^n 1^{n-l}$$

which is a contradiction because $0^n 1^{n-l} \notin L$. In $0^n 1^{n-l}$ the number of 0's are more than the number of 1's as $l \geq 1$.

Case (iii) By pumping lemma we write

$$z = 0^n 1^n$$

$$= \underbrace{000000 \dots 0000 \dots 0}_{0^{n-m}} \underbrace{001111 \dots 111111}_{0^m 1^p} \underbrace{\dots 111111}_{1^{n-p}} = 0^{n-m} 0^m 1^p 1^{n-p}$$

Here we considered $u = 0^{n-m}$, $v = 0^m 1^p$ and $w = 1^{n-p}$

By pumping lemma we write

$$z = 0^{n-m} (0^m 1^p)^i 1^{n-p}$$

For $i = 0$ we do not get a contradiction in this case, because for $i = 0$, $z = 0^{n-m} 1^{n-p}$, here m may be equal to p therefore no contradiction.

For $i = 2$ we get

$$z = 0^{n-m} (0^m 1^p)^2 1^{n-p}$$

$$= 0^{n-m} (0^m 1^p) (0^m 1^p) 1^{n-p}$$

$$= 0^n 1^p 0^m 1^n$$

which is a contradiction because $0^n 1^p 0^m 1^n \notin L$. In $0^n 1^p 0^m 1^n$, m occurrences of 0's occur after 1's which is not supported by $0^n 1^n$.

All three above cases prove that L is not regular.

Example 5.2

Prove that language $L = \{a^i b^j \mid i \neq j\}$ is not regular.

Solution We will prove that language $L = \{a^i b^j \mid i \neq j\}$ is not regular by using pumping lemma. We apply the following steps:

Step 1 Assume given language $L = \{a^i b^j \mid i \neq j\}$ is regular.

Step 2 By pumping lemma we write

$$z = a^i b^j = uvw, \text{ such that } |v| \neq 0 \text{ or } |v| \geq 1.$$

Step 3 There are two cases:

- (i) $i > j$, in this case $v = a^{i-j}$.
- (ii) $j \geq i$, in this case $v = b^{j-i}$.

Case (i) $z = uv^k w$

$$z = a^i b^j = a^i (a^{i-j})^k b^j$$

for $k = 0$, we have $z = a^i b^j$, which is a contradiction as we can see that number a 's are equal to number of b 's, therefore the language $L = \{a^i b^j \mid i \neq j\}$ is not regular.

Case (ii) $z = uv^k w$

$$z = a^i b^j = a^i (b^{j-i})^k b^i$$

for $k = 0$, we have $z = a^i b^i$, which is a contradiction as we can see that number a 's are equal to number of b 's, therefore the language $L = \{a^i b^j \mid i \neq j\}$ is not regular.

Example 5.3

Show that the language $L = \{0^p \mid p \text{ is a prime number}\}$ is not regular.

Solution

Step 1 Let us suppose L is regular language and if get a contradiction then L is not regular. Let n be number of states in the finite automaton accepting language L .

Step 2 Let p be the prime number greater than n . By pumping lemma, we have $z = 0^p = uvw$ and $|z| = |uvw| = |0^p| = p$.

By using pumping lemma, we can write $z = uvw$, with $|uv| \leq n$ and $|v| \geq 1$. u, v, w are the strings of 0's. Therefore, $v = 0^m$ for some $n \geq m \geq 1$. So $|v| = m$.

Step 3 Let $i = p + 1$. Then,

$$\begin{aligned} |uv^i w| &= |uvw| + |v^{i-1}| \\ &= p + (i-1)m \\ &= p + (i+1-1)m && \text{by } i = p+1 \\ &= p + pm \\ &= p(1+m). \end{aligned}$$

By pumping lemma $uv^i w \notin L$ with $i = p + 1$, because $|uv^i w| = p(1 + m)$, and $p(1 + m)$ is not a prime number, since it is divisible by p and $(1 + m)$ where $|1 + m| \geq 2$. Here we get a contradiction to say that L is not regular.

Example 5.4

By using pumping lemma show that the language $L = \{a^{n^2} \mid n > 1\}$ is not regular.

Solution

Step 1 Let us assume that $L = \{a^{n^2} \mid n > 1\}$ is regular. If the language $L = \{a^{n^2} \mid n > 1\}$ is regular then by mathematical induction $L = \{a^{(m-1)^2} \mid m > 0\}$ is also regular. Let us assume $n = m + 1$

Step 2 Let $z = a^{(m+1)^2}$ then $|z| = (m + 1)^2$

By pumping lemma we write

$$z = uvw \text{ such that } |uv| \leq (m + 1)^2 \text{ and } |v| \geq 1.$$

Step 3 As the string $a^{(m+1)^2}$ contains occurrences of a 's we have

$$z = a^{(m+1)^2} = a^{m^2+2m+1} = a^{m^2} a^{2m} a$$

Let $u = a^{m^2}$, $v = a^{2m}$, and $w = a$, then by pumping lemma, we have

$$z = a^{m^2} (a^{2m})^i a$$

for $i = 0$

$$z = a^{m^2} a = a^{m^2+1}$$

which is a contradiction, because $m^2 + 1$ can never be complete square for $m > 0$. In other words

$$m^2 < m^2 + 1 < (m + 1)^2$$

Therefore, L is not regular.

Example 5.5

By using pumping lemma prove that the language $L = \{a^n b^k \mid n > k \geq 0\}$.

Solution

Step 1 Let us assume that $L = \{a^n b^k \mid n > k \geq 0\}$ is regular.

Step 2 By pumping lemma we write

$$z = a^n b^k$$

$$z = uvw \text{ such that } |uv| \leq n + k, \text{ and } |v| \geq 1.$$

Step 3 The part 'v' may contain

- (i) only a's such that $v = a^{n-k}$ with $n - k \neq 0$.
- (ii) Both a's and b's such that $v = a^m b^l$, with $m, l \neq 0$.

Case (i) By pumping lemma we write

$$\begin{aligned} z &= a^n b^k \\ &= a^k a^{n-k} b^k \end{aligned}$$

Here we assume $u = a^k$, $v = a^{n-k}$, and $w = b^k$, then by pumping lemma, we have

$$z = a^k (a^{n-k})^i b^k$$

for $i = 0$

$$z = a^k b^k$$

which is a contradiction because $a^k b^k \notin L$ for any k . In $a^k b^k$ the number of a's and b's are same.

Case (ii) By pumping lemma we write

$$\begin{aligned} z &= a^n b^k \\ &= a^{n-m} a^m b^l b^{k-l} \end{aligned}$$

Here we assume $u = a^{n-m}$, $v = a^m b^l$, and $w = b^{k-l}$, then by pumping lemma, we have

$$z = a^{n-m} (a^m b^l)^i b^{k-l}$$

For $i = 2$ we get

$$\begin{aligned} z &= a^{n-m} (a^m b^l)^2 b^{k-l} \\ &= a^{n-m} (a^m b^l) (a^m b^l) b^{k-l} \\ &= a^n b^l a^m b^k \end{aligned}$$

Which is a contradiction because $a^n b^l a^m b^k \notin L$. In $a^n b^l a^m b^k$, some occurrences of b's are followed by a's, which is not the property of language L .

Example 5.6

Let $L \subseteq (a, b)^*$ be the language of all strings of even length (since ' $\wedge$ ' is even as $\wedge \in L$). Is the language L regular? If it is, what is the regular expression corresponding to it?

Solution Any string of even length can be obtained by concatenating some number of strings of length 2. Conversely, any, such concatenation has even length. It follows that

$$L = \{aa, ab, ba, bb\}^*$$

so that one regular expression corresponding to L is $\{aa, ab, ba, bb\}^*$. This expression can also be written as $((a, b)(a, b))^*$.

❖ Did you Know ?

The pumping lemma was first expressed by Y. Bar-Hillel and M. Perles, E. Shamir by the year 1961. It is a useful lemma for disproving the regularity of a specific language. It is one of the few pumping lemmas, each with similar purpose.

5.5 REGULAR LANGUAGE AND RIGHT LINEAR GRAMMAR

For every regular language there exists a right linear grammar G such that the language generated by the grammar is equivalent to regular language L .

Theorem 5.6 If $L \subseteq \Sigma^*$ is a regular language then there exists a right linear grammar G such that $L(G) = L$.

Proof We have a constructive proof of this theorem. Since the language L is regular let DFA $M = (Q, \Sigma, \delta, q_0, F)$, with $L(M) = L$ and $\Sigma \cap Q = \emptyset$.

Construct grammar $G = (Q, \Sigma, P, q_0)$, where,

$$P = \{A \rightarrow tA' \text{ where } t \in \Sigma \text{ and } A, A' \in Q \text{ with } \delta(A, t) = A'\} \cup \{B \rightarrow \wedge \text{ where } B \in F\}$$

1. the grammar G is right-linear grammar and it directly simulates DFA M .
2. if $a = a_1a_2a_3\dots a_n \in L(M)$ with $\delta(q_0, a_1) = q_1, \delta(q_1, a_2) = q_2, \dots, \delta(q_{n-1}, a_n) = q_n$ then

$$q_0 \Rightarrow a_1q_1 \Rightarrow a_1a_2q_2 \Rightarrow \dots \Rightarrow a_1a_2a_3\dots a_nq_n$$
3. since $q_n \in F$ and $q_n \rightarrow \wedge \in P$, and $a_1a_2\dots a_n \in L(M)$
4. hence, $L(M) = L(G)$.

Summary

- **Closure Properties of Regular Languages** The regular languages are closed under union, concatenation, closure, complementation, intersection and difference.
- **Shuffle** The shuffle operation on any two strings s_1 and s_2 is defined as the set of all strings that can be obtained by interleaving the positions of strings s_1 and s_2 in any way.
- **Pumping Lemma for Regular Languages** The term pumping lemma is made up of two words pumping and lemma. The word pumping refers to generate many input strings by pushing symbols in a string again and again. The word lemma refers to a subsidiary or intermediate theorem in a proof. Pumping Lemma is generally used to prove that certain languages are not regular languages.

Review Questions

- 5.1 What method is applied to prove that a certain language is not regular?
- 5.2 What will happen if pumping lemma is applied to a regular language?
- 5.3 Find the intersection of languages represented by $(0 + 1)^*0$ and $1(0 + 1)^*$.
- 5.4 Find the complement of languages represented by
- $(10 + 01)^*0$
 - $1(00 + 11)^*$
- 5.5 Find the union of languages represented by
- $(0 + 1)^*0$ and $(0 + 1)^*1$.
 - $(0 + 1)^*$, $(00 + 11)^*$ and $(01 + 10)^*$

Graded Exercises

- *5.1 Find the complement of the following DFA:

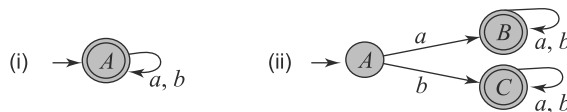


Fig. 5.4 FA of Exercise 5.1

- **5.2 Find the complement of regular languages represented by the following regular expressions:
- $ab^* + ba^*$
 - $(a + ab + ba)^*$
- *5.3 Show that there exists no finite automaton for accepting all palindromes over $(0, 1)$ without using pumping lemma.
- ***5.4 With the help of pumping lemma show that following languages are not regular:
- $L = \{a^n b^k a^n \mid n > k \geq 0\}$.
 - $L = \{a^n b^{n+3} \mid n \geq 0\}$.
- **5.5 If M_1 and M_2 be two finite automata to accept languages L_1 and L_2 . How you will construct finite automata M to accept the following languages:
- $L_1 L_2$
 - $(L_1 + L_2)^*$
- ***5.6 Consider the following DFA M_1 and M_2 accepting languages L_1 and L_2 respectively:

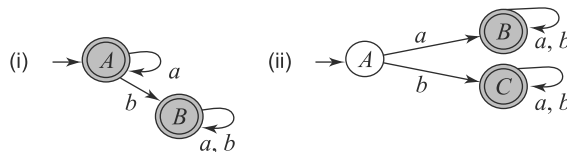


Fig. 5.5 FA of Exercise 5.1

Construct DFA M such that it accepts language L defined as

(i) $L = L_1 L_2$

(ii) $L = L_1 L_2 L_2$

(iii) $L = (L_1 L_2 + L_2 L_1)^*$

* Difficulty level 1

** Difficulty level 2

*** Difficulty level 3

Objective Questions

- Let R_1 and R_2 be regular sets defined over the alphabet Σ , then
 - $R_1 \cup R_2$ is regular
 - $R_1 \cap R_2$ is not regular
 - $\Sigma \cap R_2$ is not regular
 - R_2^* is not regular
- Let L_1 and L_2 be two regular languages, then
 - $L_1 \cup L_2$ is regular
 - $L_1 \cap L_2$ is regular
 - $(L_1 \cup L_2)^*$ is regular
 - all of these
- Which of the following is not a regular language?
 - $L = \{a^n a^{n+3} \mid n \geq 0\}$.
 - $L = \{a^n b^{n+3} \mid n \geq 0\}$.
 - $L = \{a^n a^{n+3} a^n \mid n \geq 0\}$.
 - all of these
- If $L_1 = \{a^n b^n \mid n \geq 0\}$ and $L_2 = \{s \mid s \in (a, b)^*\}$, then
 - $L_1 \cup L_2$ is regular
 - $L_1 \cap L_2$ is regular
 - $(L_1 \cup L_2)^*$ is regular
 - (a) and (c)
- Let L_1 and L_2 be two regular languages, then
 - $L_1 \cup L_2'$ is regular
 - L_1' is regular
 - $(L_1' \cup L_2')'$ is regular
 - all of these

References for Extra Reading

- Bar-Hillel, Y., M. Perles, and E. Shamir, *On formal properties of simple phrase structure grammars*, Z. Phonetik, Sprachwiss Kommunikationsforsch.
- Michael Sipser, *Introduction to the Theory of Computation*, PWS Publishing, 1997.
- Ginsburg, S., G. Rose, *Operations which preserve definability in languages*, J. ACM 10:2, 1963.

Online Sources

- http://en.wikipedia.org/wiki/Regular_language
- http://www.cse.yorku.ca/course_archive/2006-07/F/2001/handouts/lect05.pdf
- <http://www.cse.msu.edu/~tornq/360Book/RegLang/Properties/>
- http://www.inf.ed.ac.uk/teaching/courses/inf2a/slides/2007_inf2a_L09a_slides.pdf
- <http://infolab.stanford.edu/~ullman/ialc/slides/slides5.pdf>

6

Context Free Grammar and Context Free Language

❖ Chapter Objective ❖

In Chapter 3 we have seen what context-free grammars and languages are. In this chapter we will discuss their representation by trees called derivation or parse trees. We will discuss how to find a set of terminal strings for context-free languages. We will present the concept of ambiguity in CFGs and then we will illustrate how to eliminate ambiguity from CFGs. When we are concerned with equivalence of PDA and context-free grammars, we need to represent context-free grammars in a particular format called normal forms. For this purpose we will describe Chomsky and Greibach Normal forms. We will also describe the method for converting a CFG into these normal forms. We will also discuss the limitations before converting a CFG into Chomsky or Greibach normal form.

□ Introduction □

Context-free grammars were first used in the study of human languages. Context-free grammars are used as the basis in the design and implementation of a compiler. The designers of compilers often use such grammars to implement components of a compiler, like parsers, scanners, code generators, etc. Any implementation of programming language is preceded by a context-free grammar that specifies it.

The origin of the context-free grammar formalism is found in works by Noam Chomsky (1956); important later writings by Chomsky appeared in between 1959 to 1963. The related Bacus-Naur Form notation was used for the description of ALGOL by Bacus in 1959, and by Peter Naur in 1960. The Chomsky Normal Form introduced in 1959, was proved by Noam Chomsky, and the Greibach normal form was proved by Sheila Greibach in 1965.

Ambiguity in context-free grammars was first studied formally by Floyed and Cantor (in 1962), Chomsky, Schutzenberger and Greibach (in 1963).

Important applications of the context-free grammar theory have been made to compiler design. The validity of high level language statements is checked by context-free grammar and context-free language by constructing derivation trees or parse trees. Context-free languages are applied in parser design and Chomsky normal form is used in decidability (emptiness of CFG) concept of push down automata.

6.1

DEFINITION OF CONTEXT FREE GRAMMAR

A context-free grammar (CFG) is defined as 4-tuples (V_N, Σ, P, S) , where P is a set of production rules of the form

one nonterminal $\rightarrow$ finite string of terminals and/or nonterminals

This general form of productions in CFG, states that the nonterminal in left hand side of the production is free from any context (there is neither left nor right context), therefore these types of grammars are called context-free grammars.

The general form of production of context-free grammar can also be described in another form as we have seen in Chapter 3. According to that G is a context-free grammar if every production it has is of the form.

$$A \rightarrow \alpha$$

where, $A \in V_N, \alpha \in (V_N \cup \Sigma)^*$

❖ Good to Know!

Production rules are also known as grammar rules. Some text authors use the symbol ‘ $::=$ ’ instead of ‘ $\rightarrow$ ’ in productions. Also, some authors indicate nonterminals (variables) by writing them within angle brackets like

$$\langle S \rangle \rightarrow \langle X \rangle \mid \langle Y \rangle \mid \langle Z \rangle$$

$$\langle Z \rangle \rightarrow a \mid a \langle X \rangle$$

$$\langle Y \rangle \rightarrow b \mid b \langle Y \rangle$$

$$\langle Z \rangle \rightarrow \wedge$$

Here $S, X, Y, Z \in V_N$ and $a, b \in \Sigma$

Example 6.1

Show that the grammar described by the following grammar rules is a context-free grammar.

$$S \rightarrow S + S \mid S - S \mid S * S \mid S / S \mid (S) \mid a$$

Solution According to productions given, we have

$$V_N = \{S\}$$

$$\Sigma = \{+, -, *, /, (,), a\}$$

Since we see that all the given productions are of the form

$$A \rightarrow \alpha$$

where, $A \in V_N, \alpha \in (V_N \cup \Sigma)^*$. The point to be noted here is that, in the left hand side of any production the only variable is S , which does not have any left or right context (S is free from context). Therefore the given grammar is a context-free grammar.

❖ Good to Know!

(i) A grammar G , defined as $E \rightarrow E + E$, $E \rightarrow E^*E$, $E \rightarrow (E)$, $E \rightarrow id$, can also be written as $E \rightarrow E + E \mid E^*E \mid (E) \mid id$. We will use both formats in the book.

(ii) When we deal with more than one grammar simultaneously, we will use notation G_1 or G_2 or something else with derivation symbol $\Rightarrow$ to make sure a particular derivation corresponds to which grammar. For example, $S \xRightarrow{*}_{G_k} w$ states that the string 'w' is derived from production rules of grammar G_k in more than one step.

6.2

CONTEXT FREE LANGUAGE

The languages generated by context-free grammars are called context-free languages. An important example of context-free languages is the syntax of programming languages. Context-free languages are also applied in the design of a parser. These are also useful for describing block structure in programming languages. However, all the syntactic aspects of programming languages are not captured by the context-free grammar, i.e., the fact that a variable has to be declared before being used and the type correctness of expressions are not captured.

Context-free languages, like regular sets, have great importance practically e.g., in formalising the notation of parsing, simplifying the translation of programming languages, defining programming languages, and other string-processing applications. As an example, CFGs are useful for describing arithmetic expressions, with arbitrary nesting of balanced parentheses. For understanding all these aspects of programming language the study of regular expressions is a must.

Context-free languages are more complicated than regular languages. You will recall that in Chapter 2 we did not define a deterministic regular language, although we considered both deterministic and nondeterministic FAs. The reason, of course was that for any N DFA there is a DFA recognizing the same language. But it is not true in case of push down automata. We will see that deterministic and nondeterministic CFLs are quite different.

6.3

DETERMINISTIC CONTEXT FREE LANGUAGE (DCFL)

A language L is a deterministic context-free language (abbreviated as DCFL) if there is a deterministic push down automaton (abbreviated as DPDA) accepting language L . Note that a push down automaton M is said to be deterministic if there is no configuration for which M has a choice of more than one move in the same input conditions.

A context-free grammar is deterministic if and only if its rules allow a (left-to-right, bottom-up) parser to proceed without having to guess the correct continuation at any point. A **parser** can be defined as the practical implementation of PDA.

A language is deterministic context-free if and only if it is generated by some deterministic context-free grammar. Not all context-free languages are deterministic.

As an exmple, the following grammar is not deterministic:

$$S \rightarrow aS, S \rightarrow aSb, S \rightarrow b$$

While we are parsing, we cannot decide which rule to apply without knowing the rest of the string in advance. The language being described is in fact nondeterministic; it is impossible to write a deterministic context-free grammar for it.

Determinism is a stronger notion than non-ambiguity. A grammar is ambiguous if it allows two different presentations for the same sentence. Determinism requires that ambiguity must not present at any point during parsing.

Let us take an example of context-free grammar for a simplified version of arithmetic expressions:

$$S \rightarrow S+S \mid S-S \mid S*S \mid S/S \mid N$$

$$N \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid \dots \mid 9$$

We are all familiar with the ambiguity inherent in the expression $2 + 3*4$. Does it mean $(2 + 3)*4$, giving value 20, or does it mean $2 + (3*4)$, which is 14?

In this defined language we have no option of putting parentheses for clarification, because parentheses cannot be generated by any of the production system and are therefore not the symbols (terminals) in the derived language. There is no question that $2 + 3*4$ is a word in the language derived by this context-free grammar. The only question is what does this word intend in terms of calculation?

If we include the parentheses in the grammar

$$S \rightarrow (S + S) \mid (S - S) \mid (S * S) \mid (S / S) \mid N$$

then, we will not be able to generate the string $2 + 3*4$ at all. We could only produce

$$\begin{aligned} S &\Rightarrow (S+S) \\ &\Rightarrow (S + (S*S)) \\ &\Rightarrow (2 + (S*S)) \\ &\Rightarrow (2 + (3*S)) \\ &\Rightarrow (2 + (3*4)) \end{aligned}$$

or

$$\begin{aligned} S &\Rightarrow (S*S) \\ &\Rightarrow ((S + S) * S) \\ &\Rightarrow ((2 + S) * S) \\ &\Rightarrow ((2 + 3) * S) \\ &\Rightarrow ((2 + 3) * 4) \end{aligned}$$

Neither of these is an ambiguous expression.

Practically we do not need to use all these parentheses because we use the priority of operators, which says that $*$ is to be executed before $+$. We can distinguish between these two possible meanings for the expression $2 + 3*4$ by looking at the two possible hierarchical structures called derivation trees or parse trees.

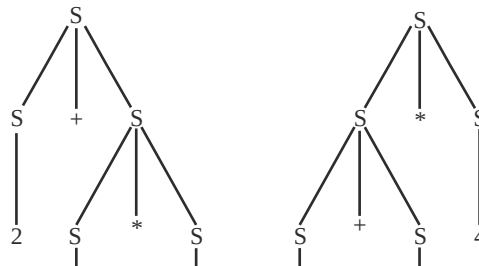


Fig. 6.1 The possible parse trees for $2 + 3*4$

Therefore an expression can be evaluated in the form of a parse tree by starting from leaves to root (bottom-up-approach), replacing each nonterminals (capital letter) as we come to it by the result of the calculation that it produces. In Fig. 6.2 the nonterminals inside the circle are evaluated first.

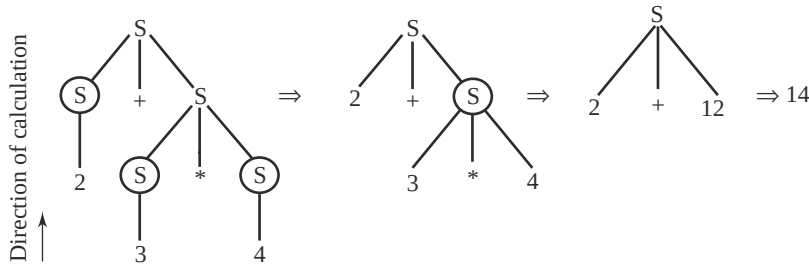


Fig. 6.2 The possible parse trees for $2 + 3 * 4$

The parse tree given in Fig. 6.1(b) can be evaluated in the same way as the parse tree given in Fig. 6.1(a) by using steps shown in Fig. 6.2.

6.4

DERIVATIONS

A derivation can be defined as the process to generate string of terminals by replacing the nonterminals in the right hand side of a production. While the terminal string 's' is generated from an S-production (the production having start symbol S in left hand side of a production) of grammar G then $s \in L(G)$. The symbol $\Rightarrow$ is used to represent a derivation. To distinguish whether it is a leftmost derivation or a rightmost derivation we use notation LMD and RMD with $\Rightarrow$. This way the notations $\xRightarrow{LMD}$ and $\xRightarrow{RMD}$ represent leftmost and rightmost derivations respectively.

In other words, derivation is the process of generation of terminal string from a given set of productions. The yield of a derivation tree is also known as derivations. If $s = 'aba'$ is a derivation, it means 's' is the string obtained by concatenating the labels of leaf nodes from left to right.

Note The definitions of leftmost derivation, rightmost derivation and yield are discussed later in this chapter.

6.5

PARSE TREES

The strings generated by a context-free grammar $G = (V_N, \Sigma, P, S)$ can be represented by a hierarchical structure called tree. (A hierarchical structure is that structure in which elements have some relationship with each other, e.g., if a node A is a parent node of B then A can be accessed through B and vice versa).

A parse tree (also called derivation tree, syntax tree, production tree, generation tree) for a context-free grammar G has the following characteristics:

- (i) Every vertex of a parse tree has a label which is either a variable (generally denoted by an uppercase letter and also called nonterminal) or a terminal from $\{\Sigma \cup \wedge\}$.
- (ii) The root of a parse tree has label S, where S is start symbol.

- (iii) The label of an internal vertex is always a variable or nonterminal.
- (iv) If a vertex A has k children with labels $A_1, A_2, A_3, A_4, \dots, A_k$, then there exists a production $A \rightarrow A_1 A_2 A_3 A_4 \dots A_k$ in context-free grammar G .
- (v) A vertex is an intermediate node if its label is $N \in V_N$.
- (vi) A vertex is a leaf node if its label is $n \in \{\Sigma \cup \wedge\}$.
- (vii) A vertex is the only child of its parent node if its label is $\wedge$.

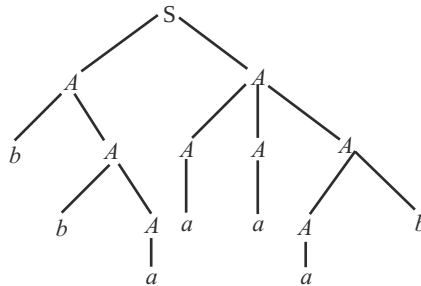


Fig. 6.3 A derivation tree of grammar G_1 giving yield 'bbaaaab'

❖ Good to Know!

There is no need to draw an arrow for direction on the edges in a derivation tree because the direction of production is always downward.

Let $G_1 = (\{S, A\}, \{a, b\}, P, S)$, where P contains the productions $S \rightarrow AA$, $A \rightarrow AAA|bA|Ab|a$, the derivation tree for string 'bbaaaab' generated by this grammar is given by Fig. 6.3 above. The only rule for formation of a derivation tree is that every nonterminal begins to grow branches leading to every symbol in right side of the production that replaces it. The nonterminal A can be replaced by 'ba' if there exists a production

$$A \rightarrow ba$$

and following subtree can be drawn for this production

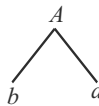


Fig. 6.4 A derivation tree for the production $A \rightarrow ba$

In Fig. 6.3 the vertices having labels 'a' or 'b' are leaves (or say leaf nodes), while the vertices having labels S and A are root and intermediate nodes respectively. In a valid set of productions these symbols must appear at least once, in left side without context.

❖ Definitions

- ❖ **Yield of a derivation tree.** The yield of a derivation tree is the concatenation of the labels of leaf nodes in left-to-right ordering without repetition. For example the yield of the derivation tree given by Fig. 6.5 (a) is 'bba'.
- ❖ **Subtree of a derivation tree.** A subtree of a derivation tree is a tree having the following characteristics:
 - (i) the root of a subtree is some vertex 'v' of derivation tree, where $v \in V_N$.
 - (ii) the vertices of subtree are descendants of 'v', with their labels, and
 - (iii) the edges of subtree connect the descendants of 'v'.

As an example, if we consider the derivation tree given by Fig. 6.3, the following are the subtrees of this derivation tree:

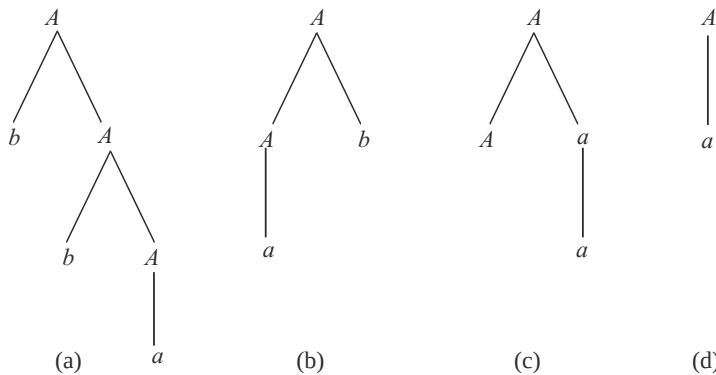


Fig. 6.5 The subtrees of derivation tree given by Fig. 6.3

The structure of the derivation tree depends upon the sequence and selection of production rules to construct a derivation tree. Therefore, a derivation 's' may be obtained either by leftmost derivation or rightmost derivation.

❖ Definitions

- ❖ **A-tree** A subtree looks like a derivation tree except that the label of the root node may not be S (the start symbol) if and only if S does not appear in the right side of any production. It is called an A-tree if the label of its root is A. Similarly if there is another subtree whose label of root node is B then it is called a B-tree.
- ❖ **Working string** The arrow symbol ' $\Rightarrow$ ' is employed between the unfinished stages of the generation of a string or a derivation. It means that there exists a possible substitution in $XY \Rightarrow abY$ if there exists a production $X \rightarrow ab$. These unfinished stages are strings of terminals and nonterminals that are generally called working strings.
- ❖ **Internal nodes** A node in a derivation tree whose label is a variable (or say nonterminal) is called a leaf node or an external node. For example, if the label of a node is $A \in V_N$, then A is an internal node.
- ❖ **Leaf node** These are also called external nodes. A node in a derivation tree, whose label is a terminal $a \in \Sigma$ or $\wedge$ is called a leaf node or external node or simply a leaf.
- ❖ **Sentential Form** A string of terminals and variables (say α) is called a sentential form if $S \xRightarrow{*} \alpha$, where $S \in V_N$ and $\alpha \in (V_N \cup \Sigma)^*$.
- ❖ **Nullable variable** A variable E in CFG G is said to be nullable if it derives (or say generates) $\wedge$ i.e., $E \xRightarrow{*} \wedge$.

6.6

FROM INFERENCE TO TREE

Theorem 6.1 If $G = (V_N, \Sigma, P, S)$ is a context-tree grammar, then $S \Rightarrow^* \alpha$ if and only if there is a derivation tree in grammar G with yield α .

Proof First of all we would like to prove that $A \Rightarrow \alpha$ if and only if there is A -tree with yield α . Once this is proved, the theorem will also be proved by assuming that $A = S$.

We prove that $A \Rightarrow \alpha$ by induction on the number of internal vertices in an A -tree (say T). When the tree T has only one internal vertex, the remaining vertices are leaf nodes and are children nodes of root A , as given by Fig. 6.6 below:

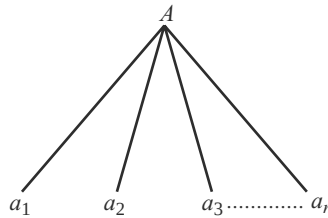


Fig. 6.6 A derivation tree for $A \Rightarrow a_1 a_2 a_3 \dots a_n$

By the definition of a derivation tree, the derivation tree given by Fig. 6.6, shows that there must be a production

$$A \rightarrow a_1 a_2 a_3 \dots a_n \quad (\text{where } a_1 a_2 a_3 \dots a_n = \alpha)$$

or we can say that there is a derivation

$$A \Rightarrow \alpha$$

Thus there is a basis for induction. Let us assume the result for all trees with at most $k - 1$ internal nodes (i.e., the nodes whose labels are variables), where $k > 1$.

Let us assume that T is an A -tree with k internal nodes, where $k \geq 2$. Let $P_1, P_2, \dots, P_n$ be the children of the root in the left-to-right ordering, with their labels $X_1, X_2, X_3, \dots, X_n$ respectively. Therefore, there must be a production

$$A \rightarrow X_1 X_2 X_3 \dots X_n$$

in P , and there is a derivation $A \Rightarrow X_1 X_2 X_3 \dots X_n$

As $k \geq 2$, then there is at least one child which is internal node. According to left-to-right ordering of leaves, α can be written as $\alpha_1 \alpha_2 \alpha_3 \dots \alpha_n$, where each α_i is obtained by the concatenation of the labels of the leaves which are descendants of vertex P_i . If P_i is an internal vertex, consider the subtree T with P_i as its root. The number of internal nodes of the subtree is less than k (as there are k internal nodes in T and at least one of them, i.e., its root is not in the subtree). So by the induction hypothesis applying to the subtree,

$$X_i \Rightarrow^* \alpha_i$$

If P_i is not an internal node (i.e., it is a leaf node), then $X_i = \alpha_i$. By using the production $A \rightarrow X_1 X_2 \dots X_n$, we get the derivation

$$\begin{aligned}
 A &\Rightarrow X_1 X_2 X_3 \dots X_n, \\
 &\Rightarrow X_2 X_3 \dots X_n, \\
 &\quad \cdot \\
 &\quad \cdot \\
 &\Rightarrow \alpha_1 \alpha_2 \alpha_3 \dots \alpha_n \\
 &= \alpha
 \end{aligned}$$

or we can say $A \Rightarrow \alpha$. According to the principle of induction, $A \Rightarrow \alpha$ whenever α is the yield of an A -tree. For the proof of ‘only if’ part, let us assume that $A \Rightarrow \alpha$. We have to construct an A -tree whose yield is α . We can do this induction through the number of steps to get $A \Rightarrow \alpha$. If there is a derivation $A \alpha$, then $A \rightarrow \alpha$ is a production in P . If $\alpha = X_1 X_2 X_3 \dots X_n$, the A -tree with yield is constructed, as given by Fig. 6.7.

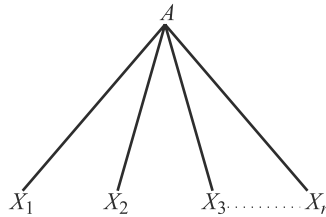


Fig. 6.7 A derivation tree for one step derivation

This is the basis for induction. Suppose the result for derivations takes at most k steps. Suppose

$$A \xRightarrow{k} \alpha$$

We can split this derivation as

$$A \Rightarrow X_1 X_2 X_3 \dots X_n \xRightarrow{k-1} \alpha$$

where $A \Rightarrow X_1 X_2 X_3 \dots X_n$ implies the production $A \rightarrow X_1 X_2 X_3 \dots X_n$ in P . In the derivation $X_1 X_2 X_3 \dots X_n \alpha$, there is one case out of two cases:

- (i) X_i is not changed throughout the derivation, or
- (ii) X_i is changed in some subsequent step

Let α_i be the substring of derived from X_i . Then

$$X_i \Rightarrow \alpha_i \quad \text{in case (i)}$$

and

$$X_i \Rightarrow \alpha_i \quad \text{in case (ii)}$$

As G is context-free grammar, therefore in every step of the derivation

$$X_1 X_2 X_3 \dots X_n \Rightarrow \alpha$$

We replace a single variable by a string. As $\alpha_1, \alpha_2, \dots, \alpha_m$ account for all the symbols in α .

$$\alpha = \alpha_1 \alpha_2 \alpha_3 \dots \alpha_n.$$

We construct the derivation tree with yield α , as follows:

As $A \rightarrow X_1 X_2 X_3 \dots X_n \in P$, we construct a derivation tree with n external (leaf) nodes whose labels are $X_1 X_2 X_3 \dots X_n$ in the left-to-right ordering. The constructed derivation tree is given by Fig. 6.8.

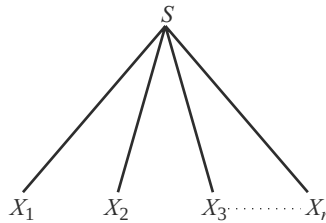


Fig. 6.8 A derivation tree with yield $X_1X_2X_3\ldots X_n$

In case (i), we leave the node P_i as it is. In case (ii), $X_i \xRightarrow{*} \alpha_i$ in less than k steps (as $X_1X_2X_3\ldots X_n \xRightarrow{m-1} \alpha$). According to induction hypothesis there exists an X_i -tree T_i with yield T_i . We attach the tree T_i at the node P_i , as P_i is root of tree T_i . The resulting constructed tree is given by Fig. 6.9.

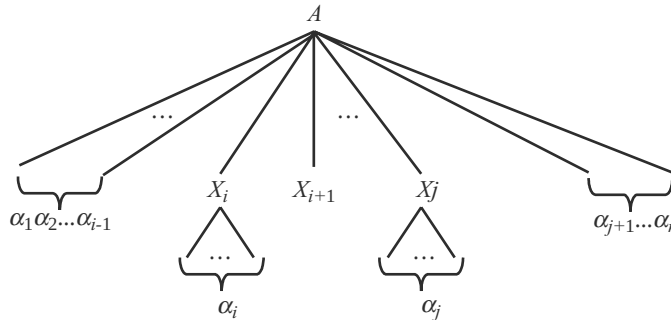


Fig. 6.9 Derivation tree with yield $\alpha_1\alpha_2\alpha_3\ldots\alpha_n$

In the Fig. 6.9, let us suppose i and j are the first and last indexes such that X_i and X_j satisfy (ii) case. Therefore, $\alpha_1\alpha_2\ldots\alpha_{i-1}$ are the labels of leaves at level 1 in T . α_i is the yield of the X_i -tree T_i , α_{i+1} is the yield of the X_{i+1} -tree T_{i+1} , and so on.

Thus we get a derivation tree with yield α . By the principle of induction we get the result for any derivation. This completes the proof of ‘only if’ part of the theorem.

6.7

DERIVATION TREE AND NEW NOTATION OF ARITHMETIC EXPRESSIONS

In the special type of context-free grammar of operators and operands, we can construct meaningful derivation trees that enable us to introduce a new notation for arithmetic expression which has direct applications to computer science. These notations are prefix and post fix notations. Both these methods of notations are useful in computer science. Compilers often convert infix to prefix notation and then to assembler code. From a derivation tree of an algebraic expression, we can get equivalent prefix and postfix notations. An algebraic expression in terms of operators and operands can be derived by an ambiguous context-free grammar.

Prefix notation is the parenthesis-free notational scheme invented by Polish logician Jan Lukasiewicz (1878–1966) and is often called polish notation. In prefix notation operator is followed by operands. For

example, in prefix notation $A + B$ is written as $+AB$. Postfix notation is reverse of prefix notation $AB+$ is the equivalent postfix notation of $A + B$.

First of all the derivation tree of expression (infix form) generated by context-free grammar, is constructed. Let us suppose the given ambiguous CFG G is defined by the following productions

$$\begin{aligned} S &\rightarrow (S) \\ S &\rightarrow S + S \\ S &\rightarrow S - S \\ S &\rightarrow S * S \\ S &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \end{aligned}$$

Here we are assuming that every operated is of single digit. Suppose the generated algebraic expression by grammar G is

$$S \Rightarrow ((6 + 5) * (3 - 4) + 2) * 1$$

We can easily construct the derivation tree for this expression by using the production sequences we have applied. Note that there is no need to show parentheses in derivation tree. The constructed derivation tree for $((6 + 5) * (3 - 4) + 2) * 1$ is given by Fig. 6.10.

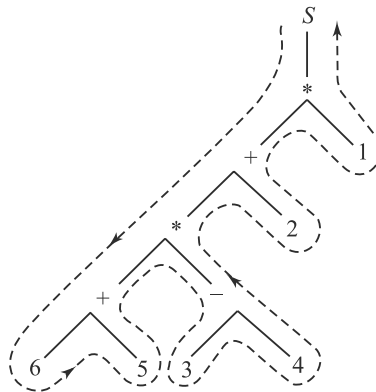


Fig. 6.10 Derivation tree for expression $((6 + 5) * (3 - 4) + 2) * 1$ with direction for evaluation of prefix notations

As the derivation tree is unambiguous, the prefix notation is also unambiguous. To convert the infix expression $((6 + 5) * (3 - 4) + 2) * 1$ into equivalent prefix notation we need a binary tree (here we consider derivation tree given by Fig. 6.10) in the sequence specified by arrows. Scanning this way, we get

$$*, +, *, 6, 5, -, 3, 4, 2, 1$$

This is the equivalent prefix expression. Make sure that neither an operator nor an operand is scanned more than once. Here we are not concerned with how this prefix expression will be evaluated.

6.8

SENTENTIAL FORMS

There may be so many productions in a context-free grammar that generate terminal strings on replacement of nonterminals in right hand side with some terminals. But the productions which have a start symbol in left hand side of a CFG have a special role. The set of all terminal strings represented

by the start symbol constitute the language of a CFG. Let $G = (V_N, \Sigma, P, S)$ be a CFG and there is some string $s \in (V_N, \Sigma)^*$ such that $S \xRightarrow{*} s$, then there exists a sentential form. If the sentential form is obtained through a leftmost derivation then it is called left sentential form. Similarly if the sentential form is obtained through a rightmost derivation then it is called right sentential form.

If the number of terminal strings represented by the start symbol are finite then there will be finite number of derivation or parse trees to represent those terminal strings, otherwise there will be infinite number of derivation trees.

6.9 RIGHTMOST AND LEFTMOST DERIVATION OF STRINGS

❖ Definition

Leftmost derivation A derivation $S \xRightarrow{*} s$ is said to be a leftmost derivation if production is applied only to the left most variable (nonterminal) at every step. It is not necessary that we can derive a derivation by start symbol only; we can also obtain a derivation from some other variable in V_N .

Let us consider a CFG G having productions $S \rightarrow S - S \mid S * S \mid (S) \mid a$. For clarity, the leftmost variable in derivation is in bold style. Suppose, first of all we use production $S \rightarrow S - S$

$$S \Rightarrow \mathbf{S} - S$$

Now we can apply a production on leftmost variable. We have

$$S \Rightarrow \mathbf{S} * S - S$$

Now we apply as S-production (here $S \rightarrow a$) on leftmost $\mathbf{S}$ to get

$$S \Rightarrow a * \mathbf{S} - S$$

$$\Rightarrow a * a - \mathbf{S}$$

$$\Rightarrow a * a - a$$

In other words, we can say that

$$S \xRightarrow{*} a * a - a$$

where, $a * a - a$ is the leftmost derivation. For more precise description, we write

$$S \xRightarrow[LM]{*} a * a - a$$

Here, LM denotes the leftmost derivation.

❖ Definition

Rightmost derivation A derivation $S \xRightarrow{*} s$ is said to be a rightmost derivation if production is applied only to the right most variable (nonterminal) at every step. It is not necessary that we can derive a derivation by start symbol only; we can also obtain a derivation from some other variable in V_N .

Let us again consider a CFG G having productions $S \rightarrow S - S \mid S * S \mid (S) \mid a$. Now we will find a rightmost derivation by using this grammar. During derivation steps, we will represent the rightmost variable in bold style. Let us start with the production $S \rightarrow S - S$

$$S \Rightarrow S - \mathbf{S}$$

$$\begin{aligned} &\Rightarrow S - a \\ &\Rightarrow S * S - a \\ &\Rightarrow S * a - a \\ &\Rightarrow a * a - a \end{aligned}$$

In other words, we can say that

$$S \Rightarrow^* a * a - a$$

where, $a * a - a$ is the rightmost derivation. For more precise description we write

$$S \Rightarrow_{RM}^* a * a - a$$

Here, RM denotes the rightmost derivation.

Let us consider the context-free grammar G having two production $S \rightarrow S + S \mid a$. The string $a + a + a$ has two distinct leftmost derivations:

$$S \Rightarrow S + S \Rightarrow a + S \Rightarrow a + S + S \Rightarrow a + a + S \Rightarrow a + a + a$$

and

$$S \Rightarrow S + S \Rightarrow S + S + S \Rightarrow a + S + S \Rightarrow a + a + S \Rightarrow a + a + a$$

The corresponding derivation trees are given by Fig. 6.11(a), (b).

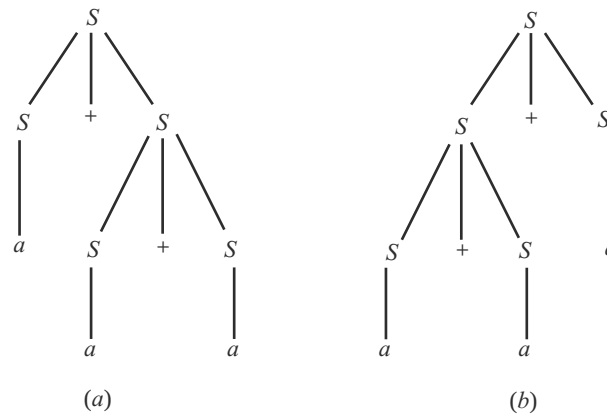


Fig. 6.11 The derivation trees for $a + a + a$

❖ Definition

Total language tree For a given context-free grammar G , we define a derivation tree with the start symbol as its root node, and its children as working string of terminals and nonterminals. The descendants of each node are all possible results of applying every applicable production to the working string, one at a time. A string of all terminals is a terminal node in the tree. This type of resultant tree is called the total language tree of the context free grammar G .

For example, if a CFG G consists of following productions:

$$S \rightarrow aa \mid bA \mid aAA, A \rightarrow b \mid a.$$

The total language tree for this CFG is given by Fig. 6.12 (see below).

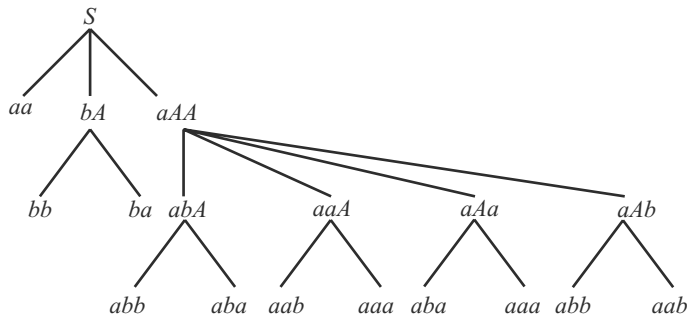


Fig. 6.12 The total language tree for grammar $S \rightarrow aa \mid bA \mid aAA, A \rightarrow b \mid a$

Example 6.2

Consider the context grammar $G = (\{S, A\}, \{0, 1\}, P, S)$ where, $P = \{S \rightarrow A \mid b, A \rightarrow aA\}$. Construct the total language tree for G .

Solution The total language tree begins with the productions $S \rightarrow A \mid b, A \rightarrow aA$

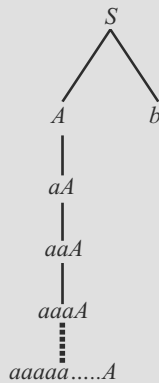


Fig. 6.13 The total language tree of Example 6.2

It is clear that the only string in the language is the single letter 'b'. Therefore, $L(G) = \{b\}$.

Example 6.3

Consider the following derivation tree (see below, Fig. 6.14).

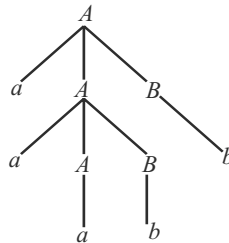


Fig. 6.14 The derivation tree of Example 6.3

Determine the yield given by this derivation tree.

Solution While we scan the labels of leaf nodes from left to right, we get a sequence a, a, a, b and b . The concatenation of these labels is $aaabb$, which is the yield of this derivation tree.

Example 6.4

Show the derivation steps and construct a derivation tree for string $aabbbb$ by using leftmost derivation with the following grammar?

$$S \rightarrow AB \mid \wedge, A \rightarrow aB, B \rightarrow Sb$$

Solution We will start with the production $S \rightarrow AB$. In each step, we will apply production on leftmost variable (shown in bold style). We have

$$\begin{aligned}
 S &\Rightarrow AB \\
 &\Rightarrow aBB && \text{(after applying } A \rightarrow aB) \\
 &\Rightarrow aSbB && \text{(after applying } B \rightarrow Sb) \\
 &\Rightarrow aABbB && \text{(after applying } S \rightarrow AB) \\
 &\Rightarrow aaBbB && \text{(after applying } A \rightarrow aB) \\
 &\Rightarrow aaSbBb && \text{(after applying } B \rightarrow Sb) \\
 &\Rightarrow aa\wedge bBb && \text{(after applying } S \rightarrow \wedge) \\
 &\Rightarrow aa\wedge SbBb && \text{(after applying } B \rightarrow Sb) \\
 &\Rightarrow aa\wedge bbbB && \text{(after applying } S \rightarrow \wedge) \\
 &\Rightarrow aa\wedge bbbSb && \text{(after applying } B \rightarrow Sb) \\
 &\Rightarrow aa\wedge bbb\wedge b && \text{(after applying } S \rightarrow \wedge) \\
 &\Rightarrow aabbbb && \text{(as } \wedge \text{ is length zero string)}
 \end{aligned}$$

The derivation tree can be constructed very easily with the help of leftmost derivation steps. First of all we will draw derivation tree for $S \rightarrow AB$ by assigning S as root and A, B as left and right child respectively, as shown in Fig. 6.15.

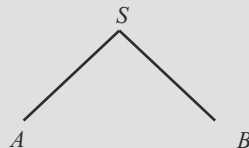


Fig. 6.15 Derivation tree for $S \Rightarrow AB$

In the leftmost derivation at this stage the variable ' A ' is the leftmost variable in the derivation, so we will apply production to ' A ' i.e., the left child of root ' S ' as we see in Fig. 6.16.

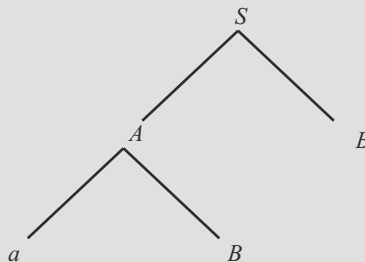


Fig. 6.16 Derivation tree for $S \Rightarrow aBB$

If we scan the leaves (leaf nodes) of the above derivation tree (Fig. 6.16) from left to right we see that B is the leftmost variable on which now the must be applied. It is also described by the second line in the derivation steps. Therefore, after application of the production $B \rightarrow Sb$ on leftmost B , we have Fig. 6.17.

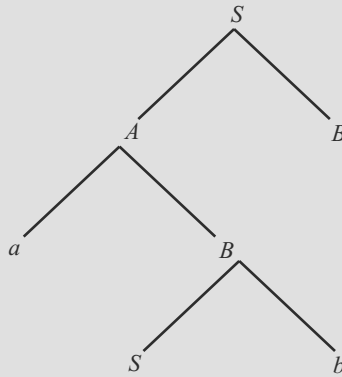


Fig. 6.17 Derivation tree for $S \Rightarrow aSbB$

Now we apply the production on S , as it is the leftmost variable in the current working string. After application of production $S \rightarrow AB$, we have derivation as following (Fig. 6.18):

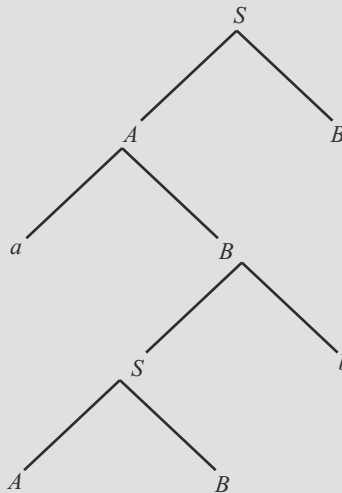
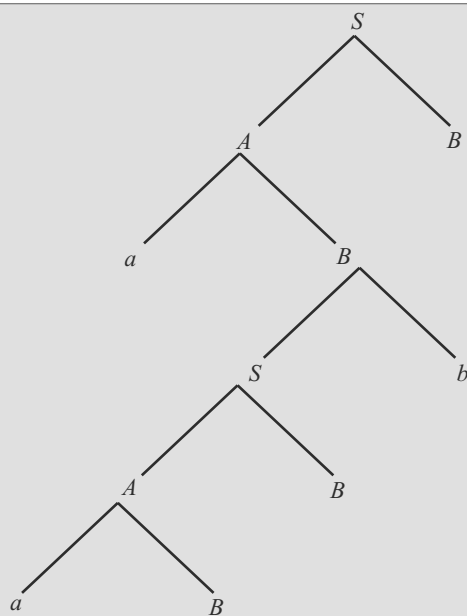


Fig. 6.18 Derivation tree for $S \Rightarrow aABbB$

Now we apply production $A \rightarrow aB$, as A is the leftmost variable in the current working string. We have



Similarly, we construct the complete derivation tree according to the order of application of production in the derivation. Figure 6.20 shows the derivation tree for yield $aa\wedge b\wedge bb\wedge b$.

All leaf nodes containing label $\wedge$ are avoided. Therefore, the derivation tree given by Fig. 6.20 can also be drawn as following (See Fig. 6.21):

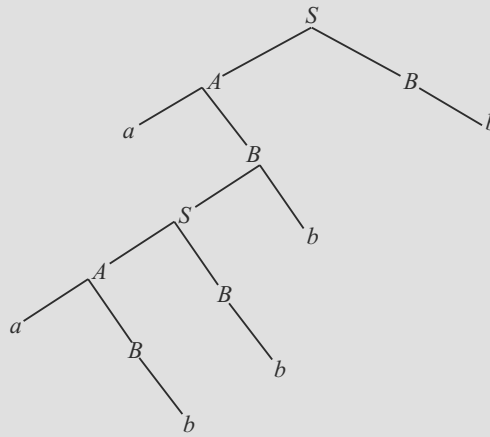


Fig. 6.21 Derivation tree for $S \Rightarrow aabbbb$

Example 6.5

Consider the context-free G , that consists of following production:

$$S \rightarrow aB \mid bA, A \rightarrow a \mid aS \mid bAA, B \rightarrow b \mid bS \mid aBB$$

For the string $aabbabab$, find (a) leftmost derivation, (b) rightmost derivation, and (c) parse tree.

Solution (a) As the string $aabbabab$ has first symbol a , therefore first of all we consider the production $S \rightarrow aB$ for leftmost derivation. Following are the leftmost derivation steps:

$$\begin{aligned} S &\Rightarrow aB \\ &\Rightarrow aaBB && \text{(After applying } B \rightarrow aBB) \\ &\Rightarrow aabB && \text{(After applying } B \rightarrow b) \\ &\Rightarrow aabbS && \text{(After applying } B \rightarrow bS) \\ &\Rightarrow aabbaB && \text{(After applying } S \rightarrow aB) \\ &\Rightarrow aabbabS && \text{(After applying } B \rightarrow bS) \\ &\Rightarrow aabbabaB && \text{(After applying } S \rightarrow aB) \\ &\Rightarrow aabbabab && \text{(After applying } B \rightarrow b) \end{aligned}$$

- (b) For rightmost derivation again we will consider the production $S \rightarrow aB$. In this derivation we will apply production on rightmost variable at every step. Following are the derivation steps:

$$\begin{aligned} S &\Rightarrow aB \\ &\Rightarrow aaBB && \text{(After applying } B \rightarrow aBB) \\ &\Rightarrow aaBbS && \text{(After applying } B \rightarrow bS) \\ &\Rightarrow aaBbaB && \text{(After applying } S \rightarrow aB) \\ &\Rightarrow aaBbabS && \text{(After applying } S \rightarrow bS) \\ &\Rightarrow aaBbabaB && \text{(After applying } S \rightarrow aB) \\ &\Rightarrow aaBbabab && \text{(After applying } B \rightarrow b) \\ &\Rightarrow aabbabab && \text{(After applying } B \rightarrow b) \end{aligned}$$

- (c) The parse tree for string $aabbabab$ is given by the figure below (Fig. 6.22):

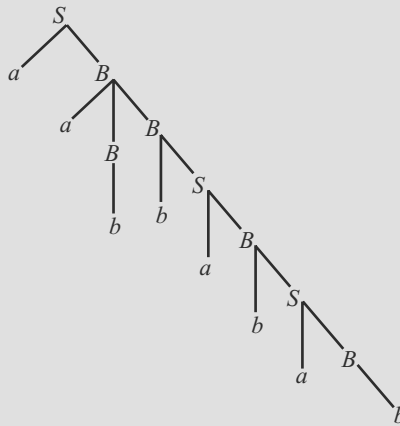


Fig. 6.22 Derivation tree for leftmost derivation of Example 6.5

Similarly, the derivation tree for rightmost derivation can also be drawn.

6.10

AMBIGUITY IN GRAMMAR AND LANGUAGE

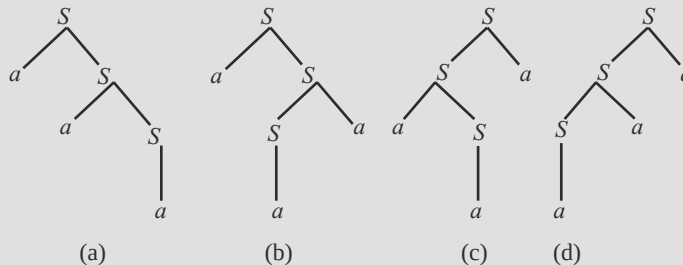
❖ Definition

Ambiguous grammar A context free grammar G is said to be ambiguous if there exists at least one string $s \in L(G)$ having two or more distinct derivation trees (or equivalently, two or more distinct left most derivations; or two or more distinct right most derivations).

Example 6.6

Show that the language of all non-null strings of a 's defined by a context-free grammar $G = (\{S\}, \{a\}, \{S \rightarrow aS \mid Sa \mid a\}, S)$ is ambiguous.

Solution By using the grammar G , the string a^3 (or say aaa) can be derived by four different ways given by Fig. 6.23 (a) – (d).

Fig. 6.23 Four possible derivation trees for aaa

The derivation steps for (a) are

$$\begin{aligned} S &\Rightarrow aS \\ &\Rightarrow aaS \\ &\Rightarrow aaa \end{aligned}$$

The derivation steps for (b) are

$$\begin{aligned} S &\Rightarrow aS \\ &\Rightarrow aSa \\ &\Rightarrow aaa \end{aligned}$$

The derivation steps for (c) are

$$\begin{aligned} S &\Rightarrow Sa \\ &\Rightarrow aSa \\ &\Rightarrow aaa \end{aligned}$$

The derivation steps for (d) are

$$\begin{aligned} S &\Rightarrow Sa \\ &\Rightarrow Saa \\ &\Rightarrow aaa \end{aligned}$$

A set of four derivation trees for string a^3 (i.e., aaa) shows that grammar G is ambiguous.

6.10.1 Another Example of Ambiguity

A standard example of ambiguity in programming a language is the ‘dangling else’ phenomenon. In C language, one might try to specify grammar rules (productions) to define both the variable (i.e., statement and expression). Two specific type statements in C language are ‘if’ statements and ‘for’ statement, and therefore the overall definition of ‘statement’ might be of the form

$$\text{statement} \rightarrow \text{if-statement} \mid \text{for-statement} \mid \dots$$

The syntax of these two types of statements can be described by the following rules:

$$\text{if-statement} \rightarrow \text{if}(\text{expression}) \text{ statement}$$

$$\text{for-statement} \rightarrow \text{for}(\text{expression}; \text{expression}; \text{expression}) \text{ statement}$$

Example 6.7

Consider the context-free grammar $G = (V_N, \Sigma, P, S)$, where

$$V_N = \{\text{statement}, \text{expression}\}$$

$$\Sigma = \{\text{if}, \text{else}, \text{fun1}(\); \text{fun2}(\); \text{expression1}, \text{expression2}, (,) \}$$

$$\begin{aligned} P = \{ &\text{statement} \rightarrow \text{if}(\text{expression}) \text{ statement}, \\ &\text{statement} \rightarrow \text{if}(\text{expression}) \text{ statement else statement}, \\ &\text{statement} \rightarrow \text{fun1}(\); \mid \text{fun2}(\); \\ &\text{expression} \rightarrow \text{expression1} \mid \text{expression2} \}. \end{aligned}$$

$$S = \{\text{statement}\}$$

Show that the context-free grammar G is ambiguous.

Solution Let us consider the statement

$$\begin{aligned} &\text{if}(\text{expression1}) \\ &\quad \text{if}(\text{expression2}) \\ &\quad \quad \text{fun1}(\); \\ &\quad \text{else} \\ &\quad \quad \text{fun2}(\); \end{aligned}$$

derived by grammar G . This statement can be written in two ways by using the productions of grammar G in two different sequences. In first derivation, the 'else' goes with the first 'if' as shown in derivation tree in Fig. 6.24.

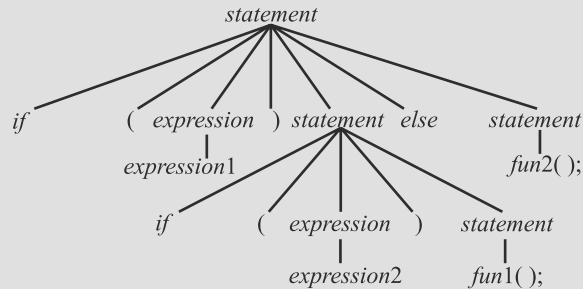


Fig. 6.24 Derivation tree for `if (expression1) if (expression2) fun1( ); else fun2( );`

The derivation given by Fig. 6.24 follows the following steps:

$$\begin{aligned}
 \text{statement} &\Rightarrow \text{if}(\text{expression}) \text{ statement } \text{else} \text{ statement} \\
 &\Rightarrow \text{if}(\text{expression1}) \text{ statement } \text{else} \text{ statement} \\
 &\Rightarrow \text{if}(\text{expression1}) \text{if}(\text{expression}) \text{ statement } \text{else} \text{ statement} \\
 &\Rightarrow \text{if}(\text{expression1}) \text{if}(\text{expression2}) \text{ statement } \text{else} \text{ statement} \\
 &\Rightarrow \text{if}(\text{expression1}) \text{if}(\text{expression2}) \text{fun1}(\text{ }); \text{else} \text{ statement} \\
 &\Rightarrow \text{if}(\text{expression1}) \text{if}(\text{expression2}) \text{fun1}(\text{ }); \text{else} \text{fun2}(\text{ });
 \end{aligned}$$

The other derivation starts with the second 'if' of the grammar. The derivation tree is given by Fig. 6.25.

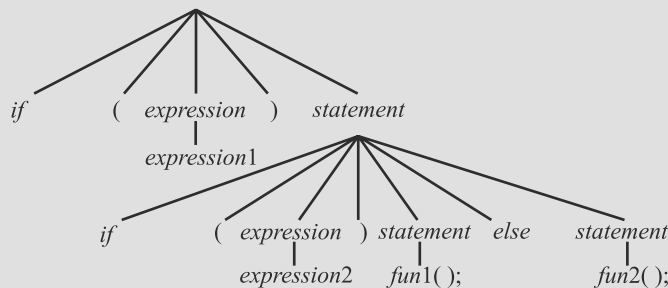


Fig. 6.25 Another derivation tree for `if (expression1) if (expression2) fun1( ); else fun2( );`

The derivation tree given by Fig. 6.25 follows the following steps:

$$\begin{aligned}
 \text{statement} &\Rightarrow \text{if}(\text{expression}) \text{ statement} \\
 &\Rightarrow \text{if}(\text{expression1}) \text{ statement} \\
 &\Rightarrow \text{if}(\text{expression1}) \text{if}(\text{expression}) \text{ statement } \text{else} \text{ statement} \\
 &\Rightarrow \text{if}(\text{expression1}) \text{if}(\text{expression2}) \text{ statement } \text{else} \text{ statement} \\
 &\Rightarrow \text{if}(\text{expression1}) \text{if}(\text{expression2}) \text{fun1}(\text{ }); \text{else} \text{ statement} \\
 &\Rightarrow \text{if}(\text{expression1}) \text{if}(\text{expression2}) \text{fun1}(\text{ }); \text{else} \text{fun2}(\text{ });
 \end{aligned}$$

A C language compiler interprets the statement the second way, but not as a result of the syntax rules given. This is the additional information with which the compiler must be furnished. The parentheses or their equivalent could be used to eliminate the ambiguity in the statement:

if(*expression1*) {*if*(*expression2*) *fun1*();} *else* *fun2*();

forces the first interpretation, while

if(*expression1*) {*if*(*expression2*) *fun1*(); *else* *fun2*();}

forces the second interpretation. Therefore, the grammar G is ambiguous.

6.10.2 Inherent Ambiguity

❖ Definition

Inherently ambiguous CFL A context-free language for which every context-free grammar is ambiguous is said to be an inherently ambiguous context-free language. For example, the language

$$L = \{a^n b^n c^m d^m \mid m, n \geq 1\} \cup \{a^n b^m c^m d^n \mid m, n \geq 1\}$$

is inherently ambiguous because there are infinitely many strings of the form $a^n b^n c^n d^n$, for $n > 1$, which must have two distinct left most derivations.

Let us consider the context-free grammar $L(G)$ with following productions:

$S \rightarrow A \mid BC$

$A \rightarrow aAd \mid aDd$

$D \rightarrow bDc \mid bc$

$B \rightarrow aBb \mid ab$

$C \rightarrow cCd \mid cd$

This grammar with start symbol S generates the language

$$L(G) = \{a^n b^n c^m d^m \mid m, n \geq 1\} \cup \{a^n b^m c^m d^n \mid m, n \geq 1\}$$

For string $aabbccdd \in L(G)$, we have two following different leftmost derivations:

(i) $S \Rightarrow A \Rightarrow aAd \Rightarrow aaDdd \Rightarrow aabDcdd \Rightarrow aabbccdd$

(ii) $S \Rightarrow BC \Rightarrow aBbC \Rightarrow aabbC \Rightarrow aabbCcd \Rightarrow aabbccdd$

Following are the derivation trees giving yield $aabbccdd$ (as given by above derivations):

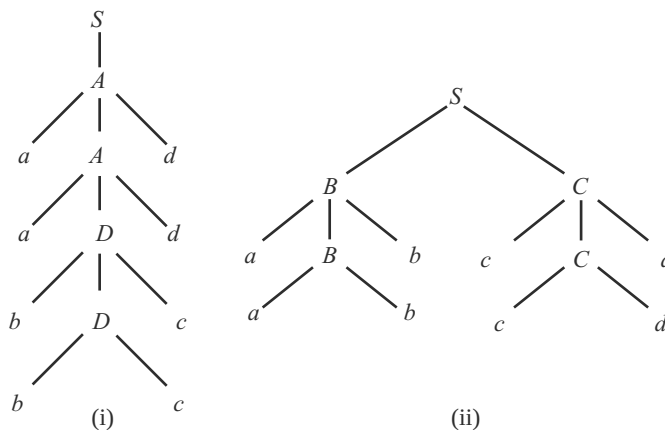


Fig. 6.26 Two different derivation trees for yield $aabbccdd$

❖ Good to Know!

Although some context free languages (CFLs) are inherently ambiguous in the sense they cannot be produced except by ambiguous grammars; ambiguity is the property of grammar rather than the language. If a CFG is ambiguous, it is often possible and usually desirable to find an equivalent unambiguous context free grammar. In section 6.12 we will discuss how an ambiguous context free grammar can be converted into an equivalent unambiguous grammar.

6.11**REMOVAL OF AMBIGUITY**

Ambiguity is the undesirable property of a context-free grammar that we might wish to eliminate. In case of ambiguity, a context-free grammar G has certain terminal strings for which there exist two or more derivation trees. If the productions of the CFG G are modified into another CFG G' in such a manner that no terminal string has more than one derivation tree and $L(G) = L(G')$, then we say the CFG G' is without ambiguity.

6.12**AMBIGUOUS TO UNAMBIGUOUS CONTEXT-FREE GRAMMAR**

Although some context-free languages are inherently ambiguous in the sense that they cannot be generated except by ambiguous grammars, ambiguity is usually a property of grammar rather than the language. If a context-free grammar is ambiguous, it is often possible and desirable to find an equivalent unambiguous context-free grammar.

Let us consider a context-free grammar $G = (V_N, \Sigma, P, S)$, with operators $+$, $*$, $(,)$, and id in Σ . Suppose, P contains the production

$$S \rightarrow S + S$$

$$S \rightarrow S * S$$

$$S \rightarrow (S)$$

$$S \rightarrow id$$

To determine an equivalent unambiguous context-free grammar with other operators is slightly different.

As we see the production $S \rightarrow S + S$ is itself enough to produce ambiguity. Therefore, we need to eliminate the productions of this form. At the same time, we keep in mind the possibility of incorporating into the grammar, the standard rules of order and operator precedence. According to operator precedence, $*$ has higher precedence over $+$. Therefore, the expression $a + b + c$ should mean $(a + b) + c$, rather than $a + (b + c)$.

As we want to eliminate the productions of the form $S \rightarrow S + S$, so we will not think about expressions involving $+$ as being sum of other expressions. They are obviously sums of something. Let us use the letter T to stand for the things which are added to create expressions, where T is nonterminal.

Like sums, the expressions can also be a product. However two expressions $a + b * c$ and $a * b + c$ are both sums, but it is more appropriate to say that the term T can be a product. Additionally, let us say that 'factors' are the things that are multiplied to produce terms. Let us represent these terms by nonterminal F .

Thus, the expressions, the most general objects, are sums of one or more terms, and terms are products of one or more factors. This hierarchy incorporates the procedure of multiplication over addition.

Let us now deal with parentheses. We say (X) could be an expression, a term (say T), or a factor (say F). Now we must select the most appropriate way of derivation of (X) . While evaluation an expression, we cannot do anything with the object inside the parentheses until the object has been evaluated. On the other hand, evaluation of an expression inside the parenthesis takes precedence over any operator outside the parenthesis. In our hierarchy, factors are evaluated first, and therefore the appropriate choice is to say (X) is a factor and ' X ' itself can be an arbitrary expression.

At this stage, expressions are sums of one or more terms, terms are products of one or more factors, and factors are either expression inside parentheses or single identifier. Hence, the 'sum of terms' can be represented as

$$S \rightarrow T + T \mid T$$

But again we would have ambiguity in the grammar. What we say instead is that an expression is either a single term or the sum of a term with some other expression. The only point is whether we want

$$S \rightarrow S + T \quad \text{or} \quad S \rightarrow T + S$$

If we wish to represent $a + b + c$ as $(a + b) + c$, we would probably choose $S \rightarrow S + T$ as more appropriate. In other words, an expression with more than one term is obtained by adding the last term to the sub expression containing all but the last term. For the same reason we choose, the production

$$T \rightarrow T * F$$

rather than the production $T \rightarrow F * T$. The resultant unambiguous context-free grammar is

$$G = (V'_N, \Sigma, P', S)$$

where,

$$V'_N = \{S, T, F\}$$

$$P' = \{S \rightarrow S + T \mid T, T \rightarrow T * F \mid F, F \rightarrow (S) \mid id\}$$

We must now to prove two things:

- (i) G' is equivalent to G (given ambiguous CFG)
- (ii) G' is the unambiguous CFG

For the proof of part (i) we use Theorem 6.2.

Theorem 6.2 Consider the context-free grammar G_1 with productions

$$S \rightarrow S + S$$

$$S \rightarrow S * S$$

$$S \rightarrow (S)$$

$$S \rightarrow id$$

and the context-free grammar G_2 with productions

$$S \rightarrow S + T \mid T,$$

$$T \rightarrow T * F \mid F,$$

$$F \rightarrow (S) \mid id$$

then $L(G_1) = L(G_2)$

Proof As first part, we have to prove that $L(G_2) \subseteq L(G_1)$. This proof is based on induction on the length of a string in $L(G_2)$. The basis step is to show that $d \in L(G_1)$, and this will show that

$$L(G_2) = L(G_1).$$

In the induction step we assume that $k \geq 1$ and every $y \in L(G_2)$ satisfying $|y| \leq k$ is in $L(G_1)$. We must show that if $x \in L(G_2)$ and $|x| = k + 1$, then $x \in L(G_1)$. Because $x \neq a$, so any derivation $x \in L(G_2)$ can be started in one of the following ways:

$$S \Rightarrow S + T$$

$$S \Rightarrow T \Rightarrow T * F$$

$$S \Rightarrow T \Rightarrow (S)$$

If x has a derivation beginning with $S \Rightarrow S + T$, then $x = y + z$, where

$$S \xRightarrow[G_2]{*} y \text{ and } T \xRightarrow[G_2]{*} z$$

since $S \xRightarrow[G_2]{*} T$ follows that $S \xRightarrow[G_2]{*} z$. Therefore, $|y|$ and $|z|$ must be of length less than or equal to k . the induction hypotheses implies that y and z are both in $L(G_1)$. Since G_1 contains the production $S \rightarrow S + S$, the string $y + z$ can be generated from S in G_1 , and hence $x \in L(G_1)$.

Now we have to show that $L(G_1) \subseteq L(G_2)$ as second part. Once again we use induction on $|x|$ as we have applied in first part; therefore the basis step is straight forward. We assume that $k \geq 1$ and for every $y \in L(G_1)$ with $|y| \leq k$, and $y \in L(G_2)$. Now we wish to show that if $x \in L(G_2)$ and $|x| = k + 1$, then $x \in L(G_1)$.

This simplest case is that in which x has a derivation in G_1 starting with $S \rightarrow (S)$. In this case $x = (y)$, for some $y \in L(G_1)$, and it follows from induction hypotheses that $y \in L(G_2)$. Therefore x can be derived in G_2 by starting with the derivation $S \Rightarrow T \Rightarrow F \Rightarrow (S)$ and then deriving y from S .

Let x be a derivation in G_1 that begins with production $S \rightarrow S + S$. Then same as before, the induction hypothesis tells that $x = y + z$, where y and z both are in $L(G_2)$, we need z to be derived from T . In other words, we assume z as a single term, the trailing part of the terms whose sum is x . By keeping this idea in mind, suppose

$$x = x_1 + x_2 + x_3 + x_4 + \dots x_{n-1} + x_n$$

where $x_i \in L(G_2)$, for $i = 1, 2, 3, \dots, n$. Here n is as large as possible. We have already determined that $n \geq 2$. The way by which n is defined, none of the x_i 's can have a derivation in G_2 that begins with $S \Rightarrow S + T$, therefore, every x_i can be derived from T in G_2 .

Suppose

$$y = x_1 + x_2 + x_3 + x_4 + \dots x_{n-1}$$

and

$$z = x_n.$$

then y can be derived from S in G_2 , since

$$S \xRightarrow[G_2]{*} T + T + T + T + \dots + T + T \quad (n-1 \text{ terms}),$$

and z can be derived from T . this shows that $x \in L(G_2)$, since we can start with the production

$$S \rightarrow S + T.$$

Finally, suppose that every production of $x \in L(G_1)$ begins with

$$S \Rightarrow S * S.$$

Then for some $y, z \in L(G_1)$, $x = y * z$. this time let us suppose

$$x = x_1 * x_2 * x_3 * x_4 * \dots * x_{n-1} * x_n$$

where each $x_i \in L(G_1)$, and n is as large as possible. Then by using induction hypothesis, we have each $x_i \in L(G_2)$. This time we have assumed that each x_i is derivable F in G_2 . We can easily rule out the case in which some x_i has a derivation in G_2 that begins with $S \rightarrow T$

$$S \Rightarrow T \Rightarrow T * F.$$

If it is true, then x_i will be of the form $y_i * z_i$ for some $y_i, z_i \in L(G_2)$. As we know that $L(G_2) \subseteq L(G_1)$, this will contradict the maximal property of the number n .

Suppose that x_i had a derivation in G_2 beginning with production $S \rightarrow S + T$, then $x_i = y_i + z_i$ for some $y_i, z_i \in L(G_2) \subseteq L(G_1)$. In that case,

$$x = x_1 * x_2 * x_3 * x_4 * \dots * x_{i-1} * y_i + z_i * x_{i+1} * \dots * x_n.$$

But this is also impossible. Suppose two assumed substrings u and v are before and after the $+$ operator, respectively, we clearly have $u, v \in L(G_2)$, and therefore, $x = u + v$. this means that we could derive x in G_2 using a derivation that begins with $S \Rightarrow S + S$, and we have assumed that this is not the case.

We may conclude that each x_i can be derived from F in G_2 . Then as we did in previous case, we can get

$$y = x_1 * x_2 * x_3 * x_4 * \dots * x_{n-1}$$

and

$$z = x_n.$$

the string y is derivable from T in G_2 , since

$$F * F * F * F * \dots * F \quad (n-1 \text{ factors})$$

is derivable from T in G_2 , therefore we may derive x from S in G_2 by starting the derivation sequences

$$S \Rightarrow T \Rightarrow T * F,$$

and therefore $x \in L(G_2)$.

Thus we see that $x \in L(G_1)$ and $x \in L(G_2)$, and hence, $L(G_1) = L(G_2)$.

Theorem 6.3 The context-free grammar with productions

$$\begin{aligned} S &\rightarrow S + T \mid T, \\ T &\rightarrow T * F \mid F, \\ F &\rightarrow (S) \mid d \end{aligned}$$

is an unambiguous grammar.

Proof If a context-free grammar has more than one leftmost derivations or more than one rightmost derivations then the grammar is said to be ambiguous grammar. If we are able to prove, that for every string $x \in L(G)$ has only one leftmost derivation from start symbol S , the grammar is unambiguous.

Our proof is based on mathematical induction on $|x|$, and it will actually be easier to prove something apparently stronger:

“For any string x derivable from one of the variable S , T or F , the string x has one leftmost derivation from that variable.”

For the basis step, we observe that d can be derived from any of the three variables, and that in each case there is only one derivation. In this induction step, we assume that $k \geq 1$ and for every ‘ y ’ derivable from S , T or F for which $|y| \leq k$, y has only one leftmost derivation from that variable. We wish to show the same result from a string ‘ x ’ with $|x| = k + 1$.

Let us consider the first case in which ‘ x ’ contains at least one $+$ not within the parentheses, ‘ x ’ can be derived from S only, and any derivation for ‘ x ’ must begin with

$$S \Rightarrow S + T$$

where, the $+$ terminal in the derivation is the last $+$ in ‘ x ’ that is not within parentheses. Therefore any leftmost derivation of ‘ x ’ from the start symbol S has the form

$$S \Rightarrow S + T \Rightarrow y + T \Rightarrow y + z$$

where, the last two steps represent leftmost derivation of ‘ x ’ from S and ‘ z ’ from T , respectively, and the $+$ is still the last one not within the parentheses. The induction hypothesis shows that ‘ y ’ has only one leftmost derivation from S , and ‘ z ’ has only one leftmost derivation from T . Therefore, ‘ x ’ has only one leftmost derivation from S .

Let us consider the next case in which ‘ x ’ contains $+$ inside the parentheses but at least one $*$ out side the parentheses. This time ‘ x ’ can be derived only from S or T . Any derivation from S must begin with

$$S \Rightarrow T \Rightarrow T * F$$

and any derivation from T must begin with

$$T \Rightarrow T * F.$$

In either case, the $*$ must be the last one in ‘ x ’ that is not within the parentheses. As in first case, the subsequent steps of any leftmost derivation must be

$$T * F \Rightarrow y * F \Rightarrow y * z$$

Consisting of a leftmost derivation of ‘ y ’ from T , followed by a leftmost derivation ‘ z ’ from F . Again, the induction hypothesis says that there is only one possible way for these derivations to proceed, and hence, there is only one leftmost derivation of ‘ x ’ from S or T .

In the final case, let us assume ‘ x ’ contains no $+$ ’s or $*$ ’s out side the parentheses, then ‘ x ’ can be derived from any of the variables. However the only derivation from S begins with

$$S \Rightarrow T \Rightarrow F \Rightarrow (S)$$

and the only derivation from T or F begins the same way, by omitting first one or two steps.

Therefore, $x = (y)$, where, $S \Rightarrow y$. By induction hypothesis, ‘ y ’ has only one leftmost derivation from S , and it follows that ‘ x ’ has only one leftmost derivation from each of these.

Example 6.8

Consider an ambiguous context-free grammar G having the following productions.

$$\begin{aligned}
S &\rightarrow S + S \\
S &\rightarrow S * S \\
S &\rightarrow a \\
S &\rightarrow b
\end{aligned}$$

Construct an unambiguous context-free grammar G_1 equivalent to G .

Solution For the production of the form $S \rightarrow S + S$, we construct a new productions set by introducing a new variable (say T) as

$$S \rightarrow S + T \mid T,$$

For the production of the form $S \rightarrow S * S$, we construct a new productions set by introducing a new variable (say F) as

$$T \rightarrow T * F \mid F$$

The productions $S \rightarrow a$, $S \rightarrow b$ will remain the same because they do not show any ambiguity. But their left hand side will be replaced by F , as

$$F \rightarrow a, F \rightarrow b$$

Hence, the constructed unambiguous context-free grammar G_1 is

$$G_1 = (\{S, T, F\}, \{a, b, *, +\}, \{S \rightarrow S + T \mid T, T \rightarrow T * F \mid F, F \rightarrow a \mid b\}, S)$$

6.13

USELESS SYMBOLS IN CFG

There are several ways in which we can restrict the format of production rules in a context-free grammar without reducing the capabilities (power to generate language) of grammar. In a context-free grammar G , it may not be necessary to use all the variables and terminals, or all the productions in P for generating the strings. So when we study a context-free language $L(G)$, we always try to eliminate productions, terminals, and nonterminals in G , which are not useful for derivation of strings belonging to $L(G)$.

Let $G = (V_N, \Sigma, P, S)$ be a context-free grammar. A symbol ' A ' is useful if there is a derivation

$$S \Rightarrow^* \alpha A \beta \Rightarrow^* s$$

for some α, β and s , where $s \in \Sigma^*$. Otherwise ' A ' is useless. There are two aspects to usefulness. First some terminal string must be derivated from ' A ' and second ' A ' must occur in some string derivable from S (the start symbol). These two conditions are however not sufficient to guarantee that ' A ' is useful, since ' A ' may occur only in sentential form that contain a variable from which no terminal string can be derived.

Let us consider another case. If $L(G)$ is a nonempty context-free language (i.e., it generates at least one terminal string), then it can be generated by a CFG G with the following properties:

- (i) Each terminal and nonterminal must be involved in the derivation of some string in $L(G)$.
- (ii) There are no productions of the form $A \rightarrow B$ (called unit production) where $A, B \in V_N$.

If $\wedge \notin L(G)$, then there is no need of productions of the form $A \rightarrow \wedge$. Let us consider a context-free grammar

$$G = (\{S, A, B, C, D\}, \{0, 1, 2\}, P, S),$$

where P is defined as:

$$P = \{S \rightarrow AB, A \rightarrow 01, B \rightarrow C \mid 1, D \rightarrow 12 \mid \wedge\}$$

We take a string $s = 011$, such that $011 \in L(G)$.

Let us assume another grammar $G = (\{S, A, B, \}, \{0, 1\}, P_1, S)$, where P_1 is defined as:

$$P_1 = \{S \rightarrow AB, A \rightarrow 01, B \rightarrow 1\}.$$

We see that $L(G) = L(G_1)$. It means the language generated by both grammars G and G_1 are the same. From grammar G_1 , we have eliminated production $B \rightarrow C$ and $D \rightarrow 12 \mid \wedge$ which are useless productions in grammar G , on the following basis:

- (i) The variable C does not derive any terminal string.
- (ii) D and 12 are not involved in any sentential form.
- (iii) $B \rightarrow C$ simply replace B by C (unit production).
- (iv) $D \rightarrow \wedge$ is a null production.

Therefore for the simplification of CFG, it is necessary to eliminate the productions, variable and terminals having the following characteristics:

- (i) a production in which a variable does not derive a terminal string
- (ii) the symbols $(V_N \cup \Sigma)$, not appearing in any sentential form
- (iii) unit productions (e. g., $A \rightarrow B$)
- (iv) null productions (e. g., $A \rightarrow \wedge$).

6.13.1 Construction of Reduced CFG

Theorem 6.4 If G is a context grammar that generates non-empty language $L(G)$ (i.e. $L(G) \neq \phi$), then we can find an equivalent reduced CFG G_1 such that each variable in G_1 derives some terminal string.

Proof Suppose $G = (V_N, \Sigma, P, S)$ be the given context-free grammar. Let us construct $G_1 = (V'_N, \Sigma, P', S')$ as reduced context-free grammar by the following steps:

Step 1

Construction of V'_N We define $W_i \subseteq V_N$. It means that W_i is subset of V_N , which says that all element of W_i will definitely be in V_N . But the reverse must be true, it is not necessary. We define W_1 as

$$W_1 = \{A \in V_N \mid \text{there exists a production } A \rightarrow s \text{ where } s \in \Sigma^*\}$$

If $W_1 = \{\phi\}$, some variable will remain after the application of any production, and so $L(G) = \phi$.

Each W_{i+1} is constructed in terms of W_i as:

$$W_{i+1} = W_i \cup \{A \in V_N \mid \text{there exists some production where } A \rightarrow \beta \text{ such that } \beta \in (\Sigma \cup W_i)^*\}.$$

According to definition of W_i , $W_i \subseteq W_{i+1}$ for all i . It means that all the variables in W_{i+1} may also be in W_i but definitely all the variables in W_i must be in W_{i+1} .

As V_N has only finite number of variables, then there will be a condition that

$$W_k = W_{k+1} \text{ for some } k \leq |V_N|$$

where $|V_N|$ tells the length of V_N (or equivalently the number of variables in V_N)

Therefore, $W_k = W_{k+m}$ for $m \geq 1$. At this stage we define

$$V'_N = W_k$$

for newly constructed grammar G_1 .

Step 2

Construction of P' The set of production rules P' is defined as:

$$P' = \{A \rightarrow \alpha \mid A, \alpha \in (V'_N \cup \Sigma)^*\}$$

This means P' contains only those productions that involve the variables in V'_N . Hence we can define the reduced grammar as

$$G_1 = (V'_N, \Sigma, P', S)$$

where $S \in V'_N$. Note that the grammar G_1 is not a completely reduced grammar because we have not proved that all the variables and terminals in G' are fully involved in deriving sentential form. For the proof of derivation of sentential form we prove the following theorem:

Theorem 6.5 For every context grammar $G_1 = (V'_N, \Sigma, P, S)$, we can construct an equivalent grammar $G' = (V''_N, \Sigma'', P'', S)$ such that every symbol in $(V''_N \cup \Sigma'')$ appears in some sentential form.

Proof The theorem states that for every $Z \in (V''_N \cup \Sigma'')$ there exists β such that $S \xRightarrow{*}_G \beta$ and Z is a symbol in the string β . Note that the Σ'' is the reduced Σ .

We construct reduced grammar $G' = (V''_N, \Sigma'', P'', S)$ by the following steps:

Step 1

Construction of V''_N . We construct W_i for $i \geq 1$. First we define $W_1 = \{S\}$, then we determine W_{i+1} in terms of W_i for each $i \geq 1$. We define W_{i+1} as:

$W_{i+1} = W_i \cup \{Z \in (V'_N \cup \Sigma) \mid \text{there exists a production } A \rightarrow \beta \text{ with } A \in W_i \text{ and } \beta \text{ containing the symbol } Z\}$.

Note that $W_i \subseteq (V'_N \cup \Sigma)$ and $W_i \subseteq W_{i+1}$. As we have finite number of symbols (terminals and nonterminals) in V'_N , then $W_k = W_{k+1}$ for some k . This means that $W_k = W_{k+j}$ for all $j \geq 0$.

Step 2

Construction of V''_N, Σ'' and P'' . We define the following:

$$V''_N = V'_N \cap W_k$$

$$\Sigma'' = \Sigma \cap W_k$$

$$P'' = \{A \rightarrow \beta \mid A \in W_i\}$$

Every symbol Z in G' appears in some sentential form, say $\alpha Z \beta$. Therefore $S \xRightarrow{*}_G \alpha Z \beta \xRightarrow{*}_G s$ for some s , i.e., G' is reduced.

Example 6.9

Consider the context-free grammar $G = (V_N, \Sigma, P, S)$ with following productions.

$$S \rightarrow AB, A \rightarrow b, B \rightarrow a, B \rightarrow D, E \rightarrow a$$

(a) Find the grammar G_1 such that every variable in G_1 derives some terminal string.

(b) Find grammar G' from G_1 such that every symbol (nonterminals and terminals) appears in some sentential form.

Solution (a) Let $G_1 = (V'_N, \Sigma, P', S)$.

(i) **Construction of V'_N .**

$W_1 = \{A, B, E\}$ because these variables have terminal strings of length zero or more in R.H.S. of the productions.

$$W_2 = W_1 \cup \{A' \in V_N \mid A' \rightarrow \alpha \text{ for some } \alpha \in (\Sigma \cup W_1)^*\}$$

$$= \{A, B, E, S\}$$

$$W_3 = W_2 \cup \{A' \in V_N \mid A' \rightarrow \alpha \text{ for some } \alpha \in (\Sigma \cup W_2)^*\}$$

$$= W_2 \cup \phi$$

$$= W_2$$

Since $W_3 = W_2$, therefore

$$V'_N = \{S, A, B, E\}$$

(ii) **Construction of P'**

$$P' = \{A' \rightarrow \alpha \mid A', \alpha \in (V'_N \cup \Sigma)^*\}$$

$$= \{S \rightarrow AB, A \rightarrow b, B \rightarrow a, E \rightarrow a\}$$

Therefore,

$$G_1 = (\{S, A, B, E\}, \{a, b\}, P', S), \text{ where, } P' = \{S \rightarrow AB, A \rightarrow b, B \rightarrow a, E \rightarrow a\}$$

(b) As we have

$$G_1 = (\{S, A, B, E\}, \{a, b\}, P', S),$$

where, $P' = \{S \rightarrow AB, A \rightarrow b, B \rightarrow a, E \rightarrow a\}$. Let us construct $G' = (V''_N, \Sigma'', P'', S)$.

(i) **Construction of $V''_N \cup \Sigma''$ (denoted by W_k)**

$$W_1 = \{S\}. \quad \text{As } S \text{ is the start symbol.}$$

$$W_2 = W_1 \cup \{Z \in (V'_N \cup \Sigma) \mid \text{there exists a production } A \rightarrow \beta \text{ with } A \in W_1 \text{ and } \beta \text{ containing the symbol } Z\}.$$

$$= \{S\} \cup \{A, B\}$$

$$= \{S, A, B\}$$

$$W_3 = \{S, A, B\} \cup \{a, b\}$$

$$= \{S, A, B, a, b\}$$

$$W_4 = W_3 \cup \{\phi\}$$

Since, $W_4 = W_3$, so $W = \{S, A, B, a, b\}$

$$\text{Now, } V''_N = V'_N \cap W_k = \{S, A, B\}$$

$$\Sigma'' = \Sigma \cap W_k = \{a, b\}$$

$$P'' = \{A \rightarrow \beta \mid A \in W\} = \{S \rightarrow AB, A \rightarrow b, B \rightarrow a\}$$

Thus the required grammar is:

$$G' = (\{S, A, B\}, \{a, b\}, \{S \rightarrow AB, A \rightarrow b, B \rightarrow a\}, S)$$

Example 6.10

A context-free grammar G is given by following productions.

$$S \rightarrow aS, S \rightarrow AB, S \rightarrow cA, A \rightarrow a$$

find a grammar G' such that every variable in G' derive some terminal string and every symbol (nonterminals and terminals) appears in some sentential form.

Solution First of all we will construct grammar G_1 such that every variable in G_1 derives some terminal string.

Suppose the given grammar is $G = (V_N, \Sigma, P, S)$ and the constructed grammar G_1 is defined as

$$G_1 = (V'_N, \Sigma, P', S)$$

The set Σ and start symbol will remain the same. Let us construct V'_N and P' as follows:

(i) **Construction of V'_N**

$$W_1 = \{A\} \text{ because there is a production } A \rightarrow a$$

$$W_2 = W_1 \cup \{A' \in V_N \mid A' \rightarrow \alpha \text{ for some } \alpha \in (\Sigma \cup W_1)^*\}$$

$$= \{A\} \cup \{S\}$$

$$= \{A, S\}$$

$$W_3 = W_2 \cup \{A' \in V_N \mid A' \rightarrow \alpha \text{ for some } \alpha \in (\Sigma \cup W_2)^*\}$$

$$= \{A, S\} \cup \{S\}$$

$$= \{A, S\}$$

$$W_4 = W_3 \cup \{\phi\}$$

Since $W_4 = W_3$, therefore

$$V'_N = \{A, S\}$$

(ii) **Construction of P'**

$$P' = \{A' \rightarrow \alpha \mid A', \alpha \in (V'_N \cup \Sigma)^*\}$$

$$= \{S \rightarrow aS, S \rightarrow cA, A \rightarrow a\}$$

Therefore,

$$G_1 = (\{S, A\}, \{a, c\}, \{S \rightarrow aS, S \rightarrow cA, A \rightarrow a\}, S)$$

Now we will construct grammar G' such that every symbol (terminals and nonterminals) appears in some sentential form. The grammar G' is defined as

$$G' = (V''_N, \Sigma'', P'', S)$$

for which the input grammar is G_1 . If $x \in V''_N \cup \Sigma''$ is in sentential form from grammar G' then x is working string to derive reduced grammar. Now, at this moment we construct a set W_k to represent $(V''_N \cup \Sigma'')$ for some k .

(iii) **Construction of $V''_N \cup \Sigma''$**

$$W_1 = \{S\}. \text{ As } S \text{ is the start symbol.}$$

$$W_2 = W_1 \cup \{Z \in (V''_N \cup \Sigma) \mid \text{there exists a production } A \rightarrow \beta \text{ with } A \in W_1 \text{ and } \beta \text{ containing the symbol } Z\}.$$

$$= \{S\} \cup \{a, S, c, A\}$$

$$= \{S, A, a, c\}$$

$$W_3 = W_2 \cup \{Z \in (V''_N \cup \Sigma) \mid \text{there exists a production } A \rightarrow \beta \text{ with } A \in W_2 \text{ and } \beta \text{ containing the symbol } Z\}.$$

$$= \{S, A, a, c\} \cup \{a\}$$

$$= \{S, A, a, c\}$$

$$W_4 = W_3 \cup \{Z \in (V''_N \cup \Sigma) \mid \text{there exists a production } A \rightarrow \beta \text{ with } A \in W_3 \text{ and } \beta \text{ containing the symbol } Z\}.$$

Copyright © 2010, Tata McGraw-Hill. All rights reserved.

$$\begin{aligned}
 &= \{S, A, a, c\} \cup \{\phi\} \\
 &= \{S, A, a, c\}
 \end{aligned}$$

Since, no new element is added to W_k , as we got $W_4 = W_3$, therefore

$$W = W_4 = \{S, A, a, c\}$$

Now we will calculate the element in V''_N and Σ'' as

$$V''_N = V'_N \cap W_k = \{S, A\}$$

$$\Sigma'' = \Sigma \cap W_k = \{a, c\}$$

P'' is defined as

$$P'' = \{A \rightarrow \beta \mid A \in W\} = \{S \rightarrow aS, S \rightarrow cA, A \rightarrow a\}$$

Thus, the required grammar G' is

$$G' = (\{S, A\}, \{a, c\}, \{S \rightarrow aS, S \rightarrow cA, A \rightarrow a\}, S)$$

6.14

ELIMINATION OF NULL AND UNIT PRODUCTIONS

There are some straight forward ways to improve a context-free grammar without changing the resulting language. In this regard, first of all certain types of productions are eliminated that may be difficult to handle and to work with. The second step in the continuation is standardisation of productions so that they all have a certain normal form.

We start by trying to eliminate null productions (also called $\wedge$ -productions) of the form $A \rightarrow \wedge$, and the unit productions of the form $B \rightarrow C$, in which one variable is simply replaced by another. To illustrate how this elimination might be useful let us suppose a grammar that contains any type of productions and consider a derivation containing the step

$$\beta \Rightarrow \gamma$$

Suppose if there is no null production, then the string γ must be at least as long as β ; and suppose if there is no unit production. β and γ can be of equal length only if this step consists of replacement of a variable by a single terminal.

The idea to eliminate unit productions is based on chaining. If there are productions $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow d$, then these production can simply be replaced by $A \rightarrow d$.

Theorem 6.6 If $G = (V_N, \Sigma, P, S)$ is a context-free grammar, then we can find a context-free grammar G' having no null productions such that

$$L(G') = L(G) - \{\wedge\}.$$

Proof The difference between grammars G and G' is of only productions, so we define G' as (V_N, Σ, P, S) . The construction steps are as follows:

Step 1 Construction of the set of nullable variables We find nullable variables recursively. A variable E in a CFG is nullable if $E \Rightarrow^* \wedge$. Nullable variable are found as:

- (i) $W_1 = \{A' \in V_N \mid A' \rightarrow \wedge \text{ is in } P\}$
- (ii) $W_{i+1} = W_i \cup \{A' \in V_N \mid \text{there exists a production } A' \rightarrow \beta \text{ with } \beta \in W_i\}$

According to the definition of W_i , $W_i \subseteq W_{i+1}$ for all i , W_i is subset of W_{i+1} , which says that all element of W_i will definitely be in W_{i+1} but it is not necessary that all elements of W_{i+1} must be in W_i .

As V_N has finite number of variables therefore, $W_{k+1} = W_k$ for some $k \leq |V_N|$, where $|V_N|$ is the number of element in V_N . So $W_{k+j} = W_k$ for all j . Let $W = W_k$, where W is the set of all nullable variables.

Now we are going to remove all those productions that contain nullable variables.

Step 2 (i) Construction of P' We will include all productions in P' , which do not have any nullable variable in the right hand side of the production.

(ii) If $A' \rightarrow X_1 X_2 X_3 \dots X_m$ is in P , the production of the $A' \rightarrow \beta_1 \beta_2 \beta_3 \dots \beta_m$ are include in P' , where $\beta_i = X_i$ if $X_i \notin W$. $\beta_i = X_i \cup \wedge$ if $X_i \in W$ and $\beta_1 \beta_2 \beta_3 \dots \beta_m \neq \wedge$. Actually the production $A' \rightarrow X_1 X_2 X_3 \dots X_m$ gives several productions in P' . The productions are obtained either by not removing any nullable variable on the right hand side of $A' \rightarrow X_1 X_2 X_3 \dots X_m$ or by removing some or all nullable variable provided some symbol appeared on the right hand side after removing.

Note In the above theorem A' is not a complement of A . We have to assume A' as a random variable such that $A' \in V_N$.

Example 6.11

A context-free grammar G is given by the following productions.

$$S \rightarrow AB, A \rightarrow \wedge, B \rightarrow \wedge, A \rightarrow a, B \rightarrow b.$$

Construct a grammar G' having no null production.

Solution Let $G' = (V'_N, \Sigma, P', S)$ be the constructed grammar such that there is no null production in set P' . First of all construct a set W of all nullable variables.

Step 1 Construction of W

$$W_1 = \{A' \in V_N \mid A' \rightarrow \wedge \text{ is in } P\}$$

$$= \{A, B\}$$

$$W_2 = W_1 \cup \{A' \in V_N \mid \text{there exists a production } A' \rightarrow \beta \text{ with } \beta \in W_1^*\}$$

$$= \{A, B\} \cup \{S\}$$

$$= \{S, A, B\}$$

$$W_3 = W_2 \cup \{A' \in V_N \mid \text{there exists a production } A' \rightarrow \beta \text{ with } \beta \in W_2^*\}$$

$$= \{S, A, B\} \cup \{\phi\}$$

$$= \{S, A, B\}$$

Thus,

$$W = \{S, A, B\}$$

Now we construct P' (the new set of productions).

Step 2 Construction of P'

(i) $A \rightarrow a$, and $B \rightarrow b$ are in required form, therefore $\{A \rightarrow a, B \rightarrow b\} \in P'$

(ii) $S \rightarrow AB$, gives $S \rightarrow A$ (from $S \rightarrow A\wedge$), $S \rightarrow B$ (from $S \rightarrow \wedge B$) and $S \rightarrow AB$, therefore, $\{S \rightarrow A, S \rightarrow B, S \rightarrow AB\} \in P'$

Thus,

$$P' = \{S \rightarrow AB, S \rightarrow A, S \rightarrow B, A \rightarrow a, B \rightarrow b\}$$

Hence, the grammar equivalent to G having no null production is given by

$$G' = (\{S, A, B\}, \{a\}, \{S \rightarrow AB, S \rightarrow A, S \rightarrow B, A \rightarrow a, B \rightarrow b\}, S)$$

Example 6.12

Show that a context-free grammar G given by following productions.

$$S \rightarrow aS, S \rightarrow AB, A \rightarrow a, B \rightarrow b$$

is a grammar having no null production.

Solution Suppose the given grammar is $G = (V_N, \Sigma, P, S)$. Let us assume the grammar G has some null production. Let $G' = (V'_N, \Sigma, P', S)$ be the grammar such that there is no null production in set P' . First of all we construct a set W of all nullable variables as

$$W_1 = \{A' \in V_N \mid A' \rightarrow \wedge \text{ is a production in } G\}$$

$$W_1 = \{\phi\}$$

$$W_2 = W_1 \cup \{B' \in V_N \mid \text{there is a production } B' \rightarrow \alpha \text{ where } \alpha \in W_1^*\}$$

$$W_2 = \{\phi\} \cup \{\phi\}.$$

Thus

$$W = \{\phi\}$$

Therefore, we can say that there is no null production in grammar G , since the set of nullable variables has no element.

Theorem 6.7 Let G be a context-free grammar with no null production, we can find a context-free grammar G' which has no unit production such that $L(G') = L(G)$.

Proof The productions of the form $D \rightarrow B$ are called unit productions, where, $D, B \in V_N$. Let A be any variable in V_N .

Step 1 First we construct the set of variable derivable from A . We define $W_i(A)$ recursively as follows:

$$W_0(A) = \{A\}$$

$$W_{i+1}(A) = W_i(A) \cup \{B \in V_N \mid C \rightarrow B \text{ is in } P \text{ with } C \in W_i(A)\}.$$

It means W_{i+1} is calculated in terms of W_i . By definition of $W_i(A)$, the $W_i(A)$ is subset of $W_{i+1}(A)$. As the number of variables in V_N are finite, then $W_{k+1}(A) = W_k(A)$ for some $k \leq |V_N|$ where $|V_N|$ tells the number of variables in V_N . Therefore, $W_{k+1}(A)$ will be equal to $W_k(A)$ for all $j \geq 0$. Here we introduce a new set $W(A)$, defined as

$$W(A) = W_k(A)$$

where, $W(A)$ is the set of variable derivable from A .

Step 2 Construction of A -productions in G' . The A -productions in G' are of the form

$$A \rightarrow \alpha \text{ whenever } B \rightarrow \alpha \text{ is in } G \text{ with } B \in W(A) \text{ and } \alpha \notin V_N$$

Now we define $G' = (V'_N, \Sigma, P', S)$ where P' is constructed using step 2 for every $A \in V_N$ and V'_N is set of variables in P'

Example 6.13

Consider a context-free grammar G given by following productions.

$$S \rightarrow XY, X \rightarrow a, Y \rightarrow Z \mid b, Z \rightarrow A, A \rightarrow B, B \rightarrow c$$

Find a context-free grammar G' equivalent to G by eliminating unit productions

Solution

Step 1 $W_0(S) = \{S\}$, and $W_1(S) = W_0(S) \cup \{\phi\}$

Hence, $W(S) = \{S\}$

Similarly, $W(X) = \{X\}$, and $W(B) = \{B\}$

Now, $W_0(Y) = \{Y\}$,

$$W_1(Y) = W_0(Y) \cup \{Z\}$$

$$= \{Y\} \cup \{Z\}$$

$$= \{Y, Z\}$$

$$W_2(Y) = W_1(Y) \cup \{A\}$$

$$= \{Y, Z\} \cup \{A\}$$

$$= \{Y, Z, A\}$$

$$W_3(Y) = W_2(Y) \cup \{B\}$$

$$= \{Y, Z, A\} \cup \{B\}$$

$$= \{Y, Z, A, B\}$$

$$W_4(Y) = W_3(Y) \cup \{\phi\}$$

$$= \{Y, Z, A, B\}$$

Hence, $W(Y) = \{Y, Z, A, B\}$

Similarly, $W_0(Z) = \{Z\}$,

$$W_1(Z) = W_0(Z) \cup \{A\}$$

$$= \{Z, A\}$$

$$W_2(Z) = W_1(Z) \cup \{B\}$$

$$= \{Z, A\} \cup \{B\}$$

$$= \{Z, A, B\}$$

$$W_3(Z) = W_2(Z) \cup \{\phi\}$$

$$= W_2(Z)$$

$$= \{Z, A, B\}$$

Hence, $W(Z) = \{Z, A, B\}$

Similarly, $W_0(A) = \{A\}$,

$$W_1(A) = W_0(A) \cup \{B\}$$

$$= \{A\} \cup \{B\}$$

$$= \{A, B\}$$

$$W_2(A) = W_1(A) \cup \{\phi\}$$

$$= W_1(A)$$

$$= \{A, B\}$$

Hence, $W(A) = \{A, B\}$

Step 2 The productions in G' are

$$S \rightarrow XY, X \rightarrow a, Y \rightarrow c \mid b, Z \rightarrow c, A \rightarrow c, B \rightarrow c$$

Thus, the grammar G' is $(\{S, X, Y, Z, A, B\}, \{a, b, c\}, P', S)$, where P' is given by step 2 (see above).

Theorem 6.8 There exists an algorithm to decide whether $\wedge \in L(G)$ for a given context-free grammar G .

Proof $\wedge$ is in $L(G)$ if and only if S (the start symbol) is nullable. The construction given by Theorem 6.7 terminates in a finite number of steps. Therefore we can prove this theorem by using the two following steps:

Step 1 First we construct a set of nullable variable (say W).

Step 2 In this step we test whether $S \in W$.

Hence, a context-free grammar that does not generate the string $\wedge$ can be written without $\wedge$ -productions.

6.15

CHOMSKY AND GREIBACH NORMAL FORM

6.15.1 Chomsky Normal Form (CNF)

In Chomsky normal form, we restrict the length of V_N and the nature of symbols in the right hand side of productions. A context-free grammar G is said to be in Chomsky normal form if every production is of the form either $A \rightarrow a$, (exactly one terminal in the right hand side of the production) or $A \rightarrow BC$ (exactly two variables in the right hand side of the production). For example, the context-free grammar G with productions $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$ is in Chomsky normal form.

$S \rightarrow \wedge$ is also in CNF if $S \rightarrow \wedge$ is in G such that $\wedge \in L(G)$ and S does not appear in the right hand side of any production. In all other cases it is must to eliminate null productions before reducing it into CNF.

Note Several text book authors do not allow $S \rightarrow \wedge$ in Chomsky normal form.

❖ Did you Know ?

The Chomsky normal form is named after Noam Chomsky. It is a normal form of context free grammars in which every production is of the form

$A \rightarrow AB$ or $A \rightarrow a$.

Thus, for a context-free grammar to be in CNF, the derivation tree has the property that every node has at most two descendents either two internal nodes or a single leaf node. It is sure that an internal node always has label of any symbol from V_N , and a leaf node always has label of any symbol from $(\Sigma \cup \wedge)$.

Reduction to CNF Let us consider a context-free grammar having productions $S \rightarrow AB \mid aC$, $A \rightarrow b$, $B \rightarrow aA$ and $C \rightarrow ABS$. Except $S \rightarrow aC$, $B \rightarrow aA$, $C \rightarrow ABS$, all the other productions are in the form required to be in CNF. For the production of the form $S \rightarrow aC$, we introduce new variable C_a such that $S \rightarrow aC$ can be replaced by

$$S \rightarrow C_a C, C_a \rightarrow a$$

which has the same derivation as $S \rightarrow aC$ has, but the productions $S \rightarrow C_a C$, $C_a \rightarrow a$ are in the form required for CNF. Similarly $B \rightarrow aA$ can be replaced by

$$B \rightarrow C_a A$$

and there is no need to write $C_a \rightarrow a$ again.

For the production of the form $C \rightarrow ABS$ we introduce another new variable (say D) such that $C \rightarrow ABS$ can be replaced by

$$C \rightarrow AD, D \rightarrow BS$$

which are in required form. For long work strings, some more new variables can be introduced to put the production into CNF.

A context-free grammar in Chomsky normal form is very useful to determine whether or not it can generate any word at all. This is the decidability concept of emptiness in CFGs. Also, Chomsky normal form is particularly useful for programmes that have to manipulate grammars.

Theorem 6.9 For every context-free grammar there is an equivalent grammar in Chomsky Normal Form (CNF).

Proof Let $G = (V_N, \Sigma, P, S)$ be a CFG. A context-free grammar G is converted into Chomsky Normal Form (CNF) by using the following steps:

Step 1 Eliminate null and unit production by applying Theorem 6.6 and Theorem 6.7 respectively. Let the modified grammar thus obtained be

$$G_1 = (V'_N, \Sigma', P', S)$$

Step 2 Elimination of terminals from right-hand side We define grammar $G_2 = (V''_N, \Sigma', P'', S)$ where P'' and V''_N are constructed as follows:

- (i) All the productions in P' of the form $A \rightarrow b$ or $B \rightarrow CD$ are included in P'' , and all the variables in V'_N are included in V''_N .
- (ii) If there is a production of the form $X \rightarrow A_1 A_2 A_3 \dots A_n$ with some terminals on right hand side, say A_i is terminal with value a_i , add a new variable C_{a_i} to V''_N and add a production $C_{a_i} \rightarrow a_i$ to P'' .

Every terminal in right hand side of the production $X \rightarrow A_1 A_2 A_3 \dots A_n$ is replaced by the corresponding new variable and the variables on right hand side are retained. The resulting production is included to P'' . This way we get $G_2 = (V''_N, \Sigma', P'', S)$.

Step 3 Restricting the number of variables on right hand side. Every production in P'' must contain either a single terminal (or $\wedge$ in case of $S \rightarrow \wedge$) or two more variables on the right hand side.

We define grammar in CNF $G' = (V'''_N, \Sigma', P''', S)$ as follows:

- (i) All productions in P'' are added to P''' if they are in required form and, all the variables in V''_N are added to V'''_N .
- (ii) If there are productions of the form $A \rightarrow A_1 A_2 A_3 \dots A_n$, where $n \geq 3$, we introduce new productions $A \rightarrow A_1 C_1$, $C_1 \rightarrow A_2 C_2$, $C_2 \rightarrow A_3 C_3$, $C_3 \rightarrow A_4 C_4$, $C_4 \rightarrow \dots$, $C_{n-2} \rightarrow A_{n-1} A_n$ and new variables $C_1, C_2, C_3, \dots, C_{n-2}$. These productions are added to P''' and new variables introduced in these productions (e.g., $C_1, C_2, \dots, C_{n-2}$) are added to V'''_N . Thus, the grammar $G' = (V'''_N, \Sigma', P''', S)$ is in CNF.

Example 6.14

Reduce the context-free grammar G into Chomsky normal form given by the following productions

$$\begin{aligned} S &\rightarrow aAC \\ A &\rightarrow aB \mid bAB \\ B &\rightarrow b \\ C &\rightarrow c. \end{aligned}$$

Solution

Step 1 First we have to eliminate all null and unit productions. But there is no such production. At the end of this step we have grammar

$$G_1 = (V'_N, \Sigma', P', S)$$

where

$$\begin{aligned} V'_N &= \{S, A, B, C\} \\ \Sigma' &= \{a, b, c\} \\ P' &= \{S \rightarrow aAC, A \rightarrow aB \mid bAB, B \rightarrow b, C \rightarrow c\} \\ S & \text{ is the start symbol, and } S \in V'_N \end{aligned}$$

Step 2 Elimination of terminals from right hand side

- (i) $B \rightarrow b, C \rightarrow c$ are in required form, therefore these productions are included in P'' .
- (ii) $S \rightarrow aAC$ can be converted into $S \rightarrow C_aAC$ and $C_a \rightarrow a$, and $A \rightarrow aB$ can be converted into $A \rightarrow C_aB$. (There is no need to write $C_a \rightarrow a$ again). $A \rightarrow bAB$ can be converted into $A \rightarrow C_bAB$ and $C_b \rightarrow b$.

Therefore,

$$\begin{aligned} V''_N &= \{S, A, B, C, C_a, C_b\} \\ P'' &= \{B \rightarrow b, C \rightarrow c, S \rightarrow C_aAC, C_a \rightarrow a, A \rightarrow C_aB, A \rightarrow C_bAB, C_b \rightarrow b\} \end{aligned}$$

At the end of this step the grammar G_2 is (V''_N, Σ, P'', S) .

Step 3 Restricting the number of variables on right hand side P'' consists of productions

$$S \rightarrow C_aAC, A \rightarrow C_bAB, A \rightarrow C_aB, B \rightarrow b, C \rightarrow c, C_a \rightarrow a, C_b \rightarrow b$$

- (i) The productions $A \rightarrow C_aB, B \rightarrow b, C \rightarrow c, C_a \rightarrow a$, and $C_b \rightarrow b$ are in required form. Therefore these productions are added to P''' . The variables A, C_a, B, C and C_b are added to V'''_N .
- (ii) $S \rightarrow C_aAC$ can be replaced by $S \rightarrow C_aC_1$ and $C_1 \rightarrow AC$. These productions are added to P''' because these are in required form.
- (iii) $A \rightarrow C_bAB$ can be replaced by $A \rightarrow C_bC_2$ and $C_2 \rightarrow AB$. These production are added to P''' because these are in required form.

At the end of this step we have context-free grammar $G' = (V'''_N, \Sigma', P''', S)$, where,

$$\begin{aligned} V'''_N &= \{S, A, B, C, C_a, C_b, C_1, C_2\} \\ \Sigma' &= \{a, b, c\} \\ P''' &= \{S \rightarrow C_aC_1, C_1 \rightarrow AC, A \rightarrow C_bC_2, C_2 \rightarrow AB, A \rightarrow C_aB, B \rightarrow b, C \rightarrow c, C_a \rightarrow a, C_b \rightarrow b\} \end{aligned}$$

The grammar G is in Chomsky normal form and equivalent to G .

6.15.2 Greibach Normal Form (GNF)

A context-free grammar is said to be in Greibach normal form if every production is of the form

$$A \rightarrow a\alpha$$

where $a \in \Sigma$, $A \in V_N$ and $\alpha \in V_N^*$. It shows that α is a string of variables having zero or more length.

❖ Did you Know ?

The Greibach normal form is named after Sheila Greibach. It is a normal form of context free grammars in which every production is of the form

$$A \rightarrow aB \text{ or } A \rightarrow a.$$

Sheila Greibach is well known for her contribution to formal languages. In addition to establishing the normal form (Greibach Normal Form) for context-free grammars, she has also been involved in investigation of properties of W-grammars, push down automata (PDA), and decidability problems. She has also contributed along with Seymour Ginsburg and Michael Harrison to context-sensitive parsing using the stack model of automata.

Grammars in Greibach normal form are typically complex and much longer than the context-free grammars from which they were derived. The Greibach normal form is useful for proving the equivalence of context-free grammar (CFGs) and Nondeterministic Push Down Automata (NPDA). When we convert a CFG into an NPDA, or vice versa, we use GNF.

A context-free grammars can be reduced to Greibach normal form (GNF) by using the following two Lemmas:

Lemma 6.1 If $G = (V_N, \Sigma, P, S)$ is a given context-free grammar, and an A -production $A \rightarrow B\gamma$ is in P . If B -productions are

$$B \rightarrow \beta_1 | \beta_2 | \beta_3 | \beta_4 | \beta_5 | \beta_6 | \dots | \beta_k.$$

If we define a new set of productions P_1 as

$$P_1 = (P - \{A \rightarrow B\gamma\}) \cup \{A \rightarrow B_i \gamma | 1 \leq i \leq k\}$$

(i.e., productions of the form $A \rightarrow B\gamma$ are eliminated by introducing the productions of the form $A \rightarrow B_i \gamma$). Then grammar $G_1 = (V_N, \Sigma, P_1, S)$ is equivalent to given context-free grammar G .

Proof If we apply $A \rightarrow B\gamma$ in some derivation $s \in L(G)$, we have to eliminate B from grammar G which is the same as applying $A \rightarrow B_i \gamma$ for some i in grammar G_1 .

Hence,

$$s \in L(G_1), \text{ i.e., } L(G_1) \subseteq L(G)$$

Similarly, instead of applying $A \rightarrow B_i \gamma$, we can also apply $A \rightarrow B\gamma$ and $B \rightarrow \beta_i$ to get

$$A \xRightarrow[G]{*} B_i \gamma$$

This shows that

$$L(G_1) \subseteq L(G)$$

This way, we can delete a variable B appearing as the first symbol on the right side of some A -productions. It provides no B -production which has B as the first symbol on right side.

In the construction, given by Lemma 6.1, B is eliminated from the production $A \rightarrow B\gamma$ by simply replacing B by the right side of every B -production.

Therefore the Lemma 6.1 is useful to eliminate A from the right side of $B \rightarrow AB$.

Lemma 6.2 If $G = (V_N, \Sigma, P, S)$ is a context-free grammar, and A -productions are

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid A\alpha_4 \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \beta_3 \mid \dots \mid \beta_n$$

where β_i 's do not start with A . Suppose Z is a new variable ($Z \notin V_N$). Suppose a context-free grammar G_1 is defined as

$$G_1 = (\{V_N \cup Z\}, \Sigma, P_1, S)$$

where P_1 is defined as follows:

(i) The set of A -productions in P_1 are

$$A \rightarrow \beta_1 \mid \beta_2 \mid \beta_3 \mid \dots \mid \beta_n \text{ and } A \rightarrow \beta_1 Z \mid \beta_2 Z \mid \beta_3 Z \mid \dots \mid \beta_n Z.$$

(ii) The set of Z -productions in P_1 are

$$Z \rightarrow \alpha_1 \mid \alpha_2 \mid \alpha_3 \mid \alpha_4 \dots \mid \alpha_m \text{ and } Z \rightarrow \alpha_1 Z \mid \alpha_2 Z \mid \alpha_3 Z \mid \alpha_4 Z \dots \mid \alpha_m Z$$

(iii) The productions for the other variables are the same as in P . Then, context-free grammar G_1 is equivalent to G .

Proof Let us consider a leftmost derivation $s \in L(G)$ for the proof of $L(G) \subseteq L(G_1)$. The only productions in $P - P_1$ are

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid A\alpha_4 \dots \mid A\alpha_m$$

If $A \rightarrow A\alpha_{i_1} \mid A\alpha_{i_2} \mid \dots \mid A\alpha_{i_k}$ are used, then $A \rightarrow \beta_j$ should be used at a later stage to eliminate A . So we have derivation

$$A \xRightarrow[G]{*} B_j A\alpha_{i_1} \alpha_{i_1} \dots \alpha_{i_k}$$

While deriving $s \in L(G)$. However, by using CFG G_1 we have

$$A \xRightarrow[G_1]{} B_j Z \xRightarrow[G_1]{} \beta_j \alpha_{i_1} Z \dots$$

$$\xRightarrow[G_1]{} B_j \alpha_{i_1} Z \alpha_{i_2} Z \dots \alpha_{i_k}$$

$$\xRightarrow[G_1]{} B_j \alpha_{i_1} \alpha_{i_2} \alpha_{i_3} \dots \alpha_{i_k}$$

$$\text{i.e., } A \xRightarrow[G_1]{} B_j A\alpha_{i_1} \alpha_{i_2} \alpha_{i_3} \dots \alpha_{i_k}$$

This way, ' A ' can be eliminated by using the productions in grammar G_1 . Therefore, $s \in L(G_1)$.

For the proof of $L(G_1) \subseteq L(G)$, let us consider a leftmost derivation of $s \in L(G_1)$.

The only productions in $P_1 - P$ are

$$A \rightarrow \beta_1 Z \mid \beta_2 Z \mid \beta_3 Z \mid \dots \mid \beta_n Z, \quad Z \rightarrow \alpha_1 Z \mid \alpha_2 Z \mid \alpha_3 Z \mid \alpha_4 Z \dots \mid \alpha_m Z \quad \text{and}$$

$$Z \rightarrow \alpha_1 \mid \alpha_2 \mid \alpha_3 \mid \alpha_4 \mid \dots \mid \alpha_m$$

If the new introduced variable Z appears in the derivation of s , is due to the application of the production $A \rightarrow \beta_j Z$ in some starting step. Also, Z can be eliminated only by a production of the form $Z \rightarrow \alpha_i \mid \alpha_j$ for some i and j in later steps. Therefore, we get

$$A \xRightarrow{G_1}^* \beta_j \alpha_{i_1} \alpha_{i_2} \alpha_{i_3} \dots \alpha_{i_k}$$

in the derivation of s . Also, we know that $A \xRightarrow{G_1}^* \beta_j \alpha_{i_1} \alpha_{i_2} \alpha_{i_3} \dots \alpha_{i_k}$. Therefore, $s \in L(G)$, which shows that $L(G_1) \subseteq L(G)$.

Theorem 6.10 Every context-free language L can be generated by a context-free grammar G in Greibach normal form. (i.e., a context-free grammar can be reduced to Greibach normal form).

Proof We prove the theorem when $\wedge$ is not in L (i.e., $\wedge \notin L$) and then we extend the construction to $\wedge \in L$. Thus there are two cases first when $\wedge \notin L$ and second when $\wedge \in L$.

Case (i) Construction of G (when $\wedge \notin L$)

Step 1 We eliminate null productions and then construct a grammar G in Chomsky Normal form which generates language L . We rename the variables as $A_1, A_2, A_3, \dots, A_n$ with $S = A_1$. Therefore, grammar G is defined as

$$\begin{aligned} G &= (V_N, \Sigma, P, S) \\ V_N &= \{A_1, A_2, A_3, \dots, A_n\} \\ \text{with } S &= A_1 \end{aligned}$$

Step 2 To obtain the productions in the form $A_i \rightarrow a\gamma$ or $A_i \rightarrow a_j\gamma$, where $i < j$, we convert the A_i -productions (for $i = 1, 2, 3, \dots, n-1$) in the form $A_i \rightarrow a_j\gamma$, such that $i < j$. The proof of such modification can be done by induction on i .

Let us consider A_1 -productions. If some A_1 -productions are of the form $A_1 \rightarrow A_1\gamma$, then we apply Lemma 6.2 to eliminate such productions. By introducing a new variable (say Z_1), we get A_1 -productions in the form $A_1 \rightarrow a$ or $A_1 \rightarrow A_j\gamma'$, where j is greater than 1, which is the basis for induction.

This way we can modify A_1 -productions, A_2 -productions, A_3 -productions, ..., A_i -productions. Let us consider A_{i+1} productions, There is no need of any modification in productions of the form $A_{i+1} \rightarrow a\gamma$. Now we consider the first symbol in the right side of remaining A_{i+1} -productions. Suppose k is the smallest index among the indices of such variables. If $k > i + 1$, then there is no need to prove anything. Otherwise we apply induction hypothesis to A_k -productions for $k \leq i$. Therefore, any A_k -production is of the form $A_k \rightarrow A_j\gamma$ where $j > k$ or $A_k \rightarrow a\gamma'$. Now we can apply Lemma 6.1 to A_{i+1} -productions whose right side starts with A_k , then the modified A_{i+1} productions are of the form

$$A_{i+1} \rightarrow A_j\gamma$$

where $j > k$ or $A_{i+1} \rightarrow a\gamma'$

We repeat the above construction by finding k for the new set of A_{i+1} -productions. Finally, the A_{i+1} -production are converted to the form $A_{i+1} \rightarrow A_j\gamma$, where $j \geq i + 1$ or $A_{i+1} \rightarrow a\gamma'$. The productions of the form $A_{i+1} \rightarrow A_{i+1}\gamma$ can be modified by applying Lemma 6.2. This way, we have converted A_{i+1} -productions in the required form. Any A_i -production is of the form $A_i \rightarrow A_j\gamma$, where $i < j$ or $A_i \rightarrow a\gamma'$ and A_n -production is of the form $A_n \rightarrow A_n\gamma$ or $A_n \rightarrow a\gamma'$ at the end of this step.

Step 3 We convert A_n -production to the form $A_n \rightarrow a\gamma$. The productions obtained in step 2 of the form $A_n \rightarrow A_n \gamma$ are eliminated by applying Lemma 6.2. As a result the A_n -production are of the form $A_n \rightarrow a\gamma$.

Step 4 Now we modify A_i -productions to the form $A_i \rightarrow a\gamma$ (for $i = 1, 2, 3, \dots, n-1$). At the end of step 3, the A_n -productions are of the form $A_n \rightarrow a\gamma$, the A_{n-1} productions are of the form $A_{n-1} \rightarrow a\gamma'$ or $A_{n-1} \rightarrow A_n \gamma$. We eliminate the productions of the form $A_{n-1} \rightarrow A_n \gamma$ by applying Lemma 6.1. The resulting A_{n-1} -productions are in the required form. We repeat the similar constructions for $A_{n-2}, A_{n-3}, \dots$ productions.

Step 5 In this step we modify Z_i -productions. We apply Lemma 6.2 every time, and we get a new variable. As a result Z_i -productions are of the form $Z_i \rightarrow \alpha Z_i$ or $Z_i \rightarrow \alpha$, where α is obtained from $A_i \rightarrow A_i \alpha$. Thus the Z_i -productions are of the form $Z_i \rightarrow a\gamma$ or $Z_i \rightarrow A_k \gamma$ for some k . At the end of step 4, the right side of any A_k -production was starting with a terminal, which we are going to change in this step. Therefore we can apply Lemma 6.1 to eliminate the productions of the form $Z_i \rightarrow A_k \gamma$. Thus, the resultant grammar G_1 obtained at the end of this step (step 5) is in GNF.

Case (ii) Construction of G (when $\wedge \in L$) We repeat the construction as in case (i), to get a grammar $G' = (V'_N, \Sigma, P', S)$ in Greibach normal form such that $L(G') = L - \{\wedge\}$. We define a new grammar G_1 as

$$G_1 = (V'_N \cup \{S\}, \Sigma, P' \cup \{S' \rightarrow S \mid \wedge\}, S).$$

The production $S' \rightarrow S$ can be eliminated by using theorem to eliminate the unit productions. Thus all S -and S' productions are in required form and G_1 is in Greibach normal form.

Thus we can apply steps 2 to 5 to a context-free grammar, whose productions are of the form $A \rightarrow A\alpha$ or $A \rightarrow \alpha$, where $\alpha \in V_N^*$ and $|\alpha| \geq 2$ (the length of α is 2 or more), to reduce it into GNF, by first converting the productions of the form $A \rightarrow \alpha$ (where $\alpha \in V_N^*$ and $|\alpha| \geq 2$) into Chomsky normal form (CNF).

Example 6.15

Reduce the context-free grammar into Greibach normal form, whose productions are

$$S \rightarrow XX, S \rightarrow a, X \rightarrow SS, X \rightarrow b.$$

Solution As the given grammar is in Chomsky normal form so there is no need to convert it into Chomsky normal form. We rename variables S and X as A_1 and A_2 respectively. Therefore, the productions of grammar are

$$A_1 \rightarrow A_2 A_2, A_1 \rightarrow a, A_2 \rightarrow A_1 A_1, A_2 \rightarrow b$$

As there is no null production so we do not need to follow Step 1. Now we proceed to Step 2.

Step 2 A_1 -productions are in required form, they are $A_1 \rightarrow A_2 A_2 \mid a$ (i.e., the form $A_i \rightarrow A_j \gamma$, where $i < j$ and $A_j \rightarrow a\gamma$). The production $A_2 \rightarrow b$ is also in required form ($A_2 \rightarrow b$ is of the form $A_i \rightarrow a\gamma$, here $\gamma = \wedge$).

The production $A_2 \rightarrow A_1 A_1$ is not in required form. We apply Lemma 6.1 to the production $A_2 \rightarrow A_1 A_1$. The resulting A_2 -production are

$$A_2 \rightarrow A_2 A_2 A_1, A_2 \rightarrow a A_1, A_2 \rightarrow b$$

Step 3 The A_2 -production $A_2 \rightarrow A_2 A_2 A_1$ is not in required form. We apply Lemma 6.2 to the production $A_2 \rightarrow A_2 A_2 A_1$. Suppose Z_2 is the newly introduced variable. The resulting productions are

$$A_2 \rightarrow aA_1, A_2 \rightarrow b, A_2 \rightarrow aA_1Z_2, A_2 \rightarrow bZ_2, \\ Z_2 \rightarrow A_2A_1, Z_2 \rightarrow A_2A_1Z_2$$

Step 4 Among the A_1 -productions we eliminate production $A_1 \rightarrow A_2A_2$ by applying Lemma 6.1. The resulting productions are

$$A_1 \rightarrow aA_2A_2, A_1 \rightarrow bA_2, A_1 \rightarrow aA_2Z_2A_2, A_1 \rightarrow bZ_2A_2$$

After modification, the set of A_1 -productions includes the following productions

$$A_1 \rightarrow a, A_1 \rightarrow aA_2A_2, A_1 \rightarrow bA_2, A_1 \rightarrow aA_2Z_2A_2, A_1 \rightarrow bZ_2A_2$$

Step 5 By applying Lemma 6.1 we modify Z_2 -productions which are not in required form. We get

$$Z_2 \rightarrow aA_1A_1 \mid bA_1 \mid aA_1Z_2A_1 \mid bZ_2A_1 \\ Z_2 \rightarrow aA_1Z_2A_1 \mid aA_1Z_2A_1Z_2 \mid bA_1Z_2 \mid bZ_2A_1Z_2$$

Now all the productions are in required form (i.e., $A \rightarrow a\alpha$, where $a \in \Sigma$ and $\alpha \in V_N^*$). Thus, the given context-free grammar after converting in Greibach normal form is given by

$$G' = (\{A_1, A_2, Z_2\}, \{a, b\}, P', A_1)$$

where A_1 is the start symbol and P' is given by following productions

$$A_1 \rightarrow a, A_1 \rightarrow aA_2A_2, A_1 \rightarrow bA_2, A_1 \rightarrow aA_2Z_2A_2, A_1 \rightarrow bZ_2A_2 \\ A_2 \rightarrow aA_1, A_2 \rightarrow b, A_2 \rightarrow aA_1Z_2, A_2 \rightarrow bZ_2, \\ Z_2 \rightarrow aA_1A_1 \mid bA_1 \mid aA_1Z_2A_1 \mid bZ_2A_1 \\ Z_2 \rightarrow aA_1Z_2A_1 \mid aA_1Z_2A_1Z_2 \mid bA_1Z_2 \mid bZ_2A_1Z_2$$

6.16

CYK ALGORITHM

In this section we will see that the parsing of a string ' s ' requires $|s|^3$ steps. We now discuss CYK algorithm to have this claim. The name CYK is taken from Cocke J., Younger D. H., and Kasami T., because of their contribution to this concept. The major limitation of this algorithm is that the grammar input must be in Chomsky Normal Form (CNF). Let us assume a CFG

$$G = (V_N, \Sigma, P, S) \text{ in CNF, and}$$

there is a string

$$s = x_1x_2x_3x_4 \dots x_n.$$

Let us define substrings

$$s_{ij} = x_ix_{i+1} \dots x_j$$

and a subset of nonterminals V_N , as

$$V_{ij} = \{X \in V_N \mid X \Rightarrow s_{ij}\}$$

This means $s \in L(G)$ iff $S \in V_{1n}$.

For the computation of V_{ij} , we observe that $X \in V_{ij}$ iff CFG G contains a production of the form $X \rightarrow x_i$, this way we can compute V_{ii} for all i from 1 to n using productions of the grammar G . For $i > j$, we see that X drives s_{ij} iff there exists a production of the form $X \rightarrow YZ$ with $Y \Rightarrow s_{ik}$ and $Z \Rightarrow s_{k+1}$ for some k varying between i and j . This can also be represented as

$$V_{ij} = \bigcup_{k \in \{i, i+1, \dots, j\}} \{X \mid X \rightarrow YZS, \text{ and } Y \in V_{ik}, Z \in V_{k+1, j}\}$$

This equation shows the flexibility for computation of all V_{ij} . We can compute

- (i) $V_{11}, V_{22}, V_{33}, \dots, V_{nn}$
- (ii) $V_{12}, V_{23}, V_{34}, \dots, V_{n-1, n}$
- (iii) $V_{13}, V_{24}, V_{35}, \dots, V_{n-n, n}$

With the help of CYK algorithm we can determine the membership for any given language for which there exists a CFG in CNF. We can see that algorithm requires $O(n^3)$ steps as there are $\frac{m(n+1)}{2}$ sets of V_{ij} . The evaluation of each set involves n terms.

Example 6.16

Determine whether the string $s = 00111$ belongs the language generated by the following CGF in Chomsky Normal Form:

$$\begin{aligned} S &\rightarrow XY \\ X &\rightarrow 0 \mid YY \\ Y &\rightarrow 1 \mid XY \end{aligned}$$

Solution We have $s_{11} = 0$, the term V_{11} can be seen as set of all nonterminals that derives 0, therefore $V_{11} = \{X\}$. Now $s_{22} = 0$, so we have $V_{22} = \{X\}$, and this way we have

$$\begin{aligned} V_{11} &= \{X\} \\ V_{22} &= \{X\} \\ V_{33} &= \{Y\} \\ V_{44} &= \{Y\} \\ V_{55} &= \{Y\} \end{aligned}$$

By using formula , we have

$$\begin{aligned} V_{ij} &= \bigcup_{k \in \{i, i+1, \dots, j\}} \{X \mid X \rightarrow YZ, \text{ and } Y \in V_{ik}, Z \in V_{k+1, j}\} \\ V_{12} &= \{X \mid X \rightarrow YZ, Y \in V_{11}, Z \in V_{22}\} \end{aligned}$$

As we have $V_{11} = \{X\}$ and $V_{22} = \{X\}$, therefore the set will contain all nonterminals that occur on the left hand side of the production whose right hand side is XX . Since there is no such production. Thus V_{12} is empty. In this continuation we have,

$$V_{23} = \{X \mid X \rightarrow YZ, Y \in V_{22}, Z \in V_{33}\}$$

Therefore the required right hand side is XY , so we have

$$V_{23} = \{S, Y\}.$$

In the same way we can compute $V_{34} = \{X\}$, $V_{45} = \{X\}$, and

$$V_{13} = \{S, Y\}, V_{24} = \{X\}, V_{35} = \{S, Y\},$$

$$V_{14} = \{X\}, V_{25} = \{S, Y\},$$

$$V_{15} = \{S, Y\}$$

The levelled computation is represented by following diagram:

Level 5	S, Y				
Level 4	X	S, Y			
Level 3	S, Y	X	S, Y		
Level 2		S, Y	X	X	
Level 1	X	X	Y	Y	Y
Input String	0	0	1	1	1

Therefore, $00111 \in s$.

❖ Did you Know ?

The CYK algorithm has three different sources: Cocke J., Younger D. H., and Kasami T. The work of Cocke J. on CYK algorithm was circulated privately and it was never published. He is recognised for his large contribution to computer architecture and optimising the compiler design. He is considered by many to be “the father of RISC (Reduced Instruction Set Computer) architecture.”

6.17

APPLICATIONS OF CFG

A context-free grammar is a useful tool for defining programming languages. In this section we will discuss how context-free grammar is used to define the syntax of C language and definition part of HTML (Hyper Text Markup Language).

Let us first discuss how the concept of context-free grammar, with the help of BNF (Bacur Naur Form) is used to define C type of a small hypothetical language construct. The BNF notation mainly consists of angular brackets $<$ and $>$ to denote variable for or nonterminals, for example $<\text{identifier}>$ denotes the class of identifier, the notation $::=$ is used to denote “is defined as”, for example,

$<\text{identifier}> ::= <\text{letter}>(<\text{letter}> | <\text{digit}>)^*$

denotes an identifier obtained by writing a letter followed by a combination of zero or more letters and digits.

By using CFG and BNF, here are some C statements:

$<\text{statement}> ::= <\text{assignment statement}> | <\text{compound-statement}> | <\text{iteration-statement}> | <\text{selection-statement}>$

The productions in BNF for iteration statements are:

$<\text{iteration-statement}> ::= \text{do } <\text{statement}> \text{while } (<\text{expression}>) |$
 $\text{while}(<\text{logical-expression}>) <\text{statement}> |$
 $\text{for } ([<\text{expression}>]; [<\text{expression}>]; [<\text{expression}>]); <\text{statement}>$

The productions in BNF for selection-statements are:

$<\text{selection-statement}> ::= \text{if } (<\text{logical-expression}>) <\text{statement}> |$
 $\text{if } (<\text{logical-expression}>) <\text{statement}> \text{ else } <\text{statement}> |$
 $\text{switch}(<\text{expression}>) \{<\text{cases}>\}$

The productions in BNF for logical-expression are:

$\langle \text{logical-expression} \rangle ::= \langle \text{comparison} \rangle | \langle \text{comparison} \rangle \&\& \langle \text{logical-expression} \rangle |$
 $\langle \text{comparison} \rangle || \langle \text{logical-expression} \rangle$

The productions in BNF for comparison are:

$\langle \text{comparison} \rangle ::= (\langle \text{boolean-operand} \rangle \langle \text{comparison-operator} \rangle (\langle \text{boolean-operand} \rangle))$

The productions in BNF for boolean-operand are:

$\langle \text{boolean-operand} \rangle ::= \text{True} | \text{False} | \langle \text{identifier} \rangle$

The productions in BNF for expression are:

$\langle \text{expression} \rangle ::= \langle \text{factor} \rangle | \langle \text{expression} \rangle + \langle \text{factor} \rangle | \langle \text{expression} \rangle - \langle \text{factor} \rangle$

The productions in BNF for factor are:

$\langle \text{factor} \rangle ::= \langle \text{factor} \rangle * \langle \text{operand} \rangle | \langle \text{operand} \rangle$

The productions in BNF for comparison-operator are:

$\langle \text{comparison-operator} \rangle ::= > | < | >= | <= | != | =$

The productions in BNF for cases are:

$\langle \text{cases} \rangle ::= \langle \text{case} \rangle \langle \text{cases} \rangle | \langle \text{default} \rangle$

The productions in BNF for case are:

$\langle \text{case} \rangle ::= \langle \text{case-head} \rangle \langle \text{cases} \rangle | \langle \text{case-head} \rangle \langle \text{statement} \rangle$

$\langle \text{case-head} \rangle ::= \text{case} \langle \text{literal} \rangle :$

The production in BNF for default case is:

$\langle \text{default} \rangle ::= \text{default} : \langle \text{statement-sequence} \rangle$

Let us now define some token classes in terms of BNF notation.

$\langle \text{input} \rangle ::= \langle \text{token} \rangle | \langle \text{comment} \rangle | \langle \text{white-space} \rangle$

$\langle \text{white-space} \rangle ::= \backslash n | \backslash t | \backslash r | \langle \text{invisible-space} \rangle$

$\langle \text{comment} \rangle ::= // \langle \text{sequence of characters} \rangle /* \langle \text{sequence of characters} \rangle /*$

$\langle \text{token} \rangle ::= \langle \text{identifier} \rangle | \langle \text{operator} \rangle | \langle \text{literal} \rangle | \langle \text{keyword} \rangle | \langle \text{separator} \rangle$

$\langle \text{identifier} \rangle ::= \langle \text{letter} \rangle (\langle \text{letter} \rangle | \langle \text{digit} \rangle)^*$

$\langle \text{letter} \rangle ::= a|b|c|d| \dots \dots \dots x|y|z|A|B|C|D| \dots \dots \dots |X|Y|Z$

$\langle \text{digit} \rangle ::= 0|1|2|3|4|5|6|7|8|9$

$\langle \text{keyword} \rangle ::= \text{int} | \text{main} | \text{void} | \text{getch} | \text{if} | \text{else} | \text{boolean}$

$\langle \text{literal} \rangle ::= \langle \text{integer-value} \rangle | \langle \text{float-value} \rangle | \langle \text{boolean-value} \rangle$

$\langle \text{integer-value} \rangle ::= \langle \text{digit} \rangle \langle \text{digit} \rangle^*$

$\langle \text{boolean-value} \rangle ::= \text{True} | \text{False}$

$\langle \text{float-value} \rangle ::= \langle \text{decimal} \rangle | \langle \text{exponential} \rangle$

$\langle \text{decimal} \rangle ::= \langle \text{signed-integer} \rangle . \langle \text{integer} \rangle | \langle \text{integer-value} \rangle | \langle \text{sign} \rangle$

Now we discuss how a context-free grammar is acceptable in the definition part of HTML. Here we will use $\rightarrow$ (the normal production notation) instead of $::=$, and underscore instead of angular brackets. The underlined part in each production is a nonterminal.

HTML-Document $\rightarrow$ $\langle \text{HTML} \rangle$ document $\langle \text{HTML} \rangle$

The productions in BNF for document are:

document $\rightarrow$ $\langle \text{head} \rangle$ head-part $\langle \text{head} \rangle \langle \text{body} [\text{body-attributes}]^* \rangle$ body-part $\langle \text{body} \rangle$

The productions in BNF for head part are:

head-part $\rightarrow$ $\langle \text{title} \rangle$ title-part $\langle \text{title} \rangle$

The productions in BNF for title part are:

title-part $\rightarrow$ string

string $\rightarrow$ $[\text{letter}] [\text{letter} | \text{digit}]^*$

digit $\rightarrow$ 0|1|2|3|4|5|6|7|8|9

letter → $a|b|c|d|\dots\dots\dots x|y|z|A|B|C|D|\dots\dots\dots|X|Y|Z$

The productions in BNF for body attributes are:

body-attributes → [background = “bg-value”][bgcolor = “color”] [text = “color”]

The productions in BNF for color settings are:

color → acqua | black | blue | fuchsia | | yellow | white

body-part → paragraph | text | line-break | image | link | map-code | from-code | comment

The productions in BNF for paragraph are:

paragraph → <p>[para-attribute]* body-part</p>

text → string [<text-marker>text </text-marker>]*

line-break →
 body-part

image →

The productions in BNF for link are:

link → linked-item-identifier

comment → <!--body-part>

text-marker → header-marker | physical-text-marker | content-based-marker | list-marker

header-marker → h1 | h2 | h3 | h4 | h5 | h6

physical-text-marker → b | bg | i | s | small | sub | tt | u

content-based-marker → cite | code | em | strong | var

list-marker → unordered-list-marker | ordered-list-marker | definition-list-marker

unordered-list-marker listed-items

listed-items → string

ordered-list-marker → listed-items

definition-list-marker → <dl> definition-list </dl>

definition-list → [definition]*

definition → <dt> string </dt> <dd> string </dd>

para-attribute → align = “align-attribute”

align-attribute → centre | left | right | justified

textual → [text] body-part

blank →

image-attribute → alt = “string” align = “align-attribute” | integer-attribute = “integer”

integer-attribute → height | width | boarder

linked-item-identifier → string | [image] + path

path → absolute-path | relative-path

absolute-path → [web-path | non-web-path] relative-path

web-path → web-protocol | DNS-name-of-host

web-protocol → web-protocol-name:

web-protocol-name → http://ftp/gopher/telnet

DNS-name-of-host → //[www | ftp][.string] + [: port]

port → integer

non-web-path → non-web-protocol mail-address

mail-address → string@string[.string]*

relative-path → /directory/file

Finally, if we substitute the required productions to the right hand side of the production

HTML-Document → <HTML> document </HTML>

The elimination of nonterminals from right hand side represents the general layout of an HTML code.

Summary

- **Context-Free Grammar** A context-free grammar (CFG) is defined as 4-tuples (V_N, Σ, P, S) , where P is a set of production rules of the form

one nonterminal $\rightarrow$ finite string of terminals and/or nonterminals

- **Context Free Languages** The languages generated by the context-free grammars are called context-free languages.

- **Deterministic Context-Free Language** A language L is a deterministic context-free language (abbreviated as DCFL) if there is a deterministic push down automaton (abbreviated as DPDA) accepting language L .

- **Deterministic Context-Free Grammar** A context-free grammar is deterministic if and only if its rules allow a (left-to-right, bottom-up) parser to proceed without having to guess the correct continuation at any point.

- **Derivations** A derivation can be defined as the process to generate a string of terminals by replacing the nonterminals in the right hand side of a production.

- **Parse Trees** The strings generated by a context-free grammar $G = (V_N, \Sigma, P, S)$ can be represented by a hierarchical structure called tree.

- **Yield of a Derivation Tree** The yield of a derivation tree is the concatenation of the labels of leaf nodes in left-to-right ordering without repetition.

- **A-tree** A subtree looks like a derivation tree except that the label of the root node may not be S (the start symbol) if and only if S does not appear in the right side of any production. It is called an *A-tree* if the label of its root is A . Similarly if there is another subtree whose label of root node is B then it is called a *B-tree*.

- **Working String** The arrow symbol ' $\Rightarrow$ ' is employed between the unfinished stages of the generation of a string or a derivation. It means that there exists a possible substitution in $XY \Rightarrow abY$ if there exists a production $X \rightarrow ab$. These unfinished stages are strings of terminals and nonterminals that are generally called working strings.

- **Internal Nodes** A node in a derivation tree whose label is a variable (or say nonterminal) is called a leaf node or an external node. For example, if the label of a node is $A \in V_N$, then A is an internal node.

- **Leaf Node** These are also called external nodes. A node in a derivation tree, whose label is a terminal or $\wedge$ is called a leaf node or external node or simply a leaf.

- **Sentential Form** A string of terminals and variables (say α) is called a sentential form if $S \Rightarrow \alpha$, where $S \in V_N$ and $\alpha \in (V_N \cup \Sigma)^*$.

- **Nullable Variable** A variable E in CFG G is said to be nullable if it derives (or say generates) $\wedge$ i.e., $E \Rightarrow \wedge$.

- **Sentential Form** Let $G = (V_N, \Sigma, P, S)$ be a CFG and there is some string $s \in (V_N, \Sigma)^*$ such that $S \Rightarrow s$, then there exists a sentential form.

- **Leftmost Derivation** A derivation $S \Rightarrow s$ is said to be a leftmost derivation if production is applied only to the leftmost variable (nonterminal) at every step. It is not necessary that we can derive a derivation by start symbol only; we can also obtain a derivation from some other variable in V_N .

- **Rightmost Derivation** A derivation $S \Rightarrow s$ is said to be a rightmost derivation if production is applied only to the rightmost variable (nonterminal) at every step. It is not necessary that we can derive a derivation by start symbol only; we can also obtain a derivation from some other variable in V_N .

- **Total Language Tree** For a given context-free grammar G , we define a derivation tree with the start symbol as its root node, and its children as working string of terminals and nonterminals. The descendants of each node are all possible results of applying every applicable production to the working string, one at

a time. A string of all terminals is a terminal node in the tree. This type of resultant tree is called the total language tree of the context-free grammar G .

- **Ambiguous Grammar** A context-free grammar G is said to be ambiguous if there is at least one string in $L(G)$ having two or more distinct derivation trees (or equivalently, two or more distinct leftmost derivations; or two or more distinct rightmost derivations).
- **Inherently Ambiguous CFL** A context-free language for which every context-free grammar is ambiguous is said to be an inherently ambiguous context-free language.
- **Reduced CFG** If G is a context grammar that generates non-empty language $L(G)$ (i.e. $L(G) \geq \phi$), then we can find an equivalent reduced CFG G_1 such that each variable in G_1 derives some terminal string. For every context grammar $G_1 = (V'_N, \Sigma, P, S)$, we can construct an equivalent grammar $G' = (V''_N, \Sigma'', P', S)$ such that every symbol in $(V''_N \cup \Sigma'')$ appears in some sentential form.
- **Chomsky Normal Form (CNF)** A context-free grammar G is said to be in Chomsky normal form if every production is of the form either $A \rightarrow a$, (exactly one terminal in the right hand side of the production) or $A \rightarrow BC$ (exactly two variables in the right hand side of the production).
- **Greibach Normal Form (GNF)** A context-free grammar is said to be in Greibach normal form if every production is of the form $A \rightarrow a\alpha$, where $a \in \Sigma$, $A \in V_N$ and $\alpha \in V_N^*$.

Review Questions

- 6.1 Construct a context-free grammar G which generates all integers.
- 6.2 Construct context-free grammars which can be used to specify the overall syntax of language like C , Pascal, etc.
- 6.3 Show that the grammar $S \rightarrow aSb \mid SS$, does not generate any terminal string.
- 6.4 Consider the context-free grammar G with productions $S \rightarrow aAS \mid a, A \rightarrow SbA \mid SS \mid ba$, show that

$$S \xRightarrow[G]{*} aabbbaa$$
- 6.5 Consider the context-free grammar G with productions $S \rightarrow 0B \mid 1A, A \rightarrow 1AA \mid 0S \mid 0, B \rightarrow 0BB \mid 1S \mid 1$ and construct a derivation tree for string 00110101 from G .
- 6.6 Show that the context-free grammar G given by productions $S \rightarrow SBS \mid a, B \rightarrow b$, is ambiguous.
- 6.7 Determine the closure of union of two languages L_1 and L_2 such that L_1 is the language of all palindromes over $\{0, 1\}$, and L_2 is the language of all strings having equal number of a 's and b 's.
- 6.8 Show that $a * b + a * b$ is in $L(G)$, where G is given by $S \rightarrow S * S \mid S + S \mid a \mid b$.
- 6.9 Show that the CFG G given by productions $S \rightarrow 0X \mid 01, X \rightarrow 0XX \mid 1, Y \rightarrow XY1 \mid 1$, is ambiguous.
- 6.10 Prove that $L(G') = L(G) - \{\wedge\}$, where G is a CFG with null productions and G' is a CFG equivalent to G after elimination of null productions.
- 6.11 Consider the CFG G whose productions are $S \rightarrow 0S \mid XY, X \rightarrow \wedge, Y \rightarrow \wedge, Z \rightarrow 1$. Construct a CFG G' without null productions generating $L(G) - \{\wedge\}$.
- 6.12 Consider a CFG G whose productions are $S \rightarrow XY, X \rightarrow 0, Y \rightarrow 1, Z \rightarrow A$ and $A \rightarrow 2$. Construct a CFG G' equivalent to G by eliminating unit productions.
- 6.13 Let CFG $G = (\{S, A, B, C, E\}, \{a, b\}, \{S \rightarrow AB, A \rightarrow a, B \rightarrow b, B \rightarrow C, E \rightarrow c\}, S)$
Construct a CFG G' such that every variable in G' derives some terminal string.
- 6.14 Construct a reduce CFG G' equivalent to the context-free grammar G whose productions are

$$S \rightarrow CA \mid AB, A \rightarrow a, B \rightarrow AB \mid BC, \text{ and } C \rightarrow aB \mid b$$

- 6.15** Consider a context-free grammar G given by the productions

$$S \rightarrow ABD, A \rightarrow aA \mid bBA, B \rightarrow b \mid CA.$$

Reduce the CFG G into Chomsky normal form.

- 6.16** Consider a context-free grammar G having productions.

$$S \rightarrow AA \mid a, A \rightarrow aB \mid ab \mid \wedge, B \rightarrow C \mid \wedge, C \rightarrow bb$$

Reduce the CFG G into Chomsky normal form.

- 6.17** Reduce context-free grammars into Greibach normal form, given by following productions:

$$(i) S \rightarrow BB \mid b, B \rightarrow SS \mid a$$

$$(ii) S \rightarrow AB, A \rightarrow BS, A \rightarrow b, B \rightarrow SA, B \rightarrow a$$

- 6.18** Reduce the following grammars into CNF and GNF:

$$(i) G = (\{S, A, B\}, \{0, 1\}, \{S \rightarrow 1A \mid 0B, A \rightarrow 0S \mid 0, B \rightarrow 1S \mid 1\}, S)$$

$$(ii) G = (\{S\}, \{a, b, c\}, \{S \rightarrow cSS \mid a \mid b\}, S)$$

$$(iii) G = (\{S, A, B\}, \{a, b\}, \{S \rightarrow ABa, A \rightarrow aab, B \rightarrow AC\}, S)$$

$$(iv) G = (\{S, A\}, \{a, b\}, \{S \rightarrow abSa \mid aAb \mid a, A \rightarrow bS\}, S)$$

- 6.19** Consider the context-free grammar given by productions $S \rightarrow aA, A \rightarrow aA \mid bA \mid \wedge$. What is the language generated by this CFG?

- 6.20** Construct an unambiguous context-free grammar for the language of all algebraic expressions involving parentheses, the operand a , and four operators $+$, $-$, $*$ and $/$.

Hints

- 6.1** The required grammar is $G = (\{S, I, D\}, \{+, -, 0, 1, 2, \dots, 9\}, P, S)$, where P is defined as

$$P = \{S \rightarrow +I \mid -I, I \rightarrow DI \mid D,$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9\}$$

- 6.2** The advantage of using high level programming language such as C, Pascal, etc., is that they allow us to write statements that look more like English, and we can use Context-free grammars to capture many of these rules of programming languages. It is easy to see how we might go about constructing CFGs to specify simple rules of English syntax. The most basic type of declarative sentence has the structure

Subject verb object

And so we could start with a production

sentence $\rightarrow$ *Subject verb object*

- 6.3** The start symbol is not nullable.

- 6.4** $S \Rightarrow aAS \Rightarrow aSbAS \Rightarrow aabAS \Rightarrow aabbaS$

$$\Rightarrow aabbaa, \text{ therefore } S \xRightarrow{*}_G aabbaa$$

- 6.6** The construction of two different derivation-trees below for the same yield shows that the grammar G is ambiguous.

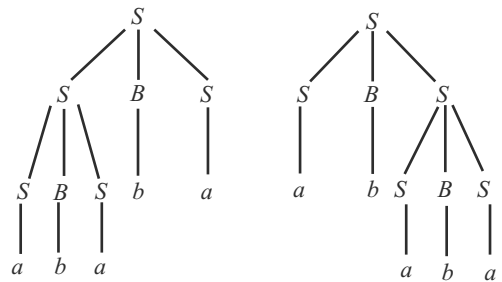


Fig. 6.27 Two different derivation trees for yield 'ababa' from grammar G

- 6.8** $S \Rightarrow S+S \Rightarrow S*S+S \Rightarrow a*S+S \Rightarrow a*S+S*S$
 $\Rightarrow a*b+S*S \Rightarrow a*b+a*S \Rightarrow a*b+a*b$

Hence, $a*b+a*b$ is in $L(G)$

- 6.9** One of the possible solutions is two different derivation trees given below

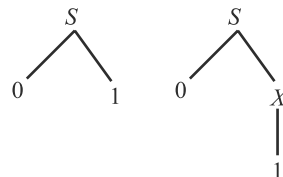


Fig. 6.28 Two different trees for yield '01' from G

6.11 The constructed grammar (without null productions) has the productions

$$S \rightarrow 0S, S \rightarrow XY, S \rightarrow X, \quad S \rightarrow Y \text{ and } Z \rightarrow 1.$$

6.12 The constructed context-free grammar G' (without unit productions) has the following productions

$$S \rightarrow XY, X \rightarrow 0, Y \rightarrow 1, Z \rightarrow 2, A \rightarrow 2$$

6.13 The constructed context-free grammar G' has following productions

$$S \rightarrow AB, A \rightarrow a, \text{ and } B \rightarrow b$$

6.14 First we find CFG G_1 such that every production generates terminal strings. We have following productions in grammar G_1

$$S \rightarrow CA, A \rightarrow a, \text{ and } C \rightarrow b$$

Now find the reduced grammar G' such that every production is in sentential form. We have following productions in reduced grammar G' :

$$\{S \rightarrow CA, A \rightarrow a, \text{ and } C \rightarrow b\}$$

6.15 The production $B \rightarrow b$ and $B \rightarrow CA$ are in required form $S \rightarrow ABD$ can be replaced with $S \rightarrow AC_1$ and $C_1 \rightarrow BD$. $A \rightarrow aA$ can be replaced with $A \rightarrow C_a A$ and $C_a \rightarrow a$. $A \rightarrow bBA$ can be replaced with $A \rightarrow C_b C_2$, $C_b \rightarrow b$ and $C_2 \rightarrow BA$. Therefore, the given grammar is converted into CNF with the productions $S \rightarrow AC_1$, $C_1 \rightarrow BD$, $A \rightarrow C_a A$, $C_a \rightarrow a$, $A \rightarrow C_b C_2$, $C_2 \rightarrow BA$, $C_b \rightarrow b$, $B \rightarrow b \mid CA$.

6.19 The language is of the form $a^n b^m$. We see that the smallest string generated by grammar is a . Therefore n must be greater or equal to 1, while m must be greater than or equal to zero. Hence, the language generated by the given grammar is $L = \{a^n b^m \mid m \geq 0, n \geq 1\}$

6.20 The constructed unambiguous grammar has following productions.

$$S \rightarrow S+T \mid T, T \rightarrow T-F \mid F, F \rightarrow F * G \mid G, \\ G \rightarrow G/H \mid H, H \rightarrow a$$

Graded Exercises

***6.1** Show that if L is an unambiguous context-free then L^* is also unambiguous.

***6.2** Construct a context-free grammar G which generates all non-palindrome strings over $\{a, b\}$.

*****6.3** Construct context-free grammars generating each of these languages:

(i) $L = \{a^i b^j c^k \mid i = j \text{ or } j \leq k\}$

(ii) $L = \{a^i b^j c^k \mid j = i + k\}$

(iii) $L = \{a^i b^j c^k \mid i = j \text{ or } j = k\}$

(iv) $L = \{a^i b^j \mid i \leq 2j\}$

***6.4** Find the language generated by the context-free grammar

$$G = (\{S, X\}, \{a, b\}, \{S \rightarrow XaXaX, X \rightarrow aX \rightarrow bX \mid \wedge\}, S)$$

****6.5** In each case, show that the context-free grammar given below is ambiguous, find an equivalent unambiguous context-free grammar:

(i) $G = (\{S\}, \{a, b\}, \{S \rightarrow SS \mid a \mid b\}, S)$

(ii) $G = (\{S, A, B\}, \{a, b\}, \{S \rightarrow ABA, A \rightarrow aA \mid \wedge, B \rightarrow bB \mid \wedge\}, S)$

(iii) $G = (\{S\}, \{a, b\}, \{S \rightarrow aaSb \mid aSb \mid \wedge\}, S)$

(iv) $G = (\{S, A, B\}, \{a, b\}, \{S \rightarrow AB, A \rightarrow aAB \mid ab, B \rightarrow abB \mid \wedge\}, S)$

***6.6** Determine whether the following CFGs are ambiguous or not?

(i) $S \rightarrow ab \mid aSb \mid BA, A \rightarrow a \mid b \mid \varepsilon, B \rightarrow aSB \mid \varepsilon$

(ii) $S \rightarrow aAb \mid aaSb \mid AaB, A \rightarrow a \mid ab \mid \varepsilon, B \rightarrow aSb \mid \varepsilon$

***6.7** Reduce the following grammar such that each production generates some terminal string

$$S \rightarrow ab \mid aSC \mid BA, A \rightarrow a \mid Cb \mid bb, B \rightarrow aB \mid \varepsilon$$

***6.8** Reduce the following grammar such that each production appears in sentential form

$$S \rightarrow ab \mid AA \mid BA, A \rightarrow a \mid Ab \mid bb, B \rightarrow aB \mid \varepsilon, C \rightarrow a \mid Cb$$

- *6.9 Eliminate null productions from the following grammar

$$S \rightarrow ab \mid aSb \mid BA, A \rightarrow a \mid b \mid \varepsilon, B \rightarrow aSB \mid \varepsilon$$

- *6.10 Eliminate unit productions from the following grammar

$$S \rightarrow ab \mid aSb \mid A, A \rightarrow a \mid b \mid B, B \rightarrow a \mid \varepsilon$$

- *6.11 Reduce the following grammar into Chomsky normal form

$$S \rightarrow ab \mid aSC \mid BA, A \rightarrow a \mid Cb \mid bb, B \rightarrow aB \mid \varepsilon$$

- *6.12 Reduce the following grammar into Greibach normal form

$$S \rightarrow a \mid AA \mid BA, A \rightarrow a \mid AB \mid b, B \rightarrow a$$

* Difficulty level 1

** Difficulty level 2

*** Difficulty level 3

Hints

- 6.2 The grammar G has productions $S \rightarrow aSb \mid bSa \mid ab \mid ba \mid abb \mid baa$.

- 6.3 (i) For the first case $i = j$ and k is arbitrary. This can be obtained by following productions.

$$X \rightarrow AC, A \rightarrow aAb \mid \wedge, C \rightarrow Cc \mid \wedge,$$

In the second case, i is arbitrary and $j \leq k$. Here we can have following productions:

$$Y \rightarrow BD, B \rightarrow Ab \mid \wedge, D \rightarrow bDc \mid E, E \rightarrow Ec \mid \wedge,$$

Finally, we start with the productions $S \rightarrow X \mid Y$.

- 6.4 X generates $(a, b)^*$. By substituting $(a, b)^*$ in production $S \rightarrow XaXaX$, we get.

$$L(G) = \{(a, b)^*a(a, b)^*a(a, b)^*\}$$

It is the language of all strings over (a, b) with at least two a 's.

Objective Questions

- Which of the following statements is false?
 - a regular language is also a context-free language.
 - a context-free language is also a regular language.
 - all context-free grammars are ambiguous.
 - both (b) and (c)
- For a derivation tree, which of the following statements is false?
 - the label of each leaf node is x , where x is a terminal.
 - the label of all nodes except leaf nodes (or leaves) is a non-terminal.
 - if the root of a subtree is A , then it is called an A -tree.
 - none of the above
- * If $S \Rightarrow s$ is in context-free grammar G , then
 - there is a rightmost derivation for ' s '.
 - there is a leftmost derivation of ' s '.
 - both (a) and (b)
 - none of these

4. A context-free grammar G is said to be ambiguous if
 - (a) it has two or more leftmost derivations for some terminal string $s \in L(G)$.
 - (b) it has two or more rightmost derivations for some terminal string $s \in L(G)$.
 - (c) neither (a) nor (b) is true
 - (d) both (a) and (b) are true
5. Which of the following statements is false?
 - (a) a CFG G is said to be a right-linear grammar if each production is of the form $A \rightarrow sB \mid s$, where $A, B \in V_N$ and $s \in \Sigma^*$.
 - (b) a CFG G is said to be a left-linear grammar if each production is of the form $A \rightarrow Bs \mid s$, where $A, B \in V_N$ and $s \in \Sigma^*$.
 - (c) a CFG G is said to be linear grammar if each production is of the form $A \rightarrow tBs \mid s$, where $A, B \in V_N$ and $s, t \in \Sigma^*$.
 - (d) none of the above
6. Which of the following statements is true?
 - (a) a CFG G is said to be self-embedded if there exists some useful variable 'A' such that $A \Rightarrow tAs$ where $s, t \in \Sigma^+$.
 - (b) a CFG G is regular if and only if it is generated by a non-self-embedding grammar.
 - (c) both (a) and (b)
 - (d) none of the above
7. If a CFG is in Chomsky normal form, then
 - (a) there is a restriction on the length of string in right side of the productions.
 - (b) there is a restriction in the nature of symbols in the right side of productions.
 - (c) both (a) and (b)
 - (d) (a) is true but (b) is false
8. Consider a CFG, $S \rightarrow aS \mid XY, X \rightarrow \wedge, Y \rightarrow \wedge$. The CFG after elimination of null production will be:
 - (a) $S \rightarrow aS \mid X \mid Y$
 - (b) $S \rightarrow aS \mid XY \mid X \mid Y \mid a$
 - (c) $S \rightarrow aS \mid X \mid Y \mid XY$
 - (d) All of these
9. Consider the CFG, given by the productions $S \rightarrow XY, X \rightarrow a, Y \rightarrow C \mid b, C \rightarrow D, D \rightarrow Z, Z \rightarrow a$. The context-free grammar after elimination of unit productions will be
 - (a) $S \rightarrow XY, X \rightarrow a, Y \rightarrow a \mid b$
 - (b) $S \rightarrow XY, X \rightarrow a, Y \rightarrow a \mid b, C \rightarrow a, D \rightarrow a, Z \rightarrow a$
 - (c) $S \rightarrow XY, X \rightarrow a, Y \rightarrow a \mid b, C \rightarrow a$
 - (d) all of the above.
10. If $L(G)$ is the language generated by grammar G given by productions

$$S \rightarrow S + S \mid S - S \mid S * S \mid S / S \mid (S) \mid a$$
 then:
 - (a) $a - a / a \in L(G)$
 - (b) $a + (a * a) / a - a \notin L(G)$
 - (c) $b - b / b \in L(G)$
 - (d) both (a) and (c)
11. If $S \rightarrow S + S \mid S * S \mid a$ are the productions of a context-free grammar G then which of the following statement is false?
 - (a) the grammar is ambiguous.
 - (b) $a + a + a * a * a$ is in language generated by the grammar.
 - (c) the grammar is in Chomsky normal form.
 - (d) $S \rightarrow S + T \mid T, T \rightarrow T * F \mid F, F \rightarrow a$ is equivalent unambiguous grammar

12. A context-free grammar having productions $A \rightarrow BC \mid a$ is in
 - (a) Greibach normal form
 - (b) Chomsky normal form
 - (c) both (a) and (b)
 - (d) neither (a) nor (b)
13. For the context-free grammar $S \rightarrow SS \mid AaAa \mid \wedge, A \rightarrow bA \mid \wedge$, which statement is false?
 - (a) A can generate b^*
 - (b) S can generate $(b^* ab^* ab^*)^*$
 - (c) $AaAa$ can generate $a^* b^*$
 - (d) both (b) and (c)
14. The language defined by the regular expression a^*bb generated by CFG
 - (a) $S \rightarrow aSbb \mid \wedge$
 - (b) $S \rightarrow aS \mid bb$
 - (c) both (a) and (b)
 - (d) none of these
15. Consider the context-free grammar with productions $S \rightarrow aSb \mid SS \mid ab$, generating language L , then
 - (a) no string in L begins with abb
 - (b) all strings in L begin with a
 - (c) all strings in L end with bb
 - (d) both (a) and (b)
16. Which of the following CFG is not ambiguous?
 - (a) $S \rightarrow SS \mid a \mid b$
 - (b) $S \rightarrow A \mid B, A \rightarrow ab \mid \wedge, B \rightarrow abB$
 - (c) $S \rightarrow ABA, A \rightarrow aA \mid \wedge, B \rightarrow bB \mid \wedge$
 - (d) none of these
17. If a context-free grammar G with $\wedge$ -productions is unambiguous, then
 - (a) a context-free grammar G' without $\wedge$ -productions is also unambiguous
 - (b) a context-free grammar G' without $\wedge$ -productions is definitely an ambiguous grammar
 - (c) $L(G')^*$ is also unambiguous, where $L(G')$ is the language generated by G' which is the CFG without $\wedge$ -productions
 - (d) both (a) and (c)
18. Consider the context-free grammar G given by the productions $S \rightarrow AA, A \rightarrow AAA \mid ba \mid Ab \mid a$, then the set of all words generated by CFG G are
 - (a) all words having even number of a 's
 - (b) no word having only b 's
 - (c) both (a) and (b)
 - (d) none of these
19. A context-free grammar G generating language $L(G)$ is given by productions $S \rightarrow SS \mid aSb \mid ab$, then
 - (a) the context-free grammar G is ambiguous.
 - (b) ' $aabbS$ ' is a sentential form for G .
 - (c) all terminal strings generated by G have an equal number of a 's and b 's.
 - (d) all of the above
20. All strings over $\{a, b\}$ in which the number of a 's are two times more than the number of b 's can be generated by CFG
 - (a) $S \rightarrow aSab \mid aab \mid \wedge$,
 - (b) $S \rightarrow aSaSbS \mid aSbSaS \mid bSaSaS \mid \wedge$,
 - (c) $S \rightarrow aab \mid aba \mid baa$,
 - (d) none of these
21. The CFG $S \rightarrow aS \mid bS \mid a \mid b$, is equivalent to:
 - (a) $(a + b)^+$
 - (b) $(a + b)(a + b)^*$
 - (c) $(a + b)^*(a + b)$
 - (d) all of these
22. The string of terminals generated by CFG $S \rightarrow AB, A \rightarrow aA \mid bA \mid a, B \rightarrow Ba \mid Bb \mid a$
 - (a) has at least two a 's
 - (b) has at least one b
 - (c) should end with ' a '
 - (d) all of these
23. The CFG with productions $S \rightarrow aB \mid bA, A \rightarrow b \mid aS \mid bAA, E \rightarrow b \mid bS \mid aBB$ generates strings of terminals which have

- (a) odd number of a 's and odd number of b 's (b) equal number of a 's and b 's
 (c) even number of a 's and even number of b 's (d) both (a) and (c)
24. The language can be generated by the CFG having productions
 (a) $S \rightarrow ab \mid aSb$, (b) $S \rightarrow aaSbb \mid ab \mid aabb$, (c) both (a) and (b) (d) none of these
25. Which of the following CFG cannot be simulated by a finite state machine?
 (a) $S \rightarrow aSb \mid ab$, (b) $S \rightarrow Sa \mid a$,
 (c) $S \rightarrow abA, A \rightarrow cB, B \rightarrow d \mid aA$, (d) all of these
26. The productions $E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid id$
 (a) are unambiguous
 (b) generate an inherently ambiguous language
 (c) generate an ambiguous language but inherently so
 (d) generate all possible fixed length valid computations for carrying out addition, subtraction, multiplication and division which can be expressed in our expression
27. The grammar $G = (\{S\}, \{0, 1\}, \{S \rightarrow SS \mid 0S1 \mid 1S0 \mid \wedge\}, S)$ generates a
 (a) context-free language (b) regular language
 (c) context-sensitive language (d) all of these
28. The grammar in above question generates a language L defined as
 (a) $\{0^n 1^n \mid n \geq 0\}$ (b) $\{s \in (0, 1)^* \mid s \text{ has equal number of 0's and 1's}\}$
 (c) $\{0^n 1^n \mid n \geq 0\} \cup \{1^n 0^n \mid n \geq 0\}$ (d) all of these
29. $L(G)$ is the language generated by $S \rightarrow aS \mid bA, A \rightarrow d \mid ccA$, then
 (a) $bccddd \in L(G)$ (b) $aadb \in L(G)$ (c) $ababccd \in L(G)$ (d) None of these
30. Which of the following statements is false?
 (a) CNF is useful for programmes that have to manipulate grammars
 (b) CNF is useful in checking of emptiness eliminating unit productions
 (c) a CFG can be converted into CNF without eliminating unit productions
 (d) none of these
31. Which of the following statements is false?
 (a) a context-free grammar generating the set accepted by a PDA is in GNF
 (b) CFG in GNF is typically complex and much longer than the CFG from which it was derived.
 (c) both (a) and (b)
 (d) none of the above
32. Which of the following is not necessary while converting a context-free grammar into Greibach Normal Form?
 (a) elimination of unit productions
 (b) elimination of null productions
 (c) reduction of given context-free grammar into equivalent Chomsky normal form
 (d) none of the above
33. The language is a:
 (a) context-free language (b) regular language (c) both (a) and (b) (d) none of these

References for Extra Reading

1. John C. Martin. *Introduction to Languages and the Theory of Computation*. McGraw Hill. 2003.
2. Łukasiewicz, Jan, *Elements of Mathematical Logic*. Translated from Polish by Olgierd Wojtasiewicz, New York, Macmillan, 1964.
3. Bala Sundara Raman, L., Ishwar, S., and Sanjeeth Kumar Ravindranath, *Context-free Grammar for Natural Language Constructs*. Proceedings of Tamil Internet, Chennai, 2003.
4. Michael Sipser, *Introduction to the Theory of Computation*, PWS Publishing, 1997.
5. Nederhof, M. J., *Regular approximations of CFLs: A grammatical view*, In International Workshop on Parsing Technologies, Massachusetts Institute of Technology, 1997.
6. Ginsburg, S. and E. H. Spanier, *Quotients of context-free languages*, Journal of the Association for Computing Machinery 10:4, 487–492, 1963.
7. Sheila A. Greibach, *A New Normal-Form Theorem for Context-Free Phrase Structure Grammars*, Journal of the ACM, 1965.
8. Sheila A. Greibach, *An Infinite Hierarchy of Context-Free Languages*, Journal of the ACM, 1969.
9. Shieber, Stuart, *Evidence against the context-freeness of natural language*. Linguistics and Philosophy, 1985.

Online Sources

1. <http://www.cs.sun.ac.za/~rw324/documents/rw324w5.pdf>
2. <http://www.kornai.com/MatLing/cflfinal.pdf>
3. www.brics.dk/~amoeller/papers/ambiguity/journal.pdf
4. <http://www.eecs.harvard.edu/~shieber/Biblio/Papers/shieber85.pdf>.
5. <http://www.cs.uiowa.edu/~hzhang/c135/contextfree-4p.pdf>



Push Down Automata

❖ Chapter Objective ❖

In this chapter we will describe the functioning of automata more advanced than a finite automaton. Our point of discussion will be focused on Push Down Automata (PDA). We will describe a model of PDA, its representation and the language accepted in two different ways called acceptance by null store (also called empty store) and acceptance by final state. In this sequence we will see the modification in a PDA model to make it more powerful in the form of two-stack PDA and auxiliary PDA. Finally we will discuss its application in a compiler in the form of parsing.

□ Introduction □

Push down automata (PDA), are a way to represent the language class called context free languages. Push down automata are abstract devices defined in theory of automata. They are similar to finite automata, except that they access a potentially unlimited amount of memory in the form of a stack. Push down automata (PDAs) are of two types, deterministic and non-deterministic. The languages accepted by non-deterministic push down automata are precisely context free languages (CFLs). If we allow a finite automaton to access two stacks instead of just one, we obtain a device (2-stack PDA) much more powerful than a push down automaton, equivalent to a Turing Machine (TM).

A finite automaton cannot accept a language having strings of the form $a^n b^n$ because a finite automaton has to remember the number of a 's in the string. For this, a finite automaton will require an infinite number of states. To overcome this problem an additional auxiliary memory in the form of stack is included. In stack the stored elements are processed in last in first out (LIFO) fashion. To accept strings of the form $a^n b^n$ given by language L , a 's are pushed (the insert operation in stack is called push and deletion is called pop) to the stack and when ' b ' encounters the topmost ' a ' from stack is deleted. Thus the matching of number of a 's and b 's is accomplished. This type of additional arrangement in a finite automaton is called push down automaton.

For context free languages PDA are not very important but the hierarchy of finite state machines with corresponding regular languages, PDA with corresponding CFLs and Turing machines with corresponding recursively enumerable sets (languages) is our major concern.

7.1

DESCRIPTION AND DEFINITION

In this section we will study the components of a push down automaton and the way it operates. A push down automaton consists of

- (i) a read only input tape
- (ii) input alphabet
- (iii) finite state control with two heads, one read only and another Read/Write
- (iv) a set of final states
- (v) an initial state
- (vi) a stack called push down store (PDS).

Both read and write operations are performed on push down store as we insert (or say push) elements into push down store or delete (or say pop) elements from push down store. The transition from one state to another in push down automaton is almost the same as in the case of finite automata. The push down automaton, on reading an input symbol from input tape and the top most symbol from push down store, moves to next state and writes (inserts) a string of symbols in push down store. Fig. 7.1 illustrates the model of push down automata.

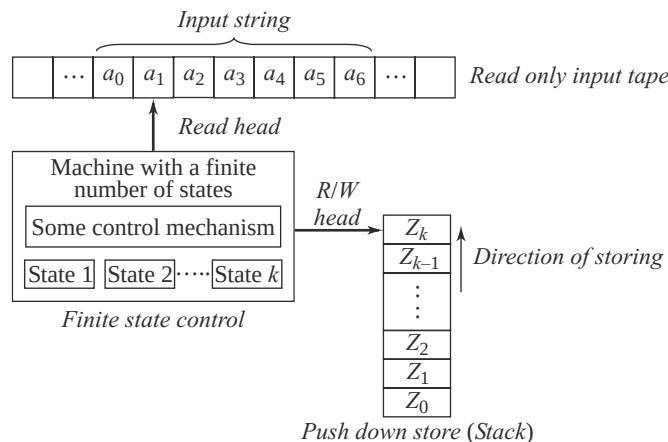


Fig. 7.1 Model of push down automata

7.2

DEFINITION AND MODEL OF PDA

A push down automata M is defined as 7-tuples $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, where

- (i) Q = a finite nonempty set of states,
- (ii) Σ = a finite nonempty set of input symbols,
- (iii) Γ = a finite nonempty set of push down symbols (i.e., the symbols in push down store),
- (iv) q_0 = initial state (start state),
- (v) Z_0 = special push down symbol called the initial symbol in push down store,

- (vi) F = the set of final states, $F \subseteq Q$
- (vii) δ = the transition function which maps from $Q \times \{\Sigma \cup \wedge\} \times \Gamma$ into the set of finite subset of $Q \times \Gamma^*$

The relation $F \subseteq Q$ shows that as maximum all states in Q may be final states, and as minimum there is no final state.

The transition function δ , explains that a push down automaton is in any state of Q , it takes input symbol from $\{\Sigma \cup \wedge\}$ and checks for top push down symbol (from Γ), as a result it makes a transition to next state (a state in set Q) and writes a string denoted by Γ^* in the top of push down store.

❖ Good to Know!

- A finite automaton with no final state has no significance because it accepts no language at all, but it is not true in case of PDA. A PDA can accept language by null or empty store also.
- It is not necessary that every time the reading head reads a symbol from read only input tape, when reading head does not read the symbol from input tape, then input symbol is assumed to be $\wedge$ (or ϵ).

7.3

LANGUAGE OF PDA

Context free and regular are the languages which are accepted by a PDA. If there is a context free grammar then there exists a context free language which can be accepted by a PDA. In other words, if a language can be represented by a context free grammar then there exists a PDA that accepts that language. If a language is context free which is not regular, then during the acceptance of that language the PDA will use its stack at least once. As we know the PDA is similar to a FA with only the difference that a PDA has additional memory in the form of stack. If we have to design a PDA for a regular language the PDA will not utilise its stack to accept the regular language.

7.4

GRAPHICAL NOTATIONS FOR PDA

At any time the state of a PDA $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, is given by:

- the state $q \in Q$ the PDA is currently in.
- the input string $x \in \Sigma^*$ which still has to be processed,
- the contents of the stack (push down store) $\alpha \in \Gamma^*$.

7.4.1 Instantaneous Description

An instantaneous description (ID) is defined as (q, x, α) , where $q \in Q$, $x \in \Sigma^*$ and $\alpha \in \Gamma^*$ for a push down automaton $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$. The instantaneous description (q, x, α) represents the present status of the push down automaton as illustrated below:

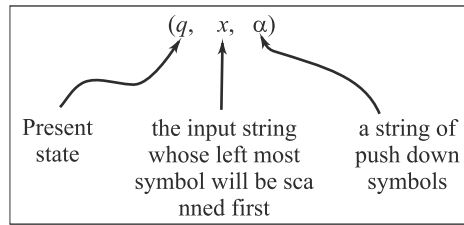


Fig. 7.2 Description of an ID of a PDA

If $x = a_1 a_2 a_3 \dots a_k$, then the first symbol to be scanned is a_1 , then a_2 and so on. If $\alpha = Z_1 Z_2 \dots Z_l$, then Z_1 is the top most symbol in push down store.

We can take an example of an instantaneous description (ID) as $(q_1, a_1 a_2 a_3 \dots a_k, Z_1 Z_2 \dots Z_l)$, this instantaneous description describes that the current state of push down automaton is q_1 , the input string to be processed is $a_1 a_2 a_3 \dots a_k$. The move of PDA in next state (depending upon the design of the PDA) depends upon the present input (the left most symbol of input string) and the top most symbol of push down store.

In an instantaneous description, input string may also be $\wedge$ (a case of NPDA). NPDA is non deterministic PDA that works non-deterministically, i.e., for some specific configuration there are two or more different instantaneous descriptions. In this case push down automata makes a $\wedge$ -move. The working of push down automata can be described in term of change in instantaneous description; it is just like the case of finite automaton where working is described in terms of change in states.

7.4.2 Move Relation

The transition from an instantaneous description (ID) to another instantaneous description is defined by the term move relation and denoted by the symbol $\vdash$. For example a push down automaton is in state q_1 , the input string is $x_1 x_2 x_3 \dots x_n$, and the symbols in push down store are $Z_1 Z_2 Z_3 \dots Z_l$ where Z_1 is the top element of the stack. If this push down automaton goes to next state q_2 by scanning x_1 from the input string and by writing a string $u_1 u_2 \dots u_m$ (say β) on place of Z_1 (the top most symbol) in push down store (PDS). The move relation for this transition can be written as

$$\begin{array}{ccc} (q_1, x_1 x_2 x_3 \dots x_n, Z_1 Z_2 Z_3 \dots Z_l) & \vdash & (q_2, x_2 x_3 \dots x_n, \overbrace{u_1 u_2 u_3 \dots u_m}^{\beta} Z_2 Z_3 \dots Z_l) \\ \text{Present ID} & & \text{Next ID} \end{array}$$

After this transition the length of input string will be decreased by one. Now the next input symbol will be x_2 and top most symbol in push down store will be u_1 .

It is not necessary that each time the string β to replace the top symbol of PDS, will be of length more than one. There are several transitions in which β is $\wedge$. For example, let us consider the following transition:

$$(q, abab, baaZ_0) \vdash (q', bab, aaZ_0).$$

In this case $\beta = \wedge$. This move relation is shown by the diagram given by Fig. 7.3 (see below).

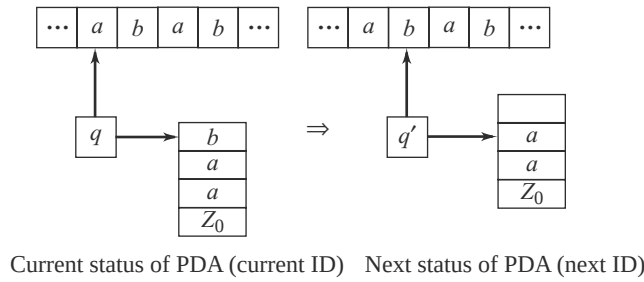


Fig. 7.3 A move relation

❖ Good to Know!

As we have used $\Rightarrow^*$ to represent a definite sequence of more than one substitutions in a derivation from a grammar G to generate the language $L(G)$, we can also define the reflexive transitive closure $\vdash^*$ which represents a definite sequence of more than one move relations.

If (q_1, x, α_1) is the present ID and after more than one move relations we get (q_n, y, α_2) , then we can write

$$(q_1, x, \alpha_1) \vdash^* (q_n, y, \alpha_2)$$

If the above relations are exactly n , then we can write

$$(q_1, x, \alpha_1) \vdash^n (q_n, y, \alpha_2)$$

If $n = 0$, it means there is no transition and the next ID will be the present ID.

Also $\vdash^*$ can be split into several parts as:

$$(q_1, x_1, \alpha_1) \vdash (q_2, x_2, \alpha_2) \vdash (q_3, x_3, \alpha_3) \vdash \dots \vdash (q_i, x_m, \alpha_m)$$

where, $x_1 x_2 \dots x_m \in \Sigma^*$, and $\alpha_1, \alpha_2, \dots, \alpha_m \in \Gamma^*$.

❖ Good to Know!

If we are dealing with more than one push down automata simultaneously, we specify the PDA during the description of a move relation. For example, a move relation for PDA A is denoted by $\vdash_A$, for PDA B by $\vdash_B$, and so on.

7.4.3 Properties of Move Relation

Property 1 If

$$(q, x, \alpha_1) \vdash^* (q', \wedge, \beta) \quad (7.1)$$

then for every $y \in \Sigma^*$, we have

$$(q, xy, \alpha_i) \vdash^* (q', y, \beta) \quad (7.2)$$

In converse if $(q, xy, \alpha_i) \vdash^* (q', y, \beta)$ for some $y \in \Sigma^*$, then $(q, x, \alpha_i) \vdash (q', \wedge, \beta)$.

Proof By referring the Fig. 7.3, the result can be understood very easily. If the PDA is in state q with string α in push down store, the moves given by equation (7.1) are affected by processing the string 'x', the PDA goes to state q' with β in push down store. The same transition is affected by starting with the input string 'xy' and processing only 'x'. Now 'y' is the string which has to be processed, as represented by equation (7.2).

Similarly, we can prove the converse part of property 1.

Property 2 If

$$(q, x, \alpha) \vdash^* (q', \wedge, \Gamma) \quad (7.3)$$

Then for every $\beta \in \Gamma^*$, we have

$$(q, x, \alpha\beta) \vdash^* (q', \wedge, \gamma\beta) \quad (7.4)$$

Proof The series of moves given by equation (7.3) can be split in finite moves as

$$(q, x, \alpha) \vdash (q_1, x_1, \alpha_1) \vdash (q_2, x_2, \alpha_2) \vdash (q_3, x_3, \alpha_3) \vdash \dots (q', \wedge, \gamma)$$

We consider $(q_n, x_n, \alpha_n) \vdash (q_{n+1}, x_{n+1}, \alpha_{n+1})$. Suppose $\alpha_n = Z_1 Z_2 Z_3 \dots Z_m$. As a result of this move, Z_1 is eliminated and some string is placed above $Z_2 Z_3 \dots Z_m$. Therefore, a sequence of element $Z_2 Z_3 \dots Z_m$ in push down store is not affected. So, we get

$$(q_n, x_n, \alpha_n\beta) \vdash (q_{n+1}, x_{n+1}, \alpha_{n+1}\beta)$$

Therefore, we get a series of moves

$$(q, x, \alpha\beta) \vdash (q_1, x_1, \alpha_1\beta) \vdash (q_2, x_2, \alpha_2\beta) \vdash (q_3, x_3, \alpha_3\beta) \vdash \dots (q', \wedge, \gamma\beta)$$

or we can say

$$(q, x, \alpha\beta) \vdash^* (q', \wedge, \gamma\beta)$$

7.4.4 Representation of PDA by Transition Diagram

A transition pushing input symbol on PDS $\delta(q_i, 0, Z_0) = \{(q_j, 0Z_0)\}$ can be represented as

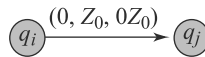


Fig. 7.4 Transition diagram for $\delta(q_i, 0, Z_0) = \{(q_j, 0Z_0)\}$

This transition diagram shows a transition from state q_i to q_j on input 0. The input 0 is pushed on to stack when Z_0 is the top element of the stack. Similarly, the transition $\delta(q_0, 0, 0) = \{(q_0, \wedge)\}$ popping an element from the stack can be represented by following transition diagram:



Fig. 7.5 Transition diagram for $\delta(q_0, 0, 0) = \{(q_0, \wedge)\}$

7.5

ACCEPTANCE BY FINAL STATE AND EMPTY STACK

The acceptance of language by the push down automaton can be defined in terms of (i) final states or (ii) push down store. Because a push down automaton can have final states like an NDFA and also an additional memory in the form of stack called push down store (PDS).

Acceptance by Final State The acceptance conditions include that the input string must be exhausted as the PDA reaches to final state. Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be a push down automaton. The set accepted by PDA in terms of final state denoted by $T(P)$ is defined as

$$T(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash (q_f, \wedge, \alpha) \text{ for some } q_f \in F, \text{ and } \alpha \in \Gamma^*\}$$

where 'w' is any input string on read only tape.

Acceptance by Null or Empty Store The acceptance conditions include that the stack must be empty as the input string is exhausted. Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \{\phi\})$ be a push down automaton (PDA) accepting language L by empty store, for this we can define a push down automaton

$$P' = (Q', \Sigma, \Gamma', \delta', q'_0, Z'_0, F')$$

which accepts language L by final state, i.e., $L = N(P) = T(P')$

where,

$N(P)$ = the set accepted by PDA P in terms of null store,

$T(P')$ = the set accepted by PDA P' in terms of final state.

The set accepted by PDA in terms of null store (empty store) denoted by $N(P)$ is defined as

$$N(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash (q', \wedge, \wedge) \text{ or } (q', \wedge, Z_0) \text{ for some } q' \in Q\}$$

where 'w' is any string on read only tape.

❖ Good to Know!

The pushdown store of PDA with only element $\wedge$ as well as Z_0 is considered to be empty.

7.6

FROM EMPTY STACK TO FINAL STATE AND VICE VERSA

The acceptance of languages by PDA is of two types: Acceptance by final state and acceptance by null or empty store. If a PDA accepts a language by empty stack then there exists an equivalent PDA that accepts the same language by final state and vice versa. When the input string is exhausted and PDA enters into some final state then this transition is represented as

$$\delta(q_1, \wedge, Z_0) = \{(q_f, \wedge)\}, \quad q_f \in F$$

If we have to show the acceptance by empty store then the above transition can be written without changing state as

$$\delta(q_1, \wedge, Z_0) = \{(q_1, \wedge)\}, \quad q_1 \in Q$$

Theorem 7.1 If PDA A accepts language L by final state, then we can find a PDA B accepting L by empty store.

Proof Let PDA A is defined as $A = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, we construct PDA B with the help of PDA A in such a way that

- (i) by the initial move of PDA B , an initial ID (instantaneous description) of PDA A is reached.
- (ii) once PDA B reaches to initial ID of PDA A , it behaves like PDA A until a final state of A is reached, and
- (iii) when PDA B reaches a final state of PDA A , it guesses whether the input string is exhausted. Then PDA B simulates PDA A or it removes all the symbols in push down store (PDS).

The actual construction of PDA B is as follows:

$$B = (Q \cup \{q_0', q_d\}, \Sigma, \Gamma \cup \{Z_0'\}, \delta_B, q_0', Z_0', \{\phi\})$$

where q_d is a dead state, $q_d \in Q$, and Z_0' is the new push down symbol for PDS of PDA B . (Note that in PDA B , there is no final state because we have to construct PDA B such that it accepts L by empty store, not by final state.)

The transition function δ_B of PDA B is defined as

$$\begin{aligned} R_1: \delta_B(q_0', \wedge, Z_0') &= \{(q_0, Z_0 Z_0')\} \\ R_2: \delta_B(q, a, Z) &= \delta(q, a, Z) && \text{for all } a \in \Sigma, q \in Q, \text{ and } Z \in \Gamma \\ R_3: \delta_B(q, \wedge, Z) &= \delta(q, \wedge, Z) \cup \{(q_d, \wedge)\} && \text{for all } q \in F, \text{ and } Z \in \Gamma \cup \{Z_0'\} \\ R_4: \delta_B(q_d, \wedge, Z) &= \{(q_d, \wedge)\} && \text{for all } Z \in \Gamma \cup \{Z_0'\} \end{aligned}$$

By using rule R_1 for PDA B , PDA B enters to an initial ID of PDA A and the push down symbol Z_0 is placed on the top of push down store.

By using rule R_1 , PDA B can simulate PDA A until it reaches a final state of PDA A . By reaching, final state of PDA A , PDA B makes a guess whether the input string is exhausted or not. When the input string is not exhausted, PDA B once again simulates PDA B . Otherwise PDA B enters to the dead state q_d .

Rule R_4 gives a $\wedge$ -move. By using rule R_4 , PDA B removes all the symbols in push down store.

For a string w , $w \in T(A)$ if and only if PDA A reaches a final state. ($T(A)$ is the acceptance of a string by PDA A by final state.) On reaching a final state of PDA A , the PDA can reach the state q_d and removes all symbols from the push down store by $\wedge$ -moves. Thus, it is clear that $w \in T(A)$ if and only if $w \in N(B)$. ($N(B)$ is the acceptance of a string by PDA B by null or empty store). We know prove that $T(A) = N(B)$.

If $w \in T(A)$, then for some $q \in F$, $\alpha \in \Gamma^*$,

$$(q_0, w, Z_0) \vdash_A^* (q, \wedge, \alpha)$$

By using rule R_2 , we get

$$(q_0, w, Z_0) \vdash_B^* (q, \wedge, \alpha)$$

By applying *property 2* of move relation on $(q_0, w, Z_0) \vdash_B^* (q, \wedge, \alpha)$, we get

$$(q_0, w, Z_0 Z_0') \vdash_B^* (q, \wedge, \alpha Z_0') \quad (7.5)$$

From rule R_1 , we have

$$(q', \wedge, Z_0 Z') \vdash_B^* (q_0, Z_0 Z')$$

By applying *property 1* of move relation on $(q', \wedge, Z_0 Z') \vdash_B^* (q_0, \wedge, Z_0 Z')$, we get following move relation

$$(q', w, Z_0 Z') \vdash_B (q_0, w, Z_0 Z') \quad (7.6)$$

From (7.6) and (7.5), we can get

$$(q', w, Z_0 Z') \vdash_B (q_0, w, Z_0 Z') \vdash_B^* (q, \wedge, \alpha Z_0')$$

or simply we can write it as

$$(q', w, Z_0 Z') \vdash_B^* (q, \wedge, \alpha Z_0') \quad (7.7)$$

By applying rule R_1 once and R_4 repeatedly on (7.7), we get

$$(q, \wedge, \alpha Z_0') \vdash (q_d, \wedge, \wedge) \quad (7.8)$$

The relations (7.7) and (7.8) imply that $(q', w, Z_0 Z') \vdash_B^* (q_d, \wedge, \wedge)$. Hence, we can say that

$$T(A) \subseteq N(B).$$

Now we have to prove that $N(B) \subseteq T(A)$. After this proof we can say $T(A) = N(B)$.

We start with a string $w \in N(B)$. This means that for some state of PDA B ,

$$(q', w, Z_0 Z') \vdash_B^* (q_d, \wedge, \wedge) \quad (7.9)$$

The initial move of PDA B is defined by rule R_1 , and the first move of (7.9) is

$$(q', \wedge w, Z_0 Z') \vdash_B (q_0, w, Z_0 Z')$$

Z_0' in the push down store can be removed only when PDA B enters into state q_d . PDA B can enter into state q_d only when it reaches to a final state q of PDA A in an earlier step. Therefore, moves of (7.9) can be split as

$$(q', \wedge w, Z_0 Z') \vdash_B (q_0, w, Z_0 Z') \vdash_B^* (q, \wedge, \alpha Z_0') \vdash_B^* (q_d, \wedge, \wedge)$$

for some $q \in F$ and $\alpha \in \Gamma^*$. But the move relation $(q_0, w, Z_0 Z') \vdash_B^* (q, \wedge, \alpha Z_0')$ can be obtained only by application of rule R_2 . So, the moves involved here are those induced by the moves of PDA A . Since Z_0' is not a push down symbol in PDA A , it lies in the bottom of PDS. Hence,

$$(q', \wedge w, Z_0 Z') \vdash_A^* (q, \wedge, \alpha), \text{ where } q \in F.$$

Therefore, $w \in T(A)$ and $N(B) \subseteq T(A)$. This, shows that

$$L = N(B) = T(A).$$

Theorem 7.2 If PDA A accepts language L by empty store, then we can find a PDA B accepting L by final state.

The proof of this theorem is left as an exercise for the readers.

7.7

DETERMINISTIC PUSH DOWN AUTOMATA

A deterministic push down automaton is one for which every input string has a unique path through initial state to final state in the machine. A nondeterministic push down automaton is one for which, at a certain time it has to select a particular path among possible paths. It means for a particular input we have two or more transitions to go to the next state. Input string is accepted by such a machine if some set of choice leads to an ACCEPT state. If all possible paths that a certain input string can follow it is always a REJECT state, then the string will be rejected. This is analogous to the definition of acceptance of non-deterministic transition diagrams.

We shall see that non deterministic push down automata are equivalent to context free grammars (CFGs) and more powerful than deterministic one. The relative strength of NPDA, DPDA and FA is given by Fig. 7.6. It shows that NPDA is more powerful than DPDA; and DPDA is more powerful than FA (DFA and NFA both).

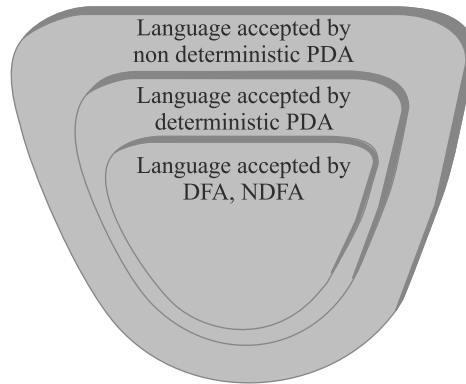


Fig. 7.6 The relative strengths of NPDA, DPDA and FA

A push down automaton $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, is said to be deterministic if for all $q \in Q, a \in \Sigma$ and $Z \in \Gamma$

- (i) $\delta(q, a, Z)$ is either empty or single move, and
- (ii) $\delta(q, \wedge, Z) = q$, it means the state is not changed on input $\wedge$.

If any of the above rules is not followed even once, the PDA is called nondeterministic.

Example 7.1

Show that a PDA M given below is deterministic.

$M = (\{q_0, q_1\}, \{0, 1, 2\}, \{0, 1, Z_0\}, \delta, q_0, Z_0, \{q_1\})$, where δ is defined as

$$\delta(q_0, 0, Z_0) = \{(q_0, 0Z_0)\}, \quad \delta(q_0, 1, Z_0) = \{(q_0, 1Z_0)\}, \quad (7.10)$$

$$\delta(q_0, 0, 0) = \{(q_0, 00)\}, \quad \delta(q_0, 1, 0) = \{(q_0, 10)\}, \quad (7.11)$$

$$\delta(q_0, 0, 1) = \{(q_0, 01)\}, \quad \delta(q_0, 1, 1) = \{(q_0, 11)\}, \quad (7.12)$$

$$\delta(q_0, 2, 0) = \{(q_1, 0)\}, \quad \delta(q_0, 2, 1) = \{(q_1, 1)\}, \quad \delta(q_0, 2, Z_0) = \{(q_1, Z_0)\} \quad (7.13)$$

$$\delta(q_1, 0, 0) = \{(q_1, \wedge)\}, \quad \delta(q_1, 1, 1) = \{(q_1, \wedge)\}, \quad (7.14)$$

$$\delta(q_1, \wedge, Z_0) = \{(q_1, Z_0)\}, \quad (7.15)$$

Solution The transition given by equation (7.10) – (7.15) have single move (*i.e.*, singleton) also $\delta(q_1, x, Z_0) = \{\phi\}$, for all $x \in \Sigma$. Therefore the given PDA is deterministic.

Unlike the design for finite state automata, in general there is no way to translate a nondeterministic push down automaton into a deterministic push down automaton. Thus, there is no deterministic push down automaton (DPDA) which recognises the language L . Nondeterministic PDAs are more powerful than deterministic PDAs.

However, we can define a similar language L' over $\Sigma = \{0, 1, \$\}$, which can be recognised by a deterministic PDA.

That is L' contains palindromes with a marker $\$$ in the middle, e.g., $01\$10 \in L'$. We define a deterministic PDA P_D' for L' :

$$P_D' = (\{q_0, q_1\}, \{0, 1, \$\}, \{0, 1, Z_0\}, \delta, q_0, Z_0, \{\phi\}),$$

where δ is defined as

$$\begin{aligned} T_1: \delta(q_0, 0, Z_0) &= \{(q_0, 0Z_0)\}, \\ T_2: \delta(q_0, 1, Z_0) &= \{(q_0, 1Z_0)\}, \\ T_3: \delta(q_0, 0, 0) &= \{(q_0, 00)\}, \\ T_4: \delta(q_0, 1, 0) &= \{(q_0, 10)\}, \\ T_5: \delta(q_0, 0, 1) &= \{(q_0, 01)\}, \\ T_6: \delta(q_0, 1, 1) &= \{(q_0, 11)\}, \\ T_7: \delta(q_0, \$, Z_0) &= \{(q_1, Z_0)\}, \quad \text{when } \$ \in L' \\ T_8: \delta(q_0, \$, 0) &= \{(q_1, 0)\}, \\ T_9: \delta(q_0, \$, 1) &= \{(q_1, 1)\}, \\ T_{10}: \delta(q_1, 0, 0) &= \{(q_1, \wedge)\}, \\ T_{11}: \delta(q_1, 1, 1) &= \{(q_1, \wedge)\}, \\ T_{12}: \delta(q_1, \wedge, Z_0) &= \{(q_1, Z_0)\} \end{aligned}$$

Now we can check whether the above PDA is deterministic.

Note If the transition T_{12} is replaced by $\delta(q_1, \wedge, Z_0) = \{(q_f, Z_0)\}$ for $P_D' = (\{q_0, q_1, q_f\}, \{0, 1, \$\}, \{0, 1, Z_0\}, \delta, q_0, Z_0, \{q_f\})$, then the PDA is nondeterministic.

7.8

NONDETERMINISTIC PUSH DOWN AUTOMATA

A nondeterministic push down automata M is defined as 7-tuples $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, where all symbols have their usual meaning as in case of deterministic PDA with only the difference of transition function. The transition function δ of nondeterministic push down automaton is defined as the mapping

$$Q \times \{\Sigma \cup \wedge\} \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$$

For example, the PDA given by following transitions is nondeterministic push down automata:

$$\delta(q_0, \wedge, E) = \{(q_0, T), (q_0, E + T)\}$$

$$\delta(q_0, \wedge, T) = \{(q_0, F), (q_0, T * F)\}$$

$$\delta(q_0, \wedge, F) = \{(q_0, a), (q_0, (E))\}$$

$$\delta(q_0, (, () = \{(q_0, \wedge)\}$$

$$\delta(q_0,), () = \{(q_0, \wedge)\}$$

$$\delta(q_0, a, a) = \{(q_0, \wedge)\}$$

$$\delta(q_0, +, +) = \{(q_0, \wedge)\}$$

$$\delta(q_0, *, *) = \{(q_0, \wedge)\}$$

The first three transitions represent that the given PDA is nondeterministic.

7.8.1 Application of Nondeterministic PDA

There exists a nondeterministic PDA for even length palindromes i.e., for language $L = \{ww^R \mid w \in (a, b)^*\}$. The PDA for language L , non-deterministically guesses the middle of the string ww^R .

Theorem 7.3 The language of palindromes $L = \{x \in \{a, b\}^* \mid x = \text{reverse}(x)\}$ cannot be accepted by any DPDA M .

Proof We assume that DPDA M accepts the language of palindrome L and then we get a contradiction. We assume DPDA M such that every move is of the form

$$\delta(q, x, X) = (q', \wedge)$$

or of the form

$$\delta(q, x, X) = (q', xX)$$

where, in both cases $x \in \{\Sigma \cup \wedge\}$. DPDA M cannot remove a symbol from the stack and place another symbol on the stack in a same move.

Next, we observe that for any string ' s ', DPDA M must eventually read every symbol of ' s '. It is adoptable, in general, that DPDA M might begin in infinite sequence of consecutive $\wedge$ -transitions, which would prevent it from processing over any more input. The reason is not that every string of a 's and b 's is a prefix of a palindrome, and by definition, a string in the language must be processed accepting the language by DPDA.

The total number of elements in push down store of DPDA M depends upon how much information M needs to remember after the processing of string ' s '. Suppose ' t ' is a string for which the stack has elements as less as possible, then

$$\delta(q_0, st, Z_0)^* = (q_t, \alpha_t)$$

for some state $q_t \in Q$, and some string $\alpha_t \in \Gamma^*$.

And for any string $t \in \Sigma^*$, any state $q \in Q$, and any string $\beta \in \Gamma^*$, if

$$\delta(q_0, st, Z_0)^* = (q, \beta)$$

then $|\beta| \geq |\alpha_t|$.

Because of our initial assumption about the moves of DPDA M , any move that involves removing a stack symbol results in a shorter stack when that move has been completed. Therefore, the result of processing the 'st' is a configuration in which no symbols currently on the stack will ever be removed subsequently.

It may also be possible that two or more different s 's correspond to the same string 'st'. But we are considering arbitrarily long strings 's', there are still infinitely many strings 'u' (in the form of 'st') having the property that processing 'u' leads DPDA M to a configuration C_u in which no symbols currently on the stack will be removed subsequently. We can select an infinity subset of strings for which the states of the corresponding configurations are all the same; and we can select further an infinite subset of strings for which the top elements in stack of corresponding configurations are also same. Therefore, there are two distinct strings u_1 and u_2 with the property that for any string $z \in \Sigma^*$,

$$\delta(q_0, u_1z, Z_0) \equiv \delta(q_u, z, X\alpha_1)$$

and

$$\delta(q_0, u_2z, Z_0) \equiv \delta(q_u, z, X\alpha_2)$$

For some $q_u \in Q$, $X \in \Gamma$, and $\alpha_1, \alpha_2 \in \Gamma^*$, and the symbol X is never removed from the push down store during the processing of string 'z'.

It follows that for any 'z', the ultimate result of processing the string ' u_1z ' is the same as the string ' u_2z '. Therefore, either both strings ' u_1z ' and ' u_2z ' are accepted or neither string is accepted. Hence, it is impossible that M accepts a language of palindromes.

7.9

EQUIVALENCE OF PDA AND CONTEXT FREE LANGUAGE

Theorem 7.4 For a language $L \subseteq \Sigma^*$ the following is equivalent:

- (a) L is derived by the context free grammar G , i.e., $L = L(G)$
- (b) L is the language of the PDA M , i.e., $L = L(M)$.

Context free languages (CFLs) can be described by context free grammars (CFGs) and can be processed by push down automata.

Now we see how to construct a PDA for a given context free grammar. Given a context free grammar $G = (V_N, \Sigma, P, S)$ generating language $L(G)$, we define a PDA $M(G)$ for CFG G

$$M(G) = (\{q_0\}, \Sigma, V_N \cup \Sigma, \delta, q_0, S, \{\phi\}) \quad (7.16)$$

where δ is defined as follows:

$$\delta(q_0, \wedge, A) = \{(q_0, \alpha) \mid A \rightarrow \alpha \in P\} \quad \text{for all } A \in V_N \quad (7.17)$$

$$\delta(q_0, a, a) = \{(q_0, \wedge)\} \quad \text{for all } a \in \Sigma \quad (7.18)$$

We have not given a set of final states because we use acceptance by empty stack. So we have used only one state q_0 .

We take an example CFG $G = (\{E, T, F\}, \{(\ ,), a, +, *\}, P, E)$ where E is start symbol and P is set of production defined as

$$P = \{E \Rightarrow T \mid E + T, T \Rightarrow F \mid T * F, F \Rightarrow a \mid (E)\}.$$

We define PDA $M(G)$ for CFG G from equation (7.16) as

$$M(G) = (\{q_0\}, \{(,), a, +, *\}, \{E, T, F, (,), a, +, *\}, \delta, q_0, E, \{\emptyset\}).$$

From equation (7.17), we get

$$\delta(q_0, \wedge, E) = \{(q_0, T), (q_0, E + T)\} \quad (7.19)$$

$$\delta(q_0, \wedge, T) = \{(q_0, F), (q_0, T * F)\} \quad (7.20)$$

$$\delta(q_0, \wedge, F) = \{(q_0, a), (q_0, (E))\} \quad (7.21)$$

From equation (7.18), we get

$$\delta(q_0, (,) = \{(q_0, \wedge)\} \quad (7.22)$$

$$\delta(q_0,),) = \{(q_0, \wedge)\} \quad (7.23)$$

$$\delta(q_0, a, a) = \{(q_0, \wedge)\} \quad (7.24)$$

$$\delta(q_0, +, +) = \{(q_0, \wedge)\} \quad (7.25)$$

$$\delta(q_0, *, *) = \{(q_0, \wedge)\} \quad (7.26)$$

How this PDA is equivalent to CFG G ? Can it accept a string $a + (a * a) \in L(G)$? We consider initial ID $(q_0, a + (a * a), E)$ to reach $(q_0, \wedge, \wedge)$ by applying the equation (7.19) to (7.26) as follows :

$$\begin{aligned} (q_0, a + (a * a), E) &\vdash (q_0, a + (a * a), E + T) \\ &\vdash (q_0, a + (a * a), T + T) \\ &\vdash (q_0, a + (a * a), F + T) \\ &\vdash (q_0, a + (a * a), a + T) \\ &\vdash (q_0, + (a * a), + T) && [\text{After applying (7.24)}] \\ &\vdash (q_0, (a * a), T) && [\text{After applying (7.25)}] \\ &\vdash (q_0, (a * a), F) \\ &\vdash (q_0, (a * a), (E)) \\ &\vdash (q_0, a * a, E)) && [\text{After applying (7.22)}] \\ &\vdash (q_0, a * a, T) \\ &\vdash (q_0, a * a, T * F)) \\ &\vdash (q_0, a * a, F * F)) \\ &\vdash (q_0, a * a, a * F)) \\ &\vdash (q_0, * a, * F)) && [\text{After applying (7.24)}] \\ &\vdash (q_0, a, F)) && [\text{After applying (7.26)}] \\ &\vdash (q_0, a, a)) \\ &\vdash (q_0,),)) && [\text{After applying (7.24)}] \\ &\vdash (q_0, \wedge, \wedge) && [\text{After applying (7.23)}] \end{aligned}$$

Therefore, $a + (a * a) \in L(G)$.

The push down automaton we have constructed is very much non-deterministic whenever we have a choice between different rules the automaton may silently choose one of the alternatives.

Example 7.2

Consider the push down automaton having no final state

$$M = (\{q_0, q_1\}, \{a, b\}, \{a, Z_0\}, \delta, q_0, Z_0, \{\emptyset\})$$

where δ is given by

$$R_1 : \delta(q_0, a, Z_0) = \{(q_0, aZ_0)\}$$

$$R_2 : \delta(q_0, a, a) = \{(q_0, aa)\}$$

$$R_3 : \delta(q_0, b, a) = \{(q_1, \wedge)\}$$

$$R_4 : \delta(q_1, b, a) = \{(q_1, \wedge)\}$$

$$R_5 : \delta(q_1, \wedge, Z_0) = \{(q_1, \wedge)\}$$

Show that the string $a^n b^n$ (for $n \geq 1$) is acceptable by PDA M by empty stack.

Solution We start with ID representing q_0 as initial state and Z_0 as the top symbol in push down store to process string $a^n b^n$ as:

$$(q_0, a^n b^n, Z_0) \vdash (q_0, a^{n-1} b^n, aZ_0).$$

$$\vdash^{n-1} (q_0, b^n, a^n Z_0)$$

$$\vdash (q_1, b^{n-1}, a^{n-1} Z_0) \quad [\text{by applying rule } R_3]$$

$$\vdash^{n-1} (q_1, \wedge, Z_0) \quad [\text{by applying rule } R_4]$$

$$\vdash (q_1, \wedge, \wedge) \quad [\text{by applying rule } R_5]$$

$\delta(q_1, \wedge, \wedge)$ represents that string $a^n b^n$ is recognised and push down store got empty. Hence, the string $a^n b^n$ is accepted by PDA M by null (empty) store.

Note that if the transitions of PDA in example 7.2 are replaced with

$$R_1 : \delta(q_0, a, Z_0) = \{(q_0, aZ_0)\}$$

$$R_2 : \delta(q_0, a, a) = \{(q_0, aa)\}$$

$$R_3 : \delta(q_0, b, a) = \{(q_0, \wedge)\}$$

$$R_4 : \delta(q_0, \wedge, Z_0) = \{(q_0, \wedge)\}$$

Then the language accepted by this PDA will be $L = \{(ab)^n \cup a^n b^n \mid n \geq 0\}$ since the sequence $ababab \dots abab$ and $aaaaaa \dots aaaaabbbbb \dots bbbb$ can be recognised by L .

Theorem 7.5 Let $L(G)$ be a context-free language generated from CFG G , then there exists a PDA M such that $L(G) = N(M)$.

Proof Suppose $\wedge \notin L(G)$ and $G = (V_N, \Sigma, P, S)$ is a context-free grammar in Greibach normal form generating the language $L(G)$. Suppose the PDA M is defined as

$$M = (\{q\}, \Sigma, V_N, \delta, q, S, \{\emptyset\}),$$

where $\delta(q, a, A)$ contains (q, γ) whenever there is a production $A \rightarrow a\gamma$ in P .

Since grammar G is in Greibach normal form (productions of the form $A \rightarrow x\alpha$, $\alpha \in V_N^*$ and $x \in \Sigma$), the PDA M simulates left most derivations of G . PDA M stores the suffix α of the left sentential form on its stack after processing the prefix 'x', we can show this derivation as,

$$S \xRightarrow[LM]{*} x\alpha \text{ if and only if } (q, x, S) \vdash_M^* (q, \wedge, \alpha) \quad (7.27)$$

First, we consider $(q, x, S) \vdash^i (q, \wedge, \alpha)$ and by using induction on i we show that $S \xRightarrow{*} x\alpha$.

The basis, $i = 0$, is trivial since $x = \wedge$ and $\alpha = S$. For the induction, suppose $i \geq 1$, and suppose $x = ya$.

Let us consider the next-to-last step,

$$(q, ya, S) \xrightarrow{i-1} (q, a, \beta) \xrightarrow{*} (q, \wedge, \alpha) \quad (7.28)$$

If we remove 'a' from the end of input string in the first 'i' instantaneous description of the sequence $(q, ya, S) \xrightarrow{i} (q, a, \beta)$, we get

$$(q, y, S) \xrightarrow{i-1} (q, \wedge, \beta)$$

Since 'a' has no effect on the moves of M until it is actually consumed from the input. According to induction hypothesis, we have

$$S \xRightarrow{*} y\beta$$

The move $(q, a, \beta) \xrightarrow{*} (q, \wedge, \alpha)$ implies that $\beta = A\gamma$ for some $A \in V_N, A \rightarrow a\eta$ is a production in grammar G and $\alpha = \eta\gamma$. Hence

$$\begin{aligned} S &\xRightarrow{*} y\beta \\ &\Rightarrow ya\eta\gamma \\ &\Rightarrow x\alpha, \end{aligned}$$

and, we conclude that $(q, x, S) \xrightarrow{i}^* (q, \wedge, \alpha)$.

Now, let us suppose $S \Rightarrow x\alpha$ is derived by left most derivation. We show by induction on i that

$$(q, x, S) \xrightarrow{i}^* (q, \wedge, \alpha).$$

The basis, $i = 0$ is again trivial. Let $i \geq 1$ and suppose that

$$S \xRightarrow{i-1} yA\gamma = ya\eta\gamma,$$

where, $x = ya$ and $\alpha = \eta\gamma$.

By using induction hypothesis, we have

$$(q, y, S) \xrightarrow{*} (q, \wedge, A\gamma) \xrightarrow{*} (q, \wedge, \alpha).$$

and this way we also have $(q, ya, S) \xrightarrow{*} (q, a, A\gamma)$. Since $A \rightarrow a\eta$ is a production in G , it follows that $\delta(q, a, A)$ contains (q, η) . Hence

$$(q, x, S) \xrightarrow{*} (q, a, A\gamma) \xrightarrow{*} (q, \wedge, \alpha)$$

follows the 'only if' part of equation(7.40).

To conclude the proof, we have only to consider the equation (7.27) with $\alpha = \wedge$, which says $S \xRightarrow{*} x$ if only if $(q, x, S) \xrightarrow{*} (q, \wedge, \wedge)$. That is $x \in L(G)$ if and only if $x \in N(M)$.

Example 7.3

Construct a push down automaton which accepts the set of all strings over $\{a, b\}$ with an equal number of a 's and b 's i.e., the language $L = \{s \in (a, b)^* \mid n_a(s) = n_b(s)\}$ by null store.

Solution Let the Constructed PDA be $M = (\{q\}, \{a, b\}, \{Z_0, a, b\}, \delta, q, Z_0, \{\phi\})$.

Since the acceptance is by null store, therefore there is no need to decide a final state. Here, we want to match the number of occurrences of a 's and b 's; we can start by storing a symbol ' a ' of input string (if the string is starting with ' a ') and continue storing until the symbol ' b ' occurs. If the top symbol in push down store is ' a ' and the current input symbol is ' b ', then top ' a ' is removed (popped) from the push down store. Thus, if ' s ' is a string of equal number of a 's and b 's, then

$$(q, s, Z_0) \xrightarrow{*} (q, \wedge, Z_0) \vdash (q, \wedge, \wedge).$$

Therefore, $s \in N(M)$. We can show that $N(M)$ is the given set of strings over $\{a, b\}$ using the following construction of δ :

$$\begin{array}{ll} T_1: \delta(q, a, Z_0) = (q, aZ_0) & \text{if string is starting with an 'a'} \\ T_2: \delta(q, b, Z_0) = (q, bZ_0) & \text{if string is starting with a 'b'} \\ T_3: \delta(q, a, a) = (q, aa) & \\ T_4: \delta(q, b, b) = (q, bb) & \\ T_5: \delta(q, a, b) = (q, \wedge) & \\ T_6: \delta(q, b, a) = (q, \wedge) & \\ T_7: \delta(q, \wedge, Z_0) = (q, \wedge) & \end{array}$$

The following is the equivalent presentation of this PDA in the form of a transition diagram:

$$(a, Z_0, aZ_0), (b, Z_0, bZ_0), (a, a, aa), (b, b, bb), (a, b, \wedge), (b, a, \wedge), (\wedge, Z_0, \wedge)$$



Fig. 7.7 Transition diagram of PDA of Example 7.3

The PDA works as following:

1. Push the first symbol of the input string, whatever it is.
2. If input symbol and top element of PDS are same then push the input symbol to PDS.
3. If input symbol and top element of PDS are different (i.e., input is 'a' and top element of PDS is 'b', or input is 'b' and top element of PDS is 'a') then pop the top element of PDS.

The basic idea for designing the required PDA is that the first input symbol of the language is pushed onto empty stack (see transitions T_1 and T_2). As the read head scans second symbol of the input, the PDA performs push operation if the input symbol and top element of push down stack are same (see transition T_3 and T_4), otherwise the PDA performs pop operation (see transition T_5 and T_6).

Thus a single 'a' pops single 'b' and a single 'b' pops single 'a' from the pushdown store. This way n number of a 's pop n number of b 's and vice versa. Therefore, if the language holds an equal number of a 's and b 's, the stack becomes empty after reading the complete input. This shows the acceptability of the required language by null store.

Note This PDA (example 7.3) also accepts language $L = \{a^n b^n \mid n \geq 0\}$ because it contains equal number of a 's and b 's. In that case transitions T_2 , T_4 , and T_5 are useless.

Example 7.4

Construct PDA M to accept the language $L = \{x \in \{a, b\}^* \mid \text{number of } a\text{'s in } x \text{ are more than number of } b\text{'s}\}$.

Solution We construct PDA M defined as

$$M = (\{q_0, q_1\}, \{a, b\}, \{Z_0, a, b\}, \delta, q_0, Z_0, \{q_1\})$$

The solution approach to this problem is almost similar to example 7.3. It reads first symbol of input and push it onto stack. The PDA reads the whole input and performs push operation if input symbol and top element of stack are same, otherwise PDA performs pop operation. This way after reading

the full input the stack does not become empty. It has at least one 'a' in PDS to show that number of a's in language are more than number of b's.

The PDA M stays in state q_0 until it decides to quit, which it does by checking that the stack has atleast one 'a' on it. In state q_0 PDA is able either to quit or to continue reading (there are strings in language L that are prefixes of longer strings in L), therefore the constructed PDA is non-deterministic. We use the current state to indicate whether there is currently a surplus of a's. We have the following transitions:

$$\begin{aligned} T_1: \delta(q_0, a, Z_0) &= (q_0, aZ_0) && \text{if string is starting with an 'a'} \\ T_2: \delta(q_0, b, Z_0) &= (q_0, bZ_0) && \text{if string is starting with a 'b'} \\ T_3: \delta(q_0, a, a) &= (q_0, aa) \\ T_4: \delta(q_0, b, b) &= (q_0, bb) \\ T_5: \delta(q_0, a, \wedge) &= (q_0, \wedge) \\ T_6: \delta(q_0, b, a) &= (q_0, \wedge) \\ T_7: \delta(q_0, \wedge, a) &= (q_1, a) && [\text{case when } n_a(x) > n_b(x)] \end{aligned}$$

The following is the equivalent presentation of this PDA in the form of a transition diagram:

$$(a, Z_0, aZ_0), (b, Z_0, bZ_0), (a, a, aa), (b, b, bb), (a, b, \wedge), (b, a, \wedge), (\wedge, a, a)$$

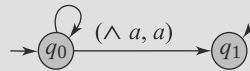


Fig 7.8 Transition diagram of PDA of Example 7.4

The deterministic PDA equivalent to this will still have the two states q_0 and q_1 , but it will enter to state q_1 whenever it reads an 'a' and the stack is empty. This move does not change the status of push down store. If it pushed an 'a', to reflect the fact that there is currently a surplus of a's, there would be no way to determine, at the point when that 'a' was about to be removed from stack, that the PDA should leave the state q_1 . The PDA will leave the state q_1 only by reading 'b' when the stack is empty except for Z_0 , that is, by reading 'b' when there is currently only one excess a, and that move also leaves the stack unchanged.

The deterministic PDA we are giving below has q_1 as final state and there is no move specified from q_1 with stack symbol 'b' or from q_0 with stack symbol 'a', because neither of these situation will ever occur.

The transitions T_1 through T_8 are of deterministic PDA.

$$\begin{aligned} T_1: \delta(q_0, a, Z_0) &= \{(q_1, Z_0)\} \\ T_2: \delta(q_0, b, Z_0) &= \{(q_0, bZ_0)\} \\ T_3: \delta(q_0, a, b) &= \{(q_0, \wedge)\} \\ T_4: \delta(q_0, b, b) &= \{(q_0, bb)\} \\ T_5: \delta(q_1, a, Z_0) &= \{(q_1, aZ_0)\} \\ T_6: \delta(q_1, b, Z_0) &= \{(q_0, Z_0)\} \\ T_7: \delta(q_1, a, a) &= \{(q_1, aa)\} \\ T_8: \delta(q_1, b, a) &= \{(q_1, \wedge)\} \end{aligned}$$

The following is the equivalent presentation of this PDA in the form of a transition diagram:

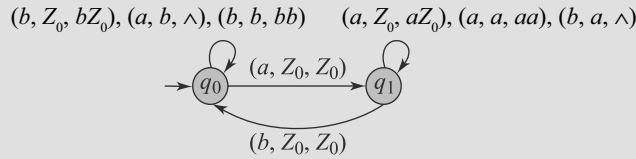


Fig. 7.9 Another transition diagram of PDA of Example 7.4

Let us take a string 'x' having more a's than b's as $x = abbabaa$. We illustrate the operation of the constructed PDA on string 'abbabaa' as

$(q_0, abbabaa, Z_0)$	$\vdash (q_1, bbabaa, Z_0)$	(by transition T_1)
	$\vdash (q_0, babaa, Z_0)$	(by transition T_6)
	$\vdash (q_0, abaa, bZ_0)$	(by transition T_2)
	$\vdash (q_0, baa, Z_0)$	(by transition T_3)
	$\vdash (q_0, aa, bZ_0)$	(by transition T_2)
	$\vdash (q_0, a, Z_0)$	(by transition T_3)
	$\vdash (q_1, \wedge, Z_0)$	(string accepted by transition T_1)

Example 7.5

Design a PDA for $L = \{s \in (a, b)^* \mid n_a(s) \neq n_b(s)\}$, i.e. the language of all strings over $\{a, b\}$ when number of a's are not equal to number of b's.

Solution The idea of designing the required PDA is almost similar to example 7.4 and 7.5. As the full input string is exhausted the stack of PDA is not empty. If the top element of stack is 'a' then number of a's in input are greater than number of b's, and if the top element of stack is 'b' then number of b's in the input are greater than number of a's. The following transitions represent the required PDA:

$$T_1: \delta(q_0, a, Z_0) = (q_0, aZ_0)$$

$$T_2: \delta(q_0, b, Z_0) = (q_0, bZ_0)$$

$$T_3: \delta(q_0, a, a) = (q_0, aa)$$

$$T_4: \delta(q_0, b, b) = (q_0, bb)$$

$$T_5: \delta(q_0, a, b) = (q_0, \wedge)$$

$$T_6: \delta(q_0, b, a) = (q_0, \wedge)$$

$$T_7: \delta(q_0, \wedge, a) = (q_f, a)$$

PDA reaches to final state q_f with $n_a(s) > n_b(s)$

$$T_8: \delta(q_0, \wedge, b) = (q_f, b)$$

PDA reaches to final state q_f with $n_b(s) > n_a(s)$

The following is the equivalent presentation of this PDA in the form of a transition diagram:

$(a, Z_0, aZ_0), (b, Z_0, bZ_0), (a, a, aa),$
 $(b, b, bb), (a, b, \wedge), (b, a, \wedge)$

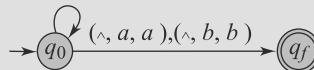


Fig. 7.10 Transition diagram of PDA of Example 7.5

Example 7.6

Construct a push down automaton M to accept the language
 $L = \{\omega 2\omega^T \mid \omega \in \{0, 1\}^*\}$
 by final state.

Solution We construct PDA M defined as

$$M = (\{q_0, q_1, q_2\}, \{0, 1, 2\}, \{0, 1, Z_0\}, \delta, q_0, Z_0, \{q_2\})$$

where δ is defined by following transitions:

$$T_1 : \delta(q_0, 0, Z_0) = \{(q_0, 0Z_0)\}$$

$$T_2 : \delta(q_0, 1, Z_0) = \{(q_0, 1Z_0)\}$$

$$T_3 : \delta(q_0, 0, 0) = \{(q_0, 00)\},$$

$$T_4 : \delta(q_0, 1, 0) = \{(q_0, 10)\},$$

$$T_5 : \delta(q_0, 0, 1) = \{(q_0, 01)\},$$

$$T_6 : \delta(q_0, 1, 1) = \{(q_0, 11)\},$$

$$T_7 : \delta(q_0, 2, 0) = \{(q_1, 0)\},$$

$$T_8 : \delta(q_0, 2, 1) = \{(q_1, 1)\},$$

$$T_9 : \delta(q_0, 2, Z_0) = \{(q_2, Z_0)\}, \quad \text{when, } 2 \in L$$

$$T_{10} : \delta(q_1, 0, 0) = \{(q_1, \wedge)\},$$

$$T_{11} : \delta(q_1, 1, 1) = \{(q_1, \wedge)\},$$

$$T_{12} : \delta(q_1, \wedge, Z_0) = \{(q_2, Z_0)\}.$$

The following is the equivalent presentation of this PDA in the form of a transition diagram:

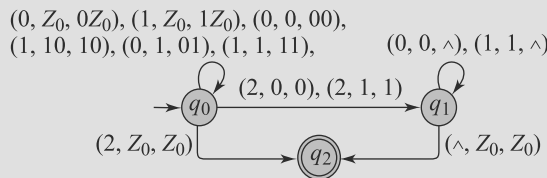


Fig 7.11 Transition diagram of PDA of Example 7.6

As the string ' ω ' is any string from $\{0, 1\}^*$ (any combination of 0's and 1's), according to transitions T_1 and T_2 , the first symbol of string is pushed onto push down store. By transition T_3 to T_6 , the remaining symbols of string ' ω ' are pushed onto push down store until the symbol '2' comes, by T_7 to T_9 , on seeing '2', the PDA M moves to state q_1 without changing the status of push down store. By T_{10} and T_{11} , the PDA M removes (deletes) the topmost symbol if it coincides with the current input symbol it means if they do not match the PDA M halts. By T_{12} the PDA M goes to final state and shows that the string $\omega 2\omega^T$ is exhausted.

Suppose the string ω over $\{0, 1\}^*$ is defined as

$$\omega = a_1 a_2 a_3 \dots a_n,$$

where each a_i is either 0 or 1. Then we have ID $(q_0, a_1 a_2 a_3 \dots a_n 2 a_n a_{n-1} \dots a_2 a_1, Z_0)$. From this ID to final ID $(q_2, \wedge, Z_0)$, we can proceed by transition rules defined through T_1 to T_{12} as follows:

$$\begin{array}{ll}
 (q_0, a_1 a_2 a_3 \dots a_n 2 a_n a_{n-1} \dots a_2 a_1, Z_0) \vdash (q_0, a_2 a_3 \dots a_n 2 a_n a_{n-1} \dots a_2 a_1, a_1 Z_0) & [\text{By } T_1] \\
 \vdash^* (q_0, 2 a_n a_{n-1} \dots a_2 a_1, a_n a_{n-1} \dots a_2 a_1 Z_0) & [\text{By } T_1 \text{ to } T_6] \\
 \vdash (q_1, a_n a_{n-1} \dots a_2 a_1, a_n a_{n-1} \dots a_2 a_1 Z_0) & [\text{By } T_7 \text{ \& } T_8] \\
 \vdash^* (q_1, \wedge, Z_0) & [\text{By } T_{10} \text{ \& } T_{11}] \\
 \vdash (q_2, \wedge, Z_0) &
 \end{array}$$

Therefore, $\omega 2 \omega^T \in T(M)$.

Example 7.7

Construct a PDA M which accepts the language $L = \{0^n 1^m 0^n \mid m, n \geq 1\}$ by null store.

Solution We construct PDA M to accept the language L by null store such that it stores 0's in stack until a 1 occurs. After storing 0's, PDA reads for 1's without changing the status of push down store. When all 1's of the input string are exhausted then stored 0's are removed (popped) from the push down store on scanning of 0's on the tape, as a result the stack is got empty (acceptance by null store). The constructed PDA is defined as

$$M = (\{q_0, q_1\}, \{0, 1\}, \{0, Z_0\}, \delta, q_0, Z_0, \{\phi\})$$

where, δ is defined by the following transition rules:

$$T_1 : \delta(q_0, 0, Z_0) = \{(q_0, 0Z_0)\}$$

$$T_2 : \delta(q_0, 0, 0) = \{(q_0, 00)\}$$

$$T_3 : \delta(q_0, 1, 0) = \{(q_1, 0)\}$$

$$T_4 : \delta(q_1, 1, 0) = \{(q_1, 0)\}$$

$$T_5 : \delta(q_1, 0, 0) = \{(q_1, \wedge)\}$$

$$T_6 : \delta(q_1, \wedge, Z_0) = \{(q_1, \wedge)\}$$

The following is the equivalent presentation of this PDA in the form of a transition diagram:

$$(0, Z_0, 0Z_0), (0, 0, 00) \quad (1, 0, 0), (0, 0, \wedge), (\wedge, Z_0, \wedge)$$

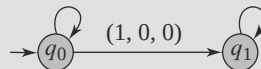


Fig 7.12 Transition diagram of PDA of Example 7.7

The transition rule T_1 stores a '0' above Z_0 in push down store, then T_2 stores 0's until 1 occurs from the input string. Now all 0's are stored in push down store until 1 occurs from the input string. By T_3 a 0 is removed from the push down store for each '0' of input string. By T_6 the push down store gets empty by removing Z_0 . This shows the acceptability of L by null store. The acceptability of string $0^n 1^m 0^n$ by PDA M can be shown as follows:

$$\begin{array}{ll}
 (q_0, 0^n 1^m 0^n, Z_0) \vdash (q_0, 0^{n-1} 1^m 0^n, 0Z_0) & (\text{By } T_1) \\
 \vdash (q_0, 1^m 0^n, 0^n Z_0) & (\text{By } T_2) \\
 \vdash (q_1, 1^{m-1} 0^n, 0^n Z_0) & (\text{By } T_3) \\
 \vdash (q_1, 0^n, 0^n Z_0) & (\text{By } T_4) \\
 \vdash (q_1, \wedge, Z_0) & (\text{By } T_5) \\
 \vdash (q_1, \wedge, \wedge) & (\text{By } T_6)
 \end{array}$$

Hence, we can say $0^n 1^m 0^n \in N(M)$.

Note If the transition T_4 is not included in example 7.7, the modified PDA becomes capable of accepting the language $L = \{0^n 10^n \mid n \geq 1\}$.

Example 7.8

Design PDA to accept language $L = \{a^n b^m c^{m+n} \mid n, m \geq 1\}$.

Solution The basic idea to design the required PDA is that as the input scanning is started all a 's and then b 's are pushed onto stack. When c 's are scanned the m number of c 's pop all b 's from the stack and remaining n number of c 's pop all the a 's (exactly n number of a 's) from the stack. As a result the input is exhausted and stack becomes empty. The following is the required transition of the PDA:

$T_1 : \delta(q_0, a, Z_0) = (q_0, aZ_0)$	first ' a ' is pushed onto stack.
$T_2 : \delta(q_0, a, a) = (q_0, aa)$	additional (i.e. $n - 1$) a 's are pushed onto stack.
$T_3 : \delta(q_0, b, a) = (q_0, ba)$	first b is pushed onto stack.
$T_4 : \delta(q_0, b, b) = (q_0, bb)$	additional (i.e. $m - 1$) b 's are pushed onto stack.
$T_5 : \delta(q_0, c, b) = (q_1, \wedge)$	first b is popped by scanning first c
$T_6 : \delta(q_1, c, b) = (q_1, \wedge)$	additional (i.e. $m - 1$) b 's are popped by scanning c 's
$T_7 : \delta(q_1, c, a) = (q_1, \wedge)$	n number of a 's are popped by scanning n number of c 's and stack finally becomes empty.
$T_8 : \delta(q_1, \wedge, Z_0) = (q_1, \wedge)$	acceptance by empty store

Following is the equivalent presentation of this PDA in the form of a transition diagram:

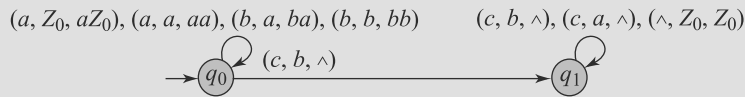


Fig. 7.13 Transition diagram of PDA of Example 7.8

Example 7.9

Design PDA for language $L = \{a^n b^{n+2} \mid n \geq 1\}$.

Solution The basic idea is that all a 's of the input string are pushed onto stack by scanning from the input tape. The n number of b 's pop n number of a 's from the stack and stack becomes empty. When second last ' b ' is scanned the PDA changes its state and by scanning last ' b ' the PDA reaches the final state. The following is the required PDA:

$$M = (\{q_0, q_1, q_2, q_3\}, \{a, b\}, \{a, Z_0\}, \delta, Z_0, q_0, \{q_3\})$$

where δ is defined as

$T_1 : \delta(q_0, a, Z_0) = (q_0, aZ_0)$	the first ' a ' is pushed onto stack.
$T_2 : \delta(q_0, a, a) = (q_0, aa)$	the remaining a 's are pushed onto stack.
$T_3 : \delta(q_0, b, a) = (q_1, \wedge)$	the first a is popped from the stack.
$T_4 : \delta(q_1, b, a) = (q_1, \wedge)$	additional a 's are popped
$T_5 : \delta(q_1, b, Z_0) = (q_2, Z_0)$	second last ' b ' is scanned.
$T_6 : \delta(q_2, b, Z_0) = (q_3, Z_0)$	the last ' b ' is scanned, and PDA reaches to final state q_3 .

The following is the equivalent presentation of this PDA in the form of a transition diagram:

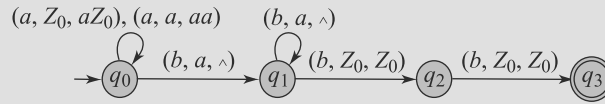


Fig 7.14 Transition diagram of PDA of Example 7.9

To show acceptability by null store or empty store we need to change T_6 by

$$\delta(q_2, \wedge, Z_0) = (q_2, \wedge) \text{ or } (q_2, Z_0)$$

for $M = (\{q_0, q_1, q_2\}, \{a, b\}, \{a, Z_0\}, \delta, Z_0, q_0, \{\emptyset\})$

Example 7.10

Design a PDA for language $L = \{a^n bc^{2n} \mid n \geq 0\}$.

Solution For designing this PDA, the basic idea is that a 's are pushed onto stack first. As the ' b ' is scanned the PDA changes its state to ensure that there is only one ' b '. The two c 's are responsible to pop one a 's from the stack (see transitions T_4 and T_5). The required PDA is

$$M = (\{q_0, q_1, q_2, q_3\}, \{a, b, c\}, \{a, Z_0\}, \delta, Z_0, q_0, \{\emptyset\})$$

where δ is defined as

$$T_0 : \delta(q_0, b, Z_0) = (q_2, Z_0) \quad [\text{in case of } n = 0]$$

$$T_1 : \delta(q_0, a, Z_0) = (q_1, aZ_0)$$

$$T_2 : \delta(q_1, a, a) = (q_1, aa)$$

$$T_3 : \delta(q_1, b, a) = (q_2, a)$$

$$T_4 : \delta(q_2, c, a) = (q_3, a)$$

$$T_5 : \delta(q_3, c, a) = (q_2, \wedge) \quad [\text{pop operation on scanning of every second } b]$$

$$T_6 : \delta(q_2, \wedge, Z_0) = (q_2, \wedge) \quad [\text{Acceptance by null store}]$$

The following is the equivalent presentation of this PDA in the form of a transition diagram:

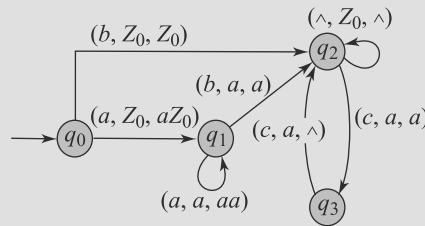


Fig 7.15 Transition diagram of PDA of Example 7.10

The acceptability of language L is by null store. To show acceptability by final state we need to replace transition T_6 by

$$\delta(q_2, \wedge, Z_0) = (q_f, Z_0), \text{ for } M = (\{q_0, q_1, q_2, q_3, q_f\}, \{a, b, c\}, \{a, Z_0\}, \delta, Z_0, q_0, \{q_f\}).$$

Example 7.11 Design a PDA for language $L = \{a^n b^{3n} \mid n \geq 0\}$.

Solution The given language can also be treated as $L = \{a^n (bbb)^n \mid n \geq 0\}$. The basic idea for designing this PDA is that all a 's are pushed onto stack after scanning and when b 's are scanned, a group of three b 's pops one ' a '. This way $3n$ number of b 's pop n number of a 's and the input is exhausted. The required PDA is

$$M = (\{q_0, q_1, q_2, q_3, q_f\}, \{a, b\}, \{a, Z_0\}, \delta, Z_0, q_0, \{q_f\}).$$

where δ is defined as

$$T_1 : \delta(q_0, \wedge, Z_0) = (q_f, Z_0)$$

Terminating transition if $\wedge \in L$ (a case for $n = 0$), $q_f \in F$.

$$T_2 : \delta(q_0, a, Z_0) = (q_0, aZ_0)$$

First ' a ' is pushed onto stack.

$$T_3 : \delta(q_0, a, a) = (q_0, aa)$$

Additional a 's are pushed onto stack.

$$T_4 : \delta(q_0, b, a) = (q_1, a)$$

First ' b ' is scanned from a group of ' bbb '.

$$T_5 : \delta(q_1, b, a) = (q_2, a)$$

Second ' b ' scanned from a group of ' bbb '.

$$T_6 : \delta(q_2, b, a) = (q_3, \wedge)$$

Third ' b ' from group of three b 's popping one ' a '.

$$T_7 : \delta(q_3, b, a) = (q_1, a)$$

First ' b ' is scanned from next group of ' bbb '.

$$T_8 : \delta(q_3, \wedge, Z_0) = (q_f, \wedge)$$

The input string is exhausted and accepted by final state

Following is the equivalent presentation of this PDA in the form of transition diagram:

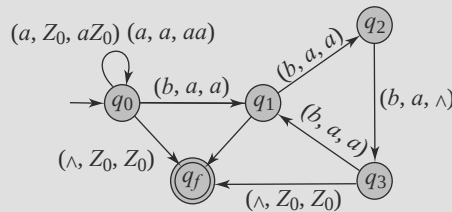


Fig 7.16 Transition diagram of PDA of Example 7.11

This PDA shows acceptability by final state. To show acceptability by null (empty) store we need to replace transition T_1 and T_8 by

$$T_1 : \delta(q_0, \wedge, Z_0) = (q_0, Z_0) \text{ and } T_8 : \delta(q_3, \wedge, Z_0) = (q_3, Z_0)$$

respectively, for $M = (\{q_0, q_1, q_2, q_3\}, \{a, b\}, \{a, Z_0\}, \delta, q_0, Z_0, \{\phi\})$.

Example 7.12 Design PDA for language of all palindromes over $\{a, b\}$ i.e.,
 $L = \{s \in (a, b)^* \mid s = \text{reverse}(s)\}$

Solution One way to design PDA for context free languages is that first we find a context free grammar corresponding to a given CFL and then design a PDA corresponding the CFG as described in section 7.7

The context free grammar for all palindromes over $\{a, b\}$ is.

$$G = (\{S\}, \{a, b\}, P, S),$$

where,

$$P = \{S \rightarrow aSa \mid bSb \mid a \mid b \mid \wedge\}$$

The PDA corresponding to CFG G is

$$M(G) = (\{q_0\}, \{a, b\}, \{a, b, S\}, \delta, q_0, S)$$

where δ is defined as

$$\delta(q_0, \wedge, S) = \{(q_0, aSa), (q_0, bSb), (q_0, a), (q_0, b), (q_0, \wedge)\}$$

$$\delta(q_0, a, a) = (q_0, \wedge)$$

$$\delta(q_0, b, b) = (q_0, \wedge)$$

The following is the equivalent presentation of this PDA in the form of a transition diagram:

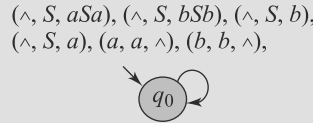


Fig 7.17 Transition diagram of PDA of Example 7.12

This PDA shows acceptability by null store or empty store, and is nondeterministic in nature.

Example 7.13

Design PDA for $L = \{a^i b^j c^k \mid i \neq j \text{ or } j \neq k\}$.

Solution The solution to this problem is in two parts. In the first part the PDA compares a 's with b 's and reaches to final state by ensuring either $i > j$ or $i < j$. In second part the PDA compares b 's with c 's and reaches to final state by ensuring either $k > j$ or $k < j$. The following are the transitions of the designed PDA:

$$T_1 : \delta(q_0, a, Z_0) = (q_0, aZ_0)$$

$$T_2 : \delta(q_0, a, a) = (q_0, aa)$$

$$T_3 : \delta(q_0, b, a) = (q_1, \wedge), (q_2, b)$$

$$T_4 : \delta(q_1, b, a) = (q_1, \wedge)$$

$$T_5 : \delta(q_1, b, Z_0) = (q_f, Z_0)$$

$$T_6 : \delta(q_1, c, a) = (q_f, a)$$

$$T_7 : \delta(q_2, b, b) = (q_2, bb)$$

$$T_8 : \delta(q_2, c, b) = (q_2, \wedge)$$

$$T_9 : \delta(q_2, c, a) = (q_f, a)$$

$$T_{10} : \delta(q_2, \wedge, b) = (q_f, b)$$

the first ' a ' is pushed onto stack.

the remaining a 's are pushed onto stack.

first b is scanned

second last ' b ' is scanned.

$i < j$, PDA reaches to final state q_f .

$i > j$, PDA reaches to final state q_f .

$k > j$, PDA reaches to final state q_f .

$j > k$, PDA reaches to final state q_f .

The following is the equivalent presentation of this PDA in the form of a transition diagram:

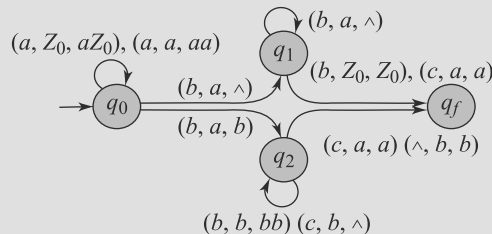


Fig 7.18 Transition diagram of PDA of Example 7.13

7.10

PDA AND REGULAR LANGUAGES

As we know a regular language is the simplest form of a formal language which can be accepted by the least powerful machine called finite automata. The pushdown automata is more powerful than finite automata, therefore pushdown automata must also accept a regular language.

Basically, a pushdown automaton is nothing but the basic model of finite automata with additional storage in the form of stack called pushdown store. If the pushdown store in the PDA is ignored then the PDA simulates the behavior of finite automata.

If we wish to design a PDA for a regular language then use of push down store is optional. For this purpose the PDA reads the input and when input is exhausted the PDA reaches the final state. The input after scanning can be pushed on to stack but it has no further utilisation.

Example 7.14

Design a PDA for $L = \{a^n \mid n \geq 0\}$.

Solution The language $L = \{a^n \mid n \geq 0\}$ is a regular language which is equivalent to regular expression a^* . There are two solutions to this problem.

(i) **No use of pushdown store** In this approach, the input string is scanned and the PDA reaches the final state as the input is exhausted. The following transitions suggest this solution:

$$T_1 : \delta(q_0, a, Z_0) = (q_0, Z_0) \text{ All } a\text{'s are scanned}$$

$$T_2 : \delta(q_0, \wedge, Z_0) = (q_f, Z_0) \text{ PDA reaches final state.}$$

When, $\wedge \in L$ the only transition used is T_2 .

(ii) **Use of pushdown store** In this approach, the input string is scanned and all symbols of input are pushed onto stack one by one. No deletion (pop operation) is done. When input is exhausted the PDA reaches the final state. The following transitions suggest this solution:

$$T_1 : \delta(q_0, a, Z_0) = (q_0, aZ_0) \quad \text{all } a\text{'s are scanned}$$

$$T_2 : \delta(q_0, \wedge, a) = (q_f, a) \quad \text{accepted if length of input is one or more.}$$

$$T_3 : \delta(q_0, \wedge, Z_0) = (q_f, Z_0) \quad \text{accepted if length is zero i.e., } \wedge \in L.$$

In both cases, acceptance is by final state.

7.11

EQUIVALENCE OF PDA AND CONTEXT FREE GRAMMAR

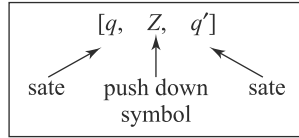
Theorem 7.6 If there is a PDA $A = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, then there exists a context-free grammar G such that the language generated by grammar G (say $L(G)$) is equivalent to the set accepted by PDA A by null store (say $N(A)$), i.e., $L(G) = N(A)$.

Proof First we construct CFG G equivalent to the PDA A , then we prove that the sets accepted by the PDA A by null store are actually the context-free languages.

Step 1 We define context-free grammar $G = (V_N, \Sigma, P, S)$, where

$$V_N = \{S\} \cup \{[q, Z, q'] \mid q, q' \in Q, \text{ and } Z \in \Gamma\}.$$

It means that any element of V_N is either the symbol S (which acts as start symbol) for grammar G , or an ordered triple whose first and third elements are states and the middle (second) element is a push down symbol as illustrated below.



The productions P in grammar G are induced by moves of PDA. P is defined by the following rules:

- R_1 : S -productions are given by $S \rightarrow [q_0, Z_0, q]$ for all states $q \in Q$
- R_2 : Each move removing a symbol from push down store is given by $(q', \wedge) \in \delta(q, a, Z)$ which induces the production $[q, Z, q'] \rightarrow a$.
- R_3 : A move that does not remove a symbol from stack is given by $(q_1, Z_1 Z_2 Z_3 \dots Z_m) \in \delta(q, a, Z)$ induces several productions of the form $[q, Z, q'] \rightarrow a[q_1, Z_1, q_2][q_2, Z_2, q_3] \dots [q_m, Z_m, q']$, for each of the states $q_2, q_3, q_4, \dots, q_m, q' \in Q$.

Therefore, each move (transition) induces several productions by rule R_3 .

Note The total number of elements in V_N except S , depend upon total number of states and total number of push down symbols in the given PDA.

Step 2 Now we prove that $L(G) = N(A)$. Note that a variable $[q, Z, q']$ in V_N indicates the present (current) state q of the PDA and top element in the push down store; also it shows the state q' in such a way that the push down store will get empty finally. This is corresponding to rule R_2 as it replaces a variable by a terminal.

We have an auxiliary result

$$[q, Z, q'] \xRightarrow{*}_G s \quad (7.29)$$

if and only if

$$(q, s, Z) \vdash (q', \wedge, \wedge) \quad (7.30)$$

We can prove the 'if' part by using mathematical induction on the number of steps in equation (7.28). If $(q, s, Z) \vdash (q', \wedge, \wedge)$, then 's' is either $a \in \Sigma$ or $\wedge$. So we have

$$\delta(q, s, Z) = (q', \wedge).$$

by rule R_2 , we get a production $[q, Z, q'] \rightarrow s$. Therefore, $[q, Z, q'] \Rightarrow s$.

Thus there is a basis for induction. Let us assume the result in equation (7.28) for k moves, then

$$(q, s, Z) \vdash^k (q', \wedge, \wedge).$$

this can be split as

$$(q, as', Z) \vdash ((q_1, s', Z_1 Z_2 Z_3 \dots Z_m) \vdash^{k-1} (q', \wedge, \wedge) \quad (7.31)$$

where $s = as'$ and $a \in (\Sigma \cup \{\wedge\})$.

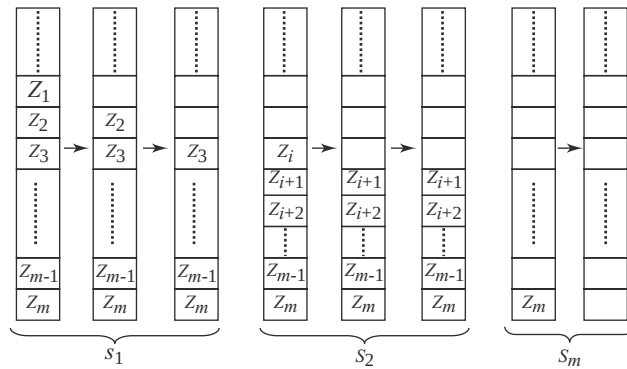


Fig. 7.19 Status of PDS (stack) during PDA operations

Let us consider the second part of (7.31). The push down store has $Z_1Z_2Z_3\dots Z_m$ initially and by applying string 's', the push down store is got empty. Each move of PDA can either remove the top symbol from push down store or replace the top symbol of push down store by some non-empty terminal string. Therefore, several moves are required to get Z_2 as the top element of push down store. Let ' s_1 ' be the prefix of string 's' such that the pushdown store has $Z_1Z_2Z_3\dots Z_m$ after applying ' s_1 '. Note that elements $Z_2Z_3\dots Z_m$ are not disturbed during application of ' s_1 '. Suppose ' s_i ' is a substring of 's' such that the push down store has $Z_{i+1}Z_{i+2}Z_{i+3}\dots Z_m$ on application of s_i . In this case $Z_{i+1}Z_{i+2}Z_{i+3}\dots Z_m$ is not disturbed while applying $s_1, s_2, s_3, \dots, s_i$. The Fig. 7.19 shows the changes in the status of push down store during the application of $s_1, s_2, s_3, \dots, s_i$.

In terms of instantaneous descriptions (IDs), we can show that

$$(q_i, s_i, Z_i) \vdash (q_{i+1}, \wedge, \wedge) \quad \text{for } i = 1, 2, 3, \dots, m, \text{ and } q_{m+1} = q' \quad (7.32)$$

Each move above requires less than k steps, by using induction hypothesis, we have

$$[q_i, Z_i, q_{i+1}] \xRightarrow{*}_G s_i \text{ for } i = 1, 2, 3, \dots, m, \quad (7.33)$$

The first part of (7.31) is given by $(q_1, Z_1Z_2Z_3\dots Z_m) \in \delta(q, a, Z)$. By using rule R_3 , we get the production

$$[q, Z, q'] \Rightarrow a[q_1, Z_1, q_2][q_2, Z_2, q_3]\dots[q_m, Z_m, q_{m+1}] \quad (7.34)$$

where $q_{m+1} = q'$. From equations (7.33) and (7.34), we have

$$[q, Z, q'] \Rightarrow as_1s_2s_3\dots s_m = s$$

According to the principle of induction

$$[q, Z, q'] \xRightarrow{*}_G s \text{ implies } (q, s, Z) \vdash (q', \wedge, \wedge)$$

Let us prove the 'only if' part by mathematical induction on the number of steps in the derivation $[q, Z, q'] \Rightarrow s$. Suppose $[q, Z, q'] \Rightarrow s$ is true, then $[q, Z, q'] \rightarrow s$ must be a production in P . This production is obtained by rule R_2 . Therefore, $s = \wedge$ or $s \in \Sigma$ and $(q', \wedge) \in \delta(q, s, Z)$. This gives the transition $(q, s, Z) \vdash (q', \wedge, \wedge)$. Thus, there is basis for induction.

Let us assume the result for derivation when the number of steps is less than k . Consider the derivation $[q, Z, q'] \xRightarrow{k} s$. This can be split as

$$[q, Z, q'] \Rightarrow a[q_1, Z_1, q_2][q_2, Z_2, q_3] \dots [q_m, Z_m, q'] \xRightarrow{k-1} s. \quad (7.35)$$

As we know that the grammar G is context-free, so we can write $s = as_1s_2s_3\dots s_m$, where

$$[q_i, Z_i, q_{i+1}] \xRightarrow{*}_G s_i, \text{ for } i = 1, 2, 3, \dots, m, \\ \text{for } q_{m+1} = q'. \text{ By induction hypothesis, we have} \\ (q_i, s_i, Z_i) \vdash^* (q_{i+1}, \wedge, \wedge) \text{ for } i = 1, 2, 3, \dots, m, \quad (7.36)$$

By applying property 1 and property 2 of move relation on equation (7.36), we get

$$(q_i, s_i, Z_i Z_{i+1} Z_{i+2} \dots Z_m) \vdash^* (q_{i+1}, \wedge, Z_{i+1} Z_{i+2} \dots Z_m) \quad (7.37)$$

and

$$(q_i, s_1 s_2 s_3 \dots s_m, Z_1 Z_2 Z_3 \dots Z_m) \vdash^* (q_{i+1}, s_{i+1} s_{i+2} s_{i+3} \dots s_m, Z_{i+1} Z_{i+2} \dots Z_m) \quad (7.38)$$

By combining the moves of equations (7.37) and (7.38), we get

$$(q_1, s_1 s_2 s_3 \dots s_m, Z_1 Z_2 Z_3 \dots Z_m) \vdash^* (q', \wedge, \wedge) \quad (7.39)$$

The first step in the derivation given by (7.35) is induced by $(q_1, Z_1 Z_2 Z_3 \dots Z_m) \in \delta(q, a, Z)$. The moves corresponding to this are

$$(q, a, Z) \vdash^* (q_1, \wedge, Z_1 Z_2 Z_3 \dots Z_m) \quad (7.40)$$

By applying property 1 of move relation on (7.38), we get

$$(q, a s_1 s_2 s_3 \dots s_m, Z) \vdash^* (q_1, s_1 s_2 s_3 \dots s_m, Z_1 Z_2 Z_3 \dots Z_m) \quad (7.41)$$

From (7.39) and (7.41), we get

$$(q, s, Z) \vdash^* (q', \wedge, \wedge).$$

According to the principle of induction, $[q, Z, q'] \xRightarrow{*}_G s$ implies $(q, s, Z) \vdash^* (q', \wedge, \wedge)$ which is the auxiliary result.

We say $s \in L(G)$ if and only if $S \xRightarrow{*} s$. Also,

$$S \xRightarrow{*} s \text{ if and only if } S \Rightarrow [q_0, Z_0, q'] \xRightarrow{*} s. \text{ (for some } q' \text{ by rule } R_1)$$

and

$$S \Rightarrow [q_0, Z_0, q'] \xRightarrow{*} s \text{ if and only if } (q_0, s, Z_0) \vdash^* (q', \wedge, \wedge) \text{ (by auxiliary result).}$$

We say $(q_0, s, Z_0) \vdash^* (q', \wedge, \wedge)$ if and only if $s \in N(A)$.

Hence, we can say that

$$N(A) = L(G).$$

This proves the theorem.

Example 7.15

Construct a context-free grammar G that accepts the set accepted by PDA M by null store (i.e., $N(M)$), where,

$$M = (\{q_0, q_1\}, \{a, b\}, \{Z_0, Z\}, \delta, q_0, Z_0, \{\emptyset\})$$

δ is given by

$$\begin{aligned}\delta(q_0, b, Z_0) &= \{(q_0, ZZ_0)\} \\ \delta(q_0, \wedge, Z_0) &= \{(q_0, \wedge)\} \\ \delta(q_0, a, Z) &= \{(q_1, Z)\} \\ \delta(q_0, b, Z) &= \{(q_0, ZZ)\} \\ \delta(q_1, a, Z_0) &= \{(q_0, Z_0)\} \\ \delta(q_1, b, Z) &= \{(q_1, \wedge)\}.\end{aligned}$$

Solution Suppose the context-free grammar G is defined as $G = (V_N, \Sigma, P, S)$ where, $\Sigma = \{a, b\}$, (because there are only terminals 'a' and 'b' in the transition of PDA M)

$$V_N = \{S, [q_0, Z_0, q_0], [q_0, Z_0, q_1], [q_0, Z, q_0], [q_0, Z, q_1], [q_1, Z_0, q_0], [q_1, Z_0, q_1], [q_1, Z, q_0], [q_1, Z, q_1]\}.$$

Note that the elements in V_N , except S , denoted by a triple are constructed by using all combinations of q_0 and q_1 (e.g., $q_0q_0, q_0q_1, q_1q_0, q_1q_1$) with Z_0 or Z in middle. Thus, there are $2^m \times n + 1$ variable in V_N (including S), if there are m unique states and n unique push down symbols.

The set of production P includes

$$\left. \begin{aligned}P_1 : S &\rightarrow [q_0, Z_0, q_0] \\ P_2 : S &\rightarrow [q_0, Z_0, q_1]\end{aligned} \right\} \quad (\text{by Rule } R_1)$$

The transition $\delta(q_0, b, Z_0) = \{(q_0, ZZ_0)\}$ gives productions

$$\left. \begin{aligned}P_3 : [q_0, Z_0, q_0] &\rightarrow b[q_0, Z, q_0] [q_0, Z_0, q_0] \\ P_4 : [q_0, Z_0, q_0] &\rightarrow b[q_0, Z, q_1] [q_1, Z_0, q_0] \\ P_5 : [q_0, Z_0, q_1] &\rightarrow b[q_0, Z, q_0] [q_0, Z_0, q_1] \\ P_6 : [q_0, Z_0, q_1] &\rightarrow b[q_0, Z, q_1] [q_1, Z_0, q_1]\end{aligned} \right\} \quad (\text{by Rule } R_3)$$

The transition $\delta(q_0, \wedge, Z_0) = \{(q_0, \wedge)\}$ gives production

$$P_7 : [q_0, Z_0, q_0] \rightarrow \wedge \quad (\text{by Rule } R_2)$$

The transition $\delta(q_0, b, Z) = \{(q_0, ZZ)\}$ gives productions

$$\left. \begin{aligned}P_8 : [q_0, Z, q_0] &\rightarrow b[q_0, Z, q_0] [q_0, Z_0, q_0] \\ P_9 : [q_0, Z, q_0] &\rightarrow b[q_0, Z, q_1] [q_1, Z_0, q_0] \\ P_{10} : [q_0, Z, q_1] &\rightarrow b[q_0, Z, q_0] [q_0, Z_0, q_1] \\ P_{11} : [q_0, Z, q_1] &\rightarrow b[q_0, Z, q_1] [q_1, Z_0, q_1]\end{aligned} \right\} \quad (\text{by Rule } R_3)$$

The transition $\delta(q_0, a, Z) = \{(q_1, Z)\}$ gives production

$$\left. \begin{aligned}P_{12} : [q_0, Z, q_0] &\rightarrow a[q_1, Z, q_0] \\ P_{13} : [q_0, Z, q_1] &\rightarrow a[q_1, Z, q_1]\end{aligned} \right\} \quad (\text{by Rule } R_3)$$

The transition $\delta(q_1, b, Z) = \{(q_1, \wedge)\}$ gives production

$$P_{14} : [q_1, Z, q_1] \rightarrow b \quad (\text{by Rule } R_2)$$

The transition $\delta(q_1, a, Z_0) = \{(q_0, Z_0)\}$ gives productions

$$\left. \begin{aligned}P_{15} : [q_1, Z_0, q_0] &\rightarrow a[q_0, Z, q_0] \\ P_{16} : [q_1, Z_0, q_1] &\Rightarrow a[q_0, Z, q_1]\end{aligned} \right\} \quad (\text{by Rule } R_3)$$

The productions P_1 to P_{16} are in P . These productions may have some useless symbols so we can reduce some symbols and then some productions to get reduced grammar by applying the theorem given in Chapter 6.

7.12

TWO STACK PDA

Up to this point we have seen that neither finite automata nor push down automata are the general models of a computer, since they can not recognise simple languages like $\{a^n b^n c^n \mid n \geq 0\}$. Finite automata were capable of recognising languages like $\{a^n \mid n \geq 0\}$. Then we added some auxiliary memory (in the form of stack) to make it a more powerful machine (called push down automaton) capable of recognising the languages like $\{a^n b^n \mid n \geq 0\}$. A push down automaton recognising $\{a^n b^n \mid n \geq 0\}$ uses stack to store a^n on push down store, then b^n on input tape is matched with a^n on push down store. This way, a PDA recognises the languages of the form $\{a^n b^n \mid n \geq 0\}$.

If we add one extra stack (push down store) to a finite automaton, definitely the power of finite automaton increases. What would happen if we added two push down stores or three, or twenty in a finite automaton? The addition of a single push down store results in the push down automaton. But what will we call a finite automaton which has more than one push down stores?

A two-stack push down automaton (also called two-push down stack machine or two-push down store machine) is like a push down automaton except that it has two push down stores, $STACK_1$ and $STACK_2$. When we wish to push a symbol ' a ' onto a $STACK$, element from a stack for the purpose of two options, we will have to specify which stack, either POP_1 or POP_2 . The functions of state $START$, $READ$, $ACCEPT$ and $REJECT$ remain the same. The input string is placed on the same read-only input tape. One important difference is that we shall insist that a two-stack PDA (abbreviated as 2-PDA) be deterministic, that is, branching will only occur at the $READ$ and POP states and there will be at most one edge from any state for any given symbol.

To understand the functioning of 2-PDA, we are defining the transition function δ of 2-PDA as

$$Q \times (\Sigma \cup \{\wedge\}) \times \Gamma \times \Gamma \rightarrow \text{finite subset of } Q \times \Gamma^* \times \Gamma^*$$

This transition function states that the move in a two-stack PDA depends on the tops of the two stacks and results in new values being pushed on two stacks. The class of a two-stack automaton is equivalent to the class of a Turing machine.

As we have designed 2-PDA to be deterministic, we can not be sure whether they are even as powerful as ordinary PDAs. In other words we can not be sure that they can accept every context-free language because the deterministic push down automata cannot.

We shall see that 2-PDAs are more powerful than PDAs, and they can accept all context-free languages and some other languages that are non-context-free. We shall also see how a 2-PDA is equivalent to a Turing Machine (TM). which is known as **Minsky's Theorem**.

The strength of a push down automaton can be increased by adding additional (extra) stacks. A 2-PDA (two-stack PDA) can accept several languages that a PDA cannot. Are there languages that a 2-PDA cannot accept? Is a 3-PDA stronger than a 2-PDA? Is a nondeterministic 2-PDA stronger than a deterministic 2-PDA? Which is stronger, a 2-PDA or a Turing machine?

These types of questions, at this point, could become very confusing. However, many of these questions are answered by a theorem given by Marvin Minsky in 1961 called Minsky's Theorem.

Theorem 7.7 (Minsky's Theorem) A language accepted by a two-stack PDA (2-PDA) can also be accepted by some Turing machine and any language accepted by a Turing machine (TM) can also be accepted by some 2-PDA (i.e., $2PDA = TM$).

Generalisation of Minsky's Model The generalised model is equivalent to a Turing machine. If we add one additional auxiliary memory in the form of stack to a PDA, then it becomes capable of recognising the languages of the form $\{a^n b^n c^n \mid n \geq 0\}$, $\{a^n b^n c^n d^n \mid n \geq 0\}$, $\{a^n b^n c^n d^n e^n \mid n \geq 0\}$, ..., and so on.

If the machine (two stack PDA) has to recognise the languages of the form $\{a^n b^n c^n \mid n \geq 0\}$, it uses first stack S_1 to remember how many a 's occur before the first b . The second stack S_2 is used to remember how many b 's occur before the first c occurs, and then pop the top element from both stacks S_1 and S_2 on every scanning of c on the input tape. The one ' c ' pops the top element from both the stacks S_1 and S_2 . This way n number of c 's pop n number of a 's (from S_1) and n number of b 's (from S_2). If at the end of string both stacks S_1 and S_2 are empty, the language $\{a^n b^n c^n \mid n \geq 0\}$ is recognised by the machine (see Fig. 7.20).

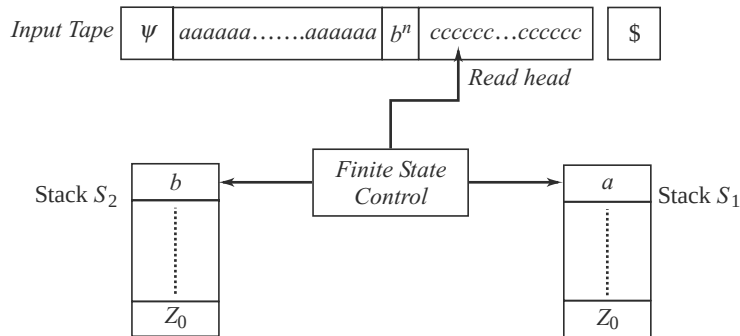


Fig. 7.20 The Minsky's model (two stack PDA)

The model of 2-PDA (equivalent Turing machine) shown in Fig. 7.20 is also capable of recognising the languages like $\{a^n b^n c^n d^n \mid n \geq 0\}$, $\{a^n b^n c^n d^n e^n \mid n \geq 0\}$, ..., and so on. Needless to say that model becomes an ordinary PDA if one stack either S_1 or S_2 is removed, and this model becomes equivalent to a finite automaton if both the stacks S_1 and S_2 are removed. If this model is expected to recognise the languages of the form $\{a^n \mid n \geq 0\}$, $\{a^n b^m \mid n, m \geq 0\}$, $\{a^n b^m c^l \mid n, m, l \geq 0\}$ i.e., regular languages, then there is no need to use any of the stacks S_1 and S_2 . If the model is expected to recognise the languages of the form $\{a^n b^n \mid n \geq 0\}$ then we can use any one stack either S_1 or S_2 .

As far as the efficiency of this model is concerned, to recognise the languages of the form $\{a^n b^n c^n d^n \mid n \geq 0\}$, the model uses first stack S_1 to store a^n , on scanning of b 's from b^n , stack S_1 empties a 's and b 's are pushed on to stack S_2 simultaneously. When the scanning of b 's is complete the stack S_1 gets empty and stack S_2 contains b^n . When c 's are scanned, the b 's popped from S_2 and c 's are pushed in to stack S_1 simultaneously. And finally d 's from d^n are scanned by emptying the stack S_1 filled with c^n .

The transitions of 2-stack PDA for $L = \{a^n b^n c^n d^n \mid n \geq 0\}$ can be written as:

$$\begin{aligned} T_1 : \delta(q_0, a, Z_1, Z_2) &= (q_0, aZ_1, Z_2) \\ T_2 : \delta(q_0, a, a, Z_2) &= (q_0, aa, Z_2) \\ T_3 : \delta(q_0, b, a, Z_2) &= (q_0, \wedge, bZ_2) \\ T_4 : \delta(q_0, b, a, b) &= (q_0, \wedge, bb) \\ T_5 : \delta(q_0, c, Z_1, b) &= (q_0, cZ_1, \wedge) \\ T_6 : \delta(q_0, c, c, b) &= (q_0, cc, \wedge) \\ T_7 : \delta(q_0, d, c, Z_2) &= (q_0, \wedge, Z_2) \\ T_8 : \delta(q_0, \wedge, Z_1, Z_2) &= (q_0, Z_1, Z_2) \end{aligned}$$

The transitions T_1 and T_2 push a 's onto stack S_1 , the transitions T_3 and T_4 push b 's onto stack S_2 by popping a 's (ensuring equal number of a 's and b 's) from stack S_1 simultaneously. The transition T_5 and T_6 push c 's of stack S_1 and pop b 's (ensuring equal number of a 's, b 's and c 's) from stack S_2 simultaneously. The transition T_7 pops c 's from stack S_1 on scanning of d 's from d^n (ensuring equal number of a 's, b 's, c 's and d 's) by getting empty the stack S_1 . When the full input string is exhausted, both the stacks get empty to ensure equal occurrences of a 's, b 's, c 's and d 's in L . The transition T_8 shows the acceptability of L by final state.

❖ Did you Know ?

Marvin Minsky, the Turing Award winner of 1969, is well known for solving Emil Post's problem of 'Tag' and Minsky's theorem. He has made his contribution in the domains of computer graphics, symbolic and mathematical computation, knowledge representation (a field of AI), computational semantics, machine perception, and connectionist learning. He is listed on Google directory as one of the all time top 6 people in the field of Artificial Intelligence. He has also been involved with advanced technologies for exploration of space.

In 1951 he built the first randomly wired neural network learning machine (called SNARC, for Stochastic Neural-Analog Reinforcement Computer). It is based on the underpinning of simulated synaptic transmission coefficients. In 1956, as a Junior Fellow at Harvard, he invented and built the first Confocal Scanning Microscope (CSM). A CSM is an optical instrument with extraordinary resolution and image quality. In the beginning of 1970s, Minsky and Papert began formulating a theory called 'The Society of Mind' that shared insights from maturity of child psychology and their experience with research on Artificial Intelligence.

Example 7.16

Design 2-stack PDA for language $L = \{a^n b^n c^n \mid n \geq 0\}$.

Solution A PDA having only one stack is not capable for recognising context sensitive languages like $L = \{a^n b^n c^n \mid n \geq 0\}$. For this purpose we use 2-stack PDA. The basic idea for designing 2-stack PDA for $L = \{a^n b^n c^n \mid n \geq 0\}$ is that the a 's in the starting of the input are pushed onto $STACK_1$ by scanning one by one from the input tape. The b 's in the middle part of the language are scanned after all a 's are scanned. All b 's are pushed onto $STACK_2$ after scanning. When the scanning of c 's is started, one ' c ' pops the top element from both the stacks (' a ' is popped from $STACK_1$ and ' b ' is popped from $STACK_2$). This way n number of c 's pop n number of a 's and b 's and as a result when the input is exhausted both the stacks get empty. The following transitions show the required 2-stack PDA:

$$T_0: \delta(q_0, \wedge, Z_1, Z_2) = (q_f, Z_1, Z_2) \quad [\text{acceptance for } n = 0]$$

$$T_1: \delta(q_0, a, Z_1, Z_2) = (q_0, aZ_1, Z_2)$$

$$T_2: \delta(q_0, a, a, Z_2) = (q_0, aa, Z_2)$$

$$T_3: \delta(q_0, b, a, Z_2) = (q_0, a, bZ_2)$$

$$T_4: \delta(q_0, b, a, b) = (q_0, a, bb)$$

$$T_5: \delta(q_0, c, a, b) = (q_1, \wedge, \wedge)$$

$$T_5: \delta(q_1, c, a, b) = (q_1, \wedge, \wedge)$$

$$T_6: \delta(q_1, \wedge, Z_1, Z_2) = (q_f, Z_1, Z_2) \quad [\text{acceptance for } n = 0]$$

for $M = (\{q_0, q_1, q_f\}, \{a, b, c\}, \{a, b, Z_1, Z_2\}, \delta, Z_1, Z_2, q_0, \{q_f\})$ where δ is defined by transition T_0 to T_6 above.

7.13

AUXILIARY PUSH DOWN AUTOMATA

Auxiliary push down automata, are push down automata with two tapes and a stack. The interesting characteristic of auxiliary push down automata (APDA) is that for a fixed amount of extra storage, the deterministic and non-deterministic versions are equivalent in language-recognising power, and the class of languages accepted by auxiliary push down automata with a given space bound is equivalent to the class of languages accepted by Turing machines of time complexity exponential in that space bound. The model of an auxiliary push down automaton is shown in Fig. 7.21. It consists of :

- (i) a read-only input tape, whose length is decided by end markers, ψ and $\$$,
- (ii) a finite state control,
- (iii) a read-write storage tape of length $S(n)$, where n is the length of the input string w , and
- (iv) a stack (push down store).

The transition in auxiliary push down automaton is determined by the state of the finite control, along with the symbol scanned from read only input tape, the read-write storage, and top symbol in the stack. In one move, an auxiliary PDA may does any or all of the following:

- (i) changes the state
- (ii) moves its input head one block left or right, but not off the input,
- (iii) writes a symbol on the block scanned by the storage head and moves the head one position left or right,
- (iv) push a symbol onto the stack or pop the top symbol form the stack.

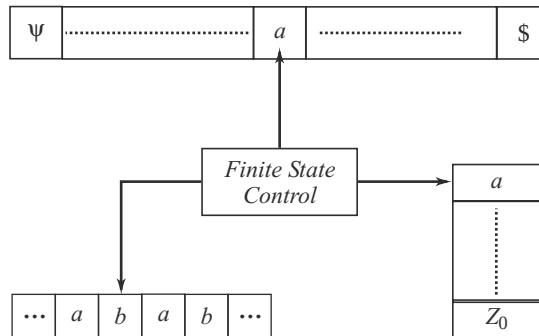


Fig. 7.21 The Model of Auxiliary Push down Automata

If auxiliary push down automaton is deterministic, it has a finite number of choices of the above type. Initially the tape heads are at the left end of the input and storage tape with the finite control in a specified initial state and stack consisting of a specified push down symbol.

The interest in auxiliary PDA originates from the fact that deterministic and non-deterministic auxiliary PDA with the same space bound are equivalent, and that $S(n)$ space on an auxiliary push down automaton is equivalent to $k^{S(n)}$ time on a Turing machine. It means, the following three statements are equivalent:

- (i) Language L is accepted by a deterministic $S(n)$ – APDA

- (ii) Language L is accepted by a non-deterministic $S(n)$ – APDA
- (iii) Language L is in $\text{DTIME}(k^{S(n)})$ for some constant k , where DTIME is time complexity

7.14

PARSING AND PDA (TOP DOWN AND BOTTOM UP)

The practical implementation of PDA is called a **Syntax Analyser**. It creates the syntactic structure (generally a parse tree) of the given program. In other words, a Syntax Analyser takes the output of lexical analyser (list of tokens) and produces a parse tree. A syntax analyser is also called as a **parser**. In other words, a parser for a CFG (Context-Free Grammar) is a program which determines whether a string w is part of the language $L(G)$. The main functions of a parser are

- they produce a parse tree if $w \in L(G)$
- they call semantic routines
- they manage syntax errors, generates error messages

As input they accept string (finite sequence of tokens). The input is read from left to right. As output they produce parse tree / error messages.

A parse tree describes a syntactic structure of the program. The syntax is defined as the physical layout of the source program. The grammars describe precisely the syntax of a language. Two kinds of grammars which compiler writers use a lot are: regular, and context free.

A parser checks input stream for syntactic correctness and prepares a framework for subsequent semantic processing. A parser is implemented as a push down automaton (PDA). YACC (yet another compiler compiler) is a tool that takes context free grammar and generates a parser for that language (again a C program).

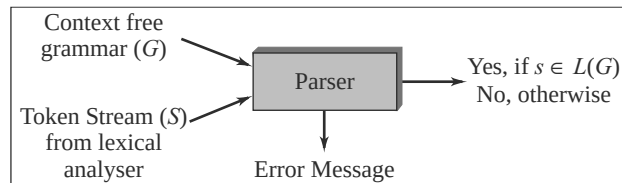


Fig. 7.22 Function of a parser

A parser works on a stream of tokens as shown in the figure below. The smallest item is a token.

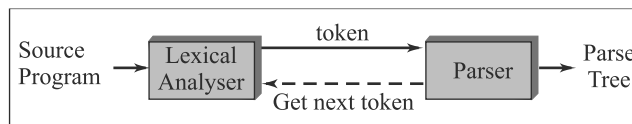


Fig. 7.23 Working of a parser

The process performed by the parser is called **parsing**. Depending on how the parse tree is created, there are different parsing techniques. These parsing techniques are categorised into two groups:

- **Top-Down Parsing**
- **Bottom-Up Parsing**

Top-Down Parsing Top-down parsing refers to the construction of the parse tree which starts at the root and proceeds towards the leaves. Efficient top-down parsers can be easily constructed manually. The examples of top-down parsing are – Recursive Predictive Parsing and Non-Recursive Predictive Parsing (LL Parsing).

The simplest way of top down parsing is to use backtracking. A parser takes the grammar and constructs the parse tree by selecting the production as per the guidance initiated by left to right scanning of the input string. For example, if the input string is $s = bcd$ and the given grammar has productions

$$S \rightarrow bX$$

$$X \rightarrow d \mid cX$$

For the construction of the parse tree for the string bcd , we start with root levelled with start symbol S . We have only one option for S as bX and also its first symbol (terminal b) is matched with the first symbol of string bcd . Now the replacement of X must be done in such a way that the second leaf node in the derivation tree should be 'c'. If X is replaced with 'd' then we will have to back track because there will be no matching in second symbol between input string and yield of the parse tree. Therefore the nonterminal X is replaced with cX . Finally the nonterminal X will be replaced with 'd' so that the yield of a parse tree is similar to input string. The construction of a parse tree is given below.

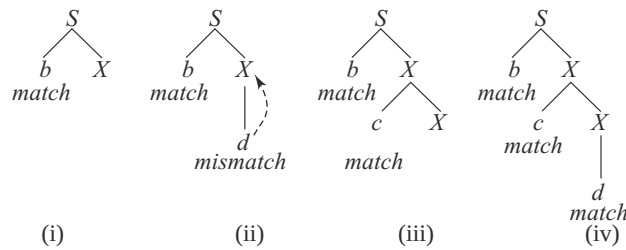


Fig. 7.24 Steps of construction of a Parse tree

Bottom-Up Parsing In bottom-up parsing, the construction of the parse tree starts at the leaves and proceeds towards the root. Normally efficient bottom-up parsers are created with the help of some software tools. Bottom-up parsing is also known as **shift-reduce parsing**. It is further divided into two Operator-Precedence and LR Parsing. The operator-Precedence is simple, restrictive, and easy to implement. The LR parsing is a much more general form of shift-reduce parsing, examples are SLR, Canonical LR, and LALR.

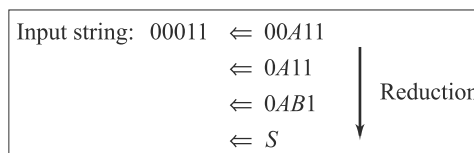
Bottom up parsers start their working with terminal string and finally reduces it to the start symbol. For example, an input string to be parsed is $s = 00011$, and given CFG G has productions

$$S \rightarrow 0AB1$$

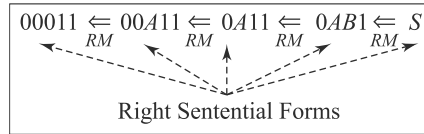
$$A \rightarrow 0A \mid 0$$

$$B \rightarrow 1B \mid 1$$

Here we see how the string 00011 is reduced in start symbol S :



This shows that the string 00011 $L(G)$ and the string 00011 is parsed. The strings of terminals and/or nonterminals during shift–reduce operations are called right sentential forms. The different right sentential forms of above examples are given below.



It is often desirable to know which substring is to be replaced at each reduction step. For this purpose we use a handle. Informally, a **handle** of a string is a substring that matches the right side of a production rule. But it is not necessary that every substring matches the right side of a production rule is a handle. A handle of a right sentential form $\gamma (\equiv \alpha \beta \omega)$ is a production rule $A \rightarrow \beta$ and a position of γ , where the string β may be found and replaced by A to produce the previous right-sentential form in a rightmost derivation of γ .

$$S \xRightarrow[RM]{*} \alpha A \omega \xRightarrow[RM]{} \alpha \beta \omega$$

If the grammar is unambiguous, then every right-sentential form of the grammar has exactly one handle. We will see that ω is a string of terminals.

A right-most derivation in reverse can be obtained by **handle pruning**. For example,

$$S = \gamma_0 \xRightarrow[RM]{} \gamma_1 \xRightarrow[RM]{} \gamma_2 \xRightarrow[RM]{} \cdots \xRightarrow[RM]{} \gamma_{n-1} \xRightarrow[RM]{} \gamma_n = \omega$$

We start from γ_n , find a handle $A_n \rightarrow \beta_n$ in γ_n , and replace β_n in by A_n to get γ_{n-1} . Then find a handle $A_{n-1} \rightarrow \beta_{n-1}$ in γ_{n-1} , and replace β_{n-1} in by A_{n-1} to get γ_{n-2} . This process is repeated until we reach S . Now we see the stack implementation of a shift–reduce parser. There are four possible actions of a shift–parser action:

1. **Shift** The next input symbol is shifted onto the top of the stack.
 2. **Reduce** Replace the handle on the top of the stack by the non-terminal.
 3. **Accept** Successful completion of parsing.
 4. **Error** The parser discovers a syntax error, and calls an error recovery routine.
- Initial stack just contains only the end marker $\$$. The end of the input string is marked by it.

A Stack Implementation of a Shift- Reduce Parser

Stack	Input	Action
\$	00011\$	shift
\$0	0011\$	shift
\$00	011\$	shift
\$000	11\$	reduce by $A \rightarrow 0$
\$00A	11\$	reduce by $A \rightarrow 0A$
\$0A	11\$	shift
\$0A1	1\$	reduce by $B \rightarrow 1$
\$0AB	1\$	shift
\$0AB1	\$	reduce by $S \rightarrow 0AB1$
\$S	\$	Accept

❖ Did you Know ?

Sheila Griebach, the name behind Greibach normal form (GNF), has also worked with Seymour Ginsburg and Michael Harrison. They have worked together in parsing of context-sensitive language with the help of the stack automata model.

7.15

DETERMINISTIC PDA AND DETERMINISTIC CFL

In the beginning of this section we have seen that a PDA is said to be deterministic if there is only one transition from the present state to the next state on an input. Also, there should be no transition on input ϵ or $\wedge$. In other words, if a PDA takes transitions by following the transition function

$$Q \times \{\Sigma \cup \wedge\} \times \Gamma \rightarrow Q$$

For example, the PDA M defined as following is deterministic:

$$M = (\{q_0, q_1\}, \{0, 1\}, \{0, 1, Z_0\}, \delta, Z_0, q_0, \{q_1\}).$$

$$T_1 : \delta(q_0, 0, Z_0) = \{(q_1, Z_0)\}$$

$$T_2 : \delta(q_0, 0, 1) = \{(q_0, 01)\}$$

$$T_3 : \delta(q_0, 1, 1) = \{(q_1, \wedge)\}$$

$$T_4 : \delta(q_1, 1, 0) = \{(q_1, 10)\}$$

$$T_5 : \delta(q_1, 0, 0) = \{(q_1, \wedge)\}$$

A context free language is said to be deterministic if there exists a deterministic PDA that exactly accepts/recognises it and also the PDA has additional capability to detect the end of input string. In a formal way, we say a context free language $L \subseteq \Sigma^*$ is deterministic if $L\$ = L(A)$ for some deterministic PDA A .

Summary

- **Push Down Automata** Push down automata (PDAs), are a way to represent the language class called context free languages.
- **Instantaneous Description** An instantaneous description (ID) is defined as (q, x, α) , where $q \in Q$, $x \in \Sigma^*$ and $\alpha \in \Gamma^*$ for a push down automaton $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$. The instantaneous description (q, x, α) represents the present status of the push down automaton.
- **Move Relation** The transition from an instantaneous description (ID) to another instantaneous description is defined by the term move relation and denoted by the symbol $\vdash$.
- **Language of PDA** Context free and regular are the languages which are accepted by a PDA. If there is a context free grammar then there exists a context free language which can be accepted by a PDA.
- **Deterministic PDA** A deterministic push down automaton is one for which every input string has a unique path through initial state to final state in the machine.
- **Deterministic CFL** A context free language is said to be deterministic if there exists a deterministic PDA that exactly accepts/recognises it.

- **Nondeterministic PDA** A nondeterministic push down automaton is one for which, at a certain time it has to select a particular path among possible paths. The transition function δ of nondeterministic push down automaton is defined as the mapping $Q \times \{\Sigma \cup \wedge\} \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$.
- **Acceptance by Final State** The acceptance conditions include that the input string must be exhausted as the PDA reaches the final state.
- **Acceptance by Null or Empty Store** The acceptance conditions include that the stack must be empty as the input string is exhausted.
- **Two-Stack PDA** A two-stack push down automaton (also called two-push down stack machine or two-push down store machine) is like a push down automaton except that it has two push down stores, $STACK_1$ and $STACK_2$.
- **Auxiliary PDA** Auxiliary push down automata, are push down automata with two tapes and a stack.
- **Syntax Analyser** The practical implementation of PDA is called a Syntax Analyser. A syntax analyser is also called a parser.
- **Top-Down Parsing** Top-down parsing refers to the construction of the parse tree which starts at the root, and proceeds towards the leaves.
- **Bottom-Up Parsing** In bottom-up parsing, the construction of the parse tree starts at the leaves and proceeds towards the root.
- **LR Parsing** The LR parsing is a much more general form of shift-reduce parsing, examples are SLR, Canonical LR, and LALR.

Review Questions

- 7.1 Show that a push down automaton M given below is deterministic

$$M = (\{q_0, q_1\}, \{0, 1\}, \{0, 1, Z_0\}, \delta, Z_0, q_0, \{q_1\}).$$

$$T_1 : \delta(q_0, 0, Z_0) = \{(q_1, Z_0)\}$$

$$T_2 : \delta(q_0, 0, 1) = \{(q_0, 01)\}$$

$$T_3 : \delta(q_0, 1, 1) = \{(q_1, \wedge)\}$$

$$T_4 : \delta(q_1, 1, 0) = \{(q_1, 10)\}$$

$$T_5 : \delta(q_1, 0, 0) = \{(q_1, \wedge)\}$$

- 7.2 Give transitions for PDA recognising the language:

$$L = \{s \in (a, b)^*\}$$

- 7.3 Construct a nondeterministic push down automaton that accepts the language generated by the grammar

$$S \rightarrow aSSS \mid ab.$$

- 7.4 Give transitions for PDA recognising the language:

$$L = \{s \in aa(a, b)^*bb \mid |s| \% 2 = 0\}$$

- 7.5 Show that the PDA M given below accepts the language even length palindromes over $\{0, 1\}$ i.e.,

$$L = \{\omega\omega^r \mid \omega \in \{0, 1\}^*\}.$$

$$M = (\{q_0, q_1\}, \{0, 1\}, \{A, B, C\}, \delta, q_0, A, \{\phi\}), \text{ where } \delta \text{ is defined as}$$

$$\delta(q_0, 0, A) = \{(q_0, BA)\}, \quad \delta(q_0, 1, C) = \{(q_0, CC), (q_1, \wedge)\},$$

$$\begin{aligned}\delta(q_0, 1, A) &= \{(q_0, CA)\}, & \delta(q_1, 0, B) &= \{(q_1, \wedge)\}, \\ \delta(q_0, 0, B) &= \{(q_0, BB), (q_1, \wedge)\}, & \delta(q_1, 1, C) &= \{(q_1, \wedge)\}, \\ \delta(q_0, 0, C) &= \{(q_0, BC)\}, & \delta(q_0, \wedge, A) &= \{(q_1, \wedge)\}, \\ \delta(q_0, 1, B) &= \{(q_0, CB)\}, & \delta(q_1, \wedge, A) &= \{(q_1, \wedge)\},\end{aligned}$$

7.6 Construct a push down automaton M accepting the set of all even-length palindromes over the alphabet $\{a, b\}$ by null store.

7.7 Give transitions for two-stack PDA recognising the language

$$L = \{a^n b^{2n} a^n b^n \mid n \geq 0\}$$

7.8 Give transitions for two-stack PDA recognising the language

$$L = \{a^n b^n a^n b^n \mid n \geq 0\}$$

7.9 Show that the PDA M

$$M = (\{q_0, q_1, q_f\}, \{a, b\}, \{S, a, b, Z_0\}, \delta, q_0, Z_0, \{q_f\}),$$

accepts all strings over $\{a, b\}$ in which number of a 's are more than number of b 's, where δ defined as

$$\begin{aligned}\delta(q_0, \wedge, Z_0) &= \{(q_1, SZ_0)\}, \\ \delta(q_1, \wedge, S) &= \{(q_1, a), (q_1, aS), (q_1, bSS), (q_1, SSb), (q_1, SbS)\}, \\ \delta(q_1, a, a) &= \{(q_1, \wedge)\}, \\ \delta(q_1, b, b) &= \{(q_1, \wedge)\}, \\ \delta(q_1, \wedge, Z_0) &= \{(q_f, Z_0)\}\end{aligned}$$

7.10 Construct push down automaton M that accepts the strings represented by following context free language

$$L = \{\omega \omega \mid \omega \in \{0, 1\}^*\}$$

Hints

7.3 $\delta(q_0, \wedge, S) = \{(q_0, aSSS), (q_0, ab)\},$

$$\delta(q_0, a, a) = \{(q_0, \wedge)\}, \delta(q_0, b, b) = \{(q_0, \wedge)\}$$

The transition $\delta(q_0, \wedge, S) = \{(q_0, aSSS), (q_0, ab)\}$ in this PDA, produces non-determinism. The acceptance of language is by null store.

7.10 The CFG representing L has productions $S \rightarrow AA, A \rightarrow aA \mid bA \mid \wedge$. A PDA for this CFG can be constructed as.

$$\delta(q_0, \wedge, S) = \{(q_0, AA)\}$$

$$\delta(q_0, \wedge, A) = \{(q_0, aA), (q_0, bA), (q_0, \wedge)\}$$

$$\delta(q_0, a, a) = \{(q_0, \wedge)\}$$

$$\delta(q_0, b, b) = \{(q_0, \wedge)\}$$

Graded Exercises

****7.1** Give transitions for PDA recognising the language:

$$L = \{s \in (a, b)^* \mid n_a(s) \geq 2n_b(s)\}$$

****7.2** Give transitions for PDA recognising the language:

$$L = \{s \in (a, b)^* \mid n_a(s) \neq 2n_b(s)\}$$

*****7.3** Give transitions for PDA recognising the language $L = \{a^p b^q c^r a^k \mid p + q = r + s\}$

***7.4** Consider the context-free grammar G with productions $S \rightarrow SS \mid [S] \mid \{S\} \mid \wedge$. CFG G generates the languages L of all balanced strings involving two types of brackets “[]” and “[]”. Construct equivalent PDA M .

****7.5** Construct non-deterministic push down automaton M that accepts the language

$$L = \{a^n b^m \mid m+n \text{ is even}\}$$

****7.6** Construct push down automaton M that accepts the language

$$L = \{a^n s b^n \mid n \geq 0, s \in (a, b)^* \text{ and } |s| \text{ is even}\}$$

****7.7** Give transitions for PDAs recognising each of the following languages:

(i) $L = \{s \in (a, b)^* \mid n_a(s) = n_b(s) + 1\}$

(ii) $L = \{s \in (a, b)^* \mid n_a(s) \neq n_b(s) \text{ and } |x| \geq 1\}$

(iii) $L = \{a^n b^{n+m} a^m \mid m, n \geq 0\}$

(iv) $L = \{s \in (a, b, c)^* \mid |s| \% 3 = 2\}$

****7.8** Design PDA for language $L = \{a^n b^n \cup a^n b^{2n} \mid m, n \geq 0\}$.

****7.9** Design PDA for language $L = \{a^n b^n a^m b^m a^l b^l \mid m, n, l \geq 0\}$.

***7.10** Design PDA for language generated by the following grammar

$$S \rightarrow aSa \mid A \mid c, A \rightarrow aAa \mid bAb \mid a \mid b \mid \varepsilon$$

***7.11** Design PDA for accepting the language generated by the following grammar

$$S \rightarrow aA \mid \varepsilon, A \rightarrow bB, B \rightarrow cS$$

***7.12** Design PDA for language $L = \{a^n b^m \mid m, n \geq 0\}$.

***7.13** Design PDA for language $L = \{(aba)^n \mid n \geq 0\}$.

****7.14** Consider the push down automaton $M = (\{q_0, q_1\}, \{0, 1\}, \{0, 1, Z_0\}, \delta, Z_0, q_0, \{q_1\})$.

$$T_1 : \delta(q_0, 0, Z_0) = \{(q_1, Z_0)\}$$

$$T_2 : \delta(q_0, 0, 1) = \{(q_0, 01)\}$$

$$T_3 : \delta(q_0, 1, 1) = \{(q_1, \wedge)\}$$

$$T_4 : \delta(q_1, 1, 0) = \{(q_1, 10)\}$$

$$T_5 : \delta(q_1, 0, 0) = \{(q_1, \wedge)\}$$

Construct a context free grammar equivalent to it.

****7.15** If a language L is accepted by a PDA M , then is it necessary that L is accepted by a PDA in which there are at most two stack symbols in addition to stack symbol Z_0 .

7.16 Give transitions for two-stack PDA recognising the languages:

***(i)** $L = \{a^n b^n a^{2n} \mid n \geq 0\}$

**** (ii)** $L = \{a^n b^n a^n b^{3n} \mid n \geq 0\}$

***(iii)** $L = \{a^n b^n c^m d^n \mid m, n \geq 0\}$.

**** (iv)** $L = \{a^n b^n c^n d^n \mid n \geq 0\}$.

* Difficulty level 1

** Difficulty level 2

*** Difficulty level 3

Hints

7.4 The PDA M is given by following transitions

$$\delta(q_0, \{, Z_0) = \{(q_1, \{Z_0\})\},$$

$$\delta(q_0, [, Z_0) = \{(q_1, [Z_0])\},$$

$$\delta(q_1, \{, \{) = \{(q_1, \{\{ \})\},$$

$$\delta(q_1, [, \{) = \{(q_1, [\{ \})\},$$

$$\delta(q_1, \{, [) = \{(q_1, \{[\})\},$$

$$\delta(q_1, [, []) = \{(q_1, [[])\},$$

$$\delta(q_1, \}, \{) = \{(q_1, \wedge)\},$$

$$\delta(q_1,], []) = \{(q_1, \wedge)\},$$

$$\delta(q_1, \wedge, Z_0) = \{(q_1, Z_0)\}.$$

7.5 The required PDA has following transitions:

$$\delta(q_0, \wedge, Z_0) = \{(q_1, SZ_0)\},$$

$$\delta(q_1, \wedge, S) = \{(q_1, AB), (q_1, CD)\},$$

$$\delta(q_1, \wedge, A) = \{(q_1, aaA), (q_1, \wedge)\},$$

$$\delta(q_1, \wedge, B) = \{(q_1, bbB), (q_1, \wedge)\},$$

$$\delta(q_1, \wedge, C) = \{(q_1, aaC), (q_1, a)\},$$

$$\delta(q_1, \wedge, D) = \{(q_1, bbD), (q_1, b)\},$$

$$\delta(q_1, a, a) = \{(q_1, \wedge)\},$$

$$\delta(q_1, b, b) = \{(q_1, \wedge)\}, \text{ and}$$

$$\delta(q_1, \wedge, Z_0) = \{(q_f, Z_0)\}$$

$$7.7 \text{ (iii) } \delta(q_0, \wedge, Z_0) = \{(q_f, Z_0)\} \quad [\text{for } m, n = 0]$$

$$\delta(q_0, a, Z_0) = \{(q_1, aZ_0)\}, \delta(q_1, a, a) = \{(q_1, aa)\}$$

$$\delta(q_0, b, Z_0) = \{(q_3, bZ_0)\}, \delta(q_3, b, b) = \{(q_3, bb)\} \quad [\text{for } n = 0]$$

$$\delta(q_1, b, a) = \{(q_2, \wedge)\}, \delta(q_2, b, a) = \{(q_2, \wedge)\}$$

$$\delta(q_2, b, Z_0) = \{(q_3, bZ_0)\}, \delta(q_3, b, b) = \{(q_3, bb)\}$$

$$\delta(q_3, a, b) = \{(q_3, \wedge)\}$$

$$\delta(q_3, \wedge, Z_0) = \{(q_f, Z_0)\}$$

Objective Questions

- Which of the following is the most powerful?
 - DFA
 - NDEFA
 - 2PDA
 - DPDA
- A push down automaton is different from a finite automaton because of:
 - a read head
 - a memory in the form of stack
 - a set of state
 - all of these
- Which of the following statement is false?
 - For a finite automaton, the working can be described in terms of change of states.
 - For a push down automaton, the working can be described in terms of change of instantaneous descriptions.
 - both (a) and (b)
 - neither (a) or (b)
- The acceptance of input strings by a push down automaton can be defined in terms of
 - only final state
 - only empty store
 - only null store
 - all of these
- For a push down automaton $A = (Q, \Sigma, \Gamma, \delta, q_0, F)$, the set $N(A)$ accepted by empty store is defined by
 - $N(A) = \{s \in \Sigma^* \mid (q_0, s, Z_0) \xrightarrow{*} (q_f, \wedge, Z)\} \text{ for some } q_f \in F \text{ and } Z \in \Gamma^*\}$
 - $N(A) = \{s \in \Sigma^* \mid (q_0, s, Z_0) \xrightarrow{*} (q_f, \wedge, \wedge)\} \text{ for some } q_f \in Q\}$
 - both (a) and (b)
 - neither (a) nor (b)
- Which of the following is true?
 - A PDA is deterministic if $\delta(q, a, Z)$ is either empty or singleton, or $\delta(q, \wedge, Z) \neq \emptyset$
 - The set accepted by a PDA by final state is defined by $\{s \in \Sigma^* \mid (q_0, s, Z_0) \xrightarrow{*} (q_f, \wedge, \alpha)\} \text{ for some } q_f \in F \text{ and } \alpha \in \Gamma^*\}$
 - The set accepted by PDA are precisely context-free languages
 - all of the above
- Which of the following statements is true

- (a) The set accepted by a PDA A by null store is accepted by some PDA B by final state.
 (b) The set accepted by a PDA A by final state is accepted by some PDA B by null store.
 (c) (a) is true but (b) is false
 (d) both (a) and (b) are true
8. The type of symbols in push down store are:
 (a) terminals and nonterminals (b) only terminals
 (c) only nonterminals (d) none of these
9. Which of the following statement is true?
 (a) $L = \{a^n b^n \mid n \geq 1\} \cup \{a^m b^{2m} \mid m \geq 1\}$ can be accepted by a deterministic PDA.
 (b) The regular set accepted by a DFA with n states is accepted to final state by a deterministic PDA with n states and one push down symbol.
 (c) The language $L = \{a^n b^n c^n \mid n \geq 1\}$ can be accepted by a deterministic PDA.
 (d) all of the above
10. A two-way push down automaton (2-PDA) can
 (a) be a PDA that is allowed to move in either direction (left or right) on its input.
 (b) accept a language $L = \{0^n 1^n 2^n \mid n \geq 1\}$
 (c) not be equivalent to a PDA
 (d) all of the above
11. If $G = (V_N, \Sigma, P, S)$ is a CFG, and there is a PDA M such that $L(G) \subseteq L(M)$ and $L(M) \subseteq L(G)$, then
 (a) $L(M) \neq L(G)$ (b) $L(M) = L(G)$ (c) G must be empty (d) both (b) and (c)
12. If the language L is accepted by a PDA in which no symbol is necessarily removed from the stack, then L is
 (a) context-free (b) regular (c) both (a) and (b) (d) only (a) not (b)
13. If $L \subseteq \Sigma^*$ is accepted by a PDA M and that for some fixed k every $x \in \Sigma^*$, no sequence of moves made by M on input x causes the stack to have more than k elements. Then
 (a) L is regular language (b) x is any even length string
 (c) the set Σ has only a single symbol (d) all of these.
14. If a language L is accepted by a PDA, then:
 (a) there is a PDA accepting the language $\{x\#y \mid x \in L \text{ and } xy \in L\}$
 (b) L is accepted by a PDA in which every move either pops top element of the stack or pushes a single symbol onto stack on the top of symbol that was previously on top, or leaves the stack unchanged.
 (c) L is accepted by a PDA that never crashes.
 (d) all the above
15. If L is accepted by a PDA, then L is also accepted by a PDA having
 (a) atmost two states (b) no $\wedge$ -transitions (c) both (a) and (b) (d) neither (a) nor (b)
16. Which of the following language cannot be accepted by any deterministic push down automaton
 (a) The set of all strings over $\{a, b\}$ consisting of equal number of a 's and b 's.
 (b) A language of palindromes $\{x \in \{a, b\}^* \mid x = \text{rev}(x)\}$.
 (c) $\{wcw^R \mid w \in (a, b)^*\}$
 (d) all of the above

17. All strings over $\{a, b\}$ having equal number of a 's and b 's can be recognised by
 - (a) a *DFA*
 - (b) an *NDFA*
 - (c) a *PDA*
 - (d) all of these
18. Which of the following statements is true
 - (a) If we add a stack to just the *DFA* model, the result is the deterministic push down automaton (*DPDA*) model.
 - (b) If we add a stack to just the *NDFA* model, the result is the nondeterministic push down automaton model.
 - (c) Adding a stack to a *FA* model makes nondeterministic behavior more powerful than deterministic behavior.
 - (d) all of the above
19. If $M = (Q, \Sigma, \delta, \Gamma, q_0, Z_0, F)$ is a *PDA*, then
 - (a) δ : mapping from $Q \times \Sigma \times \Gamma$ to finite subset of $Q \times \Gamma$
 - (b) δ : mapping from $Q \times \Sigma \times \Gamma^*$ to finite subset of $Q \times \Gamma^*$
 - (c) δ : mapping from $Q \times \Sigma \times \Gamma$ to finite subset of $Q \times \Gamma^*$
 - (d) δ : mapping from $Q \times (\Sigma \cup \{\wedge\}) \times \Gamma$ to finite subset of $Q \times \Gamma^*$
20. The instantaneous description (*ID*) in a *PDA*:
 - (a) shows the present state
 - (b) shows the string to be processed
 - (c) shows the stack contents
 - (d) all of these
21. Which of the following statements is false?
 - (a) $\mid_M^*$ is the reflexive closure of $\mid_M$
 - (b) $\mid_M^*$ is the transitive closure of $\mid_M$
 - (c) A sequence of *IDs* $I \mid_M J$ and $J \mid_N K$ implies $I \mid_{M+N} K$
 - (d) none of the above.
22. A *PDA* M simulates left most derivation of a context-free grammar G if
 - (a) the grammar is ambiguous
 - (b) the grammar is unambiguous
 - (c) the grammar is in Chomsky normal form
 - (d) the grammar is in Greibach normal form.
23. If $[q_1, Z, q_2] \Rightarrow w$ for a context-free grammar G , then:
 - (a) $(q_2, w, Z) \mid^* (q_1, \wedge, \wedge)$
 - (b) $(q_1, w, Z) \mid^* (q_2, \wedge, \wedge)$
 - (c) $(q_2, w, Z Z) \mid^* (q_1, \wedge, Z)$
 - (d) all of these
24. If $(q_i, w, Z) \mid^* (q_k, \wedge, \wedge)$, then
 - (a) $(q_i, ww', Z) \mid^* (q_k, \wedge, \wedge)$
 - (b) $(q_i, w, ZZ') \mid^* (q_k, \wedge, Z')$
 - (c) $(q_i, ww', Z Z') \mid^* (q_k, w', ZZ)$
 - (d) all of these
25. If $(q, x y z, Z) \mid_{n-1}^* (q, y z, \alpha)$ and $(q, a, a) \mid (q, \wedge, \wedge)$, then
 - (a) $(q, x y z, Z) \mid (q, a y z, a \alpha) \mid (q, y z, \alpha)$
 - (b) $(q, x y z, Z) \mid^n (q, a y, a \alpha) \mid (q, a, \alpha)$
 - (c) both (a) and (b)
 - (d) none of these
26. If $(q_0', w, Z_0') \mid (q_0, w, Z_0 Z_0')$ and $(q_0, w, Z_0 Z_0') \mid (q, \wedge, \alpha Z_0')$, then:
 - (a) $(q_0, w, Z_0 Z_0') \mid^* (q_0', \wedge, Z_0')$
 - (b) $(q_0', w, Z_0') \mid^* (q, \wedge, \alpha Z_0')$

- (c) $(q_0, w, Z_0 Z_0') \vdash^* (q, \wedge, \alpha Z_0')$, (d) all of these
27. For a context-free grammar $S \rightarrow S^*S \mid a \mid b$, the equivalent push down automaton has transition rule(s)
- (a) $\delta(q_0, \wedge, S) = \{(q_0, S^*S), (q_0, a), (q_0, b)\}$ (b) $\delta(q_0, a, a) = \{(q_0, \wedge)\}$
- (c) $\delta(q_0, b, b) = \{(q_0, \wedge)\}$ (d) all of the above
28. If 'w' is any string with an equal number of a's and b's, then:
- (a) $(q_0, w, Z_0) \vdash^* (q, \wedge, \wedge)$ for some $q \in Q$ (b) $w \in N(A)$, for some PDA A
- (c) $w \in T(A)$ for some PDA A (d) all of the above

References for Extra Reading

- Oettinger, A.G., *Automatic syntactic analysis and the pushdown store*, Proceedings of Symposia on Applied Mathematics 12, American Mathematical Society, Providence, Rhode Island, 1961.
- Autebert, J. M., J. Berstel, L. Boasson, *Context-free languages and pushdown automata*, in G. Rozenberg, A. Salomaa, eds., *Handbook of Formal Languages*, vol. I. Springer, Berlin, 1997.
- Goldstine, J., H. Leung, D. Wotschke, *Measuring nondeterminism in pushdown automata*, in R. Reischuk and M. Morvan, eds., *Theoretical Computer Science*, 18(1): 33-40, 1997.
- Goldstine, J., J.K. Price, D. Wotschke, *A pushdown automaton or a context-free grammar: which is more economical?*, *Theoretical Computer Science*, 18(1): 33-40, 1982.
- Herzog, C., *Pushdown automata with bounded nondeterminism and bounded ambiguity*, *Theoretical Computer Science*, 1997.
- John C. Martin, *Introduction to Languages and the Theory of Computation*, McGraw Hill. 2003.
- Minsky, M., *Computation: Finite and Infinite Machine*, Prentice-Hall, Inc., Englewood Cliffs, NJ. Ch.: Neural Networks 1967.
- Minsky, M. L., *Some methods of heuristic programming and artificial intelligence*, In Proc. Symposium on the Mechanisation of Intelligence, London, UK. <http://web.media.mit.edu/~minsky/>
- Salomaa, K., and S. Yu, *Degrees of nondeterminism for pushdown automata*, in L. Budach, ed., *Fundamentals of Computation Theory 1991*, Lecture Notes in Computer Science 529: 380-389. Springer, Berlin.

Online Sources

- <http://www.cs.princeton.edu/courses/archive/spr01/cs126/lectures/T2-4up.pdf>
- <http://www.cs.uky.edu/~lewis/texts/theory/automata/autointr.pdf>
- http://www.easychair.org/FLoC-06/parthasarathy_galop_floc06.pdf
- http://www.en.wikipedia.org/wiki/Automata_theory

8

Properties of Context Free Languages

❖ Chapter Objective ❖

This chapter highlights the closure and decision properties of context free languages. We will describe pumping lemma for context free languages and see how pumping lemma is applied to show that certain languages are not context free. We will discuss how a regular expression is represented by a context free grammar.

□ Introduction □

Context free grammars have a very important role in computer science. The syntax of many of the programming languages is context free. With the help of pumping lemma we can prove that the syntax of C language is context free. The decision properties of a context free language can be used to check whether a language is empty, finite or infinite. The closure properties of context free languages allow us to analyse the result when a context free language is operated with another context free language and non-context free language.

8.1 PUMPING LEMMA FOR CONTEXT FREE LANGUAGES

We give a method to prove that certain languages are not context-free. If we have to prove that a particular language is context free, we can show that there exists a push down automaton and our purpose is solved. If we have to prove that a particular language is not context free then we use pumping lemma for context free languages. In this section we see how an infinite number of strings can be generated from a given sufficiently long string in a context-free language.

Theorem 8.1 For a given context-free grammar G in CNF, T is a derivation tree in G . Then the yield of T is of length less than or equal to 2^{k-1} if the length of longest path in T is less than or equal to k .

Proof We can prove the theorem by applying induction on the length of the longest path for all subtrees of T .

Case ($K = 1$) When the longest path in a subtree (say A -tree, i.e., a tree root A) is of length 1, then the root has only one child whose label is a terminal. So the yield is of length 1.

Case ($K > 1$) When the longest path in a subtree is of length 2, then there will be exactly two children of the root whose label is a variable (because productions are in CNF). These two variables have exactly one child whose label is a terminal. In case length 2 productions are of the form $A \rightarrow BC$, $B \rightarrow b$, $C \rightarrow a$, and in this case ($K > 1$) the two subtrees are with labels B and C , shown in Fig. 8.1 below :

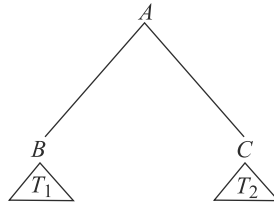


Fig. 8.1 An A -tree with subtrees T_1 and T_2

In Fig. 8.1, B and C can be assumed as roots of the subtrees T_1 and T_2 respectively. If s_1 and s_2 are the yields of subtrees T_1 and T_2 respectively, then by induction hypothesis, $|s_1| \leq 2^{k-2}$ and $|s_2| \leq 2^{k-2}$. Therefore, the yield of derivation tree T is s_1s_2 , where $|s_1s_2| \leq 2^{k-2} + 2^{k-2}$, which is equal to 2^{k-1} .

Lemma 8.1 (Pumping Lemma for CFLs) If L is any context-free language, then we can find a natural number n such that:

- (i) every $z \in L$ where $|z| \geq n$ can be written as $uvwxy$ for some strings u, v, w, x and y
- (ii) the length of ' vx ' is greater or equal to 1 (i.e., $|vx| \geq 1$)
- (iii) the length of ' vwx ' is less or equal to n (i.e., $|vwx| \leq n$)
- (iv) $uv^kwx^ky \in L$ for all $k = 0, 1, 2, 3, \dots$

Proof We have already proved that there exists an algorithm to decide whether or not $\wedge \in L$. When $\wedge \in L$, we consider $L - \{\wedge\}$ and construct a CFG $G = (V_N, \Sigma, P, S)$ in Chomsky normal form generating L . When $\wedge \notin L$, we construct G in Chomsky normal form generating language L .

Suppose there are m variables in V_N and $n = 2^m$. To prove that n is the required number, we start with $z \in L$, where $|z| \geq 2^m$. We construct a derivation tree T giving yield ' z '. If the length of the longest path in T is at most m , then length of yield ' z ' is less than or equal to 2^{m-1} . But m is assumed such that.

$$|z| \geq 2^m > 2^{m-1}$$

So, the derivation tree T has a path (say P) of length greater than or equal to $m + 1$ and P has at least $m + 2$ nodes and only the last node is a leaf node, and the remaining nodes have variables as labels. As we have assumed that there are m variable in V_N , so some variables are repeated.

We select a variable in V_N to be repeated as follows :

We start with the leaf node of path P and traverse along path P to the root of tree T . We stop when some label (say X) is repeated. Suppose v_1 and v_2 are nodes with label X , and v_1 is nearer from the root. In path P , the portion of path from v_1 to the leaf node has only one label which is repeated. Therefore, the length of path P is at most $m + 1$.

Now suppose T_1 and T_2 be two subtrees with nodes v_1 and v_2 as roots and ' z_1 ' and ' w ' as yields, respectively. Since P is the longest path in tree T , then the portion of P from v_1 to lead node is path in subtree T_1 and its length is at most $m + 1$. The length of yield ' z_1 ' is less or equal to 2^m .

Let us come to the construction. Consider a context free grammar whose productions are

$$S \rightarrow XY, X \rightarrow aY \mid a, Y \rightarrow bX \mid b$$

We construct a derivation tree for ' $ababb$ ' as given by Fig. 8.2 (i). The longest path P in derivation tree T is shown by the thick path in Fig. 8.2(i). Which is

$$S \rightarrow X \rightarrow Y \rightarrow X \rightarrow Y \rightarrow b$$

Here,

$$z = ababb, z_1 = bab, u = a, v = ba, w = b, x = \wedge, \text{ and } y = b.$$

As ' z ' and ' z_1 ' are the yields of T and T_1 respectively, we can write $z = uz_1y$. As z_1 and w are the yields of T_1 and subtree T_2 (a subtree of subtree T_1 shown by thick path in Fig. 8.2(ii)), we can write $z_1 = vwx$. Also, $|vwx| > |w|$, therefore, $|vw| \geq 1$.

Thus, we have

$$z = uvwxy$$

with $|vwx| \leq n$ and $|vx| \geq 1$. This is the proof of point (i) to (iii) of the theorem.

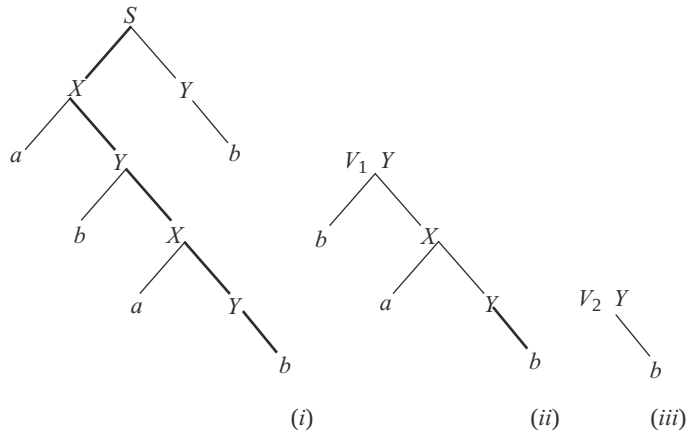


Fig. 8.2 A derivation tree T with their subtrees T_1 and T_2

According to the construction of derivation tree, T is an S -tree, T_1 and T_2 are Y -trees we get

$$S \xRightarrow{*} uYy, Y \xRightarrow{*} vYx \text{ and } Y \xRightarrow{*} w$$

Now

$$\begin{aligned} S &\xRightarrow{*} uYy \\ &\Rightarrow u w y \\ &\Rightarrow u v^0 w x^0 y \end{aligned}$$

Where, $uv^0wx^0y \in L$. For $k \geq 1$,

$$\begin{aligned} S &\Rightarrow uYy \\ &\Rightarrow uv^kYx^ky \\ &\Rightarrow uv^kwx^ky \end{aligned}$$

Where, $uv^kwx^ky \in L$. This is the proof of (iv) point of the theorem.

Application of Pumping Lemma for CFLs

We use pumping lemma to show that a language L is not context-free. First we assume that L is a context-free language then by using pumping lemma we get a contradiction. The following steps are carried out to show that a certain language is not context free :

Step 1 Assume L is context-free. We get a contradiction. Let n be a natural number obtained by using the pumping lemma.

Step 2 Choose $z \in L$, where $|z| \geq n$. By using pumping lemma we can write $z = uvwxy$. It means the string expression representing the language is divided into five parts.

Step 3 Find a suitable k so that $uv^kwx^ky \notin L$. This is a contradiction. This proves that L is not a context free language

Note The pumping lemma for context-free languages is almost similar to pumping lemma for regular languages with only the difference that in pumping lemma for regular language we divide the language string into three parts u , v and w such that $|v| \neq 0$ and $|uv| \leq n$; but in pumping lemma for context free language the language string is divided into five parts u , v , w , x , and y such that $|vx| \neq 0$ and $|uwx| \leq n$.

Example 8.1

Show that the following languages are not context-free by using pumping lemma.

- $L = \{a^n b^n c^n \mid n \geq 1\}$
- $L = \{a^n b^n a^n \mid n \geq 1\}$
- $L = \{a^p \mid p \text{ is a prime number}\}$.

Solution (a) We will prove that language $L = \{a^n b^n c^n \mid n \geq 0\}$ is not context free by using pumping lemma. The language contains the number of a 's followed by the same number of b 's and b 's are followed by the same number of c 's. If the occurrences of a 's, b 's and c 's are not same on application of pumping lemma then we say that the given language is not context free. Also, if b 's are followed by a 's or c 's are followed by b 's or a 's then we say the given language is not context free. First we will assume that the given language is context free then we will get some contradiction that will prove that the given language is not context free. We apply the following steps:

Step 1 Assume given language $L = \{a^n b^n c^n \mid n \geq 0\}$ is context free.

Step 2 By pumping lemma we write

$$z = a^n b^n c^n = uvwxy, \text{ such that } |vx| \neq 0 \text{ or } |vx| \geq 1.$$

The part vx may contain

- (i) only a 's
- (ii) only b 's
- (iii) only c 's
- (iv) both a 's and b 's
- (v) both b 's and c 's
- (vi) all a 's, b 's and c 's

Step 3 Let us find contradictions in all the cases. By pumping lemma we write

$$z = a^n b^n c^n = uv^k wx^k y.$$

Case (i) $vx = a^i$ such that $i \geq 1$

$$z = a^n b^n c^n$$

$$z = \frac{a^{n-i}}{u} \frac{a^i}{v} \frac{b^n}{w} \frac{c^n}{x} \frac{c^n}{y}$$

By pumping lemma we have

$$z = a^{n-i} (a^i)^k b^n (\wedge)^k c^n$$

for $k = 0$, we have

$$z = a^{n-i} b^n c^n$$

Here we get a contradiction because the number of a 's are less than the number of b 's and c 's ($i \geq 1$). Therefore the language $L = \{a^n b^n c^n \mid n \geq 0\}$ is not context free.

Case (ii) $vx = b^i$ such that $i \geq 1$

$$z = a^n b^n c^n$$

$$z = \frac{a^n}{u} \frac{b^{n-i}}{v} \frac{b^i}{w} \frac{c^n}{x} \frac{c^n}{y}$$

By pumping lemma we have

$$z = a^n (\wedge)^k b^{n-i} (b^i)^k c^n$$

for $k = 0$, we have

$$z = a^n b^{n-i} c^n$$

Here we get a contradiction because the number of b 's are less than the number of a 's and c 's ($i \geq 1$). Therefore the language $L = \{a^n b^n c^n \mid n \geq 0\}$ is not context free.

Case (iii) $vx = c^i$ such that $i \geq 1$

$$z = a^n b^n c^n$$

$$z = \frac{a^n b^n}{u} \frac{\wedge}{v} \frac{\wedge}{w} \frac{c^i}{x} \frac{c^{n-i}}{y}$$

By pumping lemma we have

$$z = a^n b^n (\wedge)^k \wedge (c^i)^k c^{n-i}$$

for $k = 0$, we have

$$z = a^n b^n c^{n-i}$$

Here we get a contradiction because the number of c 's are less than the number of a 's and b 's ($i \geq 1$). Therefore the language $L = \{a^n b^n c^n \mid n \geq 0\}$ is not context free.

Case (iv) $\forall x = a^i b^j$ such that $i + j \geq 1$

$$z = a^n b^n c^n$$

$$z = \frac{a^{n-i}}{u} \frac{a^i}{v} \frac{b^{n-j}}{w} \frac{b^j}{x} \frac{c^n}{y}$$

By pumping lemma we have

$$z = a^{n-i} (a^i)^k b^{n-j} (b^j)^k c^n$$

for $k = 0$, we have

$$z = a^{n-i} b^{n-j} c^n$$

Here we get a contradiction because the number of c 's are greater than either from the number of a 's or b 's ($i + j \geq 1$) or both. Therefore the language $L = \{a^n b^n c^n \mid n \geq 0\}$ is not context free.

Case (v) $\forall x = b^i c^j$ such that $i + j \geq 1$

$$z = a^n b^n c^n$$

$$z = \frac{a^n}{u} \frac{b^i}{v} \frac{b^{n-i}}{w} \frac{c^j}{x} \frac{c^{n-j}}{y}$$

By pumping lemma we have

$$z = a^n (b^i)^k b^{n-i} (c^j)^k c^{n-j}$$

for $k = 0$, we have

$$z = a^n b^{n-i} c^{n-j}$$

Here, we get a contradiction because the number of a 's are greater than either from the number of b 's or c 's ($i + j \geq 1$) or both. Therefore the language $L = \{a^n b^n c^n \mid n \geq 0\}$ is not context free.

Case (vi) $\forall x = a^i b^j c^l$ such that $i + j + l \geq 1$

$$z = a^n b^n c^n$$

$$z = \frac{a^{n-i}}{u} \frac{a^i}{v} \frac{b^{n-j}}{w} \frac{b^j c^l}{x} \frac{c^{n-l}}{y}$$

By pumping lemma we have

$$z = a^{n-i} (a^i)^k b^{n-j} (b^j c^l)^k c^{n-l}$$

for $k = 2$, we have

$$\begin{aligned} z &= a^{n-i} (a^i)^2 b^{n-j} (b^j c^l)^2 c^{n-l} \\ &= a^{n-i} a^i a^i b^{n-j} b^j c^l b^j c^l c^{n-l} \\ &= a^n a^i b^n c^l b^j c^n \end{aligned}$$

Here, we get contradiction because the number of c 's are followed by number of b 's. Therefore the language $L = \{a^n b^n c^n \mid n \geq 0\}$ is not context free.

(b) Let us suppose $L = \{a^n b^n a^n \mid n \geq 1\}$ is context free language.

Let us assume this language can be generated by some context free grammar in Chomsky Normal Form (CNF). Let us assume

$$z = a^n b^n a^n$$

is big enough for $n = 200$. Now, we show that any method for breaking ' z ' into five parts u, v, w, x, y as

$$z = u v w x y$$

means that $u v^2 w x^2 y$ cannot be in $\{a^n b^n a^n\}$.

All words in $\{a^n b^n a^n\}$ have exactly one occurrence of the substring ' ab ' no matter what the value of ' n ' is. Now if either the v -part or the x -part has the substring ' ab ' in it, then $u v^2 w x^2 y$ will have more than one substring of ' ab ', and so it cannot be in $\{a^n b^n a^n\}$, therefore, neither v nor x contains ' ab '.

Also, all words in $\{a^n b^n a^n\}$ have exactly one occurrence of the substring ' ba ' no matter what the value of n is. Now if either the v -part or the x -part has the substring ' ba ' in it, then $u v^2 w x^2 y$ has more than one such substring, which has no word in $\{a^n b^n a^n\}$. Therefore, neither v nor x contains ' ba '.

The only possibility left is that v and x must be all a 's all b 's, or Λ . Otherwise they would contain either ' ab ' or ' ba '. But if v and x are blocks of one symbol, then $u v^2 w x^2 y$ has increased one or two clumps of solid symbols (more a 's if v contains only a 's).

The b 's and the second dump of a 's were increased, but the first a 's so the exponents are no longer the same.

We see that there is no possible decomposition of ' z ' into $uvwx y$. It should be understood that we have shown that any attempt partitioning into $u v z x y$ must fail to have $u v^2 w x^2 y$ in the language.

Therefore the pumping lemma cannot successfully be applied to the language. Hence, the language $L = \{a^n b^n a^n\}$ is not context-free.

(c) Let us assume that the language $L = \{a^p \mid p \text{ is a prime number}\}$ is a context-free. If we get a contradiction, then we can prove that it is not a context free language. We apply the following steps:

Step 1 Assume the given language $L = \{a^p \mid p \text{ is prime}\}$ is context free.

Step 2 By pumping lemma we write

$$z = a^p = u v w x y, \text{ such that } |vx| \neq 0 \text{ or } |vx| \geq 1.$$

Now we have, $|z| = |u v w x y| = p$

Step 3 Suppose $vx = a^m$, such that $m \geq 1$. Now we have, $|z| = |u v w x y| = p$. We also have, $|vx| = m$.

Step 4 We already have, $z = a^p = u v w x y$. Now by pumping lemma we write $z = u v^k w x^k y$. Then we have

$$\begin{aligned} |z| &= |u v^k w x^k y| \\ &= |u v w x y| + (k-1)|vx| \\ &= p + (k-1)m \end{aligned}$$

For $k = p + 1$, we have

$$|z| = p + (p + 1 - 1)m$$

$$= p + pm$$

$$= p(1 + m)$$

This is a contradiction because the value $p(1 + m)$ is not a prime number as it is divisible by p and $(1 + m)$. Therefore the language $L = \{a^p \mid p \text{ is prime}\}$ is not context free.

8.2

DECISION PROPERTIES AND ALGORITHM

There are so many questions regarding context-free languages which are unanswerable. The reasons for this are that no such algorithms are in existence to resolve such problems. Equivalently, we say that no such algorithm is designed yet. The following are some questions which are unanswerable:

- (i) How can we say, whether a given context-free grammar is ambiguous or not?
- (ii) How can we say, whether the complement of a given context-free language is context-free or not?
- (iii) How can we say, whether the intersection of two context-free languages is context-free or not?

There may be so many questions like the above, which are unanswerable. Suppose we have a question that requires a decision procedure to answer it, we call it decidable. If we are able to prove that there exists no algorithm to answer a question, we say that the question or problem is undecidable or unsolvable. The questions above (i)-(iii) are all undecidable.

However, there are certain questions about context-free grammars that are decidable (answerable). Some of them are the following:

- (i) For a given context-free grammar G , can we say whether or not it generates any words at all? This is the question of emptiness. This decides whether or not a CFG is empty. A CFG, which is not able to generate any word at all, is called an empty context-free grammar.
- (ii) For a given context-free grammar G , can we say whether or not the language generated by G is finite or infinite? This is the question of finiteness. A context-free grammar generating finite number of words is called a finite context-free grammar.
- (iii) For a given context-free grammar G and a particular string of letters ' s ', can we say whether or not ' s ' can be generated by the CFG G ? This is the question of membership. If $L(G)$ is the language generated by G , then $s \in L(G)$ if and only if s is in the membership of CFG G .

Theorem 8.2 There is an algorithm to determine whether or not a context-free grammar can generate any word at all.

Proof The proof of this theorem is based on constructive example. We have already seen that every context-free grammar (say G) that does not generate $\wedge$ can be written without $\wedge$ -production (Theorem to eliminate $\wedge$ -productions or null productions). A word $\wedge$ is generated by the context-free grammar if and only if S , the start symbol is nullable. If S is nullable, then we have a derivation of the form.

$$S \xRightarrow{*} \wedge$$

Therefore, the problem of determining whether $\wedge$ is in the language generated by a context-free grammar has already been solved.

Let us assume the case when $\wedge \notin L(G)$, where $L(G)$ is the language generated grammar G . In this case we can easily convert the context-free grammar G into Chomsky normal form (CNF) that generates the same words as the grammar G .

If there is a production of the form

$$S \rightarrow t$$

where $t \in \Sigma$, then t is a word in the language. It means the grammar is able to generate at least one word. If there is no such production, then we purpose the following algorithm, given by step 1 and step 2.

Step 1 For each variable (non terminal) V that has some productions of the form

$$V \rightarrow t$$

where $t \in \Sigma^*$, we choose these productions and throw out (reject) all other productions for which V is on the left side of the production. We then replace V by t in all production in which V is in the right side of the production. Thus nonterminal V is eliminated altogether.

Therefore, we have changed the grammar so that it no longer accepts the same language because it may no longer be in Chomsky normal form (after substituting t in place of V). Thus, every word that can be generated from the new grammar could have been generated by the original context-free grammar. If the original (given) context-free grammar was able to generate any word then the new one does also.

Step 2 Repeat step 1 until either it eliminates S or it eliminates no new nonterminals. Whenever, S has been eliminated, the context-free grammar produces some words: if not, than it does not.

The algorithm is clearly finite because step 1 is executed finite number of times, as there are finite number of nonterminals in the Chomsky normal form. The string of terminals that will eventually replaces S is a word that could have been derived from S if we retrace it reverse the exact sequence of steps that from the terminals to S .

For example, if we consider the following derivation tree (given by Fig. 8.3)

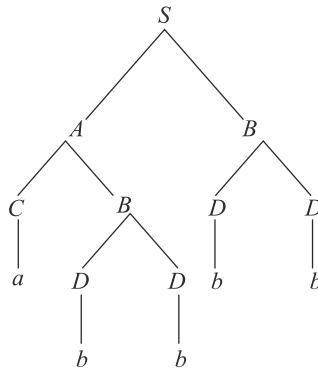


Fig. 8.3 A derivation tree generating string *abbbb*

then we can trace from leaf nodes to root as :

$$D \rightarrow b$$

It means we are coming at node label ' b ' from a node whose label is D . Therefore $D \rightarrow b$ must be a production, so we can replace all D 's with b 's in productions where D is in right side in productions. By tracing backward (from bottom to top or from leaf nodes to root), we get.

$$B \rightarrow DD,$$

therefore, $B \rightarrow DD$ is a production, so we can replace B with ' bb '. Also there is a production $C \rightarrow a$. For production $A \rightarrow CB$, we can replace A with ' abb ' and finally for production $S \rightarrow AB$, we can replace S with ' $abbbb$ '.

If we had a production $D \rightarrow c$, we could begin the back up by replacing all D 's by c instead of b .

The final conclusion is that some sequence of backward replacement will reach back to top (i.e., at start symbol) if there is any word in the language.

Example 8.2

Show that the context-free grammar G given by the following production.

$$S \rightarrow AB, A \rightarrow CA \mid CC, B \rightarrow DB \mid DD, C \rightarrow c, D \rightarrow d$$

produces at least one word.

Solution There are two productions of the form $V \rightarrow t$ where $V \in V_N$ and $t \in \Sigma^*$ therefore we can apply step 1 of Theorem 8.2 to replace all C 's by c and all D 's by d . We get

$$S \rightarrow AB, A \rightarrow cA \mid cc, B \rightarrow dB \mid dd$$

We throw out all those productions which have C or D in the left side. Now we have two productions $A \rightarrow cc$ and $B \rightarrow dd$ which are of the form $V \rightarrow t$, where $V \in V_N$ and $t \in \Sigma^*$. Therefore once again we can apply step 1 to replace all A 's by ' cc ' and all B 's by ' dd '. We get

$$S \rightarrow ccdd$$

and we throw out all those productions which have A or B in the left side. Now, we can replace S 's by ' $ccdd$ '. By the replacement of S to some terminal string, we terminate step 1.

As the start symbol S has been eliminated, the context-free grammar produces some words. This is the end of step 2. Therefore the context-free grammar produces at least one word.

Example 8.3

Show that the following context-free grammar

$$S \rightarrow AB, A \rightarrow CA \mid a, B \rightarrow DB, D \rightarrow b$$

does not produce any word at all.

Solution The productions of the form $V \rightarrow t$, (where $V \in V_N$ and $t \in \Sigma^*$), are

$$A \rightarrow a \text{ and } D \rightarrow b.$$

Therefore we can apply step 1 of Theorem 8.2 to replace all A 's by a 's and all D 's by b 's, to get

$$S \rightarrow aB, B \rightarrow bB$$

and we throw out all productions in which A or D are in the left side. Since there is no production of the form $V \rightarrow t$, where $V \in V_N$ and $t \in \Sigma^*$ in $B \rightarrow bB$, therefore, we terminate step 1, but we discover that S is not eliminated. Hence, the given context-free grammar does not produce any word at all.

Theorem 8.3 There is an algorithm to decide whether a given context-free grammar generates a finite or an infinite language.

Proof The proof is given by a constructive algorithm, and we shall show that there exists such a procedure. If the language is infinite, then there must be some words long enough so that the pumping lemma can be them. Therefore, the language generated by a context-free grammar is infinite if and only if the pumping lemma can be applied.

The central part of the pumping lemma is to find a self-embedded nonterminal A . A symbol X will be called a self-embedded nonterminal if there is a production of the form $X \rightarrow \alpha X$ or the production $X \rightarrow X\alpha$, where $\alpha \in (V_N \cup \Sigma)^*$. For example, in the context-free grammar

$$S \rightarrow AB \mid a, A \rightarrow Ab$$

the nonterminal A is self-embedded.

To check the finiteness of a language, the algorithm steps are as following :

Step 1 First determine the useless nonterminals by using the previous theorem (theorem of emptiness). Eliminate all productions involving useless nonterminals.

Step 2 Repeat steps (i) to (v) given below until a self-embedded nonterminal is found among the remaining nonterminals. For the testing of self embedded nonterminal A , perform the following steps:

- (i) Change all nonterminals A 's on the left side of productions by a different symbol (say Z) but leave all A 's on the right side of productions as it is.
- (ii) Replace all A 's by $\bar{A}$'s
- (iii) If Y is any nonterminal left side of any productions with some symbol having bar on the right side, then replace all Y 's with $\bar{Y}$.
- (iv) Repeat (iii) until nothing new is put with bar.
- (v) If Z is with bar then A is self-embedded; if not, it is not.

Step 3 If any nonterminal is self-embedded in the left side of grammar after the step 1, the language generated is infinite, otherwise the language is finite.

The explanation given in the proof of the Theorem of Emptiness is necessary to say the above procedure to be finite.

Example 8.4

Show that the language generated by the following grammar G .

$$\begin{aligned} S &\rightarrow BCa \mid bBD \mid b \\ B &\rightarrow Xb \mid bDB, C \rightarrow bBB \\ X &\rightarrow aDa \mid bB \mid aaa \\ D &\rightarrow DBbB. \end{aligned}$$

is finite.

Solution The nonterminal D is useless, while the remaining nonterminals are used in the generation of words. We now test to see whether X is self-embedded. First we eliminate productions involving D to get the following productions

$$S \rightarrow BCa \mid b$$

$$\begin{aligned}
 B &\rightarrow Xb \\
 C &\rightarrow bBB \\
 X &\rightarrow bB \mid aaa
 \end{aligned}$$

Now we introduce new symbol Z :

$$\begin{aligned}
 S &\rightarrow BCa \mid b \\
 B &\rightarrow Xb \\
 C &\rightarrow bBB \\
 Z &\rightarrow bB \mid aaa
 \end{aligned}$$

Now we put bar

$$\begin{aligned}
 \bar{X} & \text{ (X has bar)} \\
 \bar{B} &\rightarrow Xb \\
 \bar{Z} &\rightarrow bB \\
 \bar{C} &\rightarrow B \\
 \bar{S} &\rightarrow BCa
 \end{aligned}$$

As Z has the bar so the language generated by the given CFG is infinite. Hence, this CFG will generate infinite number of strings.

8.3

CLOSURE PROPERTIES OF CFLS

In the section we will consider some operations that preserve context-free language. These operations are useful not only in some constructions or proving that certain languages are context-free languages but also in proving that certain languages are not context-free. For example, we will prove that context-free languages are closed under union, concatenation and Kleene closure. A given language L can be shown not to be context free by constructing from another L that is not context free using only operations that preserve context-free languages.

Theorem 8.4 If L_1 and L_2 context-free language then $L_1 L_2$ is also context-free language.

Proof Let $G_1 = (V'_N, \Sigma, P', S')$ and $G_2 = (V''_N, \Sigma, P'', S'')$ be two context-free grammars generating languages L_1 and L_2 , respectively.

Now we have to construct a grammar G_u to generate language $L_1 \cup L_2$ such that

$$G_u = (V_N, \Sigma, P, S)$$

V'_N and V''_N have element such that $V'_N \cap V''_N = \emptyset$

Let us define V_N as

$$V_N = V'_N \cup V''_N \cup \{S\}$$

where S is a new start symbol such that $S \notin \{V'_N \cup V''_N\}$. Then P is defined as

$$P = P' \cup P'' \cup \{S \rightarrow S' \mid S''\}$$

On the one hand, if $x \in L_1 = L(G_1)$, then

$$S'' \xRightarrow{*} x_1$$

is in the grammar G_u because we can start with the production rule $S \rightarrow S'$ and continue with the derivation of x in G_1 , and similarly for $x \in L_2$. Therefore

$$L_1 \cup L_2 \subseteq L(G_u).$$

In other words if x can be derived from $S \in L(G_u)$, the first step in any derivation must be

$$S \Rightarrow S' \quad \text{or} \quad S \Rightarrow S''.$$

In the first case, all subsequent productions used must be productions in G_1 because no variable in G_2 is involved, and thus $x \in L_1 \cup L_2$. Therefore, $L(G_u) \subseteq L_1 \cup L_2$.

Theorem 8.5 If L_1 and L_2 are context-free languages, then $L_1 L_2$ is also context-free language.

Proof Let $G_1 = (V'_N, \Sigma, P', S')$ and $G_2 = (V''_N, \Sigma, P'', S'')$ be two context-free grammars generating languages L_1 and L_2 , respectively.

Suppose $G_c = (V_N, \Sigma, P, S)$ is the grammar generating $L_1 L_2$. Assume $V'_N \cap V''_N = \emptyset$. Let us define V_N for concatenated grammar G_c as

$$V_N = V'_N \cup V''_N \cup \{S\}$$

where S is the start symbol such that $S \notin \{V'_N \cup V''_N\}$. Let us define set of productions P as

$$P = P' \cup P'' \cup \{S \rightarrow S' S''\}$$

If x is a string in $L_1 L_2$, then $x = x_1 x_2$, where x_1 is in L_1 and x_2 is in L_2 . We may then derive x in grammar G_c as follows :

$$S \Rightarrow S' S''$$

$$\begin{array}{c} * \\ S'' \Rightarrow x_1 \end{array}$$

$$\begin{array}{c} * \\ \Rightarrow x_1 x_2 \end{array} x$$

Above derivation shows that x_1 is in L_1 and x_2 is in L_2 . Conversely, if x can be derived from S , then the first step of derivation must be $S \Rightarrow S' S''$ and x must be derivable from $S' S''$. Therefore, $x = x_1 x_2$, where for each i , x_i can be derived from S in grammar G_c . Hence, $x \in L_1 L_2$, which shows that concatenation of two context-free language is also context-free language.

Theorem 8.6 If L is a context-free language, then the language L^* is also context-free language.

Proof Let $G = (V'_N, \Sigma, P', S')$ be a context-free grammar generating L .

Let us suppose $G^* = (V_N, \Sigma, P, S)$ generating language L^* .

Let $V_N = V'_N \cup \{S\}$, where S is not in V'_N . The language L^* contains the string of the form $x = x_1 x_2 x_3 x_4 x_5 \dots x_m$, where each $x_i \in L$ because each x_i can be derived from S' . For the derivation x from S it is enough to be able to derive a string of m occurrences of S' . This can be accomplished by including the following productions in P

$$S \rightarrow SS' \mid \wedge \quad \text{or} \quad S \rightarrow S'S \mid \wedge$$

Suppose the set of productions for G^* is defined as

$$P = P' \cup \{S \rightarrow SS' \mid \wedge\}$$

The proof that $L^* \subseteq L(G^*)$ is straight-forward. In other words, if x is in $L(G^*)$, then it is clear that $x = \wedge$ or x can be a string of the form $S'S'S' \dots S'$ (m consecutive S') in G^* . In the second case, since the only productions in G^* beginning with S' are those in G , so we may conclude that

$$x \in L(G)^m \subseteq L(G)^*$$

Example 8.5

Suppose L_1 is the context-free language generated by productions $\{B \rightarrow AB \mid \wedge, A \rightarrow 011 \mid 1\}$ and L_2 is the context-free language generated by the productions $\{C \rightarrow DC \mid \wedge, D \rightarrow 01\}$. Construct the grammar generating language $L_1 L_2$.

Solution Let G_1 be the grammar generating L_1 and G_2 be the grammar generating L_2 . Let the grammar generating $L_1 L_2$ be $G = (V_N, \Sigma, P, S)$ and suppose L_1 is generated by the grammar $G_1 = (V_N', \Sigma', P', S')$, and L_2 is generated by the grammar $G_2 = (V_N'', \Sigma'', P'', S'')$. As G is the concatenated grammar, therefore P is defined as

$$P = P' \cup P'' \cup \{S \rightarrow S' S''\}$$

Hence, the following are productions in P :

$S \rightarrow BC$ (Here $S' = B, S'' = C$, the start symbols of grammar G_1 and G_2 respectively)

$B \rightarrow AB \mid \wedge, A \rightarrow 011 \mid 1$

$C \rightarrow DC \mid \wedge, D \rightarrow 01$

Therefore the constructed grammar is :

$$G = (\{S, A, B, C, D\}, \{0, 1\}, P, S)$$

Where, P is defined above.

Theorem 8.7 Context-free languages are closed under substitution.

Proof Suppose L is a context-free language such that $L \subseteq \Sigma^*$ and for each $a \in \Sigma$. Let L_a be a context-free language. Let L be $L(G)$ and L_a be $L(G_a)$ for each $a \in \Sigma$. Let us assume that variables of G and G_a are disjoint. Now we construct a grammar G' as :

- (i) The variables of G' are all the variables of G and the variables of G_a .
- (ii) The terminals of G' are the terminals of G_a .
- (iii) The start symbol of G' is the start symbol of G .
- (iv) The productions of G' are all productions of G_a with those productions formed by taking a production $A \rightarrow \alpha$ of G and substituting S_a (the start symbol of grammar G_a) for each instance of an $a \in \Sigma$ appearing in α .

Thus, the obtained language generated by G' is closed under substitution.

Theorem 8.8 The intersection of two CFLs may or may not be context-free

Proof We prove this theorem by breaking it into two parts:

(i) The Intersection of Two CFLs May be Context-Free We know that all regular languages are context-free, through Chomsky hierarchy. The intersection of two regular languages is also regular.

Therefore, if L_1 and L_2 are two regular languages, then L_1 and L_2 are also context-free languages. Also $L_1 \cap L_2$ is both regular and context-free.

(ii) The Intersection of Two CFLs May be Non-context Free Let the language L_1 be defined as

$$\begin{aligned} L_1 &= \{a^n b^n a^m \mid n, m \geq 1, n \text{ is not necessarily be same as } m\} \\ &= \{aba, abaa, aabba, aabbaa, \dots\} \end{aligned}$$

To prove the language L_1 to be context-free, we can represent L_1 by following context-free grammar:

$$S \rightarrow AB, A \rightarrow aAb \mid ab, B \rightarrow Ab \mid a$$

Let the language L_2 be defined as

$$\begin{aligned} L_2 &= \{a^n b^m a^m \mid n, m \geq 1, n \text{ is not necessarily the same as } m\} \\ &= \{aba, aaba, abbaa, aabbaa, \dots\} \end{aligned}$$

Make sure that L_1 and L_2 are two different languages. To prove that L_2 is context-free, we can represent L_2 by the following context-free grammar :

$$S \rightarrow AB, A \rightarrow aA \mid a, B \rightarrow bBa \mid ba$$

We observe that L_2 is a concatenation of the regular language aa^* and the context-free language (b^na^n) . We see that both languages L_1 and L_2 are context-free, but their intersection $L_1 \cap L_2$ is the language (say L) given as

$$L = L_1 \cap L_2 = \{a^n b^n a^n \mid n \geq 1\}$$

Any word in both languages has as many string a 's, followed by the same number of b 's (to be in L_1) and finally terminated by same number of a 's (to be in L_2). But L is a non-context-free language. We have proved it in an example by using pumping lemma. Therefore, the intersection of two context-free languages can be non-context-free.

Theorem 8.9 The complement of a context-free language may or may not be context-free.

Proof The proof of this theorem occurs in two parts :

(a) May be If L is a regular language, then $\bar{L}$ (compliment of L) is also regular and both are context-free languages.

(b) May not be Suppose the complement of every context-free language is context-free then if we start with two such languages L_1 and L_2 . We know that $\bar{L}_1$ and $\bar{L}_2$ are context-free by part (a). Also, we know that the union of two context-free languages is context-free. Therefore, $\bar{L}_1 \cup \bar{L}_2$ or $\overline{L_1 + L_2}$ would be context-free. Not only that, but $\overline{L_1 + L_2}$ would also be context-free, which is the complement of $(\bar{L}_1 + \bar{L}_2)$. But we also know that

$$\overline{\bar{L}_1 + \bar{L}_2} = L_1 \cap L_2$$

Therefore, the intersection of L_1 and L_2 must be context-free. We have assumed L_1 and L_2 as any arbitrary context-free languages; therefore the intersection of all CFLs would have to be context-free. But it is possible if and only if the intersection of two CFLs is also context-free and we are not sure about it.

8.4

MIXING OF CFLS AND RLS

The union of a context free language and a regular language must be context free because a regular language is itself a context free language. Is the union of a context free language and a regular language regular or not? The answer is that it is sometimes yes and sometimes not. If one language contains the other, then the union is the larger of the two languages whether it is the regular or the non-regular context free language.

There are so many cases in which the union of a non-regular context free language and a regular language is a regular language. For example, a non-regular context free language of palindromes and a regular language defined by $(a + b)^*$.

When the union of a non-regular context free language and a regular language is union of non-regular context free language $\{a^n b^n \mid n \geq 1\}$ and a regular language defined by $b^* a^*$, is non-regular context free language as seen in MyHill Nerode theorem, because each string $a^n b^n$ belongs to a different class i.e., for each class there is a unique element of b^* that completes a string in the union language.

Theorem 8.10 All regular languages are context free.

We do not give a proof here, the idea is that regular expressions can be translated into context free grammars, i.e., $a^* b^*$ can be represented by the context free grammar

$$G = (\{A, B\}, \{a, b\}, \{A \rightarrow aA \mid B, B \rightarrow bB\}, A)$$

Context free grammars are used in computer linguistics to describe natural languages.

Some of the languages which we have shown not to be regular are actually context free. The language $\{0^n 1^n \mid n \in N\}$ is given by the following grammar

$$G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S1 \mid \wedge\}, S)$$

Also the language of palindromes $\{ww^R \mid w \in \{a, b\}^*\}$ turns out to be a context free language.

8.4.1 CFG defined by Regular Expression

A regular language can be defined by regular expressions. Similarly, the language represented by regular expressions can be expressed by regular grammar. A regular grammar is also context free grammar (by Chomsky Hierarchy). One point to remember is that the constructed productions, should be of the form

$$A \rightarrow \alpha$$

where, $A \in V_N$, $\alpha \in (V_N \cup \Sigma)^*$, while converting a context free grammar into regular expression.

Theorem 8.11 Every regular language is generated by a context free grammar.

Proof Let us assume L is given regular language. Then we can design a DFA $M = (Q, \Sigma, \delta, q_0, F)$ which accepts the language L , and we have to produce a context free grammar $G = (V_N, \Sigma, P, S)$ such that the strings recognized by the DFA M forms the language generated by the CFG. We leave the Σ as it is and $V_N = Q$, so that every state becomes a nonterminal, and we set $S = q_0$. For every transition

$$\delta(q, a) = q'$$

we add a production rule

$$q \rightarrow aq'$$

and for every final state q we add a production rule

$$q \rightarrow \wedge$$

The only way of starting is from q_0 . This can be replaced with aq for every transition from q_0 to q on input 'a', and aq can then be replaced with abq' whenever there is transition

$$\delta(q, b) = q'$$

So every string thus generated consists of a string $s \in \Sigma^*$ followed by one nonterminal symbol which is a state from Q . The only way of getting rid of that nonterminal is by reaching a final state, in which case this symbol vanishes and we are left with a string that has reached a final state. Therefore the language generated by this grammar is precisely the set of strings accepted by the DFA M , which is the regular language.

Theorem 8.12 A language is regular if and only if it is generated by a CFG which is subject to the following restriction:

All productions of the form

$$P \rightarrow aP' \text{ or } P \rightarrow a \text{ or } P \rightarrow \wedge$$

where $P, P' \in V_N$, and $a \in \Sigma$.

Proof The 'if' part is shown in previous theorem (see Theorem 8.12). For the 'only if' part assume that we have a CFG whose productions comply with the given restriction. We can design an NDFA as follows. The set of nonterminals of the grammar is taken as states and one extra state say ' A '. The start symbol ' S ' is the initial state. A state $P \in V_N$ is final state if and only if there is a production $P \rightarrow \wedge$, and we define the state ' A ' to final as well. For $P, P' \in V_N$, if there is a transition from P to P' on input 'a' if there is a production of the form $P \rightarrow aP'$. There is a transition from P to A if and only if there is a production $P \rightarrow a$.

There is a path through the finite automaton labelled by a word $a_1a_2a_3\dots a_n \in \Sigma^*$ if and only if there is a sequence of applications of productions $S \Rightarrow a_1P_1 \Rightarrow a_1a_2P_2 \dots a_nP_n$. It is clear that only the last symbol in the strings generated along the path can be a nonterminal, and as the word ends this nonterminal is replaced by either a terminal or $\wedge$. It is clear it occurs if and only if the corresponding path through the NDFA ends in a final state.

Theorem 8.13 If L is a regular language then L is a context free language.

Proof There exists a constructive proof of this theorem. As we have seen earlier there exists a finite automaton for every regular language L . A context free grammar can be constructed from a DFA accepting a regular language L .

Theorem 8.14 If L has a regular context free grammar then L is a regular language.

Proof Here also there exists a constructive proof of this theorem. An NDFA can be constructed that accepts the language L for which a context free grammar can be generated.

Let $G = (V_N, \Sigma, P, S)$ be the regular context free grammar. For this grammar we construct NDFA M

$M = (Q, \Sigma, \delta, q_0, F)$, where $Q = (V_N - \Sigma) \cup \{q_f\}$, $q_0 = S$, $F = \{q_f\}$.

For each production of the form

$A \rightarrow aB$, for $a \in \Sigma$ and $A, B \in V_N - \Sigma$

add a transition

$\delta(A, a) = B$.

For each production of the form

$A \rightarrow a$, for $a \in \Sigma$

add a transition

$\delta(A, a) = q_f$.

Example 8.6

Consider the productions of context free grammar.

$S \rightarrow aS \mid bS \mid a \mid b \mid \wedge$

Generate the language of this grammar in regular expression if possible.

Solution This context free grammar is also a regular grammar. Therefore we can have a regular expression for this grammar. By considering following facts:

- (i) It generates $\wedge$, therefore the form of regular expression must be : (some expression)*.
- (ii) It also has the strings of length one, i.e., a and b .
- (iii) The production $S \rightarrow aS$ states that there is an occurrence of any number of a 's. Similarly, the production $S \rightarrow bS$ states that there is an occurrence of any number of b 's.
- (iv) The productions $S \rightarrow aS \mid bS$ states that there are all sequences of any occurrences of a 's and b 's in any order.

Above facts (i) – (iv) explain that the regular expression will be:

$(a + b)^*$

Note If we remove productions $S \rightarrow a$, $S \rightarrow b$ from example 8.6, the language generated would be the same.

Example 8.7

Construct the context free grammar G , generating the language defined by the regular expression a^*bb .

Solution Suppose the required CFG be $G = (V_N, \Sigma, P, S)$, let us divide the regular expression into two parts such that a^* is followed by bb . For a^* , we can design the productions

$S \rightarrow aS \mid \wedge$

The strings generated by productions $S \rightarrow aS \mid \wedge$ and regular expression a^* are exactly the same. But the regular expression a^*bb describes that the minimum length string is bb , therefore we must replace the production $S \rightarrow \wedge$ with $S \rightarrow bb$.

Hence, the constructed grammar is

$G = (\{S\}, \{a, b\}, \{S \rightarrow aS \mid bb\}, S)$

Note In the start of the solution of example 8.7 we have used productions $S \rightarrow aS \mid \wedge$ to generate a^* , we cannot use $S \rightarrow Sa \mid \wedge$ even though it also generates a^* . The reason is that at a later stage the productions $S \rightarrow Sa \mid bb$ will generate bba^* which is not our requirement.

Example 8.8

Construct the context free grammar G , generating the language defined by the regular expression $(a+b)^*bbb(a+b)^*$.

Solution The regular expression $(a+b)^*bbb(a+b)^*$ describes the string over $\{a, b\}$ such that there is a substring bbb enclosed within any string of a 's and b 's and also possibly an empty string $\wedge$. Make sure all the strings in $(a+b)^*bbb(a+b)^*$ are not palindromes. The first and last (third) parts need not necessarily be identical.

Let us divide the regular expression $(a+b)^*bbb(a+b)^*$ into three parts as $(a+b)^*$, bbb , and $(a+b)^*$. Suppose first $(a+b)^*$ in CFG is denoted by A and second occurrence of $(a+b)^*$ in CFG is denoted by C . Then there should be a production of the form

$$S \rightarrow AbbbC \quad \text{or} \quad S \rightarrow ABC, B \rightarrow bbb$$

where A and C are representing regular expression $(a+b)^*$ at different positions. Here we prefer the first production $S \rightarrow AbbbC$. The production for $(a+b)^*$ can be written as

$$A \rightarrow aA \mid bA \mid \wedge$$

and

$$C \rightarrow aC \mid bC \mid \wedge$$

Hence the required context free grammar is

$$G = (\{S, A, C\}, \{a, b\}, \{S \rightarrow AbbbC, A \rightarrow aA \mid bA \mid \wedge, C \rightarrow aC \mid bC \mid \wedge\}, S)$$

Example 8.9

If L is the context free language corresponding to the regular expression $(011 + 1)^*(10)^*$

construct context free grammar G generating the language L .

Solution Let us divide the regular expression $(011 + 1)^*(10)^*$ into two parts $(011 + 1)^*$ and $(10)^*$. Let L_1 be the language corresponding to sub-expression $(011 + 1)^*$ that can be generated by context free grammar G_1 and L_2 the language corresponding to sub-expression $(10)^*$ that can be generated by context free grammar G_2 .

The production of grammar G_1 can be constructed as

$$A \rightarrow 011A \mid 1A \mid \wedge$$

That will represent the expression $(011 + 1)^*$.

The production of grammar G_2 can be constructed as

$$B \rightarrow 10B \mid \wedge$$

That will represent the expression $(10)^*$.

Now we have to generate the concatenated language $L(G_1) L(G_2)$. Let this concatenated language (say $L(G)$) is generated by context free grammar G . The grammar G can be defined as

$$G = (\{S, A, B\}, \{0, 1\}, \{S \rightarrow AB, A \rightarrow 011A \mid 1A \mid \wedge, B \rightarrow 10B \mid \wedge\}, S)$$

Summary

- **Pumping Lemma for CFL** It is a method to prove that certain languages are not context-free.
- **Closure Properties of CFL** CFLs are closed under union, concatenation and closure.
- **Complement of a Context-free Language** The complement of a context-free language may or may not be context-free.
- **Intersection of Two Context-free Languages** The complement of two context-free languages may or may not be context-free.
- **CFG Defined by Regular Expression** A regular language can be defined by regular expressions. Similarly, the language represented by regular expressions can be expressed by regular grammar.
- **Applications of CFG** Context free grammar is a useful tool for defining programming languages such as to define the syntax of C language and definition part of HTML (Hyper Text Markup Language).

Review Questions

- 8.1 State True/False with justification, whether the union of a regular and a context free language is context free.
- 8.2 State True/False with justification, whether the complement of a context free language is context free.
- 8.3 Show that following languages are not context free using pumping lemma
 - (i) $L = \{a^p \mid p \text{ is a perfect square}\}$
 - (ii) $L = \{a^n b^n c^n d^n \mid n \geq 0\}$
- 8.4 Let G_1 be the CFG generating language $L(G_1)$ with productions $S \rightarrow AS \mid a \mid b$, and G_2 be the CFG generating language $L(G_2)$ with productions $A \rightarrow 011 \mid 1B$, $B \rightarrow 0B \mid 1$. Determine the grammar generating the following languages:
 - (i) $(L(G_1) + L(G_2))^* L(G_1)$
 - (ii) $L(G_1) L(G_2) L(G_1)$
- 8.5 Show that the language generated by the grammar G which has productions

$$S \rightarrow XY, X \rightarrow YZ \mid a, Y \rightarrow ZZ \mid b, Z \rightarrow a$$
 is finite.
- 8.6 State whether the context-free grammar given by productions :

$$S \rightarrow aAS \mid AB, A \rightarrow BC \mid b, B \rightarrow AA \mid a$$
 produces any word or not ? If not, give the reason.
- 8.7 Construct context-free grammars for each of the languages defined by following regular expressions :
 - (i) ab^*
 - (ii) $a^* b^*$
 - (iii) $(baa + abb)^*$
 - (iv) $ab^*(a^* + b^*)^*$
 - (v) $((aa)^* + (bb^*))^*$
 - (vi) $(ab^* + ba^* + b^*)^*$

8.8 Find the languages in terms of regular expression (if there is a possibility) defined by following context-free grammars :

(i) $S \rightarrow bAb, A \rightarrow aAa \mid bAb \mid a$, (ii) $S \rightarrow AB, A \rightarrow a, B \rightarrow b$ (iii) $S \rightarrow aS \mid aA \mid \Lambda, A \rightarrow \Lambda$,

8.9 State true/false with justification :

- (i) The intersection of two CFLs is also CFL.
- (ii) The closure of a CFL is also a CFL.
- (iii) The concatenation of CFL with a regular language is CFL.
- (iv) The union of CFL and regular language is always a regular language.
- (v) All regular languages are also context-free languages and vice versa.

8.10 Prove or disprove that the difference of two context free languages is context free.

Hints

- 8.7** (i) $S \rightarrow aB, B \rightarrow bB \mid \Lambda$,
(ii) $S \rightarrow AB, A \rightarrow aA \mid \Lambda, B \rightarrow bB \mid \Lambda$, or
 $S \rightarrow aS \mid A, A \rightarrow bA \mid \Lambda$,

(iii) $S \rightarrow AS \mid \Lambda, A \rightarrow B+C, B \rightarrow baa, C \rightarrow abb$

8.8 (i) No regular expression exists because it is a context-free language. Basically it is a language of odd

length palindromes over $\{a, b\}$ starting and ending with b and having a as middle symbol.

(ii) ab , (iii) a^*

8.9 (i) False (ii) True (iii) True (iv) False (v) False

Graded Exercises

****8.1** By using pumping lemma, show that the following languages are not context-free :

- (i) $L = \{a^n b^n c^i \mid i \neq n\}$
- (ii) $L = \{i \text{ is an integer}\}$
- (iii) $L = \{a^i b^j c^i d^j \mid i \geq 1 \text{ and } j \geq 1\}$
- (iv) The set of all strings over $\{0, 1\}$ in which the occurrences of 0's and 1's are same.

***8.2** Show that the syntax of C language is context free.

***8.3** Show that the intersection of a regular and a context free language is a context free language.

****8.4** If a CFG is in CNF, then show that all derivation trees for yield with length n have $2n-1$ internal nodes.

* Difficulty level 1

** Difficulty level 2

*** Difficulty level 3

Objective Questions

1. If L is a deterministic CFL and R is a regular language, then

- (a) $L \cap R$ is a deterministic CFL.
- (b) $L \cap R$ is regular language
- (c) (a) is true but (b) is not
- (d) (b) is true but (a) is not

2. Consider the language, $L = \{a^n b^n c^n | n \geq 1\}$, then
 - (a) L is context-free.
 - (b) L is context-sensitive.
 - (c) L is context-sensitive but not context-free.
 - (d) both (b) and (c).
3. If L_1 and L_2 are two context-free languages, then which of the following is false
 - (a) $L_1 L_2$ is a context-free language.
 - (b) $L_1 \cap L_2$ may be or may not be context-free language.
 - (c) $L_1^* L_2^*$ is not a context-free language.
 - (d) none of above
4. For a language $L = a^n b^m | n \geq m$, and $m, n \neq 1\}$ which of the following statements is true?
 - (a) Complement of L is context-free
 - (b) L is not regular
 - (c) Complement of L is not regular
 - (d) all of these
5. If $L_1 = \{a^n b^m | n \geq m\}$ and $L_2 = \{a^n b^m | m \geq n\}$ are two languages, then :
 - (a) Both languages are regular languages.
 - (b) Both languages are context-free languages.
 - (c) The union of these languages is regular.
 - (d) all of the above.
6. The language generated by the grammar $S \rightarrow AB, A \rightarrow BC | a, B \rightarrow CC | b, C \rightarrow a$ is :
 - (a) finite
 - (b) empty
 - (c) both (a) and (b)
 - (d) none of these
7. If L' is the complement of language $L = \{a^n b^m | n \geq m$, and $m, n \geq 1\}$, then :
 - (a) L is context-free
 - (b) L is not regular
 - (c) L' is not regular
 - (d) all of these
8. Which of the following statements is true?
 - (a) Any regular language has an equivalent CFG.
 - (b) Some non-regular languages cannot be generated by any CFG.
 - (c) Some regular languages cannot be generated by any CFG.
 - (d) both (a) and (b)
9. Which of the following statements is true?
 - (a) GNF is used to check the emptiness of CFG.
 - (b) GNF is used to prove the equivalence of CFG and NPDA,
 - (c) CNF is used to check the emptiness of CFG.
 - (d) both (b) and (c)
10. Which of the following statements is false?
 - (a) The intersection of two context-free languages may or may not be context-free.
 - (b) The complement of a context-free language may or may not be context-free.
 - (c) There is no such algorithm that could decide whether a CFL generates a finite or infinite language.
 - (d) none of the above
11. Let $L_1 = \{a^i b^j | i > 0\}$ and $L_2 = (a + b)^*$ be two languages, then :
 - (a) their union is CFL.
 - (b) their union is regular language.
 - (c) their intersection is regular language.
 - (d) all of these
12. Let and $L_1 = \{a^i b^j | i > j \geq 1\}$ and $L_2 = \{a^i b^j | i < j\}$. The union of L_1 and L_2 is given by
 - (a) $\{a^i b^j | i > j \geq 1\}$
 - (b) $\{a^i b^j | i, j \geq 1\}$
 - (c) $\{a^i b^j | j > j \geq 1\}$
 - (d) $\{a^i b^j | i, j \geq 1, i \neq j\}$

13. The intersection of two languages $L_1 = (a + b)^*a$ and $L_2 = b(a + b)^*b$ is :
 (a) $(a + b)^*a$ (b) $a(a + b)^*b$ (c) $(a + b)^*ab(a + b)^*$ (d) none of these
14. The union of languages $L_1 = \{a^n b^n \mid n \geq 5\}$ and $L_2 = \{\text{all strings over } (a, b)\}$
 (a) $a^*b^* \cup (a + b)$ (b) $a(a + b)^*b$ (c) $(a + b)^*$ (d) none of these

References for Extra Reading

1. Susan L. Graham, Michael A. Harrison, and Walter L. Ruzzo, *An improved context-free recogniser*, ACM, 1980.
2. Pullum, Geoffrey K.; Gerald Gazdar, *Natural languages and context-free languages*, Linguistics and Philosophy, 1982.
3. Marcus, S., *Contextual grammars and natural languages*, in G. Rozenberg, A. Salomaa, eds., *Handbook of Formal Languages*, Springer, Berlin 1969.
4. Cohen, D.I.A., *Introduction to Computer Theory*, John Willy and Sons (Asia) Pvt. Ltd. Singapore, 1991.

Online Sources

1. <http://www.cs.sun.ac.za/~rw324/documents/rw324w5.pdf>
2. <http://www.kornai.com/MatLing/cfffinal.pdf>

9

Turing Machines

❖ Chapter Objective ❖

In this chapter we will present the most powerful model of computation called Turing machine. We will see a Turing machine as more capable model compared to a push down automation and a finite automaton. We have already seen that a 2-stack PDA is much capable than finite automata and push down automata.

We will start our discussion with the model and definition of a Turing machine. Then we will design some simple Turing machines for regular languages. In this sequence we will see how Turing machines are designed for non regular languages. We will discuss recursively enumerable languages as Turing machine languages. We will see how Turing machines are designed for computable functions.

Then we will see how a specific Turing machine can be modified to make it more powerful. For this, we will discuss multi-tape, multi-head, and multi dimensional Turing machines. We will discuss how a Universal Turing Machine (UTM) is the model of a general purpose computer.

□ Introduction □

The Turing Machine (TM) was invented by Alan Turing in 1936. Off-line Turing Machines had been discussed by Hartmanis, Lewis and Stearns in 1965. Turing machines are ultimate models for computers, as they necessarily have output capabilities. Output is very important, so important that a programme without statements might seem totally useless because to the user it would never convey the result of its calculations.

The languages accepted by a Turing machines are said to be recursively enumerable. The term 'enumerable' is derived from the fact that it is precisely those languages whose strings can be enumerated or listed by a Turing machine. Turing machines are studied in both classes of languages these define (known as recursively enumerable sets), and the class of integer functions they compute (known as partial recursive functions).

In particular, a Turing machine is equivalent to the computational power of digital computers as we know today and also to all the most general mathematical concepts of computation. Now, the Turing machine has become the accepted formalisation of an effective algorithm. It is clear that one cannot

prove that the Turing machine model is equivalent to our intuitive concept of a digital computer, but there are undeniable arguments for this equivalence, which is known as *Church's hypothesis*.

Turing machine is an abstract machine that is widely accepted as a general model of computation. The basic operations of such a machine are analogous in their straightforwardness to those of the earlier machines. In particular, this model can accommodate the idea of a stored-program computer, so that we can have a single 'universal' machine to execute any algorithm by providing it an input string that includes an encoding of the algorithm.

There are a number of variations on the original Turing machine and a few of these are discussed later in this chapter. The version we begin with is not exactly the one that Alan Turing proposed, but its basic features are similar. Like a finite automaton or a push down automaton, it has a finite set of states, corresponding to Turing's terminology to the possible 'states of mind' of the human computer.

We consider a Turing machine (TM) a simple mathematical model of a general purpose computer. In other words, Turing machine models the computing power of a digital computer. Turing machines are capable of performing any calculation which can be performed by a general computing machine.

❖ Did you Know ?

Alan Matheson Turing (June 23, 1912, to June 7, 1954) the creator of Turing Machine was often criticised for his hand writing. For most of his early years, he struggled at English. But he was so popular with his original ideas that he produced exceptional solutions in mathematics. Despite producing irregular answers, Turing won almost every possible prize of mathematics at Sherburne. His headmaster at Sherburne reported: "If he is to be exclusively a scientific specialist, he is killing his time at a public school." The appraisal of this establishment was almost correct.

9.1

MODEL OF TURING MACHINES

A Turing machine can be visualised as a single, one dimensional array of cells, each of which can hold a single symbol to be processed. This array extends infinitely in both directions and is therefore capable of holding an ultimate amount of information. This information can be read and modified in any order. Such a storage device is called a tape because it is analogous to the magnetic tapes used in actual computers as secondary memory.

A Turing machine is an automaton whose temporary storage is a tape. A read-write head is associated with this tape that can move in both directions (left or right) and it can read and write a single symbol on each move. Thus a Turing machine can be thought of as a finite-state automaton connected with a read/write head.

The input to the finite state automaton, as well as the output from the finite state automaton is affected by the read/write (R/W) head which can examine one cell at a time. In one move, the machine examines the current symbol under the read/write head on the input tape and the present state of automaton to determine the following:

- (i) a new symbol to be written on the input tape in the cell under the read/write head,
- (ii) the direction of movement of read/write head along the tape (either one cell left, say L ; or one cell right, say R),
- (iii) the next state of the automaton, and
- (iv) whether to halt or not.

The Turing machine model is given by Fig. 9.1, below.

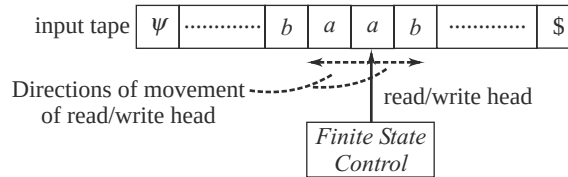


Fig. 9.1 Basic model of a Turing machine

❖ Good to Know!

A Turing machine is said to be in halt state (denoted by h or H) if it is not able to move further.

9.2

DEFINITION OF TURING MACHINE

A Turing machine (TM) denoted by M , is defined as 7-tuple,

$$M = (Q, \Sigma, \Gamma, \delta, q_0, \#, H).$$

where,

Q = a finite non-empty set of states,

Σ = a non-empty set of input symbols (alphabets) which is a subset of Γ and $\# \notin \Sigma$,

Γ = a finite non-empty set of tape symbols, i.e., $(\Sigma \cup \#)$

δ = the transition function

$$Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\},$$

It is mapping from the present state of automaton and tape symbol to next state, tape symbol and movement of head in left or right direction along the tape. This tells us that a Turing machine is in some present state $q \in Q$, after scanning an input symbol from $(\Sigma \cup \#)$ it goes to next state $q' \in Q$ by writing a symbol $x \in \Gamma$ in the current cell of input tape and finally takes a left or right move.

q_0 = the initial state, and $q_0 \in Q$

$\#$ = the blank and $\# \in \Gamma$, and

H = halt state, $H \in Q$.

Make sure that $\#$ is not a symbol. A cell with $\#$ indicates that it does not contain any alphabet at all. Also δ may not be defined for some elements of $Q \times \Gamma$.

The transition function $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ states that if a Turing Machine is in some state (from set Q), by taking a tape symbol (from set Γ), it goes to some next state (from set Q) by overwriting (replacing) the current symbol by another or same symbol and the read/write head moves one cell either left (L) or right (R) along the tape.

Let us consider the transition

$$\delta \in Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, \text{Stop}\}$$

This transition function defines how the Turing machine behaves if it is in state q and the symbol on the tape is ' x '. If

$$\delta(q, x) = \text{stop},$$

then the machine stops, otherwise if

$$\delta(q, x) = (q', y, D)$$

the machine reaches into state q' (next state), writes the symbol ' y ' on the tape (by replacing the symbol x) and moves one cell left if $D = L$, or right, if $D = R$.

Similarly, the transition function $\delta(q_0, 0) = (q_1, \#, R)$ describes that the Turing machine moves from state q_0 to q_1 on input symbol 0 by # (called blank).

A Turing machine with transitions $\delta(q_2, 1) = (q_2, 1, R)$ and $\delta(q_2, 0) = (q_3, 1, L)$ describes the process of reaching right most 1's until encountering a '0'. If 0 is replaced by 1, then there is a left move of read/write head and machine enters in state q_3 from state q_2 .

Let us consider a Turing machine which is designed in such a way that

- (i) it can scan a blank as well as input symbol from Σ ,
- (ii) the set of states may contain the halt state,
- (iii) the read/write head can move one cell left, one cell right or may be stationary in the same cell (i.e., the D may be L, R or stop),

then the transition function of such a Turing machine is defined as

$$Q \times (\Sigma \cup \{\#\}) \rightarrow Q \cup \{h\} \times (\Gamma) \times \{L, R, \text{stop}\}$$

is a partial function that is possibly undefined at certain points. If the tape of a Turing machine is of finite length and if the read/write head of a Turing machine is scanning cell 0 (the leftmost cell), the head is not allowed to take a left move. This is one of the situations in which the Turing machine is said to be crash. If this does not happen and the state is a halt state, we say that the move causes the Turing machine to halt. Note that, once the Turing machine halts, the machine cannot move further because the transition function δ is not defined for such transition.

If $q \in Q$, $q' \in Q \cup \{h\}$, $X, Y \in \Gamma \cup \{\#\}$, and $D \in \{L, R, S\}$, then we interpret the transition

$$\delta(q, X) = (q', Y, D)$$

to mean that when the Turing machine is in state q and the symbol under read/write head is X , the machine replaces symbol X by symbol Y on that cell, changes to state q' , and either moves the tape head one cell left, moves one cell right, or leaves it stationary, depending on whether D is L, R or stop , respectively.

Hanging Configuration of Turing Machine The hanging configuration of a Turing machine is defined as the configuration of the form $(q, \wedge bv)$ where $v \in \Sigma^*$, such that the transition $\delta(q, b)$ tells the

machine to take a left move. The Turing machine configuration (q', ubv) yields the configuration (q, xcy) where $x, c, y \in \Sigma^*$ in single step if and only if the transition $\delta(q', b)$ changes the configuration (q', ubv) to (q, xcy) where $u \in \Sigma^*$.

9.3

HALT AND CRASH CONDITIONS

A Turing machine is said to be in *halt* state if it is not able to move further. The set of string followed from initial state to halt state is the language accepted by a Turing machine. The halt state in a Turing machine is almost similar to a final state in a finite automaton or a push down automaton with the only difference that when a Turing machine enters into a halt state it does not take any further move.

If the read/write head of a Turing machine is over the left most cell (i.e., input ψ) and the machine is asked to take a left move it is called *crash* condition. Similarly, if the read/write head of a Turing machine is over the right most cell (i.e., input $\$$) and the machine is asked to take a right move it is also called a crash condition. Following transitions represent crash condition:

$$\delta(q_i, \psi) = q_j, \psi, L), \delta(q_i, \$) = (q_k, \$, R) \quad \text{for } q_i, q_j, q_k \in Q$$

9.4

EQUIVALENCE OF TWO TURING MACHINES

Two Turing machines M_1 and M_2 are said to be equivalent if both accept the same set of strings. In other words, two Turing machines M_1 and M_2 are said to be equivalent if both machines on scanning same set of strings terminate to some halting configuration.

9.5

REPRESENTATION OF TURING MACHINES

The following methods are used to describe the Turing machine:

- (i) Instantaneous Descriptions (IDs) using move-relations,
- (ii) Transition Table, and
- (iii) Transition Diagram (transition graph).

9.5.1 Representation by Instantaneous Descriptions

An instantaneous description of a Turing Machine is defined in terms of the entire input string and the current state. Remember, we have defined instantaneous description of a push down automaton in terms of present state, input string to be processed, and the top symbol of push down store (stack). But, the input string to be processed is not enough to define as instantaneous description of a Turing machine because the read/write head can move in both (left or right) directions.

An instantaneous description of a Turing machine M is defined as a string $\alpha\beta\gamma$, where β is the present state of Turing machine, α and γ are substrings of string ' $\alpha\gamma$ '. The string α is processed (scanned) string, while γ is the remaining string to be scanned. The first (left most) symbol of substring γ is the symbol (present input) under read/write head. Thus, an instantaneous description of a Turing machine has substring α (also called left sequence) then a present state (also called current state) followed by

present input symbol (left most symbol of substring γ) and then finally remaining symbols of substring γ (also called right sequence).

For example, if the input string is $a_1 a_2 a_3 a_2 a_1 a_2$, the symbol under read/write head is a_3 , and the Turing machine is in state q_2 then we have following snapshot of this Turing machine (see Fig. 9.2).

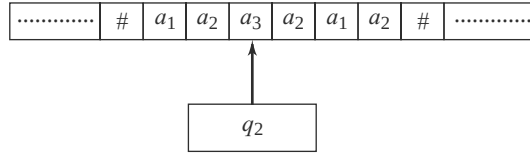
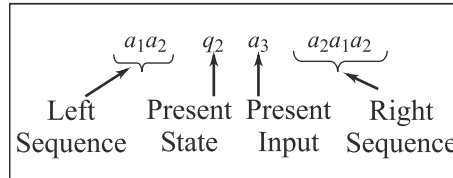


Fig. 9.2 Snapshot of a Turing machine

Below, we see that $\alpha = a_1 a_2$, $\beta = q_2$ and $\gamma = a_3 a_2 a_1 a_2$, where a_2 (the leftmost symbol of γ) is the symbol (current input symbol) under read/write head. This can also be seen as an instantaneous description as following:



Move Relations in a Turing Machine In this section we see how the transitions in a Turing machine are represented by using move relation (denoted by symbol $\vdash$). In case of pushdown automata, the change in instantaneous description (ID) of the Turing machine is induced by $\delta(q, x)$, where $q \in Q$ and x is the current input.

Suppose $\delta(q_1, x_3) = (q_2, y, R)$, and $s = x_1 x_2 x_3 \dots x_n$. The input string 's' to be processed is $x_1 x_2 x_3 \dots x_n$, and the current symbol under read/write head is x_3 . Therefore the ID before processing the input symbol x_3 is

$$x_1 x_2 q_1 x_3 x_4 \dots x_n.$$

After the processing of x_3 , the resulting ID will be

$$x_1 x_2 y q_2 x_4 x_5 \dots x_n,$$

because according to $\delta(q_1, x_3) = (q_2, y, R)$, the symbol ' x_3 ' is replaced by ' y ' and there is a right move. Therefore, the current symbol under read/write head is ' x_4 '. This change of ID can be represented by

$$x_1 x_2 q_1 x_3 x_4 \dots x_n \vdash x_1 x_2 y q_2 x_4 x_5 \dots x_n.$$

If we denote an ID by I_j for some j , and another ID by I_k ; and the machine is able to reach from ID I_j to I_k in some moves then we can represent these transitions by using reflexive-transitive closure of relation $\vdash^*$ as given below

$$I_j \vdash^* I_k$$

Also, $\vdash^*$ could be split as

$$I_j \vdash I_2 \vdash I_3 \vdash I_4 \vdash \dots \vdash I_{k-1} \vdash I_k$$

for some IDs, $I_2, I_3, \dots, I_{k-1}$.

9.5.2 Representation by Transition Table

In this section we will define δ in terms of a table called transition table (or state table). For transition,

$$\delta(q_1, a_1) = (q_2, a_2, D),$$

we can write a_2Dq_2 under a_1 -column and q_1 -row in the transition table constructed. So if we get a_2Dq_2 in the transition table, it means that a_2 is overwritten in the current cell, D gives the direction (of movement) of the read/write head either left (L) or right (R), and q_2 is the next state in which the Turing machine enters. An example of a Turing machine is given below, which is represented by a transition table (Table 9.1). This Turing machine has seven states with q_1 as initial state and H as halt state (also called accepting state). The tape symbols are 1, 2, 3 and # (blank).

Table 9.1 A Turing machine represented by a transition table

PRESENT STATE	INPUT-TAPE SYMBOLS			
	1	2	3	#
$\rightarrow q_1$	$\#Rq_2$	—	—	$\#Rq_1$
q_2	$1Rq_2$	$\#Rq_3$	—	$\#Rq_2$
q_3	—	$2Rq_3$	$\#Rq_4$	$\#Rq_3$
q_4	—	—	$3Lq_5$	$\#RH$
q_5	$1Lq_6$	$2Lq_5$	—	$\#Lq_5$
q_6	$1Lq_6$	—	—	$\#Rq_1$
H	—	—	—	—

In Table 9.1, we see that if the Turing machine is in state q_1 then on input 1 it goes to next state q_2 replacing 1 by # (the blank) and the read/write head moves one cell in right direction. Similarly, if the machine is in state q_5 , then on input 2 the machine remains in state q_5 replaces 2 by 2 and the read/write head moves one cell left.

Example 9.1

Show the computation sequence for string '1022' for the Turing machine given by the following table:

Table 9.2 Transition table of Example 9.1

PRESENT STATE	INPUT-TAPE SYMBOLS			
	0	1	2	#
$\rightarrow q_1$	$\#Rq_2$	$1Rq_2$	—	$\#Rq_1$
q_2	$0Rq_2$	$\#Rq_3$	$\#Rq_3$	$\#Rq_2$
q_3	—	$1Rq_3$	$\#Rq_4$	$\#Rq_3$
q_4	—	—	$2Lq_5$	$\#LH$
q_5	$0Lq_6$	$1Lq_5$	—	$\#Lq_5$
q_6	$0Lq_6$	—	—	$\#Rq_1$
H	—	—	—	—

Solution q_1 is the initial state and input string is '1022'. The computation sequence is given as $q_1 \#1022\# \mid \#q_1 1022\# \mid \#1q_2 022\# \mid \#10q_2 22\# \mid \#10\#q_3 2\# \mid \#10\#\#q_4 \#\# \mid \#10\#H\#\#\#$. As $\delta(H, \#)$ is not defined in transition table, the Turing machine halts.

9.5.3 Representation by Transition Diagram

We had introduced transition diagrams in Chapter 2 to represent finite automata. In this section we are going to represent Turing machines. The states of Turing machines are represented by using exactly the same notations we used for finite automata. Also, we use directed edges to show transition from one state to another. The labels of edges are triples (a set of three parameters) of the form (α, β, D) , where $\alpha, \beta \in \Gamma$ and $D \in \{L, R, stop\}$. The directed edge from state q to q' with label (α, β, D) represents the transition

$$\delta(q, \alpha) = (q', \beta, D)$$

where, D is either L, R , or $stop$. The triple (α, β, D) indicates that the symbol under read/write head is α , and α is replaced by β as a result of processing of symbol α and the direction of movement of read/write head is described by the value of D .

Let us consider a triple $(1, \#, R)$ as a label of edge from state q_3 to q_4 . We say that the symbol 1 is under read/write head. The symbol 1 is replaced by # (the blank) while reaching the machine in state q_4 from q_3 and there is a right movement of read/write head. The transition system of a Turing machine can be defined as 5-tuple $(q_p, \alpha, \beta, D, q_N)$, where,

q_p = the present state, and $q_p \in Q$

α = the current symbol under read/write head,

β = the replaced symbol,

D = the direction of head movement L, R or $stop$, and

q_N = the next state, and $q_N \in Q$.

Thus, each Turing machine can be described by a collection of 5-tuples representing (α, β, D) as the label of the directed edges. Make sure that the notations for initial and intermediate states are almost same as in transition diagrams of finite automata (e.g., an initial state by a circle with an incoming arrow, an intermediate state by just a circle) and a halt state also by a circle with state name h or H . Figure 9.3 is a transition system of a Turing machine.

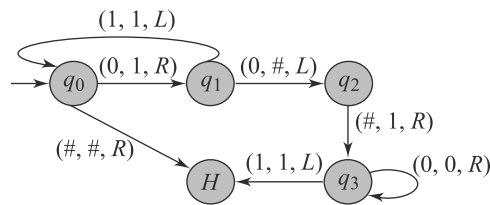


Fig. 9.3 The transition diagram of a Turing machine

In Fig. 9.3, we see that if a Turing machine is in state q_1 , then on input 0 it goes to state q_2 replacing 0 by # with a left movement of read/write head; if the input is 1 then machine goes to state q_0 without replacing current symbol with another symbol (or replacing 1 by 1) with a left movement of read/write head. Finally, when the machine reaches to halt state, it does not take further move.

Example 9.2

Consider the Turing machine given by the following table:

Table 9.3 Transition table of Example 9.2

PRESENT STATE	INPUT-TAPE SYMBOLS	
	1	#
$\rightarrow q_0$	1R q_0	1R q_2
q_1	1R q_1	#L q_2
q_2	#L q_3	—
q_3	1L q_3	#RH
H	—	—

Draw the transition diagram of this Turing machine.

Solution As the transition table shows that q_0 is the initial state and H is the halt state. The transition diagram is given by Fig. 9.4.

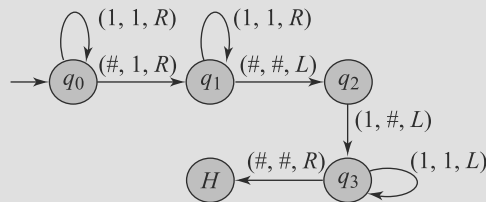


Fig. 9.4 Transition diagram of Example 9.2

9.6

DESIGNS FOR TURING MACHINES

We start with the following theorem:

Theorem 9.1 Every regular language has a Turing machine that accepts exactly it.

Proof Let us consider a regular language L . We have a finite automaton that can accept language L . We change the labels of edges 0 and 1 by $(0, 0, R)$ and $(1, 1, R)$, respectively, from one state to another, where $(0, 1 \in \Sigma)$. We change the initial state to the word START and eliminate the final state(s) by converting these into nonfinal states and add halt state from state(s) it/these was/were connected, with label(s) $(\#, \#, R)$ leading to a halt state.

The Turing machine reads the input string moving from one state to another exactly as we have the Turing machine in finite automaton. When we reach to the end of the input string, the machine halts when the read/write head reads # (the blank) in the next cell. If the Turing machine state corresponds to

the final state of finite automaton, we take the edge labelled $(\#, \#, R)$ to halt. Thus, the accepting strings are the same for a Turing machine and a finite automaton.

A string $s \in \Sigma^*$ is said to be accepted by a Turing machine defined as $M = (Q, \Sigma, \Gamma, \delta, q_0, \#, H)$ if

$$q_0 s \vdash^* \alpha H \beta$$

for some $H \in Q$ and $\alpha, \beta \in \Gamma^*$. The Turing machine M does not accept string 's' if the Turing machine does not halt, i.e., it does not reach to halt state. In other words, the language accepted by a Turing machine M (denoted by $L(M)$), is the set of those strings in Σ^* that cause Turing machine to enter in a halt state when placed justified at the left, on the tape of a Turing machine, with the Turing machine in state q_0 (initial state), and the read/write head of the Turing machine at the left most cell.

It is important to note that there are several ways an input string might fail to be accepted by a Turing machine. First, it can lead to some non-halting configuration from which the Turing machine cannot move (i.e., to some combination of non-halting states and current tape symbols for which the transition function is not defined). The second thing is, at some point in the processing of the string, the tape head in scanning the first or leftmost cell and the next move specifies moving the head left, off the end of the tape. In either of these cases, we say that the Turing machine crashes, which, of course, is not the same as saying that it halts. There is still a third possibility; an input string might cause the Turing machine to enter in an infinite loop (a never-ending sequence of moves). We can describe these three cases informally as follows:

A string can fail to be accepted by being explicitly rejected, as a result of the crashing of the machine, or it can fail to be accepted because the machine is unable to make up its mind whether to accept or not.

The difference between these outcomes is that in the second case there is no outcome. Someone waiting for the answer is left in suspense. The machine continues to make moves, but the observer is never sure that it is not about to halt or crash.

A Turing machine is the most powerful machine among all automata. If we have to design a Turing machine, if for that purpose a finite automaton can also be designed, then we construct finite automata and transform it into an equivalent Turing machine by changing the label on transition arrows and introducing a halt state after the final state(s) (see Fig. 9.5) and some new initial state.

Basically we use the following steps for designing a Turing machine for regular languages:

- (i) Construct the finite automaton M for regular language L having no $\wedge$ -moves.
- (ii) Introduce a new initial state by making transition from this to previous initial state by input $(\#, \#, R)$, and convert previous initial state into intermediate state.
- (iii) Introduce a halt state by making transitions from all the final states of finite automata by input $(\#, \#, R)$, and convert final state(s) into nonfinal states.
- (iv) Replace all inputs at different arrows of finite automata from *input* to $(input, input, R)$.

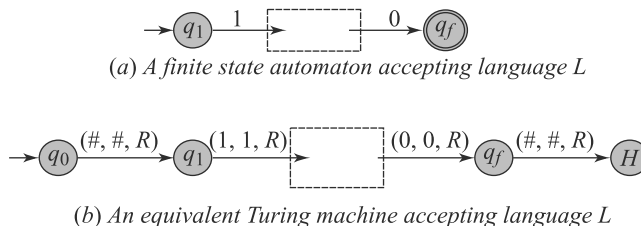


Fig. 9.5 Transforming an FA into an equivalent Turing machine

Example 9.3

Construct a Turing machine accepting the language accepted by finite automata given below (Fig. 9.6).

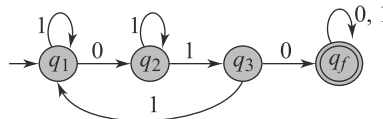


Fig. 9.6 Finite automata of Example 9.3

Solution The given finite automaton accepts the language

$$L = \{x \in \{0, 1\}^* \mid x \text{ contains the substring } 010\}.$$

The Turing machine constructed to accept the language L is given by Fig. 9.7. It consists of six states, input alphabets and tape symbols are $\{0, 1\}$. As the language L is regular, therefore the Turing machine can process input string essentially the way a finite automaton is forced to move the head to the right at each step and never changing (replacing) the tape symbol. Make sure that this type of processing is not sufficient to recognise a non-regular language.

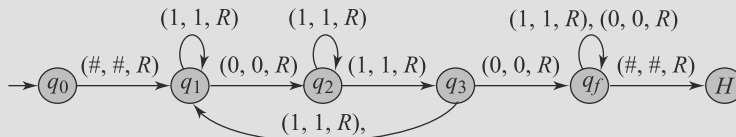


Fig. 9.7 Turing machine of Example 9.3

As soon as the Turing machine discovers the substring '010' on the tape, it halts, and thereby accepts the entire input string, even though it may not have read all of it. This is in contrast to a finite automaton or a pushdown automaton; in either of these cases, 'accept' means to accept the string of input symbols read so far. Since all moves in the Turing machine are to the right (R), it cannot halt (H state) without reading the blank to the right of the last input symbol.

Note In Fig. 9.5(b) and 9.7 the state q_f is not a final state. It can also be renamed with any state like q_7 or q_8 .

Example 9.4

Construct a Turing machine that accepts all strings over $\{0, 1\}$ with an even number of 0's and even number of 1's.

Solution If we construct a finite automaton to accept all strings over $\{0, 1\}$ with an even number of 0's and an even number of 1's, we have FA given by Fig. 9.8.

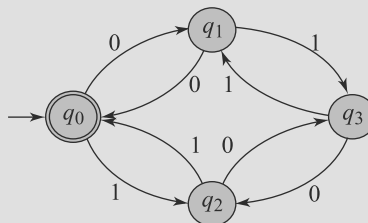


Fig. 9.8 FA of Example 9.4

Now we have to transform the FA given by Fig. 9.8 into a Turing machine that accepts the same language. The required Turing machine is given by Fig. 9.9.

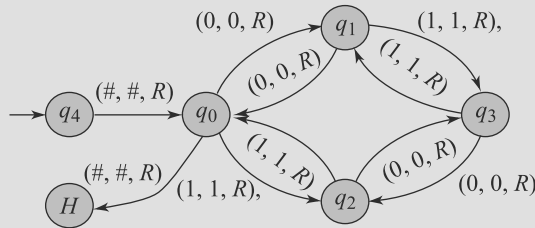


Fig. 9.9 Turing machine of Example 9.4

In this transition diagram q_0 is the initial state and H is the halt state.

Example 9.5

Design a Turing machine to recognise all strings having of an odd number of 1's over $\{0, 1\}$.

Solution The design of the required Turing machine is made by defining moves in the following manner:

- The Turing machine starts reading input, it enters to state q_1 or halt state from initial state q_0 on input 1 and remains in current state q_0 for input '0'. Reaching to halt state from initial state on input 0^*1 ensures that there is only one '1' in the string.
- When the machine is in state q_1 it returns to state q_0 on input 1 and remains in q_1 on input 0.
- The repeated transitions $q_0 \rightarrow q_1 \rightarrow q_0$ ensure even occurrence of 1's. When there is transition from q_0 to halt state H , the number of 1's in complete string become odd.

The designed Turing machine is given by the following diagram:

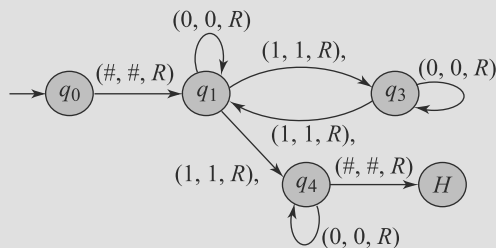


Fig. 9.10 Turing machine of Example 9.5

In all previous examples of a Turing machine, we have not allowed a Turing machine to take a left move. The reason is that we have designed Turing machines so far for regular languages. In case of non-regular language the designed Turing machine is expected to take left move one or more times. Here we are discussing some complicated Turing machines. Before designing a Turing machine the following guidelines should be kept in mind:

- The fundamental objective in scanning a symbol from a tape by read/write head should be known and make sure what to do in the future. Also, the machine must remember the past

symbol scanned. This could be done very easily if the Turing machine goes to a unique next state on each input.

- (ii) The total number of states must be as less as possible. The minimum state Turing machine can be designed by changing the states only when there is a change in the written symbol or when there is a change in the movement of read/write head.

Example 9.6

Design a Turing machine that recognises the presence of sub string '101' and replaces it with '110'.

Solution (1) The basic idea for designing such Turing machine is that we have to look for sub string '101' and replace it by '110'. The following transition diagram represents the required Turing machine.

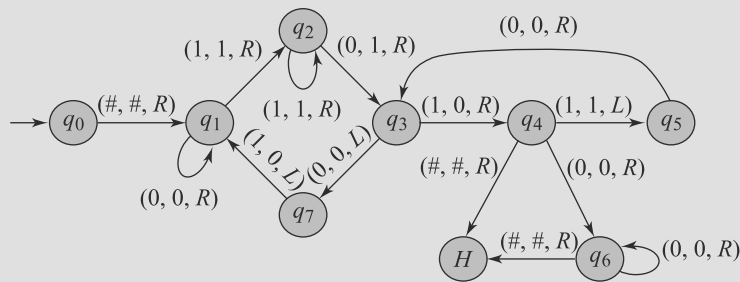


Fig. 9.11 Turing machine of Example 9.6

Let us simulate this Turing machine for input = #01011#

$q_0\#01011\# \mid \#q_101011\# \mid \#0q_11011\# \mid \#01q_2011\# \mid \#011q_311\# \mid \#0110q_41\# \mid \#011q_501\#$
 $\mid \#0110q_31\# \mid \#01100q_4\# \mid \#01100\#H$

Solution (2) First we design NFA for $(0, 1)^*101(0, 1)^*$, then we convert it into DFA and finally we convert this DFA into a Turing machine. We replace all inputs by the same symbol except the substring 101 is replaced by 110. The finite automaton for $(0, 1)^*101(0, 1)^*$ is

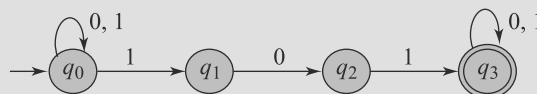


Fig. 9.12 NFA of Example 9.6

Now we convert this NFA into DFA. We get the following

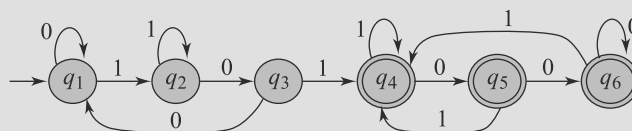


Fig. 9.13 DFA of Example 9.6

Now we convert this DFA into Turing machine, to get

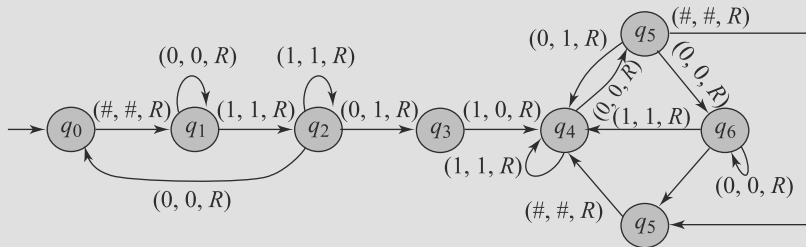


Fig. 9.14 Another Turing machine of Example 9.6

Note that the above Turing machine does not take any left move, because this Turing machine is a transformation from a finite automaton.

Example 9.7

Construct a Turing machine that produces 2's complement of input binary sequence.

Solution The basic idea to design such a Turing machine is that the input string is scanned right to left by first reaching to the right of input sequence. The logic is that 0's are not replaced with any other symbol during scanning. As the first '1' is scanned the machine takes left move and the whole sequence from that point to left end is complemented by replacing 0's by 1's and 1's by 0's. As the machine reads blank (the #), it goes to halt state.

This way the designed machine is given by the following diagram:

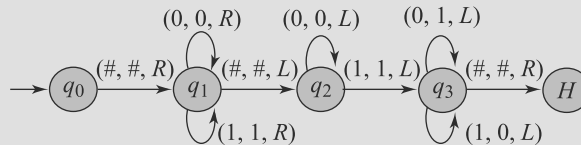
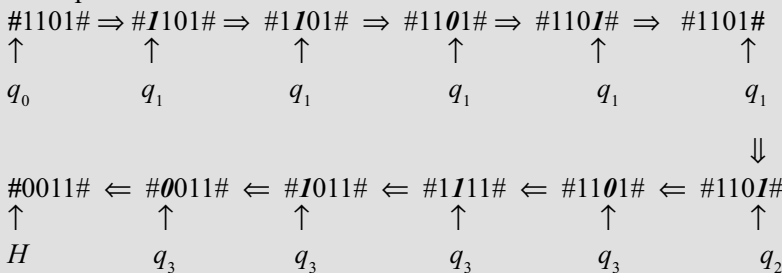


Fig. 9.15 Turing machine of Example 9.7

For input #1101#



Example 9.8

Design a Turing machine for $f(n) = n + 3$.

Solution Here, we are representing n by its equivalent unary value, *i.e.*, a number 4 is represented by '1111'. The basic idea for designing a Turing machine for $f(n) = n + 3$ is that the number of 1's are scanned and the head moves in the right direction. As the blank # is under read/write head, after scanning it is replaced by '1' and the machine takes a right move. Again the blank # is under read/write head the # after scanning is replaced by '1' and the machine takes a right move. Once again the same operation is performed. This way three #'s are replaced by '1' and the head *stops* and the machine reaches to a *halt* state. As a result, three #'s are replaced by 1's to increase the occurrence of 1's by three. The designed machine is given by the following diagram

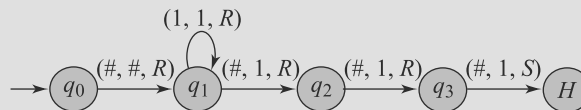


Fig. 9.16 Turing machine of Example 9.8

If we wish to design a Turing machine for $f(n) = n + 1$, then we will have to replace '#' by '1' only once.

Example 9.9

Design a Turing machine to accept the language $L = \{0^n 1^n \mid n \geq 0\}$.

Solution Here, we assume that the string $s \in L$ is enclosed with blanks *i.e.*, the input string is $\#0^n 1^n\#$. The smallest string in this family is $\#\#$. To design a Turing machine for L we adopt the following steps:

- If two consecutive blanks *i.e.*, $\#\#$ are scanned the machine reaches to halt state (for $n = 0$).
- If the leftmost symbol in the string is '0', it is replaced by X and the head moves right till the leftmost '1' is encountered in the string. It is replaced by Y and head moves left.
- Repeat (ii) with the leftmost '0'.
- All 0's are replaced by X and 1's are replaced by Y to ensure the equal occurrences of 0's and 1's in L .

The above steps are illustrated by the following sequence of operations:

```

#aaaaaaaaa.....aaaaabbbbbbbbbbb.....bbbbbb#
      ↓
#Xaaaaaaaaa.....aaaaaYbbbbbbbbbb.....bbbbbb#
      ↓
#XXaaaaaaaaa.....aaaaaYYbbbbbbbbbb.....bbbbbb#
      ↓*
#XXXXXXXXXX.....XXXXXYYYYYYYYYYY.....YYYYYY#
  
```

By using the above approach the following Turing machine is designed:

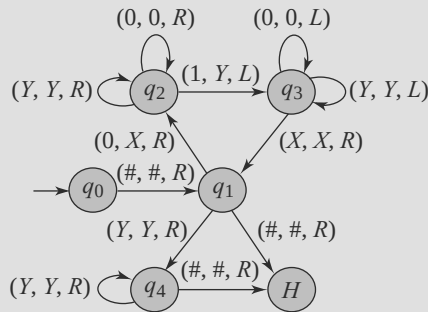


Fig. 9.17 Turing machine of Example 9.9

Example 9.10

Design a Turing machine for $f(m, n) = m + n$.

Solution The basic idea of designing a Turing machine for function $f(m, n) = m + n$ is that m and n are represented by unary number on input tape separated by symbol $+$. The read/write head of the Turing machine starts reading 1's and takes right move, as the symbol $+$ is scanned it is replaced by '1' and the machine takes right move until blank $\#$ comes. After reading one blank cell the machine takes a left move and replaces '1' by $\#$. The occurrence of consecutive 1's on the tape is result $m + n$ in unary form. This way the designed Turing machine is given by the following diagram.

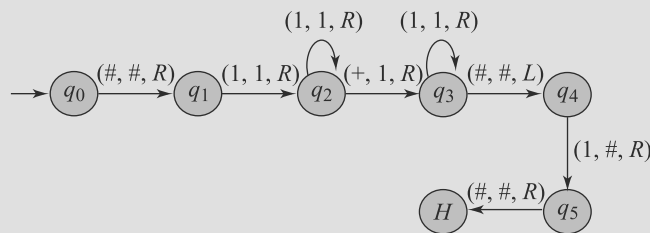


Fig. 9.18 Turing machine of Example 9.10

Below are the transition sequences for input $m = 2, n = 2$

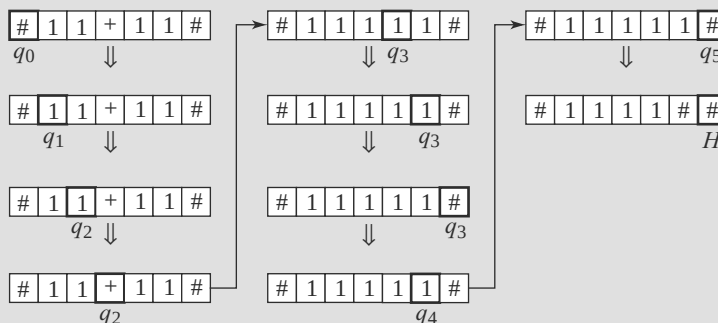


Fig. 9.19 Turing machine transitions of Example 9.10

9.7

PROGRAMMING TECHNIQUES

When a normal Turing machine is trained in such a manner that it can accept input data as well as procedure then it becomes a universal Turing machine. As we know a universal Turing machine requires input data as well as a procedure (or say program) to perform a desired operation. In this section we will define a set of steps to program a Turing machine. Let us consider following rules:

Rule 1 Each command for a Turing machine has to be assigned a unique name.

Rule 2 A command in a Turing machine has four elements, and these elements should occur in some specific order:

- (i) name of the command
- (ii) what the current cell scanned of input tape must contain to continue
- (iii) The action is to be performed if condition (ii) is satisfied. There are four actions –
 - a:** Write an 'a' in current cell by overwriting whatever is there.
 - b:** Write a 'b' in current cell by overwriting whatever is there.
 - L:** Move one cell left along the input tape.
 - R:** Move one cell right along the input tape.
- (iv) The name of the command has to be carried out next, such as 'CMD22'

Rule 3 The above four elements have to be separated, such as CMD1, a, L, CMD8 is a single command which indicates that the machine has to scan cell 1 to search for 'a' if it contains an 'a' the machine will take a left move to execute CMD8.

Rule 4 Unless otherwise not verified, the input tape is supposed to be blank (all a's). Under such conditions, the program will write a never ending series of a's to the left:

CMD1, a, b, CMD2
 CMD2, b, L, CMD1

Rule 5 If a particular command is called within the program and that command does not exist, the machine halts. The program fragment given below writes two b's and halts

CMD1, a, b, CMD2
 CMD2, b, L, CMD3
 CMD3, a, b, CMD4

This program halts because the command CMD3, a, b, CMD4 searches for CMD4, which is not defined.

Rule 6 A Turing machine program must be of finite length, even if it does not halt. The power of a Turing machine does not require infinitely long programs any more than an infinitely long tape.

Rule 7 There may exist two commands for the same name, for example

CMD_k, a, L, CMD_{k+1}
 CMD_k, b, R, CMD_{k+1}

This is useful in a condition when we are not sure whether the currently scanned cell contains an 'a' or a 'b'. This set of command tells us that the head moves left if an 'a' is scanned or moves right if 'b' is scanned.

Rule 8 An important rule for Turing machine programming is that the programs must be consistent. The best way to insure consistency is not to use more than two commands of the same name ever.

Rule 9 When a command is inapplicable, because it requires the scanner to be pointing to an 'a' when it is pointing to a 'b' or vice versa, then the machine will look for another command of the same name that can be executed. If there is no such command, it will be skipped and look for the next numbered command in the sequence; if there is no command, then the machine will halt.

Rule 10 Always we must know which command is to be executed first. A major advantage of using numerical names (even if they are not pure numerals) is that we may specify that programmes will always begin with the lowest numbered command in the sequence.

Rule 11 When a command ends with the instruction to execute CMD_k next, the Turing machine will look for $C_k, 0, \Delta, \Delta$ first, and then for $C_k, 1, \Delta, \Delta$, and perform whichever one applies, depending on whether the currently scanned cell contains an 'a' or 'b'. (Here, Δ is a placeholder standing in for any appropriate elements of Turing commands.)

If we are not concerned with simplicity, we can write Turing commands more compendiously so that $CMD1, b, a, CMD3$ would become "Command 1: IF 'b', THEN WRITE 'a', GOTO Command 3" or even:

```
Command 1;
Begin {Command 1}
IF CurrentCell is 'b' THEN WRITE 'a';
    GOTO Command 3;
End; {Command 1}
```

There are only two ways to halt: calling a command that does not exist, and run out of commands by executing or skipping each one. Programmers use both methods to force a canonical halt.

9.8

TURING MACHINE AND COMPUTATION

Turing Machine to Digital Computer A digital computer can simulate finite control, and mount one disk that holds the region of the Turing machine tape around the tape head. When the head of the tape moves off this region, the computer prints an order to have its disk moved to the top of the right or left heap, and the top of the other heap mounted.

Digital Computer to Turing Machine Simulating a digital computer into a Turing machine is at the level of stored instructions and words of memory. A Turing machine has one tape that holds all the used memory locations, and their contents. Other Turing machine tapes hold the instruction counter, memory address, computer input file, and scratch. Instruction cycle of a computer simulated by:

1. Find the word indicated by the instruction counter on the memory tape.
2. Examine the instruction code (a finite set of options), and get the contents of any memory words mentioned in the instruction, using the scratch tape.
3. Perform the instruction, changing any words' values as needed, and adding new address-value pairs to the memory tape, if required.

Limits of Algorithmic Computation Interactive computation is more expressive than algorithmic computation. That is, there are problems which cannot be solved by algorithmic computation, but for which an interactive system can be built.

Complex systems, consisting of many interactive components, are even more expressive than sequential interactive systems, capable of exhibiting emergent behaviours whose nature is not yet well understood.

What is interesting in Turing completeness is that it is the most meticulous definition of “what task can be performed by algorithmic computation”: it is a hypothesis and is impossible to prove. There exist no such problems that can be solved by a series of mechanical computation but cannot be solved by Turing machines. This has never been seen. This is impossible to prove because the concept of “mechanical computation” is an observant, not a formal one.

The major restriction with Turing machines is that they cannot model the strength of a particular arrangement well. For example, contemporary models of stored programme computers are essentially instances of a more specific form of an abstract machine known as Random Access Machine (RAM) or RASP (Random Access Stored Program Machine) model. Similar to Universal Turing Machine the RASP stores its programme instructions in memory external to its instructions of finite state machine. In contrast to UTM, the RASP model has infinite number of different registers as memory cells. The finite state machine of RASP is capable of indirect addressing.

Reversible-algorithmic computation and quantum computation are the other areas which are not paced by the Turing machines. The existing approach to quantum computation is an evolution of classical reversible-algorithmic computation (a sequence of elementary logically reversible transformations, represented by unitary transformations).

❖ Did you Know ?

As an undergraduate student at King's College, Cambridge from 1931, Alan Turing entered a world more encouraging to free-ranging thought. His 1932 reading of the then new work of Von Neumann on the logical foundations of quantum mechanics, helped the transition from emotional to rigorous intellectual enquiry. At the same time, this was when his homosexuality became a definitive part of his identity.

By 1933 Turing had already introduced himself to Russell and Whitehead's *Principia Mathematica* and so to the then arcane area of mathematical logic. Bertrand Russell had thought of logic as a solid foundation for mathematical truth, but many questions had since been raised about how truth could be captured by any formalism.

Turing was elected a fellow of King's college, Cambridge in 1935 for a dissertation on the Gaussian error function which proved fundamental results on the probability theory, namely central limit theorem. Although the central limit theorem had recently been discovered Turing was not aware of it and discovered it independently.

9.9

TYPES OF TURING MACHINES

9.9.1 Nondeterministic Turing Machines

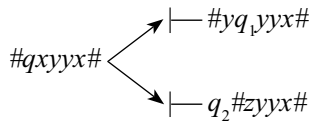
A non-deterministic Turing machine is the same automaton as given in section 9.4, except that δ is now a function

$$Q \times \Gamma \rightarrow 2^{Q \times \Gamma \times \{L, R\}}$$

As always when this property (non-determinism) is involved, the range of δ is a set of all possible transitions, any of which can be chosen arbitrarily by the machine. For example, if a Turing machine has transition specified by

$$\delta(q, x) = \{(q_1, y, R), (q_2, z, L)\},$$

is non-deterministic in nature. For an input string 'xyyx', there are two possible moves when a Turing machine is in state q . The moves are,



where, # is blank. It is not clear what role a non-deterministic machine plays for a Turing machine for computing functions. Non-deterministic automata are usually viewed as acceptors. A non-deterministic Turing machine is assumed to accept a string 's' if there is any possible sequence of moves such that

$$q_0 \# s \# \vdash^* \alpha H \beta$$

where H is halt state and $\alpha, \beta \in \Gamma^*$.

A non-deterministic machine may have moves available that lead to a non-halting state or to an infinite loop. However, as always with non-determinism, these alternatives are not relevant.

A non-deterministic Turing machine is not more powerful than a deterministic one. To show this we need to provide a deterministic equivalent for the non-determinism. Non-determinism can be viewed as a deterministic backtracking (return back to the path followed) algorithm, and a deterministic machine can simulate a non-deterministic machine as long as it can handle the book keeping involved in the back tracking. To see this, let us consider an alternative view of non-determinism, one which is useful in many arguments is:

‘A non deterministic machine can be seen as a machine that has the ability to replicate itself whenever necessary. When more than one moves are possible, the machine produces as many replicas as needed and assigns each replica the task of carrying out one of the alternatives.’

Ultimately, replication is certainly not within the power of present computers. Nevertheless, the process can be simulated by a deterministic Turing machine.

9.9.2 Linear Bounded Automata

While the power of the standard Turing machine cannot be extended by complicating the structure of a tape, it is possible to limit it by restricting the way in which the tape can be used. The way of limiting the tape capabilities is to allow the machine to use only that part of the tape occupied by the input. This way, more space is available for long input strings than for short strings, generating another class of

machines called linear bounded automata (LBA). The model of linear bounded automaton is given by Fig. 9.20 below.

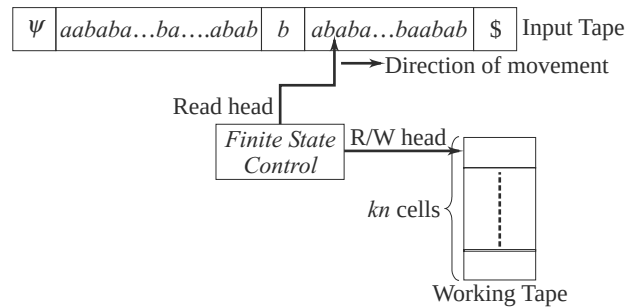


Fig. 9.20 The basic linear bounded automata model

The model of LBA consists of two tapes (i) input tape, and (ii) working tape. There is a read only head on the input tape that is allowed to move one cell at a time in the right direction only. Another head is the read/write head associated with the working tape that can modify the contents of a working tape in any way, without any restriction. The symbols ψ and $\$$ on input tape (called left-end marker and right-end marker respectively), decide the length of input tape. The symbols ψ and $\$$ are entered in left-most and right cells of input tape, respectively.

If the input tape contains n cells then input strings ' s ' may be at most $n - 2$ length, i.e., $|s| \leq n - 2$. The string ' s ' can be recognised by a linear bounded automaton if it can also be recognised by a Turing machine using at most ' kn ' cells of input tape, where k is a constant specified in the description of linear bounded automaton. The value of k is independent from the input string but is purely a property of the machine. The string to be processed is assumed to be closed within end-markers ψ and $\$$.

An LBA is a nondeterministic Turing machine (which has a single tape) whose length is not infinite but bounded by a linear function of the length of the input string.

Formally, we can define a linear bounded automaton as 7-tuple, $(Q, \Sigma, \Gamma, \delta, q_0, \#, F)$, where

Q = a finite nonempty set of states

Σ = a nonempty set of input symbols, and $\# \notin \Sigma$,

Γ = a finite nonempty set of tape symbols, and $\#, \psi, \$ \in \Gamma$
($\#$ is blank, ψ and $\$$ are left and right-end markers, respectively),

δ = transition function, which maps from $Q \times \Sigma$ into $Q \times \Sigma \times \{L, R\}$.

q_0 = the initial state,

$\#$ = the blank,

F = the set of final states, and $F \subseteq Q$.

The transition function δ will be co-domain of $Q \times \Sigma$ into $2^{Q \times \Sigma \times \{L, R\}}$ to accommodate nondeterminism. This definition of LBA is assumed to be nondeterministic. This is not just a matter of convenience but necessary for the discussion of LBAs. We can define deterministic linear bounded automata, but it is not known whether they are equivalent to the nondeterministic version.

In 1960, Myhill introduced an automaton model today known as deterministic linear bounded automaton. Thereafter, Landweber proved that the languages accepted by a deterministic LBA are

always context-sensitive. In 1964, Kuroda introduced the more general model of (nondeterministic) linear bounded automata, and showed that the languages accepted by them are precisely the context-sensitive languages.

Theorem 9.2 If L is any context sensitive language then there exists a Linear Bounded Automaton that accepts it.

Proof For any context sensitive language we can construct a linear bounded automaton which has a two-track tape. It can be simulated in such a way that when we place a string ' s ' on the first track and produce sentential form on the second track, by comparing its contents with the contents of the first track every time. When there is empty string ($\varepsilon \in L$), then linear bounded automaton halts without accepting.

Theorem 9.3 If L is any language recognised by a Linear Bounded Automaton then L is a context sensitive language.

Proof Let $M = (Q, \Sigma, \Gamma, \psi, \$, \delta, q_0, \#, F)$ be a linear bounded automaton with $\psi, \$$ as left and right end markers. Assume $L(M) = L$, then we can construct a context sensitive grammar $G = (V_N, \Sigma, P, S)$. In the construction of context sensitive grammar G , V_N consists of nonterminals of the form (α, β) where $\alpha \in \Sigma$, and β is a string of the form $s, qs, q\psi s$ or $s\$$ or $qs\$$, where $q \in Q$, and $s \in \Gamma$.

The linear bounded automata accept context sensitive languages. Make a note that the study of context-sensitive languages is important from the implementation point of view, because several compiler languages lie between context-free and context-sensitive languages.

9.9.3 Modifications of Turing machines

In this section we see other alternative definitions of a Turing machine that could serve well for our definition of a standard Turing machine. The major conclusions that can be drawn about the power of a Turing machine are largely free from the explicit structure chosen for a Turing machine.

Complicating the standard Turing machine by giving it a more complex storage device will not have any effect on the power of the automaton. A series of computations that can be performed on such a new platform will still fall under the class of a mechanical computation and, therefore, it can be performed using a standard model. Many variations on the basic model of a Turing machine (Section 9.3) are possible. For example, a Turing machine can be considered with more than one tape, or with tapes that can be extended in several dimensions (e.g., two or three dimensional).

❖ Did you Know ?

Alan Turing had a life-long interest in machines. According to his mother, "Alan had dreamt of inventing a typewriter as a boy. He could well have begun by asking himself what was meant by calling a typewriter mechanical." During his PhD, Alan Turing built a Boolean-logic multiplier.

In 1935 as a young master student, Alan Turing took a challenge. He had been enthused by the lectures of logician M. A. Newman. He learned about Godel's work and the Entscheidungs problem from them. In his obituary of Alan Turing, Newman wrote:

'To the question—what is a mechanical process, Alan Turing gave the distinctive answer—something that can be done by a machine, and he embarked on the highly agreeable task of analysing the concept of a general purpose computing machine.'

There are at least two reasons for looking at these alternative approaches. First, in many cases they might seem to have the potential for increasing the computing power of the basic model; however, they can be shown not to do so. Thus, constituting a sort of evidence for the generality of the Turing machine by considering more sophisticated models, and by seeing in each case that an ordinary Turing machine can do every thing that these models can. Second, some of the variants are often more convenient to work with than the basic model of a Turing machine. An algorithm that would require considerable book keeping machinery if carried out on an ordinary Turing machine might be very easy to describe on a three-tape machine.

When we compare the two classes of abstract machines with reference to ‘computing power’ it is important to be precise about the criteria we are going to use. At this stage, we do not want to consider speed, efficiency, or convenience; we are concerned only with whether the two types of machines can serve the same purpose and get the same results. A machine in the extended family of Turing machines gives an answer, first by halting (reaching to H state) or failing to halt, and second by producing a particular output when it halts.

Multitape Turing Machines A Turing machine with several input tapes is said to be a multitape Turing machine. In a multitape Turing machine each tape is controlled by its own independent read/write head. A multitape Turing machine with k tapes is shown in Fig. 9.21.

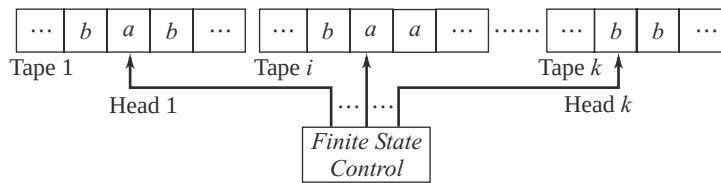


Fig. 9.21 A Multitape Turing machine model

Typically an n -tape Turing machine can be defined as $M = (Q, \Sigma, \delta, \Gamma, q_0, \#, H)$, where, all the parameters have the same meanings as in the basic definition of a Turing machine, but the transition function δ is defined as

$$Q \times \Gamma^n \rightarrow Q \times \Gamma^n \times \{L, R, stop\}^n$$

To show the equivalence between a multitape and a standard Turing machine, we argue that a standard Turing machine A can be simulated by any given multitape Turing machine M , and conversely, that any standard Turing machine can simulate a multitape Turing machine.

The point to be noted is, when we claim that a Turing machine with multiple tape is no more powerful than a standard (basic) one, we are making a statement only about what can be done by these machines, particularly, what language can be accepted.

Multidimensional Turing Machines A Turing machine is said to be a multidimensional Turing machine if its input tape can be viewed as extending infinitely in more than one dimension. The formal definition of a two-dimensional Turing machine involves a transition function δ of the form

$$Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, U, D, stop\}$$

All symbols have their usual meanings except U and D . The symbols U and D specify movement of read/write head in up and down directions, respectively. A diagram of a two-dimensional Turing machine is given by Fig. 9.22.

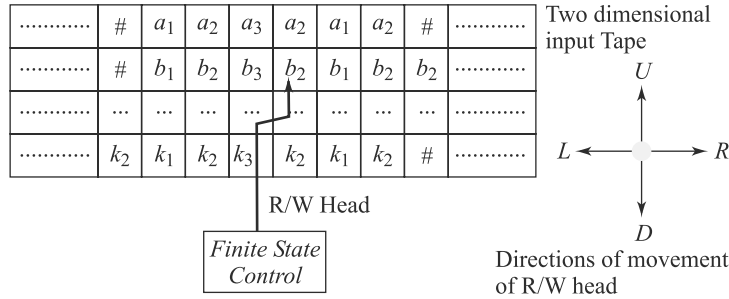


Fig. 9.22 A Multidimensional Turing machine model

Multihead Turing Machines A multihead Turing machine $M = (Q, \Sigma, \delta, \Gamma, q_0, \#, H)$, can be viewed as a Turing machine with a single tape and a single finite state control (also called control unit) but with several independent read/write heads. The transition function of a multihead Turing machine can be defined as

$$Q \times \Gamma_T \rightarrow Q \times \Gamma_T \times \{L, R, stop\}^n$$

where, $\Gamma_T = \Gamma \times \Gamma \times \Gamma \times \Gamma \times \Gamma \times \Gamma \dots \times \Gamma$, and n is the number of read/write heads.

A move of the multihead Turing machine depends upon the state and the symbol scanned by each head. In one move, the read/write heads may take move independently left, right or may remain stationary. A five-head Turing machine (a multihead Turing machine with five heads) is shown by Fig. 9.23.

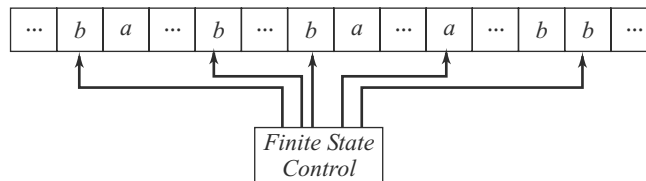


Fig. 9.23 A Multihead Turing machine model (five heads)

Off-Line Turing Machines An off-line Turing machine is a multitape Turing machine whose input tape is read-only (write operation is not allowed). Such Turing machines do not allow the head to move on the input tape off the region between the end-markers ψ and $\$$. An off-line Turing machine can simulate any Turing machine A by using one more tape than Turing machine A . The reason for using an extra tape is that the off-line Turing machine makes a copy of its own input into the extra tape, and it then simulates Turing machine A as if the extra tape were A 's input.

Combining Turing Machines For complicated and large tasks, two or more Turing machines can be combined. If we wish to combine two Turing machines M_1 and M_2 as a single Turing machine to accept

the language $L = L(M_1)L(M_2)$, then the halt state of Turing machine M_1 is combined with the initial state of the Turing machine M_2 .

9.10

UNIVERSAL TURING MACHINE

A Turing machine is said to be a universal Turing machine if it can accept:

- (i) the input data, and
- (ii) an algorithm (description) for computation.

This is precisely what a general-purpose digital computer does. A digital computer accepts a programme (a set of instructions in sequence) written in high level language. Thus, a general-purpose Turing machine will be called a **Universal Turing Machine** (say U) if it is powerful enough to simulate the behavior of any digital computer, including any Turing machine itself. More precisely, a universal Turing machine can simulate the behavior of an arbitrary Turing machine over any set of input symbols.

Thus, it is possible to create a single machine that can be used to compute any computable sequence. If this machine is supposed to be supplied with the tape on the beginning of which is written the input string of quintuple separated with some special symbol of some computing machine M , then the Universal Turing machine U will compute the same strings as those by M . The model of an Universal Turing machine is considered to be a theoretical breakthrough that led to the concept of stored programme computing device.

The Turing machines we have studied in earlier sections are special-purpose (limited capabilities) computers. Designing a general-purpose Turing machine is a more complex task. Once the transitions of Turing machine are defined, the machine is restricted to carrying out one particular type of computation. Digital computers, on the other hands, are general purpose machines that can be programmed to do different jobs at different times. Consequently, Turing machines cannot be considered equivalent to general purpose digital computers until they are designed to be reprogrammed.

By modifying our basic model of a Turing machine we can design a universal Turing machine. The modified Turing machine must have a large number of states for simulating even a simple behavior. We modify our basic model by:

- (i) increasing the number of read/write heads
- (ii) increasing the number of dimensions of input tape (*i.e.*, making it two, three or more dimensional),
- (iii) adding a special purpose memory (such as stacks or special purpose registers).

All the above modifications in the basic model of a Turing machine will almost speed up the operation of the machine, but do not increase the computing power of doing something more than what the basic model of a Turing machine can do.

A number of ways can be used to explain to show that Turing machines are useful models of real computers. Any thing that can be computed by a real computer can also be computed by a Turing machines. A Turing machine, for example, can simulate any type of functions used in programming languages. Recursion and parameter passing are some typical examples. A Turing machine can also be used to simplify the statements of an algorithm.

If we talk about the differences of a Turing machine and a real computer, the major point is manipulation of unbounded amount of data. A Turing machine is not very capable of handling it in

a given finite amount of time. Also, Turing machines are not designed to receive unbounded input as many real programmes like word processors, operating system, and other system softwares.

❖ Did you Know ?

Finding small Universal Turing machines has recently become a popular venture. For a long time the smallest universal Turing machine was in existence due to Marvin Minsky, who discovered it as a 7-state 4-symbol machine using 2-tag system in 1962. Stephen Wolfram introduced a 2-state 5-symbol universal Turing machine. Stephen Wolfram announced a prize of US \$25000 for the proof/disproof of the guesswork that an even simpler 2-state 3-symbol Turing machine is universal. This prize was awarded to Alex Smith, an undergraduate student of Electronics and Computer Engineering at Birmingham University in 2007. After some time in 2007, Vaughan Pratt (Stanford University) announced that he had discovered an imperfection in that proof.

9.11

CHURCH-TURING HYPOTHESIS

9.11.1 Church's Thesis

We begin with questions like, 'How powerful are Turing machines? Are there some algorithms we can write in any programming language that cannot be translated into a Turing machine?' The answers are, 'we do not think so'. There is nothing better that we know of, and it is generally agreed that we will never find anything better.

We are now in fuzzy state, but no one has yet come up with a problem that can be solved algorithmically that cannot be solved by a Turing machine, and there is considerable experimental evidence to suggest that no one ever will.

The reason of this uncertainty is the uncertain definition of an algorithm. After all, computers can only do what an algorithm tells them to do. We have an intuitive concept of what an algorithm is, but we do not have an exact definition. For example, *C* language programs seem like precise statements of an algorithm, but we would not want to say that the only thing computers can do is followed by *C* language programs.

An effective procedure is another name of an algorithm. There is no precise definition of an effective procedure, but we expect it to have the following properties:

- (i) Its description is finite.
- (ii) It will execute in a finite amount of time.
- (iii) It requires a finite amount of storage.
- (iv) It is deterministic. In this context, deterministic means that it always produces the same answer for a given input.

A truly amazing fact about Turing machines is that they seem to be capable of solving any problem for which an effective procedure exists. According to Alonzo Church, 'there is an effective procedure to solve a problem if and only if there is a Turing machine which solves that problem or equivalently there exists a Turing machine to solve a problem if and only if there exists an effective (mathematical) procedure to solve that problem'.

This statement is called a thesis (*Church's Thesis or Church-Turing Thesis*) rather than a theorem because it is fundamentally not provable. Provable or not, Church's thesis is generally accepted to be true. In fact many computer scientists define an effective procedure as something for which a Turing machine exists.

According to Church's Thesis, a Turing machine can be treated as the most general computing system. Therefore, finding an algorithm for a Yes/No problem is equivalent to constructing a Turing machine which can execute an algorithm.

Church's thesis is also known as the Church-Turing hypothesis because of the combined research of Alonzo Church and Alan Turing which appeared in the *American Journal of Mathematics* in 1936. The Church-Turing hypothesis says that the class of decision problems that can be solved by any reasonable model of computation is exactly the same as the class of decision problems that can be solved by Turing machine programmes.

Since 1936, several thousand different models of computation have been proposed, for example

- Post systems
- λ -Calculus
- Godel-Herbrand-Kleene equational calculus
- Unlimited register machine

Every single one of these satisfies the Church-Turing hypothesis. In other words we can say that anything that can be solved by a Turing machine can be programmed in any one of the systems given above and vice-versa.

The importance of the Church-Turing hypothesis is that it allows us to consider computability results without any loss of generality.

❖ Did you Know ?

Alonzo Church is best remembered for Church's Thesis (1936), which says that there exists no decision procedure for the full predicate calculus. Church's Thesis appeared in an unsolvable problem under elementary number theory published in the *American Journal of Mathematics*. Church founded the *Journal of Symbolic Logic* in 1936 and remained the editor till 1979. His major work was based on mathematical logic, recursion theory and, needless to say, in theoretical computer science. He was responsible for creating the λ -calculus in the 1930s which today is a very useful tool for computer scientists. In 1956, he wrote the book entitled, *Introduction to Mathematical Logic*. He had 31 doctoral students including Turing, S. Kleene, Kermeny and Smullyan. D.I.A. Cohen was also a student of Church. All of these are well known personalities in the field of the theory of Automata.

9.11.2 Post Machine

In 1936, Emil Post created a machine called 'Post Machine', which he hoped would prove to be the 'Universal Algorithm Machine'. A Post-Turing machine can be seen as a program formulation of a Turing machine comprising a variant of Emil Post's Turing equivalent model of computation. A Post Turing machine uses a binary alphabet, an infinite sequence of binary storage memory location, and a primitive programming language having instructions for bidirectional movement of head among the storage locations. The first condition that must be satisfied by such a machine is that any language which can be precisely defined by a human being should be accepted by some version of this machine. This should make it more powerful than a finite automaton or a pushdown automaton.

A Post Machine (abbreviated as PM) can be defined as 5-tuple (Q, Σ, S, R, A) , where

Q = nonempty set of states having start (initial) state and some halt states
(called *ACCEPT* and *REJECT* states)

Σ = alphabet of input letters including the #, i.e., the blank

S = linear storage location called *STORE* in the form of queue

R = set of *READ* states, which remove the left most character from the *STORE*

A = set of *ADD* states, which concatenate a character onto the right end of the string in the *STORE*

❖ Good to Know!

ADD states are different from PUSH states of pushdown automaton. Post machines have no PUSH states. It is possible to have a separate ADD state for every letter in Σ and Γ .

A post machine is similar to a pushdown automaton, but with the following differences: A post machine is deterministic, it has an auxiliary memory in the form of queue instead of a stack (as in PDA), and the input is assumed to have been previously loaded onto the queue. For example, if the input string is 011, then the symbol currently at the front of the queue is '0'. Items can be inserted only to the rear end of the queue, and deleted only from the front end.

We can specify the configuration of a Post machine by specifying the state and contents of the queue. If the original marker Z_0 is currently in the queue so that the string in queue is of the form $\alpha Z_0 \beta$, then the queue can be thought of as representing the tape of a Turing machine. Most of the moves of a Turing machine can be simulated by the Post machine.

9.11.3 Counter Turing Machines

According to Minsky's theorem, a 2-stack PDA is equivalent to a Turing machine. A stack can be implemented using two counters when the bits on the stack are thought of as representing a binary number, with the top being the least significant bit. Inserting a zero on to the top of the stack is equivalent to making a number two times. Pushing a one is equivalent to doubling and adding one. The pop operation is equivalent to dividing by two, where the remainder is the bit being popped. These ways two counters can simulate a stack. Therefore, a Turing machine can be implemented by using four counters.

A counter can be implemented in a straight forward manner using a two tape Turing machine. The value of the counter can be represented in unary format, for example a number 4 can be written as '1111'. In a counter machine increment is applied on each scan if the input is '1'. An increment means that a '1' is added on the tape, and the head moves right. Similarly, a decrement means a '1' is replaced with a blank (i.e., #), and the head moves left. The test of '0' is equivalent to testing whether the tape is empty. Additionally, a two tape deterministic Turing machine can be simulated by a deterministic Turing machine with one tape.

A counter machine M is equivalent to a Turing machine M_2 if for every input $s \in \Sigma^*$

- ❖ If Turing machine M' accepts s , then M executes the instruction *ACCEPT*,
 - ❖ If Turing machine M' rejects s , then M terminates without executing *ACCEPT*, and
- If Turing machine M' is inside a loop on input s , then M does not terminate.

9.12

LANGUAGE ACCEPTED BY TURING MACHINE

As we know the Turing machine is the most powerful automaton, therefore it can accept any type of language. A language L is said to be accepted by a Turing machine M if for all strings $s \in L$, there exists a halting configuration. For this purpose the input string is placed on the input tape of a Turing machine and if starting from the left, the read/write head reads the right most symbol of the string by reaching in halt state, the string is said to be accepted by the Turing machine M .

For a given Turing machine $M = (Q, \Sigma, \delta, \Gamma, q_0, \#, H)$, the set of strings $s \in L$, is said to be accepted by M if

$$q_0 \# s \# \mid^* \alpha H \beta$$

where H is halt state and $\alpha, \beta \in \Gamma^*$.

The set of languages accepted by a Turing machines are called **Recursively Enumerable languages** or simply **RE** languages.

❖ Did you Know ?

Alan Turing was found dead by his cleaner when his mother came in his lab on 8 June 1954. He had had the day before of cyanide poisoning and a half-eaten apple was found nearby his bed. His mother believed he had accidentally taken cyanide in his body from his fingers after a part-time chemistry experiment, but it is more believable that he had successfully contrived his own death to allow her alone to believe this. The forensics examiner's judgment was suicide.

9.13

RECURSIVE AND RECURSIVELY ENUMERABLE LANGUAGE

A formal language L is said to be recursively enumerable if there exists a Turing machine that accepts it. It is easy to see that a language for which an enumeration procedure exists is definitely a recursive enumerable language. We simply compare the given input string against successive strings derived by the enumeration procedure.

A language is recursive if and only if there exists a membership algorithm for it. Therefore, a language L on Σ is said to be recursive if there exists a Turing machine that accepts the language L and it halts on every $w \in \Sigma^+$.

We now turn to some basic properties of recursive languages, which include the following:

- (i) The complement of a recursive language is recursive.
- (ii) The union of two recursive languages is recursive.

In practice there are some recursively enumerable languages which are not recursive. If a language L and its complement L' are both recursively enumerable, then both languages L and L' are recursive languages. If a language L is recursive then its complement L' is also recursive, and consequently both the languages are recursively enumerable. Also, the family of recursive languages is a proper subset of the family of recursively enumerable languages. The union of two recursively enumerable languages is recursively enumerable.

Example 9.11

Is the family of recursively enumerable languages closed under union?

Solution Suppose L_1 and L_2 are two recursively enumerable languages and the Turing machine that accept these languages are M_1 and M_2 , respectively. When represented with an input 's', we choose Turing machine M_1 or M_2 to process the input 's' non-deterministically. As a result there is a Turing machine that accepts the languages $L_1 \cup L_2$.

Therefore, the family of recursively enumerable languages is closed under union.

Theorem 9.4 Every context-sensitive language is recursive.

Proof Suppose L is the language generated by the context-sensitive grammar G . This language L can also be accepted by a linear bounded automaton M , which is a special type of nondeterministic Turing machine. Now we have to show, how we can modify automaton M so that it still accepts language L and any sequence of moves causes it to halt or crash. Then we can say that the language L is recursive.

Suppose the modified machine M' uses left-end and right-end markers ψ and $\$$, respectively. The linear bounded automaton M simulates a derivation in grammar G , using the second track of the input tape between end-markers ψ and $\$$. At the end of each iteration of simulation, machine M' performs two additional steps:

- (i) M' copies the current string of the derivation onto the empty portion of the tape to the right, and
- (ii) M' determines whether the most recent string is a duplicate of any of the previous α_i 's and crashes if it is.

By implementing these considerations, it is no longer possible for M' to loop forever. Eventually, one of three possibilities is there:

- (i) Either the current string in the simulated derivation matches the original input 'a', in which case the machine M' halts; or
- (ii) an iteration of the loop results in a string longer than 'a', in which case machine M' crashes; or
- (iii) one of the strings obtained shows up for the second time, in which case the machine M' also crashes.

The string 'a' for which some sequence of moves of machine M' results in the first outcome are precisely those that can be generated by grammar G . Thus, we can conclude that language L is accepted by a non-deterministic Turing machine for which every sequence of moves ends in a crash or a halt state.

9.13.1 Non-recursively Enumerable Languages

Turing machines are enumerable. Since recursively enumerable languages are those whose strings are accepted by a Turing machine, therefore, the set of recursively enumerable languages is also enumerable. The power set of an infinite set is not enumerable. Therefore, there must be languages that cannot be computed by a Turing machine.

According to the Church-Turing thesis (see Chapter 10), a Turing machine can be designed for a problem for which an effective procedure exists. Therefore, there are some languages that can be defined by any effective procedure.

Theorem 9.5 If a language L and its complement L' are both recursively enumerable, then both are recursive.

Proof Let us suppose that a language L is recursively enumerable. That means, there exists a Turing machine T_1 that when given any string of the language, halts and accepts the string.

Now, let us suppose that the complement of language L is denoted by $L' = \{\omega \mid \omega \notin L\}$, is also recursively enumerable. That means, there is some other Turing machine T_2 which when given any string of L' , halts and accepts that string.

Thus, it is clear that any string over some alphabet set Σ belongs to either L or L' . Hence, any string will cause either T_1 or T_2 or both to halt.

Now, let us construct a new Turing machine that emulates both T_1 and T_2 by alternating moves between them. When any one of the two stops, we can tell to which language the string belongs. Thus, we have constructed a Turing machine that for each input halts with an answer whether or not the string belongs to L . Therefore, L and L' are recursive languages.

Theorem 9.6 There exists a recursively enumerable language that is not recursive.

Proof Let us consider the language

$$L = \{\omega_i \mid \omega_i \in L(T_i)\}.$$

Here, we will show that L is itself recursively enumerable, but, let us now consider its complement

$$L' = \{\omega_j \mid \omega_j \notin L(T_j)\}.$$

If L' is recursively enumerable, then there must be a Turing machine that recognises it. Thus the Turing machine must be in the enumeration somewhere, let us call it T_k .

Now, let us discuss the question “Does ω_k belong to L ?”

- If ω_k belongs to L than T_k accepts this string. But T_k accepts only those strings that do not belong to L . Thus, we have a contradiction.
- If ω_k does not belong to L , then it belongs to L' and is accepted by T_k . But since T_k accepts ω_k , ω_k must belong to L . Thus, again, we have a contradiction.

We have now defined a recursively enumerable language L and shown by contradiction that L' is not recursively enumerable.

As we have seen earlier that if a language is recursive then its complement must also be recursive. That means, if language L is recursive then L' would also be recursive, and hence recursively enumerable. But it is not necessary that L' is recursively enumerable. Therefore it must not be recursive.

9.13.2 Undecidable Recursively Enumerable Languages

The Diagonalisation Language L_d The diagonalisation language L_d is the set of strings w_i such that $w_i \notin L(M_i)$. That is, L_d contains all strings w such that the Turing machine M with code w does not accept w . L_d is not recursively enumerable and therefore there exists no Turing machine to accept it. This is one of the examples of undecidable or unsolvable problems.

The diagonalisation language L_d consists of all strings ‘ w ’ such that the Turing machine M does not accept the string ‘ w ’ when the code of M is w_i . The reason of being called the diagonalisation language so, is illustrated in figure below.

$i \setminus j =$	1	2	3	4	5	
1	0	0	1	0	1	
2	1	1	0	1	0	
3	0	1	0	1	0	
4	1	0	1	1	0	
5	0	1	1	0	1	
.						
.						

Diagonal

Fig. 9.24 Illustration of diagonalisation language

The Fig. 9.24 shows that for all i and j , whether the Turing machine M_i accepts the string w_j . The i^{th} row can be thought as a characteristic vector for the language accepted by the Turing machine M_i . The diagonal values tell whether the Turing machine M_i accepts w_i or not. To construct the diagonal language, we find the complement of the diagonal string. If the above table is correct then the complement of the diagonal string must begin with 10100... The approach of finding the complement of the diagonal by constructing the characteristic vector of a language which is not the language that appears in any row representing a string is called diagonalisation. The diagonalisation language is not recursively enumerable since there exists no Turing machine that accepts it.

The Universal Language L_u The universal language L_u is the set of binary strings that encode a pair (M, w) (by putting 111 between the code for M and w) where M is a Turing machine with the binary input alphabet (i.e., strings from $(0, 1)^*$) and $w \in L(M)$. There is a Turing machine U , often called the universal Turing machine, such that $L_u = L(U)$. It has three tapes: One for the code of (M, w) , one for the code of the simulated tape of M , and one for the code of the state of M . Thus U simulates M on w , and U accepts (M, w) if and only if M accepts w .

The organisation of a universal Turing machine for universal language L_u is given by Fig. 9.25.

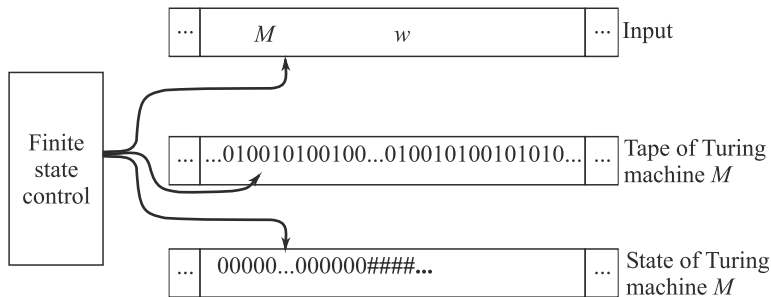


Fig. 9.25 Organisation of a Universal Turing machine for universal language L_u

The universal language L_u is recursively enumerable. Any Turing machine M may not halt when the input string w does not belong to the language, thus U will have the same behavior as M on w . Hence, L_u is recursively enumerable but it is not recursive. This can be treated as one of the examples of decidable or solvable problems.

9.14

TURING MACHINE AND TYPE-0 GRAMMAR

The languages accepted by a Turing Machines can be generated by Type-0 grammars. Thus, Turing machines are the automata that accept Type-0 languages, and Type-0 languages are the formal languages that are accepted by Turing machines. In this section we construct type-0 grammar (unrestricted grammar) generating the set accepted by a Turing machine. The construction of productions of type-0 grammar is a two step process:

Step 1 We construct productions which transform the string $[q_0\psi s\$]$ into the string $[H\#]$, where q_0 is the initial state, H is halt state, ψ and $\$$ are left-end and right-end markers, respectively. In place of a halt state H , a final state can also be taken. The grammar obtained in this step is called the transformational grammar.

Step 2 In this step we obtain inverse production rules by reversing the production of the transformational grammar to get the required type-0 grammar G .

The construction of type-0 grammar is in such a way that the string ' s ' is accepted by a Turing machine A if and only if $s \in L(G)$.

Construction of Type-0 Grammar The acceptance of a string ' s ' by the Turing machine A corresponds to the transformation of initial instantaneous description (ID) enclosed within brackets (e.g., $[q_0\psi s\$]$) into the final instantaneous description enclosed within brackets (e.g., $[q_f\#]$ or $[H\#]$), where q_0 is initial and q_f is final state and H is halt state. The length of instantaneous description may change if read/write head reaches to the left-end or right-end (e.g., when the left-hand side or right-hand side bracket (i.e., $]$) is reached). Therefore, we get productions corresponding to transitions of instantaneous descriptions with (i) no change in length, and (ii) change in length. The construction of grammar corresponding to TM given by a transition table involving two steps:

Step 1 (i) No change in length of IDs

- (a) If there is a right move like $a_k R q_l$ corresponding to q_i -row and a_j -column in the transition table of the Turing machine then the induced production is:

$$q_i a_j \rightarrow a_k q_l$$

- (b) If there is a left-move like $a_k L q_l$ corresponding to q_i -row and a_j -column in the transition table then the induced productions are:

$$a_m q_i a_j \rightarrow q_l a_m a_k \quad \text{for all } a_m \in \Gamma$$

(ii) Change in length of IDs

- (a) (In case of left-end (left bracket)). If there is a left-move like $a_k L q_l$ corresponding to q_i -row and a_j -column in the transition table then the induced production is:

$$[q_i a_j \rightarrow [q_l \# a_k$$

If ' $\#$ ' occurs next to the left bracket, it can be deleted by including the production $[\# \rightarrow [$

- (b) (In case of right-end (right bracket)) if ' $\#$ ' occurs to the left of $]$ (the right bracket), it can be removed by including the productions

$$a_j \#] \rightarrow a_j] \quad \text{for all } a_j \in \Gamma.$$

When the read/write head moves to the right of $]$, the length is increased due to insertion of ‘#’, and corresponding productions are:

$$q_i] \rightarrow q_i \#] \quad \text{for all } q_i \in Q$$

(iii) **Introduction of end-markers** The following productions are included to introduce end-markers:

$$\begin{aligned} a_i &\rightarrow [q_0 \psi a_i \\ a_i &\rightarrow a_i \$] \end{aligned}$$

for all $a_i \in \Gamma$, $a_i \neq \#$ and q_0 is initial state. To remove the brackets from $[q_f \#]$ or $[H\#]$, we include the production

$$[q_f \#] \rightarrow S \text{ or } [H\#] \rightarrow S$$

where $q_f \in F$ and S is the newly introduced symbol called the start symbol of constructed grammar.

Step 2 We reverse all the arrows of the productions obtained in step 1, to get the required type-0 grammar. This way the productions obtained are called inverse productions. The same thing can also be performed by interchanging left side of productions by right side of the productions. The grammar thus obtained is also called generative grammar.

Example 9.12

Consider a Turing machine given by the following transition diagram:

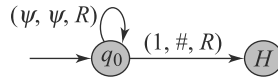


Fig. 9.26 Turing machine of Example 9.12

Obtain the inverse productions of type-0 grammar corresponding to this Turing machine.

Solution In this transition diagram q_0 is the initial and H is the state.

Step 1 (i) We get following production corresponding to right moves:

$$q_0 \psi \rightarrow \psi q_0, \quad q_0 1 \rightarrow \#H$$

(ii) (a) The production corresponding to left-end is

$$[\# \rightarrow [$$

(b) The productions corresponding to right-end are

$$\begin{aligned} \#\#] &\rightarrow \#], & l\# &\rightarrow 1], \\ q_0] &\rightarrow q_0\#, & H] &\rightarrow H\#]. \end{aligned}$$

(iii) By introducing end-markers, we have following productions:

$$1 \rightarrow [q_0 \psi 1, \psi \rightarrow \psi \$], \quad 1 \rightarrow 1 \$], \quad [H\#] \rightarrow S$$

Here, S is the introduced start symbol.

Step 2 We now inverse the production rules obtained in step 1. This is done by reversing the arrows of all production rules. We get following production rules:

$$\begin{array}{lll}
S \rightarrow [H \#], & & \\
\psi q_0 \rightarrow q_0 \psi, & & \\
\#H \rightarrow q_0 1, & [\rightarrow [\#, & \#] \rightarrow \#\#, \\
1] \rightarrow 1\#, & q_0 \# \rightarrow q_0], & 1\$] \rightarrow 1, \\
H\#] \rightarrow H, & q_0 \psi 1 \rightarrow 1, & \psi \$] \rightarrow \psi.
\end{array}$$

9.15

UNDECIDABLE PROBLEMS ABOUT TURING MACHINES

When we talk about decidability or undecidability results, we must always find out what the domain of the problem is, because this may affect the conclusion of the results. A problem may be decidable on some domain but not on another domain. Specifically, a single instance of a problem is always decidable, since the answer is either true or false.

A decision problem is *decidable* (or say solvable) if there possibly exists a Turing machine that will always halt in a finite amount of time, producing a ‘Yes’ or ‘No’ (‘True’ or ‘False’) answer. A decision problem is undecidable if a Turing machine may run forever without producing an answer. Some examples of undecidable problems are:

- (i) Does a given Turing machine M halt on all inputs?
- (ii) Does a Turing machine M halt for any input?
- (iii) Does a Turing machine M halt while given a blank input tape?
- (iv) Do two Turing machines M_1 and M_2 accept the same language?
- (v) Is the language accepted by a Turing machine finite?
- (vi) Does the language accepted by a Turing machine contain any two strings of the same length?

Of course, there exist logically determined undecidable problems, because a property can be asked whether it is true for any or all integers. For example, the halting problem of Turing machines asks if a computer program will halt at any future time on specific input string. Real computers (or say digital computers) halt when the electric supply goes off. They consist of only a limited amount of memory. The halting problem is about an abstract computer that runs infinitely and has no fixed limit on memory space. It can keep asking for another disk and can ask to read and write operation on any disk it recently used.

Digital computers follow an exact set of rules. We always know what the next step in computation is and thus what computation a computer will do at any time. But we are not sure in general if it will ever do something like halt. If we wait long enough and it halts we will be able to know this. To prove that a computer never halts require something more than following the steps the computer takes.

There is nothing special about halting. We can have an equivalent problem like when we wish to ask if the computer program will ever accept more inputs than the specified. It is a general phenomenon and every user has experienced this problem while waiting for a response from his computer performing a huge computation. A user never knows if it requires rebooting or will eventually respond.

A Turing machine is able to solve a decision problem because the language of strings representing *yes-instance* is recursive. Informally, we would like to say that a decision problem is unsolvable if there is no general algorithm capable of deciding every instance. We may describe a problem as unsolvable if the corresponding language having *yes-instance* encoded by some appropriate encoding function is

not recursive. In particular, any non-recursive language L immediately yields an unsolvable decision problem, the membership problem for L : Given a string x , is $x \in L$?

Let $e(T)$ be the encoding of the Turing machine T . One example of a non-recursive language is

$$SA = \{s \in \{0, 1\}^* \mid s = e(T) \text{ for some Turing machine } T, \text{ accepting } s\}.$$

Here, SA stands for ‘self-accepting’. Strings in SA represent (via the encoding function e) Turing machine T that accepts its own encoding $e(T)$. In other words, SA is the language of encoded *yes-instance* for the decision problem self-accepting.

Note *An instance is yes-instance for which the answer is ‘yes’.*

When we start with the languages SA , we could have taken the unsolvable problem to be the following: Given a string x of 0’s and 1’s, “Is x an element of SA ?” This is precisely what we have referred to as, the membership problem for SA . It is slightly different from self accepting, because before we can even start thinking about whether a Turing machine accepts its own encoding, we must first decide whether the given string of 0’s and 1’s represents a Turing machine at all. Here, we do not distinguish between these two problems. The reason is that whenever we choose an encoding function for any decision problem, we always maintain that there is an algorithm for decoding strings. The membership problem for SA is unsolvable because the problem of self-accepting is inherently difficult, not because of any complications involved in decoding strings.

9.15.1 Halting Problem of Turing Machines

In this section we will first discuss unsolvable problems. A class of problems with two outputs (Yes/No or True/False) is called solvable (tractable or decidable) if there exists some definite algorithm which always halts (say terminates) with any of two outputs (True/False), else the class of problem is called unsolvable (intractable or undecidable).

The halting problem of a Turing machine can simply be expressed as: For a given Turing machine M and given input ‘ ω ’, does the Turing machine M perform a computation that eventually halts, while the Turing machine starts in the initial configuration $q_0\#\omega\#$? Also, it can be asked whether or not Turing machine M can be applied to input ‘ ω ’, or simply (M, ω) halts or does not halt.

If a given Turing machine recognises a language L , we assume that the Turing machine halts (e.g., it has no next move) whenever the input string is accepted. However, there may be several words for which this Turing machine will never halt.

The halting problem of a Turing machine is an unsolvable problem, but we can run the machine with a given input. If it halts, then a specific case is answered. If the machine does not halt, then the conclusion is that the machine will never halt by recognising some pattern within the computation.

Theorem 9.7 The halting problem of a Turing machine is undecidable.

Proof Let us assume a Turing machine M_1 is encoded for unary computation. The input to the machine M_1 is string (T, x) upon the Turing machine M_1 always halts. If ‘ x ’ is already unary encoded then definitely it will make more sense to write the set (T, x) . The Turing machine M_1 will halt in state H_A if T halts on ‘ x ’ and it will halt in H_R if T does not halt.

We can use M_1 to construct another Turing machine M_2 as follows:

In Turing machine M_2 the halting state H_A is referred to just another non-halting state. This way, if the Turing machine M_1 halts in H_A the Turing machine M_2 will go into an infinite loop

$$f_{M_2} = (x, H_A) = (x, H_A, R)$$

for all $x \in \Sigma^*$. If the Turing machine M_1 halts in H_R then the Turing machine M_2 will also. A halt state for M_2 has no purpose since M_2 will never reach to that state. Any dummy state can serve this purpose. This way Turing machine M_1 halts on the input (T, x) only when the Turing machine M_2 is unable to do so. If Turing machine M_1 does not halt on (T, x) then M_2 does not either.

Now we apply Turing machine M_1 to itself to point out the inner contradiction of every existence of Turing machine M_1 . To this end, we can construct Turing machine M_3 from Turing machine M_2 as follows:

For a given string $a = (T, x)$ the very first thing is that the Turing machine M_3 does is to duplicate it to (a, a) and then sets Turing machine M_2 loose on string (a, a) .

If Turing machine M_3 halts on M_3 then Turing machine M_2 halts on (M_3, M_3) , that means M_1 halts in H_R with the input (M_3, M_3) . But in case if M_1 halts on some input then by contradiction we can say that Turing machine M_2 can not halt on that input and hence Turing machine M_3 cannot halt on Turing machine M_3 . The Turing machine M_3 is already unary encoded. This is the contradiction.

On the other hand, if Turing machine M_3 does not halt on M_3 , then M_2 does not halt on the input (M_3, M_3) , that means M_1 halts in state H_A on input (M_3, M_3) , that means M_3 halts on the input M_3 , which is another contradiction.

❖ Did you Know ?

In 1936, Alan Turing published a paper entitled 'On Computable Numbers' with an application to the Entscheidungs problem. It was a paper in which Turing introduced an abstract machine, which is now called Turing machine. He gave an idea of an infinity of possible Turing machines, each corresponding to a different 'definite method' or procedure. His work introduced a concept of enormous practical significance: The idea of the Universal Turing Machine (UTM). He introduced the Universal Turing Machine as one general purpose machine for all possible tasks. It is very difficult now not to think of a Turing machine as a computer programme, and the mechanical functionality of interpreting and training the programme as what the computer itself does. This way, the Universal Turing Machine incorporated the critical principle of the digital computer and as a result it was considered as a single machine which can be turned to any well-defined task by being supplied with the appropriate programme.

9.16 TURING MACHINE AS LANGUAGE ACCEPTOR AND GENERATOR

There are three different ways of looking at a Turing machine:

- (i) Turing machine as acceptor (as it accepts recursive and recursively enumerable sets).
- (ii) Turing machine as generator (as it computes a total recursive or a partial recursive function).
- (iii) Turing machine as an algorithm (as it solves or partially solves a class of Yes/No problems).

As language acceptor, let $L = \{s \mid s = s^R\}$ be the palindrome language over $\{a, b\}$. The machine will halt in state q_Y if candidate string $s \in L$ or state q_N if $s \notin L$. A two track tape can be used, in which the upper track holds the candidate string s , the lower track holds $\times$'s as we check-off read symbols. First, the machine reads symbol A in s , it write A back on first track and $\times$ on second track. If $A = a$, move to state $[q_R, a]$, otherwise move to state $[q_R, b]$. Now the tape head is moved right, without altering the tape until either a blank cell is found or the second track contains a $\times$. Now the machine takes a left move.

If the input on track one of the tape does not match the symbol stored in the state, move to failure state q_N and halt. If the input on track one of the tape does match the symbol stored in the state, then if the second track contains a $\times$, the machine moves to accept state q_y and halts, else it marks second track with a $\times$ and moves left. It continues to move left, without altering the tape until the second track contains a $\times$. Now the machine moves right and start the process again.

As generator, a multitape Turing machine can use one tape for output writing words over some alphabet separated by a symbol, say #, where words once written are never changed. If M is a Turing machine that accepts a language $L = (0, 1)^*$, and there is another Turing machine M_2 that generates L . After generation of strings $L = \{\wedge, 0, 1, 00, 01, 10, 11, \dots\}$, these are run on the Turing machine and printed on the output tape if accepted.

As function evaluator, Turing machines consider the addition function $a + b$ on non-negative integers. Each input string is encoded on input tape as $0^a 1 0^b$. The Turing machine will halt with 0^{a+b} on the tape.

The machine reads 0's and rewrites them on the tape moving right until the 1 is read. 1 is replaced with a 0, and the head continues moving right without changing the tape until a blank is found. Finally, it moves left and replaces the 0 with a blank.

9.17

TURING TRANSDUCER

The study of nonprobabilistic computations has been employed to understand abstract models of Turing transducers, deterministic and nondeterministic. We use similar abstract models, called probabilistic Turing transducers, for the study of probabilistic computations.

A **Turing transducer** exhibits similarity to a Turing machine having two different tapes for input and output purposes. The block diagram of a Turing transducer is given by Fig. 9.27.

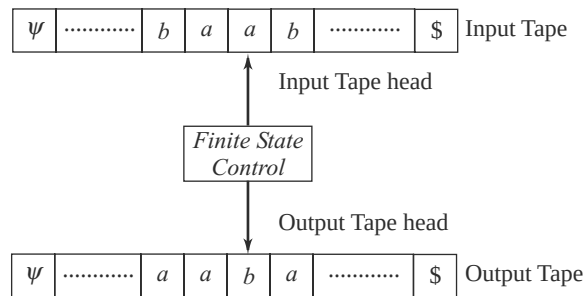


Fig. 9.27 Model of a Turing transducer

As the Turing transducer M starts to work, it has input string ' s ' on the input tape and the output tape is blank. If Turing transducer M can make a computation so as to remove the input string ' s ' from the input tape after reading and writes it after computation on the output tape (in the form of string ' t ' as computation of string ' s '). Then the Turing transducer M enters in some final state or say halt state.

A Turing transducer is a variant of the basic Turing machine model. We say a Turing transducer M can compute the function $f: \Sigma^* \rightarrow \Sigma^*$, where Σ is the input alphabet, if for every string $x \in \Sigma^*$ in which the computation of M halts on input ' x ' in state H . In the formal approach, we can define a probabilistic Turing transducer as a Turing transducer that can be viewed as non-determinism in random. Formally,

a **probabilistic Turing transducer** is a Turing transducer $M = (Q, \Sigma, \Gamma, \Delta, q_0, \#, F)$ whose computation sequences C are defined if the two conditions below hold:

- (i) C starts at an initial configuration.
- (ii) Whenever C is finite, it ends either at an accepting configuration or a non-accepting configuration from which no move is possible.

A probabilistic Turing machine is equivalent to a Turing machine except that it contains an additional instruction to allow an execution path randomly chosen. A ‘write’ instruction is an example of such instruction. In ‘write’ instruction, the value input is random and it is equally distributed between the symbols in the alphabet set of a Turing machine. Probabilistic Turing machines can have two different and valid transitions from some present state. At each present state, the probabilistic Turing machine selects between valid transitions by giving them equal probabilities of being selected. By using this model of a Turing machine, we can compute recursive functions on real numbers and natural numbers as well. The probabilistic Turing machine can produce different results for different executions.

A computation of a probabilistic Turing machine is said to be an accepting computation if it ends in an accepting configuration. Otherwise, the computation is considered to be a non-accepting (or say rejecting) computation.

On a given input, a probabilistic Turing transducer may have both accepting and non-accepting computation for some set of strings as input. Each computation of a probabilistic Turing transducer is similar to that of a non-deterministic Turing transducer, the only exception arising upon reaching a configuration from which more than one move is possible. In such a condition, the choice between the possible transitions is decided randomly, with an equal probability of each transition to take place.

A probabilistic Turing machine M is said to accept a language L if

- (i) on input x from L , M has probability $1 - e(x) > \frac{1}{2}$ for an accepting computation,
 - (ii) on input x not from L , M has probability $1 - e(x) > \frac{1}{2}$ for a non-accepting computation,
- where, $e(x)$ is the error probability of M .

A partial function is said to be a Turing computable if there is a Turing transducer that can compute exactly that function. For a partial function, if there exists a Turing transducer that can compute that function and halt for every input of the domain, the function is said to be a total Turing-computable function.

Turing Machine as Enumerator An enumerator can be considered as a 2-tape Turing machine in which the first tape of the machine is the working tape and the second one is the output tape. There exists no input to an enumerator. The enumerator performs its function by printing the strings of the working tape to the output tape. This way a Turing machine as enumerator is able to enumerate all the strings in a language by printing them on to the output tape. The class of enumerable languages is equivalent to the class of all recognisable languages.

❖ Did you Know ?

As an honor to Alan Turing, every year an award named ‘Turing Award’ is given to the greatest contributor in the field of mathematical logic. Some of the famous recipients of the Turing Award are: Marvin Minsky (1969), John McCarthy (1971), Edsger W. Dijkstra (1972), Donald E. Knuth (1974), Michael Rabin and Dona Scott (1976), John Bacus (1977), Robert W. Floyd (1978), Stephen A. Cook (1982), Ken Thompson and Dennis M. Ritchie (1983), John Hopcroft and Robert Tarjan (1986), Peter Naur (2005), etc.

Summary

- **Turing Machine** A Turing machine is an automaton whose temporary storage is a tape. A read-write head is associated with this tape that can move in both directions (left or right) and it can read and write a single symbol on each move.
- **Halt State** A Turing machine is said to be in *halt* state if it is not able to move further.
- **Crash in TM** If the read/write head of a Turing machine is over the left most cell (i.e., input ψ) and the machine is asked to take a left move it is called *crash* condition. Similarly, if the read/write head of a Turing machine is over the right most cell (i.e., input $\$$) and the machine is asked to take a right move it is also called a crash condition.
- **Language of TM** The set of string followed from initial state to halt state is the language accepted by a Turing machine.
- **Equivalence of Two TMs** Two Turing machines M_1 and M_2 are said to be equivalent if both accept the same set of strings. In other words, two Turing machines M_1 and M_2 are said to be equivalent if both machines on scanning the same set of strings terminate to some halting configuration.
- **Representation of TMs** The following methods are used to describe a Turing machine: (i) Instantaneous Descriptions (IDs) using move-relations, (ii) Transition Table, and (iii) Transition Diagram (transition graph).
- **Non-Deterministic TM** A non-deterministic Turing machine is defined as a function $Q \times \Gamma \rightarrow 2^{Q \times \Gamma \times \{L, R\}}$.
- **UTM** A Turing machine is said to be a Universal Turing machine if it can accept (i) the input data, and (ii) an algorithm (description) for computation.
- **Church's Thesis** There exists an effective procedure to solve a problem if and only if there is a Turing machine which solves that problem, or equivalently there exists a Turing machine to solve a problem if and only if there exists an effective (mathematical) procedure to solve that problem.
- **Recursive and Recursively Enumerable Language** A formal language L is said to be recursively enumerable if there exists a Turing machine that accepts it. It is easy to see that a language for which an enumeration procedure exists is definitely a recursive enumerable language.

Review Questions

- 9.1 Show that there exists a Turing machine M for which the halting problem is unsolvable.
- 9.2 Is it possible to simulate a Turing machine on a general-purpose computer and vice versa?
- 9.3 Show that a language $L \subseteq \Sigma^*$ is recursively enumerable if and only if L can be enumerated by some Turing machine.
- 9.4 Prove that if a language L is recursive, then its complement is also recursive.
- 9.5 Consider the Turing machine given by Table 9.1. Draw transition diagram to represent this.
- 9.6 Draw a transition table of a Turing machine given by Fig. 9.10.
- 9.7 What can you say about the Turing machine given by following transition table:

Table 9.4 Transition table of Exercise 9.7

PRESENT STATE	INPUT TAPE SYMBOLS		
	1	2	#
$\rightarrow q_0$	$1Rq_0$	$2Rq_1$	$\#RH$
q_1	$1Rq_1$	$2Rq_0$	—
H	—	—	—

- 9.8** Design a Turing machine that can accept the language denoted by regular expression 11^* .
- 9.9** Show the transitions to construct a Turing machine that accepts language
 $L = \{s \mid \text{length of 's' is even, over } \{0, 1\}\}.$
- 9.10** Design a Turing Machine that can accept all even length palindromes over $\{0, 1\}$.
- 9.11** Design a Turing Machine to compute the greatest common divisor of two integers m and n .
- 9.12** Design a Turing machine that accepts the language given by $\{0, 1\}^* \{010\}$.
- 9.13** Design a Turing machine that can accept all strings over $\{0, 1\}$ having an even number of 0's.
- 9.14** Find the grammar generating the set accepted by the linear bounded automaton M given by the following transition table:

Table 9.5 Transition table of Exercise 9.14

PRESENT STATE	INPUT TAPE SYMBOLS			
	ψ	$\$$	0	1
$\rightarrow q_0$	ψRq_0	—	$1Lq_1$	$0Rq_1$
q_1	ψRq_f	—	$1Rq_2$	$1Lq_0$
q_1	—	$\$Lq_0$	$1Rq_2$	$1Rq_2$
q_f	—	—	$0Lq_f$	—

- 9.15** Consider the Turing machine described by the following transition table to obtain the corresponding inverse production rules.

Table 9.6 Transition table of Exercise 9.15

PRESENT STATE	INPUT-TAPE SYMBOLS	
	1	#
$\rightarrow q_0$	$\#Rq_1$	$\#RH$
q_1	$\#Rq_0$	—

- 9.16** Design a Turing machine to shift one cell right the string on input tape.
- 9.17** Design a Turing machine TM that computes the function $f(n) = n - 3$ for $n \geq 4$.
- 9.18** Construct type-0 grammar generating the language accepted by the following Turing machine:

$$\delta(q_0, 0) = \delta(q_0, 1) = (q_1, \#, R)$$

$$\delta(q_0, \#) = (H, \#, R)$$

$$\delta(q_1, 0) = \delta(q_1, 1) = (q_0, \#, R)$$

Hints

9.5 The required transition diagram is given below:

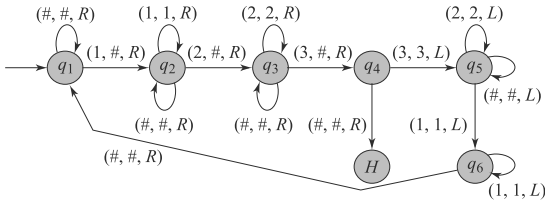


Fig. 9.28 Turing machine of Exercise 9.5

9.6 Table 9.7 Transition table of Exercise 9.6

PRESENT STATE	INPUT-TAPE SYMBOLS		
	0	1	#
$\rightarrow q_4$			$\#Rq_0$
q_0	$0Rq_1$	$1Rq_2$	$\#RH$
q_1	$0Rq_0$	$1Rq_3$	-
q_2	$0Rq_3$	$1Rq_0$	-
q_3	$0Rq_2$	$1Rq_1$	-
H	-	-	-

9.7 This Turing machine can accept all strings having even occurrences of 2's over $\Sigma = \{1, 2\}$.

9.8 The minimum length string in $L = \{11^*\}$ is '1'. The input string on the tape is in the form $\#1111 \dots 11\#$. When a machine is in initial state (q_0) it scans # to go to state q_1 . In state q_1 it looks for 1 as input to go to state q_2 . In state q_2 zero or more numbers of 1's can be scanned depending upon the length of input. In state q_2 when the machine scans the '#' it reaches to halt state. The following are the supported transitions of this solution approach:

$$\delta(q_0, \#) = (q_1, \#, R), \delta(q_1, 1) = (q_2, 1, R), \delta(q_2, 1) = (q_2, 1, R), \delta(q_2, \#) = (H, \#, stop)$$

9.9 The required Turing machine has the following transitions:

$$\delta(q_0, 0) = \delta(q_0, 1) = (q_1, \#, R), \delta(q_0, \#) = (H, \#, R), \delta(q_1, 0) = \delta(q_1, 1) = (q_0, \#, R),$$

where H is the halt state.

9.12 The required Turing machine consists of the following transitions:

$$\delta(q_0, \#) = (q_1, \#, R)$$

$$\begin{aligned} \delta(q_1, 0) &= (q_2, 0, R), & \delta(q_1, 1) &= (q_1, 1, R) \\ \delta(q_2, 0) &= (q_2, 0, R), & \delta(q_2, 1) &= (q_3, 1, R) \\ \delta(q_3, 0) &= (q_4, 0, R), & \delta(q_3, 1) &= (q_1, 1, R) \\ \delta(q_4, 0) &= (q_2, 0, R), & \delta(q_4, 1) &= (q_3, 1, R) \\ \delta(q_4, \#) &= (H, \#, stop) \end{aligned}$$

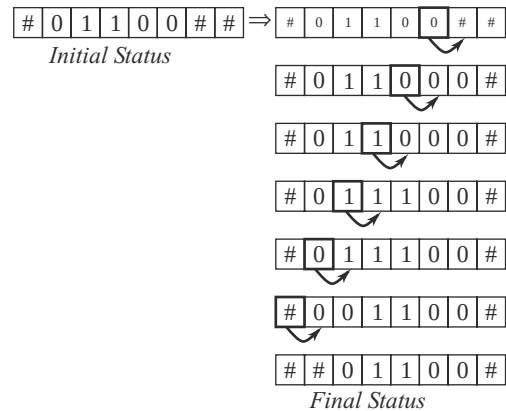
Here, H is halt state.

9.13 The following are the required transitions:

$$\begin{aligned} \delta(q_0, \#) &= (q_1, \#, R) \\ \delta(q_1, 0) &= (q_2, 0, R), & \delta(q_1, 1) &= (q_1, 1, R), \\ \delta(q_1, \#) &= (H, \#, stop) \\ \delta(q_2, 0) &= (q_1, 0, R), & \delta(q_2, 1) &= (q_2, 1, R) \end{aligned}$$

Here, H is halt state.

9.16 The input string s initially will be as $\#s\#$ on input tape. After the right shift operation the string will be as $\#\#s\#$. The design of the required Turing machine is similar to the left shift operation except the machine used to take right move instead of left move. First of all the machine goes to the rightmost cell, reads the symbol and copies it into one cell right and takes a left move, and repeats this procedure until the machine reaches to the left most cell. For example, if input string is 01100 then



The Turing machine for this purpose is given by the following table:

Table 9.8 Transition table of Exercise 9.18

PRESENT STATE	NEXT STATE		
	0	1	#
$\rightarrow q_0$	$0Rq_0$	$1Rq_0$	$\#Lq_1$
q_1	$0Rq_2$	$1Rq_4$	$\#Rq_5$
q_2	$0Lq_3$	$0Lq_3$	$0Lq_3$
q_3	$0Lq_1$	$1Lq_1$	$\#Lq_1$
q_4	$1Lq_3$	$1Lq_3$	$1Lq_3$
q_5	$\#RH$	$\#RH$	—
H	—	—	—

9.17 The given function simply reduces the value of n by 3. The required Turing machine is given by the following transitions:

$$T_1 : \delta(q_0, 0) = \{H, 0, R\}$$

$$T_2 : \delta(q_0, 1) = \{q_1, 0, R\}$$

$$T_3 : \delta(q_1, 0) = \{H, 0, R\}$$

$$T_4 : \delta(q_1, 1) = \{q_2, 0, R\}$$

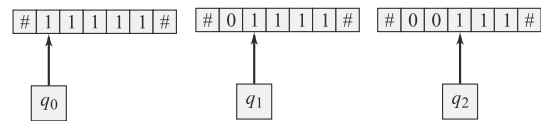
$$T_5 : \delta(q_2, 0) = \{H, 0, R\}$$

$$T_6 : \delta(q_2, 1) = \{H, 0, R\}$$

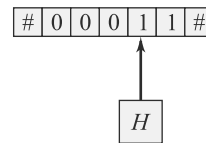
Here, q_0 is the initial state and H is halt state. The machine works on equivalent unary value of given

value of n , i.e., after a sequence of 0's the number of consecutive 1's represent n . For example, 00111 represent 3.

Let us see the functioning of this machine by using a constructive example. Consider the input tape of a Turing machine having unary value 5. The processing steps are as follows (see Fig. 9.29 and 9.30):

**Fig. 9.29** Turing machine steps

Finally we get 00011 on input tape giving unary value 2, also the machine is in final state, as shown below:

**Fig. 9.30** Turing machine in halt state

Graded Exercises

- **9.1** Construct a Turing machine that accepts the language $L = \{0^n 1^n 2^n \mid n \geq 1\}$.
- *9.2** Design a Turing machine that can compute $n \bmod 2$.
- *9.3** Design a Turing machine that accepts all palindromes over $\{0, 1\}$.
- *9.4** Design a Turing machine that removes an element from a particular position decreasing the length of the string by one.
- ***9.5** Design a Turing machine to compute $f(m, n) = m.n$.
- **9.6** Design a Turing Machine to compute $f(m, n) = \begin{cases} m - n & \text{if } m > n \\ 0 & \text{if } m \leq n \end{cases}$
- **9.7** Design a Turing machine that accepts language $L = \{a^n b^{2n} \mid n \geq 0\}$.
- *9.8** Design a Turing machine that accepts language $L = \{0^n 1^m 2^n \mid m, n \geq 1\}$.
- *9.9** Design a Turing machine that accepts language $L = \{ww \mid w \in (0, 1)^*\}$.
- *9.10** Determine the language accepted by a Turing machine represented by the following transitions:
 $\delta(q_0, \#) = (q_1, \#, R)$, $\delta(q_1, 0) = (q_2, 1, R)$, $\delta(q_2, 1) = (q_0, 0, R)$, and $\delta(q_2, \#) = (H, \#, R)$
- 9.11** Design a Turing machine that can accept language
 *(i) $L = \{(ab)^*a\}$.

- *(ii) $L = \{s \in (ab)^*a \mid |s| \% 4 = 3\}$.
- *(iii) $L = \{s \in (a, b)^* \mid s \text{ contains at most two } a\text{'s and at most two } b\text{'s}\}$.
- *(iv) $L = \{s \in (a, b)^* \mid s \text{ contains at least two } a\text{'s and at least three } b\text{'s}\}$.
- ** (v) $L = \{a^n b^n c^m \mid m, n \geq 0\}$.
- ** (vi) $L = \{a^n s c^n \mid s \in (a, b)^*, n \geq 0\}$.

***9.12** Design a Turing machine that can compute the function

- (i) $f(m, n) = m + n - 1$
- (ii) $f(n) = 2n$

* Difficulty level 1

** Difficulty level 2

*** Difficulty level 3

Hints

9.1 The key idea to construct the required Turing machine is that we match each 0, 1 and 2 by replacing them in order by x, y, z , respectively. At the end, we check that all original symbols have been rewritten.

9.2 The numeric function assigning to each natural number n the remainder when n is divided by 2 can be computed by moving to the end of the input string, making a pass from right to left in which the 1's are counted and simultaneously removed, and either leaving a single '1' or leaving nothing. The Turing machine that perform this computation is given by Fig. 9.31 below

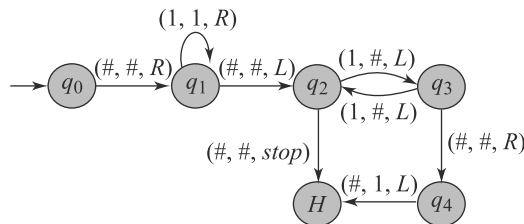
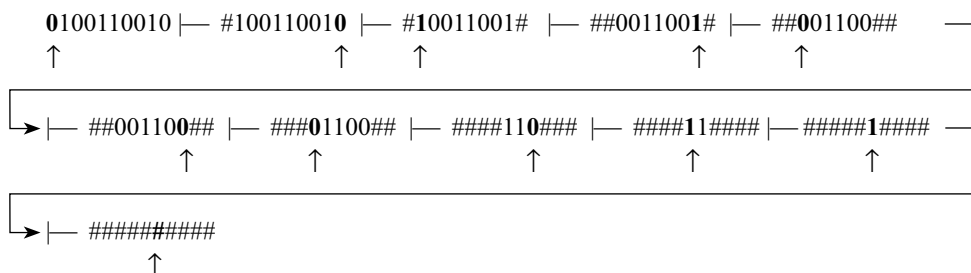


Fig. 9.31 Turing machine of Exercise 9.11

9.3 The strategy for accepting palindromes is to simply remove (i.e., replace a symbol with #) the leftmost symbol remembering it, and move to the rightmost symbol. If the leftmost and rightmost symbols are same then remove the rightmost symbol also and move to the leftmost symbol. This process is repeated until the string get emptied. Note that if the string is odd length then single symbol (middle element of the string) remains on the tape.

This strategy can directly be applied to design a Turing machine for this purpose. Blank cells can be used as left and right end of the string. Let us consider the palindrome 0100110010. The designed Turing machine should handle this string as follows:



The equivalent machine designed for this purpose is given by the following Table:

Table 9.9 Transition table of Exercise 9.3

PRESENT STATE	NEXT STATE		
	0	1	#
$\rightarrow q_0$	$\#Rq_1$	$\#Rq_4$	$\#RH$
q_1	$0Rq_1$	$1Rq_1$	$\#Lq_2$
q_2	$0Lq_3$	-	$\#RH$
q_3	$0Lq_3$	$1Lq_3$	$\#Rq_0$
q_4	$0Rq_4$	$1Rq_4$	$\#Lq_5$
q_5	-	$\#Lq_6$	$\#RH$
q_6	$0Lq_6$	$1Lq_6$	$\#Rq_0$
H	-	-	-

9.4 The strategy to remove a particular element from a given string is that the pointed (selected) element is replaced by a blank, and then all the symbols right to this blank are shifted one cell left. This way the blank cell (created to remove a symbol) is removed.

The Turing machine designed for this purpose first replaces the pointed symbol with blank (say #) and then moves one cell left. If this cell contains symbol 0 then the machine replaces it with another symbol say, B . This way the location of the deleted symbol can easily be detected. Now the machine shifts all the symbols right to A or B one by one. The required Turing machine for this purpose is given below:

Table 9.10 Transition table of Exercise 9.4

PRESENT STATE	NEXT STATE				
	0	1	#	A	B
$\rightarrow q_0$	$\#Lq_1$	$\#Lq_1$	-	-	-
q_1	ARq_2	BRq_2	-	-	-
q_2	-	-	$\#Rq_3$	-	-
q_3	$0Lq_4$	$1Lq_6$	$\#Lq_7$	-	-
q_4	$0Rq_5$	$0Rq_5$	$0Rq_5$	-	-
q_5	$0Rq_3$	$1Rq_3$	$\#Rq_3$	-	-
q_6	$1Rq_5$	$1Rq_5$	$1Rq_5$	-	-
q_7	$\#Rq_8$	$\#Rq_8$	-	-	-
q_8	$0Lq_8$	$1Lq_8$	$\#Lq_8$	$0RH$	$1RH$
H	-	-	-	-	-

Objective Questions

1. A Turing machine
 - (a) is a simple mathematical model of a general purpose computer
 - (b) models the computing power of a computer

- (c) is capable of performing any calculation which can be performed by any computing machine
(d) all of the above
2. A Turing machine is similar to a finite automaton with only the difference of
(a) read/write head (b) input tape (c) finite state control (d) all of the above
 3. An ID of a Turing machine can be defined in terms of
(a) current state and input to be processed (b) current state and entire input string
(c) current state only (d) input string only
 4. A string Σ^* is accepted by a Turing machine if
(a) $q_0 s \xrightarrow{*} \alpha H \alpha_2$ for some $\alpha_1, \alpha_2 \in \Gamma^*$ and $H \in Q$
(b) $q_0 s \xrightarrow{*} \alpha_1 q' \alpha_2$ for some $\alpha_1, \alpha_2 \in \Gamma^*$ and $q' \in Q$
(c) $q_0 s \xrightarrow{*} \alpha q \#$ for some $\alpha \in \Gamma^*$ and $q_0 \in H$
(d) all of the above
 5. A Turing machine with transition $\delta(q, 1) = \{(q_1, 1, R), (q_2, 0, L)\}$ is a
(a) deterministic (b) non-deterministic (c) both (a) and (b) (d) all of the above
 6. An instantaneous description of a Turing machine is $a_1 a_2 a_3 q_5 a_4 a_5$ then
(a) the symbol under read/write head is a_4 (b) left-sequence is $a_1 a_2 a_3$
(c) right-sequence is a_5 (d) all of the above
 7. A Turing machine represented by a transition table has entry $1Lq_4$ corresponding to q_3 -row and 0-column then which of the following statements is false?
(a) the symbol under read/write head is 0 (b) next state is q_4
(c) q_3 is the initial state (d) all of the above
 8. Which of the following statements is false
(a) A 2-PDA is equivalent to a Turing machine
(b) A Turing machine with $\delta: Q \times \Gamma \rightarrow 2^{Q \times \Gamma \times \{L, R\}}$ is a nondeterministic Turing machine.
(c) No Turing machine can be considered with more than one tape.
(d) None of the above.
 9. A the Universal Turing machine influences the concept of
(a) computability (b) stored-programme computers
(c) both (a) and (b) (d) none of these
 10. The number of internal states in a Universal Turing machine should be at least
(a) 4 (b) 3 (c) 2 (d) 1
 11. If there exists a Turing machine, which when applied to any problem in the class, terminates if the correct answer is yes and may or may not terminate otherwise, is called a
(a) partially solvable (b) undecidable (c) unstable (d) both (a) and (b)
 12. Which of the following is not possible algorithmically
(a) non-deterministic Turing machine to deterministic Turing machine
(b) non-deterministic PDA to deterministic PDA
(c) NDFA to PDA
(d) regular grammar to context-free grammar

13. A finite machine having finite tape length without rewinding capability and unidirectional tape movement is called
(a) Turing machine (b) PDA (c) DFA (d) all of these
14. Which of the following statements is false
(a) A Turing machine is more powerful than a finite state machine because it has halt state.
(b) A finite state machine can be assumed to be a Turing machine of finite tape length, with rewinding capability and unidirectional tape movement.
(c) both (a) and (b)
(d) none of the above
15. Which of the following statements is true
(a) A pushdown machine behaves like a Turing machine when it contains two auxiliary memories.
(b) A finite state machine can be assumed to be a Turing machine of finite tape length, rewinding capability, and unidirectional tape movement.
(c) both (a) and (b)
(d) none of the above
16. Which of the following statements is false
(a) The recursiveness problem of Type-0 grammar is unsolvable
(b) There is no algorithm that can determine whether or not a given Turing machine halts with a complete blank tape when it starts with a given tape configuration.
(c) The problem of determining whether or not a given context-sensitive language is context-free is a solvable problem.
(d) none of the above
17. Which of the following statements is false
(a) If a language is not recursively enumerable, then its complement cannot be recursive.
(b) The family of recursive languages is closed under union.
(c) The family of recursive languages is closed under intersection.
(d) none of the above.
18. Which of the following statements is true
(a) If language L_1 is recursive and language L_2 is recursively enumerable then $L_2 - L_1$ is necessarily recursively enumerable.
(b) The complement of a context-free language must be recursive.
(c) both (a) and (b).
(d) none of the above.
19. Which of the following statements is false
(a) Every context-sensitive language is recursive (b) Every recursive language is context-sensitive.
(c) both (a) and (b) (d) none of the above

References for Extra Reading

1. Turing, A.M. *On computable numbers, with an application to the Entscheidungs problem*, Proc. London Math. Soc., 42, 1936.
2. Ben-Amram, A.M., O. Berkman and H. Petersen, *Element distinctness on one-tape Turing machines*, Acta Informatica, 40: 81-94, 2003.
3. Hartmanis, J., *Context free languages and Turing machine computations*, Proceedings of Symposia on Applied Mathematics 19, American Mathematical Society, Providence, Rhode Island, 1967.
4. Myhill, J., *Linear bounded automata*, WADD TR-57-624, 112-137, Wright Patterson Air Force Base, Ohio, 1957.
5. John E. Hopcroft and Jeffrey D. Ullman, *Introduction to Automata Theory, Languages and Computation*, Addison-Wesley Publishing, 1997.
6. Kuroda, S.Y., *Classes of languages and linear bounded automata*, Information and Control, 7: 207-223, 1964.
7. Rozenberg, G. and A. Salomaa, eds., *Handbook of Formal Languages*, 3 vols. Springer, Berlin, 1997.

Online Sources

1. <http://www.cs.princeton.edu/courses/archive/spr01/cs126/lectures/T2-4up.pdf>
2. <http://www.cs.uky.edu/~lewis/texts/theory/automata/autointr.pdf>
3. http://www.en.wikipedia.org/wiki/Automata_theory
4. http://en.wikipedia.org/wiki/Linear_bounded_automaton

10

Undecidability and Computability

❖ Chapter Objective ❖

In this chapter we will discuss the role of decision problems in computation. We will describe unsolvable problems involving context free languages and undecidable problems that are recursively enumerable. We will see the halting problem of a Turing machine as an undecidable problem. Then we will discuss the undecidability of the Post Correspondence Problem. In this sequence we will describe the Modified Post Correspondence Problem as a subset of a Post Correspondence Problem. We will see some languages that are not recursively enumerable.

The discussion on the recursive function theory will include partial recursive and primary recursive functions followed by Ackermann's function. The concept of reducing one undecidable problem to another is discussed by following Rice's theorem. Our discussion will be finished by discussing a brief introduction to Computational Complexity, Rewriting Systems, Matrix Grammar, and Markov Algorithm.

□ Introduction □

Some undecidability results have been experienced in the recursive function theory, since they involve problems which belong to this field. One of the simplest problems is the halting problem: Decide if a given programme halts for a given input. In fact, virtually every non-trivial problem about the behavior of programmes is undecidable. However, in the current section we are interested primarily in problems from other areas of mathematics. One such area is rewriting systems. A typical example of an undecidable problem is the following:

For given strings x, y , 'Can x be transformed into y by repeated application of a given set of rewriting rules?' Each rewriting rule has the form 'replace an occurrence of u (within the current string) by v '.

The theory of automata and formal languages are closely related. Also it includes so many decidability and undecidability results. One of the most famous undecidability results was established by Matijasevic in 1970. This says : There exists no algorithm to decide if a given polynomial equation (in several variables) has a solution.

Many important examples of decidable and undecidable problems occur in mathematical logic. If the halting problem was the only undecidable problem in the universe, then we might feel better. Unfortunately, there are numbers of very useful problems we would like to solve that happen to be undecidable.

In the middle of the decade of 1930, there was a great attempt to rigorously define computability and algorithms. In 1934 Gödel pointed out that primitive recursive functions can be computed by a finite procedure. Also, Gödel hypothesised that any function computable by a finite procedure can be specified by a recursive function. The year 1936 was also the time of the independently designed computing machine called Turing machine by Alan Turing and Alonzo Church. This machine was able to carry out a finite procedure.

10.1

UNSOLVABLE PROBLEMS INVOLVING CFLS

The problems for which there exists a mathematical solution or the problems that can be solved by a Turing machine are called decidable or solvable problems. There exist solution algorithms for some decision problems involving context-free grammars and context-free languages. The membership problem for context-free languages (Given a CFG G and a string s , Is $s \in (L(G)?$) is unsolvable. However there are some other problems which are unsolvable, and in this chapter we have considered two techniques for obtaining unsolvability results involving context-free grammars.

The first approach uses the unsolvability of the Post Correspondence Problem (discussed in this Chapter previous section). We start with the description of a useful construction in which two context-free grammars are obtained from an instance of Post's Correspondence Problem (PCP). Let i be the instance of PCP determined by the pairs

$$(\alpha_1, \beta_1), (\alpha_2, \beta_2), (\alpha_3, \beta_3), \dots, (\alpha_n, \beta_n)$$

where, α_i and β_i are the strings over Σ for some i .

Suppose a set C is defined as

$$C = \{C_1, C_2, C_3, C_4, \dots, C_n\}$$

where, $C_i \notin \Sigma$ for $i = 1, 2, 3, \dots, n$. The terminal symbol of both grammars will be the symbols of $\Sigma \cup C$. Let G_α be the context free grammar with start symbol S_α and production of the form

$$S_\alpha \rightarrow \alpha_i C_i \mid \alpha_i S_\alpha C_i \quad \text{for } i = 1, 2, 3, \dots, n$$

and let G_β be the context free grammar with start symbol S_β and production of the form

$$S_\beta \rightarrow \beta_i C_i \mid \beta_i S_\beta C_i \quad \text{for } i = 1, 2, 3, \dots, n$$

Then $L(G_\alpha)$ is the language of the form

$$\alpha_{i_1} \alpha_{i_2} \alpha_{i_3} \alpha_{i_4} \dots \alpha_{i_k} C_{i_k} \alpha_{i_{k-1}} \dots C_{i_1} \quad \text{for } k \geq 1$$

and $L(G_\beta)$ is the language same as $L(G_\alpha)$ with only the difference that each α_{i_j} is replaced by β_{i_j} for all j .

The second approach, a somewhat more direct approach to decision problems involving context-free languages, is to develop a set of string representing Turing machine computations and to show that they can be described in terms of context-free grammars.

10.2 UNDECIDABLE PROBLEMS THAT ARE RECURSIVELY ENUMERABLE

The recursively enumerable languages are so general that any question we raise about them is undecidable. When we talk about the Rice's theorem, any nontrivial property of a recursively enumerable language is undecidable. The nontrivial properties of recursively enumerable languages include: Whether a language L is empty? Whether L is infinite? Whether the language L contains two different strings of the same length? The membership algorithm of recursively enumerable languages is undecidable.

Some undecidable problems for CFG are: Is the intersection of two context-free languages context-free? Is a context free grammar ambiguous?

In the next section, we will see that the Post's Correspondence Problems (PCP) and Modified Post's Correspondence Problems (MPCP) are undecidable. In reference to proof of this, we can say that since the membership problem is undecidable, the MPCP is undecidable.

Halting Problem of TM The halting problem of a Turing machine is undecidable. For a given input, we cannot say whether a particular Turing machine M will halt or not. In other words, there is no algorithm for determining, given a Turing machine M and an input string ' s ', whether ' s ' is accepted by M . Also, for a certain fixed Turing machine M_f , there is no algorithm for determining, given an input string ' s ', whether the Turing machine M_f accepts ' s '.

10.3 POST CORRESPONDENCE PROBLEM

In this section we will see that a combinatorial problem called Post's Correspondence Problem (PCP) is undecidable. Although the proof is rather complicated, the problem can be understood very easily even by someone who does not know anything about Turing machines. Using its undecidability is one way of showing that a number of decision problems involving context-free grammars are also unsolvable.

The *PCP* was first formulated by Emil Leon Post in 1946. Later, this problem was found to have many applications in the theory of formal languages. The *PCP* over an alphabet Σ belongs to a class of Yes/No problems and is stated as follows:

For two lists $\alpha = (\alpha_1, \alpha_2, \alpha_3, \dots, \alpha_n)$, and $\beta = (\beta_1, \beta_2, \beta_3, \dots, \beta_n)$ of non-empty strings over the alphabet $\Sigma = \{0, 1\}$, the *PCP* determines whether or not there exist $i_1, i_2, i_3, \dots, i_m$, where $1 \leq i_j \leq n$, such that

$$\alpha_{i_1} \alpha_{i_2} \alpha_{i_3} \alpha_{i_4} \dots \alpha_{i_m} = \beta_{i_1} \beta_{i_2} \beta_{i_3} \beta_{i_4} \dots \beta_{i_m}$$

The instance is a *Yes-instance* if there is such a sequence, and we call the sequence a solution of the instance.

It may be helpful in visualising the problem to think of n distinct groups of dominoes, each domino from the i^{th} group having the string α_i on the top half and the string β_i on the bottom half (as shown by Fig. 10.1), and to imagine that there are infinite identical dominoes in each group. Finding a solution for this instance means sequencing up one or more dominoes in a horizontal row, each one positioned vertically, so that the string formed by their top halves matches the string formed by their bottom halves.

α_1	α_2	α_3	...	α_n
β_1	β_2	β_3	...	β_n

α_{i_1}	α_{i_2}	α_{i_3}	...	α_{i_k}
β_{i_1}	β_{i_2}	β_{i_3}	...	β_{i_k}

Fig. 10.1 Dominoes and halves of post correspondence problem

Example 10.1

Consider the Post correspondence system described by the following lists
 $\alpha = (10, 01, 0, 100, 1)$, $\beta = (101, 100, 10, 0, 010)$

Does this PCP have a solution?

Solution First of all we observe the lists in distinct groups of dominoes as shown in Fig. 10.2 (see below).

<table><tr><td>10</td></tr><tr><td>101</td></tr></table>	10	101	<table><tr><td>01</td></tr><tr><td>100</td></tr></table>	01	100	<table><tr><td>0</td></tr><tr><td>10</td></tr></table>	0	10	<table><tr><td>100</td></tr><tr><td>0</td></tr></table>	100	0	<table><tr><td>1</td></tr><tr><td>010</td></tr></table>	1	010
10														
101														
01														
100														
0														
10														
100														
0														
1														
010														
Domino 1	Domino 2	Domino 3	Domino 4	Domino 5										

Fig. 10.2 Groups of distinct dominoes

In any solution instance of PCP, domino 1 must be used first because it is the only one in which the two strings begin with the same symbol. One of the possible solutions is the following (see Fig. 10.3).

10	1	01	0	100	100	0	100
101	010	100	10	0	0	10	0

Fig. 10.3 A possible solution of Example 10.1

Here we see that

$$\begin{aligned}
 \alpha_{i_1} \alpha_{i_2} \alpha_{i_3} \alpha_{i_4} \alpha_{i_5} \alpha_{i_6} \alpha_{i_7} \alpha_{i_8} &= 10 \ 1 \ 01 \ 0 \ 100 \ 100 \ 0 \ 100 \\
 &= 101 \ 010 \ 100 \ 10 \ 0 \ 0 \ 10 \ 0 \\
 &= \beta_{i_1} \beta_{i_2} \beta_{i_3} \beta_{i_4} \beta_{i_5} \beta_{i_6} \beta_{i_7} \beta_{i_8}
 \end{aligned}$$

What we have done here is, we have put α_i and β_i (for some $i = 1, 2, 3, \dots, n$) in the following way:

$$\begin{aligned}
 10 \ 1 \ 01 \ 0 \ 100 \ 100 \ 0 \ 100 &= 101 \ 010 \ 100 \ 10 \ 0 \ 0 \ 10 \ 0 \\
 \alpha_1 \ \alpha_5 \ \alpha_2 \ \alpha_3 \ \alpha_4 \ \alpha_4 \ \alpha_3 \ \alpha_4 &= \beta_1 \ \beta_5 \ \beta_2 \ \beta_3 \ \beta_4 \ \beta_4 \ \beta_3 \ \beta_4
 \end{aligned}$$

Here, $m = 8$, that means $(1, 5, 2, 3, 4, 4, 3, 4)$. Thus the given PCP has a solution.

As we know a PCP is an unsolvable problem, there are at least two cases when we can say that a particular PCP has no solution at all.

- If the first symbol in a domino does not match in both upper and lower halves, and this is true for all remaining dominoes.
- If for all dominoes, the lengths of strings of a particular half (either lower or upper) are either more or less.

For example, PCP with $\alpha = (01, 11, 010, 1)$ and $\beta = (10, 01, 101, 0)$ has no solution by (i), and a PCP with $\alpha = (11, 011, 101, 0)$ and $\beta = (101, 1011, 0110, 10)$ has no solution by (ii).

Note If a PCP has a solution then it has infinite numbers of solutions. As in example 10.1, we see that the PCP has a solution $(1, 5, 2, 3, 4, 4, 3, 4)$. Needless to say that this PCP also has solutions, $(1, 5, 2, 3, 4, 4, 3, 4, 1, 5, 2, 3, 4, 4, 3, 4, 1, 5, 2, 3, 4, 4, 3, 4, 1, 5, 2, 3, 4, 4, 3, 4)$, and so on.

If the first substring used in *PCP* is always α_1 and β_1 then the *PCP* is called a modified Post's correspondence problem (*MPCP*). An instance of the modified Post's correspondence problem is exactly the same as an instance of *PCP*. In other words, a solution for the instance is required to begin with domino 1 (i.e., α_1 and β_1). In *MPCP*, a solution consists of a sequence of zero or more integers $i_2, i_3, \dots, i_m$ such that

$$\alpha_1, \alpha_{i_2}, \alpha_{i_3}, \alpha_{i_4}, \dots, \alpha_{i_m} = \beta_1, \beta_{i_2}, \beta_{i_3}, \beta_{i_4} \dots \beta_{i_m}$$

Undecidability in Ambiguity of CFGs The Post Correspondence Problem is a convenient tool to study undecidable questions for CFLs. As we know there exists no algorithm for deciding whether any given context free grammar is ambiguous. But how can we prove it?

Let us consider two sequence of strings

$$X = (u_1, u_2, \dots, u_m) \text{ and } Y = (v_1, v_2, \dots, v_m)$$

over some alphabet Σ . Let us choose a new set of distinct symbols $a_1, a_2, a_3, \dots, a_m$ such that

$$\{a_1, a_2, a_3, \dots, a_{m-1}, a_m\} \cap \Sigma = \emptyset$$

and consider two context free languages

$$L_x = \{u_i u_j, \dots, u_1 u_k a_k a_1, \dots, a_j a_i\}$$

defined over X and $\{a_1, a_2, a_3, \dots, a_{m-1}, a_m\}$
and

$$L_y = \{v_i v_j, \dots, v_1 v_k a_k a_1, \dots, a_j a_i\}$$

defined over Y and $\{a_1, a_2, a_3, \dots, a_{m-1}, a_m\}$

Let G be the context free grammar defined as

$$G = (V_N, \Sigma', P, S)$$

Where

$$V_N = \{S, S_x, S_y\}$$

$$\Sigma' = \{a_1, a_2, a_3, \dots, a_{m-1}, a_m\} \cup \Sigma$$

$$P = P_x \cup P_y$$

$$P_x = \{S \rightarrow S_x, S_x \rightarrow u_i S_x a_i \mid u_i a_i \text{ for } i = 1, 2, \dots, n\}$$

$$P_y = \{S \rightarrow S_y, S_y \rightarrow v_i S_y a_i \text{ for } i = 1, 2, \dots, n\}$$

Now we take

$$G_x = (\{S, S_x\}, \{a_1, a_2, a_3, \dots, a_{m-1}, a_m\} \cup \Sigma, P_x, S)$$

and

$$G_y = (\{S, S_y\}, \{a_1, a_2, a_3, \dots, a_{m-1}, a_m\} \cup \Sigma, P_y, S)$$

then

$$L_x = L(G_x)$$

and

$$L_y = L(G_y)$$

Therefore,

$$L(G) = L_x \cup L_y$$

It is quite easy to say that G_x and G_y by themselves are unambiguous. If a string in $L(G)$ ends with a_i , then its derivation with grammar G_x must have started with $S \Rightarrow u_i S_x a_i$. Similarly, we can say that at any later stage which rule has to be applied. Therefore, if grammar G is ambiguous it must be because of string ' ω ' for which there are two derivations

$$S \Rightarrow S_x \Rightarrow u_i S_x a_i \xRightarrow{*} u_i u_j \dots u_k a_k \dots a_j a_i = \omega$$

and

$$S \Rightarrow S_y \Rightarrow v_i S_y a_i \Rightarrow v_i v_j \dots v_k a_k \dots a_j a_i = \omega$$

If G is ambiguous, then the *PCP* with the pair (X, Y) has a solution and if grammar G is unambiguous, then the *PCP* has no solution. If there existed an algorithm for solving the ambiguity problem, we could adapt it to solve the *PCP*, but, since there is no algorithm for the *PCP*, we conclude that the ambiguity problem is undecidable.

10.4

MODIFIED POST CORRESPONDENCE PROBLEM

If the first substring used in a *PCP* is always α_1 and β_1 then the *PCP* is called a modified Post's correspondence problem (*MPCP*). An instance of the modified post's correspondence problem is exactly the same as an instance of *PCP*. In other words, a solution for the instance is required to begin with domino 1 (i.e., α_1 and β_1). In *MPCP*, a solution consists of a sequence of zero or more integers $i_2, i_3, \dots, i_m$ such that

$$\alpha_1 \alpha_{i_2} \alpha_{i_3} \alpha_{i_4} \dots \alpha_{i_m} = \beta_1 \beta_{i_2} \beta_{i_3} \beta_{i_4} \dots \beta_{i_m}$$

10.5

LANGUAGES THAT ARE NOT RECURSIVELY ENUMERABLE

A language L is said to be recursively enumerable if $L = L(M)$ for some arbitrary Turing machine. The diagonalisation language denoted by L_d and universal language denoted by L_u (already discussed in Chapter 9) are undecidable problems that are not recursively enumerable.

10.6

CONTEXT SENSITIVE LANGUAGES

The languages that cannot be accepted by a push down automaton are called context sensitive languages. Examples of some context sensitive languages which we have already seen in previous chapters are:

$$L = \{a^n b^n c^n \mid n \geq 0\}$$

$$L = \{a^n \mid n \text{ is a prime number}\}$$

$$L = \{a^n \mid n \text{ is a perfect square}\}$$

$$L = \{a^n \mid n \geq 0\}$$

The grammars generating such languages are called context sensitive grammars. There is at least one production in context sensitive grammar such that it has either left or right or both contexts in the left hand side of a production. For example, the language $L = \{a^n b^n c^n \mid n \geq 1\}$ can be generated by a grammar having productions

$$S \rightarrow aSAC \mid abc, cA \rightarrow Ac, bA \rightarrow bb,$$

The productions $bA \rightarrow bb$ and $cA \rightarrow Ac$ have left contexts.

10.7

COMPUTABILITY

All computer programs may be viewed as computing functions. Since all computer employ binary notations, such functions are defined over set of binary strings. The question, ‘What can be solved by computers?’ is equivalent to what decision problems can be solved.

Computability is the concept that explores what we mean when we say that something cannot be done by a Turing machine or digital computer.

Up to this point we have talked about what Turing machines can do. We now look at what they cannot do. We have considered automata as accepting devices until Chapter 9. Since Chapter 9 we have seen automata as computing devices. To find whether a given problem is solvable by automata the problem reduces to the evolution of functions on the set of natural numbers or a given alphabet by mechanical means.

A function f in a particular domain will be called a computable function if there exists a Turing machine that computes the value of function f for all arguments in its domain. Equivalently a function will be called incomputable if no such Turing machine exists. There may be a Turing machine that can compute function f on parts of its domain, but the function f will be called a computable function if only if there is a Turing machine that computes the function on the whole of its domain.

If a Turing machine takes a sequence of numbers as input and gives only one number as output, we say that the computer acts as a mathematical function. Any operation which is defined on all sequences of some numbers and that can be performed by a Turing machine is called Turing-computable or simply computable.

10.8

RECURSIVE FUNCTION THEORY

In this section we will see the definitions and properties of partial recursive and primary recursive functions.

10.8.1 Partial Function

A partial function $f: X \rightarrow Y$ (read as f from X to Y) is defined as a rule which assigns at most one element of Y to every element of X . In order words a partial function f may be undefined at certain points (the points not in the domain of f).

As an example, if R denotes the set of real numbers, the rule $f: R \rightarrow R$ given by $f(r) = +\sqrt{r}$ is a partial function because $f(r)$ is defined as a real number when r is negative. But $f'(r) = r^2$ is a total function defined as $f': R \rightarrow R$.

10.8.2 Total Function

A total function $f: X \rightarrow Y$ is defined as a rule which assigns a unique element of Y to every element of X . In other words if a function f may be defined at all points in the domain of f will be called a total function.

A partial or total function $f: X^k \rightarrow X$ is also called a function of k variables and denoted by $(f(x_1, x_2, x_3, \dots, x_k))$. For example, $f(x_1, x_2) = x_1 + x_2$ is a function of two variables, then we have $f(2, 3) = 5$, here 2 and 3 are called the arguments and 5 is the value.

10.8.3 Recursive Functions

The recursive functions are a class of functions from natural numbers to natural numbers which are computable in some intuitive sense. In computability theory, the recursive functions are considered as those functions that can be computed by Turing machines. Recursive functions are much related to primitive recursive functions and their inductive definition builds upon that of the primitive recursive functions.

A function is known as a recursive function if it can be obtained from the initial functions by applying a finite number of applications of recursion, composition and minimisation in any order over regular functions.

In other words, a function is known as a recursive function if and only if it can be obtained from the initial functions by applying a finite number of applications of recursion, composition and minimisation.

In general, a recursive function may be a partial function or a total function. Let us consider a function $f_1(x_1, x_2, x_3, \dots, x_n, y)$ as a total function over N . The function f_1 is a regular function if there exists some natural number y_0 such that

$$f_m(x_1, x_2, x_3, \dots, x_n, y_0) = 0$$

for all values $x_1, x_2, x_3, \dots, x_n$ in N . For example, $f_m(x, y) = \min(x, y)$ is a regular function because

$$f_m(x, 0) = 0 \text{ for all } x \in N.$$

A function $f(x_1, x_2, x_3, \dots, x_n)$ over N can be defined from a total function $f(x_1, x_2, x_3, \dots, x_n, y)$ by minimisation if:

- (i) the function $f(x_1, x_2, x_3, \dots, x_n)$ is the least value of all y 's such that

$$f_y(x_1, x_2, x_3, \dots, x_n, y) = 0,$$

if it exists.

- (ii) the function $f(x_1, x_2, x_3, \dots, x_n)$ is undefined if there is no ' y ' such that

$$f_y(x_1, x_2, x_3, \dots, x_n, y) = 0.$$

10.8.4 Primitive Recursive Function

Now we discuss the class of primitive recursive function by applying certain operations on primitive recursive functions. Before defining primitive recursive functions, we need to define some basic functions called initial functions.

10.8.5 Initial Functions

The initial functions over the set of natural numbers N , are defined as follows:

The Zero Function Z It is defined as $Z(x) = 0$, for all $x \in N$. For example, $Z(5) = 0$.

The Successor Function S It is defined as $S(x) = x + 1$. The successor function increases the value of input by 1. For example, $S(6) = 7$.

The Projection Function $\cup_i^n$ It is defined as $\cup_i^n(x_1, x_2, x_3, \dots, x_n) = x_i$ for $1 \leq i \leq n$. The projection function $\cup_i^n$ selects the i^{th} value from given n values in a set. For example,

$$\cup_4^6(1, 3, 5, 8, 9, 23) = 8.$$

If $i = n = 1$, then $\cup_1^1(x) = x$ for all $x \in N$. In this case $\cup_1^1$ is simply called an identity function.

Some initial functions are also defined over $\Sigma = \{a, b\}$. Some cases of such functions are:

The Nil Function It is defined as $nil(\Sigma^*) = \wedge$, where $\wedge$ is a string of length zero. For example, $nil(babb) = \wedge$, for $babb \in \Sigma^*$.

The Cons Function This function concatenates two strings, and is defined as $cons(x_1, x_2) = x_1x_2$ for all $x_1, x_2 \in \Sigma^*$. For example, $cons(ba, bb) = babb$. Note that $cons(x_1, x_2) \neq cons(x_2, x_1)$.

The Composition Function If $f_1, f_2, f_3, f_4, \dots, f_m$ are partial functions of n variables and f_p is a partial function of l variables, then the composition function is also a partial function of n variable. The composition (of f_p with f_1, f_2) function is given by

$$f_p(f_1(x_1, x_2, x_3, x_4, x_5, \dots, x_n), f_2(x_1, x_2, x_3, x_4, x_5, \dots, x_n), \dots, f_m(x_1, x_2, x_3, x_4, x_5, \dots, x_n))$$

over $X = \{x_1, x_2, x_3, x_4, x_5, \dots, x_n\}$. For example, f_1, f_2 , and f_3 are three partial functions of two variables and f_p is a partial function of three variables then the composition function is given by $f_p(f_1(x_1, x_2), f_2(x_1, x_2), f_3(x_1, x_2))$.

Example 10.2

Consider three partial functions of two variables defined as $f_1(x_1, x_2) = x_1 + x_2$, $f_2(x_1, x_2) = 2x_1 + 3x_2$ and f_p is a partial function of two variables defined as $f_p(x_1, x_2) = x_1 + x_2$. Determine the composition of f_p with f_1, f_2 .

Solution Given $f_1(x_1, x_2) = x_1 + x_2$ and $f_2(x_1, x_2) = 2x_1 + 3x_2$

$$\begin{aligned} f_p(f_1(x_1, x_2), f_2(x_1, x_2)) &= f_p(x_1 + x_2, 2x_1 + 3x_2) \\ &= x_1 + x_2 + 2x_1 + 3x_2. \end{aligned}$$

Therefore the composition of f_p with f_1, f_2 can be expressed by some function com as

$$com(x_1, x_2) = x_1 + x_2 + 2x_1 + 3x_2.$$

10.8.6 Functions Defined by Recursion

A function $f(x)$ over the set of natural numbers N , can be defined by recursion if there exists a constant $k \in N$ and a function $g(x, y)$ such that

$$f(0) = k,$$

$$\text{and } f(n+1) = g(n, f(n)).$$

By using function on n , $f(n)$ can be defined for all n . Here $f(0) = k$, is the basis for induction. Once the value of $f(n)$ is evaluated, the value of the function $f(n+1)$ can be evaluated.

Let us define the factorial of n (represented as $n!$). The basis for induction is the initial value 1 of factorial 0.

$$f(0) = 1$$

$f(n+1)$ is defined by a function g in terms of n and $f(n)$, as

$$f(n+1) = g(n, f(n)),$$

where,

$$g(x_1, x_2) = S(x_1) * x_2.$$

In other words, we say that $f(n+1)$ is defined as

$$f(n+1) = (n+1) * f(n).$$

Thus a function f of $n+1$ variables can be defined by recursion if there exists a function f_n of n variables, and a function f_{n+2} of $n+2$ variables. The function f is defined as follows

$$f(x_1, x_2, x_3, x_4, x_5, \dots, x_n, 0) = f_n(x_1, x_2, x_3, x_4, x_5, \dots, x_n)$$

$$\text{and } f(x_1, x_2, x_3, x_4, x_5, \dots, x_n, y+1) = f_{n+2}(x_1, x_2, x_3, x_4, \dots, x_n, y, f(x_1, x_2, x_3, x_4, \dots, x_n)).$$

Make sure that function f can be evaluated for all arguments $x_1, x_2, x_3, x_4, x_5, \dots, x_n, y$ by induction on y for fixed variables $x_1, x_2, x_3, x_4, x_5, \dots, x_n$. This process is repeated for every variable $x_1, x_2, x_3, x_4, x_5, \dots, x_n$.

Let us now define primitive recursive functions over N and $\{a, b\}$. A total function f over N is called a primitive recursive function if:

- (i) it is any one of the three initial functions
- (ii) it is obtained by applying recursion and composition finite number of times to some or all three initial functions.

As an example, a function f defined as $f(x, y) = x + y$ is a primitive recursive function because it can be expressed in terms of initial functions.

10.8.7 μ -recursive Function

The set S of μ -recursive partial functions, is defined by following rules:

- (i) All initial functions are elements of S .
- (ii) Any function obtained from elements of S by composition is also an element of S .
- (iii) If a total function $f: N^{n+1} \rightarrow N$ for any $n \geq 0$ is in S , then the function $M_f: N^{n+1} \rightarrow N$ defined by

$$M_f(Z) = \mu y [f(Z, y) = 0]$$

is an element of S .

- (iv) No other functions are in S .

All μ -recursive partial functions are computable, and every computable function $f: N^{n+1} \rightarrow N$ is μ -recursive. If a function defined as $f: N \rightarrow N$ is a μ -recursive total function that is bijection from N to N , then its inverse f^{-1} is also μ -recursive.

10.8.8 Gödel Numbering

Gödel numbering, an encoding scheme to assign numbers to statements and formulae, an axiomatic system, was developed by logician Kurt Gödel in 1930s. By using these numbers Gödel was able to describe logical relations among objects in the system by numeric formulae expressing relations among the numbers.

Gödel number of a sequence of natural numbers, from any finite sequence $x_0, x_1, x_2, \dots, x_n$ of natural numbers is defined as

$$g_n(x_0, x_1, x_2, \dots, x_n) = 2^{x_0} 3^{x_1} 5^{x_2} 7^{x_3} \dots (PN(n))^{x_n}$$

Where $PN(n)$ is the n^{th} prime number. The Gödel number of any sequence is greater than or equal to 1, and every integer greater or equal to 1 is the Gödel number of a sequence.

If $g_n(x_0, x_1, x_2, x_3, x_4, x_5, \dots, x_m) = g_n(y_0, y_1, y_2, y_3, y_4, y_5, \dots, y_m, y_{m+1}, \dots, y_{m+k})$

$$\prod_{i=0}^m PN(i)^{x_i} = \prod_{i=0}^m PN(i)^{y_i} \prod_{i=m+1}^{m+k} PN(i)^{y_i}$$

The function g_n is not one-to-one. For example,

$$g_n(1, 0, 2) = g_n(1, 0, 2, 0, 0) = 2^1 3^0 5^2.$$

If two sequences have the same Gödel number, then they are identical with the difference that they may end with a different number of 0's.

In general, for a particular $n \geq 1$, any positive integer is the Gödel number of at most one sequence of n integer. If we wish to decode a Gödel number 'g' to find a sequence $x_0, x_1, x_2, \dots, x_n$, then we proceed by factoring 'g' into prime numbers raised to appropriate powers. For each i , x_i is the number of items $PN(i)$ appears as factor of 'g'. For example, the number 1750 has the factorisation

$$12250 = 2^1 5^3 7^2 = 2^1 3^0 5^3 7^2$$

and is therefore the Gödel number of the sequence 1,0,3,2 (or any other sequence obtained from this by adding additional 0's, e.g., 1, 0, 3, 2, 0, 0). The prime number 19 is the Gödel number of the sequence (0, 0, 0, 0, 0, 0, 1) since $19 = PN(7)$.

The Gödel numbering technique can be applied to Turing machines. A function f computed by a Turing machine is computed by applying a sequence of steps. If these steps can be managed to represent operations on numbers, then there will be a way of building the function f from more elementary functions. Since a Turing machine's move can be thought as a transformation of the machine from one configuration to another and it can be represented by a number.

We start by assigning a number to each state. The halt state, say I_p , is assigned by the number '0'. If Q is the set of states having elements $q_0, q_1, q_2, \dots, q_n$, we assume q_0 as initial state. The obvious number to use in the description of Read/Write head position is the number of the tape block (also called cell or square) just scanning. We assign number '0' to the blank symbol (denoted by #) on input tape, and we assume that the nonblank tape symbols are 1, 2, 3, ..., k . This allows us to define the tape number of the Turing machine at any point to be the Gödel number of the sequence of symbols currently on the

input tape. The point to be noted is that we are identifying ‘#’ (the blank) with ‘0’, the tape number is the same and it does not matter how many trailing blanks we include in the sequence. The input tape of a blank tape is 1.

As we know the configuration of a Turing machine is determined by the state, the Read/Write head position, and the current symbol under Read/Write head, we define the configuration number to be the number

$$g_n(q, P, N)$$

where,

q = present state number,

P = current R/W head position, and

N = current tape number.

Gödel Numbering of Strings The technique of Gödel numbering allows extension of the definitions of primitive and μ -recursive functions involving strings. If $\Sigma = \{a_1, a_2, a_3, a_4, \dots, a_s\}$ the Gödel number $g_n(x)$ of the string $x = \alpha_{i_0} \alpha_{i_1} \alpha_{i_2} \alpha_{i_3} \dots \alpha_{i_m} \in \Sigma^*$ is defined by

$$\begin{aligned} g_n(x) &= g_n(i_0, i_1, i_2, i_3, \dots, i_m) \\ &= 2^{i_0} 3^{i_1} 5^{i_2} 7^{i_3} \dots (PN(m))^{i_n} \end{aligned}$$

The Gödel number of null string (i.e., Λ) is defined to be 1.

❖ Did you Know ?

Kurt Gödel is well known for his renowned incompleteness theorem. The incompleteness theorem states that all mathematical questions are not computable. In his family, young Kurt Gödel was known as ‘Mr. Why’ because of his unquenchable curiosity. According to his brother Rudolf, ‘At the age of six or seven Gödel suffered from a rheumatic fever. He completely recovered but for the rest of his life he remained convinced that his heart had suffered permanent damage.’

Gödel solved several technical issues such as formal proofs, statements of encoding, and the very concept of probability of natural numbers. He achieved this using a process with the help of Gödel numbering which he invented.

In later life, Kurt Gödel suffered a period of mental insatiability and illness. He was suffering from obsessive fear of being poisoned. He would not eat unless his wife Adele tasted his eatables for him. Late in 1977, Adele was hospitalised for six months and could not taste Gödel’s eatables any more in her absence. He refused to eat. Eventually, he was starving himself to death. His weight was only 30 Kg when he died. In his death certificate it was certified that he died because of imbalanced consumption of nutrition and inanition caused by a personality disorder.

10.9

ACKERMANN’S FUNCTION

Ackermann’s function is defined by the rules :

- (i) $A(0, y) = y + 1$
- (ii) $A(x + 1, 0) = A(x, 1)$
- (iii) $A(x + 1, y + 1) = A(x, A(x + 1, y))$

An Ackermann's function $A(x, y)$ can be determined for every (x, y) , therefore $A(x, y)$ is a total function. Note that Ackermann's function is recursive but not primitive recursive.

Example 10.3

Compute Ackermann's function $A(1, 2)$ and $A(2, 1)$.

Solution Computation of $A(1, 2)$

$$\begin{aligned}
 A(1, 2) &= A(0 + 1, 1 + 1) \\
 &= A(0, A(1, 1)) \\
 &= A(0, A(0 + 1, 0 + 1)) \\
 &= A(0, A(0, A(1, 0))) \\
 &= A(0, A(0, A(0 + 1, 0))) \\
 &= A(0, A(0, A(0, 1))) \\
 &= A(0, A(0, 2)) \\
 &= A(0, 3) \\
 &= 4
 \end{aligned}$$

Computation of $A(2, 1)$

$$\begin{aligned}
 A(2, 1) &= A(1 + 1, 0 + 1) \\
 &= A(1, A(1, 1)) \\
 &= A(1, 3) \\
 &= A(0 + 1, 2 + 1) \\
 &= A(0, A(1, 2)) \\
 &= A(0, 4) \quad [\text{By } A(1, 2) = 4] \\
 &= 5
 \end{aligned}$$

10.10**REDUCING ONE UNDECIDABLE PROBLEM TO ANOTHER**

Although we are free to think about a decision problem on an informal level, a Turing machine must deal with the corresponding language directly. Since, Turing machines are our official models of computation, we begin with a definition of language being reducible to another, and we will then be able to reformulate the definition in terms of the problems themselves.

Suppose L_1 and L_2 are two languages over the alphabets Σ_1 and Σ_2 , respectively, then we say L_1 is reducible to L_2 (denoted by $L_1 \leq L_2$) if there is a Turing-computable function

$$f: \Sigma_1^* \rightarrow \Sigma_2^*$$

so that for any x in Σ_1^*

$$x \in L_1 \text{ if and only if } f(x) \in L_2.$$

It is common to refer to this type of reducibility as many-to-one reducibility, and the relation $\leq$ is often written as $\leq_m$.

Here, the idea is, if $L_1 \leq L_2$ is able to solve the membership problem for L_2 then it should be allowed to solve the problem for L_1 . This is exactly what the above definition tells. If there is a string x in Σ_1^* , and we want to decide whether $x \in L_1$, we can try to approach the problem indirectly by computing $f(x)$ and

trying to decide whether that string is in L_2 . Because of the fact that $x \in L_1$ if and only if $f(x) \in L_2$, the answer to the second question is guaranteed to give the answer to the first. Determining membership in L_1 is not harder than determining membership in L_2 , in the sense that the only additional work involved is computing f .

Additionally, if L_1 and L_2 are two languages over the alphabets Σ_1 and Σ_2 , respectively, and $L_1 \leq L_2$, then if L_2 is recursive then L_1 is also. Equivalently, if L_1 is not recursive then L_2 cannot be recursive.

Finally, if P_1 and P_2 are two any decision problems, we can say that P_1 is reducible to P_2 , (i.e., $P_1 \leq P_2$) if there exists an algorithmic procedure that allows it for a given arbitrary instance I_1 of P_1 , to find an instance $F(I_1)$ of P_2 so that every I_1 is *Yes-instance* of P_1 if and only if $F(I_1)$ is a *Yes-instance* of P_2 .

In addition to this, if P_2 can be solved algorithmically, then P_1 can also be.

10.11

RICE'S THEOREM

Rice's theorem has an important result in the theory of recursive functions. A property of partial functions is trivial or say insignificant if it is able to hold for all partial recursive functions or for none. Rice's theorem shows that for any non-trivial property of partial functions, the following question is undecidable: Whether a given algorithm can compute a partial function with such property.

As an example, let us consider the following variant of the halting problem:

Consider the property of a partial function F as, if function F is defined for argument 1. It is observably non-trivial, since there exist partial functions which are defined for argument 1 and others that are not defined for argument 1. The 1-halting problem can be seen as the problem of deciding of any algorithm whether it is able to define a function with this property. Rice's theorem shows that the 1-halting problem is undecidable.

Theorem 10.1 (Rice's Theorem) Any non-trivial property about the language recognised by a Turing machine is undecidable. An important property about a Turing machine can be expressed as the language of all Turing machines, encoded as strings, which satisfy that property. The property P is 'about the language recognised by Turing machine' if whenever

$$L(M) = L(N)$$

then P contains (the encoding of) M if and only if it contains (the encoding of) N . The property is non-trivial if there is at least one Turing machine that has the property, and at least one Turing machine that has not.

Proof Here we assume that a Turing machine that is able to recognise the empty language does not have the property P . If it does, we take the complement of P . The undecidability of the complement would immediately entail the undecidability of property P .

In order to reach a contradiction, assume P is decidable. it means there is a halting Turing machine (say B) that recognises the description of Turing machines that satisfies P . Using a Turing machine B we can simulate a Turing machine A that accepts the language

$$L = \{(M, s) \mid M \text{ is the description of a Turing machine that accepts the string 's'}\}.$$

As the later problem is undecidable this will show that B cannot exist and P must be undecidable as well.

Let MP be a Turing machine that satisfies P (as P is non-trivial). The Turing machine A operates as follows:

- (i) on input (M, s) , (description of) a Turing machine $C(M, s)$ is created as:
 - (a) on input x , let the Turing machine M run on the string ' s ' until it accepts (equivalently if M does not accept $C(M, s)$ it will run for an infinite time).
 - (b) Turing machine MP runs on X . (Accepted if and only if MP does), $C(M, s)$ accepts the same language as MP if M accepts ' s ', and $C(M, s)$ accepts the empty language if M does not accept ' s '.

Thus if M accepts ' s ' the Turing machine $C(M, s)$ has the property P , and otherwise it does not.

- (ii) The description of $C(M, s)$ is given to Turing machine B . If B accepts then the input (M, s) is accepted, otherwise rejected.

10.12

COMPUTATIONAL COMPLEXITY

The computation complexity theory is concerned with the question, 'For which decision problems efficient algorithms exist?' This raises questions like:

- ❖ What resources do we wish to employ efficiently?
- ❖ What do we mean by efficient?

Two significant complexity measures of our interest are TIME and SPACE complexities, that means the number of steps an algorithm takes to return its answer in worst case; and the amount of memory required to run the algorithm.

The *time complexity* of a decision problem f is the running time of the best algorithm A for f . This way we say that a decision problem f has time complexity $T(n)$ if there is an algorithm A for f such that the number of steps taken by A on inputs of length n are always less than or equal to $T(n)$.

The *space complexity* of a decision problem p can be defined as the amount of memory consumed by the best algorithm A for p . Therefore, we say that a decision problem p has space complexity $S(n)$ if there is an algorithm A for f such that the number of memory locations used by algorithm A on inputs of length n is always less than or equal to $S(n)$.

The above definitions allow us to classify decision problems according to their time or space complexity. This way we can define complexity classes. For example, $TIME(n)$ is the set of decision problems for which an algorithm exists, which runs in at most n steps on inputs of length n . More precisely, $TIME(n^3)$ is the set of decision problems for which an algorithm exists that runs in at most n^3 steps on inputs of length n .

In general, $TIME(T(n))$ is the set of decision problems for which an algorithm exists that runs in at most $T(n)$ steps on all inputs of length n .

Note A decision problem P , for which there is an algorithm taking at most n^k steps on inputs of length n is called a *polynomial time complexity decision problem*.

TIME and SPACE Complexity of a Turing Machine

The time complexity of a Turing machine M is the function T defined on the natural numbers as follows:

‘For a natural number n , $T(n)$ is the maximum number of moves the Turing machine M can make on any input string of length n . If there is an input string x having length n such that Turing machine M loops forever on input x , $T(n)$ is undefined.’

Now we talk about the space complexity function S of Turing machine M . If no input string of length n causes Turing machine M to use an infinite number of blocks on input tape, then $S(n)$ is the maximum number of blocks on tape used by Turing machine M for any input string of length n .

Note For a multitape Turing machine ‘number of blocks on tape’ means the maximum of the numbers for the individual tapes.

10.13

REWRITING SYSTEMS

A word is said to be in some normal form if it is the least word under the ordering of the rewriting system that can define a particular group element. Therefore, there exists always a unique word in normal form for each group element, and it can be determined by the group generators and the ordering on words in the left hand side of any of the reduction rules in the system as a substring. Words in normal form are always irreducible, but the converse is true if and only if the rewriting system is confluent.

Term Rewriting System A term rewriting system is collection of rewrite rules used to transform terms (expressions, strings in some formal language) into equivalent terms. It is the process of transforming an expression according to certain reduction rules. The most important forms are beta reduction (application of a lambda abstraction to one or more argument expressions) and delta reduction (application of a mathematical function to the required number of arguments). An extension of a term rewriting system which uses graph reduction on terms represented by directed graphs to avoid duplication of work by sharing expressions.

String Rewriting System A string rewriting system is a substitution system used to perform computation using Markov algorithms or to create certain types of fractals such as the Cantor set or Menger sponge. A fractal is a set which is self-similar. A curve is said to be self-similar if, for every piece of the curve, there is a smaller piece that is similar to it.

10.14

MATRIX GRAMMAR

Matrix grammars have been seen as one of the classical topics of formal languages, more particularly, regulated rewriting. The earliest work of this type on context free grammars have been seen in this type of control and, matrix grammars still pose interesting questions. One such class of problems is concerned with the leftmost derivation. Moreover, we can find a characterisation of the recursively enumerable languages for matrix grammars with the leftmost control defined on class of a defined partition of the set of nonterminal alphabet.

We have seen that the languages over a one-letter alphabet generated by a context-free matrix grammar are always regular. In this sequence, we can generate a decision procedure for the question of whether a context-free matrix language is finite.

We can improve some results based on the number of variables (nonterminal symbols) needed in matrix grammars, programmed grammars, and graph-controlled grammars with appearance checking for generating arbitrary recursively enumerable languages. The important result is that the number of nonterminals used in the appearance checking mode can be restricted to two.

In the case of graph controlled grammars (or programmed grammars) with appearance checking, the number of nonterminals can be reduced to three, but in the case of matrix grammars with appearance checking, for nonterminals are required, in which three of them are used in appearance checking mode, or otherwise only two nonmaterials are required to be used in appearance checking mode. But we cannot bind the total number of nonterminals.

10.15

MARKOV ALGORITHM

The Markov algorithm can be seen as a string rewriting system that uses grammar-like rules to be operated on strings of symbols. Markov algorithms have been shown to have adequate power to be a general model of computation, and can thus be shown to be equivalent in power to a Turing machine. Since it is Turing-complete, Markov chains can represent any mathematical expression from its simple notation.

The Markov Algorithm Machine Markov Algorithm (*MA* in short, also called Normal Algorithm) stands as an associative model of computation based on pattern matching and substitution. The model is equivalent to all the other models of computation (such as Turing machines and λ -calculus) that are at the basis of various classes of programming languages. The class of languages circumscribed by the *MAs* addresses mainly rule-based languages useful for artificial intelligence applications, in particular for expert systems development. However, these languages can be seen as general purpose, offering a mix of declarative and imperative programming flavour. They are equipped with interesting off-track data representation and control features that allow for direct coding for high abstraction solving strategies. The solution of a problem is much on the side of the problem rather than being distorted due to the programing constructs the programming language offers. The Markov Algorithmic Machine (*MAM*) consists of:

- a data register, called (*DR*)
- a control unit (*CU*), and
- an algorithm store (*AS*)

Data A *MAM* works with data represented as strings. The data register *DR* stores a string, called *R*, from the set $\{A_B \cup A_D\}^+ \cup \{\wedge\}$, where

$$\begin{aligned} A_B \cap A_L &= \emptyset \\ A_B &= \text{the base symbols alphabet;} \\ A_L &= \text{the local (working) symbols alphabet;} \\ \wedge &= \text{the empty string.} \end{aligned}$$

The sets A_B and A_L cannot contain reserved symbols that are used to build an algorithm. The data register has an unlimited capacity. It extends to the right as much as necessary. For the correct execution of the algorithm stored in *AS*, the initial string in *DR* (before the algorithm starts) and the final string in

DR (after the algorithm terminates) must contain only symbols from A_B . Symbols from A_L can appear in DR during the execution of the algorithm only.

Rules A Markov algorithm (MA) is built from rules of the form:

identification_pattern $\rightarrow$ substitution_pattern[.]

identification_pattern $\rightarrow$ symbol*

substitution_pattern $\rightarrow$ symbol*

symbol $\rightarrow$ constant | generic_variable | local_variable

where, constant is a symbol from A_B and local_variable is a symbol from A_L .

A generic_variable is a conventional symbol that designates a generic symbol from a subset of A_B . By convention, generic variables are noted by the letter g , possibly decorated by subscript and/or superscript indexes. The set of all legitimate values a generic variable g can be bound to is called the domain of the variable and is denoted by $Dom(g)$. The following restrictions apply:

- During the execution of an MA , a generic variable from a rule can be bound to a unique symbol from its domain while the rule is applied. The scope of a generic variable spans the algorithm within which it appears.
- A rule is correct if any generic variable from its R.H.S. occurs also in the L.H.S. of a rule.

Note A rule can be textually terminated by a dot. Such a rule is a terminal rule. If applied it stops the MAM .

Algorithms Mainly, a Markov Algorithm can be seen as an ordered set of rules which are known as the body of the algorithm, to be enhanced with declarations such that:

- structure A_B into subsets, and
- specify the domains of the generic variables used in the body of the algorithm.

But by convention an algorithm is described as follows:

Algorithm name base_alphabet_declaration;
 [generic_var_declaration;]*
 [label : rule;]*
 end [name]
 base_alphabet_declaration $\rightarrow$ ([set [, set] *])
 set $\rightarrow$ subset_of A_B | (set) | set set_constructor set
 subset_of A_B $\rightarrow$ subset_name | {constant [, constant] *}
 set_constructor $\rightarrow$ $\cup$ | $\cap$ | $\setminus$
 generic_var_declaration $\rightarrow$ set generic_var [, generic_var] *
 label $\rightarrow$ natural_number

Rules are numbered according to their position in the algorithm. We assume that the first rule has the label 1, whereas i^{th} rule has the label i .

By convention, a symbol that occurs in a rule and that is not declared as a constant from A_B is considered a local variable. A local variable is similar to a literal in a conventional programming language.

The syntax of an MA is of little importance as far as the specification makes it clear which are the domains of generic variables and which is the meaning of the symbols used in the rules of the algorithm

(constants from A_B , local variables, generic variables). As an example, the algorithm `set_difference` removes from DR all symbols that are in the set B .

The Markov algorithm assigns the following priorities to the grammar rules (production system):

- (i) The production rules are assigned with numbers of their respective orders.
- (ii) There is a production to indicate the termination of the process. Here, the production is of the form

$$\alpha \times \beta \rightarrow [.]$$

- (iii) When more than one productions are applied at a time, the decision is made on the basis of the index assigned to the productions. The production of a smaller index is applied first.
- (iv) When a string ' ω ' has many occurrences of X , and there is a production of the form $\alpha \times \beta \rightarrow [.]$, then algorithm decides to which X the production will be applied.

Summary

- **Decidable or Solvable Problems** The Problems for which there exist mathematical solutions or the problems that can be solved by a Turing machine are called decidable or solvable problems.
- **Intractable Problems** The intractable problems are those for which no effective algorithm exists.
- **Halting Problem of TM** The halting problem of Turing machine is undecidable. For a given input, we cannot say whether a particular Turing machine M will halt or not.
- **Post Correspondence Problem** For two lists $\alpha = (\alpha_1, \alpha_2, \alpha_3, \dots, \alpha_n)$, and $\beta = (\beta_1, \beta_2, \beta_3, \dots, \beta_n)$ of non-empty strings over the alphabet $\Sigma = \{0, 1\}$, the *PCP* determines whether or not there exist $i_1, i_2, i_3, \dots, i_m$, where $1 \leq i_j \leq n$, such that

$$\alpha_{i_1} \alpha_{i_2} \alpha_{i_3} \alpha_{i_4} \dots \alpha_{i_m} = \beta_{i_1} \beta_{i_2} \beta_{i_3} \beta_{i_4} \dots \beta_{i_m}$$

- **MPCP** If the first substring used in *PCP* is always α_1 and β_1 then *PCP* is called a modified Post's correspondence problem (*MPCP*).
- **Context Sensitive Languages** The languages that cannot be accepted by a push down automaton are called context sensitive languages.
- **Computable Function** A function f in a particular domain will be called a computable function if there exists a Turing machine that computes the value of function f for all arguments in its domain.
- **Partial Function** A partial function $f: X \rightarrow Y$ (read as f from X to Y) is defined as a rule which assigns at most one element of Y to every element of X . In other words a partial function f may be undefined at certain points (the points not in the domain of f).
- **Total Function** A total function $f: X \rightarrow Y$ is defined as a rule which assigns a unique element of Y to every element of X . In other words if a function f may be defined at all points in the domain of f it will be called a total function.
- **Recursive Functions** Recursive functions are a class of functions from natural numbers to natural numbers which are computable in some intuitive sense. In the computability theory, the recursive functions are considered as those functions that can be computed by Turing machines.
- **Zero Function Z** It is defined as $Z(x) = 0$, for all $x \in N$.

- **Successor Function S** It is defined as $S(x) = x + 1$.
- **Projection Function $\cup_i^n$** It is defined as $\cup_i^n (x_1, x_2, x_3, \dots, x_n) = x_i$, for $1 \leq i \leq n$.
- **Nil Function** It is Defined as $nil(\Sigma^*) = \wedge$, where $\wedge$ is a string of length zero. For example, $nil(babb) = \wedge$, for $babb \in \Sigma^*$.
- **Cons Function** This function concatenates two strings, and is defined as $cons(x_1, x_2) = x_1x_2$ for all $x_1, x_2 \in \Sigma^*$.
- **Composition Function** If $f_1, f_2, f_3, f_4, \dots, f_m$ are partial functions of n variables and f_p is a partial function of l variables, then the composition function is also a partial function of n variable. The composition (of f_p with f_1, f_2) function is given by
$$f_p(f_1(x_1, x_2, x_3, x_4, x_5, \dots, x_n), f_2(x_1, x_2, x_3, x_4, x_5, \dots, x_n), \dots, f_m(x_1, x_2, x_3, x_4, x_5, \dots, x_n))$$
over $X = \{x_1, x_2, x_3, x_4, x_5, \dots, x_n\}$.
- **Reducing one Undecidable Problem to Another** If L_1 and L_2 are two languages over the alphabets Σ_1 and Σ_2 , respectively, then we say L_1 is reducible to L_2 (denoted by $L_1 \leq L_2$) if there is a Turing-computable function $f: \Sigma_1^* \rightarrow \Sigma_2^*$ so that for any x in Σ_1^* , $x \in L_1$ if and only if $f(x) \in L_2$.
- **Rice's Theorem** Rice's theorem states that for any non-trivial property of partial functions, the question of whether a given algorithm computes a partial function with this property is undecidable.

Review Questions

- 10.1 Prove that if L_1 and L_2 are languages over alphabets Σ_1 and Σ_2 , respectively, and $L_1 \leq L_2$, then if L_2 is recursive, L_1 is also recursive.
- 10.2 Show that, for arbitrary context-free grammar G_1 and G_2 , the problem " $L(G_1) \cap L(G_2)$ is context-free" is undecidable.
- 10.3 Prove that for $|\Sigma| = 1$, that the post correspondence problem is decidable.
- 10.4 Give the solution of giving post correspondence problems:
 - (i) $A = \{aab, aabb, bb, bab\}$ and $B = \{ab, bbb, bbb, aba\}$
 - (ii) $A = \{ab, a, b\}$ and $B = \{abb, ba, bb\}$
 - (iii) $A = \{bb, baa, bbb\}$ and $B = \{bbb, aab, bb\}$
 - (iv) $A = \{a, bba, aab\}$ and $B = \{ba, aaa, ba\}$
- 10.5 Show that concatenation function (denoted by *concat*) is a primitive recursive function.
- 10.6 Define a function for subtraction.
- 10.7 Evaluate Ackermann's function for $A(2, 3)$ and $A(3, 2)$.
- 10.8 Determine Ackermann's function for $A(10, 8)$
- 10.9 Show that the max function defined as

$$\max(x, y) = \begin{cases} 1 & \text{if } x > y \\ 0 & \text{if } x \leq y \end{cases}$$

is a primitive recursive.

- 10.10 Show that the characteristic function of set of all odd integers is recursive.

Hints

10.3 For a single-letter alphabet, there is a solution of *PCP* if and only if there is some subset I of $\{1, 2, 3, \dots, n\}$ such that

$$\sum_{i \in I} |w_i| = \sum_{i \in I} |v_i|$$

Since there exists only a finite number of subsets, they can all be checked and therefore we can say that the problem is decidable.

10.4 (i) The problem has a solution for the sequence given by $x_3x_4x_1 = y_3y_4y_1$.

(ii) The problem has no solution because for each substring $x_i \in A$ and $y_i \in B$, we have

$$|x_i| < |y_i| \text{ for all } i.$$

(iii) The problem has a solution for the sequence given by $x_1x_3 = y_1y_3$.

(iv) The problem has no solution because no pair has a common non-empty initial substring.

10.5 The concatenation function can be defined as $\text{concat}(x, y) = xy$. It can be represented by composition and recursion to the initial functions.

10.6 First we define a predecessor function as:

$$\begin{aligned} \text{pred}(0) &= 0, \\ \text{pred}(y + 1) &= y. \end{aligned}$$

by using this definition we define the subtraction function as:

$$\begin{aligned} \text{subtr}(x, 0) &= x, \\ \text{subtr}(x, y + 1) &= \text{pred}(\text{subtr}(x, y)). \end{aligned}$$

10.9 The function *max* can be defined in terms of subtraction function *subtr*, as we have defined in Exercise 10.6, $\text{max}(x, y) = \text{subtr}(1, \text{subtr}(1, \text{subtr}(x, y)))$.

Graded Exercises

- ***10.1** If there is a reduction A to B , then prove that if A is undecidable then so is B .
- ****10.2** Prove that the Post Correspondence Problem is undecidable.
- ***10.3** If L is a Turing-decidable language then prove that its complement L' is also Turing-decidable.
- ***10.4** Prove that the Post Correspondence Problem with sets $A = (01, 10, 11, 00)$ and $B = (10, 00, 10, 11)$ is unsolvable.
- ***10.5** Show that there exists no context free grammar for language $L = \{a^n b^n c^n \mid n \geq 0\}$.
- ****10.6** Prove that the diagonalisation language L_d and universal language L_u are not recursively enumerable.
- ****10.7** Prove the following for Ackermann function: $A(3, x) = 2^{x+3} - 3$.
- ****10.8** If a language L and its complement L , both are recursively enumerable, then show that L is recursive language.

* Difficulty level 1

** Difficulty level 2

*** Difficulty level 3

Objective Questions

1. A problem is said to be an unsolvable decision problem if
 - (a) it is not possible to build a Turing machine that will halt in a finite amount of time, producing a 'Yes' or 'No' answer
 - (b) there is no algorithm that takes an input as an instance of the problem and determines whether the answer to that instance is 'Yes' or 'No'

- (c) both (a) and (b)
(d) none of the above
2. Which of the following is not undecidable
(a) Whether a Turing machine halts on all inputs?
(b) For a given $x \in \{a, b\}^*$, is x an element of language of palindromes over $\{a, b\}$?
(c) Given a context-free grammar G , is $L(G)$ empty?
(d) none of the above
3. Which of the following statements is false
(a) A problem whose language is recursive is said to be decidable.
(b) The theory of undecidability is concerned with the existence or non-existence of algorithms for solving problems with infinity instances.
(c) The complement of a recursive language is never recursive.
(d) all of the above.
4. A post correspondence problem is solvable if
(a) $|\Sigma| = 1$ (b) $|\Sigma| = 2$ (c) $|\Sigma| = \infty$ (d) none of these
5. Which of the following instances of the post correspondence problem have a viable sequences
(a) $\{(a, aa), (aa, aba), (aba, baa), (baa, abaa)\}$ (b) $\{(ab, abb), (ba, aaa), (aa, a)\}$
(c) $\{(ba, bab), (abb, bb), (bab, abb)\}$ (d) all of the above.
6. Which of the following statements is false
(a) A PCP with two lists $x = (1, 10111, 10)$ and $y = (111, 10, 0)$ has a solution
(b) A PCP can be treated as a game of dominoes.
(c) The PCP over Σ for $|\Sigma| \geq 2$ is unsolvable.
(d) none of the above.
7. The value of Ackermann's function $A(2, 2)$ is
(a) 5 (b) 6 (c) 7 (d) 8.
8. Which of the following Arckemann's function is defined correct
(a) $A(1, x) = x + 2$ (b) $A(2, x) = 2x + 2$ (c) $A(1, x) = x + 3$ (d) $A(2, x) = 3x + 1$.
9. Which of the following statements is false
(a) A total recursive function is also a partial recursive function.
(b) The function $f(x, y) = y^x$ is not a primitive recursive function.
(c) The function $f(x) = x/2$ is a partial function over N .
(d) none of above
10. Which of the following function(s) is (are) primitive recursive
(a) $f(x) = x^2$ (b) $f(x, y) = \max(x, y)$ (c) $f(x, y) = \begin{cases} 1 & \text{if } x > y \\ 0 & \text{if } x \leq y \end{cases}$ (d) All of these.
11. Which of the following property of recursively enumerable sets is not decidable
(a) Emptiness (b) Finiteness (c) Regularity (d) all of those
12. Which of the following is a partial recursive function
(a) $f(x) = x/2$ (b) $f(x, y) = x - y$
(c) $f(x) = \text{integral part of } \sqrt{x}$ (d) all of these

13. If f is a total function N to n and $f = O(p)$ for some polynomial function p , then there are constant A and B such that for every n
 - (a) $f(n) \geq A p(n) + B$
 - (b) $f(n) \leq A p(n) + B$
 - (c) $f(n) \geq B p(n) + A$
 - (d) none of these
14. If a language L can be recognised by a multitape Turing machine with time complexity f , then L can be recognised by a one-tape machine with time complexity
 - (a) $O(f^3)$
 - (b) $O(f^2)$
 - (c) $O(f)$
 - (d) none of these.
15. Suppose L_1 and L_2 are languages over Σ_1 and Σ_2 , respectively, and $L_1 \leq_p L_2$ then
 - (a) $L'_2 \leq_p L'_1$
 - (b) $L_2 \leq_p L_1$
 - (c) $L'_1 \leq_p L'_2$
 - (d) both (b) and (c)
16. Computation complexity is concerned with
 - (a) time complexity
 - (b) space complexity
 - (c) the number of reversals in the direction of travel of the tape head on a single-tape Turing machine.
 - (d) all of the above.
17. Which of the following statements is false
 - (a) Set of recursively enumerable language is closed under union.
 - (b) If a language and its complement are both regular, then the language is recursive.
 - (c) Recursive languages are closed under complementation.
 - (d) none of the above
18. Recursively enumerable languages are closed under
 - (a) concatenation
 - (b) union
 - (c) intersection
 - (d) all of these.
19. Recursive languages are
 - (a) a proper super set of context-free languages.
 - (b) recognisable by Turing machines.
 - (c) both (a) and (b)
 - (d) none of the above.
20. If there exists a Turing machine M that accepts every word in languages L and either rejects or loops for every word that is not in L , it is said to be
 - (a) recursive
 - (b) partially-recursive
 - (c) recursively enumerable
 - (d) none of these
21. Which of the following problem is solvable?
 - (a) writing a universal Turing machine
 - (b) determination of an arbitrary Turing machine as a universal Turing machine.
 - (c) determination of universal Turing machine to halt on some input.
 - (d) all of the above
22. Which of the following functions is not primitive recursive
 - (a) $f: N \rightarrow N$ defined by $f(x) = \lfloor \log_2(x+1) \rfloor$
 - (b) $f: N \rightarrow N$ defined by $f(x) = \lfloor \sqrt{x} \rfloor$
 - (c) $f: N^2 \rightarrow N$ defined by $f(x, y) = \max(x, y)$
 - (d) none of these.
23. If a function $g: N^2 \rightarrow N$ is primitive recursive, then $f: N \rightarrow N$ is also primitive recursive if it is defined as
 - (a) $f(x) = \sum_{i=0}^x g(x, i)$
 - (b) $f(x) = x^2$
 - (c) $f(x) = \sum_{i=0}^x g(x, i)$
 - (d) None of these.
24. Which of the following statements is true
 - (a) All computable functions are μ -recursive
 - (b) All μ -recursive partial functions are computable

- (c) All initial functions are elements of the set of μ -recursive functions.
 (d) all of the above.
25. Which of the following numbers cannot be represented by the Gödel number of sequence?
 (a) 0 (b) 7 (c) 10 (d) none of these.
26. The number 12 is the Gödel number of the sequence:
 (a) (3, 4) (b) (4, 3) (c) (2, 1) (d) none of these
27. Which of the following statement is false
 (a) Recursive functions are Turing computable
 (b) Gödel numbering converts the operation of the Turing machine into numeric quantities.
 (c) Turing computable functions are partial recursive.
 (d) none of above

References for Extra Reading

1. Turing, A.M. *On computable numbers, with an application to the Entscheidungs problem*, Proc. London Math. Soc., 42, 1936.
2. Bucher, W., H.A. Maurer, K. and Culik, II & D. Wotschke, *Concise description of finite languages, Theoretical Computer Science*, 1981.
3. Christos Papadimitriou, *Computational Complexity*, Addison Wesley, 1993.
4. Church, Alonzo, *An Unsolvble Problem of Elementary Number Theory*, American Journal of Mathematics 1936.
5. Davis, Martin, Ron Sigal and Elaine J. Weyuker, *Computability, Complexity, and Languages and Logic: Fundamentals of Theoretical Computer Science*, San Diego: Academic Press, Harcourt, Brace & Company, 1994.
6. Post, E.L., *A variant of a recursively unsolvable problem*, Bull. Amer. Math. Soc., 1946.
7. Rogers Jr., H., *Theory of recursive functions and effective computability*, McGraw-Hill, 1967.
8. John C Martin. *Introduction to Languages and the Theory of Computation*. McGraw Hill. 2003.
9. John E. Hopcroft and Jeffrey D. Ullman, *Introduction to Automata Theory, Languages and Computation*, Addison-Wesley Publishing, 1997.
10. Ju. V. Matijasevic, *Enumerable sets are diophantine*, Dokl. Akad. Nauk SSSR 191 (1970), 279-282. English transi.: Soviet Math. Doklady 11 (1970), 354-358.
11. Kleene, C., and Stephen, *General Recursive Functions of Natural Numbers*, Presented to the American Mathematical Society, 1935.
12. Davis, M., *Computability and unsolvability*, McGraw-Hill, 1958.
13. McGinn, Colin. *Problems in Philosophy: the Limits of Inquiry*. Blackwell, USA, 1993.
14. Minsky, Marvin, *Computation: Finite and Infinite Machines*, First, Prentice-Hall, NJ, 1967.
15. Papadimitriou, C.H., *Computational Complexity*, Addison-Wesley, Reading, MA, 1994.
16. Rosser, J.B., *An Informal Exposition of Proofs of Godel's Theorem and Church's Theorem*, Journal of Symbolic Logic, 1939
17. Cook, S., *The Complexity of Theorem Proving Procedures*, ACM Symposium of Theory of Computing, 1971.

18. Turing Alan, M., *On Computable Numbers, With an Application to the Entscheidungs problem*. Proceedings of the London Mathematical Society series 2, 1936-37

Online Sources

1. www.springerlink.com/index/exx688n21703x586.pdf
2. <http://people.cis.ksu.edu/~stough/forlan/book-and-slides/slides-5.3-twoup.pdf>
3. www.cs.rpi.edu/~drinep/modcomp/class21.ppt
4. www.en.wikipedia.org/wiki/Undecidable_problem
5. www.earlham.edu/~peters/courses/logsys/recursive.htm
6. www.haskell.org/haskellwiki/Recursive_function_theory

11

NP-Completeness

❖ Chapter Objective ❖

In this chapter we will discuss the time complexity and related issues. We will start with the role of hardware in the execution time of a program. Then we will introduce the time complexity followed by growth rate of functions. We will discuss polynomial time and polynomial time reduction of a problem. The concept of deterministic and non-deterministic algorithm will be discussed to describe NP completeness. In this sequence we will differentiate the P and NP classes. The classes of problems will be described with the help of examples. Finally, we will discuss the Cook's theorem.

□ Introduction □

In the 1960's and 1970's the concept of designing a hierarchy of complexity classes for problems that are based on finite sets was developed. As we use computers with finite memory storage to resolve computational problems, it is very significant for any non-trivial algorithm designing. Most important complexity classes P (problems solvable in polynomial time) and NP (problems whose solution certificate cannot be verified in polynomial time) have an important role in this field.

No polynomial time algorithm has yet been discovered to solve an NP -complete problem, nor has anyone yet claimed to be able to prove that no polynomial time algorithm can exist for any one of them. This so-called $P \neq NP$ question has been one of the deepest research problems in theoretical computer science since it was first presented in 1971 by Stephen Cook.

It is obvious that all P problems are in NP , but is the inverse true? It is one of the most important theoretical problems today. The most fruitful result of this conception is that the complexity classes have so-called complete problems within themselves. A problem of a class is complete if anyone can solve any other problem of this class in polynomial time having a polynomial time algorithm for the first one. Hence, complete problems are hardest in their own classes and as they exist we can select any of them to advance solving techniques for the entire class. The concept of complete problems for such a class can be generalised to hard problems for the class by inclusion of all other problems, whose polynomial time algorithm can ensure polynomial time solvability for that class. Therefore, there are NP -complete and NP -hard problems.

Logicians should be quite pleased that satisfiability for the propositional calculus is *NP*-complete. It means they will still need to prove theorems since it seems unlikely that anyone will develop a computer program to do so. However, as computer experts need to see problems which are closer home. This can also be considered as more than a theoretical exercise because we can observe that any problem which is *NP*-complete is a candidate for approximation because no sub-exponential time bounded algorithms are known for such problems.

We will give a few examples of *NP*-complete problems, but the problems now known to be *NP*-complete number are in the thousands, and the list is growing constantly. The book by Garey and Johnson remains a very good reference for a general discussion of the topic and consists of a wide list of such problems. *NP*-completeness is still a somewhat mysterious property. Some decision problems are in *P*, and others that seem similar turn out to be *NP*-complete.

❖ Did you Know?

The Clay Mathematics Institute has announced that anyone who shows the ability to resolve any *NP*-Complete problem, will get a \$ 1 million prize.

11.1

TIME COMPLEXITY

The time taken by an algorithm or a procedure to solve a problem is known as the time complexity. However the time taken by the programming implementation of an algorithm depends not only on the complexity of the algorithm but also on the speed of the hardware. In other words, the time required by an algorithm for solving a problem (solvable) depends not only on the size of the problem's input and the number of operations that the algorithm uses, but also on the hardware and software used to execute the solution. For example, if the same algorithm with the same inputs is applied on a PC and a super computer to solve a problem, the time taken by the super computer is much less than the time taken by a PC, because a super computer executes instructions one million times faster than a personal computer.

The reason of a super computer being much faster than a PC is that the super computer applies the concept of multi processing. It is a multi processor system that incorporates parallel computing. For this purpose the parallel version of an algorithm is used. The parallel version of an algorithm takes less time to compute data as compared to normal serial algorithm for that purpose. Parallel computing models use MIMD algorithms.

The random access machine (RAM) is a model of a one-address computing device. It consists of a program, a memory, a read only input tape, and a write only output tape. The memory consists of an unbounded sequence of registers. Each register can hold a single integer value. One register is an accumulator where computations are performed. The worst case time complexity of a random access machine program is the function $f(n)$. It is the maximum time taken by the program to execute over all inputs of size n .

The sequential algorithm for multiplication of two matrices of order $l \times m$ and $m \times n$ has a complexity $\Theta(n^3)$ as the algorithm requires $l \times m \times n$ additions and the same number of multiplications. Thus the multiplying two $n \times n$ matrices using the sequential algorithm is clearly $\Theta(n^3)$.

When we talk about the parallel version of the matrix multiplication, the hypercube SIMD model with n^3 processors has complexity $\Theta(\log n)$ for multiplication of two matrices of order $n \times n$.

Irrespective of the size of a solvable problem and the solution used to solve it, the super computer solves the problem roughly a million times faster than a PC, if the same solution steps are used on both the machines to solve that problem.

11.2

GROWTH RATE OF FUNCTIONS

To evaluate the efficiency of an algorithm a real time unit such as micro second and neon second should not be used. Rather, logical units that express a relationship between the size n of a file or an array and the amount of time t required to process the data should be used. If there is a linear relationship between the size n and time t

$$t_1 = cn_1$$

then increase of data by a factor of 5 results in the increase of the execution time by the same factor, if $n_2 = 5n_1$ then $t_2 = 5t_1$, similarly, if $t_1 = \log_2 n$, then doubling of n increases the value of t by only 1 unit of time, therefore if $t_2 = \log_2(2n)$, then $t_2 = t_1 + 1$.

A function expressing the relationship between n and t is usually much more complex, and calculating such a function is important only in regard to large bodies. Any terms, which do not substantially change the function's magnitude, should be eliminated from the function. The resulting function gives only an approximate measure of efficiency of the original function. However, this approximation is sufficiently close to the original, especially for a function that processes large quantities of the data. This measure of efficiency is called asymptotic complexity and is used when calculating a function is difficult and impossible and only approximations are found. To illustrate the first case, let us consider

$$f(n) = n^2 + 100n + \log_{10} n + 1001$$

For small values of n , the last term, 1001 is the largest. When n equals 10, the second term (i.e., $100n$) and last 1001 terms are on equal footing with the other terms making the same contribution to the function value. When the value of n reaches 100 the first and second term make that same contribution to the result. But when n becomes larger than 100 the contribution of the second term becomes less significant. Hence for large values of n , due to quadratic growth of the first term (i.e., n^2), the value of function $f(n)$ depends mainly on the value of the first term.

11.2.1 Well-Known Asymptotic Growth Rate Notations

The five well known notations are given below:

- ❖ O – Pronounced as 'big-oh', for example $O(n^2)$ is big oh of n^2 .
- ❖ Ω – Pronounced as 'big omega', for example $\Omega(n^2)$ is omega of n^2 .
- ❖ Θ – Pronounced as 'theta', for example $\Theta(n^2)$ is theta of n^2 .
- ❖ o – Pronounced as 'little – oh', for example $o(n^2)$ is little oh of n^2 .
- ❖ ω – Pronounced as 'little-omega', for example $\omega(n^2)$ is little omega of n^2 .

The Notation O (big - oh)

It provides asymptotic upper bound for a given function. Consider $f(n)$ and $g(n)$ are two functions each from the set of natural numbers N or set of positive real numbers to positive real numbers R . Then $f(n)$ is said to be $O(g(n))$, if there exist two positive numbers constants C and k , such that

$f(n) \leq Cg(n)$
for all $n \geq k$.

Example 11.1

Consider a function defined as
 $f(n) = 2n^3 + 3n^2 + 1$

Show that,

- (i) $f(n) = O(n^3)$
- (ii) $f(n) = O(n^4)$
- (iii) $n^3 = O(f(n))$
- (iv) $n^4 \neq O(f(n))$.

Solution

(i) We have, $f(n) = 2n^3 + 3n^2 + 1$
 $\leq 2n^3 + 3n^3 + 1.n^3 = 6n^3$ for all $n \geq 1$

therefore, there exists $C = 6$ and $k = 1$ such that

$$f(n) \leq C.n^3 \quad \text{for all } n \geq k$$

$$= O(n^3)$$

(ii) On the basis of (i) above,

$$f(n) \leq 6n^4 \quad \text{for all } n \geq 1$$

Now we find C and k as

$$f(1) = 2 + 3 + 1 = 6 \quad (\text{from } f(n) = 2n^3 + 3n^2 + 1, \text{ for } n = 1)$$

$$f(2) = 2.2^3 + 3.2^2 + 1 = 29$$

$$f(3) = 2.3^3 + 3.3^2 + 1 = 82$$

When we consider $f(n) = n^4$, we get

$$f(1) = 1$$

$$f(2) = 16$$

$$f(3) = 81$$

for $C = 2$ and $k = 3$ we have

$$f(n) \leq 2n^4 \quad \text{for all } n \geq k$$

therefore,

$$f(n) = O(n^4)$$

(iii) For $C = 1$ and $k = 1$ we have

$$n^3 \leq C(2n^3 + 3n^2 + 1) \quad \text{for all } n \geq k$$

therefore,

$$n^3 = O(f(n))$$

(iv) We can prove this part by contradiction. Consider, there exist positive constants C and k such that

$$n^4 \leq C(2n^3 + 3n^2 + 1) \quad \text{for all } n \geq k$$

therefore,

$$n^4 \leq C(2n^3 + 3n^3 + n^3) = 6Cn^3 \quad \text{for all } n \geq k$$

thus,

$$n^4 \leq 6Cn^3 \quad \text{for all } n \geq k$$

it implies that $n \leq 6C$ for all $n \geq k$, but for $n = \text{Max}(6C + 1, k)$, the previous statement is not true. Hence, we get a contradiction, therefore, $n^4 \neq O(f(n))$.

The Ω Notation It provides an asymptotic lower bound for a given function. Let $f(n)$ and $g(n)$ be two functions, each from the set of natural numbers.

There exists $f(n) = \Omega(g(n))$, if there exist two positive number constants C and k such that

$$f(n) \geq C(g(n)) \quad \text{for all } n \geq k$$

Example 11.2

Consider the functions

$$f(n) = 2n^3 + 3n^2 + 1 \text{ and } h(n) = 2n^3 - 3n^2 + 2,$$

show that,

- (i) $f(n) = \Omega(n^3)$
- (ii) $h(n) = \Omega(n^3)$
- (iii) $h(n) = \Omega(n^2)$
- (iv) $n^3 = \Omega(h(n))$.

Solution

(i) For $C=1$, we have

$$f(n) \geq Cn^3 \quad \text{for all } n \geq 1$$

therefore,

$$f(n) = \Omega(n^3)$$

(ii) $h(n) = 2n^3 - 3n^2 + 2$

Suppose, C and $k > 0$ are two constants such that

$$2n^3 - 3n^2 + 2 \geq Cn^3 \quad \text{for all } n \geq k$$

Now, we have

$$(2 - C)n^3 - 3n^2 + 2 \geq 0 \quad \text{for all } n \geq k$$

for $C = 1$ and $k \geq 3$, we have $h(n) = \Omega(n^3)$.

(iii) Let the following be true:

$$2n^3 - 3n^2 + 2 = \Omega(n^2)$$

then there exist positive numbers C and k such that

$$2n^3 - 3n^2 + 2 \geq Cn^2 \quad \text{for all } n \geq k$$

which, gives

$$2n^3 - (3 + C)n^2 + 2 \geq 0$$

It can easily be seen that the lesser the value of constant C , the better the probability of the above inequality being true. Therefore, to begin with $C = 1$ and try to find a value of k such that

$$2n^3 - 4n^2 + 2 \geq 0$$

for $n \geq 2$, the above holds. Therefore, $k=2$ is such that

$$2n^3 - 4n^2 + 2 \geq 0 \quad \text{for all } n \geq k$$

this proves that, $h(n) = \Omega(n^2)$

(iv) Let us consider the following inequality is true

$$n^3 = \Omega(2n^3 - 3n^2 + 2)$$

therefore, let $C > 0$ and $k > 0$ be two constants such that

$$n^3 \geq C(2n^3 - 3n^2 + 2)$$

we see that the above inequality is true for $C = \frac{1}{2}$, $k=1$, Therefore,

$$n^3 = \Omega(h(n))$$

The Θ Notation (Theta) It provides simultaneously both, the asymptotic lower bound and asymptotic upper bound for a given function. Let us consider, $f(n)$ and $g(n)$ are two functions, each from the set of natural numbers or positive numbers (real and integer both) then

$$f(n) = \Theta(g(n))$$

If there exist positive constants C_1 , C_2 and k such that

$$C_2 g(n) \leq f(n) \leq C_1 g(n) \quad \text{for all } n \geq k$$

Note that for any two functions $f(n)$ and $g(n)$

$$f(n) = \Theta(g(n))$$

if and only if $f(n) = \Theta(g(n))$ and $f(n) = \Omega(g(n))$.

Example 11.3

Consider the function.

$$f(n) = 2n^3 + 3n^2 + 1$$

show that

- (i) $f(n) = \Theta(n^3)$
- (ii) $f(n) \neq \Theta(n^2)$.

Solution

(i) For $C_1 = 3$, $C_2 = 1$ and $k = 4$ we have,

$$C_2 n^3 \leq f(n) \leq C_1 n^3, \quad \text{for all } n \geq k$$

This shows that,

$$f(n) = \Theta(n^3)$$

(ii) We will show it by contradiction, such that no constant C_1 exists. Let,

$$2n^3 + 3n^2 + 1 \leq C_1 n^2 \quad \text{for all } n \geq k$$

then,

$$n^3 \leq C_1 n^2 \quad \text{for all } n \geq k$$

it means

$$n \leq C_1 \quad \text{for all } n \geq k$$

but for $n = \text{Max}(C_1 + 1, k)$, is not true.

The o Notation Let $f(n)$ and $g(n)$ be two functions, each from the set of natural numbers or positive (integer or real) numbers and $C > 0$ be any constant, then

$$f(n) = o(g(n))$$

If there exists natural number k satisfying

$$f(n) < Cg(n) \quad \text{for all } n \geq k \geq 1$$

The constant C does not depend on $f(n)$ and $g(n)$.

Example 11.4

Consider $f(n) = 2n^3 + 3n^2 + 1$, then show that

- (i) $f(n) = o(n^i)$ for any $i \geq 4$
- (ii) $f(n) \neq o(n^i)$ for $n \leq 3$

Solution

(i) Suppose, $C > 0$

$$2n^3 + 3n^2 + 1 < Cn^i$$

it means

$$2 + 3/n + 1/n^3 < Cn^{i-3}$$

For $i = 4$, the inequality $2 + 3/n + 1/n^3 < Cn^{i-3}$ becomes

$$2 + 3/n + 1/n^3 < Cn$$

If we take $k = \text{Max}(7/C, 1)$, we get

$$2n^3 + 3n^2 + 1 < Cn^4 \quad \text{for } n \geq k$$

In general,

$$n^i > n^4 \quad \text{for } i \geq 4$$

therefore,

$$2n^3 + 3n^2 + 1 < Cn^i \quad \text{for } i \geq 4 \text{ and for all } n \geq k$$

with $k = \text{Max}(7/C, 1)$.

(ii) We show this relation by contradiction. Let,

$$f(n) = o(n^i) \quad \text{for } i \leq 3$$

then there exist positive constants C and k such that

$$2n^3 + 3n^2 + 1 < Cn^i \quad \text{for all } n \geq k$$

by dividing the above inequality by n^3 we get,

$$2 + 3/n + 1/n^2 < Cn^{i-3} \quad \text{for } i \leq 3 \text{ and } n \geq k$$

As C is any constant, so we take $C = 1$, the above inequality is reduced to

$$n^{i-3} \leq 1 \quad \text{for } i \leq 3 \text{ and } n \geq k \geq 1$$

therefore,

$$2 + 3/n + 1/n^2 \leq 1 \quad \text{for } n \leq 3$$

But it cannot be true. This contradiction shows the required relation.

The ω Notation (little omega) Let $f(n)$ and $g(n)$ be two functions each from the set of natural numbers or set of positive (integer or real) numbers, and $C > 0$ be some constant, then

$$f(n) = \omega(g(n))$$

If there exists a positive integer k such that

$$f(n) > Cg(n) \quad \text{for all } n \geq k$$

Example 11.5

Consider function $f(n) = 2n^3 + 3n^2 + 1$ then show that $f(n) = \omega(n)$ and $f(n) = \omega(n^2)$.

Solution Let C be any positive constant such that

$$2n^3 + 3n^2 + 1 > Cn$$

to find out $k \geq 1$ to satisfy the condition of the bound ω , we divided the above inequality by n to get

$$2n^2 + 3n + 1/n > C$$

Let k be constant such that $k \geq C+1$, then, for all $n \geq k$

$$2n^2 + 3n + 1/n \geq 2n^2 + 3n > 2k^2 + 3k > 2C^2 + 3C > C$$

Therefore, $f(n) = \omega(n)$

Let us again consider, for any $C > 0$.

$$2n^3 + 3n^2 + 1 > Cn^2$$

then

$$2n + 3 + 1/n^2 > C$$

Now for $n \geq k$ we have

$$2n + 3 + 1/n^2 \geq 2n + 3 > 2k + 3 > 2C + 3 > C$$

Hence, $f(n) = \omega(n^2)$.

11.3

POLYNOMIAL TIME

We are now going to discuss *NP*-completeness by formalising the concept of polynomial time solvable problems. Such types of problems are generally called tractable but only in a philosophical sense, not mathematically.

A problem is regarded as a tractable problem if it requires time $\Theta(n^k)$ (pronounced as theta of n^k , which is order of n^k , where k is a finite number). It is reasonable to regard a problem as an intractable problem if it requires running time $\Theta(n^{100})$. In practice, there are very few problems that require time of the order of such a high-degree polynomial. The polynomial-time computable problems encountered in practice typically require much less time. Thus a problem solvable in time $\Theta(n^k)$, for a small k is said to be solvable in polynomial time.

For many reasonable models of computation, a problem that can be solved in polynomial time using some arbitrary model can be solved in polynomial time by another model. The class of polynomial-time solvable problems consists of typical closure properties, as polynomials are closed under addition, composition and multiplication operations.

In the complexity theory, polynomial time can be defined as computation time of a problem where the time, $O(n^k)$ is not greater than a polynomial function of problem size n . Mathematicians sometimes use the notion of ‘polynomial on the length of the input’ as a definition of a fast computation as opposed to super-polynomial time, which is anything slower than that. Exponential time (whose order is $O(e^k)$) is one example of a super-polynomial time.

The concept of the size of the problem is difficult to define precisely. In general, the size of a problem is measured in terms of the size of input. The size of the input of a problem can be expressed informally through examples. In case of multiplication of two matrices of size $n \times n$, the size of the problem may be taken as n^2 , i.e., the number of elements in each matrix to be multiplied.

Also, we may have an idea about growth rate and its importance in the comparative study of algorithms, which can be designed to solve problems.

Let us consider two algorithms to solve a problem, having time complexities defined as

$$T_1(n) = 1000n^2 \text{ and } T_2(n) = 5n^4$$

where, the size of the problem is assumed to be n .

As comparison, we have

$$T_1(n) \geq T_2(n) \text{ for } n \leq 14, \text{ and}$$

$$T_1(n) \leq T_2(n) \text{ for } n \geq 15$$

and, the ratio $T_2(n)/T_1(n)$ increases as fast as the value of n increases. Thus, the growth rate of $T_2(n)$ is more than the growth rate of $T_1(n)$.

Before proceeding forward, we define some important terms which are used later in this chapter again and again.

❖ Definitions

- ❖ **Polynomial time algorithm** A polynomial-time algorithm is an algorithm with integer inputs $a_1, a_2, a_3, \dots, a_n$ if it runs in time polynomial in $\log_2 a_1, \log_2 a_2, \log_2 a_3, \dots, \log_2 a_n$, that is polynomial in the length of its binary encoded inputs.
- ❖ **Polynomial time reducibility** If L_1 and L_2 are two languages, over Σ_1 and Σ_2 , respectively, then L_1 is said to be polynomial-time reducible to L_2 (represented as $L_1 \leq_p L_2$), if there is a function $f: \Sigma_1^* \rightarrow \Sigma_2^*$ such that for any string $x \in \Sigma_1^*$, $x \in L_1$ if and only if $f(x) \in L_2$ and f can be computed in polynomial time (i.e., there is a Turing machine that computes f with polynomial time complexity).
- ❖ **Polynomial-time computable function** A function $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ is called polynomial time computable if there exists a polynomial time algorithm A that produces $f(x)$ as output on any input $x \in \{0, 1\}^*$.
- ❖ **Polynomial related encodings** Two encodings e_1 and e_2 are said to be polynomially related if there exists two polynomial-time computable functions f_1 and f_2 such that for any $i \in I$, there exist $f_1(e_1(i)) = e_2(i)$ and $f_2(e_2(i)) = e_1(i)$.
- ❖ **Language accepted in polynomial time** A language L is said to be accepted in polynomial time by an algorithm A if it can be accepted by algorithm A and also, if there is a constant k such that for any string $x \in L$, the algorithm A accepts string x in time $O(n^k)$.
- ❖ **Language decidable in polynomial time** A language L is said to be decidable in polynomial time by an algorithm A if there exists a constant k such that for any string $x \in \{0, 1\}^*$, the algorithm correctly decides whether $x \in L$ in time $O(n^k)$.
- ❖ **Undecidable problems** These are also called unsolvable problems. A problem is intractable if possibly there exists no polynomial time algorithm for it. In other words, as the complexity of intractable problems increases, the time required to solve those increases at an exponential rate. According to another definition, a collection of intractable problems is the set of problems have been proven not to have polynomial algorithm.

11.4

POLYNOMIAL TIME REDUCTION

A polynomial time reduction is a reduction which is computable by a deterministic Turing machine in polynomial time. It is a many-one reduction, therefore it is also called a polynomial-time many-one reduction. Polynomial time reductions are very important and are widely used because of their power to perform many transformations between important problems. This concept of reducibility can be used in the standard definitions of several complete complexity classes, such as NP-complete. However, within the class of problems P , polynomial time reductions are inappropriate for problem solving because any problem in class P can be reduced in polynomial time to almost any other problem of that class.

If A and B are two yes/no problems, the problem A is reducible in problem B (represented as $A \leq B$), if A reduces to B in polynomial time.

The representation $L_1 \leq_p L_2$ means that there exists a polynomial time computable function f such that $x \in L_1$, if and only if $f(x) \in L_2$. If we had a procedure to solve L_2 in polynomial time, then we could use it to solve L_1 in polynomial time.

11.5

P AND NP CLASSES

We call a problem tractable (solvable or decidable) if the space (space complexity) and time (time complexity) required for implementing the algorithm are reasonable. A decision problem is tractable if there exists an algorithm to solve the problem and the time required can be expressed in terms of a polynomial say $p(n)$, where n is the length of input string.

The class P is defined as the set of all decision problems solvable by a deterministic algorithm in polynomial time. The class NP is defined as the set of all decision problems solvable by a nondeterministic algorithm in polynomial time.

Deterministic algorithms are treated as a special case of nondeterministic problems, so we can conclude that $P \subseteq NP$. The most famous unsolvable problem in computer theory is whether

$$P = NP \quad \text{or} \quad P \neq NP.$$

The class P consists of those problems that are solvable in polynomial time. Also there is an important class of problems called NPC (i.e., NP -complete). If a problem is in the class NP and is as hard as any problem in class NP we refer it as NP -complete.

A brief summary of P and NP classes of problems is given below:

P Problems (P : Polynomial) It is the set of all polynomial solvable problems. The key features are:

- (i) P is closed under addition, composition and multiplication.
- (ii) P is independent of particular formal models of computation or implementations.
- (iii) Any problem that does not belong to P is hard.
- (iv) A problem in P does not necessarily have an efficient algorithm.
- (v) P is subset of NP .

NP Problems (NP : Nondeterministic Polynomial) It is the set of all problems that can be solved if we always guess correctly what computation path we should follow. The key features are:

- (i) NP -problems deal with decision problems as opposed to optimisation problems.
- (ii) Answers of these problems can be verified in polynomial time.

11.6

NP-COMPLETENESS

The idea of *NP*-completeness was first introduced by Stephen Cook in 1971, and the first example of an *NP*-complete problem he came to know was the satisfiability problem. The satisfiability problem proved by S. Cook was *NP*-complete.

An *NP*-complete problem is one for which the corresponding language is *NP*-complete. A language L is *NP*-complete if $L \in NP$ and L is *NP*-hard.

The idea of *NP*-completeness has great significance in complexity theory. Many of the cost optimisation problems, particularly in business planning and management science, are *NP*-complete and currently solved using heuristic methods and other approximations.

NP-complete problems are the set of problem that are both *NP* and *NP*-Hard. *NP*-complete problems are equivalent in the sense that if one problem has an efficient algorithm (i.e., in class P), then all *NP*-complete problems have efficient algorithms. If any *NP*-complete problem can be solved in polynomial time, then every *NP*-complete problem has a polynomial time algorithm (or procedure). *NP*-complete problems are intractable because the range of *NP*-complete problems studied up to present time, nobody has discovered a polynomial time solution to any of them yet.

The importance of the class of *NP*-complete problems is that it includes several problems that are natural and have been examined seriously for efficient solutions. None of these problems is known to have a polynomial-time solution. If a problem has been proved to be *NP*-complete then the problem is hard.

Properties of the Class of NP-complete Problems

- (i) No polynomial time algorithm has been found for any *NP*-complete problem.
- (ii) It has not been established that polynomial-time algorithms for these problems are not in existence.
- (iii) If a polynomial-time algorithm is found for any *NP*-complete problems, then there will be polynomial time algorithms for all such problems.
- (iv) It can also be proved that if no polynomial time algorithm exists for any such problem, then definitely for any other such problem, it will not exist.

11.7

NP-HARD

The concept of *NP*-hardness plays an important role in the discussion about the relationship between the complexity classes namely P and *NP*. Most of the time, it is also used to define the complexity class *NP* complete which can be seen as the intersection of *NP* and *NP*-hard stands for non-polynomial. Although it is widely supposed that there are no polynomial-time algorithms for these problems, this has never been proved.

In an other sense, a problem H is said to be *NP*-hard if for every decision problem L in *NP* there is a prediction machine, that has a prediction for solving problem H and such a machine can solve the problem L in polynomial time.

The class of *NP*-hard can also be defined in terms of a set or property of computational search problems. A problem will be said to be *NP*-hard if solving it in polynomial time would make it possible

to solve all problems in class NP in polynomial time. If we are able to reduce an NP problem to an NP -hard problem and then it can be solved in polynomial time, then we could solve all NP problems.

The complexity class of decision problems are fundamentally harder than those that can be solved by a nondeterministic Turing machine in polynomial time. When a decision version of a combinational optimisation problem is proved to belong to the class of NP -complete problems, which includes well-known problems such as satisfiability, travelling salesman and their optimisation version are NP -hard.

The computational complexity theory says that NP -hard is the class of decision problems that contain all problems H such that for all decision problems $L \in NP$ and there is a polynomial time many-to-one reducibility to H . This class can be described as containing the decision problems that are at least as hard as any problem in class NP . This perception is supported by the fact that if we are able to find an algorithm A that solves one of these problems H in polynomial time then we will be able to get reduced form of this problem to H and then executing the algorithm A . Let us assume a language L to be NP -complete. Then

- (i) $L \in NP$
- (ii) $L' \in NP$ and $L' \leq L$ (L' is polynomially reduced to L). Here L' is complement of L .

NP -hard assumes that the language L satisfies property (ii), but not necessarily property (i). Therefore, NP -hard problems are the set of all problems L such that any NP problems can be reduced to L in polynomial time. A problem P_1 is polynomial time reducible to P_2 (denoted by $P_1 \leq P_2$), if

- (i) There exists a one-to-one mapping from input I_1 of P_1 to input I_2 of P_2 .
- (ii) There exists $P_1(I_1)$ if and only if $P_2(I_2)$.

An example of an NP -hard problem can be seen as the following decision problem – namely called *subset-sum problem*, which can be described as : A set of integer values is given and there is a question, ‘Does any non-empty subset of them add up to zero?’

This is a Yes/No question, and seems to be NP -complete. There are decision problems which are NP -hard but not NP -complete and as an example we have the halting problem of Turing machine. It is very easy to show that the halting problem of a Turing machine is NP -hard but not NP -complete.

NP-hard and NP-complete Languages

A language L is called NP -hard language if $L_1 \leq_p L$ for every $L_1 \in NP$, where the notation $\leq_p$ denotes the polynomial-time reducibility relation. A language L is said to be NP -complete if $L \in NP$ and L is NP -hard. If L_1 and L_2 are two languages for which L_1 is NP -hard and $L_1 \leq_p L_2$, then L_2 is also NP -hard. If L is any NP -complete language, then there is a computable function $f \in P$ if and only if $P = NP$.

Corollary 11.1 If L is any NP -hard language then L can not be in class P unless we have $P = NP$.

Proof Assume $L \in P$, and assume L' is any arbitrary language in NP . Since the language L is NP -hard, the language L' must be polynomial time reducible to L . therefore $L' \in P$. but L' was chosen arbitrary from NP , and hence $P = NP$.

Theorem 11.1 If L_1 and L_2 be two any languages. If L_1 is NP -hard and if $L_1 \leq_p L_2$ then L_2 is NP -hard.

Proof Assume L is any language in NP . Since the language L_1 is NP -hard, therefore $L \leq_p L_1$. Now the relation $\leq_p$ is transitive relation, so $L \leq_p L_2$. Thus every language in L in NP is polynomial time reducible to L_2 . Hence L_2 is NP -hard.

11.8

COOK'S THEOREM

The statement that the satisfiability problem is NP -complete is known as Cook's theorem, proved by Stephen Cook.

The Satisfiability problem (denoted as SAT) is defined as: for a given Boolean formula, is there some way to assign truth value (TRUE or FALSE) to the variables of the formula, so that the formula evaluates to TRUE? For a given Boolean formula we say it is satisfiable if there exists a way to assign truth value to the variables such that the final result is TRUE.

Theorem 11.2 SAT is NP -complete.

Proof For each problem $A \in NP$, we have $A \leq_M^P SAT$ where the relation $A \leq_M^P SAT$ represents that A is polynomial time many-one reducible to SAT. Let $A = L(M)$ for a one tape nondeterministic Turing machine $TM M = (Q, \Sigma, \delta, \Gamma, q_0, \#, H)$ with time bound $p(n)$ for some polynomial p . Now we need to find Boolean formula F_x for each input string ' x ' such that $x \in A$ if and only if F_x is satisfiable. For this purpose we need to take a string ' x ' of length n . Since the Turing machine M halts on input x in $p(n)$ moves with at most $p(n) + 1$ configuration of computation of $M(x)$. Let us assume there are exactly $p(n)+1$ configurations and we allow $\alpha \vdash \alpha$ for α being the halting configuration. Next we assume that each configuration is of length $p(n)$ for each symbol ' a ' either in Γ or in $Q \times \Gamma$.

We define a Boolean variable $y_{i,j,a}$ for each triple (i, j, a) with $0 \leq i \leq p(n)$, $1 \leq j \leq p(n)$ and $a \in \Gamma \cup (Q \times \Gamma)$ to denote the property that j^{th} square of the i^{th} configuration of computation of $M(x)$ has the symbol ' a '.

Now we construct a Boolean formula F_x of variables $y_{i,j,a}$ such that F_x is satisfiable if and only if following conditions hold under the consideration of above interpretation of variable $y_{i,j,a}$:

1. For each pair (i, j) where $0 \leq i \leq p(n)$, and $1 \leq j \leq p(n)$ there exists $a \in \Gamma'$ such that $y_{i,j,a} = 1$.
2. For each pair (j, a) where $1 \leq j \leq p(n)$ and $a \in \Gamma'$ there exists $y_{0,j,a} = 1$ if and only if j^{th} symbol in the initial configuration of $M(x)$ is ' a '.
3. There exists $y_{p(n),j,a} = 1$ for some j where $1 \leq j \leq p(n)$ and some $a \in F \times \Gamma$.
4. The $(i+1)^{st}$ configuration is one of the next configuration of the i^{th} configuration for each i such that $0 \leq i \leq p(n)$, with this interpretation of variable $y_{i,j,a}$, it is clear that the above four conditions are equivalent to condition that the configuration depend by the variables $y_{i,j,a}$ is halting configuration of a Turing machine $M(x)$.

The first three conditions can be translated in to Boolean formula as follows:

$$F_1 = \prod_{i=0}^{p(n)} \prod_{j=0}^{p(n)} \left(\sum_{a \in \Gamma'} y_{i,j,a} \right) \prod_{a,b \in \Gamma', a=b} (y_{0,j,a} \vee y_{0,j,b})$$

$$F_2 = y_{0,1,(q_0,x_1)} \prod_{j=1}^n (y_{0,j,x_j}) \prod_{j=n+1}^{p(n)} (y_{0,j,B})$$

$$F_3 = \sum_{j=1}^{p(n)} \sum_{a \in F \times \Gamma} y_{p(n),j,a}$$

where $1 \leq j \leq n$ and x_j is the j^{th} symbol of x .

As fourth condition, we see a simple relation between consecutive configurations. Let a_k be the k^{th} symbol of α for each configuration α , and each k , $1 \leq k \leq p(n)$. Now we assume that the i^{th} configuration of Turing machine $M(x)$ is α and $(i+1)^{\text{st}}$ configuration is β .

Now we can verify that this fact is locally on the symbols $\alpha_{k-1}, \alpha_k, \alpha_{k+1}, \beta_k, \beta_{k+1}$, where $1 \leq k \leq p(n)$. More specifically the following conditions are equivalent to :

1. If $\alpha_{k-1}, \alpha_k, \alpha_{k+1} \notin Q \times \Gamma$ then $\beta_k = a_k$
2. If $a_k \in Q \times \Gamma$ then $a_{k-1} a_k a_{k+1} \xrightarrow[M]{*} \beta_{k-1} \beta_k \beta_{k+1}$, where $\xrightarrow[M]{*}$ is sub-relation of $\vdash_M$, which is restricted to configuration of length 3.

For translation condition (2) into a Boolean formula we define for each pair $(q, a) \in Q \times \Gamma$ and i, j , $0 \leq i \leq p(n)$, and $1 \leq j \leq p(n)$, a Boolean formula $F_{q,a,i,j}$ that can encode the relation, with $\alpha_k = (q, a)$. As an instance, if (q, a, b, R) and (q, a, r', b', L) are the only two sets that beginning with (q, a) then

$$F_{q,a,i,j} = \prod_{c,d \in \Gamma} y'_{i,j-1,c} + y'_{i,j,(q,a)} + y'_{i,j+1,d} + y_{i+1,j-1,c} y_{i+1,j+1,(r,d)} + y_{i+1,j-1,(r',c)} y_{i+1,j,b'} y_{i+1,j+1,d}$$

Now we can define F_y as follows:

$$F_y = \prod_{c,d \in \Gamma} \prod_{j=1}^{p(n)} (y'_{i,j-1,a} + y'_{i,j,b} + y'_{i,j+1,c} + y'_{i+1,j,b} \prod_{(q,a) \in Q \times \Gamma} F_{q,a,i,j})$$

Now we define $F_x = F_1 F_2 F_3 F_4$. It is clear to state that F_x is satisfiable if and only if M accepts x . Now we verify that F_x is of polynomial of length $|x|$ and it can be uniformly generated from x . Let us assume

$$|\Gamma'| = m, \text{ and } |\delta| = r.$$

Now we can see that F_1 is of length $O(m^2(p(n))^2)$, F_2 is of length $O(p(n))$, F_3 is of length $O(m p(n))$, and F_4 is of length $O(r m^3(p(n))^2)$. Thus the formula F_x is of length $O(r m^3(p(n))^2)$. Additionally these formulas except F_2 , do not depend on input x , and F_2 can easily be derived from x . Therefore F_x is polynomial time computable from x .

Lemma 11.1 If $L_1, L_2 \subseteq \{0, 1\}^*$ are languages such that $L_1 \leq_p L_2$ then $L_2 \in P$ implies $L_1 \in P$.

Proof Suppose A_2 is a polynomial time algorithm that decides L_2 , and F is a polynomial time reduction algorithm that computes the reduction function f .

Let us construct a polynomial-time algorithm A_1 that decides L_1 . Figure 11.1 illustrates the construction of A_1 . For a given input $x \in \{0, 1\}^*$, the algorithm A_1 uses polynomial time reduction algorithm F to transform x into $f(x)$, and then it uses A_2 to test whether $f(x) \in L_2$. The output of A_2 is the values produced as the output from A_1 .

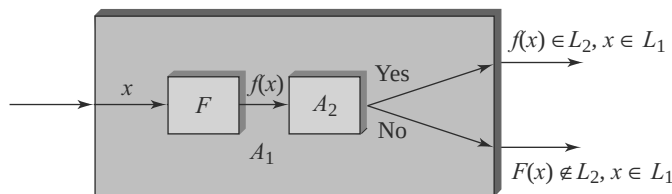


Fig. 11.1 Illustration of algorithm A_1

The algorithm runs in polynomial time, because both F and A_2 run in polynomial time.

Theorem 11.3 If any NP -complete problem is polynomial time solvable, then $P = NP$ (i.e., if any problem in NP is not polynomial time solvable, then no NP -complete problem is polynomial time solvable).

Proof Let L be in P and also $L \in NPC$ (NP -Completeness). For any $L' \in NP$, we have $L' \leq_p L$ by the definition of NP -Completeness. By Lemma 11.1, we also have $L' \in P$. This proves that if any NP -complete problem is polynomial time solvable, then $P = NP$.

The alternative part of the theorem is the contrapositive of the statement what we have proved.

❖ Did you Know ?

Stephen Cook introduced the **theorem of NP-completeness** through his famous 1971 paper entitled 'The Complexity of Theorem Proving Procedures'. This way the idea of NP -completeness was first introduced and as a satisfiability problem, the first example of an NP -complete problem came to known before the linguistics. The satisfiability problem proved by S. Cook was NP -complete.

Stephen Cook is the 1982 recipient of the Turing award. He was awarded a Steacie fellowship in 1977 and a Killam Research fellowship as well in 1982. He received computer science teaching awards in 1989 and 1995, respectively.

11.9

SOME NP -COMPLETE PROBLEMS

There are several NP -complete problems in various fields like graph theory, propositional calculus, operations research, etc. There are hundreds of such problems so far. Some of them are stated below:

- (i) *The Vertex-Cover Problem* Given a graph, to find a vertex cover of minimum size is the vertex-cover problem. In other words, in this optimisation problem, as a decision problem we have to determine whether a graph has a vertex cover of given size k .
- (ii) *Satisfiability Problem* Given a formula involving propositional variables and logical connectives (e.g., AND, OR, NOT, etc.), 'Does there exists a choice of truth values for the variables for which the given formula assumes the truth value 1?'
- (iii) *The Hamiltonian Cycle Problem* Given an undirected graph $G = (V, E)$, a Hamiltonian cycle is a simple cycle that contains each vertex in V . A graph that contains a Hamiltonian cycle is said to be Hamiltonian. The Hamiltonian cycle problem is defined as, 'Does a graph G has a Hamiltonian cycle?'
- (iv) *The Travelling Salesman Problem.* Given n vertices (or say cities), the distance between them and a number T , Does there exist a tour program for a salesman to visit all the cities so that the total distance travelled is almost T ?
- (v) *Examination Scheduling Problem.* Given a list of courses, a list of conflicts between them, and an integer k ; 'Is there an exam schedule consisting of k dates such that there are no conflicts between courses which have examinations on the same date?'

- (vi) *The Colorability Problem* Given a planner undirected graph $G = (V, E)$ with node degree at most four, then ‘Is the graph G 3-colorable?’
- (vii) *Simple Optimal Linear Arrangement Problem* Given an undirected graph $G = (V, E)$ and an integer k . If there are n vertices in graph, then ‘Is there a one-to-one function $f: V \rightarrow \{1, 2, 3, \dots, n\}$.’
- (viii) *The Zero-one Programming Problem* Given m simultaneous equations,

$$\sum_{j=1}^m a_{ij}s_j = b_i, \text{ for } i = 1, 2, 3, 4, \dots, n$$

Does there exist values, zero or one for s_j such that the above equations are satisfied?

- (ix) *The Simple Max Cut Problem* Given a graph $G = (V, E)$ and an integer k , ‘Does V has a subset V_1 such that there are at least k edges $(u, v) \in E$ with $u \in V_1$ and $v \notin V_1$?’
- (x) *The Subset Sum Problem.* A finite set $S \subset N$ with a target $t \in N$ is given, the subset sum problem is defined as whether a subset $S' \subseteq S$.

Summary

- **Tractable Problem** A problem is regarded as a tractable problem if it requires time $\Theta(n^k)$ (pronounced as theta of n^k , which is order of n^k , where k is a finite number). A decision problem is tractable if there exists an algorithm to solve the problem and the time required can be expressed in terms of a polynomial say $p(n)$, where n is the length of input string.
- **Polynomial Time** Polynomial time can be defined as computation time of a problem where the time, $O(n^k)$ is not greater than a polynomial function of problem size n .
- **Class P** The class P is defined as the set of all decision problems solvable by a deterministic algorithm in polynomial time.
- **Class NP** The class NP is defined as the set of all decision problems solvable by a nondeterministic algorithm in polynomial time.
- **P Problems** It is the set of all polynomial solvable problems.
- **NP Problems** It is the set of all problems that can be solved if we always guess correctly what computation path we should follow.
- **NP -hard** A problem is NP -hard if solving it in polynomial time would make it possible to solve all problems in class NP in polynomial time. NP -hard refers to the class of decision problems that contain all problems H such that for all decision problems $L \in NP$ there is a polynomial time many-to-one reduction to H .
- **NP -hard Language** A language L is called NP -hard language if $L_1 \leq_p L$ for every $L_1 \in NP$, where the notation $\leq_p$ denotes the polynomial-time reducibility relation.
- **The Notation O** It provides asymptotic upper bound for a given function. Consider $f(n)$ and $g(n)$ are two functions each from the set of natural numbers or set of positive real numbers to positive real numbers.
- **The Ω Notation** It provides an asymptotic lower bound for a given function. Let $f(n)$ and $g(n)$ be two functions, each from the set of natural numbers.
- **The Θ Notation** It provides simultaneously both, the asymptotic lower bound and asymptotic upper bound for a given function.

- **The o Notation** Let $f(n)$ and $g(n)$ be two functions, each from the set of natural numbers or positive (integer or real) numbers and $C > 0$ be any constant, then $f(n) = o(g(n))$. If there exists natural number k satisfying $f(n) < Cg(n)$, for all $n \geq k \geq 1$.
- **Cook's Theorem** The statement that the satisfiability problem is NP -complete is known as Cook's theorem, proved by Stephen Cook

Review Questions

- 11.1 Give the difference between polynomial and exponential time.
- 11.2 What is polynomial time reduction?
- 11.3 If every instance of problem A is an instance of problem B , and if B is hard then A is hard. True or false? Explain.
- 11.4 Prove that the relation $\leq_p$ is transitive relation on languages.
- 11.5 Show that both P and NP are closed under the operation of concatenation and Kleene closure.
- 11.6 Show that the travelling salesman problem is NP -complete.
- 11.7 Determine $\Theta(n)$, and $o(n)$ for the following
 - (i) $f(n) = 16n^3 + 48n$
 - (ii) $f(n) = n^3 + n$
 - (iii) $f(n) = 12n + 8$
 - (iv) $f(n) = 8.12^3 + 4$
- 11.8 Consider the function $f(n) = 2n^3 + 3n^2 + 1$, show that $f(n) \neq \Theta(n)$

Graded Exercises

- * 11.1 For languages $A, B \subseteq \{0, 1\}^*$, suppose, $A \oplus B = A \setminus B \cup B \setminus A$ then show that $A \leq_p A \oplus B$ and $B \leq_p A \oplus B$.
- * 11.2 Explain why the fact that $L \in NP$ does not obviously imply that $L' \in NP$, the complement of L .
- * 11.3 Show that if there is an NP -complete language L whose complement L' is NP , then the complement of any language in NP is also in NP .
- ** 11.4 Show that the satisfiability of Boolean formulas in three-conjunctive normal form is NP -complete.
- ** 11.5 Let L_A be the language such that $L_A \leq_p L_B$ for some L_A to be NP -complete, then show that L_B is NP hard.
- *** 11.6 If $L_A, L_B \subseteq (a, b)^*$ are languages such that $L_A \leq_p L_B$ then show that $L_A \in P \Rightarrow L_B \in P$.
- ** 11.7 If there is an undirected bipartite graph G with odd number of vertices then prove that graph G is nonhamiltonian.
- *** 11.8 Show that a language $L \in NP$, can be decided by an algorithm in time $2^{O(n^k)}$ for some constant k .
- ** 11.9 Give an efficient algorithm to solve Hamiltonian path in polynomial time on directed acyclic graphs.
- *** 11.10 For two non negative functions $f(n)$ and $g(n)$, prove that there exists neither $f(n) = O(g(n))$ nor $g(n) = O(f(n))$.

* Difficulty level 1

** Difficulty level 2

*** Difficulty level 3

Objective Questions

1. A problem is *NP*-complete if
 - (a) solution can be verified quickly
 - (b) a quick algorithm to solve this problem can be used to solve all other *NP* problems quickly
 - (c) it can be solved in polynomial time
 - (d) all of the above
2. If a problem *A* is polynomial time reducible to *B* (i.e., $A \leq_p B$) and $B \in P$, then
 - (a) problem *A* is also in *P*
 - (b) problem *A* can not be in *P*
 - (c) problem *A* is not harder than *B*
 - (d) both (a) and (b)
3. If $A \leq_p B$ and *B* is in *NP*, then
 - (a) if *B* is recursive, *A* is also recursive
 - (b) *A* is in *NP*
 - (c) both (a) and (b)
 - (d) none of these.
4. Which of the following problems is not *NP*-complete
 - (a) satisfiability problem
 - (b) vertex cover problem
 - (c) integer linear programming problem
 - (d) none of these
5. If *A* is *NP*-complete then *A* is a member of *P*
 - (a) if and only if $P = NP$
 - (b) if and only if $P \neq NP$
 - (c) the statement (a) is always true but sometimes (b) is also true.
 - (d) none of the above
6. The set *A* is many-one polynomial time reducible to the set *B* if and only if
 - (a) there exists any recursive function
 - (b) there is a recursive function $g(x)$ which can be compute in polynomial time for all x such that $x \in A$ if and only if $g(x) \in B$
 - (c) there exists a recursive function $g(x)$ which can be computed in exponential time for all x
 - (d) none of the above
7. Which of the following statements is correct
 - (a) halting problem of Turing machine is *NP*-complete
 - (b) the complexity class *NP*-complete is the intersection of *NP* and *NP*-hard classes.
 - (c) there are also some decision problems which are *NP*-hard but not *NP*-complete.
 - (d) only (b) and (c).
8. Which of the following statements is incorrect
 - (a) some *NP*-hard problems are also in *NP*
 - (b) all *NP*-complete problems are *NP*-hard
 - (c) all *NP*-hard problems are *NP*-complete
 - (d) none of the above
9. All *NP* problems can be solved if
 - (a) an *NP* problem can be reduced to an *NP*-hard problem before solving it in polynomial time
 - (b) an *NP* problem can be reduced to an *NP*-hard problem solving it in exponential at time
 - (c) both (a) and (b)
 - (d) none of the above

10. Which of the following is not correct
 - (a) the class of decision problem that can be solved by a deterministic sequential machine in polynomial time is known as P
 - (b) the class of decision problem that can be verified in polynomial time is known as NP
 - (c) the class of problems that can be understood as the class NP -hard consequently is NP -complete or harder
 - (d) none of the above
11. A language L is NP -complete language
 - (a) if $L \in NP$
 - (b) then $P = NP$ if and only if $L \in P$
 - (c) then for every language $L_1 \in NP$ there is a polynomial time transformation from L to L_1
 - (d) all of the above
12. A problem is polynomial-time solvable, if
 - (a) there exists an algorithm to solve it in time $O(n^k)$ for some constant k
 - (b) there exists an algorithm to solve it in time $\Theta(2^n)$
 - (c) both (a) and (b)
 - (d) none of the above
13. The travelling salesman problem is

(a) NP but not NP -complete	(b) NP -complete
(c) neither NP nor NP -complete	(d) none of these
14. If A is NP -complete and there is a polynomial time reduction of A into B , then B is

(a) NP -complete	(b) Not necessary to be NP -complete
(c) cannot be NP -complete	(d) None of these
15. If there is a language L such that $L \in P$, then

(a) complement of L is also in P .	(b) complement of L is in NP
(c) both (a) and (b)	(d) none of these
16. A language L in NP does not obviously imply that

(a) complement of L is in NP	(b) complement of L is in P
(c) both (a) and (b)	(d) None of these
17. Which of the following statements is not correct
 - (a) If a language L is in P space then its complement is also in P
 - (b) If a language L is in NP then its complement L' is also in NP
 - (c) If a language L is in P then its complement L' is also in P .
 - (d) none of the above.
18. If there is an NP -complete language L whose complement is $L' \in NP$, then the complement of any language in NP is in

(a) P	(b) NP	(c) both (a) and (b)	(d) none of these
---------	----------	----------------------	-------------------
19. Consider the operation problem given below and state which option is correct for a sequence $a_1, a_2, a_3, \dots, a_n$ of integers, is there any subset j for $j = 1, 2, 3, \dots, n$ such that $\sum_{i \in j} a_i = \sum_{i \notin j} a_i$

- (a) given problem is NP -complete by constructing a reduction from the k -colorability problem
 - (b) given problem is NP -complete by constructing a reduction from the vertex cover problem
 - (c) given problem is NP -complete by constructing a reduction from the subset-sum problem
 - (d) none of the above
20. Both problems NP and P are closed under:
- (a) Kleene closure
 - (b) Concatenation
 - (c) Union
 - (c) Intersection

References for Extra Reading

1. Cormen, Leiserson, Rivest, and Stein, *Introduction to Algorithm*, PHI, New Delhi, 2000.
2. Horowitz, and Sahani, *Computer Algorithms*, Galgotia Publications Pvt. Ltd. 1999.
3. John E. Hopcroft, and Jeffrey D. Ullman, *Introduction to Automata Theory, Languages and Computation*, Addison-Wesley Publishing, 1997.
4. Garey, M. and D. Johnson, *Strong NP-Completeness Results: Motivation examples and Implications*, Bell Lab, NJ, 1976.
5. Garey, M. R. and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*.
6. Michael J. Quinn, *Parallel Computing: Theory and Practice*, 2nd edition, Tata McGraw Hill, 2002

Online Sources

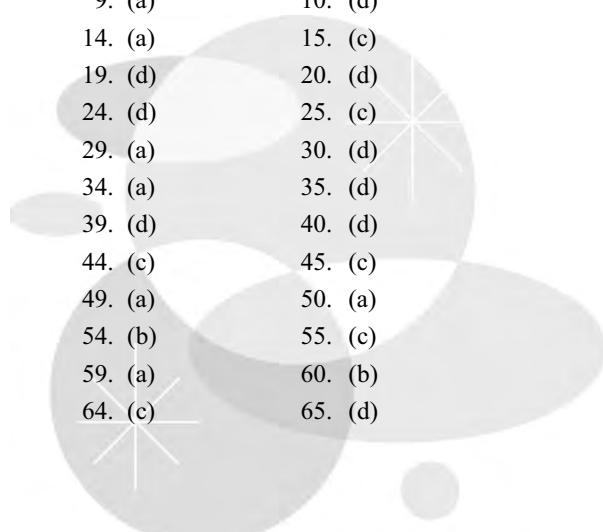
1. www.claymath.org/millennium/P_vs_NP/Official_Problem_Description.pdf
2. www.en.wikipedia.org/Polynomial-time-reduction
3. www.cs.cmu.edu/afs/cs/academic/class/15451-s06/www/lecture/08intractability.pdf



Appendix

Answers

- | | | | | | |
|-----------|---------|---------|---------|---------|---------|
| 1. | 1. (d) | 2. (d) | 3. (d) | 4. (c) | 5. (d) |
| | 6. (b) | 7. (b) | 8. (c) | 9. (b) | 10. (c) |
| 2. | 1. (b) | 2. (a) | 3. (c) | 4. (c) | 5. (b) |
| | 6. (b) | 7. (a) | 8. (b) | 9. (d) | 10. (a) |
| | 11. (a) | 12. (b) | 13. (c) | 14. (b) | 15. (d) |
| | 16. (d) | 17. (c) | 18. (d) | 19. (c) | 20. (d) |
| | 21. (d) | 22. (a) | 23. (c) | 24. (a) | 25. (d) |
| | 26. (a) | 27. (d) | 28. (a) | 29. (d) | 30. (d) |
| | 31. (a) | 32. (b) | 33. (d) | 34. (b) | |
| 3. | 1. (d) | 2. (d) | 3. (b) | 4. (c) | 5. (d) |
| | 6. (d) | 7. (c) | 8. (d) | 9. (a) | 10. (d) |
| | 11. (d) | 12. (d) | 13. (d) | 14. (a) | 15. (c) |
| | 16. (b) | 17. (d) | 18. (d) | 19. (d) | 20. (d) |
| | 21. (c) | 22. (d) | 23. (d) | 24. (d) | 25. (c) |
| | 26. (a) | 27. (b) | 28. (b) | 29. (a) | 30. (d) |
| | 31. (c) | 32. (a) | 33. (d) | 34. (a) | 35. (d) |
| | 36. (c) | 37. (b) | 38. (a) | 39. (d) | 40. (d) |
| | 41. (c) | 42. (a) | 43. (b) | 44. (c) | 45. (c) |
| | 46. (c) | 47. (b) | 48. (c) | 49. (a) | 50. (a) |
| | 51. (a) | 52. (a) | 53. (a) | 54. (b) | 55. (c) |
| | 56. (b) | 57. (c) | 58. (d) | 59. (a) | 60. (b) |
| | 61. (d) | 62. (c) | 63. (d) | 64. (c) | 65. (d) |
| | 66. (d) | 67. (d) | 68. (b) | | |



- 4.**
- | | | | | |
|---------|---------|---------|---------|---------|
| 1. (c) | 2. (b) | 3. (b) | 4. (d) | 5. (d) |
| 6. (a) | 7. (b) | 8. (c) | 9. (b) | 10. (b) |
| 11. (b) | 12. (a) | 13. (d) | 14. (b) | 15. (c) |
| 16. (b) | 17. (c) | 18. (b) | 19. (d) | 20. (d) |
| 21. (d) | 22. (a) | 23. (d) | 24. (d) | 25. (d) |
| 26. (d) | 27. (b) | 28. (c) | | |
- 5.**
- | | | | | |
|--------|--------|--------|--------|--------|
| 1. (a) | 2. (d) | 3. (b) | 4. (d) | 5. (d) |
|--------|--------|--------|--------|--------|
- 6.**
- | | | | | |
|---------|---------|---------|---------|---------|
| 1. (d) | 2. (d) | 3. (c) | 4. (d) | 5. (d) |
| 6. (c) | 7. (c) | 8. (b) | 9. (b) | 10. (a) |
| 11. (c) | 12. (b) | 13. (d) | 14. (d) | 15. (d) |
| 16. (a) | 17. (d) | 18. (c) | 19. (d) | 20. (b) |
| 21. (d) | 22. (a) | 23. (b) | 24. (c) | 25. (a) |
| 26. (c) | 27. (a) | 28. (b) | 29. (d) | 30. (c) |
| 31. (c) | 32. (d) | 33. (d) | | |
- 7.**
- | | | | | |
|---------|---------|---------|---------|---------|
| 1. (c) | 2. (b) | 3. (d) | 4. (d) | 5. (b) |
| 6. (d) | 7. (d) | 8. (a) | 9. (b) | 10. (d) |
| 11. (b) | 12. (b) | 13. (a) | 14. (d) | 15. (c) |
| 16. (b) | 17. (c) | 18. (d) | 19. (d) | 20. (d) |
| 21. (d) | 22. (d) | 23. (b) | 24. (d) | 25. (a) |
| 26. (b) | 27. (d) | 28. (d) | | |
- 8.**
- | | | | | |
|---------|---------|---------|---------|---------|
| 1. (c) | 2. (d) | 3. (c) | 4. (d) | 5. (d) |
| 6. (a) | 7. (d) | 8. (d) | 9. (d) | 10. (c) |
| 11. (b) | 12. (d) | 13. (d) | 14. (c) | |
- 9.**
- | | | | | |
|---------|---------|---------|---------|---------|
| 1. (d) | 2. (a) | 3. (b) | 4. (a) | 5. (b) |
| 6. (d) | 7. (c) | 8. (c) | 9. (c) | 10. (c) |
| 11. (a) | 12. (b) | 13. (a) | 14. (a) | 15. (c) |
| 16. (d) | 17. (d) | 18. (a) | 19. (b) | |
- 10.**
- | | | | | |
|---------|---------|---------|---------|---------|
| 1. (c) | 2. (d) | 3. (d) | 4. (a) | 5. (b) |
| 6. (d) | 7. (c) | 8. (a) | 9. (b) | 10. (d) |
| 11. (d) | 12. (d) | 13. (b) | 14. (b) | 15. (c) |
| 16. (d) | 17. (d) | 18. (d) | 19. (c) | 20. (c) |
| 21. (a) | 22. (d) | 23. (c) | 24. (d) | 25. (a) |
| 26. (c) | 27. (d) | | | |

- 11.
- | | | | | |
|---------|---------|---------|---------|---------|
| 1. (d) | 2. (a) | 3. (c) | 4. (d) | 5. (a) |
| 6. (b) | 7. (d) | 8. (c) | 9. (a) | 10. (d) |
| 11. (d) | 12. (a) | 13. (b) | 14. (a) | 15. (a) |
| 16. (a) | 17. (b) | 18. (b) | 19. (c) | 20. (a) |



Index

A

- Acceptability
 - DFA 40
 - NFA 43
 - PDA 251
 - TM 345
- Acceptance by
 - empty stack 251, 255-256
 - final state 251, 255-256
- Accepting state 34
- Ackermann's Function 376-377
- Alan M Turing 97, 140, 317, 318, 335, 338, 343, 345, 355, 366
- Algebraic laws for *RE* 142
- Alonzo Church 140, 366
- ALGOL 98, 192
- Algorithm 390-392
 - Complexity 390
 - Deterministic 390
 - Mrakov 381
 - Nondeterministic 390
- Alphabets 7
- Ambiguity
 - Grammar 195, 202, 210
 - Inherent 213
 - Language 210
- Ambiguous
 - Context free grammar 210-213
 - to unambiguous CFG 214-219

Applications of

- Arden's theorem 149
- CFG 237
- Finite Automata 80
- NPDA 260
- Regular expressions 165-167
- Arden's theorem 148
- Arithmetic expressions 104, 201-202
- A-tree 198, 295
- Automaton 29
- Auxiliary PDA 282

C

- Cardinality 2
- Cartesian product 7
- Complexity
 - Computational 379
 - Space 379
 - Time 379, 391
- CFG of Regular Expression 309
- Chomsky Noam 126, 192
 - Hierarchy 124-125
 - Normal form (CNF) 192, 228-230
- Church Thesis 140-342
- Church-Turing Hypothesis 318, 342
- Class
 - Equivalence 69-71
 - P* and *NP* 390, 399
- Closure properties
 - CFL 305-308

RE languages 345–348
 Regular Language 178–179
 Regular sets 156
 Closures 11, 99
 Kleene 99, 140, 178
 Positive 100, 178
 Reflexive 11
 Symmetric 11
 Transitive 12
 Complement 158, 179, 180
 Set 4
 Language 308
 Computability 371
 Computable functions 13–16, 371–374
 Computational Complexity 379
 Concatenation 8, 198
 Construction of Reduced CFG 220–224
 Context free
 Grammar 193–239
 Language 194
 Context Sensitive
 Grammar 119, 371
 language 119, 371
 Contradiction 185–188, 297–301
 Conversion
 Mealy to Moore machine 66
 Moore to Mealy 65
 NFA to DFA 44
 Cook's theorem 402–403
 CYK algorithm 235

D

Duality principle 7
 Decision Properties
 and algorithm 301–305
 CFLs 192, 301, 305–308
 RLs 153, 182
 Defining languages 102
 DeMorgan's law 180–181
 Design of Turing Machines 325
 Deterministic
 CFL 194, 286
 FA 30, 46, 179

PDA 258–296
 TM 319
 Derivation 196
 left most 203, 206
 Right most 203–204
 Tree 195–207
 tree of arithmetic expressions 201–202
 Difference
 CFL and DCFL 286
 DFA and NFA 47
 DPDA and NPDA 258–269
 DTM and NTM 336
 Mealy and Moore machine 56
 Set 179

E

Elimination of
 null productions 224–226
 unit productions 224, 226–227
 epsilon or $\wedge$ -moves 47
 Emil L Post 97, 367
 Emptiness 182
 Equivalence
 Ambiguous and unambiguous CFG
 215–219
 Class 11, 69
 DFA and NFA 44
 FA and 2-way FA 54
 FA with and without $\wedge$ -moves 48
 Mealy and Moore machine 68
 PDA and CFL 286
 PDA and context free language 261
 Regular expressions 152
 Turing Machines 321
 Extended transition function to strings 77

F

FA with output 55
 Factorial 103
 Final state 34
 Finite automata 29–83, 138
 and RE 144
 DFA 31, 39–41

- model 30
- NFA 41
- with epsilon transitions 35, 39
- without epsilon transitions 39
- FA to Moore machine 63–64
- Finite state machine 30
 - model 30
- Finiteness 182
- Formal languages 92, 98–105
- Function 13–15
 - μ -Recursive 374
 - Composition 373
 - Cons 373
 - defined by Recursion 102–104
 - initial 372
 - Nil 373
 - Partial 371
 - Primitive recursive 372
 - Projection 373
 - Successor 373
 - Total 372
 - Turing computable 371

G

- Graph 16–22
 - Types 22
 - Operation 21
- Generalised Transition Graph (GTG) 35
- Gödel
 - Hypothesis 366
 - Kurt 140, 366, 375–376
 - Numbering 375
- Grammars 105–126, 257
 - Classification 119
 - type-0 or Unrestricted 119
 - type-1 or context sensitive 119, 371
 - type-2 or context free 120, 193–239
 - type-3 or regular 120
- Graphical notations for PDA 251–254
- Graphs and trees 16
- Greibach
 - Normal Form (GNF) 231–235
 - Sheila 192, 231

- Growth rate
 - notation 392
 - of functions 392–397

H

- Hamiltonian cycle 404
- Halt state 319
- Halting and Crash Condition 182, 321
- Halting Problem of Turing Machines 367
- Head
 - Read 32, 250
 - Read/Write 250
- Homomorphism 159

I

- Identity rules for RE 141
- Infinite language 116
- Inference to tree 199
- Inherent ambiguity 213
- Initial Functions 372–373
- Initial state 31, 33, 34, 43
- Input tape 32
- Instantaneous Description (ID)
 - PDA 251–252
 - Turing Machine 321–322
- Intermediate
 - Node 198
 - State 36
- intersection
 - languages 158, 179, 180, 181, 307–308
 - sets 4

K

- Kleene Theorem 146

L

- Lexical analysis 283
- Language 7, 106–126
 - Classification 119
 - non-recursively Enumerable 370
 - type-0 or Unrestricted 119
 - type-1 or context sensitive 119, 346, 370

type-2 or context free 120, 194

type-3 or regular 120

Language acceptability

DFA 39, 40

NDA 43

PDA 251

TM 345

Languages 7

and their relations 120

LBA (Linear Bounded Automata) 236–238

Lexicographic order 12, 100

Limitations of

Algorithmic computation 335

finite state machines (FA) 83

PDA 279

Linear Bounded Automata (LBA) 336–337

M

Mathematical induction 23–24

Machine from Regular Expression 145–148

Markov Algorithm 381

Matrix Grammar 380–381

Machines

Finite state 30

Mealy 55–68

Moore 55–68

Turing 318–355

Michel Rabin 97

Minimization of FA 69

by Equivalence Class 69

by Subset Construction 72

Minsky Marvin 281, 342

Model 280

Theorem 279

Mixing of CFLs and RLs 309

Modifications with Turing machines 338–341

Modified Post Correspondence Problem 370

Moore Edward E 63

Membership 366

Move relation

PDA 252–254

TM 322

Myhill-Nerode theorem 153

N

NDA or NFA 41

Next state 31

NFA with and without epsilon transitions
47–52

Noam Chomsky 97, 126, 192

Non deterministic

finite automata (NFA or NDA) 41

push down automata 259

Turing Machines 336

Non-context free Languages

Non-regular languages 167, 184–188

Nonterminals (variables) 106–107

Normal forms of CFG 228–235

NP

complete problems 390–391, 404

Completeness 390, 400

Hard 390, 400–401

Null or empty string 8

Null production 224–228

Nullable variables 198

Non-recursively enumerable languages 345

O

Operations on

languages 122–123

sets 3

Operators of RE 129

P

P and NP classes 390, 399

Palindromes 9

Parse trees 195–207

Parsing 192, 283

bottom up 283–285

top down 283–284

Partial Function 371–373

Partition 72

PDA 120, 192, 249–286
 and CFG 274
 and Regular Language 274
 Definition and Model 250
 empty stack to final state 257
 final state to empty stack 256
 Model 250
 Representation 250
 PDS (Push Down Store/Stack) 250
 Polynomial time 397–398
 Algorithm 398
 time reduction 399
 Post Correspondence Problem 366–370
 Post machine 343–344
 Power set 3, 42
 Precedence of operators in RE 141
 Prefix 9, 201
 Primitive Recursive Function 372
 Present state 31, 42
 Postfix 201–202
 Polish notation 201
 Problem
 Colorability 405
 Hamiltonian cycle 404
 Intractable 397
 Satisfiability 402
 Solvable (decidable) 348, 352
 Subset sum 405
 Tractable 397
 Traveling sales man 404
 Unsolvable (Undecidable) 347, 352
 Vertex cover 404
 Processing of strings in DFA 40
 Programming techniques in TM 333
 Proof techniques 22–24
 Properties of
 Move Relation 253–254, 322
 Recursively enumerable languages 346–347
 Transition function 76
 Proving languages not to be regular 184

Pumping lemma for
 CFLs 294–3–1
 RLs 181, 183–188

R

Recursion 373
 Recursive
 Definition 102
 Function 372
 Function Theory 371
 Language 96, 345
 Recursively Enumerable
 Languages 121, 345
 Sets 317, 353
 Reducibility 377–378
 Reducing one problem to another 377–378
 Regular expressions 138–167
 and regular grammar 163–164
 to FA 145
 from FA 144
 Regular Grammar
 Left linear and right linear 164
 from FA 159
 to FA 161
 Regular language 120, 139, 179
 regular set 155
 Relations 9
 Antisymmetric 10, 12
 Equivalence 11
 Inverse 10
 Reflexive 10, 12
 Symmetric 10
 Transitive 10, 12
 Types 10
 Relations between
 Sets 2–3
 Kleene and positive closure 99–101
 Removal of ambiguity 213
 Reverse 8
 Rewriting Systems 365, 380
 Rice's Theorem 367, 378–379
 Right Linear Grammar 189

S

Sentential forms 198, 202

Set

Empty 2

Equal 2

Finite 3

Identities 5-6

Intersection 3

Notation 2

Null 3

Operations 3-5

Partition 6

Power 3

Representation in Computer 6

Super 3

theory 2

Types 2

Union 3

Universal 3

Simulating

FA 36

TM 344, 348

Space complexity 379

Stack 249-282

Stephen Cole Kleene 97, 101, 140

Stephen Cook 390, 402, 404

String 7

Empty 8

Null 8

Working 198, 204

Length 8

Subset 3

Substitution 108

Shuffle 181

Subtree 198

Suffix 9

T

Terminal 106-107

Time Complexity 379, 391

Total language tree 204-205

Transition equations

FA 33

PDA 256, 258-274

TM 320

Transition Function

2-stck PDA 279

2-way FA 52

DFA 31

LBA 337

NDFA 42

PDA 251

TM 319-320

Transition graph (diagram) 33

DFA 36, 46

NDFA 46

PDA 254, 265-273

TM 324-332

Transition table

DFA 36, 46

NFA 46

TM 323, 325

Transpose 156

Turing Machine 119, 138

Computation 334, 371

Counter 344

Definition 319

Enumerator 355

Generator 353-354

Language accepter 353-354

Model 318

Multi dimensional 339-340

Multi head 340

Multi tape 339, 380

Offline 340

Representation 319

Transducer 354-355

Type-0 grammar 349-351

Undecidable problems 351-353

Two stack PDA 279-281

Two way finite automata 52-55

Types of Turing machines 336-341

U

Undecidability 365

Unambiguous grammar 214

Undecidable Problems 366

CFG 366, 369-370

Recursively Enumerable 347–348, 367
Union
 Languages 158
 Sets 140
Unit production 224–228
Universal TM 341
Unsolvable Problems Involving CFLs 351–353
Useless symbols in CFG 219

V

Variable (see nonterminals)
Vertex cover problem 404

W

Working string 198, 204

Y

Yes-instance 343
Yield 198

Z

Zero function 373